

# Scaling Up Relaxed Memory Verification with Separation Logics

Dissertation zur Erlangung des Grades  
des Doktors der Ingenieurwissenschaften  
der Fakultät für Mathematik und Informatik  
der Universität des Saarlandes

vorgelegt von Hoang-Hai Dang

Saarbrücken, 2024

Last updated for typo fixes on September 10, 2026.

TAG DES KOLLOQUIUMS

10. September, 2024

DEKAN DER FAKULTÄT FÜR MATHEMATIK UND INFORMATIK

Prof. Dr. Roland Speicher

PRÜFUNGSAUSSCHUSS

Vorsitzender: Prof. Dr. Sven Apel

Gutachter: Prof. Dr. Derek Dreyer

Prof. Dr. Mark Batty

Dr. Viktor Vafeiadis

Akademischer Mitarbeiter: Dr. Léo Stefanescu

## Abstract

---

Reasoning about concurrency in a realistic, non-toy language like C/C++ or Rust, which encompasses many interweaving complex features, is very hard. Yet, *realistic* concurrency involves relaxed memory models, which are significantly harder to reason about than the simple, traditional concurrency model that is sequential consistency. To *scale up* verifications to realistic concurrency, we need a few ingredients: (1) strong but abstract reasoning principles so that we can avoid the too tedious details of the underlying concurrency model; (2) modular reasoning so that we can compose smaller verification results into larger ones; (3) reasoning extensibility so that we can derive new reasoning principles for both complex language features and algorithms without rebuilding our logic from scratch; and (4) machine-checked proofs so that we do not miss potential unsoundness in our verifications. With these ingredients in hand, a logic designer can flexibly accommodate the intricacy of relaxed memory features and the ingenuity of programmers who exploit those features.

In this dissertation, I present how to develop strong, abstract, modular, extensible, and machine-checked separation logics for realistic relaxed memory concurrency in the Iris framework, using multiple layers of abstractions. I report two main applications of such logics: (i) the verification of the Rust type system with a relaxed memory model, where relaxed memory effects are encapsulated behind the safe interface of libraries and thus are not visible to clients, and (ii) the compositional specification and verification of relaxed memory libraries, in which relaxed memory effects are exposed to clients.

## Zusammenfassung

---

Programmverifikation von nebenläufigen Programmen in einer realistischen Programmiersprache wie C/C++ oder Rust, die viele komplexe, miteinander verflochtene Sprachkonstrukte enthält, ist sehr schwierig. Realistische nebenläufige Programme basieren auf *schwachen Speicherkonsistenzmodellen*, in denen sich die Beweisführung im Vergleich zum traditionellen, sequentiellen Speicherkonsistenzmodell (SC) erheblich schwieriger gestaltet. Um die Verifikation solcher realistischen nebenläufigen Programme zu ermöglichen benötigen wir mehrere Voraussetzungen: (1) starke Beweisregeln die die mühsamen Details des zugrundeliegenden Speicherkonsistenzmodells abstrahieren, (2) modulare Beweistechniken die es erlauben, die Verifikation in kleinere, mundgerechte Beweise aufzuteilen, (3) eine erweiterbare Verifikationslogik, in der neue Beweistechniken hinzugefügt werden können, ohne die Korrektheit der gesamten Logik erneut beweisen zu müssen (4) maschinengeprüfte Beweise, die die Korrektheit der Logik und der durchgeführten Beweise garantiert. Mit diesen Voraussetzungen kann ein Logikdesigner die Komplexität des schwachen Speicherkonsistenzmodells und den Einfallsreichtum der Programmierer, die sich dessen Funktionen zu Nutze machen, flexibel berücksichtigen.

In dieser Dissertation stelle ich vor, wie man starke, abstrakte, modulare, erweiterbare und maschinengeprüfte Separationslogiken für realistische schwache Speicherkonsistenz in dem Framework Iris mit Hilfe von mehreren Abstraktionsebenen erstellen kann. Ich berichte über zwei Hauptanwendungen dieser Logiken: (i) die Verifikation des Typsystems von Rust auf Basis eines schwachen Speicherkonsistenzmodells, bei dem die Auswirkungen schwacher Speicherkonsistenz hinter der sicheren Programmschnittstelle abstrahiert und somit für Clients unsichtbar sind, und (ii) die modulare Spezifikation und Verifikation von Programmbibliotheken mit schwacher Speicherkonsistenz, bei denen die Auswirkungen schwacher Speicherkonsistenz für Clients sichtbar sind.



# Acknowledgments

---

My PhD work was long and difficult. Fortunately, in the end, it has been a fruitful journey—I would not have been able to imagine where I am today. And for that, I am grateful to be able to work with and learn from ingenious and amazing people during that time.

I would like to deeply thank my advisor, Derek Dreyer, for his giving me the opportunity and support to embark on this arduous journey. Derek has always managed to surround himself with the most interesting research problems, and, as a result, I as his student, had multiple chances to work with some of those. I remember my first project with him was about using the very first version of Iris to encode high-order reasoning principles for continuations. I started the project without knowledge on Iris and reasoning about continuations, but Derek had encouraged and helped me through it. For that first project, I would also additionally like to thank David Swasey for his various guiding discussions, even though most did not make sense to me at the time. After that, Derek put me on the true beginning of my PhD life: the iGPS project. I still do not understand how or if Derek had foreseen that I could survive that! With some luck, I got through it and learnt a lot, and for that, I also would like to thank Janno Kaiser, Ori Lahav, and Viktor Vafeiadis for their teaching me about relaxed memory and Coq. After the success of iGPS, I have had the chances to work on more difficult problems, and Derek had always been there for me. I still remember the night of 2018, when we were running towards the deadline of the PLDI submission, and Derek found the bug in the race detector of ORC11, and we had to decide to submit the RustBelt Relaxed paper later. Thank you Derek, for always being vigilant and on the point.

I thank Mark Batty and Viktor Vafeiadis for serving as reviewers of my dissertation, and Sven Apel as chair and Léo Stefanesco as academic member of my committee. I would also like to thank all of the collaborators that I have worked with—without them, I would have achieved nothing. I would like to thank Jacques-Henri Jourdan, Ralf Jung, Jeehoon Kang, Azalea Raad, William Mansky, Robbert Krebbers, Jaehwang Jung, Jaemin Choi, Duc-Thuan Nguyen, Joshua Yanovski, Lennard Gäher, Michael Sammler, and Simon Spies. I am forever grateful to Rose Hoberman for her teaching of English, scientific writing, and communication skills.

Living in Germany would be impossible without my friends. I thank my dear Vietnamese friends in Saarbrücken, in Luxembourg, in Vienna, and in Antibes. I would also thank my friends that I got to know at MPI-SWS: Heiko Becker, Debasmita Lohar, Reinhard Munz, Ezgi Cicek, Rodolphe Lepigre, Gaurav Parthasarathy, and Raphaël Monat.

Last but not least, I thank my beloved family: my parents and my big brother, for having also been there for me, across the continents. Without them, I would not have even arrived in this continent and would not have met so many great people. I dedicate this dissertation to my mother and my father.

Saarbrücken, September 2024

*Dặng Hoàng Hải*



# Contents

---

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| Abstract                                                                         | iii       |
| Zusammenfassung                                                                  | iii       |
| Acknowledgments                                                                  | v         |
| Contents                                                                         | ix        |
| List of Figures                                                                  | xiii      |
| List of Tables                                                                   | xv        |
| Glossary                                                                         | xvii      |
| <b>1 Introduction</b>                                                            | <b>1</b>  |
| 1.1 Reasoning about Relaxed Memory Concurrency                                   | 2         |
| 1.2 RustBelt Relaxed: Verifying the Soundness of Rust's Type System in RMC       | 3         |
| 1.3 Compass: Strong and Compositional Specifications of Relaxed-Memory Libraries | 5         |
| 1.4 Structure                                                                    | 8         |
| 1.5 Publications and Collaborations                                              | 9         |
| <b>I OPERATIONAL SEMANTICS FOR RELAXED MEMORY</b>                                | <b>11</b> |
| <b>2 Background: Relaxed Memory Models</b>                                       | <b>15</b> |
| 2.1 C11, Intuitively                                                             | 15        |
| 2.2 RC11, Formally                                                               | 17        |
| <b>3 ORC11: Operational Repaired C11</b>                                         | <b>27</b> |
| 3.1 Understanding Relaxed Memory with Views                                      | 27        |
| 3.2 Basic Machine State Definitions                                              | 30        |
| 3.3 View-based RMC Semantics                                                     | 34        |
| 3.4 The Data-Race Detector                                                       | 39        |
| 3.5 Comparison with iGPS Race Detector                                           | 42        |
| 3.6 The Correspondence between RC11 and ORC11                                    | 44        |
| <b>4 The Relaxed <math>\lambda_{\text{Rust}}</math> Language</b>                 | <b>47</b> |
| 4.1 Language Syntax                                                              | 47        |
| 4.2 Language Expression Reductions                                               | 50        |
| 4.3 The Complete Operational Semantics of Relaxed $\lambda_{\text{Rust}}$        | 54        |
| <b>5 Related Work</b>                                                            | <b>57</b> |
| <b>II SEPARATION LOGIC FOR RELAXED MEMORY</b>                                    | <b>59</b> |
| <b>6 More Background: Iris, A Framework for Concurrent Separation Logics</b>     | <b>63</b> |
| 6.1 Basic Rules                                                                  | 64        |
| 6.2 Ghost State and Resource Algebras                                            | 65        |
| 6.3 Invariants and Fancy Updates                                                 | 67        |

|            |                                                                                            |            |
|------------|--------------------------------------------------------------------------------------------|------------|
| 6.4        | Hoare Triples . . . . .                                                                    | 69         |
| 6.5        | Adequacy . . . . .                                                                         | 69         |
| 6.6        | Some Common Rules for WPs and Hoare Triples . . . . .                                      | 70         |
| 6.7        | Weakest Pre-conditions and Invariants . . . . .                                            | 71         |
| 6.8        | Properties of Propositions . . . . .                                                       | 72         |
| 6.9        | The Method of Fictional Separation . . . . .                                               | 73         |
| 6.10       | The Physical State Interpretation . . . . .                                                | 75         |
| 6.11       | An Instantiation Example for Simple Heaps . . . . .                                        | 75         |
| <b>7</b>   | <b>A Base Logic for RMC in Iris</b>                                                        | <b>79</b>  |
| 7.1        | Thread-local Configurations as Expressions . . . . .                                       | 79         |
| 7.2        | Basic Local Assertions for View-based RMC . . . . .                                        | 81         |
| 7.3        | Primitive Memory Rules . . . . .                                                           | 84         |
| 7.4        | Resource Algebras for Basic Local Assertions . . . . .                                     | 92         |
| 7.5        | State Interpretation . . . . .                                                             | 93         |
| 7.6        | Proofs of Some Primitive Rules and Adequacy . . . . .                                      | 96         |
| <b>8</b>   | <b>vProp: View-monotone Predicates</b>                                                     | <b>99</b>  |
| 8.1        | View-monotone Predicates . . . . .                                                         | 99         |
| 8.2        | Model of iRC11 Weakest Pre-conditions . . . . .                                            | 101        |
| 8.3        | Fence Modalities . . . . .                                                                 | 102        |
| 8.4        | Objective Propositions and The Objective Modality . . . . .                                | 105        |
| 8.5        | View-explicit Modalities . . . . .                                                         | 106        |
| 8.6        | The Subjective Modality . . . . .                                                          | 109        |
| <b>9</b>   | <b>Non-Atomic Points-To</b>                                                                | <b>111</b> |
| 9.1        | The Interface of Non-Atomic Points-To . . . . .                                            | 111        |
| 9.2        | The Model of Non-Atomic Points-To . . . . .                                                | 112        |
| <b>10</b>  | <b>Atomic Points-To</b>                                                                    | <b>115</b> |
| 10.1       | The Interface of the Atomic Points-To Assertion . . . . .                                  | 116        |
| 10.2       | The Model of the Atomic Points-To Assertion . . . . .                                      | 126        |
| <b>11</b>  | <b>Invariants in Relaxed Memory</b>                                                        | <b>133</b> |
| 11.1       | Objective Invariants . . . . .                                                             | 134        |
| 11.2       | Cancelable Invariants . . . . .                                                            | 136        |
| 11.3       | Non-Atomic Invariants . . . . .                                                            | 143        |
| <b>12</b>  | <b>Example Verifications with iRC11</b>                                                    | <b>145</b> |
| 12.1       | Release-Acquire Message-Passing . . . . .                                                  | 145        |
| 12.2       | Release-Acquire Message-Passing with Reclamation . . . . .                                 | 149        |
| 12.3       | Spawn and Join . . . . .                                                                   | 154        |
| 12.4       | A Release-Acquire Treiber Stack . . . . .                                                  | 156        |
| <b>13</b>  | <b>Related Work</b>                                                                        | <b>167</b> |
| <b>III</b> | <b>RUSTBELT MEETS RELAXED MEMORY</b>                                                       | <b>171</b> |
| <b>14</b>  | <b>Challenge: RustBelt and Relaxed Memory</b>                                              | <b>173</b> |
| 14.1       | Task 1: Re-prove the Safety of Rust Libraries under RMC . . . . .                          | 174        |
| 14.2       | Task 2: Re-prove the Safety of the $\lambda_{\text{Rust}}$ Type System under RMC . . . . . | 176        |
| 14.3       | Contributions of RustBelt Relaxed . . . . .                                                | 176        |



|                                                                                    |                |
|------------------------------------------------------------------------------------|----------------|
| <b>15 The Lifetime Logic of SC RustBelt</b>                                        | <b>179</b>     |
| 15.1 Borrowing in Rust . . . . .                                                   | 179            |
| 15.2 The Lifetime Logic Primer, in SC . . . . .                                    | 181            |
| <b>16 Lifetime Logic Meets Relaxed Memory</b>                                      | <b>187</b>     |
| 16.1 More Rules for the Lifetime Logic . . . . .                                   | 187            |
| 16.2 Other Forms of Borrows . . . . .                                              | 191            |
| 16.3 Adaption of the Lifetime Logic’s Model in iRC11 . . . . .                     | 195            |
| <b>17 GPS Single-Location Protocols</b>                                            | <b>201</b>     |
| 17.1 Surface-level GPS Protocols in iRC11 . . . . .                                | 201            |
| 17.2 Middleware GPS Protocols in iRC11 . . . . .                                   | 218            |
| 17.3 The Model of GPS Protocols . . . . .                                          | 221            |
| <b>18 Verification of <code>RwLock</code></b>                                      | <b>225</b>     |
| 18.1 RMC Implementation of a Reader-Writer Lock . . . . .                          | 225            |
| 18.2 The Semantic Model of the Reader-Writer Lock Type . . . . .                   | 229            |
| 18.3 Proof Sketches of the Library’s Operations . . . . .                          | 233            |
| <b>19 Verification of <code>Arc</code></b>                                         | <b>237</b>     |
| 19.1 The Core <code>Arc</code> library . . . . .                                   | 237            |
| 19.2 Verification of Core <code>Arc</code> with Cancelable GPS Protocols . . . . . | 239            |
| 19.3 Verification of <code>Arc</code> ’s Full APIs . . . . .                       | 245            |
| 19.4 Insufficient Synchronization in <code>get_mut</code> . . . . .                | 249            |
| <b>20 Related Work</b>                                                             | <b>251</b>     |
| <br><b>IV COMPASS</b>                                                              | <br><b>253</b> |
| <b>21 Background: Strong Specifications with Logical Atomicity</b>                 | <b>257</b>     |
| 21.1 Sequential Specifications for Queues . . . . .                                | 257            |
| 21.2 SC Specifications with Logical Atomicity . . . . .                            | 258            |
| 21.3 Logically Atomic Specifications in RMC with Views . . . . .                   | 260            |
| <b>22 Strong Compass Specifications with Richer Partial Orders</b>                 | <b>263</b>     |
| 22.1 Graph-Based Specs to Encode Partial Orders . . . . .                          | 263            |
| 22.2 Weaker Specs by Abandoning Abstract States . . . . .                          | 268            |
| 22.3 Implementing Compass Specs in iRC11 . . . . .                                 | 269            |
| <b>23 Verifications of Stacks and Queues</b>                                       | <b>273</b>     |
| 23.1 Queue Specs and Verification of the Michael-Scott Queue . . . . .             | 273            |
| 23.2 Stack Specs and Verification of the Treiber Stack . . . . .                   | 287            |
| <b>24 Exchangers and the Elimination Stack</b>                                     | <b>291</b>     |
| 24.1 The Elimination Stack . . . . .                                               | 291            |
| 24.2 A Strong Spec for Exchangers . . . . .                                        | 292            |
| 24.3 Verifying the Elimination Stack . . . . .                                     | 295            |
| <b>25 Related Work</b>                                                             | <b>297</b>     |
| <b>26 Conclusion</b>                                                               | <b>301</b>     |
| <b>Bibliography</b>                                                                | <b>305</b>     |



## List of Figures

---

|     |                                                                                                                      |     |
|-----|----------------------------------------------------------------------------------------------------------------------|-----|
| 1.1 | Dependency graph of this dissertation's chapters and the concepts that they introduce. . .                           | 7   |
| 2.1 | Message-Passing examples in C11/RC11. . . . .                                                                        | 16  |
| 2.2 | Candidate executions of several MP examples. . . . .                                                                 | 20  |
| 2.3 | Illustrations of derived relations. . . . .                                                                          | 22  |
| 2.4 | A racy execution of a racy MP program. . . . .                                                                       | 23  |
| 2.5 | Several <i>forbidden</i> (inconsistent) executions in C11/RC11. . . . .                                              | 24  |
| 2.6 | Load-buffering (LB) and Out-of-thin-air (OOTA) behaviors. . . . .                                                    | 25  |
| 3.1 | View-based explanation of MP behaviors. . . . .                                                                      | 29  |
| 3.2 | Computations of post thread-views for read and write operations. . . . .                                             | 36  |
| 3.3 | View-based machine semantics. . . . .                                                                                | 37  |
| 3.4 | Data-race free (DRF) pre-conditions. . . . .                                                                         | 40  |
| 3.5 | Data-race free (DRF) post-conditions. . . . .                                                                        | 42  |
| 4.1 | The relaxed $\lambda_{\text{Rust}}$ language syntax. . . . .                                                         | 48  |
| 4.2 | Some syntactic sugars for $\lambda_{\text{Rust}}$ . . . . .                                                          | 49  |
| 4.3 | CPS notations for $\lambda_{\text{Rust}}$ . . . . .                                                                  | 50  |
| 4.4 | Relaxed $\lambda_{\text{Rust}}$ expression semantics. . . . .                                                        | 52  |
| 4.5 | The combined 1-thread semantics of ORC11 machine semantics and $\lambda_{\text{Rust}}$ expression semantics. . . . . | 54  |
| 4.6 | Threadpool semantics. . . . .                                                                                        | 55  |
| 6.1 | An excerpt of Iris grammar. . . . .                                                                                  | 64  |
| 6.2 | Basic rules of several Iris connectives. . . . .                                                                     | 65  |
| 6.3 | Basic rules of Iris ghost ownership and basic updates. . . . .                                                       | 66  |
| 6.4 | Some rules for Iris invariants and fancy updates. . . . .                                                            | 68  |
| 6.5 | Some common rules for Iris weakest pre-conditions and Hoare triples. . . . .                                         | 71  |
| 6.6 | Some rules for Iris weakest pre-conditions and invariants. . . . .                                                   | 72  |
| 6.7 | Some properties of timeless propositions and persistent propositions. . . . .                                        | 73  |
| 6.8 | Several rules for the $\text{AUTH}(M)$ RA. . . . .                                                                   | 74  |
| 7.1 | Pure primitive WPs in the RMC base logic . . . . .                                                                   | 80  |
| 7.2 | Main properties of the base logic's local assertions . . . . .                                                       | 83  |
| 7.3 | The base logic's primitive Hoare rules for fences . . . . .                                                          | 85  |
| 7.4 | The base logic's primitive Hoare rules for non-atomic reads and writes . . . . .                                     | 86  |
| 7.5 | The base logic's primitive Hoare rules for atomic reads and writes . . . . .                                         | 87  |
| 7.6 | The base logic's primitive Hoare rule for CASes . . . . .                                                            | 88  |
| 7.7 | The base logic's primitive WP rule for CASes . . . . .                                                               | 91  |
| 7.8 | Several agreements between the global ghost state and local assertions . . . . .                                     | 95  |
| 7.9 | Several update rules for the global ghost state and local assertions . . . . .                                       | 95  |
| 8.1 | iRC11 rules for fence modalities . . . . .                                                                           | 103 |
| 8.2 | iRC11 rules for objective propositions and the objective modality . . . . .                                          | 106 |
| 8.3 | iRC11 rules for view-explicit modalities . . . . .                                                                   | 108 |

|       |                                                                                                                |     |
|-------|----------------------------------------------------------------------------------------------------------------|-----|
| 8.4   | iRC11 rules for the subjective modality . . . . .                                                              | 110 |
| 9.1   | Rules for iRC11 non-atomic points-to . . . . .                                                                 | 112 |
| 10.1  | Basic properties of assertions related to the atomic points-to . . . . .                                       | 117 |
| 10.2  | Conversions between the non-atomic and atomic points-to assertion . . . . .                                    | 119 |
| 10.3  | iRC11 read rules with the atomic points-to assertion . . . . .                                                 | 121 |
| 10.4  | iRC11 write rules with the atomic points-to assertion . . . . .                                                | 123 |
| 10.5  | An iRC11 CAS rule with the atomic points-to assertion . . . . .                                                | 125 |
| 10.6  | An iRC11 CAS rule with the atomic points-to in single-writer mode . . . . .                                    | 126 |
| 10.7  | Several properties of ghost abstractions for the atomic RA . . . . .                                           | 128 |
| 11.1  | iRC11 rules for objective invariants . . . . .                                                                 | 135 |
| 11.2  | iRC11 rules for cancelable invariants . . . . .                                                                | 137 |
| 11.3  | Stronger iRC11 rules for cancelable invariants . . . . .                                                       | 139 |
| 11.4  | Properties of the RA FRACVIEWR for cancelable invariants . . . . .                                             | 142 |
| 11.5  | The interface of non-atomic invariants . . . . .                                                               | 144 |
| 12.1  | Message-Passing with Loops . . . . .                                                                           | 146 |
| 12.2  | Hoare proof outlines for <b>mp</b> . . . . .                                                                   | 148 |
| 12.3  | Message-Passing with Reclamation . . . . .                                                                     | 150 |
| 12.4  | Hoare proof outlines for <b>mp_reclaim</b> . . . . .                                                           | 151 |
| 12.5  | Derived iRC11 atomic access rules with the view-join modality . . . . .                                        | 153 |
| 12.6  | A Spawn-and-Join library . . . . .                                                                             | 154 |
| 12.7  | Hoare proof outlines for <b>SPAWN-SPEC</b> . . . . .                                                           | 156 |
| 12.8  | A simple release-acquire implementation for Treiber stacks . . . . .                                           | 157 |
| 12.9  | Bag or per-element specifications for Treiber stacks . . . . .                                                 | 159 |
| 12.10 | Hoare proof outlines for <b>try_push_swap</b> . . . . .                                                        | 163 |
| 12.11 | Hoare proof outlines for <b>try_pop</b> . . . . .                                                              | 164 |
| 14.1  | Key rules for cancelable invariants in Iris-SC . . . . .                                                       | 175 |
| 15.1  | Selected rules of SC RustBelt’s lifetime logic . . . . .                                                       | 182 |
| 15.2  | The life cycle of borrows and lifetimes . . . . .                                                              | 182 |
| 15.3  | MP verified with the lifetime logic in Iris-SC. . . . .                                                        | 184 |
| 16.1  | More selected rules for lifetimes and full borrows, ported to $\lambda_{\text{Rust}} + \text{ORC11}$ . . . . . | 189 |
| 16.2  | Selected rules for other borrow alternatives, sound in $\lambda_{\text{Rust}} + \text{ORC11}$ . . . . .        | 193 |
| 17.1  | Rules for GPS Persistent Concurrent Protocols . . . . .                                                        | 204 |
| 17.2  | CAS Rules for GPS Persistent Concurrent Protocols . . . . .                                                    | 206 |
| 17.3  | Rules for auxillary assertions of GPS Single-Writer Protocols . . . . .                                        | 210 |
| 17.4  | Selected rules for Cancelable Single-Writer GPS Protocols . . . . .                                            | 213 |
| 17.5  | Selected basic rules for Atomic-Borrows-based GPS Protocols . . . . .                                          | 215 |
| 17.6  | Selected read and write rules for Atomic-Borrows-based GPS Protocols . . . . .                                 | 217 |
| 17.7  | A CAS rule for Atomic-Borrows-based GPS Protocols . . . . .                                                    | 218 |
| 17.8  | Selected rules for assertions of middleware GPS protocols . . . . .                                            | 220 |
| 18.1  | A $\lambda_{\text{Rust}}$ version of Rust’s RMC <b>RwLock&lt;T&gt;</b> . . . . .                               | 228 |
| 19.1  | Implementation of Core <b>Arc</b> . . . . .                                                                    | 239 |

|      |                                                                                                 |     |
|------|-------------------------------------------------------------------------------------------------|-----|
| 19.2 | Counting permissions for Core Arc . . . . .                                                     | 241 |
| 19.3 | A truncated history of the Arc counter . . . . .                                                | 247 |
| 21.1 | Specifications of Queue operations, from sequential, to SC concurrency and strong RMC . . . . . | 259 |
| 21.2 | A Message-Passing (MP) client with Queues . . . . .                                             | 261 |
| 22.1 | Compass Specs for Queues . . . . .                                                              | 264 |
| 22.2 | A proof sketch of Message Passing with queues . . . . .                                         | 267 |
| 23.1 | Full $\text{LAT}_{\text{hb}}^{\text{abs}}$ specs for queue . . . . .                            | 274 |
| 23.2 | A release-acquire Michael-Scott queue . . . . .                                                 | 275 |
| 23.3 | An RMC Herlihy-Wing Queue . . . . .                                                             | 277 |
| 23.4 | $\text{LAT}_{\text{hb}}$ specs for stack . . . . .                                              | 288 |
| 23.5 | $\text{LAT}_{\text{hb}}$ Stack Consistency . . . . .                                            | 289 |
| 24.1 | $\text{LAT}_{\text{hb}}$ specs for exchangers (excerpt, simplified). . . . .                    | 293 |



## List of Tables

---

|      |                                                                                                   |     |
|------|---------------------------------------------------------------------------------------------------|-----|
| 16.1 | Comparison of borrow types . . . . .                                                              | 192 |
| 18.1 | A summary of Rust types for <code>RwLock&lt;T&gt;</code> and its lock guards . . . . .            | 226 |
| 19.1 | An excerpt of Rust's <code>Arc&lt;T&gt;</code> and <code>Weak&lt;T&gt;</code> APIs . . . . .      | 245 |
| 19.2 | Rust's implementation (excerpt) of <code>Arc::get_mut</code> and <code>Arc::drop</code> . . . . . | 246 |





## *Glossary*

---

**SC** Sequential Consistency

**RMC** Relaxed Memory Consistency/Concurrency

**CSL** Concurrent Separation Logic

**C11** C/C++ 2011 Standards

**RC11** Repaired C11

**ORC11** Operational Repaired C11



# 1

## Introduction

---

Reasoning about concurrency is hard, due to the explosion of possible interactions between threads running in parallel. In the traditional concurrency model of *sequential consistency*<sup>1</sup>, every thread takes turns to execute its atomic instructions, and the behavior of a concurrent program is defined as all interleavings of all threads' atomic instructions. As such, if one needs to verify some property of the program, one would need to check that property for every possible interleaving of the atomic instructions performed by the threads. This is low level and hard to scale: if we want to compose our *verified* libraries, then we would have to look at the compositions of their interleavings, and we would have to make sure that the properties they have been verified against are compatible with interleaving composition. In order to scale verification to more intricate programming language features and algorithms, we need more *abstract* and *modular* reasoning principles.

CONCURRENT SEPARATION LOGICS<sup>2</sup> (hereafter, CSLs) provide a feasible approach to abstract and modular control of *interference*: instead of thinking in terms of interleavings, we can reason about each thread more modularly by thinking in terms of the *resources* that the thread owns. The resources owned by each thread are “separated” from those of other threads, and encode the thread's *permissions* on the shared memory's fragments that it owns. As a result, they can restrict how other threads may interfere with the current thread's execution. This “separation” idea has led to long research lines on highly expressive logics or logic frameworks<sup>3</sup> that have been applied to various sophisticated concurrency verification problems. Among these problems includes reasoning about realistic *relaxed memory concurrency*<sup>4</sup>—the main focus of this dissertation.

RELAXED MEMORY CONCURRENCY. Sequential consistency (hereafter, SC)—the interleaving model of concurrency in which threads take turns accessing the global state, and all threads share the same view of that state—does not reflect what is going on in modern multicore programming languages. In reality, multicore hardware employ rich hierarchies of *caches* to improve memory access performance, with which a CPU's write may not immediately reach the main memory, or may not be immediately visible to all other cores, or may not be visible to all other cores at the same time. To further improve performance, both hardware

<sup>1</sup>Lamport, “How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs” [Lam79].

<sup>2</sup>O'Hearn, “Resources, concurrency, and local reasoning” [O'H07]; Brookes, “A Semantics for Concurrent Separation Logic” [Bro07].

<sup>3</sup>Just to list a few: [VP07; FFS07; Fen09; Fu+10; DY+10; JB12; SB14; RPDG14; RP+16; Nan+14; SWT18; Kro+20; TDB13; Jun+15; Jun+18b; Cha+21; FKB21; G+22].

<sup>4</sup>[VN13; TVD14; DV16; DV17; Kai+17; Sve+18; He+18; Dan+20a; MJP20].

and compilers can analyze dependencies of memory accesses to apply optimizations: if the effects of two memory accesses are independent, they can be executed independently. In short, from the perspective of programmers, memory accesses instructions can be executed *out-of-order* in modern programming languages.

To match this modern reality, we need models of so-called *relaxed memory concurrency* (hereafter, RMC) at the programming-language level that provide an abstraction over different hardware architectures and compilers. However, due to the complexity of hardware behaviors and desirable optimizations, the formal semantics of RMC models (at both hardware level and language level) still require extensive ongoing research.<sup>5</sup> Nevertheless, the goal of this dissertation is *not* to find the right model that captures all relaxed memory features. Here, I take as assumption a language-level memory model whose features have stabilized over years of research, and present how to build RMC separation logics that can scale up to very substantial verification efforts.

<sup>5</sup>[Bat+11; Kan+17; Pul+18; Flu+17; Lah+17; Pul+19; CV19; PLV19; Lee+20; Cho+21b; Sim+20; Cho+22; Lee+23].

## 1.1 Reasoning about Relaxed Memory Concurrency

This dissertation focuses on the relaxed memory model of C/C++, which was first proposed in the C++11 standard and was formalized by Batty et al.,<sup>6</sup> and is now broadly adopted<sup>7</sup> by the RMC models of Rust, Java, OCaml, JavaScript, and WebAssembly.<sup>8</sup> The C/C++ RMC model (hereafter, C11) supports a variety of different *consistency levels* for shared-memory accesses, which intuitively dictate how much reordering can be applied to the accesses. For programmers who demand the simpler SC concurrency model where there is strong *synchronization* between threads (so that they have the same view of shared memory), SC accesses are available.<sup>9</sup> This strength, however, comes at the cost of disabling reordering optimizations and inserting expensive memory fences into the compiled code. The weaker consistency levels of *release/acquire* and *relaxed* allow one to trade off synchronization strength in return for more efficient compiled code. These different consistency levels are widely employed in performance-critical concurrency libraries such as locks, reference-counting, stacks, queues, read-copy-update (RCU), and so on.

Compared to SC, reasoning about RMC is significantly more complicated: relaxed-memory programs have many more behaviors depending on which consistency levels are employed. In fact, some useful reasoning principles in SC logics are no longer sound for reasoning about relaxed behaviors. Furthermore, such behaviors are defined in C11 not in the familiar style of interleavings, but by an *axiomatic* semantics, in which the allowed behaviors of a program are defined by enumerating candidate executions (represented as “event graphs”) and then restricting attention to the executions that obey various coherence axioms. Vafeiadis et al. overcome these challenges and provided the first abstract and modular reasoning principles for C11 in form of various RMC separation logics.<sup>10</sup>

However, in building these logics, Vafeiadis et al. were not able to use the standard model of Hoare-style program specifications from

<sup>6</sup>Batty et al., “Mathematizing C++ concurrency” [Bat+11].

<sup>7</sup>potentially partially, with minor modifications or simplifications.

<sup>8</sup>[BP19; WRP19; DSM18; Wat+20].

<sup>9</sup>The word “consistency”, used *e.g.*, in sequential consistency or relaxed memory consistency, can be understood as the consistency among the views of the threads (or processors) on shared memory.

<sup>10</sup>[VN13; DV16; DV17; TVD14].

prior CSLs because notions like “the machine states before and after executing a command  $c$ ” do not have a clear meaning in C11’s axiomatic semantics. Instead, they had to come up with new, non-standard models of separation logic in terms of predicates on event graphs. Unfortunately, the complexity of these new models has made them challenging to adapt and extend to more complex settings, for example in verifying Rust’s type system. Furthermore, although the soundness of these logics has been verified formally in Coq, there has thus far been no tool support to perform machine-checked verifications of RMC programs or libraries in these logics.

To scale the reasoning principles of concurrent separation logic to realistic languages like C/C++ or Rust, which encompass many interweaving complex features, we need a few ingredients: (1) strong but abstract reasoning principles so that we can avoid the too tedious details of the underlying concurrency model; (2) modular reasoning so that we can compose smaller verification results into larger ones; (3) reasoning extensibility so that we can derive new reasoning principles for both complex language features and algorithms without rebuilding our logic from scratch; and (4) machine-checked verifications so that we do not miss potential bugs in our proofs—both in soundness proofs of our logics and in program verifications. Only recently was it possible to acquire these ingredients at once with the CSL framework Iris,<sup>11</sup> which comes with strong tactic support in Coq.<sup>12</sup> Using Iris, Jung et al.<sup>13</sup> have verified the soundness of the Rust’s type system, and thus have demonstrated the scalability of CSLs to complex languages such as Rust, even though only for the SC memory model. Meanwhile, my collaborators and I had previously re-proven the soundness of Vafeiadis et al.’s RSL and GPS logics in Iris, and demonstrated the possibility of building extensible RMC separation logics, even though only for a small fragment of the C11 model.<sup>14</sup>

I, together with my collaborators, have developed strong, abstract, modular, extensible, and machine-checked RMC separation logics in Iris that scale to substantial verification efforts, for an also substantial fragment of C11 whose features have stabilized over years of research, namely the RC11 (Repaired C11) model.<sup>15</sup> In this dissertation, I present the abstractions needed to build such logics. I report two main contributions that rely on those logics:

1. RustBelt Relaxed:<sup>16</sup> the soundness proof of the Rust’s type system in RMC, where relaxed memory effects are encapsulated behind the safe interface of libraries and thus are not visible to clients; and
2. Compass:<sup>17</sup> the compositional specification and verification of relaxed memory libraries, where relaxed memory effects are exposed to clients of such libraries.

## 1.2 RustBelt Relaxed: Verifying the Soundness of Rust’s Type System in RMC

Rust<sup>18</sup> is a young and evolving programming language that aims to

<sup>11</sup>Jung et al., “Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning” [Jun+15]; Jung et al., “Higher-order ghost state” [Jun+16]; Krebbers et al., “The Essence of Higher-Order Concurrent Separation Logic” [Kre+17]; Jung et al., “Iris from the ground up: A modular foundation for higher-order concurrent separation logic” [Jun+18b].

<sup>12</sup>Krebbers et al., “Interactive Proofs in Higher-Order Concurrent Separation Logic” [KTB17]; Krebbers et al., “MoSeL: A General, Extensible Modal Framework for Interactive Proofs in Separation Logic” [Kre+18].

<sup>13</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [Jun+18a].

<sup>14</sup>Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

<sup>15</sup>Lahav et al., “Repairing sequential consistency in C/C++11” [Lah+17].

<sup>16</sup>Dang et al., “RustBelt Meets Relaxed Memory” [Dan+20a].

<sup>17</sup>Dang et al., “Compass: strong and compositional library specifications in relaxed memory separation logic” [Dan+22].

<sup>18</sup>Klabnik and Nichols, *The Rust Programming Language* [KN18].

bring safety to systems programming. Specifically, Rust provides low-level control over data layout and resource management à la modern C++, while at the same time offering strong high-level guarantees (such as type and memory safety) that are traditionally associated with safe languages like Java. In fact, Rust takes a step further, statically preventing more forms of anomalous behavior, such as data races and iterator invalidation, that safe languages typically fail to rule out. Rust strikes its delicate balance between safety and control using a *substructural* type system, in which types not only classify data but also represent *ownership* of resources, such as the right to read, write, or reclaim a piece of memory. By tracking ownership in the types, Rust is able to prohibit dangerous combinations of mutation and aliasing, a well-known source of programming pitfalls and security vulnerabilities in C/C++ and Java.

Nevertheless, Rust’s ownership-based type system is not always expressive enough to type-check very delicate programming idioms, e.g., some pointer-based data structures, synchronization abstractions, garbage collection mechanisms. To allow for these mechanisms, Rust supports extension to the type system via *libraries* whose implementations internally utilize *unsafe features* (e.g., unchecked type casts, array accesses without bounds checks, or accesses of “raw” pointers who are untracked by the type system). Given that these libraries are not checked by the type system, it is now the responsibility of libraries developers to make sure that these extensions are actually *safe*, in the sense that they have properly encapsulated the uses of unsafe features within their “safe APIs”. Unfortunately, as the language is evolving and libraries are being updated or created, it is not clear what such encapsulation formally means.

RustBelt<sup>19</sup> is the first work on the formal foundations of the Rust programming language, in which it covers not only the soundness of the ownership-based type system, but also the safe encapsulation by Rust’s extensions via libraries. RustBelt managed to formalize such interactions between the type system and the extensions in the presence of complex language features like recursive types and higher-order state. Furthermore, all proofs were machine-checked in Coq. Unfortunately, while ground-breaking, RustBelt assumes the SC memory model. Therefore, even though RustBelt’s results increase the confidence in the safety of Rust’s type system and libraries, the results cannot yet be applied to actual Rust code, which relies on the C11 memory model.

To circumvent this problem, we developed RustBelt Relaxed (or RB<sub>r1x</sub>, for short), the first formal validation of the soundness of Rust under RMC. Although based closely on the original RustBelt, RB<sub>r1x</sub> takes a significant step forward by accounting for the safety of the more weakly consistent memory operations that real concurrent Rust libraries actually use. For the most part, we were able to verify Rust’s uses of relaxed-memory operations as is. Only in the implementation of one Rust library (*Arc*) did we need to strengthen the consistency level of two memory reads (from relaxed to acquire) in order to make our verification go through. And in one of these cases, our attempt to verify the original (more relaxed) access led us to expose it as the source of a previously undetected data race in the library. Our fix for this race has since been merged into the

<sup>19</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [Jun+18a].

Rust codebase.<sup>20</sup>

**SYNCHRONIZED GHOST STATE.** The main technical challenge of porting RustBelt to RMC is relevant not just to Rust but to relaxed-memory verification in general: namely, that **existing work on separation logic does not provide an adequate foundation for reasoning about resource reclamation under relaxed memory**. Resource reclamation under relaxed memory intertwines *resource accounting* and *physical synchronization*: one needs to make sure that resources provided to every client thread must be returned completely and with proper synchronization. As a result, more care is needed when designing and proving relaxed memory reasoning rules. Fortunately, in  $\text{RB}_{\text{rlx}}$  we show that changes in the rules needed to support reclamation are minimal and can be handled fairly routinely, thanks to a novel notion of *synchronized ghost state*: ghost state that is tied to physical synchronization so that it can be used for safe, well-synchronized resource accounting.

### 1.3 Compass: Strong and Compositional Specifications of Relaxed-Memory Libraries

Existing RMC separation logics have been applied to verify tricky RMC algorithms such as locks, stacks, queues, read-copy-update,<sup>21</sup> and reference counting,<sup>22</sup> as well as the  $\text{RB}_{\text{rlx}}$  work. However, these works (except Cosmo<sup>23</sup>—see more below) only verify implementations against some “reasonable” specifications that are sufficient for their respective purposes, but do not necessarily capture their full functional correctness. For example, as we will see, even with unsafe features, the RMC libraries verified in  $\text{RB}_{\text{rlx}}$  only need specifications strong enough to verify the soundness of Rust’s type system, which focuses on safety and does not expose relaxed behaviors to users. As another example, the queue specification in GPS<sup>24</sup> only captures the fact that a dequeue is synchronized with the enqueue that it is matched with, but *not* the standard first-in-first-out (FIFO) property of queues. Stronger functional correctness CSL specifications (from now on, *specs* for short) for RMC libraries thus are needed, especially for clients that build new libraries out of smaller ones and rely on certain relaxed behaviors of the constituent libraries to verify their library’s implementation.

However, unlike in the SC setting, in RMC verification research there is no canonical way to specify full functional correctness of a library that may expose relaxed behaviors. While *linearizability*<sup>25</sup> is the de facto standard correctness condition for concurrent libraries, it does not extend to many highly concurrent libraries, including those in RMC: these libraries tend to have less synchronization or control, and it may be that a *linearization* is extremely difficult to construct (*e.g.*, Herlihy-Wing queue) or that the library has no useful sequential behaviors (*e.g.*, exchangers<sup>26</sup>). Therefore, various linearizability-like criteria have been proposed as alternatives,<sup>27</sup> especially for relaxed memory.<sup>28</sup> These works essentially share one basic idea in relaxing linearizability: instead of requiring a *total order* on a library’s operations, one only requires that

<sup>20</sup>Jourdan, *Insufficient synchronization in Arc::get\_mut* [Jou18].

<sup>21</sup>Tassarotti et al., “Verifying read-copy-update in a logic for weak memory” [TDV15].

<sup>22</sup>Doko and Vafeiadis, “Tackling Real-Life Relaxed Concurrency with FSL+” [DV17].

<sup>23</sup>Mével et al., “Cosmo: a concurrent separation logic for multicore OCaml” [MJP20].

<sup>24</sup>Turon et al., “GPS: navigating weak memory with ghosts, protocols, and separation” [TVD14]; Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

<sup>25</sup>Herlihy and Wing, “Linearizability: A Correctness Condition for Concurrent Objects” [HW90].

<sup>26</sup>[SLS05; HRV15].

<sup>27</sup>[Hen+13; JR14; Der+14; Haa+16; Nei94; AKY10; Bur+14; CRR15].

<sup>28</sup>[Bur+12; BDG13; Jag+13; Doh+18; Don+18; Raa+19; EE19; Kri+20].

<sup>29</sup>Rocha Pinto et al., “TaDA: A Logic for Time and Data Abstraction” [RPDG14]; Svendsen and Birkedal, “Impredicative Concurrent Abstract Predicates” [SB14]; Jung et al., “Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning” [Jun+15]; Jung et al., “The future is ours: prophecy variables in separation logic” [Jun+20].

<sup>30</sup>Mével and Jourdan, “Formal verification of a concurrent bounded queue in a weak memory model” [MJ21].

<sup>31</sup>Mével et al., “Cosmo: a concurrent separation logic for multicore OCaml” [MJP20].

<sup>32</sup>Dolan et al., “Bounding data races in space and time” [DSM18].

<sup>33</sup>Raad et al., “On library correctness under weak memory consistency: specifying and verifying concurrent libraries under declarative consistency models” [Raa+19].

operations respect some *partial orders*. These works, however, have little support for modular client reasoning. Therefore, we want to improve the proposed relaxations of linearizability with *Hoare-style* specs to support better modular reasoning about clients who rely on strong correctness guarantees of RMC libraries.

Accordingly, our starting point is *logical atomicity*,<sup>29</sup> a key proof technique to achieve strong specs *and* modular client reasoning in (SC) CSLs. Logically atomic specs are similar to Hoare-triple based specs, but they allow *atomic access* to the exact, up-to-date *abstract state* of the data structure. As such, they provide the abstraction that an operation takes effect atomically on the data structure’s abstract state, so that clients can build a concurrent protocol to govern how the data structure is used (how the state can evolve). If the client wants to compose multiple data structures, they can further build a protocol for multiple abstract states, all the while enjoying the benefits of separation logics.

Logical atomicity has been applied mostly in the SC setting, and only recently did Mével and Jourdan<sup>30</sup> demonstrate its use to give stronger CSL specs for RMC libraries. Unsurprisingly, the application of the technique needs to account for relaxed behaviors: Mével and Jourdan needed to combine logical atomicity with the tracking of some *synchronization* information among library operations, reminiscent of the partial orders from the relaxations of linearizability. But they only needed limited synchronization tracking, because their logic, Cosmo,<sup>31</sup> is sound only for the Multicore OCaml memory model,<sup>32</sup> and they only gave one spec for a concurrent queue and verified one client.

Consequently, the Cosmo-style specs does not scale to libraries or clients that rely on *interacting* relaxed behaviors. More specifically, while Cosmo specs expose *internal* (to the implementation) synchronizations among operations, they do not take into account how additional *external* synchronizations created by clients or other libraries can affect the behaviors of the library in question.

LOGICAL ATOMICITY AND RICHER PARTIAL ORDERS. We generalize Mével and Jourdan’s approach by combining **logical atomicity** with **richer partial orders** inspired by the relaxations of linearizability, so that we can give stronger specs for more weakly consistent libraries, in the more relaxed memory model RC11. But, given the plethora of partial orders from those relaxations of linearizability, which one should we use? We believe the *event-graph* based criteria proposed by Raad et al.<sup>33</sup> (“Yacovet”) are the most general, because in that framework a verifier can give a library stronger or weaker specs by choosing the partial orders they prefer and by stating suitable library-specific *consistency conditions* on the partial orders. Therefore, we decided to encode Yacovet criteria in our separation logic and enhance them further with logical atomicity. As such, we can give strong and compositional Hoare-style specs for RMC libraries, with better support for modular client reasoning, in a new framework called Compass. We demonstrate the strength, satisfiability, and support for client reasoning of our specs with multiple mechanized libraries and client verifications.



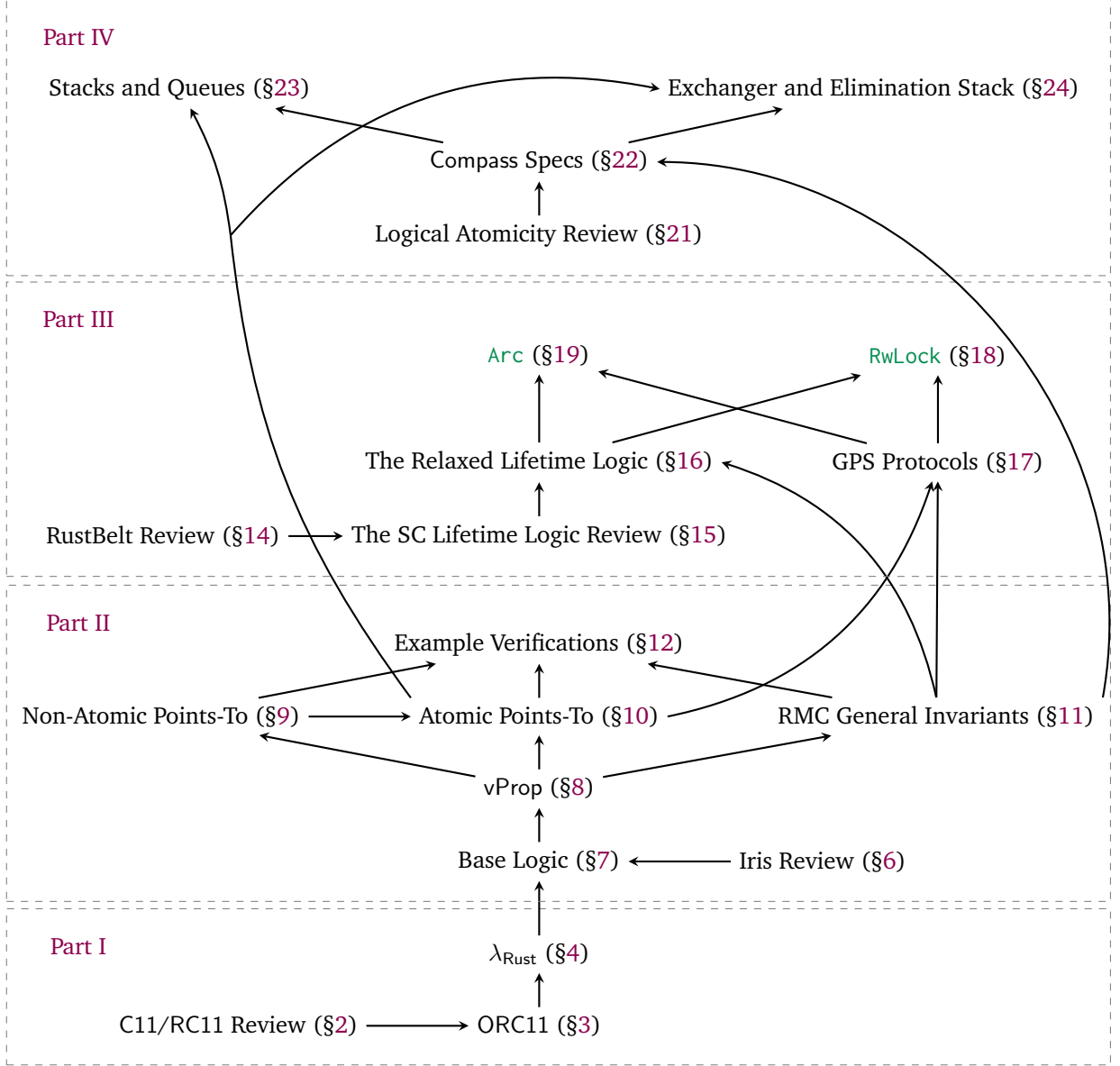


FIGURE 1.1: Dependency graph of this dissertation's chapters and the concepts that they introduce.

## 1.4 Structure

This dissertation is composed of three parts: **Part II** presents the basic layers needed to build RMC separation logics with Iris, while **Part III** and **Part IV** discuss how such logics can be extended and/or applied for RustBelt Relaxed and Compass, respectively. **Part III** and **Part IV** are independent from each other, but both rely on materials presented in **Part II**. Each part will discuss the context, the challenges, the solutions, and the results separately, as well as related and future work in detail. The conclusion (**Chapter 26**) provides a high level summary and potential future research directions. Note that **Figure 1.1** (page 7) provides the dependency graph for the main chapters in this dissertation.

**Part I** presents ORC11, an *operational* variant of RC11 that is needed to instantiate Iris. It provides a brief background review on relaxed memory models, which readers who are familiar with the topics can skip. The most important feature of ORC11 is its *race detector*—an operational account for *data races*, which need meticulous care and significantly complicate the soundness proof of iRC11.<sup>34</sup>

**Part II** discusses the features and the construction of iRC11, our extensible RMC separation logic for ORC11. It first provides a review of the Iris framework. The main chapters of **Part II** flesh out the abstraction layers needed to build the various core reasoning principles of iRC11: its *modalities*, and its *non-atomic* and *atomic* points-to assertions, and its forms of invariants, including *cancelable invariants* that employ *synchronized ghost state*. The atomic points-to assertion is a novel contribution of this dissertation that has not been published elsewhere. The extensibility of the construction will be demonstrated by the fact that iRC11 not only can incorporate *all* reasoning principles from all other RMC separation logics, but also can extend and combine them with iRC11’s own novel reasoning principles.

**Part III** discusses the proofs of the RustBelt Relaxed work. It first provides an overview of Rust and RustBelt, and briefly explains the soundness proof the Rust’s type system, which crucially depends on the *lifetime logic*. The remaining chapters of **Part III** elaborate on how iRC11’s *synchronized ghost state* and *cancelable invariants* are used in re-proving the lifetime logic and in re-verifying the concurrent standard libraries of Rust that use relaxed memory operations (e.g., `Mutex`, `RwLock`, or `Arc`). The library verifications depend crucially on a combination of cancelable invariants and *GPS single-location protocols*.<sup>35</sup> A bit of history on how the bug in `Arc` was found will be provided.<sup>36</sup>

**Part IV** presents the Compass specification framework. It starts by reviewing logical atomicity in both SC and RMC settings, as well as the event-graph based Yacovet specs. It then presents how to encode Yacovet specs in iRC11 with logical atomicity. The remaining chapters present the library verifications and client verifications of various RMC data structures, relying on a general notion of *multi-location invariants* in combination with the atomic points-to assertions. I will also touch on the topic of *helping* (cooperation) with logical atomicity, and its role in the specs of exchangers. Some of the specifications and verifications are

<sup>34</sup>It indeed delayed the publication of the `RBr1x` work by a year.

<sup>35</sup>Turon et al., “GPS: navigating weak memory with ghosts, protocols, and separation” [TVD14]; Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

<sup>36</sup>Jourdan, *Insufficient synchronization in `Arc::get_mut`* [Jou18].

the first-ever performed in the relaxed memory setting.

## 1.5 Publications and Collaborations

This dissertation contains the work of the following two papers:

- [Dan+20a]: Hoang-Hai Dang, Jacques-Henri Jourdan, Jan-Oliver Kaiser, Derek Dreyer. “RustBelt Meets Relaxed Memory”, appeared in POPL 2020.
- [Dan+22]: Hoang-Hai Dang, Jaehwang Jung, Jaemin Choi, Duc-Thanh Nguyen, William Mansky, Jeehoon Kang, and Derek Dreyer. “Compass: Strong and Compositional Library Specifications in Relaxed Memory Separation Logic”, appeared in PLDI 2022.

While reusing much of the text from these two papers, the dissertation provides substantially more in-depth details—many of which have not been presented before—in a coherent structure. The following contents are new and have not been discussed elsewhere:

- §3.4: the details of ORC11’s race detector;
- §7: the detailed model of the iRC11 base logic;
- §8: the models of various iRC11’s modalities;
- §9: the model of the *non-atomic* points-to assertion, which depends tightly on ORC11’s race detector;
- §10: the model of the *atomic* points-to assertion;
- §11: the detailed interfaces and models of iRC11 *objective invariants* and *cancelable invariants*;
- §12: several example verifications demonstrating many *intermediate* abstractions provided by iRC11;
- §16: more details on how the lifetime logic was ported to iRC11;
- §17: the detailed model of GPS *single-location protocols*, built atop atomic points-to;
- §19: the detailed verification of Rust’s standard library *Arc*;
- §22: the detailed interpretations of Compass specs in iRC11 with logical atomicity;
- §23: the verifications of stacks and queues against Compass specs;
- §24: a more complete spec of the exchanger with *helping*.

Some of the ideas in this work were originally developed in iGPS ([Kai+17]: Jan-Oliver Kaiser, Hoang-Hai Dang, Derek Dreyer, Ori Lahav, and Viktor Vafeiadis. “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris”, appeared in ECOOP 2017) of which I was a co-author. Although iGPS is not a part of this dissertation, I contributed to those ideas and have ported them fully into iRC11.

COLLABORATIONS. The two papers mentioned above, on which this dissertation is based, are the results of delightful collaborations. Although I led the efforts in both works, they would not have been achievable without the team efforts of many fellow researchers.

For  $\text{RB}_{\text{rlx}}$ , the ORC11’s race detector and the model of GPS protocols were inspired by those developed for iGPS, which in turn was the result of collaborations with Jan-Oliver (Janno) Kaiser. The flaw of the initial ORC11’s race detector was found by Derek Dreyer, and after I fixed the design, it was Janno that led the (on-paper) correspondence proof between RC11 and ORC11. I proved most of the soundness of the iRC11 logic, but I collaborated with Jacques-Henri Jourdan to construct the models of several iRC11 modalities. It was Jacques-Henri who used iRC11 to re-prove the soundness of the lifetime logic. I re-verified the Rust concurrent libraries by substantially extending the original proofs in SC RustBelt.<sup>37</sup> It was also Jacques-Henri’s original suggestion to re-prove the model of GPS protocols directly on top of iRC11, instead of on top of the base logic from Iris’ instantiation as in iGPS. I only completed that task 2 years later.

For Compass, I encoded the Yacovet specs in iRC11 with logical atomicity, and verified library implementations against those specs. I collaborated with Jaehwang Jung and Jaemin Choi to refine those specs to cater to the *linearizability-style* specs, but those specs are not included in this dissertation. Together with all other co-authors, we performed the client verifications that used the specs reported in [Dan+22].

COQ ARTIFACTS. Unless noted explicitly, all definitions and proofs in this dissertation are formalized in Coq. The follow repositories contain the respective Coq developments and instructions for how to build and use them.

- ORC11: <https://gitlab.mpi-sws.org/iris/gpfs1/-/tree/master/orc11>
- iRC11: <https://gitlab.mpi-sws.org/iris/gpfs1/-/tree/master/gpfs1>
- $\text{RB}_{\text{rlx}}$ : [https://gitlab.mpi-sws.org/iris/lambda-rust/-/tree/masters/weak\\_mem](https://gitlab.mpi-sws.org/iris/lambda-rust/-/tree/masters/weak_mem)
- Compass: <https://gitlab.mpi-sws.org/iris/gpfs1/-/tree/master/gpfs1-examples>

<sup>37</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [Jun+18a].

## Part I

### OPERATIONAL SEMANTICS FOR RELAXED MEMORY



This thesis discusses the features and the construction of iRC11, a concurrent separation logic that is sound for the relaxed memory model RC11.<sup>38</sup> We do not assume prior knowledge either on relaxed memory models or concurrent separation logics. Therefore, in this part, we discuss the semantics of relaxed memory concurrency. We start with [Chapter 2](#) to review relaxed memory models defined in *axiomatic* style, specifically for the C11<sup>39</sup> and RC11 models. Readers familiar with RMC can freely skip this review, unless they are interested in the specific details of the RC11 model. Then, in [Chapter 3](#), we present our first contribution: ORC11, an *operational* version of RC11 that is geared to complement the  $\lambda_{\text{Rust}}$  language used in RustBelt,<sup>40</sup> which is presented in [Chapter 4](#). Developing an operational semantics for RC11 is a necessary prerequisite for instantiating Iris. The bottom half of [Figure 1.1](#) visualizes the dependency among these chapters.

<sup>38</sup>Lahav et al., “Repairing sequential consistency in C/C++11” [[Lah+17](#)].

<sup>39</sup>Batty et al., “Mathematizing C++ concurrency” [[Bat+11](#)].

<sup>40</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [[Jun+18a](#)].





# 2

## Background: Relaxed Memory Models

---

The goal of hardware and language relaxed memory models is to give an abstraction for the possibility (or impossibility) of *out-of-order* behaviors for relaxed memory accesses, which are induced by hardware and/or compiler optimizations. The models can be defined in form of either *operational* or *axiomatic* semantics. RMC operational semantics typically involve some kind of buffers (e.g., write buffers in x86-TSO)<sup>1</sup> to delay the effects of memory accesses and thus make them appear out-of-order. Axiomatic semantics, on the other hand, define a set of constraints (*axioms*) on several *partial orders* among memory accesses in a *candidate execution*—accesses not so tightly ordered can thus be executed out-of-order. In this chapter, we review the axiomatic semantics of C11<sup>2</sup> and RC11.<sup>3</sup> More specifically, we review the intuitive semantics of C11 in §2.1, then a formal excerpt of RC11’s partial orders and axioms in §2.2. In §3, we present ORC11, our operational version of RC11.

<sup>1</sup>Sewell et al., “x86-TSO: a rigorous and usable programmer’s model for x86 multi-processors” [Sew+10].

<sup>2</sup>Batty et al., “Mathematizing C++ concurrency” [Bat+11].

<sup>3</sup>Lahav et al., “Repairing sequential consistency in C/C++11” [Lah+17].

### 2.1 C11, Intuitively

The C11 memory model offers several different modes of memory accesses, including non-atomic (**na**), relaxed (**rlx**), release (**rel**), acquire (**acq**), and sequentially consistent (**sc**). Non-atomic accesses are “normal” data accesses, meaning that it is the programmer’s responsibility to ensure that they are properly synchronized through other means. If they are not properly synchronized—i.e., there is a data race involving non-atomics—then C11 says the whole program has *undefined behavior*, or UB for short. The remaining modes, collectively called *atomic* accesses, are allowed to be racy and are indeed used to establish synchronization among non-atomic accesses.

**Example 2.1** (Message-Passing). To explain what synchronization actually means, we explore the *Message-Passing* examples in Figure 2.1. In Example 2.1(a), we initialize two memory locations  $\ell_x$  and  $\ell_y$  to 0 non-atomically, then spawn two threads  $\pi$  (on the left) and  $\rho$  (on the right). Thread  $\pi$  intends to pass a “message” to  $\rho$ . The message, 42, is stored in  $\ell_x$  (line  $\pi 1$ ).  $\pi$  then sets the boolean flag  $\ell_y$  to 1 (line  $\pi 2$ ), to signal to  $\rho$  that the message is ready to be received. Once  $\rho$  sees the flag set (line  $\rho 1$ ), it attempts to read the message from  $\ell_x$  (line  $\rho 2$ ). However, both the intended value of 42 as well as the initial value of 0 could be

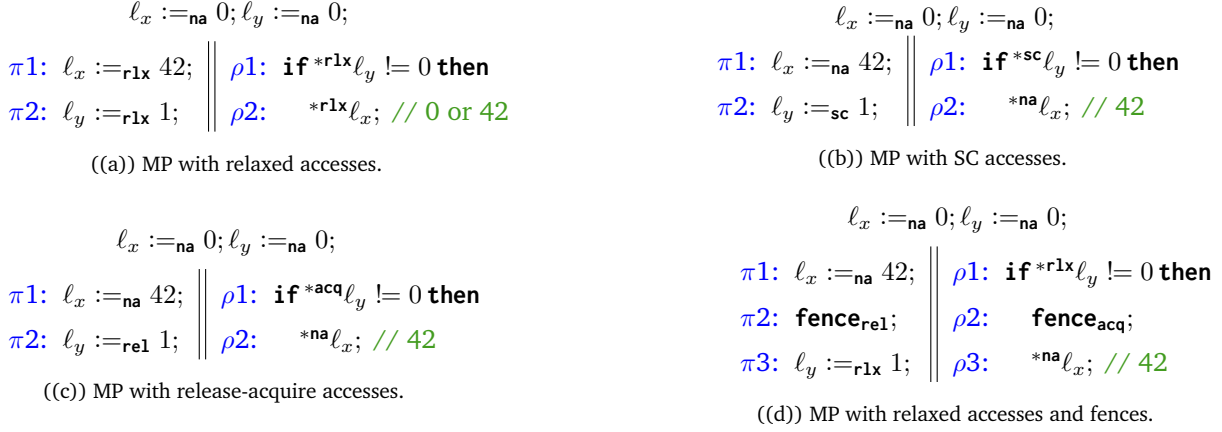


FIGURE 2.1: Message-Passing examples in C11/RC11.

read. That is, even though  $\rho$  has read 1 from  $\ell_y$ , it is not guaranteed to read 42 from  $\ell_x$ . This is because the relaxed accesses of  $\ell_y$  are not enough to establish synchronization between  $\pi$  and  $\rho$ .

In C11, threads are not synchronized by default: they each have their own perspective on the values in shared memory, and thus may observe memory events in different order. In [Example 2.1\(a\)](#), thread  $\rho$  may see that  $\pi 2$  ( $\pi$ 's write to  $\ell_y$ ) is executed out-of-order, before  $\pi 1$  ( $\pi$ 's write to  $\ell_x$ ), and therefore  $\rho$  reads 0 from  $\ell_x$  in line  $\rho 2$ . What is happening under the hood is that hardware and/or compilers may deduce that  $\pi$ 's writes are of *independent* memory locations, and thus may reorder them.<sup>4</sup>

C11, however, also provides certain ways of performing accesses such that all threads can agree that one access is ordered before the other. In particular, the remaining examples in [Figure 2.1](#) present several ways to create the *happens-before* relation between  $\pi$ 's write to  $\ell_x$  ( $\pi 1$ ) and  $\rho$ 's read from  $\ell_x$  ( $\rho 2$ ). We say to “establish synchronization” is to guarantee somehow that two memory events of interest are in the happens-before relation. Relaxed accesses are the weakest atomic accesses in C11 and do *not* guarantee happens-before. Thus, in [Example 2.1\(a\)](#), the relaxed accesses on  $\ell_y$  do not establish synchronization between the accesses on  $\ell_x$ .

SC ACCESSES (**sc**) are the strongest option to establish synchronization, and we use them in [Example 2.1\(b\)](#) for the accesses of  $\ell_y$ .<sup>5</sup> If  $\rho$ 's read of  $\ell_y$  (line  $\rho 1$ ) is not zero, then it reads from  $\pi$ 's write of 1 to  $\ell_y$  (line  $\pi 2$ ). By C11's semantics of SC accesses,  $\pi 2$  happens before  $\rho 1$ . Furthermore, SC accesses *should* prevent all<sup>6</sup> reorderings of other *intra-thread* accesses around them—i.e.,  $\pi 1$  cannot be reordered to after  $\pi 2$ , and  $\rho 2$  cannot be reordered to before  $\rho 1$ . As a result, we know that  $\pi 1$  happens before  $\rho 2$ —or in other words, that  $\rho$ 's read of  $\ell_x$  is *synchronized with*  $\pi$ 's write to it. Since the write of 42 is the most recent write to  $\ell_x$ , we know that thread  $\rho$  must read 42 in  $\rho 2$ .

RELEASE-ACQUIRE ACCESSES. Instead of using the costly SC accesses, we can use the release-acquire idiom to establish synchronization, as in [Example 2.1\(c\)](#). Here,  $\pi$  uses a release (**rel**) write in  $\pi 2$ , and  $\rho$  uses an

<sup>4</sup>Note that from thread  $\pi$ 's point of view, such reordering does not really matter as it cannot distinguish the effects, which, on the contrary, are distinguishable to the concurrently running thread  $\rho$ .

<sup>5</sup>According to C11, SC accesses can have subtle behaviors when mixed with other kinds of accesses. We refer interested readers to the RC11 paper ([Lah+17]) for more details. In this dissertation, we do not focus on SC accesses. We only mention them here for the purpose of demonstration.

<sup>6</sup>Both C11 and RC11 do not give such a guarantee, but SC accesses are required to be as strong as release/acquire accesses, so they prevent enough reorderings for our MP example to work.

acquire (**acq**) read in  $\rho 1$ . If  $\rho 1$  reads 1 from  $\pi 2$ , C11’s release-acquire semantics on the location  $\ell_x$  says that  $\pi 2$  happens before  $\rho 1$ . Furthermore, a release write prevents reordering other intra-thread reads and writes that appear *before* it to *after* it, so, again,  $\pi 1$  cannot be reordered to after  $\pi 2$ . Conversely, an acquire read prevents reordering other intra-thread reads and writes that appear *after* it to *before* it, so  $\rho 2$  cannot be reordered to before  $\rho 1$ . Consequently, we still have  $\pi 1$  happens before  $\rho 2$ . Note that release and acquire accesses are less costly to implement than SC accesses because they allow more reordering around them. Nevertheless, they are quite sufficient to establish synchronization in many RMC algorithms.<sup>7</sup>

<sup>7</sup>In x86-TSO ([Sew+10]), release and acquire accesses are the default and weakest atomic accesses.

**RELEASE-ACQUIRE FENCES.** We can also achieve release-acquire synchronization using relaxed accesses with *fences*, as in [Example 2.1\(d\)](#). Here, thread  $\pi$  performs a release fence (**fence<sub>rel</sub>**) after the write to  $\ell_x$ , and then relaxedly (**rlx**) writes to  $\ell_y$ . Meanwhile,  $\rho$  performs an acquire fence (**fence<sub>acq</sub>**) once it relaxedly reads 1 from  $\ell_y$ . Note that there is no happens-before relation between the relaxed accesses of  $\ell_y$ , but C11 guarantees happens-before between the accesses of  $\ell_x$  (lines  $\pi 1$  and  $\rho 3$ ) through chains of the form “release fence  $\rightarrow$  relaxed write  $\rightarrow$  relaxed read  $\rightarrow$  acquire fence”. That is, synchronization is guaranteed between the events *before* the release fence and the events *after* the acquire fence if the two fences are connected by the relaxed write and read.

In terms of reordering, a release fence prevents reorderings of other writes before it (to after it), while an acquire fence prevents reorderings of other reads after it (to before it). Combining those restrictions with the fact that  $\rho 1$  reads from  $\pi 3$ , we have that  $\pi 1$  happens before  $\rho 3$ .

**DATA RACES.** Note that in [Example 2.1\(a\)](#), where we do not have sufficient synchronization between the accesses to  $\ell_x$ , the worst thing can happen is that  $\rho$  would read unwanted values. However, if we were to replace the **rlx** accesses of  $\ell_x$  with non-atomic accesses (**na**), it would constitute a data race and the program would exhibit undefined behavior. In the remaining examples in [Figure 2.1](#), we always have sufficient synchronization (and hence no races) between the accesses of  $\ell_x$ , so we can use non-atomic accesses for those.

## 2.2 RC11, Formally

The axiomatic semantics of C11/RC11 relaxed memory models are defined in two steps:

- first, we generate a set of *candidate executions* for the program of interest, in form of *graphs* whose vertices are memory *events* generated by the program’s memory accesses and whose edges are several *partial orders* among the events;
- then, the behaviors of the program are those candidate executions that satisfy the model’s *consistency axioms*.

In the following, we provide an excerpt of the RC11 formalization—following Lahav et al.<sup>8</sup> closely with minor presentation deviations—that

<sup>8</sup>Lahav et al., “Repairing sequential consistency in C/C++11” [Lah+17].

are relevant to the features used in this dissertation. Interested readers can consult the original paper. Note that the formalizations in this chapter are also not included in our Coq developments. Again, they can be found in the RC11 paper’s artifacts.

### 2.2.1 Basic Definitions

A relaxed memory model only concerns about the possible orders between memory accesses, and thus can be separated from the language syntax. Therefore, we can delay our language syntax much later (Chapter 4). For the bare minimum, we assume the abstract types *Loc* for memory locations and *Val* for values stored in memory, with meta-variables  $\ell \in Loc$  and  $v \in Val$ , respectively.

First, we need the type of memory access consistency mode:

**Definition 2.2** (Memory Access Consistency Mode).

$$o \in AccessMode ::= \mathbf{sc} \mid \mathbf{acq} \mid \mathbf{rel} \mid \mathbf{relacq} \mid \mathbf{rlx} \mid \mathbf{na}.$$

*AccessMode*’s LATTICE

$$o_1 \sqsubseteq o_2$$

$$\begin{array}{ccc} \mathbf{na} \sqsubseteq o & \mathbf{rlx} \sqsubseteq \mathbf{acq} & \mathbf{rlx} \sqsubseteq \mathbf{rel} \\ \mathbf{rel} \sqsubseteq \mathbf{relacq} & \mathbf{acq} \sqsubseteq \mathbf{relacq} & o \sqsubseteq \mathbf{sc} \end{array}$$

**Definition 2.3** (Memory Access Event). Each memory access generates an event of type *MemEvent*, with the meta-variable  $\varepsilon$ .

$$\varepsilon \in MemEvent ::= R^o(\ell, v) \mid W^o(\ell, v) \mid U^{o_r, o_w}(\ell, v_r, v_w) \mid F^o.$$

Specifically:

- $R^o(\ell, v)$ : a Read of  $v$  from  $\ell$ , with access mode  $o \in \{\mathbf{na}, \mathbf{rlx}, \mathbf{acq}, \mathbf{sc}\}$ .
- $W^o(\ell, v)$ : a Write of  $v$  to  $\ell$ , with access mode  $o \in \{\mathbf{na}, \mathbf{rlx}, \mathbf{rel}, \mathbf{sc}\}$ .
- $U^{o_r, o_w}(\ell, v_r, v_w)$ : a read-modify-write (Update) to  $\ell$ , with read value  $v_r$  and write value  $v_w$ , and read access mode  $o_r \in \{\mathbf{rlx}, \mathbf{acq}, \mathbf{sc}\}$ , and write access mode  $o_w \in \{\mathbf{rlx}, \mathbf{rel}, \mathbf{sc}\}$ .<sup>9</sup>
- $F^o$ : a memory Fence, with  $o \in \{\mathbf{acq}, \mathbf{rel}, \mathbf{relacq}, \mathbf{sc}\}$ .

<sup>9</sup>Alternatively, RC11 models an Update event as a Read event immediately followed by a Write event. Here we follow C11. It is only a matter of presentation.

**Definition 2.4** (Memory Event Projections). For a memory event  $\varepsilon$ , the projections *loc*, *mod*, *val<sub>r</sub>*, and *val<sub>w</sub>* respectively give  $\varepsilon$ ’s location, access mode, read value and write value when applicable. More specifically, *loc* is only applicable for R, W, and U events; *val<sub>r</sub>* is applicable for R and U events; and *val<sub>w</sub>* is applicable for W and U events.

For U events, *mod* is defined as follows:

- $U^{\mathbf{rlx}, \mathbf{rlx}}(\_).mod ::= \mathbf{rlx}$
- $U^{\mathbf{acq}, \mathbf{rel}}(\_).mod ::= \mathbf{relacq}$
- $U^{\mathbf{rlx}, \mathbf{rel}}(\_).mod ::= \mathbf{rel}$
- $U^{\mathbf{sc}, \mathbf{sc}}(\_).mod ::= \mathbf{sc}$
- $U^{\mathbf{acq}, \mathbf{rlx}}(\_).mod ::= \mathbf{acq}$

**Notation 2.5** (Update Event Access Mode). Consequently, we also use the following shorthand notations for Update events:

- $U^{rlx}(\_) ::= U^{rlx,rlx}(\_)$
- $U^{relacq}(\_) ::= U^{acq,rel}(\_)$
- $U^{rel}(\_) ::= U^{rlx,rel}(\_)$
- $U^{sc}(\_) ::= U^{sc,sc}(\_)$
- $U^{acq}(\_) ::= U^{rlx,acq}(\_)$

**Notation 2.6** (Memory Event Sets). The notations  $R$ ,  $W$ ,  $U$ , and  $F$  respectively denote sets of Read, Write, Update, and Fence events.

We may also combine event sets, e.g.,  $RW ::= R \cup W$ . We use subscript and superscript respectively to filter the sets by accessed location and access mode, e.g.,  $W_{\ell}^{\text{rel}} ::= \{\varepsilon \in W \mid \varepsilon.\text{loc} = \ell \wedge \varepsilon.\text{mod} \sqsupseteq \text{rel}\}$ .

**Notation 2.7** (Memory Event Relations). For a binary relation on events  $R \in \text{MemEvent} \times \text{MemEvent}$ ,  $R^?$ ,  $R^+$ , and  $R^*$  respectively denote its reflexive, transitive, and reflexive-transitive closures.  $\text{dom}(R)$  and  $\text{codom}(R)$  denote the domain and co-domain of  $R$ , respectively.

The notation  $R_1 ; R_2$  denotes the left composition of two relations  $R_1$  and  $R_2$ . We assume that  $;$  binds stronger than  $\cup$  and  $\setminus$ . The notation  $[A]$  stands for the identity relation on the set  $A$ . Consequently,  $[A] ; R$  can be understood as filtering  $R$  on the left with  $A$ , while  $R ; [B]$  filters  $R$  on the right with  $B$ . That is,  $[A] ; R = \{(\varepsilon_a, \varepsilon_b) \in R \mid \varepsilon_a \in A\}$ , and  $R ; [B] = \{(\varepsilon_a, \varepsilon_b) \in R \mid \varepsilon_b \in B\}$ . Finally,  $[A] ; R ; [B] = R \cap (A \times B)$ .

Given a function  $f$ ,  $=_f$  and  $\neq_f$  denote the binary relations of pairs that are  $f$ -equal and  $f$ -non-equal, respectively:

$$\begin{aligned} =_f &::= \{(\varepsilon_a, \varepsilon_b) \mid f(\varepsilon_a) = f(\varepsilon_b)\} \\ \neq_f &::= \{(\varepsilon_a, \varepsilon_b) \mid f(\varepsilon_a) \neq f(\varepsilon_b)\} \end{aligned}$$

Meanwhile, given a relation  $R'$ ,  $R|_{R'}$  denotes the filtering of  $R$  with respect to  $R'$ , i.e.,  $R|_{R'} ::= R \cap R'$ . For example,  $R|_{=_{\text{loc}}}$  and  $R|_{\neq_{\text{loc}}}$  denote the relation  $R$  restricted to same and different locations, respectively.

### 2.2.2 Execution Graphs

**Definition 2.8** (Execution Graph). An execution graph  $G$  is a tuple  $(E, \text{po}, \text{rf}, \text{mo})$ :

- $E$  is the set of memory events ( $\text{MemEvent}$ ) in  $G$ .
- The *program order*  $\text{po}$  is a *strict*<sup>10</sup> partial order that orders each thread's event by the program's control flow. For simplicity, RC11 assumes that for each location  $\ell$ ,  $E$  contains a Write event  $\varepsilon_0^\ell ::= \text{wna}(\ell, 0)$  as the initialization for  $\ell$ .  $\text{po}$  is then required to order initialization events before all other events, i.e.,  $E_0 \times (E \setminus E_0) \subseteq \text{po}$  where  $E_0 ::= \{\varepsilon_0^\ell \in E\}$  is the set of  $E$ 's initialization events.
- The *reads-from* relation  $\text{rf}$  relates a write with a read that reads from it, i.e.,

$$(i) \quad \text{rf} \subseteq [WU] ; =_{\text{loc}} ; [RU];^{11} \text{ and}$$

<sup>10</sup>It is irreflexive, i.e.,  $(\varepsilon, \varepsilon) \notin \text{po}$ .

<sup>11</sup>Recall that an Update event is considered both a read and a write.

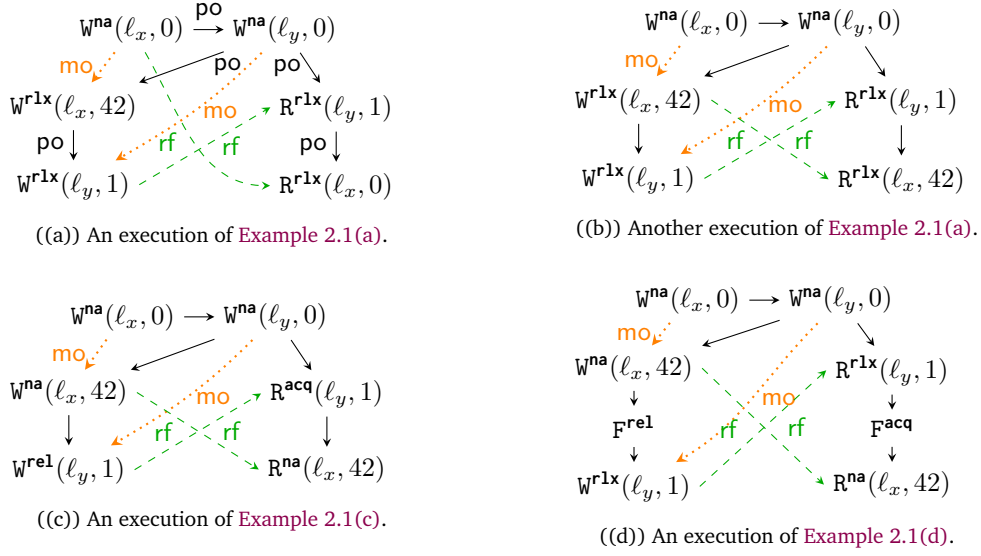


FIGURE 2.2: Candidate executions of several MP examples.

(ii) **rf** respects written and read values:  $\varepsilon_w.\text{val}_w = \varepsilon_r.\text{val}_r$  for all  $(\varepsilon_w, \varepsilon_r) \in \text{rf}$ ; and

(iii) **rf** is injective: if  $(\varepsilon_w^1, \varepsilon_r) \in \text{rf}$  and  $(\varepsilon_w^2, \varepsilon_r) \in \text{rf}$  then  $\varepsilon_w^1 = \varepsilon_w^2$ .

- The *modification order* **mo** is a strict partial order that gives a *strict total order* on the write events of each location. That is, **mo** is a disjoint union of the relations  $\{\text{mo}_\ell\}_{\ell \in \text{Loc}}$  where  $\text{mo}_\ell$  is a strict total order on  $(\text{WU})_\ell$ .

The components are also used as projections, e.g.,  $G.\text{mo}$ . In cases where  $G$  is clear in the context, we may also drop the “ $G$ .” part and just use **mo**.

**Definition 2.9** (Candidate Execution). Execution graphs of a program  $\mathcal{P}$  encode *prefixes* of traces of events generated by the program’s memory accesses and fences. A execution  $G$  is a *candidate execution* if it represents a *full* trace generated by the whole program  $\mathcal{P}$ .

**Example 2.10** (Candidate Executions for MP). **Figure 2.2** gives a few candidate executions for several MP examples in **Figure 2.1**. We use filled arrows, **dotted** arrows, and **dashed** arrows—with the same colors—for **po**, **mo**, and **rf** edges, respectively, between events. To avoid cluttering, we sometimes elide edge labels and instead use the arrow style to make the edge’s type evident.

**Definition 2.11** (Complete Execution). An execution  $G$  is *complete* if every read reads some written value, i.e.,  $G.\text{R} \subseteq \text{codom}(G.\text{rf})$ . A candidate execution is always complete, but the reverse is not always true.

**Definition 2.12** (Derived Relations). RC11 defines the following derived partial orders on execution graphs.

$$\begin{aligned}
\text{rb} &::= (\text{rf}^{-1} ; \text{mo}) \setminus [E] && (\text{reads before}) \\
\text{eco} &::= (\text{rf} \cup \text{mo} \cup \text{rb})^+ && (\text{extended coherence order}) \\
\text{rs} &::= [\text{WU}] ; \text{po}|_{=\text{loc}}^? ; [\text{WU} \sqsupset \text{rlx}] ; (\text{rf} ; [\text{U}])^* && (\text{release sequence}) \\
\text{sw} &::= [E \sqsupset \text{rel}] ; ([F] ; \text{po})^? ; \text{rs} ; \text{rf} ; && (\text{synchronized-with}) \\
&\quad [\text{RU} \sqsupset \text{rlx}] ; (\text{po} ; [F])^? ; [E \sqsupset \text{acq}] \\
\text{hb} &::= (\text{po} \cup \text{sw})^+ && (\text{happens-before}) \\
\text{psc} &::= \dots \text{ (elided)} && (\text{partial SC})
\end{aligned}$$

- The reads-before relation **rb** relates a Read event  $\varepsilon_r$  and a Write event  $\varepsilon_w$ , where  $\varepsilon_r$  reads from **(rf)** a write that is **mo**-before  $\varepsilon_w$ . The “ $\setminus [E]$ ” part is to exclude the case where an Update event reads from itself.
- The extended coherence order **eco** is the transitive closure of **rf**, **mo**, and **rb**, and is defined by RC11 to remedy C11’s behaviors for SC accesses and fences.<sup>12</sup>
- The release sequence **rs** of a Write event  $\varepsilon_w$  contains (i) all later same-thread, same-location ( $\text{po}|_{=\text{loc}}$ -later) atomic writes ( $\text{WU} \sqsupset \text{rlx}$ ) including the write  $\varepsilon_w$  itself—hence the reflexive closure (?) of  $\text{po}|_{=\text{loc}}$ , as well as (ii) all Updates that recursively read from such writes.
- The synchronized-with relation **sw** defines *inter*-thread synchronization. A release event  $\varepsilon_a \in E \sqsupset \text{rel}$  is synchronized with an acquire event  $\varepsilon_b \in E \sqsupset \text{acq}$ , if  $\varepsilon_b$  (or, in case  $\varepsilon_b$  is a Fence event, some atomic Read event that is po-before  $\varepsilon_b$ ) reads from the release sequence of  $\varepsilon_a$  (or in case  $\varepsilon_a$  is a Fence event, some atomic Write event that is po-after  $\varepsilon_a$ ). Note that the relation **rs** ; **rf** is between a Write event and a Read event. The relations  $([F] ; \text{po})^?$  and  $(\text{po} ; [F])^?$  allow us to extend **rs** ; **rf** to fences that come po-before and po-after the Write and the Read events in **rs** ; **rf**, respectively.
- Most importantly, the happens-before relation **hb** formally defines what *global* synchronization means, as the transitive closure of the *inter*-thread synchronization **sw** relation and the *intra*-thread program order **po**.
- Finally, the partial SC relation **psc** is defined by RC11 to rectify SC behaviors, using a diligent combination of **mo**, **rf**, **rb**, **eco**, and **hb**. The exact definition, however, is not in the focus of this dissertation and therefore elided.

<sup>12</sup>Lahav et al., “Repairing sequential consistency in C/C++11” [Lah+17].

**Example 2.13** (Illustrations of Derived Relations). Figure 2.3 demonstrates the derived relations on several execution graphs. Figure 2.3(e) especially demonstrates a fairly complex instance of the release sequence **rs** relation with 4 threads, of which the middle 2 threads use Updates (atomic read-modify-write instructions).

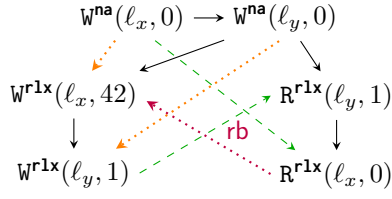
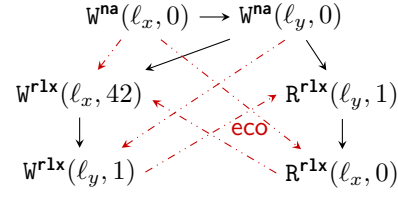
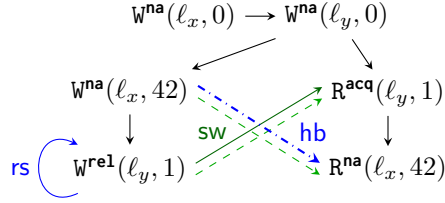
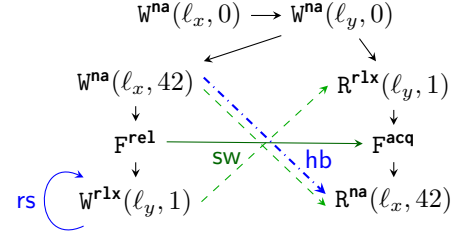
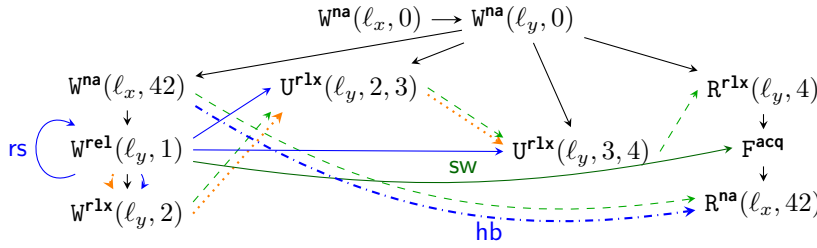
((a)) **rb** on the execution in Figure 2.2(a).((b)) **eco** (transitive closure elided) derived from Figure 2.3(a).((c)) **rs**, **sw**, and a part of **hb** on the execution in Figure 2.2(c).((d)) **rs**, **sw**, and a part of **hb** on the execution in Figure 2.2(d).((e)) More complex **rs**, **sw**, and **hb** on an execution with Updates.

FIGURE 2.3: Illustrations of derived relations.

We use **dotted** arrows, **dash-dot-dotted** arrows, **filled** arrows, **filled** arrows, and **dash-dotted** arrows, respectively for **rb**, **eco**, **rs**, **sw**, and **hb** edges.

### 2.2.3 Consistency

**Definition 2.14** (RC11-consistency). An execution  $G$  is *RC11-consistent* if it is complete (Definition 2.11) and

- **hb** ; **eco**<sup>?</sup> is irreflexive; and (RC11-COHERENCE)
- **psc** is acyclic; and (RC11-SC)
- $\text{po} \cup \text{rf}$  is acyclic. (RC11-NO-OOTA)

**RC11-COHERENCE** is the main axiom that give sane behaviors to most memory operations—see Proposition 2.19 below. **RC11-SC** is the main contribution of the RC11 work to give better semantics for SC accesses and fences, which, again, is not in the focus of this dissertation and is only stated here for completeness. The **RC11-NO-OOTA** condition is a simple fix to forbid *load-buffering* (LB) behaviors, and therefore forbids the *out-of-thin-air* problem<sup>13</sup>—see Remark 2.21 below.

<sup>13</sup>Boehm and Demsky, “Outlawing ghosts: avoiding out-of-thin-air results” [BD14].



$\ell_x :=_{\text{na}} 0; \ell_y :=_{\text{na}} 0;$   
 $\pi 1: \ell_x :=_{\text{na}} 42; \quad \parallel \quad \rho 1: \text{if } *_{\text{rlx}} \ell_y \neq 0 \text{ then}$   
 $\pi 2: \ell_y :=_{\text{rlx}} 1; \quad \parallel \quad \rho 2: \quad *_{\text{na}} \ell_x; \text{ // racy}$   
 ((a)) A racy MP program.

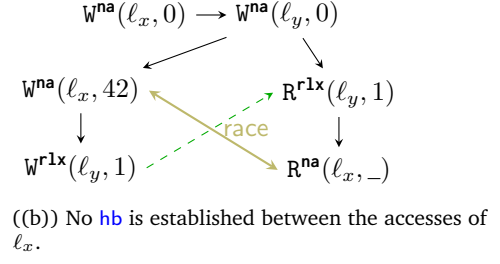


FIGURE 2.4: A racy execution of a racy MP program.

#### 2.2.4 Data Races

**Definition 2.15** (Races). Two events  $\varepsilon_a$  and  $\varepsilon_b$  are *conflicting* in an execution  $G$  if they are on the same location and one of them is a write, i.e.,  $\varepsilon_a \neq \varepsilon_b$  and  $\varepsilon_a.\text{loc} = \varepsilon_b.\text{loc}$  and  $\{\varepsilon_a, \varepsilon_b\} \cap G.(\text{WU}) \neq \emptyset$ .

The pair  $(\varepsilon_a, \varepsilon_b)$  is called a *race* in  $G$ , denoted  $(\varepsilon_a, \varepsilon_b) \in G.\text{race}$ , if they are conflicting in  $G$  and neither happens before the other, i.e.,  $(\varepsilon_a, \varepsilon_b) \notin \text{hb} \cup \text{hb}^{-1}$ .

**Definition 2.16** (Racy Executions). An execution  $G$  is called *racy* if there is some racy event pair in  $G$  such that one of them is a non-atomic access, i.e.,  $\exists(\varepsilon_a, \varepsilon_b) \in G.\text{race} \wedge \{\varepsilon_a, \varepsilon_b\} \cap E^{\text{na}} \neq \emptyset$ .

**Example 2.17** (Racy Execution of MP). A racy MP program and one of its racy executions is given in Figure 2.4. The race is between the non-atomic accesses of  $\ell_x$ , where no **hb** edge is established between the accesses, because we use only relaxed accesses for  $\ell_y$ .

#### 2.2.5 Program Behaviors

**Definition 2.18** (RC11 Program Behavior). A program  $\mathcal{P}$  has *undefined behavior* (UB) under RC11 if it has some racy RC11-consistent execution. Otherwise, its behaviors are defined by the set of RC11-consistent *full* executions of  $\mathcal{P}$ .

**Proposition 2.19** (RC11 and C11 Coherence). **RC11-COHERENCE** is equivalent to the conjunction of the following C11 axioms:<sup>14</sup>

- **hb** is irreflexive. (C11-HB)
- **rf**; **hb** is irreflexive. (C11-NO-FUTURE-READ)
- **mo**; **rf**; **hb** is irreflexive. (C11-CoRW)
- **mo**; **hb** is irreflexive. (C11-CoWW)
- **mo**; **hb**; **rf**<sup>-1</sup> is irreflexive. (C11-CoWR)
- **mo**; **rf**; **hb**; **rf**<sup>-1</sup> is irreflexive. (C11-CoRR)

**Example 2.20** (C11 Coherence). C11 coherence axioms are demonstrated by several forbidden (non-consistent) behaviors<sup>15</sup> in Figure 2.5.

- **C11-HB** ensures that **hb** is a strict partial order.
- **C11-NO-FUTURE-READ** (Figure 2.5(a)) says that a read may not happen before the write that it reads from.

<sup>14</sup>Lahav et al., “Repairing sequential consistency in C/C++ +11” [Lah+17], §3.4, Proposition 1.

<sup>15</sup>Batty et al., “Mathematizing C++ concurrency” [Bat+11], §2.7.

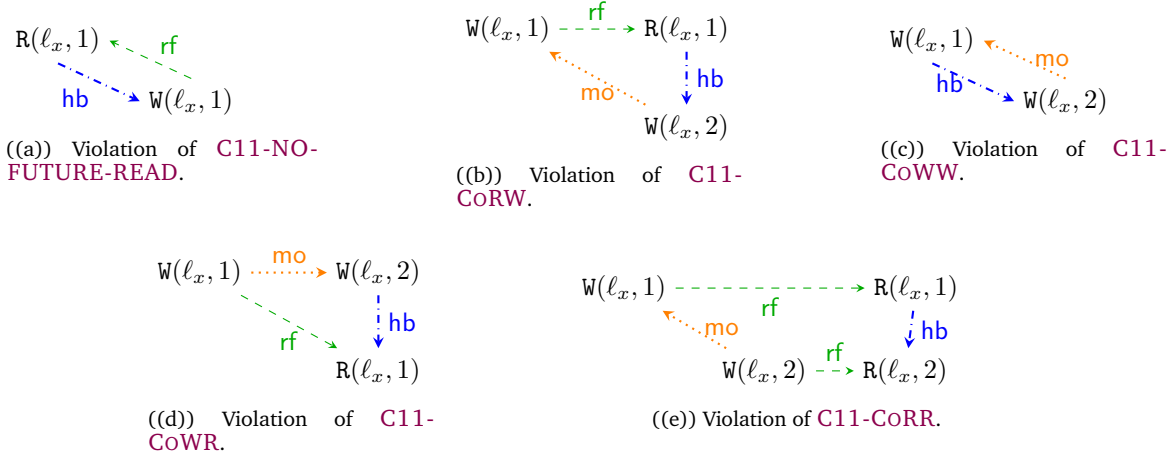


FIGURE 2.5: Several *forbidden* (inconsistent) executions in C11/RC11.

- **C11-CoRW** (Figure 2.5(b)) requires that a read may not happen before a write that **mo**-before the write it reads from.
- **C11-CoWW** (Figure 2.5(c)) requires that **mo** and **hb** may not disagree.
- **C11-CoWR** (Figure 2.5(d)) requires that a read may not read from a write that is already hidden by (**mo**-before) another write that happens-before it.
- **C11-CoRR** (Figure 2.5(e)) requires that two reads connected by **hb** may not read from writes with the inverse order in **mo**.

**Remark 2.21** (LB and OOTA). The C11 memory model allows the so-called Load-Buffering (LB) behavior, while the RC11 memory model simply forbids it with **RC11-NO-OOTA**. Figure 2.6(a) gives an example program with an execution demonstrating its LB behavior in Figure 2.6(c). Here, one can think that the reads (loads) are buffered until the writes are completed, and then they can both read 1. The execution in Figure 2.6(c) is consistent in C11, and so such behavior is allowed in C11.

The problem with LB is that, the same execution in Figure 2.6(c) justifies an *undesirable* behavior of the program in Figure 2.6(b), where the reads read 1, even though 1 does not appear in the program: it appears out of thin air (OTA)! OTA behaviors are forbidden by the *informal* C11 standard, and are not exhibited in any implementation. However, it is formally non-trivial to distinguish LB, which is desirable, from OTA, which is not. Several solutions are already proposed to distinguish them,<sup>16</sup> but they result in more involved semantics. Furthermore, the LB behavior itself is rather non-local and makes it hard to build high-level, logic-based reasoning—see [Sve+18] for an attempt.

RC11 resolves to a simpler solution: forbidding LB behaviors altogether, by requiring  $\text{po} \cup \text{rf}$  to be acyclic (**RC11-NO-OOTA**). Similar to existing logics,<sup>17</sup> we also adopt this solution in ORC11 (which is an operational version of RC11), as it simplifies the construction of our separation logic. Recent work by Ou and Demsky<sup>18</sup> suggests that the performance

<sup>16</sup>Kang et al., “A promising semantics for relaxed-memory concurrency” [Kan+17]; Chakraborty and Vafeiadis, “Grounding thin-air reads with event structures” [CV19].

<sup>17</sup>[VN13; DV16; DV17].

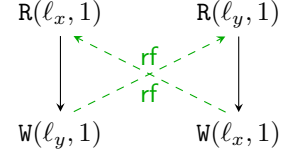
<sup>18</sup>Ou and Demsky, “Towards understanding the costs of avoiding out-of-thin-air results” [OD18].

$$\begin{array}{l} *rlx \ell_x; // 1 \\ \ell_y := rlx 1; \end{array} \parallel \begin{array}{l} *rlx \ell_y; // 1 \\ \ell_x := rlx 1; \end{array}$$

(a) A program with LB behaviors.

$$\begin{array}{l} a := *rlx \ell_x; // 1 \\ \text{if } (a) \text{ then} \\ \ell_y := rlx a; \end{array} \parallel \begin{array}{l} b := *rlx \ell_y; // 1 \\ \text{if } (b) \text{ then} \\ \ell_x := rlx b; \end{array}$$

(b) A program with OOTA behaviors in C11.



(c) An execution with LB behaviors.

overhead of working with RC11 vs. C11 may not be so significant in practice. Even more recently, Lee et al.<sup>19</sup> suggests forbidding LB behaviors in the source language model (e.g., C and C++) but still allowing LB in the intermediate representation (IR) semantics. This would result in a compromising memory model where logics like iRC11 can be used for verifications at the source level, and where all optimizations for non-atomics can still be performed at the IR level, and therefore imposing no overhead on non-atomics.

**Remark 2.22** (Consume Accesses and Locks). Unlike C11, RC11 does not consider *consume* accesses, which is a premature feature not implemented by major compilers, nor *locks*, which can be implemented with release-acquire accesses.

FIGURE 2.6: Load-buffering (LB) and Out-of-thin-air (OOTA) behaviors.

<sup>19</sup>Lee et al., “Putting Weak Memory in Order via a Promising Intermediate Representation” [Lee+23].

**CHAPTER SUMMARY.** This chapter reviews the high-level intuition for the behaviors of C11 atomic accesses, as well as an excerpt of the RC11 formalization in form of axiomatic semantics. The distinctive feature of the RMC axiomatic semantics is the use of axioms to constrain partial orders among memory events. While this style results in very concise definitions,<sup>20</sup> it may take time to get used to. Nevertheless, the main inconvenience of axiomatic semantics is that the behaviors are encoded in the axioms that are stated rather *globally* on relations that span multiple events across multiple threads, making it difficult to prove soundness of *thread-local*, Hoare-style CSLs directly on top of those semantics.<sup>21</sup> In the next chapter, we present ORC11, an operational version of RC11 that is more convenient to build our separation logic iRC11 in Iris.

<sup>20</sup>In contrast, the formulation of ORC11 in Chapter 3 is much more verbose.

<sup>21</sup>Yet it is still achievable, by annotating resources on incoming and outgoing edges of an event node, as Vafeiadis et al. demonstrated with RSL and FSL ([VN13; DV16]).



# 3

## ORC11: Operational Repaired C11

Following iGPS,<sup>1</sup> we need an operational semantics for relaxed memory so that it can be instantiated in the Iris framework. We extend iGPS’s operational semantics for release-acquire and non-atomics (RA+NA) to include relaxed accesses and fences. The result is ORC11—Operational Repaired C11.<sup>2</sup>

Features-wise, ORC11 is closely related to the axiomatic semantics of RC11.<sup>3</sup> Most importantly, it forbids load-buffering (LB) behaviors, *i.e.*,  $\text{po} \cup \text{rf}$  is acyclic. Construction-wise, ORC11 follows the *view*-based approach to operational semantics for relaxed memory.<sup>4</sup> More concretely, it follows the promising semantics formalization<sup>5</sup> but *without* promises, and thus forbids LB. The promising semantics, however, does not model non-atomics.<sup>6</sup> Meanwhile, ORC11 needs to employ a race detector to formalize races on non-atomics which, as we will see, in the presence of relaxed accesses, are more tricky to get right than iGPS’s race detector.

Consequently, ORC11 is defined by two sub-semantics: the *view*-based *machine* semantics that focuses on relaxed behaviors (§3.3) and the *race-detector* semantics that focuses on UB-triggering races (§3.4). In §3.6, we sketch a paper proof of correspondence between ORC11 and RC11.

The *expression* semantics, which defines the reductions of language expressions, can fortunately be mostly separated from the relaxed memory model that is ORC11. Chapter 4 will present the relaxed  $\lambda_{\text{Rust}}$  language which combines the expression reductions together with ORC11.<sup>7</sup>

But first, let us give a high-level, intuitive explanation of RMC using views.

### 3.1 Understanding Relaxed Memory with Views

The *view*-based approach to operational semantics for relaxed memory allows for a more *thread-local* characterization of relaxed effects. In particular, each thread in the program has its own *local view* which represents its subjective *observations* on the *globally-shared* memory.

For example, a thread  $\pi$ ’s local view may record (but not limited to) the writes to memory that the thread has observed, *e.g.*, those writes that *happen before* the current program counter  $\text{PC}_\pi$  of the thread  $\pi$ . More concretely, if we follow the language of iGPS<sup>8</sup> and track writes to memory in views, we can define a view as a map from memory locations

<sup>1</sup>Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

<sup>2</sup>The definitions in this chapter are formalized in Coq in the repository <https://gitlab.mpi-sws.org/iris/gpfs1/-/tree/master/orc11>.

<sup>3</sup>Lahav et al., “Repairing sequential consistency in C/C++11” [Lah+17].

<sup>4</sup>Steinke and Nutt, “A unified theory of shared memory consistency” [SN04]; Lahav et al., “Taming release-acquire consistency” [LGV16]; Podkopaev et al., “Operational Aspects of C/C++ Concurrency” [PSN16].

<sup>5</sup>Kang et al., “A promising semantics for relaxed-memory concurrency” [Kan+17].

<sup>6</sup>It models *plain* accesses, who can race without triggering undefined behaviors.

<sup>7</sup>And thus the semantics of the relaxed  $\lambda_{\text{Rust}}$  language is a combination of three sub-semantics: the expression semantics, the *view*-based machine semantics, and the *race-detector* semantics.

<sup>8</sup>Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

<sup>9</sup>Timestamps are typically just natural numbers, but can be more complex depending on the memory model.

to timestamps:  $View ::= Loc \xrightarrow{\text{fin}} Time$ , where the timestamps are indices into an ordering of the writes to a location.<sup>9</sup> With  $V(\ell) = t$ , we say that the view  $V$  has *seen* or observed the write to  $\ell$  identified by the timestamp  $t$ . When thread  $\pi$  writes to a location  $\ell$  in shared memory, a new write event  $\varepsilon_w$  is added to the shared memory with some fresh timestamp  $t_w$ , and thread  $\pi$  updates its local view (its observations)  $V_\pi$  accordingly to include  $t_w$  for  $\ell$ .

However, it is not necessary that another thread  $\rho$  observes that write  $\varepsilon_w$  by thread  $\pi$  immediately. In the terminology of views, we say that thread  $\rho$ 's local view  $V_\rho$  does not include the timestamp  $t_w$  for  $\ell$ . In order to observe the write  $\varepsilon_w$ , thread  $\rho$  needs to perform physical synchronization with thread  $\pi$ , so that thread  $\pi$ 's local view  $V_\pi$  (which includes  $\varepsilon_w$ ) is incorporated or *joined* into  $V_\rho$ . Then,  $V_\pi$  is *included* in  $V_\rho$ :  $V_\pi \sqsubseteq V_\rho$ . The *view inclusion* relation therefore approximates synchronization, or more formally, the happens-before (**hb**) relation. Consequently, as threads execute and their local views grow over time, they occasionally synchronize with one another by *sending* their local views to other threads.

**Example 3.1** (Racy MP with Views). Consider again the racy MP example in Figure 2.4 (Example 2.17), where we use relaxed accesses for  $\ell_y$  and therefore we are not guaranteed a happens-before relation between the conflicting non-atomic accesses to  $\ell_x$  by thread  $\pi$  and thread  $\rho$ . In the language of views, this racy behavior can be explained as follows, using Figure 3.1(a).

- After thread  $\pi$  writes 42 to  $\ell_x$ , its local view is  $V_\pi^1$ , as illustrated in Figure 3.1(a) with an arrow pointing to after the write. As an approximation of **hb**,  $V_\pi^1$  also tracks the po relation in  $\pi$ , and thus it includes the *freshly created* timestamp  $t_x^1$  for the write 42 to  $\ell_x$ , i.e.,  $V_\pi^1(\ell_x) = t_x^1$ .
- Similarly, after thread  $\pi$  writes 1 to  $\ell_y$ , its local view  $V_\pi^2$  includes the timestamp  $t_y$ :  $V_\pi^2(\ell_y) = t_y^2$ . More importantly,  $V_\pi^1 \sqsubseteq V_\pi^2$ : a thread-local view only grows, so as to maintain that happens-before (**hb**) contains program order (**po**).
- Unfortunately, because we use a relaxed access in writing 1 to  $\ell_y$ , thread  $\pi$  does *not* release its local view (neither  $V_\pi^1$  nor  $V_\pi^2$ ) with that write.
- So even if thread  $\rho$  reads that write of 1 to  $\ell_y$ , it does *not* acquire the timestamp  $t_x^1$  (for the write of 42 to  $\ell_x$ ) into its local view after the read  $V_\rho^1$ .
- Consequently, when reading  $\ell_x$  non-atomically in the next line, thread  $\rho$ 's current local view  $V_\rho^1$  is not guaranteed to include  $t_x^1$ . In the operational semantics, performing a non-atomic operation without having observed *all* writes to the same location in the local view constitutes a race.

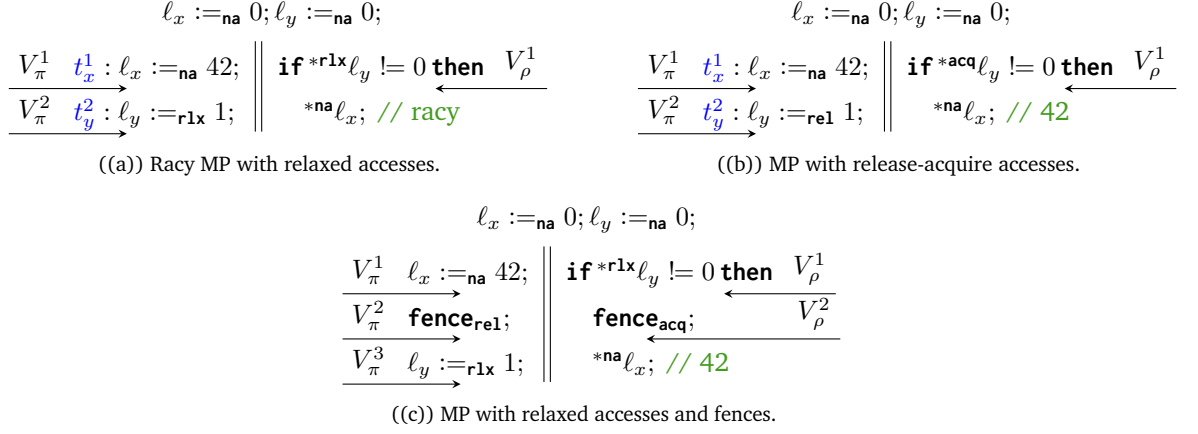


FIGURE 3.1: View-based explanation of MP behaviors.

Put it differently, that thread  $\rho$ 's view before the non-atomic read of  $\ell_x$  does not include  $t_x^1$  *approximates* the fact that the non-atomic write to  $\ell_x$  does not happen before the non-atomic read, hence a race.

**Example 3.2** (Release-Acquire MP with Views). Similarly, the release-acquire synchronization in Figure 2.1(c) (Example 2.1) can also be explained with views, using Figure 3.1(b). Because we instead use a release write of 1 to  $\ell_y$ , thread  $\pi$  *releases* its local view  $V_\pi^2$  through the write (which also include the write itself). When thread  $\rho$  reads that write using an acquire read, it *acquires* that view into its local view  $V_\rho^1$ . Accordingly,  $V_\pi^2 \sqsubseteq V_\rho^1$ , so thread  $\rho$  has observed all writes to  $\ell_x$ , and can safely read  $\ell_x$  non-atomically. Furthermore, thread  $\rho$  reads from  $\ell_x$ 's latest write, which is 42.

In other words,  $V_\pi^2 \sqsubseteq V_\rho^1$  encodes the synchronized-with (**sw**) relation between the release-acquire pair, and transitively the happens-before relation (**hb**). Effectively, thread  $\pi$ 's write of 42 to  $\ell_x$  happens before thread  $\rho$ 's read from  $\ell_x$ , the race is excluded, and the expected behavior results.

**Example 3.3** (Fence-MP with Views). The view-based explanation for the MP example with fences in Figure 2.1(d) (Example 2.1) is a bit more interesting, using Figure 3.1(c).

- Here, thread  $\pi$ 's relaxed write to  $\ell_y$ , like in Example 3.1, does not release the *current* local view  $V_\pi^3$  at the point of the write. However, unlike in Example 3.1,  $\pi$ 's release fence which comes before guarantees that the write to  $\ell_y$  does release  $\pi$ 's local view *before* the fence, i.e.,  $V_\pi^1$ .
- On the other side,  $\rho$ 's read of  $\ell_y$ , if reads 1, will acquire  $V_\pi^1$ , but does not immediately *join*  $V_\pi^1$  into its local view  $V_\rho^1$  right after the read, i.e.,  $V_\pi^1 \not\sqsubseteq V_\rho^1$ . Instead, later, thread  $\rho$ 's acquire fence will perform that join, so that after the acquire fence,  $V_\pi^1 \sqsubseteq V_\rho^2$ . Consequently, we again have the **hb** relation between the non-atomic write and read of  $\ell_x$ .

In other words, to explain fences behaviors in terms of views, we requires more views than just the *current* thread-local view: a release fence *stores*

the current view (by the time of the fence), so that it can be released through some *later* relaxed write, while an acquire fence *restores* (into the current view) some view that have been acquired by some *earlier* relaxed read. This is in agreement with the definition of the synchronized-with (*sw*) relation (Definition 2.12).

**SUMMARY.** Views are an approximation of the happens-before (*hb*) relation that is more thread-local and can help simplify the soundness proof of RMC separation logics. However, we need more intricate uses of views to handle fences (§3.3), which need multiple views, and to handle data races (§3.4), which, due to their subtle interactions with relaxed (*rlx*) accesses, require views to have a more complex structure than just a map from locations to timestamps.

### 3.2 Basic Machine State Definitions

We define the basic definitions of ORC11’s machine state, whose most important components are the *globally-shared* memory and the *thread-local* views. First, we note some extra features that affect the formal definitions of ORC11.

<sup>10</sup>Leroy et al., *The CompCert Memory Model, Version 2* [Ler+12].

**Pointer Arithmetic** The  $\lambda_{\text{Rust}}$  language (Chapter 4) adopts the CompCert model for locations,<sup>10</sup> where allocations and deallocations are done in *blocks*, and a location consists of a block index  $i$  and an offset  $n$  into that block, and pointer arithmetics can only be performed within the same block. Consequently, we need to model explicit allocations and deallocations of blocks.

**Uninitialized Memory**  $\lambda_{\text{Rust}}$  also allows memory to be uninitialized, with the only safe operations are reading and writing to uninitialized memory—other uses of values read from uninitialized memory are undefined behavior. We follow  $\lambda_{\text{Rust}}$ , which in turn follows Lee et al.<sup>11</sup> to use a *poison value*  $\text{\textcircled{X}}$  for uninitialized memory.

<sup>11</sup>Lee et al., “Taming undefined behavior in LLVM” [Lee+17].

**Data Races** To handle the interactions between races and *rlx* accesses, ORC11 views cannot just simply track write events (like in iGPS and what we have seen in §3.1). Instead, ORC11 views need to track both read and write events.

**Definition 3.4** (ORC11 Basic Types).

$$\begin{aligned} \pi, \rho \in \text{Thread} &::= \mathbb{N}^+ \\ \ell \in \text{Loc} &::= (i, n) \quad i \in \mathbb{N}^+, n \in \mathbb{Z} \\ v \in \text{Val} &::= \text{\textcircled{X}} \mid \dots \\ \omega \in \text{MemVal} &::= \dagger \mid \spadesuit \mid v \in \text{Val} \\ t \in \text{Time} &::= \mathbb{N}^+ \\ \alpha \in \text{ActIds} &::= 2^{\mathbb{N}^+} \end{aligned}$$

- A thread-id  $\pi$  or  $\rho$  is a positive number.



- A location  $\ell$  is a pair of block index  $i$  (which is a positive number) and an offset  $n$ .
- The value type  $Val$  can still be abstract, but should include the poison value  $\star$ .
- The *memory value* type  $MemVal$  is the type for values stored in locations the global memory, which can be in  $Val$  or be the two additional values  $\dagger$  and  $\clubsuit$  to respectively mark the *allocated* and *deallocated* states of a location.
- A timestamp  $t$  is a positive number.
- A set of *actions*  $\alpha$  is a set of positive numbers, which will be used to track sets of reads and writes.

**Definition 3.5** (ORC11 Memory Access Event).

$$\varepsilon \in MemEvent ::= | R^o(\ell, v) | W^o(\ell, v) | U^{or, ow}(\ell, v_r, v_w) | F^o \\ | A(\ell, n \in \mathbb{N}^+) | D(\ell, n \in \mathbb{N}^+).$$

We extend memory events (Definition 2.3) to include two new event types: the *allocation* event type  $A$  and *deallocation* event type  $D$ . Both event types carry the *base* location  $\ell$  of a block, and the size  $n$  of that block.

**Definition 3.6** (Views).

$$V \in View ::= Loc \xrightarrow{\text{fin}} \{w : Time, aw : ActIds, nr : ActIds, ar : ActIds\}$$

A (simple) view is a finite, partial map from locations to tuples of one timestamp and three sets of actions. For a view  $V$  and a location  $\ell$ ,

- $V(\ell).w$  is the timestamp of the latest write to  $\ell$  that  $V$  has seen.
- $V(\ell).aw$  is the set of *atomic* writes to  $\ell$  that  $V$  has seen.
- $V(\ell).nr$  is the set of *non-atomic* reads from  $\ell$  that  $V$  has seen.
- $V(\ell).ar$  is the set of *atomic* reads from  $\ell$  that  $V$  has seen.

**Definition 3.7** (Views' Join Semi-Lattice). The bottom element of views is the empty map  $\emptyset$ . The inclusion relation and the join operation for views are defined as follows.

$$\begin{aligned} V_1 \sqsubseteq V_2 &::= \forall \ell. V_1(\ell).w \leq V_2(\ell).w \wedge V_1(\ell).aw \subseteq V_2(\ell).aw \\ &\quad \wedge V_1(\ell).nr \subseteq V_2(\ell).nr \wedge V_1(\ell).ar \subseteq V_2(\ell).ar \\ V_1 \sqcup V_2 &::= \lambda \ell. \{ w := \max(\{V_1(\ell).w, V_2(\ell).w\}); \\ &\quad aw := V_1(\ell).aw \cup V_2(\ell).aw; \\ &\quad nr := V_1(\ell).nr \cup V_2(\ell).nr; \\ &\quad ar := V_1(\ell).ar \cup V_2(\ell).ar \} \end{aligned}$$

Note that  $V_1 \sqcup V_2$  is only defined for locations that are in the domains of either  $V_1$  or  $V_2$ , i.e.,  $\text{dom}(V_1 \sqcup V_2) = \text{dom}(V_1) \cup \text{dom}(V_2)$ . If some location

$\ell$  is not in one view, then the value in the other view takes over for the join.

For view inclusion, we consider by default:

$$\begin{aligned} V_1(\ell) &\subseteq V_2(\ell) \text{ if } \ell \notin \text{dom}(V_1) \\ V_1(\ell) &\not\subseteq V_2(\ell) \text{ if } \ell \in \text{dom}(V_1) \wedge \ell \notin \text{dom}(V_2) \end{aligned}$$

**Definition 3.8** (Thread-Views).

$$\mathcal{V} \in \text{ThreadView} ::= \{\text{rel} : \text{Loc} \xrightarrow{\text{fin}} \text{View}, \text{frel} : \text{View}, \text{cur} : \text{View}, \text{acq} : \text{View}\}$$

<sup>12</sup>This is inspired by thread-views of the promising semantics ([Kan+17]).

A thread-view<sup>12</sup> is used to track the observations of a thread, and has four components. For a thread  $\pi$  to have the thread-view  $\mathcal{V}$  at its current program counter  $\text{PC}_\pi$ ,

- $\mathcal{V}.\text{cur}$  is the actual, *current* view of  $\pi$ , which includes all reads and writes that happen before the current counter  $\text{PC}_\pi$ .
- $\mathcal{V}.\text{acq}$  is the *acquire* view of  $\pi$ . It tracks the observations acquired by  $\pi$ 's earlier relaxed reads, and will be restored into  $\pi$ 's current view *after* the next acquire or SC fence. In other words, it tracks all reads and writes that happen before  $\pi$ 's next acquire or SC fence.
- $\mathcal{V}.\text{frel}$  is the *release-fence* view of  $\pi$ . It tracks all reads and writes that happen before  $\pi$ 's most recent release or SC fence, and it can be released by  $\pi$ 's later relaxed writes.
- $\mathcal{V}.\text{rel}$  is a finite, partial function that tracks *per-location release* views for  $\pi$ . For a location  $\ell$ ,  $\mathcal{V}.\text{rel}(\ell)$  is the release view of  $\pi$ 's most recent release write to  $\ell$ , and can be released by  $\pi$ 's later relaxed writes to the same location  $\ell$ . This view is needed to model the release sequence (*rs*, Definition 2.12) for  $\ell$ .

**Property 3.9** (Thread-Views Wellformedness). The following properties must hold for a thread-view  $\mathcal{V}$ :

- $\text{dom}(\mathcal{V}.\text{rel}) \subseteq \text{dom}(\mathcal{V}.\text{cur})$  (TVIEW-DOM)
- $\forall \ell. \mathcal{V}.\text{rel}(\ell) \subseteq \mathcal{V}.\text{cur}$  (TVIEW-REL)
- $\mathcal{V}.\text{frel} \subseteq \mathcal{V}.\text{cur}$  (TVIEW-FREL)
- $\mathcal{V}.\text{cur} \subseteq \mathcal{V}.\text{acq}$  (TVIEW-CUR)

**Definition 3.10** (Thread-Views' Join Semi-Lattice). The bottom element of thread-views, also denoted by  $\emptyset$ , is the tuple of an empty release map and empty (bottom) views. The inclusion relation and the join operation for thread-views are defined as follows.

$$\begin{aligned} \mathcal{V}_1 &\subseteq \mathcal{V}_2 ::= (\forall \ell. \mathcal{V}_1.\text{rel}(\ell) \subseteq \mathcal{V}_2.\text{rel}(\ell)) \wedge \mathcal{V}_1.\text{frel} \subseteq \mathcal{V}_2.\text{frel} \\ &\quad \wedge \mathcal{V}_1.\text{cur} \subseteq \mathcal{V}_2.\text{cur} \wedge \mathcal{V}_1.\text{acq} \subseteq \mathcal{V}_2.\text{acq} \\ \mathcal{V}_1 \sqcup \mathcal{V}_2 &::= \{ \text{rel} := \lambda \ell. \mathcal{V}_1.\text{rel}(\ell) \sqcup \mathcal{V}_2.\text{rel}(\ell); \\ &\quad \text{frel} := \mathcal{V}_1.\text{frel} \sqcup \mathcal{V}_2.\text{frel}; \\ &\quad \text{cur} := \mathcal{V}_1.\text{cur} \sqcup \mathcal{V}_2.\text{cur}; \\ &\quad \text{acq} := \mathcal{V}_1.\text{acq} \sqcup \mathcal{V}_2.\text{acq} \} \end{aligned}$$

**Definition 3.11** (Global Memory).

$$\begin{aligned}\mathcal{M} \in \text{MsgPool} &::= \text{Loc} \xrightarrow{\text{fin}} \text{Time} \xrightarrow{\text{fin}} \left\{ \text{val} : \text{MemVal}, \text{view} : \text{View}^? \right\} \\ m \in \text{ExtMsg} &::= \left\{ \text{ts} : \text{Time}, \text{val} : \text{MemVal}, \text{view} : \text{View}^? \right\}\end{aligned}$$

The global memory, or the *message pool*  $\mathcal{M}$  contains all write messages to all locations. It is a finite, partial map from locations to timestamps to a pair of a written value and an *optional* view ( $\text{View}^?$ ).

For a location,  $\mathcal{M}(\ell)$  contains all write messages to  $\ell$ , ordered by timestamps. The timestamp order of  $\mathcal{M}(\ell)$  encodes the per-location modification order  $\text{mo}_\ell$  (Definition 2.8) for  $\ell$ .

For some timestamp  $t$ , the pair  $\mathcal{M}(\ell)(t)$  carries the information about a write to  $\ell$  identified by the timestamp  $t$ . If the write is an non-atomic write, then  $\mathcal{M}(\ell)(t).\text{view} = \text{None}$ . Otherwise, if the write is an atomic write, then  $\mathcal{M}(\ell)(t).\text{view} = \text{Some}(V)$  for some view  $V$  that is called the (released) view of the write. As a shorthand notation for the option type, we write  $\perp$  for None, and simply write  $V$  for  $\text{Some}(V)$ .<sup>13</sup>

The message type  $\text{ExtMsg}$  combines the timestamp with the value and the optional view into a single message. As such, the message pool can be seen as a map from locations to messages:  $\text{MsgPool} \approx \text{Loc} \xrightarrow{\text{fin}} \wp(\text{ExtMsg})$ .

**Property 3.12** (View Closedness).

$$\boxed{V \in \mathcal{M}} \quad \boxed{V \in \mathcal{M}}$$

A view  $V$  is said to be closed in  $\mathcal{M}$  if  $V$  only contains write messages in  $\mathcal{M}$ , i.e.,  $\forall \ell, V(\ell).w \in \mathcal{M}(\ell)$ . The definition is lifted point-wise for thread-views.

**Property 3.13** (Global Memory Wellformedness). A global memory  $\mathcal{M}$  is wellformed if the following hold.

$$\begin{aligned}\forall \ell, m. m \in \mathcal{M}(\ell) \wedge m.\text{view} \neq \perp &\Rightarrow m.\text{view}(\ell') \in \mathcal{M} \quad (\text{WF-MEM-CLOSED}) \\ \forall \ell, m. m \in \mathcal{M}(\ell) \wedge m.\text{view} \neq \perp &\Rightarrow m.\text{view}(\ell).w = m.\text{ts} \quad (\text{WF-MSG-VIEW}) \\ \forall \ell, t. \mathcal{M}(\ell)(t).\text{val} = \dagger &\Rightarrow t = \min(\text{dom}(\mathcal{M}(\ell))) \quad (\text{WF-MEM-ALLOC}) \\ \forall \ell, t. \mathcal{M}(\ell)(t).\text{val} = \clubsuit &\Rightarrow t = \max(\text{dom}(\mathcal{M}(\ell))) \quad (\text{WF-MEM-DEALLOC})\end{aligned}$$

**WF-MEM-CLOSED** requires that  $\mathcal{M}$  is closed in itself, i.e., any view of any write messages in  $\mathcal{M}$  only refers to messages also in  $\mathcal{M}$ . **WF-MSG-VIEW** requires that the view of a write message contains exactly the timestamp of that message. **WF-MEM-ALLOC** (resp. **WF-MEM-DEALLOC**) requires that if a write is an allocation (resp. deallocation) then it must be the minimum (resp. maximum) write event for that location.

**Definition 3.14** (Global Machine State).

$$\begin{aligned}\mathcal{N} \in \text{RaceView} &::= \text{View} \\ \varsigma \in \text{GlobalState} &::= \text{MsgPool} \times \text{RaceView}\end{aligned}$$

The global machine state  $\varsigma$  is a pair  $(\mathcal{M}, \mathcal{N})$  of the global memory  $\mathcal{M}$  and a simple view  $\mathcal{N}$  that is the state of the race detector (§3.4).

**Property 3.15** (Global Machine State Wellformedness). A global state  $(\mathcal{M}, \mathcal{N})$  is wellformed if

<sup>13</sup>Note that instead of using the option type, we can also require that the view  $V$  of a non-atomic write to be empty ( $\emptyset$ ).

- $\mathcal{M}$  is wellformed (Property 3.13); and
- $\mathcal{N}$  is closed in  $\mathcal{M}$ ; and
- $\mathcal{N}$  observes the deallocations in  $\mathcal{M}$ , i.e.,  $\forall \ell, t. \mathcal{M}(\ell)(t).val = \Phi \Rightarrow t \leq \mathcal{N}(\ell).w$ .

### 3.3 View-based RMC Semantics

We define the view-based semantics of ORC11, which describes the interactions between thread-views  $\mathcal{V}$ 's and the global memory  $\mathcal{M}$ . First we need a few auxiliary definitions.

**Definition 3.16** (Memory Value Injection).

$$\boxed{\omega \equiv v}$$

The injection of memory values (*MemVal*) into values (*Val*) is defined by the following rules.

$$\begin{array}{ll} \text{MVAL-VAL} & \text{MVAL-AVAL} \\ v \equiv v & \dagger \equiv \clubsuit \end{array}$$

That is, if the memory value is a value  $v$ , then it is returned as is. If the memory value is the allocated value  $\dagger$ , then poison  $\clubsuit$  is returned. There is no injection of the deallocated value  $\Phi$  into values.

**Definition 3.17** (Unallocated Locations).

$$\boxed{\ell \in \text{unalloc}(\mathcal{M})}$$

A location  $\ell$  is called *unallocated* in  $\mathcal{M}$  if it has not been allocated or it has been deallocated in  $\mathcal{M}$ .

$$\frac{\ell \notin \text{dom}(\mathcal{M})}{\ell \in \text{unalloc}(\mathcal{M})} \qquad \frac{\exists t. \mathcal{M}(\ell)(t) = (\Phi, \_)}{\ell \in \text{unalloc}(\mathcal{M})}$$

**Notation 3.18** (Function Computations). We use the notation  $(x) ? y : z$  to denote the expression that if  $x$  is true, then  $y$  is returned, otherwise  $z$  is returned.

For a finite, partial function  $f$ , the notation  $f[x \leftarrow y]$  denotes the same function  $f$  but with the key  $x$  updated to the value  $y$ .

For a record  $r$ , the notation  $\{r[x := y]\}$  (including braces) denotes the same record  $r$  but with the key  $x$  updated to the value  $y$ .

**Remark 3.19** (Conditions on Allocations and Deallocations). C11 only specifies that the lifetime of an object is from its allocation to deallocation, but does not specify a synchronization condition or possible races between allocation/deallocation and normal accesses. Here, we employ the following conditions that are widely thought to be reasonable.

- The allocation of a block must happen-before all accesses to it. (ALLOC-SAFE)
- The deallocation of a block must happen-after all accesses to it. (DEALLOC-SAFE)

We consider violations of these conditions data races and, thus, undefined behavior. Correspondingly, we will treat the semantics of allocations and deallocations as that of non-atomic writes.

We now define two functions (written in form of relations) to compute the resulting thread-view of a thread after a read or a write, in [Figure 3.2](#).

**Definition 3.20** (Post-Read Thread-Views).

$$\mathcal{V} \xrightarrow{R:o,\ell,t,V_r,r} \mathcal{V}'$$

**OM-POST-READ-TVIEW** ([Figure 3.2](#)) computes the thread-view *after* a read  $\mathcal{V}' = (V_{\text{rel}}, V_{\text{frel}}, V'_{\text{cur}}, V'_{\text{acq}})$  from the thread-view *before* the read  $\mathcal{V} = (V_{\text{rel}}, V_{\text{frel}}, V_{\text{cur}}, V_{\text{acq}})$ , using the read's access mode  $o$  and location  $\ell$ , the timestamp  $t$  and the view  $V_r \in \text{View}^+$  of the write message that the read reads from, and a fresh action id  $r$  ( $\{r\} \in \text{ActIds}$ ) that identifies the read.<sup>14</sup> The computation is as follows.

- $V_{\text{cur}}(\ell).w \leq t$ : the read only reads a write event (identified by  $t$ ) that is not **mo**-earlier than the current view. Intuitively, this restriction helps establish axioms like **C11-CoWR** and **C11-CoRR**.
- $V_r(\ell) \leq t$ : the operational semantics maintains an invariant that the timestamp  $t$  of the write to  $\ell$  is the maximum timestamp in the write's view  $V_r$ .
- The view  $V$  tracks the identifying information of the read and the write that the read reads from. In particular,  $V.w$  is the timestamp  $t$  of the write. The read id  $r$  is added to the non-atomic read component  $V.nr$  if this is a non-atomic read ( $o = \text{na}$ ), otherwise  $r$  is added to the atomic read component  $V.ar$ .
- A read only changes the current and acquire (simple) views of  $\mathcal{V}$ .
- The view  $V$  is joined into both the new current and acquire views  $V'_{\text{cur}}$  and  $V'_{\text{acq}}$ , so that both views observe at least the read and the write.
- If this is an atomic read ( $\text{rlx} \sqsubseteq o$ ), then the view  $V_r$  of the write is also joined into the acquire view  $V'_{\text{acq}}$ . This encodes the delayed synchronization of relaxed reads, where the view  $V_r$  sent over the write is temporarily stored in the acquire view  $V'_{\text{acq}}$  and will only later be restored into the current view with an acquire fence (recall [Example 3.3](#)).
- If this is at least an acquire read ( $\text{acq} \sqsubseteq o$ ), then the view  $V_r$  of the write is immediately joined into the current view  $V'_{\text{cur}}$  (recall [Example 3.2](#)).
- The computation maintains wellformedness of thread-views ([Property 3.9](#)).

<sup>14</sup>Note that if  $o = \text{na}$ , then  $V_r = \perp$  (None).

**Definition 3.21** (Post-Write Thread-Views).

$$\mathcal{V} \xrightarrow{W:o,\ell,t,V_r^?,V_w^?} \mathcal{V}'$$

**OM-POST-WRITE-TVIEW** ([Figure 3.2](#)) computes the thread-view *after* a write  $\mathcal{V}' = (V'_{\text{rel}}, V_{\text{frel}}, V'_{\text{cur}}, V'_{\text{acq}})$  from the thread-view *before* the write  $\mathcal{V} = (V_{\text{rel}}, V_{\text{frel}}, V_{\text{cur}}, V_{\text{acq}})$ , using the write's access mode  $o$  and location  $\ell$ , and a fresh timestamp  $t$  to identify the write, and the view  $V_r \in \text{View}^+$  that the write reads from in case it is an Update (U). Additional, it computes the view  $V_w$  of the write itself. The computation is as follows.

## OM-POST-READ-TVIEW

$$\begin{array}{c}
V_{\text{cur}}(\ell).w \leq t \quad V_r(\ell) \leq t \\
V = [\ell \leftarrow \{\mathbf{w} := t; \mathbf{aw} := \emptyset; \mathbf{nr} := (o = \mathbf{na}) ? \{r\} : \emptyset; \mathbf{ar} := (o \sqsubseteq \mathbf{rlx}) ? \{r\} : \emptyset\}] \\
V'_{\text{cur}} = (\mathbf{acq} \sqsubseteq o) ? V_{\text{cur}} \sqcup V \sqcup V_r : V_{\text{cur}} \sqcup V \\
V'_{\text{acq}} = (\mathbf{rlx} \sqsubseteq o) ? V_{\text{acq}} \sqcup V \sqcup V_r : V_{\text{acq}} \sqcup V \\
\hline
(V_{\text{rel}}, V_{\text{frel}}, V_{\text{cur}}, V_{\text{acq}}) \xrightarrow{\mathbf{R}:o,\ell,t,V_r,r} (V_{\text{rel}}, V_{\text{frel}}, V'_{\text{cur}}, V'_{\text{acq}})
\end{array}$$

## OM-POST-WRITE-TVIEW

$$\begin{array}{c}
V_{\text{cur}}(\ell).w < t \\
V = [\ell \leftarrow \{\mathbf{w} := t; \mathbf{aw} := (\mathbf{rlx} \sqsubseteq o) ? \{t\} : \emptyset; \mathbf{nr} := \emptyset; \mathbf{ar} := \emptyset\}] \\
V'_{\text{cur}} = V_{\text{cur}} \sqcup V \quad V'_{\text{acq}} = V_{\text{acq}} \sqcup V \\
V' = V_{\text{rel}}(\ell) \sqcup (\mathbf{rel} \sqsubseteq o) ? V'_{\text{cur}} : V \quad V'_{\text{rel}} = V_{\text{rel}}[\ell \leftarrow V'] \\
V_w = (\mathbf{rlx} \sqsubseteq o) ? V' \sqcup V_{\text{frel}} \sqcup V_r : \perp \\
\hline
(V_{\text{rel}}, V_{\text{frel}}, V_{\text{cur}}, V_{\text{acq}}) \xrightarrow{\mathbf{W}:o,\ell,t,V_r,V_w} (V'_{\text{rel}}, V_{\text{frel}}, V'_{\text{cur}}, V'_{\text{acq}})
\end{array}$$

FIGURE 3.2: Computations of post thread-views for read and write operations.

- $V_{\text{cur}}(\ell).w < t$ : the fresh timestamp  $t$  picked for the write must be **no**-later than the current view of  $\ell$ .
- The view  $V$  tracks the identifying information of the write. In particular,  $V.w$  is the timestamp  $t$  of the write.  $t$  is also added to the atomic write component  $V.\mathbf{aw}$  if this is an atomic write.
- The write updates the current and acquire (simple) views, and the component  $\ell$  of the per-location release view  $V_{\text{rel}}$  of  $\mathcal{V}$ .
- The view  $V$  is joined into both the new current and acquire views  $V'_{\text{cur}}$  and  $V'_{\text{acq}}$ , so that both views observe at least the write.
- The view  $V'$  is the new release write for the location  $\ell$ , and is updated into to  $V'_{\text{rel}}(\ell)$  ( $V'_{\text{rel}} = V_{\text{rel}}[\ell \leftarrow V']$ ). In particular, if this is at least a release write, then  $\ell$ 's new release view  $V'$  also includes the *new* current view  $V'_{\text{cur}}$ . (Otherwise, if the write is at most a relaxed write, then the release views remain unchanged.) This means that the write releases its current view immediately (recall also [Example 3.2](#)), and this release write starts a new release sequence ([Definition 2.12](#)) for  $\ell$ , so that po-later relaxed writes to the same  $\ell$  will indeed release  $V'_{\text{cur}}$ .
- The view  $V_w$  of the write itself is  $\perp$  if this is a non-atomic write. Otherwise, it includes at least (i) the new release view  $V'_{\text{rel}}(\ell)$  for  $\ell$ , and (ii) the view  $V_{\text{frel}}$  of the most recent same-thread release fence, and (iii) the view  $V_r$  of another write that this write reads from in case it is an Update. All of these views establish this write's effects as a part of a release sequence for  $\ell$  (see also [Example 2.13](#)).
- The computation maintains wellformedness of the thread-views ([Property 3.9](#)).

Finally, we can define the view-based semantics of ORC11.

$$\begin{array}{c}
\text{OM-READ} \\
\frac{\ell \notin \text{unalloc}(\mathcal{M}) \quad \mathcal{M}(\ell)(t) = (\omega, V_r) \quad \omega \equiv v \quad \mathcal{V} \xrightarrow{\text{R:o}, \ell, t, V_r, r} \mathcal{V}'}{\mathcal{M} \mid \mathcal{V} \xrightarrow{\text{R}^o(\ell, v), r, []} \mathcal{M} \mid \mathcal{V}'} \\
\\
\text{OM-WRITE} \\
\frac{\ell \notin \text{unalloc}(\mathcal{M}) \quad t \notin \mathcal{M}(\ell) \quad \mathcal{M}' = \mathcal{M}[\ell \leftarrow \mathcal{M}(\ell)[t \leftarrow (v, V_w)]] \quad \mathcal{V} \xrightarrow{\text{W:o}, \ell, t, \perp, V_w} \mathcal{V}'}{\mathcal{M} \mid \mathcal{V} \xrightarrow{\text{W}^o(\ell, v), \perp, [(t, v, V_w)]} \mathcal{M}' \mid \mathcal{V}'} \\
\\
\text{OM-UPDATE} \\
\frac{\ell \notin \text{unalloc}(\mathcal{M}) \quad \mathcal{M}(\ell)(t_r) = (v_r, V_r) \quad t_w = t_r + 1 \quad t_w \notin \mathcal{M}(\ell) \quad \mathcal{M}' = \mathcal{M}[\ell \leftarrow \mathcal{M}(\ell)[t_w \leftarrow (v_w, V_w)]] \quad \mathcal{V} \xrightarrow{\text{R:o}_r, \ell, t_r, V_r, r} \mathcal{V} \xrightarrow{\text{W:o}_w, \ell, t_w, V_r, V_w} \mathcal{V}'}{\mathcal{M} \mid \mathcal{V} \xrightarrow{\text{U}^{o_r, o_w}(\ell, v_r, v_w), r, [(t_w, v_w, V_w)]} \mathcal{M}' \mid \mathcal{V}'} \\
\\
\text{OM-ACQ-FENCE} \\
\mathcal{M} \mid \mathcal{V} \xrightarrow{\text{F}^{\text{acq}}, \perp, []} \mathcal{M} \mid (\mathcal{V}.\text{rel}, \mathcal{V}.\text{frel}, \mathcal{V}.\text{acq}, \mathcal{V}.\text{acq}) \\
\\
\text{OM-REL-FENCE} \\
\mathcal{M} \mid \mathcal{V} \xrightarrow{\text{F}^{\text{rel}}, \perp, []} \mathcal{M} \mid ([\ell \leftarrow \mathcal{V}.\text{cur} \mid \ell \in \text{dom}(\mathcal{V}.\text{rel})], \mathcal{V}.\text{cur}, \mathcal{V}.\text{cur}, \mathcal{V}.\text{acq}) \\
\\
\text{OM-ALLOC} \\
\frac{\ell = (i, n') \quad \{i\} \times \mathbb{N} \# \text{dom}(\mathcal{M}) \quad \mathcal{M}' = \mathcal{M}[\ell + m \leftarrow [t_m \leftarrow (\dagger, \perp)] \mid m \in [0, n]] \quad \mathcal{V} \xrightarrow{\text{W:na}, \ell+0, t_0, \perp, \perp} \dots \xrightarrow{\text{W:na}, \ell+m, t_m, \perp, \perp} \dots \xrightarrow{\text{W:na}, \ell+(n-1), t_{n-1}, \perp, \perp} \mathcal{V}' \quad ms = [(t_m, \dagger, \perp) \mid m \in [0, n]]}{\mathcal{M} \mid \mathcal{V} \xrightarrow{\text{A}(\ell, n), \perp, ms} \mathcal{M}' \mid \mathcal{V}'} \\
\\
\text{OM-FREE} \\
\frac{\ell = (i, n') \quad \text{dom}(\mathcal{M}) \cap \{i\} \times \mathbb{N} = \{i\} \times ([\geq n', < n' + n]) \quad \forall m \in [0, n]. \ell + m \notin \text{unalloc}(\mathcal{M}) \quad \forall m \in [0, n], t \in \text{dom}(\mathcal{M}(\ell + m)). t < t_m \quad \mathcal{M}' = \mathcal{M}[\ell + m \leftarrow [t_m \leftarrow (\Phi, \perp)] \mid m \in [0, n]] \quad \mathcal{V} \xrightarrow{\text{W:na}, \ell+0, t_0, \perp, \perp} \dots \xrightarrow{\text{W:na}, \ell+m, t_m, \perp, \perp} \dots \xrightarrow{\text{W:na}, \ell+(n-1), t_{n-1}, \perp, \perp} \mathcal{V}' \quad ms = [(t_m, \Phi, \perp) \mid m \in [0, n]]}{\mathcal{M} \mid \mathcal{V} \xrightarrow{\text{D}(\ell, n), \perp, ms} \mathcal{M}' \mid \mathcal{V}'}
\end{array}$$

FIGURE 3.3: View-based machine semantics.

**Definition 3.22** (ORC11 View-based Reductions).  $\boxed{\mathcal{M} \mid \mathcal{V} \rightarrow \mathcal{M}' \mid \mathcal{V}'}$

The relation  $\mathcal{M} \mid \mathcal{V} \xrightarrow{\varepsilon, r^?, ms} \mathcal{M}' \mid \mathcal{V}'$  relates a pair of global memory  $\mathcal{M}$  and a local thread-view  $\mathcal{V}$  before a machine step that generates a memory event  $\varepsilon$  to a corresponding pair  $\mathcal{M}'$  and  $\mathcal{V}'$  after the step.  $r$  is an optional action id associated with the event if it is a read, and  $ms$  is a list of write messages generated by the event if it is a write. The rules of the view-based reductions are given in [Figure 3.3](#).

- **OM-READ** says that a read  $\text{R}^o(\ell, v)$  does not change the global memory  $\mathcal{M}$ , and is only possible if  $\ell$  is alive in  $\mathcal{M}$ ,<sup>15</sup> whose memory value  $\omega$  is injected into the read value  $v$  (so that a read of an uninitialized location will return the poison value  $\clubsuit$ ).<sup>16</sup> The thread-view is updated from  $\mathcal{V}$  to  $\mathcal{V}'$  using the timestamp  $t$  and the view  $V_r$  of

<sup>15</sup>see [Definition 3.17](#).<sup>16</sup>see [Definition 3.16](#).

the write, and the fresh action id  $r$  for the read, following [Definition 3.20](#).

- **OM-WRITE** says that a write  $W^o(\ell, v)$  is only possible if  $\ell$  is alive in  $\mathcal{M}$ , and  $\mathcal{M}$  is updated to  $\mathcal{M}'$  with a new message  $(t, v, V_w)$  for  $\ell$ , where  $t$  is a fresh timestamp in  $\mathcal{M}$  for  $\ell$ , and  $V_w$  is computed following [Definition 3.21](#), which also defines how  $\mathcal{V}'$  is computed. Note that there is no other constraint on the timestamp  $t$ , e.g., it does not need to be the next largest timestamp for  $\ell$  in  $\mathcal{M}$ . This allows “holes” in the set of used timestamps, so that writes to  $\ell$  by other threads may come in later in ORC11 machine execution order, but actually ends up **mo**-earlier than the writes made by the thread in question. But do note also that [Definition 3.21](#) requires that the timestamp  $t$  is at least **mo**-later than the writes for  $\ell$  seen by the current thread-view, so as to guarantee that **mo** $_{\ell}$  agrees with the current thread’s po.
- **OM-UPDATE** combines the effects of both **OM-READ** and **OM-WRITE**, saying that an update  $U^{o_r, o_w}(\ell, v_r, v_w)$  reads from an existing write message  $(t_r, v_r, V_r)$  and updates the memory  $\mathcal{M}$  with a new write message  $(t_w, v_w, V_w)$ . The new write message’s timestamp  $t_w$  must be fresh for  $\ell$  in  $\mathcal{M}$ , and must be next to the read message’s timestamp  $t_r$ :  $t_w = t_r + 1$ , so as to exclude holes between the two messages in **mo** $_{\ell}$ , and thus to disallow other threads’ concurrent writes to come in between this update and the write that it reads from. This guarantees the *uniqueness* of a successful update event  $U$  who represents the effects of RMW instructions: if multiple RMW instructions are racing on reading the same value, then only one of them will successfully perform a write. Finally, the new thread-view  $\mathcal{V}'$  is computed from  $\mathcal{V}$  using  $r$ ,  $V_r$ , and  $V_w$ , following [Definition 3.20](#) and then [Definition 3.21](#).
- **OM-ACQ-FENCE** simply joins the thread-view’s acquire component  $\mathcal{V}.acq$  into the new current component  $\mathcal{V}'.cur$ , restoring views acquired through earlier relaxed reads and thus establishing synchronizations (recall [Example 3.3](#)).
- **OM-REL-FENCE** stores the thread-view’s current component  $\mathcal{V}.cur$  into the new release components  $\mathcal{V}'.rel$  (the per-location release views) and  $\mathcal{V}'.frel$  (the release-fence view).
- **OM-ALLOC** says that an allocation  $A(\ell, n)$  of a fresh block (whose base is  $\ell$ ) inserts  $n$  write messages  $ms$  into the global memory  $\mathcal{M}$ , each for a location in the block. The new write messages  $ms$  all have the allocated memory value  $\dagger$ . The new thread-view  $\mathcal{V}'$  is computed by applying [Definition 3.21](#) for  $n$  consecutive non-atomic writes.
- Finally, **OM-FREE** says that a deallocation  $D(\ell, n)$  requires that  $\ell$  is indeed the base location of a block whose size is  $n$ , and all locations in the block are alive ( $\forall m \in [0, n). \ell + m \notin \text{unalloc}(\mathcal{M})$ ). The



deallocation inserts  $n$  write messages  $ms$  into the global memory  $\mathcal{M}$ , all with the deallocated memory value  $\clubsuit$ , and the maximal timestamps  $(\forall m \in [0, n], t \in \text{dom}(\mathcal{M}(\ell + m)). t < t_m)$ .

**Property 3.23** (Wellformedness of View-based Reductions). The pair  $\mathcal{M} \mid \mathcal{V}$  is wellformed if  $\mathcal{V}$  is wellformed (Property 3.9) and  $\mathcal{M}$  is wellformed (Property 3.13) and  $\mathcal{V}$  is closed in  $\mathcal{M}$  (Property 3.12).

**Lemma 3.24** (Invariant of ORC11 View-based Reductions). The ORC11 view-based reductions (Definition 3.22) maintain wellformedness (Property 3.23), and maintain that the thread-views only grow ( $\mathcal{V} \sqsubseteq \mathcal{V}'$ ).

### 3.4 The Data-Race Detector

The goal of the race detector, as the name suggest, is to raise undefined behavior (UB) if the program is racy in C11/RC11 (Definition 2.16 and 2.18). That is, if a program may have a RC11-consistent execution graph that is racy, then the program must also have a ORC11 execution where the data-race detector (defined in this section) raises UB.

In this work, we model UB as *stuckness*: we say that the execution gets stuck if there is no further reduction possible when the reducing expression has not reaches a value. (If the reducing expression is already a value, then the execution has safely terminated.)<sup>17</sup> We will not see the expression reductions until §4.2, so in the following, we simply consider stuckness as “there is no further reduction possible for the current machine state”.

The aim of the race detector is to make sure locally that racy accesses where at least one is non-atomic must get stuck. For this, the race detector relies on the global machine state, which includes the global memory  $\mathcal{M}$  and the race detector’s state  $\mathcal{N} \in \text{View}$ , in combination of the executing thread  $\pi$ ’s thread-view  $\mathcal{V}$ . In practice, only the current component  $\mathcal{V}.cur \in \text{View}$  will be used, because that view encodes what have happened before the thread  $\pi$ ’s program counter  $PC_\pi$ , and recall that races are due to the lack of **hb** edges between conflicting accesses.

Recall from Definition 3.6 that both  $\mathcal{N}$  and  $\mathcal{V}.cur$  track, for each location  $\ell$ , the most recent write timestamp and sets of action ids for atomic writes, non-atomic reads, and atomic reads. The differences are that (1)  $\mathcal{N}$  tracks all actions that have been performed globally by *all* threads, while  $\mathcal{V}.cur$  only tracks locally what  $\pi$  has observed, and (2)  $\mathcal{N}.w(\ell)$  only tracks the globally most recent *non-atomic* write for  $\ell$ , not the most recent write for  $\ell$ .

The race detector checks for *data-race freedom* (DRF) for each memory access on an  $\ell$  that  $\pi$  is going to perform. If it is not data-race free, then the execution gets stuck. Otherwise, the race detector state  $\mathcal{N}$  is updated correspondingly to track the newly performed access. The race detector is therefore defined using two definitions: the DRF pre-condition (Definition 3.25) which defines the pre-condition of a data-race free access, and the DRF post-condition (Definition 3.26) which computes the post state  $\mathcal{N}'$  for the race detector.

<sup>17</sup>There may be several ways for an execution to run into UBs, *i.e.*, to get stuck (e.g., performing computations on poison  $\clubsuit$ , see §4.2), so it may be beneficial to distinguish the different reasons for the different UB types, rather than collapsing all of them into a single stuck state. This can be done by introducing error machine states. However, in this work, we do not need such details, and therefore decide to simply use stuckness.

$$\begin{array}{c}
\text{DRF-READ-NA} \\
\frac{\mathcal{N}(\ell).w \leq \mathcal{V}.\text{cur}(\ell).w \quad \forall t \in \text{dom}(\mathcal{M}(\ell)). t \leq \mathcal{V}.\text{cur}(\ell).w \quad \mathcal{N}(\ell).\text{aw} \sqsubseteq \mathcal{V}.\text{cur}(\ell).\text{aw}}{\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\mathbf{R}^{\text{na}}(\ell, v))} \\
\\
\text{DRF-WRITE-NA} \\
\frac{\mathcal{N}(\ell) \sqsubseteq \mathcal{V}.\text{cur}(\ell) \quad \forall t \in \text{dom}(\mathcal{M}(\ell)). t \leq \mathcal{V}.\text{cur}(\ell).w}{\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\mathbf{W}^{\text{na}}(\ell, v))} \\
\\
\text{DRF-READ-AT} \\
\frac{\mathbf{rlx} \sqsubseteq o \quad \mathcal{N}(\ell).w \leq \mathcal{V}.\text{cur}(\ell).w}{\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\mathbf{R}^o(\ell, v))} \\
\\
\text{DRF-WRITE-AT} \\
\frac{\mathbf{rlx} \sqsubseteq o \quad \mathcal{N}(\ell).w \leq \mathcal{V}.\text{cur}(\ell).w \quad \mathcal{N}(\ell).\text{nr} \sqsubseteq \mathcal{V}.\text{cur}(\ell).\text{nr}}{\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\mathbf{W}^o(\ell, v))} \\
\\
\text{DRF-UPDATE} \\
\frac{\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\mathbf{R}^{or}(\ell, v_r)) \quad \mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\mathbf{W}^{ow}(\ell, v_w))}{\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\mathbf{U}^{or, ow}(\ell, v_r, v_w))} \\
\\
\text{DRF-ALLOC} \\
\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\mathbf{A}(\ell, n)) \\
\\
\text{DRF-DEALLOC} \\
\frac{\forall i \in [ < n ], t' \in \text{dom}(\mathcal{M}(\ell + i)). t' \leq \mathcal{V}.\text{cur}(\ell).w \quad \forall i \in [ < n ]. \mathcal{N}(\ell + i) \sqsubseteq \mathcal{V}.\text{cur}(\ell + i)}{\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\mathbf{D}(\ell, n))} \\
\\
\text{DRF-FENCE} \\
\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\mathbf{F}^o)
\end{array}$$

FIGURE 3.4: Data-race free (DRF) pre-conditions.

**Definition 3.25** (DRF Pre-conditions).

$$\boxed{\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\varepsilon)}$$

$\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\varepsilon)$  says that the memory access  $\varepsilon$  is data-race free when executed with the global state  $(\mathcal{M}, \mathcal{N})$  by a thread whose thread-view is  $\mathcal{V}$ . The rules are given in [Figure 3.4](#).

- **DRF-READ-NA** says that a non-atomic read from  $\ell$  is data-race free if the thread has observed *all* writes to  $\ell$ , atomic and non-atomic, tracked globally by  $\mathcal{M}$  and  $\mathcal{N}$ . Note that the conditions concerning  $\mathcal{V}.\text{cur}(\ell).w$  by themselves are not sufficient: they only maintain that  $\mathcal{V}$  has observed the  $\text{mo}_\ell$ -maximum non-atomic write, which is only sufficient to guarantee observations of all non-atomic writes which must happen sequentially. The condition  $\mathcal{N}(\ell).\text{aw} \sqsubseteq \mathcal{V}.\text{cur}(\ell).\text{aw}$  guarantees the observations of all atomic writes: the atomic writes can be safely concurrent with one another, so a set of timestamps are needed instead of just a simple timestamp. Note that a non-atomic read can be safely executed concurrently with other reads, atomic or non-atomic.
- **DRF-WRITE-NA** says that a non-atomic write to  $\ell$  is data-race free if the thread has observed *all* memory accesses to  $\ell$ .

- **DRF-READ-AT** says that an atomic read from  $\ell$  is data-race free if the thread has observed the latest non-atomic write to  $\ell$ . Note that an atomic read can be safely executed concurrently with other reads, atomic or non-atomic, and atomic writes.
- **DRF-WRITE-AT** says that an atomic write to  $\ell$  is data-race free if the thread has observed *all non-atomic accesses*, reads or writes.
- **DRF-UPDATE** is simply a combination of **DRF-READ-AT** and **DRF-WRITE-AT**. Note that an Update (U) does not support non-atomic (**na**) accesses.
- **DRF-ALLOC** says that the allocation of a fresh block is always data-race free. Note that we do not model out-of-memory errors.
- **DRF-DEALLOC** is simply an iteration of **DRF-WRITE-NA** for the whole block.
- **DRF-FENCE** says that fences are never racy.

**Definition 3.26** (DRF Post-conditions). <sup>18</sup>

$$\mathcal{N} \xrightarrow[\text{racing-ghost}]{\varepsilon, r^?, ms} \mathcal{N}'$$

<sup>18</sup>The racing-ghost notation is due to Jan-Oliver Kaiser.

The relation  $\mathcal{N} \xrightarrow[\text{racing-ghost}]{\varepsilon, r^?, ms} \mathcal{N}'$  defines how a race-free event  $\varepsilon$  for  $\ell$  updates the global race detector state from  $\mathcal{N}$  to  $\mathcal{N}'$ , using the optional action id  $r$  if  $\varepsilon$  is a read, and the list of write messages  $ms$  if  $\varepsilon$  is a write. The rules are given in [Figure 3.5](#). Note that we use the record update notation defined in [Notation 3.18](#).

- **DRF-POST-READ-NA** requires that the action id  $r$  is picked fresh globally to identify this read, and the race detector's component for tracking  $\ell$ 's non-atomic reads ( $\mathcal{N}'(\ell).nr$ ) is extended with  $r$ .
- **DRF-POST-WRITE-NA** says that a non-atomic write with the message  $m$  simply extends the race detector's component for tracking  $\ell$ 's non-atomic writes ( $\mathcal{N}'(\ell).w$ ) with the write timestamp  $m.ts$ .
- **DRF-POST-READ-AT** says that the effect of an atomic read on  $\mathcal{N}$  is similar to that of a non-atomic one (**DRF-POST-READ-NA**), but instead changes the component for tracking  $\ell$ 's atomic reads.
- **DRF-POST-WRITE-AT** says that the effect of an atomic write on  $\mathcal{N}$  is similar to that of a non-atomic one (**DRF-POST-WRITE-NA**), but instead changes the component for tracking  $\ell$ 's atomic writes.
- **DRF-POST-UPDATE** combines the effects of **DRF-POST-READ-AT** and **DRF-POST-WRITE-AT**.
- **DRF-POST-ALLOC** says that the race detector state is extended with simple observations on the non-atomic write timestamps  $m_i.ts$  for the whole newly allocated block.
- **DRF-POST-DEALLOC** is an iteration of **DRF-POST-WRITE-NA** for the whole block that is deallocated.

$$\begin{array}{c}
\text{DRF-POST-READ-NA} \\
\frac{r \notin \mathcal{N}(\ell).\text{nr} \quad \mathcal{N}' = \mathcal{N}[\ell \leftarrow \{\mathcal{N}(\ell) [\text{nr} := \mathcal{N}(\ell).\text{nr} \cup \{r\}]]}{\mathcal{N} \xrightarrow{\text{R}^{\text{na}}(\ell, v), r, []} \mathcal{N}'} \\
\\
\text{DRF-POST-WRITE-NA} \\
\frac{\mathcal{N}' = \mathcal{N}[\ell \leftarrow \{\mathcal{N}(\ell) [\text{w} := m.\text{ts}]]}{\mathcal{N} \xrightarrow{\text{W}^{\text{na}}(\ell, v), \perp, [m]} \mathcal{N}'} \\
\\
\text{DRF-POST-READ-AT} \\
\frac{\text{rlx} \sqsubseteq o \quad r \notin \mathcal{N}(\ell).\text{ar} \quad \mathcal{N}' = \mathcal{N}[\ell \leftarrow \{\mathcal{N}(\ell) [\text{ar} := \mathcal{N}(\ell).\text{ar} \cup \{r\}]]}{\mathcal{N} \xrightarrow{\text{R}^o(\ell, v), r, []} \mathcal{N}'} \\
\\
\text{DRF-POST-WRITE-AT} \\
\frac{\text{rlx} \sqsubseteq o \quad \mathcal{N}' = \mathcal{N}[\ell \leftarrow \{\mathcal{N}(\ell) [\text{aw} := \mathcal{N}(\ell).\text{aw} \cup \{m.\text{ts}\}]]}{\mathcal{N} \xrightarrow{\text{W}^o(\ell, v), \perp, [m]} \mathcal{N}'} \\
\\
\text{DRF-POST-UPDATE} \\
\frac{r \notin \mathcal{N}(\ell).\text{ar} \quad \mathcal{N}' = \mathcal{N}[\ell \leftarrow \{\mathcal{N}(\ell) [\text{ar} := \mathcal{N}(\ell).\text{ar} \cup \{r\}; \text{aw} := \mathcal{N}(\ell).\text{aw} \cup \{m.\text{ts}\}]]}{\mathcal{N} \xrightarrow{\text{U}^{o_r, o_w}(\ell, v_r, v_w), r, [m]} \mathcal{N}'} \\
\\
\text{DRF-POST-ALLOC} \\
\frac{\mathcal{N}' = \mathcal{N}[\ell + i \leftarrow \{\text{w} := m_i.\text{ts}, \text{aw} := \emptyset, \text{nr} := \emptyset, \text{ar} := \emptyset\} \mid i \in [ < n]]}{\mathcal{N} \xrightarrow{\text{A}(\ell, n), \perp, [m_0 \dots m_{n-1}]} \mathcal{N}'} \\
\\
\text{DRF-POST-DEALLOC} \\
\frac{\mathcal{N}' = \mathcal{N}[\ell + i \leftarrow \{\mathcal{N}(\ell + i) [\text{w} := m_i.\text{ts}]\} \mid i \in [ < n]]}{\mathcal{N} \xrightarrow{\text{D}(\ell, n), \perp, [m_0 \dots m_{n-1}]} \mathcal{N}'} \\
\\
\text{DRF-POST-FENCE} \\
\mathcal{N} \xrightarrow{\text{F}^o, \perp, []} \mathcal{N}
\end{array}$$

FIGURE 3.5: Data-race free (DRF) post-conditions.

- **DRF-POST-FENCE** says that fences do not affect the race detector state.

**Lemma 3.27.** *The DRF post-conditions (Definition 3.26) only grow the data-race view, i.e.,  $\mathcal{N} \sqsubseteq \mathcal{N}'$ . When combined with the view-based reductions (Definition 3.22), the DRF post-conditions also maintain wellformedness of the global machine state (Property 3.15).*

### 3.5 Comparison with iGPS Race Detector

ORC11 is the first operational semantics that incorporates a race detector for non-atomic accesses into a language with release-acquire accesses, relaxed accesses, and fences. ORC11's race detector extends the race detector Kaiser et al.<sup>19</sup> developed for iGPS, in order to address the extra effects of relaxed accesses. To explain the necessity of this extension, we

<sup>19</sup>Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

first discuss why the approach of Kaiser et al. does not scale to relaxed accesses.

The iGPS race detector, introduced by Kaiser et al. for the release-acquire/non-atomic (RA+NA) fragment of C11, is somewhat unusual in that it does not in fact detect all races in *every* execution. Instead, although iGPS forbids write-before-read races—that is, races where a write is interleaved before a racing read—it *allows* read-before-write races—where a read is interleaved before a racing write.

**Example 3.28** (iGPS Race Detector Asymmetry). To illustrate this asymmetry, consider the following example code:

$$\begin{array}{l} \ell_x := 0; \\ * \ell_x \parallel \ell_x := 37 \end{array}$$

In this program there are two possible interleavings, both of which are considered racy by C11. The iGPS race detector detects a race in the interleaving where the read from  $\ell_x$  is executed after the write to  $\ell_x$ , but it does not detect a race in the interleaving where  $\ell_x$  is read first.

The upside of iGPS’s approach is that reads do not need to be tracked by the race detector, which reduces the amount of state in the operational semantics. The downside, of course, is that some races are not detected—a seemingly rather severe problem for a race detector! The reason this is not a problem in iGPS is that Hoare triples imply absence of races for *all* executions of a program. In order to be able to claim that the iGPS *logic* ensures absence of data races according to C11, it thus suffices for the race detector in the operational semantics to detect a race on *some* execution of every program that is racy according to C11. And indeed it does: for programs with only release-acquire and non-atomic accesses (the domain of iGPS), for any execution with a read-before-write race, there is always a differently interleaved execution with a write-before-read race, which iGPS’s race detector will detect.

In the presence of relaxed accesses, however, the iGPS race detector is no longer sufficient, because the property mentioned above is no longer true. That is, it is possible to construct programs that have executions in which the read-before-write races happen, but there is no interleaving where the write will be executed before the read.

**Example 3.29** (No Write-before-Read Races). Consider the following program:

$$\begin{array}{l} \ell_x := 0; \ell_y := 0; \\ * \ell_x; \ell_y := \text{r1x } 1 \parallel \text{while } (*\text{r1x } \ell_y == 0); \ell_x := 37 \end{array}$$

Here, the non-atomic read in the left thread is guaranteed to be executed before the non-atomic write to  $\ell_x$  in the right thread, and there is no interleaving where the reverse can happen. The iGPS race detector would *not* declare this program racy, but the two accesses to  $\ell_x$  are not related by happens-before and are thus considered a race by C11.

To account for such programs, ORC11’s race detector extends iGPS’s, which already tracked non-atomic writes, to track *all* memory access events, including atomic writes, and atomic and non-atomic reads in the local thread-views (see [Definition 3.6](#)). These events then must be sent across threads to perform synchronization and to ensure data-race freedom. In [Example 3.29](#) above, the non-atomic read of  $\ell_x$  by the left thread, when executed, will add a fresh read event  $r_{na}$  into  $\mathcal{N}$ ’s global set of non-atomic reads for  $\ell_x$  ( $\mathcal{N}(\ell_x).nr$ ). The non-atomic write of  $\ell_x$  by the right thread is guaranteed to be executed after the read by the left thread. However, when the write is executed, the race detector requires that the right thread must have observed in its local current view *all* read events (**DRF-WRITE-NA**), including  $r_{na}$ , in order to be deemed non-racy. Since the right thread did not synchronize with the left thread to obtain  $r_{na}$  in its local view, its write to  $\ell_x$  will be declared racy by the ORC11 race detector.

### 3.6 The Correspondence between RC11 and ORC11

To show a correspondence between ORC11 and RC11, we exploit the fact that ORC11 is very close to the “promise-free” fragment of Kang et al.<sup>20</sup> extended with non-atomics and a race detector. Kang et al. already proved a formal correspondence between their promise-free fragment and C11.<sup>21</sup> Building on their result, we show an *on-paper* correspondence proof between ORC11 and RC11. This proof has been provided in the technical appendix of the RB<sub>r1x</sub> paper,<sup>22</sup> and we only sketch its summary here.

It is worth noting that our language is actually the combination of the memory model of ORC11 and the  $\lambda_{Rust}$  expression language ([Chapter 4](#)). Together, ORC11 +  $\lambda_{Rust}$  is more conservative and declares more programs as having undefined behaviors than RC11. For example,  $\lambda_{Rust}$  can trigger UBs if we compare two values that are incomparable ([Definition 4.9](#)). Furthermore, the race detector in ORC11 is stronger, *i.e.*, detects more races, than RC11. In particular, ORC11 does not permit reducing an RMW operation with the **acq** access mode in the presence of an unsynchronized non-atomic read *even though* the RMW itself synchronizes with the non-atomic read. In contrast, the self-synchronizing nature of RMW leads to RC11 accepting this particular behavior as non-racy.

Consequently, we cannot show an equivalence in behaviors between ORC11 +  $\lambda_{Rust}$  and RC11. Instead, we also combine RC11 with  $\lambda_{Rust}$ , and relates ORC11 +  $\lambda_{Rust}$  with RC11 +  $\lambda_{Rust}$ . To simplify the proof, we allow RC11 to take  $\lambda_{Rust}$ ’s expression reduction steps that are disallowed with ORC11. In particular, the declarative semantics in RC11 +  $\lambda_{Rust}$  may compare arbitrary values with each other, whereas ORC11 +  $\lambda_{Rust}$  will get stuck in some of these cases.

Ultimately, we want to show that if a program has defined behaviors in ORC11 +  $\lambda_{Rust}$ ,<sup>23</sup> then it must have defined behaviors in RC11 +  $\lambda_{Rust}$ .<sup>24</sup> Conversely, we show that any (UB-triggering) racy RC11-consistent execution<sup>25</sup> can be replayed as a racy execution in ORC11.

<sup>20</sup>Kang et al., “A promising semantics for relaxed-memory concurrency” [\[Kan+17\]](#).

<sup>21</sup>[\[Kan+17\]](#), Appendix B: Proof of Theorem 5.

<sup>22</sup>Dang et al., *Technical Appendix: RustBelt Meets Relaxed Memory* [\[Dan+20b\]](#), §2.

<sup>23</sup>which is a proof dischargeable using our separation logics

<sup>24</sup>see [Definition 2.18](#)

<sup>25</sup>see [Definition 2.14](#) and [Definition 2.16](#)

Following Kang et al., we decompose the correspondence proof into 2 steps. First, we prove that any racy RC11 execution of the program can be replayed as a racy execution in the *Operational Graph Semantics* (OGS). Second, we prove that the racy OGS execution can be replayed as a racy execution in ORC11. The OGS is designed to be an intermediate mixture of RC11 and ORC11: its state is a RC11-consistent execution graph  $G$ , and every memory access operationally extends  $G$  without violating OGS's own race detector. OGS's race detector is stated in terms of execution graphs, but encodes exactly the rules of ORC11's race detector.<sup>26</sup> OGS is then extended with the same  $\lambda_{\text{Rust}}$ 's expression reduction steps as that of RC11 (which can compare arbitrary values).

We then show that (1a) any non-racy, consistent execution of RC11 +  $\lambda_{\text{Rust}}$  can be replayed as a non-racy or racy consistent execution of OGS +  $\lambda_{\text{Rust}}$ ,<sup>27</sup> and (1b) any program that has racy executions in RC11 +  $\lambda_{\text{Rust}}$  also has racy executions in OGS +  $\lambda_{\text{Rust}}$ .<sup>28</sup> Then, to relate OGS +  $\lambda_{\text{Rust}}$  and ORC11 +  $\lambda_{\text{Rust}}$ , we relate OGS's state with the timestamp assignments in ORC11's thread-view and race detector states. We then show that (2a) any non-racy, consistent execution of OGS +  $\lambda_{\text{Rust}}$  can be replayed as a non-stuck or stuck execution of ORC11 +  $\lambda_{\text{Rust}}$ ,<sup>29</sup> and (2b) any racy execution of OGS +  $\lambda_{\text{Rust}}$  can get stuck in ORC11 +  $\lambda_{\text{Rust}}$ .<sup>30</sup> Finally, combining all of those results, we show that any program that has racy executions in RC11 +  $\lambda_{\text{Rust}}$  can get stuck in ORC11 +  $\lambda_{\text{Rust}}$ .<sup>31</sup>

<sup>26</sup>see Figure 3.4

<sup>27</sup>[Dan + 20b], Lemma 3.

<sup>28</sup>[Dan + 20b], Lemma 4.

<sup>29</sup>[Dan + 20b], Lemma 6.

<sup>30</sup>[Dan + 20b], Lemma 7.

<sup>31</sup>[Dan + 20b], Theorem 1.

CHAPTER SUMMARY. This chapter explains the view-based semantics and the race-detector semantics of ORC11, and illustrates how it is related to RC11. In the next chapter, we present the  $\lambda_{\text{Rust}}$  language and explain how to combine it with ORC11 to achieve our target language that will be used to instantiate Iris.





# 4

## The Relaxed $\lambda_{\text{Rust}}$ Language

---

The *relaxed*  $\lambda_{\text{Rust}}$  language retrofits the original RustBelt’s  $\lambda_{\text{Rust}}$ <sup>1</sup> on top of the relaxed memory semantics of ORC11. In this chapter, we briefly review  $\lambda_{\text{Rust}}$  and formally define how to “plug” it in with ORC11 to obtain our target language.

It worths noting up front that we (nor the origin RustBelt’s authors) do not plan to tackle the Herculean task of giving a formal definition of the *complete* Rust language: the core  $\lambda_{\text{Rust}}$  calculus only captures central features of the Rust language, and the original semantics assume a SC memory model,<sup>2</sup> and we extend the operational semantics to cover relaxed memory accesses. Nevertheless, the reasoning principles we develop in this dissertation are not restricted to  $\lambda_{\text{Rust}}$  or Rust, and can be applied to other languages that employ the RC11 memory model (or stronger memory models).

<sup>1</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [Jun+18a].

<sup>2</sup>with a race detector that intuitively implements reader-writer locks on non-atomics locations.

### 4.1 Language Syntax

**Definition 4.1** ( $\lambda_{\text{Rust}}$  Grammar). The grammar is given in Figure 4.1.  $\lambda_{\text{Rust}}$  is a lambda calculus with:

- values that can be poison ( $\star$ ),<sup>3</sup> a block-based location, an integer (meta-variable  $z \in \mathbb{Z}$ ), or a recursive function (meta-variable  $f$ ) that has a list of binders ( $\bar{x}$ ) for the arguments.
- expressions (meta-variable  $e$ ) that can be a value; or a variable ( $x$ ); or a path operator ( $e.e$ ) where the second operand (called the offset) must evaluate to an integer offset of the first operand; or a binary operator; or function application ( $e(\bar{e})$ ) where the arguments are a list expressions; or a (switch) case block (**case**  $e$  **of**  $\bar{e}$ ) that allows branching into a list of expressions; or a fork operator that supports forking new (detached) threads; or a memory instruction which can be a read, a write, a compare-and-swap (**CAS**), or a fence instruction with different consistency modes;<sup>4</sup> or an explicit allocation or deallocation instruction.

<sup>3</sup>see §3.2.

<sup>4</sup>see Definition 2.2.

**Definition 4.2** ( $\lambda_{\text{Rust}}$  Left-to-Right Evaluation Contexts). The reduction strategy of  $\lambda_{\text{Rust}}$ ’s expressions is encoded using *evaluation contexts*  $K \in \text{ECtx}$ . The approach of evaluation contexts decomposes an expression  $e$  into an evaluation context  $K$  and an expression  $e'$  that can perform a

$$\begin{aligned}
v \in \text{Val} &::= \text{⋆} \mid \ell \mid z \mid \mathbf{rec} \, f(\bar{x}) := e \\
e \in \text{Expr} &::= \mid v \mid x \mid e.e \\
&\mid e + e \mid e - e \mid e \leq e \mid e == e \\
&\mid e(\bar{e}) \\
&\mid \mathbf{case} \, e \mathbf{of} \, \bar{e} \\
&\mid \mathbf{fork} \{ e \} \\
&\mid {}^*o e \mid e_1 :=_o e_2 \mid \mathbf{CAS}^{of,or,ow}(e_0, e_1, e_2) \mid \mathbf{fence}_o \\
&\mid \mathbf{alloc}(e) \mid \mathbf{free}(e_1, e_2) \\
K \in \text{Ctx} &::= \mid \bullet \\
&\mid K.e \mid v.K \\
&\mid K + e \mid v + K \mid K - e \mid v - K \mid K \leq e \mid v \leq K \mid K == e \mid v == K \\
&\mid K(\bar{e}) \mid v(\bar{v} \mathbin{++} [K] \mathbin{++} \bar{e}) \\
&\mid \mathbf{case} \, K \mathbf{of} \, \bar{e} \\
&\mid {}^*o K \mid K :=_o e \mid v :=_o K \\
&\mid \mathbf{CAS}^{of,or,ow}(K, e_1, e_2) \mid \mathbf{CAS}^{of,or,ow}(v_0, K, e_2) \mid \mathbf{CAS}^{of,or,ow}(v_0, v_1, K) \\
&\mid \mathbf{alloc}(K) \mid \mathbf{free}(K, e_2) \mid \mathbf{free}(e_1, K)
\end{aligned}$$

FIGURE 4.1: The relaxed  $\lambda_{\text{Rust}}$  language syntax.

*primitive reduction* and so will be evaluated next, satisfying  $e = K[e']$ . The empty context  $\bullet$  is called the “hole” where the next-to-be-evaluated expression is filled in.

The evaluation strategy is left-to-right call-by-value, and is given in [Figure 4.1](#). Let us consider an example evaluation of an assignment  $e_1 :=_o e_2$ :

- The expression is first decomposed into  $K_1 = \bullet :=_o e_2$  and  $e_1$ , which allows  $e_1$  to be evaluated first.
- After  $e_1$  is evaluated to a value  $v_1$ , the expression is  $K_1[v_1] = v_1 :=_o e_2$ , which can be decomposed into  $K_2 = v_1 :=_o \bullet$ , which now allows  $e_2$  to be evaluated.
- Once  $e_2$  is evaluated to a value  $v_2$ , the expression is  $K_2[v_2] = v_1 :=_o v_2$ , which is decomposed into  $\bullet$  and  $v_1 :=_o v_2$ .
- The primitive reduction of assignments then can kick in and complete the evaluation.

**Notation 4.3** ( $\lambda_{\text{Rust}}$  Syntactic Sugars). Several syntactic sugars are taken as-is from the original RustBelt, given in [Figure 4.2](#). Specifically:

- Non-recursive functions  $(\lambda[\bar{x}]. e)$  simply ignore the recursive function argument. **let** bindings are used to declare local variables in  $\lambda_{\text{Rust}}$ , which are pure and do not occupy memory (they are not mutable nor addressable). They are simply evaluated and then substituted into the remaining expression, hence the definition using functions. Sequential composition is defined using **let** bindings.

$$\begin{aligned}
& \lambda[\bar{x}].e ::= \mathbf{rec\_}([\bar{x}]) := e \\
& \mathbf{let } x = e \mathbf{ in } e' ::= (\lambda[x].e')([e]) \\
& \quad e'; e ::= \mathbf{let\_} = e' \mathbf{ in } e \\
& \mathbf{false} ::= 0 \\
& \mathbf{true} ::= 1 \\
& \mathbf{if } e_0 \mathbf{ then } e_1 \mathbf{ else } e_2 ::= \mathbf{case } e_0 \mathbf{ of } [e_2, e_1] \\
& \quad *e ::= *_{\mathbf{na}} e \\
& e_1 := e_2 ::= e_1 :=_{\mathbf{na}} e_2 \\
& \mathbf{new} ::= \lambda[\text{size}]. \mathbf{if } \text{size} == 0 \mathbf{ then } (42, 1337) \mathbf{ else } \mathbf{alloc}(\text{size}) \\
& \mathbf{delete} ::= \lambda[\text{size}, \text{ptr}]. \mathbf{if } \text{size} == 0 \mathbf{ then } \text{⊥} \mathbf{ else } \mathbf{free}(\text{size}, \text{ptr}) \\
& \mathbf{memcpy} ::= \mathbf{rec } \mathbf{memcpy}([\text{dst}, \text{len}, \text{src}]) := \\
& \quad \mathbf{if } \text{len} \leq 0 \mathbf{ then } \text{⊥} \mathbf{ else } \\
& \quad \quad \text{dst}.0 := \text{src}.0; \\
& \quad \quad \mathbf{memcpy}([\text{dst}.1, \text{len} - 1, \text{src}.1]) \\
& e_1 :=_n *e_2 ::= \mathbf{memcpy}(e_1, n, e_2) \\
& e :=_{\text{inj } i} () ::= e.0 := i \\
& e_1 :=_{\text{inj } i} e_2 ::= e_1.0 := i; e_1.1 := e_2 \\
& e_1 :=_{\text{inj } i, n} *e_2 ::= e_1.0 := i; e_1.1 :=_n *e_2 \\
& \mathbf{skip} ::= \mathbf{let } x = \text{⊥} \mathbf{ in } \text{⊥} \\
& \mathbf{newlft} ::= \text{⊥} \\
& \mathbf{endlft} ::= \mathbf{skip}
\end{aligned}$$
FIGURE 4.2: Some syntactic sugars for  $\lambda_{\text{Rust}}$ .

- We use 0 and 1 as boolean values **false** and **true**, respectively. This allows us to define **if**-branching using **case**: if  $e_0$  is **false**, then the expression with index 0 in the list  $[e_2, e_1]$ , which is  $e_2$  is picked to be evaluated next; and if  $e_0$  is **true**, then the expression with index 1, which is  $e_1$ , is picked.
- We suppress the access modes for reads and writes if they are **na**.
- **new** and **delete**, unlike **alloc** and **free**, never get stuck when the provided block size is 0. **new** simply just returns some location in that case.
- **memcpy** copies  $\text{len}$  cells from  $\text{src}$  to  $\text{dst}$  using the path operator. The “assign with length” notation  $(e_1 :=_n *e_2)$  uses **memcpy**.
- There is no language primitive to define compound data structures. Instead, they can be implemented in memory using pointer arith-

$$\begin{aligned}
\mathbf{letcont} \ k(\bar{x}) &:= e \ \mathbf{in} \ e' ::= \mathbf{let} \ k = (\mathbf{rec} \ k(\bar{x}) := e) \ \mathbf{in} \ e' \\
\mathbf{jump} \ k(\bar{e}) &::= k(\bar{e}) \\
\mathbf{funrec} \ f(\bar{x}) \ \mathbf{ret} \ k &:= e ::= \mathbf{rec} \ f([k] \uparrow \bar{x}) := e \\
\mathbf{call} \ f(\bar{e}) \ \mathbf{ret} \ k &::= f([k] \uparrow \bar{e})
\end{aligned}$$
FIGURE 4.3: CPS notations for  $\lambda_{\text{Rust}}$ .

metic. In particular,  $e_1 \stackrel{\text{inj } i}{=} e_2$  simulates *tagged union* (sum types): the location at offset  $e.0$  stores the active tag  $i$  (the case of the sum) while the location at offset  $e.1$  stores the actual value evaluated from  $e_2$ .  $e_1 \stackrel{\text{inj } i}{=}^n e_2$  is the notation for writing a tagged union whose values span multiple locations (memory cells).

- Finally, **newlft** and **endlft** are “ghost instructions” that have no interesting operational behaviors, and are only needed for logical soundness of the RustBelt type system (Part III).

**Notation 4.4** (Continuation-Passing Style). Programs in  $\lambda_{\text{Rust}}$  can be presented in continuation-passing style (CPS). This allows for encoding more complex control-flow constructs like labeled **break** or early **return**. This also makes  $\lambda_{\text{Rust}}$  closer to Rust’s *Mid-level Intermediate Representation* (MIR)<sup>5</sup> than to surface Rust. The notations are given in Figure 4.3.

<sup>5</sup>Matsakis, *Introducing MIR* [Mat16].

- Continuations (meta-variable  $k$ ) can be defined with **letcont**  $k(\bar{x}) := e \ \mathbf{in} \ e'$  where  $\bar{x}$  is a list of binders that will be instantiated with the arguments  $\bar{e}$  when a continuation is called with **jump**  $k(\bar{e})$ .
- CPS functions can be declared with **funrec**  $f(\bar{x}) \ \mathbf{ret} \ k := e$  where  $f$  is the binder for a recursive function,  $\bar{x}$  the list of binders for the arguments, and  $k$  the binder for the return continuation that will be called when the function returns. The return continuation takes only one argument for the return value. Accordingly, CPS functions can be called using **call**  $f(\bar{e}) \ \mathbf{ret} \ k$  where  $\bar{e}$  are the list of arguments and  $k$  the return continuation argument.

*Note 4.5.* Most syntactic sugars (Notation 4.3) and the CPS notations (Notation 4.4) are needed for the type system in RustBelt (Part III).

## 4.2 Language Expression Reductions

The complete semantics is defined by three sub semantics: the view-based machine semantics (§3.3), the race-detector semantics (§3.4), and the expressions semantics defined in this section. The complete semantics will be given in §4.3.

We first define some more auxiliary definitions.

**Definition 4.6** (Readable Memory Value).

$$\omega \in \text{Readable}(\ell, \mathcal{M}, \mathcal{V})$$

The predicate  $\text{Readable}(\ell, \mathcal{M}, \mathcal{V})$  defines the set of memory values *readable* from  $\ell$  for the global memory  $\mathcal{M}$  by a thread  $\pi$  whose thread-view is

currently  $\mathcal{V}$ .

$$\omega \in \text{Readable}(\ell, \mathcal{M}, \mathcal{V}) ::= \exists t. \mathcal{M}(\ell)(t) = (\omega, \_) \wedge t \leq \mathcal{V}.\text{cur}(\ell)$$

That is, the thread  $\pi$  can only read a memory value  $\omega$  that is in the memory for  $\ell$  and is not **mo**-earlier than  $\pi$ 's current view  $\mathcal{V}.\text{cur}$  for  $\ell$ . This is the same condition as **OM-READ** (Definition 3.22), but additionally concerns values.

**POINTER COMPARISON** is a problem on its own,<sup>6</sup> especially for dead pointers. In this work, for simplicity, we follow the original RustBelt work and assume the most conservative choice that avoids UB:<sup>7</sup> unallocated pointers can non-deterministically be compared equal even though their representations are not.

**Definition 4.7** (Value Equality).

$$\boxed{\mathcal{M} \vdash v_1 = v_2}$$

The results of *equality* comparison in  $\mathcal{M}$  are defined by the following rules. (Recall that  $z$  is the meta-variable for integers.)

$$\mathcal{M} \vdash z = z \quad \mathcal{M} \vdash \ell = \ell \quad \frac{\ell_1 \in \text{unalloc}(\mathcal{M}) \vee \ell_2 \in \text{unalloc}(\mathcal{M})}{\mathcal{M} \vdash \ell_1 = \ell_2}$$

**Definition 4.8** (Value Inequality).

$$\boxed{\vdash v_1 \neq v_2}$$

The results of *inequality* comparison are defined by the following rules.

$$\frac{z_1 \neq z_2}{\vdash z_1 \neq z_2} \quad \frac{\ell_1 \neq \ell_2}{\vdash \ell_1 \neq \ell_2} \quad \vdash \ell \neq 0 \quad \vdash 0 \neq \ell$$

That is, two values compare in-equal if their representations are different, and locations are never null (0). Note that this means that equality and inequality are not mutually exclusive for unallocated pointers.

**Definition 4.9** (*Val* (non-UB) Comparability).

$$\boxed{\vdash v_1 =^? v_2}$$

Two values are *comparable*, and thus may be compared equal and/or in-equal, if they satisfy the following rules.

$$\vdash z_1 =^? z_2 \quad \vdash \ell_1 =^? \ell_2 \quad \vdash \ell =^? 0 \quad \vdash 0 =^? \ell$$

**Definition 4.10** ( $\lambda_{\text{Rust}}$  Expression Reductions).

$$\boxed{\mathcal{M}, \mathcal{V} \vdash e \xrightarrow{\varepsilon^?} e'_1, e'_2^?}$$

The expression reduction relation  $\mathcal{M}, \mathcal{V} \vdash e \xrightarrow{\varepsilon^?} e'_1, e'_2^?$  says that under the global memory  $\mathcal{M}$  and the thread-view  $\mathcal{V}$ , the expression  $e$  reduces in one step to  $e'_1$ , potentially with an optional memory event  $\varepsilon^?$  and an optional expression  $e'_2$  that will be running concurrently in a newly forked thread. Only memory operations will generate a memory event  $\varepsilon$ , and only **fork**  $\{e'_2\}$  generates the new thread's expression  $e'_2$ . The rules for the expression reduction are given in Figure 4.4.

- **OE-CTX** is the general rule that drives the evaluation strategy through evaluation contexts (see Definition 4.2). The remaining rules are the *primitive reductions* that only reduce in one step.

<sup>6</sup>Kang et al., “A formal C memory model supporting integer-pointer casts” [Kan+15]; Memarian et al., “Into the depths of C: elaborating the de facto standards” [Mem+16]; Lee et al., “Reconciling high-level optimizations and low-level code in LLVM” [Lee+18]; Besson et al., “CompCertS: A Memory-Aware Verified C Compiler Using a Pointer as Integer Semantics” [BBW19]; Memarian et al., “Exploring C semantics and pointer provenance” [Mem+19]; Lepigre et al., “VIP: verifying real-world C idioms with integer-pointer casts” [Lep+22].

<sup>7</sup>UB is not a choice because pointer comparison is possible in safe Rust.

$$\begin{array}{c}
\text{OE-ECTX} \quad \frac{e \rightarrow e'_1, e'_2}{\mathcal{M}, \mathcal{V} \vdash K[e] \rightarrow K[e'_1], e'_2} \quad \text{OE-PROJ} \quad \frac{}{\mathcal{M}, \mathcal{V} \vdash \ell.n \rightarrow \ell +_{\mathbb{Z}} n} \quad \text{OE-ADD} \quad \frac{z_1 +_{\mathbb{Z}} z_2 = z'}{\mathcal{M}, \mathcal{V} \vdash z_1 + z_2 \rightarrow z'} \quad \text{OE-SUB} \quad \frac{z_1 -_{\mathbb{Z}} z_2 = z'}{\mathcal{M}, \mathcal{V} \vdash z_1 - z_2 \rightarrow z'} \\
\\
\text{OE-LE-TRUE} \quad \frac{z_1 \leq_{\mathbb{Z}} z_2}{\mathcal{M}, \mathcal{V} \vdash z_1 \leq z_2 \rightarrow \mathbf{true}} \quad \text{OE-LE-FALSE} \quad \frac{z_1 >_{\mathbb{Z}} z_2}{\mathcal{M}, \mathcal{V} \vdash z_1 \leq z_2 \rightarrow \mathbf{false}} \quad \text{OE-EQ-TRUE} \quad \frac{\mathcal{M} \vdash v_1 = v_2}{\mathcal{M}, \mathcal{V} \vdash v_1 == v_2 \rightarrow \mathbf{true}} \\
\\
\text{OE-EQ-FALSE} \quad \frac{\vdash v_1 \neq v_2}{\mathcal{M}, \mathcal{V} \vdash v_1 == v_2 \rightarrow \mathbf{false}} \quad \text{OE-APP} \quad \frac{|\bar{x}| = |\bar{v}|}{\mathcal{M}, \mathcal{V} \vdash (\mathbf{rec} f(\bar{x}) := e)(\bar{v}) \rightarrow e[\mathbf{rec} f(\bar{x}) := e/f, \bar{v}/\bar{x}]} \\
\\
\text{OE-CASE} \quad \frac{0 \leq i < |\bar{e}|}{\mathcal{M}, \mathcal{V} \vdash \mathbf{case} i \mathbf{ of } \bar{e} \rightarrow \bar{e}_i} \quad \text{OE-FORK} \quad \frac{}{\mathcal{M}, \mathcal{V} \vdash \mathbf{fork} \{e\} \rightarrow \text{poison}, e} \\
\\
\text{OE-READ} \quad \frac{}{\mathcal{M}, \mathcal{V} \vdash *o \ell \xrightarrow{\mathbf{R}^o(\ell, v)} v} \quad \text{OE-WRITE} \quad \frac{}{\mathcal{M}, \mathcal{V} \vdash \ell :=_o v \xrightarrow{\mathbf{W}^o(\ell, v)} \text{poison}} \\
\\
\text{OE-CAS-FAIL} \quad \frac{\mathbf{rlx} \sqsubseteq o_f, o_r, o_w \quad \forall \omega \in \text{Readable}(\ell, \mathcal{M}, \mathcal{V}). \exists v'. \omega \equiv v' \wedge \vdash v_1 =^? v' \quad \vdash v_1 \neq v_r}{\mathcal{M}, \mathcal{V} \vdash \mathbf{CAS}^{o_f, o_r, o_w}(\ell, v_1, v_2) \xrightarrow{\mathbf{R}^{of}(\ell, v_r)} \mathbf{false}} \\
\\
\text{OE-CAS-SUCC} \quad \frac{\mathbf{rlx} \sqsubseteq o_f, o_r, o_w \quad \forall \omega \in \text{Readable}(\ell, \mathcal{M}, \mathcal{V}). \exists v'. \omega \equiv v' \wedge \vdash v_1 =^? v' \quad \mathcal{M} \vdash v_1 = v_r}{\mathcal{M}, \mathcal{V} \vdash \mathbf{CAS}^{o_f, o_r, o_w}(\ell, v_1, v_2) \xrightarrow{\mathbf{U}^{or, ow}(\ell, v_r, v_2)} \mathbf{true}} \\
\\
\text{OE-FENCE} \quad \frac{}{\mathcal{M}, \mathcal{V} \vdash \mathbf{fence}_o \xrightarrow{\mathbf{F}^o} \text{poison}} \quad \text{OE-ALLOC} \quad \frac{n > 0}{\mathcal{M}, \mathcal{V} \vdash \mathbf{alloc}(n) \xrightarrow{\mathbf{A}(\ell, n)} \ell} \quad \text{OE-FREE} \quad \frac{n > 0}{\mathcal{M}, \mathcal{V} \vdash \mathbf{free}(n, \ell) \xrightarrow{\mathbf{D}(\ell, n)} \text{poison}}
\end{array}$$

FIGURE 4.4: Relaxed  $\lambda_{\text{Rust}}$  expression semantics.

- **OE-PROJ** says that the path operator simply computes a new location with offset  $n \in \mathbb{Z}$  from  $\ell$ , using the meta-level operator  $+_{\mathbb{Z}}$  which is defined as  $(i, n') +_{\mathbb{Z}} n = (i, n' +_{\mathbb{Z}} n)$ . (Note that the offsets are integers, and use the integer operator  $+_{\mathbb{Z}}$ .)
- **OE-ADD**, **OE-SUB**, **OE-LE-TRUE**, and **OE-LE-FALSE** say that integer operators reduce according to the meta-level integer operators. Note that we have no reduction rules for poison ( $\text{poison}$ ), which means that any computation using poison will get stuck. Also recall that **true** and **false** are just notations for 1 and 0, respectively.
- **OE-EQ-TRUE** and **OE-EQ-FALSE** say that comparison reduce according to equality and inequality comparisons, respectively.<sup>8</sup> This means that comparing unallocated locations can non-deterministically reduce to either **true** or **false**.
- **OE-CASE** says that a **case** block reduces if the *choice* index  $i$  is an actual index into the list of expressions. Then the expression  $\bar{e}_i$  will

<sup>8</sup>see Definition 4.7 and Definition 4.8.

be picked to reduce. Note that no expression in the list  $\bar{e}$  is reduced before the **case** is reduced.

- **OE-APP** says that function application reduces once all arguments have been evaluated to a list of values  $\bar{v}$ .<sup>9</sup> It is also required that the list of binders and the list of arguments have the same length. Then, the reduction *substitutes* the arguments for the binders, including the recursive function binder  $f$ , in the function's body.
- **OE-FORK** says that **fork**  $\{e\}$  returns poison in the forking thread (so that the return value should not be used), and  $e$  will be used for the newly forked thread (see §4.3).
- **OE-READ** says that a read simply reduces to the value  $v$  that comes with the read event that  $R^o(\ell, v)$  it is tied to. The memory event will be used to match this reduction with the view-based machine (§3.3) and the race detector (§3.4) in the complete semantics (§4.3).
- **OE-WRITE** says that a write reduces to poison and is tied to the corresponding write event.
- A compare-and-swap instruction **CAS** <sup>$o_f, o_r, o_w$</sup>  $(\ell, v_1, v_2)$  takes three *atomic* access modes:  $o_f$  is the order that will be used when the **CAS** fails, in which case it acts like a read with the mode  $o_f$ ; otherwise, if the **CAS** succeeds, then it acts as both a read with the mode  $o_r$  and a write with the mode  $o_w$ . The **CAS** *atomically* (i) reads the location  $\ell$ , (ii) compares the value read  $v_r$  with  $v_1$ , and (iii) if the values are equal, writes  $v_2$  to  $\ell$ . **OE-CAS-FAIL** and **OE-CAS-SUCC** therefore both require that for any memory value  $\omega$  *readable*<sup>10</sup> by the **CAS**, its injected value  $v'$  must be comparable<sup>11</sup> with  $v_1$ .
  - In case of failure, **OE-CAS-FAIL** says that it must be that  $v_1$  is in-equal to the read value  $v_r$ , and the reduction reduces to **false** and is tied to the read event  $R^{o_f}(\ell, v_r)$ .
  - In case of success, **OE-CAS-SUCC** says that it must be that  $v_1$  is equal to the read value  $v_r$ , and the reduction reduces to **true** and is tied to the update event  $U^{o_r, o_w}(\ell, v_r, v_2)$ .

Note again that this means that the **CAS** can non-deterministically fail or succeed if  $v_1$  and what  $\ell$  stores can be unallocated location values.

- **OE-FENCE** says that a fence also reduces to poison and is tied to the corresponding fence event.
- Finally, **OE-ALLOC** and **OE-FREE** both require that the provided block size  $n$  is positive, and respectively are tied to the memory events  $A(\ell, n)$  and  $D(\ell, n)$ . The allocation call returns the base location  $\ell$  of the newly allocated block, while the deallocation call returns poison.

*Note 4.11.* In the expression reductions, the global memory  $\mathcal{M}$  is used in equality comparison (**OE-EQ-TRUE**). Other than that, it is only used

<sup>9</sup>Recall that by the **Definition 4.2** for evaluation contexts, the arguments are evaluated left-to-right.

<sup>10</sup>*i.e.*, those values that are not yet overshadowed by  $\mathcal{V}.cur(\ell)$ —see **Definition 4.6**.

<sup>11</sup>see **Definition 4.9**.

$$\begin{array}{c}
\text{ForkView}(\mathcal{V}) ::= (\emptyset, \emptyset, \mathcal{V}.\text{cur}, \mathcal{V}.\text{cur}) \\
\\
\text{OC-PURE} \quad \frac{\mathcal{M}, \mathcal{V} \vdash e \rightarrow e', e_f^?}{(\mathcal{M}, \mathcal{N}) \mid (e, \mathcal{V}) \xrightarrow{\perp, (e_f, \text{ForkView}(\mathcal{V}))^?}_t (\mathcal{M}, \mathcal{N}) \mid (e', \mathcal{V})} \\
\\
\text{OC-MEM} \quad \frac{\forall \varepsilon'. (\mathcal{M}, \mathcal{V} \vdash e \xrightarrow{\varepsilon'} \_, \perp) \wedge (\mathcal{M} \mid \mathcal{V} \xrightarrow{\varepsilon', \text{ForkView}(\mathcal{V})} \_ \mid \_) \implies \mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\varepsilon')}{\frac{\mathcal{M}, \mathcal{V} \vdash e \xrightarrow{\varepsilon} e', \perp \quad \mathcal{M} \mid \mathcal{V} \xrightarrow{\varepsilon, r^?, ms} \mathcal{M}' \mid \mathcal{V}' \quad \mathcal{N} \xrightarrow{\varepsilon, r^?, ms} \mathcal{N}'}{(\mathcal{M}, \mathcal{N}) \mid (e, \mathcal{V}) \xrightarrow{\varepsilon, \perp}_t (\mathcal{M}', \mathcal{N}') \mid (e', \mathcal{V}')}}
\end{array}$$

FIGURE 4.5: The combined 1-thread semantics of ORC11 machine semantics and  $\lambda_{\text{Rust}}$  expression semantics.

together with the thread-view  $\mathcal{V}$  for comparison in **CAS** (**OE-CAS-FAIL** and **OE-CAS-SUCC**).

### 4.3 The Complete Operational Semantics of Relaxed $\lambda_{\text{Rust}}$

**Definition 4.12** (1-Thread Reductions).  $\boxed{\varsigma \mid (e, \mathcal{V}) \xrightarrow{\varepsilon^?, (e_f, \mathcal{V}_f)^?}_t \varsigma' \mid (e', \mathcal{V})'}$

The combined *1-thread* (single-thread) reductions of ORC11 machine semantics and  $\lambda_{\text{Rust}}$  expression semantics is given in **Figure 4.5**. The configuration  $(\mathcal{M}, \mathcal{N}) \mid (e, \mathcal{V})$  is called a 1-thread configuration which includes the thread's executing expression  $e$ , the thread-view  $\mathcal{V}$ , and the global state  $\mathcal{M}$  and  $\mathcal{N}$ . The pair  $(e, \mathcal{V})$  is also called a *thread-local configuration*. The combined 1-thread reductions define how a 1-thread configuration is transformed in one reduction step, possibly generating an optional memory event  $\varepsilon$  and an optional pair of expression and thread-view  $(e_f, \mathcal{V}_f)$  for a newly forked thread.

- **OC-PURE** allows for a pure step that only reduces the expression of the configuration, and potentially generates an expression  $e_f$  for a new thread. In that case, the thread-view  $\text{ForkView}(\mathcal{V})$ , derived from  $\mathcal{V}$ , is picked for the newly forked thread. The definition of  $\text{ForkView}(\mathcal{V})$  encodes our choice for **fork**'s behaviors with respect to synchronization: the forked (child) thread should be synchronized with the forking thread, but a **fork** does not act as a release fence for the forked thread, so its release views are empty ( $\emptyset$ ).
- **OC-MEM** allows for a memory step that simultaneously (i) reduces the expression  $e$  in one step to  $e'$  ( $\mathcal{M}, \mathcal{V} \vdash e \xrightarrow{\varepsilon} e', \perp$ , **Definition 4.10**) with a memory event  $\varepsilon$ , and (ii) makes a view-based machine step ( $\mathcal{M} \mid \mathcal{V} \xrightarrow{\varepsilon, r^?, ms} \mathcal{M}' \mid \mathcal{V}'$ , **Definition 3.22**) and a race detector step ( $\mathcal{N} \xrightarrow{\varepsilon, r^?, ms} \mathcal{N}'$ , **Definition 3.26**) with the *same* memory event  $\varepsilon$ , potentially with an optional read action  $id$   $r^?$  and a list of write messages  $ms$ . The reduction needs to be race-free, *i.e.*, for any potential memory step that the current configuration can make and thus generate an event  $\varepsilon'$ , it must be that  $\mathcal{M}, \mathcal{N}, \mathcal{V} \vdash \text{RaceFree}(\varepsilon')$  (**Definition 3.25**).



$$\text{OT-STEP} \quad \frac{\mathcal{T}(\pi) = (e, \mathcal{V}) \quad (\mathcal{M}, \mathcal{N}) \mid (e, \mathcal{V}) \xrightarrow{\varepsilon^?, (e_f, \mathcal{V}_f)^?} (\mathcal{M}', \mathcal{N}') \mid (e', \mathcal{V}')}{(\mathcal{M}, \mathcal{N}) \mid \mathcal{T} \rightarrow (\mathcal{M}', \mathcal{N}') \mid \mathcal{T}[\pi \leftarrow (e', \mathcal{V}')] [\rho \leftarrow (e_f, \mathcal{V}_f) \mid (e_f, \mathcal{V}_f)^? \neq \perp \wedge \rho \notin \text{dom}(\mathcal{T})]}$$

FIGURE 4.6: Threadpool semantics.

Finally, we can lift the 1-thread semantics to the complete, concurrent semantics with a thread-pool.

**Definition 4.13** (Thread-pools). A thread-pool  $\mathcal{T}$  is a partial, finite map from thread-ids to pairs of expressions and thread-views, i.e.,

$$\mathcal{T} \in \text{ThreadPool} ::= \text{Thread} \xrightarrow{\text{fin}} (\text{Expr} \times \text{ThreadView})$$

**Definition 4.14** (Threadpool Reductions).

$$\boxed{\varsigma \mid \mathcal{T} \rightarrow \varsigma' \mid \mathcal{T}'}$$

The thread-pool semantics is given in [Figure 4.6](#). It defines the reduction of a *thread-pool configuration*  $\varsigma \mid \mathcal{T}$  that includes the global machine state ([Definition 3.14](#)) and the thread-pool for all threads. **OT-STEP** says that a thread-pool reduction just picks some random thread  $\pi$  in the thread-pool and make a 1-thread step using the 1-thread configuration  $\varsigma \mid (e, \mathcal{V})$  for the global state and thread  $\pi$ . The results of the 1-thread step are then used to update the thread-pool configuration. In case thread  $\pi$  forks a new thread with  $(e_f, \mathcal{V}_f)$ , then a fresh thread-id  $\rho \notin \text{dom}(\mathcal{T})$  is picked to insert the newly forked thread into the thread-pool.

**CHAPTER SUMMARY.** This chapter presents the  $\lambda_{\text{Rust}}$  language and explains how to combine it with the machine semantics of ORC11 to achieve our target language. In the next chapter, we instantiate Iris with this language to obtain a vanilla separation logic for RMC. Note that Iris takes as input the 1-thread reductions ([Definition 4.12](#)) of a language and defines its own thread-pool reductions. We only state the thread-pool reductions ([Definition 4.14](#)) for completeness, which is similar to (but simpler than) that of Iris.



# 5

## Related Work

---

Podkopaev et al.<sup>1</sup> develop an operational account of a subset of C11 that includes relaxed accesses and non-atomics. However, it lacks support for fences and thus could not be used as is to build a logic to verify libraries that use fences *e.g.*, Rust’s `Arc`. Their semantics also does not forbid the data race in [Example 3.29](#).

Doherty et al.<sup>2</sup> develop an operational semantics based on event graphs for the release/acquire/relaxed fragment of RC11. They also develop an invariant-based logic geared towards automated verification for programs in that fragment. Their operational semantics supports neither non-atomics nor fences.

Kang et al.’s *promising semantics*<sup>3</sup> proposes using *promises* to fix C11’s out-of-thin-air problem without prohibiting load-store reordering on relaxed accesses, as RC11 and ORC11 do. The promising semantics as well as its versions 2.0 and 2.1,<sup>4</sup> however, does not include non-atomic accesses, as the authors did not consider the problem of undefined behaviors (UBs). Instead, the semantics has plain accesses that can race among one another and still have defined semantics.

Recently, Cho et al.<sup>5</sup> extends version 2.1 (PS2.1) with UB-inducing non-atomics (dubbed  $PS^{na}$ ) in order to verify optimizations on non-atomics across atomics. Non-atomics in  $PS^{na}$  can also be promised, and write-write races with a non-atomic access invokes UB, while read-write races results in the read of undef (which is more defined than the poison  $\text{⌘}$  value).<sup>6</sup> The race detection mechanism of  $PS^{na}$  is similar to that of ORC11: a non-atomic access is racy if the thread is not locally aware of another access to the same location with a bigger timestamp (similar to `DRF-READ-NA` and `DRF-WRITE-NA`); and an atomic access is racy if the thread is not aware of another non-atomic access to the same location with a bigger timestamp (similar to `DRF-READ-AT`, `DRF-WRITE-AT` and `DRF-UPDATE`). Interestingly, the implementation of  $PS^{na}$ ’s race detector is much simpler: instead of using a global non-atomic view  $\mathcal{N}$  as in ORC11,  $PS^{na}$  distinguishes between atomic and non-atomic messages in the global memory  $\mathcal{M}$ , and the check for races simply looks at the timestamps and the messages’ types. We believe that ORC11 can benefit from such a simplification.

The follow-up work of  $PS^{na}$  by Lee et al.<sup>7</sup> proposes a two-layered semantics: a source language semantics *without* promises, and an intermediate representation (IR) semantics that can have promises (dubbed

<sup>1</sup>Podkopaev et al., “Operational Aspects of C/C++ Concurrency” [[PSN16](#)].

<sup>2</sup>Doherty et al., “Verifying C11 programs operationally” [[Doh+19](#)].

<sup>3</sup>Kang et al., “A promising semantics for relaxed-memory concurrency” [[Kan+17](#)].

<sup>4</sup>Lee et al., “Promising 2.0: global optimizations in relaxed memory concurrency” [[Lee+20](#)]; Cho et al., “Modular data-race-freedom guarantees in the promising semantics” [[Cho+21b](#)].

<sup>5</sup>Cho et al., “Sequential reasoning for optimizing compilers under weak memory concurrency” [[Cho+22](#)].

<sup>6</sup>Lee et al., “Taming undefined behavior in LLVM” [[Lee+17](#)].

<sup>7</sup>Lee et al., “Putting Weak Memory in Order via a Promising Intermediate Representation” [[Lee+23](#)].

<sup>8</sup>Cho et al., “Sequential reasoning for optimizing compilers under weak memory concurrency” [Cho+22].

<sup>9</sup>[Pul+18; Flu+17; Pul+19; Sim+20].

<sup>10</sup>Cho et al., “Revamping hardware persistency models: view-based and axiomatic persistency models for Intel-x86 and Armv8” [Cho+21a].

<sup>11</sup>Simner et al., “ARMv8-A System Semantics: Instruction Fetch in Relaxed Architectures” [Sim+20].

<sup>12</sup>Simner et al., “Relaxed virtual memory in Armv8-A” [Sim+22].

$\text{PS}^{\text{IR}}$ ) but *only for race-detection purpose*. The paper also comes with a proposal to hardware developers to justify the semantics of  $\text{PS}^{\text{IR}}$ . If the proposal is accepted, it means that iRC11 can be applied to most C and C++ RMC programs. As of now, iRC11 can only be used for promise-free programs (a property which can be checked externally), and the verification results can be ported to the full semantics using the DRF theorems by Cho et al.<sup>8</sup>.

Meanwhile, semantics for low-level languages, such as that of the ARM ISA,<sup>9</sup> do not employ UBs. Regardless, their modern semantics include hardware optimizations such as speculative execution and multiple layers of caches, resulting in complex memory models. It remains to be seen if the view-based approach can be used to model various hardware features—there have been view-based semantics for non-volatile memory,<sup>10</sup> but, to the best of our knowledge, not for instruction caches<sup>11</sup> nor virtual memory.<sup>12</sup>

## Part II

### SEPARATION LOGIC FOR RELAXED MEMORY



This part discusses the features and the construction of iRC11. We give a brief review of the Iris separation logic framework in [Chapter 6](#) and discuss the instantiation of Iris with ORC11 in [Chapter 7](#), which results in the *base logic* for ORC11. The base logic, however, is very close to the operational semantics, and only provides basic separation reasoning principles. [Chapter 8](#), following iGPS,<sup>13</sup> presents the first abstraction layer that gives rise to the iRC11 logic: *view-monotone predicates*, or vProp for short. The chapter also presents several RMC-specific modalities of iRC11 in vProp, some of which are inspired by FSL and Cosmo.<sup>14</sup> [Chapter 9](#) and [Chapter 10](#) present the construction for the core ownership assertions of iRC11: the *non-atomic* and *atomic points-to*. [Chapter 11](#) introduces *invariants*—the standard principle for concurrently sharing resources—but with RMC-specific limitations. Finally, [Chapter 12](#) ends this part with several simple example verifications of RMC programs and libraries using iRC11. The bottom half of [Figure 1.1](#) visualizes the dependency among these chapters.

<sup>13</sup>Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [[Kai+17](#)].

<sup>14</sup>Doko and Vafeiadis, “A Program Logic for C11 Memory Fences” [[DV16](#)]; Mével et al., “Cosmo: a concurrent separation logic for multicore OCaml” [[MJP20](#)].





# 6

## More Background: Iris, A Framework for Concurrent Separation Logics

In this chapter, we give a quick review of the concurrent separation logic framework Iris.<sup>1</sup> Readers who are familiar with Iris can skip this review, and jump to [Chapter 7](#) for the instantiation of Iris with our relaxed  $\lambda_{\text{Rust}}$  + ORC11 language. On the other hand, for readers who prefer a deep dive into the details of Iris, please consult the journal paper [\[Jun+18b\]](#).

Iris as a framework contains (i) a higher-order, resource-aware, step-indexing *base logic* with bunched implications (BI),<sup>2</sup> (ii) extensions<sup>3</sup> that support program verification (*i.e.*, program logics with *weakest preconditions* and *impredicative invariants*) for an input language with an operational interleaving semantics, and (iii) a general Iris Proof Mode (IPM)<sup>4</sup> that supports interactive resource reasoning and that can be instantiated with any BI logics.

An excerpt of Iris grammar is given in [Figure 6.1](#). Propositions in Iris belong to the type `iProp`, which has a *step-indexing* model on *resources*.<sup>5</sup>

**Concept 6.1** (Iris Base Logic). The Iris base logic supports the common logical connectives (False, True,  $\Rightarrow$ ,  $\wedge$ ,  $\vee$ ). `iProp` propositions are to be interpreted with resources in mind, so the “classical” conjunction  $P \wedge Q$  should be read as  $P$  and  $Q$  hold relying on the *same* resources. The base logic allows embedding *pure* facts  $\phi$  which exist at the meta-level and which naturally do not occupy resources.

- It is a BI logic: the *separating* conjunction  $P * Q$  says that  $P$  and  $Q$  hold on *disjoint* resources, and the *wand* implication  $P \multimap Q$  holds on some resource  $r$  that can be combined with some resource  $s$  where  $P$  holds to obtain the resource  $r \cdot s$  where  $Q$  holds.
- The base logic also supports *higher-order* logical quantification ( $\forall, \exists$ ) and recursive predicates ( $\mu$ ) because the quantified variable  $x$  can also be an `iProp`. Recursive predicates need to be *guarded*: occurrences of  $x$  in the body need to be under a *later* modality  $\triangleright$ .
- The later modality is the materialization of the step-indexing model in the logic:  $\triangleright P$  intuitively means that  $P$  holds in the *next* step, so  $P$  only becomes available until the program takes a step (and so decrements the step-index). Step-indexing guarantees the logic’s soundness in the presence of recursive higher-order quantifications, impredicative invariants, and higher-order ghost state.

<sup>1</sup>Jung et al., “Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning” [\[Jun+15\]](#); Jung et al., “Higher-order ghost state” [\[Jun+16\]](#); Krebbers et al., “The Essence of Higher-Order Concurrent Separation Logic” [\[Kre+17\]](#); Jung et al., “Iris from the ground up: A modular foundation for higher-order concurrent separation logic” [\[Jun+18b\]](#).

<sup>2</sup>O’Hearn and Pym, “The logic of bunched implications” [\[OP99\]](#); Ishiaq and O’Hearn, “BI as an Assertion Language for Mutable Data Structures” [\[IO01\]](#).

<sup>3</sup>derived from the base logic

<sup>4</sup>Krebbers et al., “Interactive Proofs in Higher-Order Concurrent Separation Logic” [\[KTB17\]](#); Krebbers et al., “MoSeL: A General, Extensible Modal Framework for Interactive Proofs in Separation Logic” [\[Kre+18\]](#).

<sup>5</sup>[\[Jun+18b\]](#), §4.

$$\begin{aligned}
P \in \text{iProp} ::= & \text{(* base logic *)} \\
& | \phi \mid \text{False} \mid \text{True} \mid P \Rightarrow Q \mid P \wedge Q \mid P \vee Q \mid P * Q \mid P \multimap Q \mid \exists x. P \mid \forall x. P \mid \mu x. P \\
& | \triangleright P \mid \Box P \mid \{a : \overline{M}\}^\gamma \mid \dot{\Rightarrow} P \mid \dots \\
& \text{(* program logic *)} \\
& | \boxed{P}^\mathcal{N} \mid \varepsilon_1 \dot{\Rightarrow}^{\varepsilon_2} P \mid \text{wp}_\mathcal{E} e \{v. Q\} \mid \dots
\end{aligned}$$

FIGURE 6.1: An excerpt of Iris grammar.

- The *persistent* modality  $\Box P$  says that  $P$  is known to hold without some *exclusive* resource, so that  $P$  can be freely duplicated, i.e.,  $P \Rightarrow P * P$ .
- The proposition  $\{a : \overline{M}\}^\gamma$  asserts the ownership of an element  $a$  of a *resource algebra*  $M$  for the *ghost location*  $\gamma$ . In case the resource algebra  $M$  is clear in context, we simply write  $\{a\}^\gamma$ .
- The *basic update* modality  $\dot{\Rightarrow} P$  hides away some *ghost updates* that can be performed to achieve  $P$ .

**Concept 6.2** (Iris Program Logic). To support program verification for a language, several constructions can be derived from the base logic:<sup>6</sup>

- $\boxed{P}^\mathcal{N}$  asserts the existence of some global *invariant* that holds the resource  $P$ , under some *invariant name* that is in the *namespace*  $\mathcal{N}$ . Namespaces provide some hierarchy to sets of invariant names.
- The *fancy update*  $\varepsilon_1 \dot{\Rightarrow}^{\varepsilon_2} P$  hides away some logical updates (including ghost updates) that can be performed to achieve  $P$ . The logical updates involve accessing (opening and closing) invariants and therefore trading resources with the involved invariants. The *masks*  $\varepsilon_1$  and  $\varepsilon_2$  are sets of invariant names that identify the invariants that hold (unopened) *before* and *after* the update, respectively.
- The *weakest pre-condition*  $\text{wp}_\mathcal{E} e \{v. Q\}$  assert the resources needed for  $e$  to safely execute<sup>7</sup> and maintain the invariants in  $\mathcal{E}$  at every step, and if  $e$  terminates to a value  $v$ , then the post-condition  $Q$  holds. Note that the “if” signals that the program logic by default only guarantee *partial*, not *total* correctness.

<sup>6</sup>Not all connectives in the Iris base logic are primitives, some are also derived.

<sup>7</sup>never get stuck

## 6.1 Basic Rules

Figure 6.2 provides several basic rules of many connectives in the Iris base logic. The *logical entailment*  $P \vdash Q$  says that  $Q$  is derivable from  $P$  using the rules of the logic. Intuitively, its interpretation is that for any resource and step-index where  $P$  holds,  $Q$  should also hold. The notation  $P \dashv\vdash Q$  stands for  $P \vdash Q$  and  $Q \vdash P$ .

The rules in Figure 6.2 are very general rules that apply to all Iris propositions. For example, they include commutativity, associativity, and distributivity among connectives and modalities; **PERS-ELIM** tells us that

$$\begin{array}{c}
 \text{True} * P \dashv\vdash P \\
 P * Q \dashv\vdash Q * P \\
 (P * Q) * R \dashv\vdash P * (Q * R)
 \end{array}
 \qquad
 \begin{array}{c}
 \text{SEP-MONO} \\
 \frac{P_1 \vdash Q_1 \quad P_2 \vdash Q_2}{P_1 * P_2 \vdash Q_1 * Q_2}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{WAND-INTRO-ELIM} \\
 \frac{P * Q \vdash R}{P \vdash Q \multimap R}
 \end{array}$$

Several rules for the persistent modality.

$$\begin{array}{c}
 \frac{P \vdash Q}{\Box P \vdash \Box Q}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{PERS-IDEMP} \\
 \Box \Box P \dashv\vdash \Box P
 \end{array}
 \qquad
 \begin{array}{c}
 \text{PERS-ELIM} \\
 \Box P \vdash P
 \end{array}
 \qquad
 \begin{array}{c}
 \Box P \wedge \Box Q \dashv\vdash \Box P * \Box Q \\
 \Box(P * Q) \dashv\vdash \Box P * \Box Q
 \end{array}
 \qquad
 \begin{array}{c}
 \Box \exists x. P \dashv\vdash \exists x. \Box P \\
 \Box \forall x. P \vdash \forall x. \Box P
 \end{array}$$

Several rules for the later modality.

$$\begin{array}{c}
 \text{LATER-INTRO} \\
 P \vdash \triangleright P
 \end{array}
 \qquad
 \begin{array}{c}
 \text{LATER-MONO} \\
 \frac{P \vdash Q}{\triangleright P \vdash \triangleright Q}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{LÖB} \\
 (\triangleright P \Rightarrow P) \vdash P
 \end{array}
 \qquad
 \begin{array}{c}
 \triangleright(P * Q) \dashv\vdash \triangleright P * \triangleright Q \\
 \Box \triangleright P \dashv\vdash \triangleright \Box P
 \end{array}
 \qquad
 \begin{array}{c}
 \tau \text{ inhabited} \\
 \frac{}{\triangleright \exists x : \tau. P \dashv\vdash \exists x : \tau. \triangleright P} \\
 \triangleright \forall x. P \dashv\vdash \forall x. \triangleright P
 \end{array}$$

FIGURE 6.2: Basic rules of several Iris connectives.

we can always get  $P$  from  $\Box P$ ; or **LATER-INTRO** says that if we have  $P$  now, we also have  $P$  in the next step; or **LÖB** allows us to do Löb induction: if having  $P$  in the next step is sufficient to achieve  $P$  now, then we can have  $P$  now.

## 6.2 Ghost State and Resource Algebras

The ghost ownership assertion  $\boxed{a : M}^\gamma$  requires  $a$  to be an element of a resource algebra  $M$ . Resource algebras give the separation structure for ghost state in separation logics, and are a generalization of Iris for *partial commutative monoids* (PCMs).

**Definition 6.3** (Resource Algebras). A resource algebra (RA) is a tuple  $(M, \text{valid} : M \rightarrow \text{Prop}, | - | : M \rightarrow M^?, (\cdot) : M \times M \rightarrow M)$  where the type  $M$  has a commutative, associative *composition*  $(\cdot)$ ; a *core* function  $(| - |)$  that computes a optional *per-element* unit (in  $M^?$ ) for each element  $a \in M$ ; and a *validity* predicate ( $\text{valid}$ ) to indicate legal compositions.

Compared to PCMs, RAs force composition to be total, and instead regain partiality with validity. Furthermore, where a PCM has a single unit element  $\varepsilon$ , an RA can have multiple per-element units  $|a|$ , and some elements may not have a unit, *i.e.*,  $|a| = \perp$ .

The following properties must hold for an RA.

$$\begin{array}{ll}
 \forall a, b. a \cdot b = b \cdot a & (\text{RA-COMM}) \\
 \forall a, b, c. (a \cdot b) \cdot c = a \cdot (b \cdot c) & (\text{RA-ASSOC}) \\
 \forall a. |a| \in M \Rightarrow |a| \cdot a = a & (\text{RA-CORE-ID}) \\
 \forall a. |a| \in M \Rightarrow ||a|| = a & (\text{RA-CORE-IDEMP}) \\
 \forall a. |a| \in M \wedge a \preceq b \Rightarrow |b| \in M \wedge |a| \preceq |b| & (\text{RA-CORE-MONO}) \\
 \forall a, b. \text{valid}(a \cdot b) \Rightarrow \text{valid}(a) & (\text{RA-VALID-OP}) \\
 \text{where } a^? \cdot \perp = \perp \cdot a^? = a^? & \\
 a \preceq b ::= \exists c \in M. b = a \cdot c & (\text{RA-INCL})
 \end{array}$$

$$\begin{array}{c}
\text{GHOST-ALLOC} \\
\frac{\text{valid}(a)}{\dot{\Rightarrow} \exists \gamma. [\bar{a}]^\gamma} \\
\\
\text{GHOST-VALID} \\
[\bar{a}]^\gamma \Rightarrow \text{valid}(a) \\
\\
\text{GHOST-OP} \\
[\bar{a} \cdot b]^\gamma \Leftrightarrow [\bar{a}]^\gamma * [\bar{b}]^\gamma \\
\\
\text{GHOST-UPDATE-GEN} \\
\frac{a \rightsquigarrow B}{[\bar{a}]^\gamma \dot{\Rightarrow} \exists b \in B. [\bar{b}]^\gamma} \\
\\
\text{GHOST-UPDATE} \\
\frac{a \rightsquigarrow b}{[\bar{a}]^\gamma \dot{\Rightarrow} [\bar{b}]^\gamma} \\
\\
\text{BUPD-MONO} \\
\frac{P \vdash Q}{\dot{\Rightarrow} P \vdash \dot{\Rightarrow} Q} \\
\\
\text{BUPD-INTRO} \\
P \vdash \dot{\Rightarrow} P \\
\\
\text{BUPD-TRANS} \\
\dot{\Rightarrow} \dot{\Rightarrow} P \vdash \dot{\Rightarrow} P \\
\\
\text{BUPD-FRAME} \\
Q * \dot{\Rightarrow} P \vdash \dot{\Rightarrow} (Q * P)
\end{array}$$

where  $P \dot{\Rightarrow} Q ::= \Box(P * \dot{\Rightarrow} Q)$ .

FIGURE 6.3: Basic rules of Iris ghost ownership and basic updates.

**RA-CORE-ID** says that if a core exists for some  $a$ , then it is a unit for  $a$ , and **RA-CORE-MONO** says that the core function maintains *inclusion*, defined by **RA-INCL** using composition. **RA-VALID-OP** says that if a composition is considered valid, all of its components must also be valid.

A *unit* RA (uRA) has a unit element  $\varepsilon$  satisfying:

$$\text{valid}(\varepsilon) \quad \forall a \in M. \varepsilon \cdot a = a \quad |\varepsilon| = \varepsilon$$

The rules for ghost ownership are given in Figure 6.3. **GHOST-ALLOC** lets us allocate a fresh ghost location  $\gamma$  with an initial, valid element that is  $a$ . Under the hood, this is a ghost update to the global *ghost heap* to insert the fresh ghost location  $\gamma$ , and this update is hidden in the basic update modality  $\dot{\Rightarrow}$ . **GHOST-VALID** says that ghost ownership maintains validity. More importantly, **GHOST-OP** says that the RA composition gives the separation structure to ghost ownership.

**Notation 6.4** (Basic Viewshifts).

$$\boxed{P \dot{\Rightarrow} Q}$$

Intuitively, *basic viewshift*  $P \dot{\Rightarrow} Q$  says that the resources owned by  $P$  can be turned into the resources owned by  $Q$  using some ghost updates.

$$P \dot{\Rightarrow} Q ::= \Box(P * \dot{\Rightarrow} Q)$$

**GHOST-UPDATE-GEN** and **GHOST-UPDATE** allow us to update some ghost state, using the basic viewshift. **GHOST-UPDATE-GEN** allows for a non-deterministic update: some element in  $b$  will be picked after the update. To maintain consistency of separation, the premises of these rules require that ghost updates are *frame-preserving*.

**Concept 6.5** (Frame-preserving Ghost Updates).

$$\boxed{a \rightsquigarrow b}$$

When doing a ghost update for  $[\bar{a}]^\gamma$ , one must remember that one only has a *part* of the ghost location  $\gamma$ : with separation (*i.e.*, using **GHOST-OP**), other parts of  $\gamma$  that are compatible with  $a$ , called the *frame*, are owned by other parties. As such, for any  $b$  that we update  $a$  to, one must maintain that  $a$  is also compatible with the frame, *i.e.*, one cannot allow  $b$  composed with the frame being *invalid*, leading to inconsistency in the logic (turning a valid state into an invalid one). The relation  $a \rightsquigarrow b$  encodes the fact that the update from  $a$  to  $b$  maintains validity of the whole ghost state, *i.e.*, it is a frame-preserving update.

$a \rightsquigarrow b$  is derived from the more general definition  $a \rightsquigarrow B$  which says that  $a$  can be frame-preservingly updated to any element in  $B$ :

$$a \rightsquigarrow B ::= \forall c^? \in M^?. \text{valid}(a \cdot c^?) \Rightarrow \exists b \in B. \text{valid}(b \cdot c^?)$$

(RA-FRAME-UPD-GEN)

$$a \rightsquigarrow b ::= a \rightsquigarrow \{b\}$$

(RA-FRAME-UPD)

In this definition, “the frame” is universally quantified as  $c$ .

Figure 6.3 also provides some structural rules for basic updates.

### 6.3 Invariants and Fancy Updates

Invariants can be seen as logical, global spaces where resources can be stored for concurrent accesses.<sup>8</sup> The catch is that accesses must be (*physically*) *atomic*—take place during a single step of computation—and invariants must be re-established after each access, so that they indeed hold *invariantly* (*i.e.*, after each step). As such, invariants are used to build concurrent protocols on pieces of shared state, *i.e.*, to constrain how clients can change them.

The construction of Iris invariants, however, is not tied to the notion of atomic expression. Instead, it uses *invariant namespaces* and *masks* to track which invariants are opened (being accessed), and only subsequently tie weakest pre-conditions to masks to enforce atomicity. We will see that in §6.7. Here, we look at the vanilla rules of Iris invariants.

The proposition  $\boxed{I}^{\mathcal{N}}$  asserts the existence of  $I$  in the global invariant space with some invariant name  $\iota$  in the namespace  $\mathcal{N}$ .<sup>9</sup> Several rules for Iris invariants are given in Figure 6.4, which rely on *fancy updates*, which in turn generalize basic updates with masks. Intuitively,  $\varepsilon_1 \multimap^{\varepsilon_2} P$  represents ownership of resources such that, assuming that the invariants in  $\varepsilon_1$  are *enabled* (they are not opened) before, one can perform frame-preserving updates and afterwards obtaining  $P$  and having the invariants in  $\varepsilon_2$  enabled. As such, if the masks  $\varepsilon_1$  and  $\varepsilon_2$  are different, some invariants may have been opened to achieve  $P$ , or some other resources must have been returned to close some invariants. Furthermore, masks prevent reentrancy: an invariant cannot be opened again without being closed first.

This intuition is on display in the invariant access rule **INV-ACC**:

- If we know that the set of enabled invariants  $\varepsilon_1$  includes the namespace  $\mathcal{N}$ —meaning that the invariants in  $\mathcal{N}$  are not opened yet, then we can open all invariants in  $\mathcal{N}$  with the fancy update  $\varepsilon_1 \multimap^{\varepsilon_1 \setminus \mathcal{N}}$  (so after that only the invariants in  $\varepsilon_1 \setminus \mathcal{N}$  are enabled).
- Furthermore, if we know that  $\boxed{I}^{\mathcal{N}}$ , *i.e.*,  $I$  is stored in one of those invariants in  $\mathcal{N}$  that are to be opened, then we get access to the resources of  $I$ , but under a *later* modality ( $\triangleright I$ ). The later ensures that  $I$  is *guarded*, because invariants are impredicative, *e.g.*,  $I$  can refer to  $\boxed{I}^{\mathcal{N}}$  itself.

<sup>8</sup>They are indeed implemented in Iris as a global chunk of resources, see ([Jun+18b], §7.1).

<sup>9</sup>The meta-variable  $I$  is preferred over  $P$  to indicate resources that are stored in invariants.

$$\begin{array}{c}
\text{INV-ALLOC} \\
\triangleright I \vdash \models_{\mathcal{E}} \boxed{I}^{\mathcal{N}} \\
\\
\text{FUPD-BUPD} \\
\vdash P \vdash \models_{\mathcal{E}} P \\
\\
\text{FUPD-MONO} \\
\frac{P \vdash Q}{\varepsilon_1 \models_{\mathcal{E}_2} P \vdash \varepsilon_1 \models_{\mathcal{E}_2} Q} \\
\\
\text{FUPD-FRAME} \\
Q * \varepsilon_1 \models_{\mathcal{E}_2} P \vdash \varepsilon_1 \uplus \varepsilon_2 \models_{\mathcal{E}_2 \uplus \varepsilon_1} (Q * P) \\
\\
\text{INV-ACC} \\
\frac{\mathcal{N} \subseteq \mathcal{E}}{\boxed{I}^{\mathcal{N}} \vdash \varepsilon \models_{\mathcal{E} \setminus \mathcal{N}} (\triangleright I * (\triangleright I \varepsilon \setminus \mathcal{N} \multimap^{\mathcal{E}} \text{True}))} \\
\\
\text{FUPD-INTRO} \\
P \vdash \models_{\mathcal{E}} P \\
\\
\text{FUPD-INTRO-MASK} \\
\frac{\mathcal{E}_2 \subseteq \mathcal{E}_1}{\text{True} \vdash \varepsilon_1 \models_{\mathcal{E}_2} \varepsilon_2 \models_{\mathcal{E}_1} \text{True}} \\
\\
\text{FUPD-TRANS} \\
\varepsilon_1 \models_{\mathcal{E}_2} \varepsilon_2 \models_{\mathcal{E}_3} P \vdash \varepsilon_1 \models_{\mathcal{E}_3} P
\end{array}$$

where  $\models_{\mathcal{E}} ::= \varepsilon \models^{\mathcal{E}}$  and  $P \varepsilon \multimap^{\mathcal{E}'} Q ::= P * \varepsilon \models^{\mathcal{E}'} Q$ .

FIGURE 6.4: Some rules for Iris invariants and fancy updates.

- Finally, we also get a “closing” update,  $(\triangleright I \multimap^{\mathcal{E} \setminus \mathcal{N}} \models^{\mathcal{E}} \text{True})$ , which allows us to return the invariant resources, also under a later,  $\triangleright I$ , we can close all invariants and re-establish that  $\mathcal{E}$  is enabled.

Note that even though all invariants in  $\mathcal{N}$  are opened during the access, we only take out  $\triangleright I$  and so we only need to return  $\triangleright I$ . The resources of other invariants are untouched and are kept safe under the “closing” update. However, this means that the access rule does not support accessing two invariants stored under the same namespace together. To access two invariants  $I_1$  and  $I_2$  together, we need to allocate them in two *disjoint* namespaces  $\mathcal{N}_1 \# \mathcal{N}_2 \subseteq \mathcal{E}$ . Then we can apply **INV-ACC** twice, first with  $\boxed{I_1}^{\mathcal{N}_1}$  and  $\mathcal{E}$ , then with  $\boxed{I_2}^{\mathcal{N}_2}$  and  $\mathcal{E} \setminus \mathcal{N}_2$ , to get access to  $\triangleright I_1 * \triangleright I_2$ .

**INV-ALLOC** allows us to allocate an invariant  $I$  with some fresh invariant name  $\iota$  picked non-deterministically from  $\mathcal{N}$ . Note that we only need to provide  $I$  under a later, and that  $\models_{\mathcal{E}}$  is a notation for a fancy update that does not change masks. This rule justifies the use of namespaces: if we had use only invariant names, then when accessing invariants we would have to deal with disjointness for names which are allocated dynamically. Instead, in this setup with namespaces, we only have to deal with disjointness of namespaces which can be picked statically. Note that this also means that both masks and namespaces need to contain an infinite number of names.

**Figure 6.4** also provides some structural rules for fancy updates. Importantly, **FUPD-BUPD** says that a fancy update includes a basic update.

**Notation 6.6** (Fancy Updates and Wand Viewshifts).  $\models_{\mathcal{E}}$  denotes non-mask-changing fancy updates  $\varepsilon \models^{\mathcal{E}}$ , and  $P \varepsilon \multimap^{\mathcal{E}'} Q$  denotes *wand viewshifts*, a combination of wand implication and fancy update:

$$P \varepsilon \multimap^{\mathcal{E}'} Q ::= P * \varepsilon \models^{\mathcal{E}'} Q$$

$P \multimap_{\mathcal{E}} Q$  denotes non-mask-changing wand viewshifts.

Plain viewshifts are the persistent version of wand viewshifts:

$$\begin{aligned}
P \varepsilon \Rightarrow^{\mathcal{E}'} Q &::= \Box(P * \varepsilon \models^{\mathcal{E}'} Q) \\
P \Rightarrow_{\mathcal{E}} Q &::= \Box(P * \varepsilon \models_{\mathcal{E}} Q)
\end{aligned}$$

## 6.4 Hoare Triples

Once we instantiate Iris with a language that has an interleaving operational semantics defined in the style of evaluation contexts, the Iris problem logic provides us a notion of *weakest pre-conditions* propositions that encode partial correctness of expressions. (We will see the instantiation of the relaxed  $\lambda_{\text{Rust}}$  language in [Chapter 7](#).) Intuitively, the proposition  $\text{wp}_{\mathcal{E}} e \{v. Q\}$  asserts the ownership of some resources with which  $e$  can execute safely (i.e., it never gets stuck) while maintaining the invariants in  $\mathcal{E}$  at every step, and if  $e$  evaluates to a value  $v$ , then we arrive at some resources at which the *post-condition*  $Q$  holds.

The goal of program verification for some program  $e$  is to prove that the weakest pre-condition with some suitable target post-condition is derivable from some sufficient resources—the *pre-condition*—using the rules of the program logic. This can be seen more concretely in the Iris definition of *Hoare triples*.

**Definition 6.7** (Iris Hoare Triples). Hoare triples in Iris are defined in terms of weakest pre-conditions:

$$\{P\} e \{v. Q\}_{\mathcal{E}} ::= \Box (P \multimap \text{wp}_{\mathcal{E}} e \{v. Q\})$$

The persistent modality ( $\Box$ ) guarantees that the precondition  $P$  is sufficient to prove the weakest pre-condition—that is, the wand does not need extra resources. The intuitive interpretation of Hoare triples is straightforward: the precondition  $P$  is sufficient for the expression  $e$  to safely execute while maintaining the invariants in  $\mathcal{E}$ , and if  $e$  terminates to  $v$  then  $Q$  holds.

## 6.5 Adequacy

Weakest pre-conditions and Hoare triples are also Iris propositions (iProp), and are in fact also defined entirely in the logic of Iris to encode the aforementioned intuition. However, once we have proven a weakest pre-condition or a Hoare triple for some program, we would like to achieve a guarantee *outside of the logic* that the program executes safely under the target operational semantics. This is called *adequacy*: for each instantiation of Iris with some *language*  $\Lambda$ , we need to prove once and for all the following Theorem.

**Theorem 6.8** (Iris Adequacy). *If  $\vdash \text{wp}_{\top} e \{v. \phi(v)\}$  is derivable in the Iris program logic for  $\Lambda$  where  $\phi(v)$  is a pure (meta-level) fact, then the following holds.*

$$\begin{aligned} \forall \pi, \mathcal{T}, \sigma. ([\pi \mapsto e], \sigma_{\text{init}}) \rightarrow_{\Lambda}^* (\mathcal{T}, \sigma) \Rightarrow \\ \forall v. \mathcal{T}(\pi) = v \Rightarrow \phi(v) & \quad (\text{IRIS-ADEQUACY-VAL}) \\ \wedge \quad \forall \rho, e_{\rho}. \mathcal{T}(\rho) = e_{\rho} \Rightarrow (e_{\rho} \text{ is a value} \vee \text{red}(e_{\rho}, \sigma)) & \quad (\text{IRIS-ADEQUACY-NO-STUCK}) \end{aligned}$$

That is, if we start running  $e$  with the initial global configuration  $([\pi \mapsto e], \sigma_{\text{init}})$ —where the threadpool only has a single thread  $\pi$  with

the expression  $e$  and the initial global state  $\sigma_{\text{init}}$  is typically empty—then for any configuration reachable  $(\mathcal{T}, \sigma)$  through the reflexive, transitive closure of the threadpool reduction  $\rightarrow_{\Lambda}$  for  $\Lambda$ , (**IRIS-ADEQUACY-VAL**) if thread  $\pi$  has reduces to a value  $v$ , the pure fact  $\phi(v)$  must hold, and (**IRIS-ADEQUACY-NO-STUCK**) for any thread  $\rho$  with the expression  $e_{\rho}$ , it is not stuck: either  $e_{\rho}$  is a value, or  $e_{\rho}$  is still reducible on  $\sigma$  ( $\text{red}(e_{\rho}, \sigma)$ ).

Note that the mask of the weakest pre-condition is  $\top$ , which means that all invariants (any allocated) hold at every step. Note also that deriving  $\vdash \text{wp}_{\top} e \{v. \phi(v)\}$  is the same as deriving a Hoare triple with a trivial pre-condition, i.e.,  $\vdash \{\text{True}\} e \{v. Q\}_{\top}$ .

## 6.6 Some Common Rules for WPs and Hoare Triples

On the other hand, during program verification, in order to derive  $\text{wp}_{\mathcal{E}} e \{v. Q\}$ , one relies on the various rules for weakest pre-conditions (WP). Many of those rules are language-specific: after instantiate Iris with our target language  $\Lambda$ , we need to extend the logic with WP rules for the primitive instructions of  $\Lambda$ . We will see those primitive rules for our relaxed  $\lambda_{\text{Rust}} + \text{ORC11}$  language in [Chapter 7](#). Here, we look at some WP rules that are not so language-specific.

More concretely, these are the typical rules we would expect for a lambda-calculus based language with an operational semantics using evaluation contexts and fork-based concurrency. These rules, as well as their corresponding derived Hoare-triple versions, are given in [Figure 6.5](#). Note that when reading the WP rules, by reading them *backward*—as typically when we use a rule of form  $\text{wp}_{\mathcal{E}} e' \{\Psi\} \vdash \text{wp}_{\mathcal{E}} e \{\Phi\}$  to turn the verification goal  $\text{wp}_{\mathcal{E}} e \{\Phi\}$  into the goal  $\text{wp}_{\mathcal{E}} e' \{\Psi\}$ , we are driving the symbolic execution of the program *forward*: the expression  $e$  reduces to the expression  $e'$ . Note that we also use the meta-variables  $\Phi$  and  $\Psi$  for predicates on values ( $\text{Val} \rightarrow \text{iProp}$ ) which can be used for post-conditions (which so far have been written as  $v. Q$ ).

- **WP-VAL** says that if the expression has reached a value  $v$ , then we simply have to prove the post-condition  $\Phi(v)$ . **HOARE-VAL** is derivable from **WP-VAL** with  $\Phi ::= \lambda w. w = v$ .
- **WP-MONO** allows for monotonicity for the post-conditions: to prove a WP with the post-condition  $\Psi$  we may want to prove a WP with the stronger post-condition  $\Phi$ . It also additionally provides monotonicity for masks: if we can prove a WP relying on fewer invariants (with the smaller mask  $\mathcal{E}_1$ ), then that WP also works with more invariants (with the bigger mask  $\mathcal{E}_2$ ).<sup>10</sup> The well-known consequence rule **HOARE-CONS** is derivable from **WP-MONO**.
- **WP-BIND** is the core rule to drive sequential composition. It makes use of evaluation contexts. To prove a WP for  $K[e]$ , we prove a WP for  $e$  whose post-condition is another WP that plugs the resulting  $v$  into the “continuation”  $K$ . This corresponds to executing  $e$  first and then the continuation  $K$ . **HOARE-BIND** is derivable from **WP-BIND**.

<sup>10</sup>So that from a  $\text{wp}_{\mathcal{E}} e \{\Phi\}$  one can, for example, obtain  $\text{wp}_{\top} e \{\Phi\}$  with the top ( $\top$ ) mask, and subsequently apply adequacy.



|                                                                                                                                                    |                                                                                                                                                                                                                      |                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <p>WP-VAL</p> $\Phi(v) \vdash \text{wp}_{\mathcal{E}} v \{ \Phi \}$                                                                                | <p>WP-MONO</p> $\frac{\mathcal{E}_1 \subseteq \mathcal{E}_2}{(\forall v. \Phi(v) \multimap \models_{\mathcal{E}_2} \Psi(v)) * \text{wp}_{\mathcal{E}_1} e \{ \Phi \} \vdash \text{wp}_{\mathcal{E}_2} e \{ \Psi \}}$ |                                                                                                                                        |
| <p>WP-BIND</p> $\text{wp}_{\mathcal{E}} e \{ v. \text{wp}_{\mathcal{E}} K[v] \{ \Phi \} \} \vdash \text{wp}_{\mathcal{E}} K[e] \{ \Phi \}$         | <p>WP-FRAME</p> $P * \text{wp}_{\mathcal{E}} e \{ \Phi \} \vdash \text{wp}_{\mathcal{E}} e \{ v. P * \Phi(v) \}$                                                                                                     |                                                                                                                                        |
| <p>WP-LAM</p> $\triangleright \text{wp}_{\mathcal{E}} e[v/x] \{ \Phi \} \vdash \text{wp}_{\mathcal{E}} (\lambda x. e)v \{ \Phi \}$                 | <p>WP-FORK</p> $\triangleright \Phi() * \triangleright \text{wp}_{\top} e \{ \_ . \text{True} \} \vdash \text{wp}_{\mathcal{E}} \mathbf{fork} \{ e \} \{ \Phi \}$                                                    |                                                                                                                                        |
| <p>HOARE-VAL</p> $\{ \text{True} \} v \{ w. w = v \}_{\mathcal{E}}$                                                                                | <p>HOARE-BIND</p> $\frac{\{ P \} e \{ v. Q \}_{\mathcal{E}} \quad \forall v. \{ Q \} K[v] \{ w. R \}_{\mathcal{E}}}{\{ P \} K[e] \{ w. R \}_{\mathcal{E}}}$                                                          | <p>HOARE-FRAME</p> $\frac{\{ P \} e \{ v. Q \}_{\mathcal{E}}}{\{ P * R \} e \{ v. Q * R \}_{\mathcal{E}}}$                             |
| <p>HOARE-CONS</p> $\frac{P \vdash P' \quad \{ P' \} e \{ v. Q' \}_{\mathcal{E}} \quad \forall v. Q' \vdash Q}{\{ P \} e \{ v. Q \}_{\mathcal{E}}}$ | <p>HOARE-LAM</p> $\frac{\{ P \} e[v/x] \{ w. Q \}_{\mathcal{E}}}{\{ P \} (\lambda x. e)v \{ w. Q \}_{\mathcal{E}}}$                                                                                                  | <p>HOARE-FORK</p> $\frac{\{ P \} e \{ \_ . \text{True} \}_{\top}}{\{ P \} \mathbf{fork} \{ e \} \{ \_ . \text{True} \}_{\mathcal{E}}}$ |

FIGURE 6.5: Some common rules for Iris weakest pre-conditions and Hoare triples.

and **WP-MONO**. Note that Hoare-triple rules should be read as the separating conjunction of the premises implies the conclusion.

- **WP-FRAME** allows for “framing”: if the post-condition requires us to prove  $P$  that has nothing to do with the execution of  $e$ , then we can frame  $P$  out and separately prove the WP for  $e$  with the remaining post-condition  $\Phi$ . The famous frame rule **HOARE-FRAME** is derivable from **WP-FRAME**.
- **WP-LAM** is a primitive rule for a language with beta-reduction. The rule says that we need to prove the WP for the expression after beta-reduction. However, because the reduction takes a step, we only need to prove the new WP under a later. **HOARE-LAM** is derivable from **WP-LAM** and **LATER-INTRO**.
- **WP-FORK** is a primitive rule for a language with fork-based concurrency. We need to prove, only under a later because the reduction takes a step, (i) the current post-condition  $\Phi()$  for the current thread (assuming the returned value of **fork** is unit), and (ii) a WP for the newly forked thread  $\rho$  (with the expression  $e$ ) with a trivial post-condition, which signifies that forked threads are “detached” by the default.<sup>11</sup> The rule **HOARE-FORK** is derivable from **WP-FORK**, and in that rule we can see that some resources  $P$  can be transferred from the forking thread to the forked thread.

<sup>11</sup>Iris, however, does support picking a fixed, non-trivial post-condition for all forked threads. But such a post-condition only restricts the set of possible final states of the forked threads. If one wants to communicate a post-condition  $Q$  back to the forking thread, however, one can implement a join operation (using an extra location) to wait for the thread  $\rho$  to receive  $Q$ —see §12.3.

## 6.7 Weakest Pre-conditions and Invariants

Figure 6.6 provides some rules for the interaction between WPs (Hoare triples) and invariants and fancy updates.

- **WP-FUPD** says that we can perform fancy updates around an expression if our goal is a WP. Combining this rule with **FUPD-MONO**

$$\begin{array}{c}
\text{WP-FUPD} \\
\frac{}{\models_{\mathcal{E}} \text{wp}_{\mathcal{E}} e \{v. \models_{\mathcal{E}} \Phi(v)\} \vdash \text{wp}_{\mathcal{E}} e \{\Phi\}}
\\
\text{WP-INV} \\
\frac{\triangleright I \vdash \text{wp}_{\mathcal{E} \setminus \mathcal{N}} e \{v. \triangleright I * \Phi(v)\} \quad \text{atomic}(e) \quad \mathcal{N} \subseteq \mathcal{E}}{\boxed{I}^{\mathcal{N}} \vdash \text{wp}_{\mathcal{E}} e \{\Phi\}}
\\
\text{HOARE-VS} \\
\frac{P \Rightarrow_{\mathcal{E}} P' \quad \{P'\} e \{v. Q'\}_{\mathcal{E}} \quad \forall v. Q' \Rightarrow_{\mathcal{E}} Q}{\{P\} e \{v. Q\}_{\mathcal{E}}}
\\
\text{HOARE-INV} \\
\frac{\{\triangleright I * P\} e \{v. \triangleright I * Q\}_{\mathcal{E} \setminus \mathcal{N}} \quad \text{atomic}(e) \quad \mathcal{N} \subseteq \mathcal{E}}{\boxed{I}^{\mathcal{N}} \vdash \{P\} e \{v. Q\}_{\mathcal{E}}}
\end{array}$$

where  $P \Rightarrow_{\mathcal{E}} Q ::= \Box(P * \models_{\mathcal{E}} Q)$  (see [Notation 6.6](#)).

FIGURE 6.6: Some rules for Iris weakest pre-conditions and invariants.

and **FUPD-FRAME** ([Figure 6.4](#)), around a WP we can *eliminate* any hypotheses with fancy updates in our proof context if they have masks smaller than  $\mathcal{E}$ . With **FUPD-BUPD**, we can additionally perform ghost updates, and with **FUPD-INTRO-MASK** and **INV-ACC**, we can also open invariants and close them immediately to obtain some duplicable *knowledge*. Furthermore, **HOARE-CONS** can be strengthened to **HOARE-VS**.

- **WP-ATOMIC** allows us to perform *mask-changing* updates around (physically) *atomic* instructions—those whose execution takes place in a single step of computation. If  $e$  is atomic, then to prove  $\text{wp}_{\mathcal{E}_1} e \{\Phi\}$ , we can perform a mask-changing fancy update from the mask  $\mathcal{E}_1$  to the  $\mathcal{E}_2$ , and prove a WP for the mask  $\mathcal{E}_2$  whose post-condition must perform a reverse mask-changing update from  $\mathcal{E}_2$  back to  $\mathcal{E}_1$  before establishing the original post-condition.
- **WP-INV** allows us to open invariants around atomic instructions, and it is derivable from **WP-ATOMIC**. If we let  $\mathcal{E}_1 = \mathcal{E}$ , and  $\mathcal{E}_2 = \mathcal{E} \setminus \mathcal{N}$ , we can use **WP-ATOMIC** and then **INV-ACC** to open the invariant  $\boxed{I}^{\mathcal{N}}$  around a goal of  $\text{wp}_{\mathcal{E}} e \{\Phi\}$ , and use  $\triangleright I$  for the execution of the atomic expression  $e$ , if we can re-establish  $\triangleright I$  after the step. **HOARE-INV** is easily derivable from **WP-INV**.

## 6.8 Properties of Propositions

There are two important properties of propositions that make proofs in step-indexing separation logics more convenient. Several rules for these two properties are given in [Figure 6.7](#).

**Property 6.9** (Timeless Propositions). *Timeless* propositions are those who are independent of the step index, and thus are not affected by the later modality. Concretely **TIMELESS-DISCRETE** allows them to be used immediately—we say that the later is *stripped off*—using a fancy update. Timelessness is maintained structurally in many cases, such

|                                                                                                        |                                                                                 |                                                                                                                |                                                                                                  |                                                                         |
|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| $\frac{\text{TIMELESS-VS}}{\text{timeless}(P)} \quad \triangleright P \vdash \models_{\varepsilon} P$  |                                                                                 | $\frac{\text{TIMELESS-DISCRETE}}{M \text{ is discrete}} \quad \text{timeless}(\{a : M^{\gamma}\})$             |                                                                                                  | $\frac{\text{TIMELESS-PURE}}{\text{timeless}(\phi)}$                    |
| $\frac{\text{PERSISTENT-DUP}}{\text{persistent}(P)} \quad P \Leftrightarrow P * P$                     | $\frac{\text{PERSISTENT-PERS}}{\text{persistent}(\Box P)}$                      | $\frac{\text{PERSISTENT-PURE}}{\text{persistent}(\phi)}$                                                       | $\frac{\text{PERSISTENT-CORE}}{ a  = a} \quad \text{persistent}(\{a : \gamma\})$                 | $\frac{\text{PERSISTENT-INV}}{\text{persistent}(\Box^{\mathcal{N}} I)}$ |
| $\frac{\text{TIMELESS-SEP}}{\text{timeless}(P)} \quad \text{timeless}(Q) \quad \text{timeless}(P * Q)$ | $\frac{\text{TIMELESS-PERS}}{\text{timeless}(P)} \quad \text{timeless}(\Box P)$ | $\frac{\text{PERSISTENT-SEP}}{\text{persistent}(P)} \quad \text{persistent}(Q) \quad \text{persistent}(P * Q)$ | $\frac{\text{PERSISTENT-LATER}}{\text{persistent}(P)} \quad \text{persistent}(\triangleright P)$ |                                                                         |

FIGURE 6.7: Some properties of timeless propositions and persistent propositions.

as in **TIMELESS-SEP**. Some typical timeless propositions are pure (meta-level) facts (**TIMELESS-PURE**), and ownership of a ghost element whose RA is discrete (**TIMELESS-DISCRETE**), *i.e.*, the equivalence relation is not step-indexed.<sup>12</sup>

**Property 6.10** (Persistent Propositions). Intuitively, *persistent* propositions are those who do not consume resources. As such, they can be freely duplicated and be used many times without being consumed, as in **PERSISTENT-DUP**. For this reason, persistent propositions are often called *knowledge*, in contrast to non-persistent ones, which are generally called *resources*. Persistency is maintained structurally, for example, as in **PERSISTENT-SEP** and **PERSISTENT-LATER**. Naturally,  $\Box P$  is persistent (**PERSISTENT-PERS**). Some typical persistent propositions are pure facts (**PERSISTENT-PURE**), the knowledge of some invariant (**PERSISTENT-INV**), or ghost ownership of some core element (**PERSISTENT-CORE**).

<sup>12</sup>A step-indexing family of equivalence relations is needed for higher-order ghost state.

## 6.9 The Method of Fictional Separation

When instantiating Iris with our target language, apart from the expected WP rules mentioned above, we need to derive WP rules for our language's primitives (*e.g.*, for reads and writes). These derivations, as well as most derivations of rules that we will see in later chapters, all follow the method of *fictional separation*.<sup>13</sup> It is a method to turn the ownership of some monolithic, non-splittable resource  $r$  into separable ones. The splitting of  $r$  is fictional: we construct some RA that mirrors  $r$  and that has a suitable composition to provide the desirable separation structure. In other words, we create a *ghost* copy of  $r$  and we split the copy, while maintaining that the copy is in sync with the original  $r$ . For this, we need the *authoritative* RA.

**Definition 6.11** (The Authoritative Resource Algebra). The authoritative RA,<sup>14</sup> denoted  $\text{AUTH}(M)$ , assumes a *unital* RA  $M$ , and provides two types of elements for some element  $a \in M$ : the *authoritative* element  $\bullet a$ , and the *fragmentary* element  $\circ a$ . The elements satisfy (but are not limited to) the rules given in **Figure 6.8**.

<sup>13</sup>Jensen and Birkedal, “Fictional Separation Logic” [JB12].

<sup>14</sup>Jung et al., “Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning” [Jun+15].

$$\begin{array}{ll}
\forall a. |\circ a| = \circ |a| & (\text{AUTH-FRAG-CORE}) \\
\forall a, b. \circ (a \cdot b) = \circ a \cdot \circ b & (\text{AUTH-FRAG-OP}) \\
\forall a, b. a \preceq b \Rightarrow \circ a \preceq \circ b & (\text{AUTH-FRAG-MONO}) \\
\forall a. \text{valid}(\bullet a) \Leftrightarrow \text{valid}(a) & (\text{AUTH-AUTH-VALID}) \\
\forall a. \text{valid}(\circ a) \Leftrightarrow \text{valid}(a) & (\text{AUTH-FRAG-VALID}) \\
\forall a, b. \neg \text{valid}(\bullet a \cdot \bullet b) & (\text{AUTH-AUTH-OP-VALID}) \\
\forall a, b. \text{valid}(\bullet a \cdot \circ b) \Leftrightarrow b \preceq a \wedge \text{valid}(a) & (\text{AUTH-BOTH-VALID}) \\
\forall a_1, b_1, a_2, b_2. (a_1, b_1) \rightsquigarrow_{\mathcal{L}} (a_2, b_2) \Rightarrow \bullet a_1 \cdot \circ b_1 \rightsquigarrow \bullet a_2 \cdot \circ b_2 & (\text{AUTH-UPDATE})
\end{array}$$

FIGURE 6.8: Several rules for the  $\text{AUTH}(M)$  RA.

That is, fragmentary elements preserve core, composition and thus inclusion of  $M$  (**AUTH-FRAG-CORE**, **AUTH-FRAG-OP**, and **AUTH-FRAG-MONO**); both types of elements preserve validity (**AUTH-AUTH-VALID** and **AUTH-FRAG-VALID**), and the authoritative element is *exclusive* (**AUTH-AUTH-OP-VALID**). Most importantly, a valid composition of the authoritative element of  $a$  and a fragmentary element of  $b$  implies that  $b$  is included in  $a$  (**AUTH-BOTH-VALID**). This is why the RA is named “authoritative”: the authoritative element includes all fragmentary elements.

We then can use the authoritative element as *the* ghost copy for our monolithic resource  $r$ , and the fragments as its splittable counterparts.

**Concept 6.12** (The Method of Fictional Separation). **To fictionally separate a monolithic, non-splittable resource  $r$ :**

1. Design an RA  $M$  that mirrors the original resource  $r$  and that has the desirable separation structure *i.e.*, an appropriate composition.
2. Apply the authoritative RA to  $M$ , *i.e.*,  $\text{AUTH}(M)$ , and keep the ownership of the authoritative part  $\bullet r$  in sync with  $r$ .
3. Use the fragmentary parts  $\circ$  to build local assertions.
4. Derive rules for local assertions that update the splittable fragmentary parts  $\circ$ , in conjunction with  $\bullet r$  and thus with  $r$ , but having  $r$  and  $\bullet r$  *hidden*, typically by using an invariant.

To update the fragmentary elements together with the authoritative one, we use **AUTH-UPDATE**, which says that  $\bullet a_1 \cdot \circ b_1$  can frame-preservingly updated to  $\bullet a_2 \cdot \circ b_2$  if  $(a_1, b_1)$  can be *locally updated* to  $(a_2, b_2)$ .

**Definition 6.13** (Local Updates).

$$(a_1, b_1) \rightsquigarrow_{\mathcal{L}} (a_2, b_2)$$

A pair  $(a_1, b_1)$  can be locally updated to a pair  $(a_2, b_2)$ , if for any (optional) frame  $a_f^?$  completing  $b_1$  to  $a_1$ ,  $a_f^?$  also complete  $b_2$  to  $a_2$ :

$$\begin{aligned}
(a_1, b_1) \rightsquigarrow_{\mathcal{L}} (a_2, b_2) ::= \\
\forall a_f^?. \text{valid}(a_1) \wedge a_1 = b_1 \cdot a_f^? \Rightarrow \text{valid}(a_2) \wedge a_2 = b_2 \cdot a_f^?
\end{aligned}$$

In the case of **AUTH-UPDATE**, this means that the frame  $a_f^?$  is the fragmentary frame of  $\circ b_1$ , and when updating  $\bullet a_1$  together with  $\circ b_1$ , we need to maintain that the update respects  $a_f^?$ .

Now, Iris allows use to fictionally separate the physical machine state of our target language through the *physical state interpretation*.

## 6.10 The Physical State Interpretation

In fact, Iris requires us—the instantiator—to provide the state interpretation predicate  $S : \text{State} \rightarrow \text{iProp}$  where  $\text{State}$  is the type of the global physical state. This predicate is used in the definition of Iris weakest pre-conditions.

**Definition 6.14** (Iris WP, simplified).

$$\begin{aligned} \text{wp}_{\mathcal{E}}^S e \{ \Phi \} ::= & \\ & e \in \text{Val} \wedge \models_{\mathcal{E}} \Phi(e) \\ \vee & \left( e \notin \text{Val} \wedge \forall \sigma. S(\sigma) \multimap^* \right. \\ & \quad \mathcal{E} \models^{\varnothing} \left( \text{red}(e, \sigma) * \forall e', \sigma', e_f. (e, \sigma) \rightarrow_t (e', \sigma', e_f) \multimap^* \right. \\ & \quad \quad \left. \triangleright \models^{\mathcal{E}} (S(\sigma') * \text{wp}_{\mathcal{E}}^S e' \{ \Phi \} * \text{wp}_{\top}^S e_f \{ v. \text{True} \}) \right) \left. \right) \end{aligned}$$

Weakest pre-condition is defined as a recursive iProp predicate (guarded by a later modality), with two cases. In case the expression  $e$  is already a value, then the post-condition  $\Phi(e)$  must hold. Otherwise, assuming  $S(\sigma)$  for the current global physical state  $\sigma$  *before* a step, (i)  $e$  must be safe to take a 1-thread reduction step ( $\rightarrow_t$ ) in  $\sigma$ , i.e.,  $e$  is reducible in  $\sigma$  ( $\text{red}(e, \sigma)$ ), and (ii) for any resulting configuration  $(e', \sigma', e_f)$  of such a 1-thread reduction step, the state interpretation  $S(\sigma')$  for the physical state  $\sigma'$  *after* the step must hold, and the weakest pre-conditions hold recursively for  $e'$  and the forked expression  $e_f$ . The fancy updates enable ghost updates and invariant accesses around a single reduction step.

From this definition, we see that the proposition  $S(\sigma)$  is needed to perform a step and must be re-established afterwards, where  $\sigma \in \text{State}$  is the current physical state of the machine. As such, we can use  $S$ , whose definition is up to us (the instantiator) to pick, to fictionally separate the physical state  $\sigma$ . Specifically, we will use  $S$  to keep the current state  $\sigma$  in sync with our authoritative ghost copy  $\bullet \sigma$ , and give out the fragmentary element  $\circ \sigma$ —which can be separated into smaller elements—to define our local assertions. Conveniently through  $S$ , the WP definition not only keeps the physical state and the ghost copy in sync for us, but also hides them away, so our only remaining tasks are to define a suitable RA that enable the desirable separation on  $\text{State}$  and to prove our primitive WP rules (which will need to update the physical state, thus the authoritative ghost copy and the corresponding ghost fragments).

## 6.11 An Instantiation Example for Simple Heaps

To make it more concrete, we briefly consider an instantiation example for an SC language whose physical state is simple heap—a map from locations to values, i.e.,  $\text{State} ::= \text{Loc} \xrightarrow{\text{fin}} \text{Val}$ .<sup>15</sup> Let us call this language

<sup>15</sup>This example is rephrased from [Jun+18b] and [Kai+17].

$\lambda_{\text{HEAP}}$ . What we want is to derive the *small-footprint* rules for reads and writes, using the local *points-to* assertion.

$$\begin{array}{ll} \text{HEAP-READ} & \text{HEAP-WRITE} \\ \{\ell \mapsto v\} * \ell \{w. w = v * \ell \mapsto v\} \top & \{\ell \mapsto v\} \ell := v' \{(). \ell \mapsto v'\} \top \end{array}$$

To do so, we pick a suitable RA to split a heap  $\sigma$  into multiple *singleton* heaps of the form  $[\ell \leftarrow v]$ , which can then be used to define  $\ell \mapsto v$ . This RA is called **SHEAP**, whose type is exactly *State* and composition is union of finite maps, but composition is only valid between disjoint maps. That is,  $\text{valid}(\sigma \cdot \sigma') \Leftrightarrow \text{dom}(\sigma) \cap \text{dom}(\sigma') = \emptyset$ . The state interpretation and the points-to assertion are then defined as:

$$\begin{aligned} S(\sigma) &::= [\bullet \sigma : \text{AUTH}(\text{SHEAP})]_{\gamma_{\text{HEAP}}}^{\gamma_{\text{HEAP}}} \\ \ell \mapsto v &::= [\circ [\ell \leftarrow v] : \text{AUTH}(\text{SHEAP})]_{\gamma_{\text{HEAP}}}^{\gamma_{\text{HEAP}}} \end{aligned}$$

That is, the state interpretation  $S(\sigma)$  is the ghost ownership of the authoritative heap  $\bullet \sigma$  at the ghost location  $\gamma_{\text{HEAP}}$ , and the points-to assertion  $\ell \mapsto v$  is the ghost ownership of the fragmentary singleton heap  $\circ [\ell \leftarrow v]$  at the same ghost location  $\gamma_{\text{HEAP}}$ . The ghost location  $\gamma_{\text{HEAP}}$  is allocated before the program runs (a proof that indeed needs to be done in Adequacy—see §6.5), and is needed to establish the agreement between  $\ell \mapsto v$  and the current physical state  $\sigma$ , indirectly through  $S$ . To see this in action, let us look at the proofs of **HEAP-READ** and **HEAP-WRITE**. Both proofs first proceed by unfolding the definitions of Hoare triples (§6.4, Definition 6.7), and WP (Definition 6.14).

*Proof sketch of **HEAP-READ**.* After introducing the assumptions and clearing the fancy update (using **FUPD-INTRO-MASK** and **FUPD-MONO**),<sup>16</sup> we arrive at the following goal.

<sup>16</sup>see §6.7

| Context:                                                                                                                                                                                                                                                                                                                                      | Goal:                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| $\ell \mapsto v \ (= \ [\circ [\ell \leftarrow v]]_{\gamma_{\text{HEAP}}}^{\gamma_{\text{HEAP}}})$<br>$S(\sigma) \ (= \ [\bullet \sigma]_{\gamma_{\text{HEAP}}}^{\gamma_{\text{HEAP}}})$                                                                                                                                                      | $\text{red}(*\ell, \sigma) \wedge \forall e', \sigma', e_f. \dots$                                                                        |
| We first need to show that $e$ is reducible on $\sigma$ .                                                                                                                                                                                                                                                                                     |                                                                                                                                           |
| With <b>GHOST-OP</b> and <b>GHOST-VALID</b> (see §6.2), from our assumptions, we have $\text{valid}(\bullet \sigma \cdot \circ [\ell \leftarrow v])$ , and then by <b>AUTH-BOTH-VALID</b> , we have $[\ell \leftarrow v] \preceq \sigma$ and $\text{valid}(\sigma)$ , so $\sigma = [\ell \leftarrow v] \uplus \sigma_f$ for some $\sigma_f$ . |                                                                                                                                           |
| By the definition of <b>SHEAP</b> 's composition, we then know $\sigma(\ell) = v$ .                                                                                                                                                                                                                                                           |                                                                                                                                           |
| Since $\ell \in \text{dom}(\sigma)$ , we can show $\text{red}(*\ell, \sigma)$ . Our remaining goal is                                                                                                                                                                                                                                         |                                                                                                                                           |
| $\ell \mapsto v * S(\sigma)$                                                                                                                                                                                                                                                                                                                  | $\forall e', \sigma', e_f. (*\ell, \sigma) \rightarrow_t (e', \sigma', e_f) \multimap^{\circ} \triangleright^{\circ} \vdash^{\top} \dots$ |
| Looking at the operational semantics of $(*\ell, \sigma) \rightarrow_t (e', \sigma', e_f)$ , we learn that $e' = \sigma(\ell) \wedge \sigma' = \sigma \wedge e_f = \perp$ , so our goal is                                                                                                                                                    |                                                                                                                                           |
| $\ell \mapsto v * S(\sigma)$                                                                                                                                                                                                                                                                                                                  | $\triangleright^{\circ} \vdash^{\top} S(\sigma) * \text{wp}_{\top}^S \sigma(\ell) \{w. w = v * \ell \mapsto v\}$                          |
| After clearing the later and fancy update modalities and simplifying (using <b>LATER-INTRO</b> and <b>FUPD-MONO</b> again),                                                                                                                                                                                                                   |                                                                                                                                           |
| $\ell \mapsto v * S(\sigma)$                                                                                                                                                                                                                                                                                                                  | $S(\sigma) * \sigma(\ell) = v * \ell \mapsto v$                                                                                           |

Since we already know  $\sigma(\ell) = v$ , we are done.  $\square$

Note that to apply **GHOST-OP** in the above proof, it is important that  $S(\sigma)$  and  $\ell \mapsto v$  use the same ghost location  $\gamma_{\text{HEAP}}$ .

*Proof sketch of **HEAP-WRITE**.* Similar to the proof of **HEAP-WRITE**, by owning  $\ell \mapsto v$  we know that  $\sigma(\ell) = v$  where  $\sigma$  is the current physical state, so we can prove  $\text{red}(\ell := v', \sigma)$ . We then introduce all assumptions and clear the fancy update and the later from the goal. However, we additionally need to use **FUPD-TRANS** to keep a fancy update  $\models_{\top}$ : we need to update our ghost copy so that it is in sync with the new physical state after the step. Our goal then looks as follow.

| Context:                                                                                           | Goal:                                                                               |
|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| $\ell \mapsto v * S(\sigma)$                                                                       |                                                                                     |
| $(\ell := v', \sigma) \rightarrow_{\text{t}} (e', \sigma', e_{\text{f}})$                          | $\models_{\top} S(\sigma') * \text{wp}_{\top}^S e' \{(). \ell \mapsto v'\} * \dots$ |
| From the operational semantics of $\rightarrow_{\text{t}}$ , we learn that                         |                                                                                     |
| $e' = () \wedge \sigma' = \sigma[\ell \leftarrow v'] \wedge e_{\text{f}} = \perp$ , so our goal is |                                                                                     |
| $\ell \mapsto v * S(\sigma)$                                                                       | $\models_{\top} S(\sigma') * \text{wp}_{\top}^S () \{(). \ell \mapsto v'\}$         |
| After simplifying, we arrive at                                                                    |                                                                                     |
| $\ell \mapsto v * S(\sigma)$                                                                       | $\models_{\top} S(\sigma') * \ell \mapsto v'$                                       |

This goal pins down to an update of our ghost copy to sync with the new state  $\sigma'$ . Indeed, after applying **GHOST-OP**, then **FUPD-BUPD**, and then **GHOST-UPDATE**, we have to prove the following frame-preserving update.

$$\bullet \sigma \cdot \circ [\ell \leftarrow v] \rightsquigarrow \bullet \sigma' \cdot \circ [\ell \leftarrow v']$$

Applying **AUTH-UPDATE**, we have to show

$$(\sigma, [\ell \leftarrow v]) \rightsquigarrow_{\Sigma} (\sigma', [\ell \leftarrow v'])$$

This is easy. We know that the frame  $\sigma_{\text{f}}$  that completes  $\sigma$  with  $[\ell \leftarrow v]$  is disjoint from  $[\ell \leftarrow v]$ :  $\sigma = \sigma_{\text{f}} \uplus [\ell \leftarrow v]$ . Thus we can show

$$(\sigma_{\text{f}} \uplus [\ell \leftarrow v], [\ell \leftarrow v]) \rightsquigarrow_{\Sigma} (\sigma_{\text{f}} \uplus [\ell \leftarrow v'], [\ell \leftarrow v'])$$

easily by looking at the definition of local updates (**Definition 6.13**). We are done because  $\sigma' = \sigma[\ell \leftarrow v'] = \sigma_{\text{f}} \uplus [\ell \leftarrow v']$ .  $\square$





# 7

## *A Base Logic for RMC in Iris*

---

In this chapter, we demonstrate how to instantiate the Iris framework with our  $\lambda_{\text{Rust}} + \text{ORC11}$  semantics (defined in [Chapter 4](#)) to achieve a “vanilla” relaxed-memory CSL. Even though vanilla, this so-called *base logic* for our target language is already quite expressive, because it is derived from the Iris program logic: it is a higher-order CSL with higher-order ghost state, impredicative invariants, and admits the WP and Hoare rules listed in [Chapter 6](#). In this chapter, we establish more WP and Hoare rules for our language’s relaxed memory primitives. These rules form the bottom-most basis of our logic, from which all other constructs of the higher-level iRC11 logic will be derived.

ROADMAP. [§6.11](#) already gives an instantiation example for a language with simple heaps, but it worths articulating the process, more specifically for our  $\lambda_{\text{Rust}} + \text{ORC11}$  semantics:

1. We instantiate Iris with the 1-thread reductions ([Definition 4.12](#)) of  $\lambda_{\text{Rust}} + \text{ORC11}$ . Since we are in a relaxed memory semantics with views, the resulting base logic will have to expose them. [§7.1](#) discusses how views generally show up in our base-logic WP and Hoare triple rules.
2. In contrast to an SC logic whose main local assertion is points-to, we need new local assertions with appropriate separation structure to handle relaxed effects and data races. [§7.2](#) discusses the design (interfaces) of those new assertions.
3. [§7.3](#) presents the desired small-footprint, primitive rules that use the new local assertions.
4. [§7.4](#) presents the resource algebras needed to give a model for the new local assertions and proves their properties.
5. Finally, [§7.5](#) defines the state interpretation  $S$  for our language, and [§7.6](#) provides proofs of some primitive rules as well as adequacy.

### 7.1 Thread-local Configurations as Expressions

The Iris framework requires as input a language with a reduction relation  $(e, \sigma) \rightarrow_t (e', \sigma', e_f)$ —which we call a 1-thread reduction—where  $e$  is the expression of the thread being evaluated and  $\sigma$  is the physical machine

$$\begin{array}{l}
\text{BL-WP-BIND} \\
\text{wp}_{\mathcal{E}}(e, \mathcal{V}) \{(v, \mathcal{V}_v). \text{wp}_{\mathcal{E}}(K[v], \mathcal{V}_v) \{\Phi\}\} \vdash \text{wp}_{\mathcal{E}}(K[e], \mathcal{V}) \{\Phi\} \\
\\
\text{BL-WP-FRAME} \\
P * \text{wp}_{\mathcal{E}}(e, \mathcal{V}) \{\Phi\} \vdash \text{wp}_{\mathcal{E}}(e, \mathcal{V}) \{(v, \mathcal{V}_v). P * \Phi(v, \mathcal{V}_v)\} \\
\\
\text{BL-WP-VAL} \qquad \qquad \qquad \text{BL-WP-LAM} \\
\Phi(v, \mathcal{V}) \vdash \text{wp}_{\mathcal{E}}(v, \mathcal{V}) \{\Phi\} \qquad \qquad \qquad \triangleright \text{wp}_{\mathcal{E}}(e[v/x], \mathcal{V}) \{\Phi\} \vdash \text{wp}_{\mathcal{E}}((\lambda x. e)v, \mathcal{V}) \{\Phi\} \\
\\
\text{BL-WP-FORK} \\
\triangleright \Phi(\text{fork}, \mathcal{V}) * \triangleright \text{wp}_{\top}(e, \text{ForkView}(\mathcal{V})) \{\_ \text{True}\} \vdash \text{wp}_{\mathcal{E}}(\text{fork } \{e\}, \mathcal{V}) \{\Phi\} \\
\\
\text{BL-WP-PLUS} \qquad \qquad \qquad \text{BL-WP-IF} \\
\triangleright \text{wp}_{\mathcal{E}}(n +_{\mathbb{Z}} m, \mathcal{V}) \{\Phi\} \vdash \text{wp}_{\mathcal{E}}(n + m, \mathcal{V}) \{\Phi\} \qquad \qquad \qquad \triangleright \text{wp}_{\mathcal{E}}((b) ? e_1 : e_2, \mathcal{V}) \{\Phi\} \vdash \text{wp}_{\mathcal{E}}(\text{if } b \text{ then } e_1 \text{ else } e_2, \mathcal{V}) \{\Phi\}
\end{array}$$

FIGURE 7.1: Pure primitive WPs in the RMC base logic

<sup>1</sup>In fact, Iris supports forking multiple threads, so  $e_f$  can be a list of expressions.

state. The resulting  $e'$  is the thread's new expression, and  $\sigma'$  the new physical state, and a new thread may be forked with the expression  $e_f$ .<sup>1</sup> Satisfying this requirement of a 1-thread reduction is rather straightforward for traditional SC languages, but for our RMC language, we need a little bit more care: the execution of a thread needs not only the globally-shared physical state, but also some thread-local state—specifically in this case, a thread-view. This can be seen clearly in our 1-thread reduction relation  $\varsigma \mid (e, \mathcal{V}) \xrightarrow{\varepsilon^?, (e_f, \mathcal{V}_f)^?}_{\text{t}} \varsigma' \mid (e', \mathcal{V}')$  (Definition 4.12).

Aside from some notation mismatches, we need to fit our 1-thread reduction relation to what Iris requires. The solution is simple: we instantiate what Iris considers “expressions” with pairs of expressions and thread-views  $(e, \mathcal{V})$ —our thread-local configurations. This is only a change of perspective: what Iris really requires is a 1-thread relation that describes how a thread-local configuration reduces together with the global state, but Iris has often been instantiated with languages where the thread-local configuration is just an expression. This perspective applies to languages with *thread-local state*, which in our case is a thread-view, but in other languages can be, for example, a call stack or an abstraction for some hardware component.

The effects of picking expression-thread-view pairs as “Iris expressions” are most visible in weakest pre-conditions and Hoare triples. They will generally have the following forms;

$$\begin{array}{l}
\text{wp}_{\mathcal{E}}(e, \mathcal{V}) \{(v, \mathcal{V}_v). Q\} \\
\{P\}(e, \mathcal{V}) \{(v, \mathcal{V}_v). Q\}_{\mathcal{E}}
\end{array}$$

That is, a WP or Hoare triple encodes the behaviors of a thread-local configuration  $(e, \mathcal{V})$  where  $\mathcal{V}$  is the thread-view that  $e$  starts executing with. Therefore it may be necessary that  $\mathcal{V}$  satisfies certain properties that can be stated in the precondition  $P$ . The configuration, if terminates, will reduce to a configuration  $(v, \mathcal{V}_v)$  where  $\mathcal{V}_v$  is the thread-view after the execution. Properties of  $\mathcal{V}_v$ , like those of  $v$ , can be stated in the post-condition, which now has the type  $\text{Expr} \times \text{ThreadView} \rightarrow \text{iProp}$ .

Figure 7.1 presents a few *pure* WP rules that do not involve memory.

They are the expected WP rules from §6.6, but adapted with an arbitrary thread-view that mostly stays unchanged. Most notably, **WP-BIND** propagates the thread-view after the expression has finished to the evaluation context, and **BL-WP-FORK** picks the correct thread-view for the forked thread. Rules for binary operations are in the same form as **BL-WP-PLUS**, and the rule for **case** is similar to **BL-WP-IF**, and thus are elided.

However, for ORC11 memory operations, we know that expressions cannot run with arbitrary thread-views, as that may cause data races. The safety properties of thread-views are therefore unavoidable in the logic, but we can keep them manageable in form of new local assertions.

## 7.2 Basic Local Assertions for View-based RMC

In traditional separation logics, the points-to assertion is essential to achieve thread-modular reasoning: when interacting with the shared global state, it is sufficient to just have a points-to  $\ell \mapsto v$  to access the location  $\ell$ , keeping the rest of the global state out of the picture and in the “frame”. Thanks to the frame rule, traditional separation logics enjoy the simple, thread-modular, small-footprint rules like **HEAP-READ** and **HEAP-WRITE** (§6.11).

We have the same goal for our RMC base logic: we want to achieve small-footprint rules that only require minimal ownership of bits of the global state for the operations in question, and let the frame rule do its job. Unfortunately, the bits of the global state needed for our RMC memory accesses are rather involved: we need (i) the memory  $h$  of the location  $\ell$  being accessed, (ii) the thread-view  $\mathcal{V}$  of the executing thread, and (iii) the global race detector view for  $\ell$  ( $\mathcal{N}(\ell)$ ). Most importantly, the safety and the result of an access depends on the relations between the thread-view  $\mathcal{V}$  and the location’s memory  $h$  and the global race detector view  $\mathcal{N}$ . We therefore need more assertions to make those relations explicit. We present our choice of local assertions below. In §7.4, we will present the RAs needed to fictionally separate the machine state and define these local assertions within the logic.

**Definition 7.1** (Local Assertions for the Base Logic). These assertions concern knowledge or resources over the executing thread’s thread-view, the memory of the location being accessed, the race detector state, and their relations. All assertions are in  $\text{iProp}$ .

- The *seen thread-view* observation  $\text{Seen}(\mathcal{V})$  is a persistent knowledge that some thread’s thread-view  $\mathcal{V}$  is closed in the global memory.<sup>2</sup> This assertion is needed to guarantee that a memory access is grounded in the global memory.
- The *history ownership* assertion  $\text{Hist}_q(\ell, h)$  is a fractional ownership<sup>3</sup> of the write messages  $h \in \text{History}$  of the location  $\ell$  in the global memory, where  $\text{History} ::= \text{Time} \xrightarrow{\text{fin}} \left\{ \text{val} : \text{Val}, \text{view} : \text{View}^? \right\}$ . The fraction  $q \in (0, 1]$  denotes shared or full ownership of  $\ell$ ’s history. The allocated assertion  $\text{Local}_A(\ell, h, V)$  says that the simple view  $V$

<sup>2</sup>see Property 3.12, §3.3

<sup>3</sup>Boyland, “Checking interference with fractional permissions” [Boy03].

has observed the knowledge that  $\ell$ 's history  $h$  has been allocated. Both assertions are needed to perform any access to  $\ell$ .

- The *non-atomic read* assertion  $\text{Read}_q^{\text{na}}(\ell, \alpha)$  is the fractional ownership of a *subset*  $\alpha$  of  $\ell$ 's non-atomic reads in the global race detector state. That is,  $\alpha \subseteq \mathcal{N}(\ell).\text{nr}$  if  $\mathcal{N}$  is the global race detector state. A related persistent knowledge is the non-atomic read observation  $\text{Local}_R^{\text{na}}(\ell, \alpha, V)$ , which asserts that the simple view  $V$  has observed a subset  $\alpha$  of  $\ell$ 's non-atomic reads. Both assertions are needed to perform race-free non-atomic reads on  $\ell$ .
- The *atomic read* assertion  $\text{Read}_q^{\sqsupset \text{r1x}}(\ell, \alpha)$  is the fractional ownership of a *subset*  $\alpha$  of  $\ell$ 's atomic reads in the global race detector state. That is,  $\alpha \subseteq \mathcal{N}(\ell).\text{ar}$ . The persistent knowledge  $\text{Local}_R^{\sqsupset \text{r1x}}(\ell, \alpha, V)$  asserts that  $V$  has observed a subset  $\alpha$  of  $\ell$ 's atomic reads. Both assertions are needed to perform race-free atomic reads on  $\ell$ .
- The *atomic write* assertion  $\text{Write}_q^{\sqsupset \text{r1x}}(\ell, \alpha)$  is the fractional ownership of a *subset*  $\alpha$  of  $\ell$ 's atomic writes in the global race detector state. That is,  $\alpha \subseteq \mathcal{N}(\ell).\text{aw}$ . The persistent knowledge  $\text{Local}_W^{\sqsupset \text{r1x}}(\ell, \alpha, V)$  asserts that  $V$  has observed a subset  $\alpha$  of  $\ell$ 's atomic writes. Both assertions are needed to perform race-free atomic writes on  $\ell$ .
- Last but not least, the *block ownership* assertion  $\dagger_q^n \ell$  is inherited from RustBelt. This ownership is created at allocation of a block whose base location is  $\ell$ , and is only needed at deallocation of that block. The ownership guarantees that the whole block is deallocated together, *i.e.*, any thread holding a fraction of the block knows that the constituent locations are still alive. The assertion simply tracks the location  $\ell$  and the size  $n \in \mathbb{N}$  of the block.

These assertions satisfy several properties, given in [Figure 7.2](#).

**Property 7.2** (Seen Thread-view Observations). The seen thread-view observation is timeless and persistent ([BL-SEEN-TIMELESS](#) and [BL-SEEN-PERS](#)).<sup>4</sup> As  $\text{Seen}(\mathcal{V})$  is only a snapshot of some thread's thread-view at some point, it can be joined with others ([BL-SEEN-JOIN](#)), or can be forked to get old snapshots ([BL-SEEN-DOWNCLOSED](#)). With  $\text{Seen}(\mathcal{V})$ , we know that the thread-view  $\mathcal{V}$  is closed in the global memory  $\mathcal{M}$ , but we do not have a rule for this property here. We will see how the property is established by the state interpretation in [§7.6](#).

**Property 7.3** (History Ownership). The assertion  $\text{Hist}_q(\ell, h)$  is timeless ([BL-HIST-TIMELESS](#)) and fractional ([BL-HIST-FRAC-VALID](#) and [BL-HIST-FRAC](#)). Owning a fraction of the assertion is sufficient to know the history of  $\ell$ , as implied by [BL-HIST-AGREE](#). A change to the history requires the full fraction, written as  $\text{Hist}(\ell, h)$  without the fraction  $q = 1$ , which is exclusive, as in [BL-HIST-EXCL](#). ([BL-HIST-EXCL](#) is derivable from [BL-HIST-FRAC-VALID](#) and [BL-HIST-FRAC](#).)

[BL-HIST-DROP-SINGLETON](#) allows us to truncate the current history to just a singleton of the latest write. This is a convenient abstraction for  $\ell$ 's

<sup>4</sup>see [§6.8](#)

|                                                                                                                                                                                                               |                                                                                                                                                                             |                                                                                                                                                                                                          |                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| BL-SEEN-TIMELESS<br>$\text{timeless}(\text{Seen}(\mathcal{V}))$                                                                                                                                               | BL-SEEN-PERS<br>$\text{persistent}(\text{Seen}(\mathcal{V}))$                                                                                                               | BL-SEEN-JOIN<br>$\text{Seen}(\mathcal{V}) * \text{Seen}(\mathcal{V}') \dashv\vdash \text{Seen}(\mathcal{V} \sqcup \mathcal{V}')$                                                                         | BL-SEEN-DOWNCLOSED<br>$\frac{\mathcal{V} \sqsubseteq \mathcal{V}'}{\text{Seen}(\mathcal{V}') \vdash \text{Seen}(\mathcal{V})}$ |
| BL-HIST-TIMELESS<br>$\text{timeless}(\text{Hist}_q(\ell, h))$                                                                                                                                                 | BL-HIST-FRAC-VALID<br>$\text{Hist}_q(\ell, h) \vdash q \in (0, 1]$                                                                                                          | BL-HIST-FRAC<br>$\text{Hist}_q(\ell, h) * \text{Hist}_{q'}(\ell, h) \dashv\vdash \text{Hist}_{q+q'}(\ell, h)$                                                                                            |                                                                                                                                |
| BL-HIST-AGREE<br>$\text{Hist}_q(\ell, h) * \text{Hist}_{q'}(\ell, h') \vdash h = h'$                                                                                                                          | BL-HIST-EXCL<br>$\text{Hist}(\ell, h) * \text{Hist}(\ell, h') \vdash \text{False}$                                                                                          | BL-HIST-DROP-SINGLETON<br>$\frac{h(t) = (v, V) \quad t = \max(\text{dom}(h))}{\text{Hist}(\ell, h) \Rightarrow_{\mathcal{E}} \text{Hist}(\ell, [t \leftarrow (v, V)])}$                                  |                                                                                                                                |
| BL-NAR-TIMELESS<br>$\text{timeless}(\text{Read}_q^{\text{na}}(\ell, \alpha))$                                                                                                                                 | BL-NAR-FRAC-VALID<br>$\text{Read}_q^{\text{na}}(\ell, \alpha) \vdash q \in (0, 1]$                                                                                          | BL-NAR-JOIN<br>$\text{Read}_q^{\text{na}}(\ell, \alpha) * \text{Read}_{q'}^{\text{na}}(\ell, \alpha') \vdash \text{Read}_{q+q'}^{\text{na}}(\ell, \alpha \cup \alpha')$                                  |                                                                                                                                |
| BL-ATR-TIMELESS<br>$\text{timeless}(\text{Read}_q^{\sqsupset \text{rlx}}(\ell, \alpha))$                                                                                                                      | BL-ATR-FRAC-VALID<br>$\text{Read}_q^{\sqsupset \text{rlx}}(\ell, \alpha) \vdash q \in (0, 1]$                                                                               | BL-ATR-JOIN<br>$\text{Read}_q^{\sqsupset \text{rlx}}(\ell, \alpha) * \text{Read}_{q'}^{\sqsupset \text{rlx}}(\ell, \alpha') \vdash \text{Read}_{q+q'}^{\sqsupset \text{rlx}}(\ell, \alpha \cup \alpha')$ |                                                                                                                                |
| BL-ATW-TIMELESS<br>$\text{timeless}(\text{Write}_q^{\sqsupset \text{rlx}}(\ell, \alpha))$                                                                                                                     | BL-ATW-FRAC-VALID<br>$\text{Write}_q^{\sqsupset \text{rlx}}(\ell, \alpha) \vdash q \in (0, 1]$                                                                              | BL-ATW-FRAC<br>$\text{Write}_q^{\sqsupset \text{rlx}}(\ell, \alpha) * \text{Write}_{q'}^{\sqsupset \text{rlx}}(\ell, \alpha) \dashv\vdash \text{Write}_{q+q'}^{\sqsupset \text{rlx}}(\ell, \alpha)$      |                                                                                                                                |
| BL-ATW-AGREE<br>$\text{Write}_q^{\sqsupset \text{rlx}}(\ell, \alpha) * \text{Write}_{q'}^{\sqsupset \text{rlx}}(\ell, \alpha') \vdash \alpha = \alpha'$                                                       | BL-NAL-JOIN<br>$\text{Local}_R^{\text{na}}(\ell, \alpha, V) * \text{Local}_R^{\text{na}}(\ell, \alpha', V) \vdash \text{Local}_R^{\text{na}}(\ell, \alpha \cup \alpha', V)$ |                                                                                                                                                                                                          |                                                                                                                                |
| BL-ATRL-JOIN<br>$\text{Local}_R^{\sqsupset \text{rlx}}(\ell, \alpha, V) * \text{Local}_R^{\sqsupset \text{rlx}}(\ell, \alpha', V) \vdash \text{Local}_R^{\sqsupset \text{rlx}}(\ell, \alpha \cup \alpha', V)$ |                                                                                                                                                                             |                                                                                                                                                                                                          |                                                                                                                                |
| BL-ALLOC-MONO<br>$\frac{V \sqsubseteq V'}{\text{Local}_A(\ell, h, V) \vdash \text{Local}_A(\ell, h, V')}$                                                                                                     |                                                                                                                                                                             | BL-NAL-MONO<br>$\frac{V \sqsubseteq V'}{\text{Local}_R^{\text{na}}(\ell, \alpha, V) \vdash \text{Local}_R^{\text{na}}(\ell, \alpha, V')}$                                                                |                                                                                                                                |
| BL-ATRL-MONO<br>$\frac{V \sqsubseteq V'}{\text{Local}_R^{\sqsupset \text{rlx}}(\ell, \alpha, V) \vdash \text{Local}_R^{\sqsupset \text{rlx}}(\ell, \alpha, V')}$                                              |                                                                                                                                                                             | BL-ATWL-MONO<br>$\frac{V \sqsubseteq V'}{\text{Local}_W^{\sqsupset \text{rlx}}(\ell, \alpha, V) \vdash \text{Local}_W^{\sqsupset \text{rlx}}(\ell, \alpha, V')}$                                         |                                                                                                                                |
| BL-BLOCK-TIMELESS<br>$\text{timeless}(\dagger_q^n \ell)$                                                                                                                                                      | BL-BLOCK-FRAC-VALID<br>$\dagger_q^n \ell \vdash q \in (0, 1]$                                                                                                               | BL-BLOCK-JOIN<br>$\dagger_q^n \ell * \dagger_{q'}^{n'} (\ell + n) \dashv\vdash \dagger_{q+q'}^{n+n'} \ell$                                                                                               |                                                                                                                                |

FIGURE 7.2: Main properties of the base logic's local assertions

physical write events: while we need to maintain a set of write events (instead of a single value like in SC) because they may be still visible to some threads, once we know that certain writes are no longer visible, we can simply forget about them. In particular, if one can perform a race-free non-atomic write, all previous writes must be unreachable and should be forgotten, because it would be racy to read them then. Consequently, unlike the physical memory that only grows with more write messages, the history  $h$  in  $\text{Hist}_q(\ell, h)$  is not monotone—it grows during a period of atomic accesses, but will shrink back to a singleton with a non-atomic write. In later chapters, we will use **BL-HIST-DROP-SINGLETON** to switch between non-atomic and atomic access modes.

**Property 7.4** (Race Detector Ownership). The ownership assertions for parts of the race detector state are also timeless and fractional. Like

history ownership, fractions of the atomic write assertion  $\text{Write}_q^{\text{rlx}}(\ell, \alpha)$  maintain agreement on  $\ell$ 's set of atomic writes in the race detector (**BL-ATW-FRAC**), and the full fraction is required to update the set. A fraction  $q$  of the read assertions  $\text{Read}_q^{\text{na}}(\ell, \alpha)$  or  $\text{Read}_q^{\text{rlx}}(\ell, \alpha)$  on the other hand does not maintain agreement. Instead, a fraction only maintains that the set  $\alpha$  is a subset of the race detector's sets for non-atomic and atomic reads, respectively. This difference is due to the fact that, while writes maintain a total order ( $\text{mo}_\ell$ ) and thus must be updated with the full fraction, we do not enforce an order among concurrent reads, so each thread needs only some fraction of a read assertion to independently track its own reads, and sets of reads can be joined together using **BL-NAR-JOIN** or **BL-ATR-JOIN**.

**Property 7.5** (Local Observations). The local observations (for allocation, non-atomic and atomic reads, and atomic writes) are pure facts, and thus are timeless and persistent. In fact, their definitions are as follow.

$$\begin{aligned}\text{Local}_A(\ell, h, V) &::= \exists t \in \text{dom}(h). t \sqsubseteq V(\ell).w \\ \text{Local}_W^{\text{rlx}}(\ell, \alpha, V) &::= \alpha \sqsubseteq V(\ell).aw \\ \text{Local}_R^{\text{na}}(\ell, \alpha, V) &::= \alpha \sqsubseteq V(\ell).nr \\ \text{Local}_R^{\text{rlx}}(\ell, \alpha, V) &::= \alpha \sqsubseteq V(\ell).ar\end{aligned}$$

More importantly, they are *view-monotone*, i.e., if one holds on a smaller view, it also holds on a bigger view (e.g., see **BL-ALLOC-MONO**). View monotonicity is an important property that we will rely on heavily (see Chapter 8).

The local observations for reads can be joined together using **BL-NAL-JOIN** and **BL-ATRL-JOIN**.

**Property 7.6** (Block Ownership). The block ownership assertion is also timeless and fractional. **BL-BLOCK-JOIN** allows splitting and joining not just with the fractions, but also with the offsets. As such, for each location in a block one can own its bit of the block without needing to know the block size, and is guaranteed that the block is still alive.

### 7.3 Primitive Memory Rules

We now see how the local assertions are meant to be used in our primitive memory rules, given in Figures 7.3 to 7.6. Recall that in our base logic, the executing “expression” is a thread-local configuration of the actual expression and the executing thread's thread-view. The rules for allocation and deallocation are similar to that of non-atomic writes (**BL-HOARE-WRITE-NA**), but with the block ownership assertions. For the sake of simplicity, we will present them in a cleaner form in Chapter 9 (see **NA-ALLOC** and **NA-DEALLOC**).

#### 7.3.1 Rules for Fences

Figure 7.3 presents the simplest memory-related primitive rules, for release and acquire fences. Both **BL-HOARE-REL-FENCE** and **BL-HOARE-ACQ-FENCE** requires  $\text{Seen}(\mathcal{V})$  as the pre-condition for a fence running with the

$$\begin{array}{l}
\text{BL-HOARE-REL-FENCE} \\
\{ \text{Seen}(\mathcal{V}) \} (\text{fence}_{\text{rel}}, \mathcal{V}) \{ (\text{poison}, \mathcal{V}'). \mathcal{V}' \sqsubseteq \mathcal{V} * \text{Seen}(\mathcal{V}') * \mathcal{V}'.\text{frel} = \mathcal{V}'.\text{cur} \}_{\mathcal{E}} \\
\\
\text{BL-HOARE-ACQ-FENCE} \\
\{ \text{Seen}(\mathcal{V}) \} (\text{fence}_{\text{acq}}, \mathcal{V}) \{ (\text{poison}, \mathcal{V}'). \mathcal{V}' \sqsubseteq \mathcal{V} * \text{Seen}(\mathcal{V}') * \mathcal{V}'.\text{cur} = \mathcal{V}'.\text{acq} \}_{\mathcal{E}}
\end{array}$$

thread-view  $\mathcal{V}$ . Their post-conditions say that the fence instructions will return poison  $\text{poison}$  with a new thread-view  $\mathcal{V}' \sqsubseteq \mathcal{V}$ .<sup>5</sup> The effects of the fences are approximated by the properties of  $\mathcal{V}'$ . In case of an acquire fence, the current component of  $\mathcal{V}'$  is updated to include its acquire component, exactly reflecting **OM-ACQ-FENCE** (§3.3). In case of a release fence, the release-fence component of  $\mathcal{V}'$  is updated to include its current component. This only approximates **OM-REL-FENCE** (also §3.3) because it hides away (in the relation  $\mathcal{V}' \sqsubseteq \mathcal{V}$ ) the changes to the per-location release views  $\mathcal{V}'.\text{rel}$ . We made this abstraction to keep the rule simple, as we have never needed such detailed information on  $\mathcal{V}'.\text{rel}$ .

FIGURE 7.3: The base logic's primitive Hoare rules for fences

<sup>5</sup>see **OE-FENCE**, §4.2

### 7.3.2 Rules for Reads and Writes

The rules for reads and writes are given in **Figure 7.4** and **Figure 7.5**.

**NON-ATOMIC READS.** To guarantee data-race freedom, **BL-HOARE-READ-NA** requires a pre-condition that implies **DRF-READ-NA** (§3.4). That is, the pre-condition ensures that the executing thread has observed all writes (non-atomic and atomic) to  $\ell$ . The pre-condition thus includes:

- $\text{Seen}(\mathcal{V})$ , like in other memory-related rules, to know the lower bound of the executing thread's thread-view; and
- a fraction  $q$  of the current singleton history  $\text{Hist}_q(\ell, [t \leftarrow (v, V^?)])$  where  $(t, v, V^?)$  is  $\ell$ 's latest write; and
- the knowledge  $\text{Local}_A(\ell, [t \leftarrow (v, V^?)], \mathcal{V}.\text{cur})$  that the current thread-view  $\mathcal{V}.\text{cur}$  has observed not only  $\ell$ 's allocation but also its latest write; and
- a fraction  $q$  of the race detector's atomic writes set  $\text{Write}_q^{\text{rlx}}(\ell, \alpha_w)$  and the knowledge  $\text{Local}_w^{\text{rlx}}(\ell, \alpha_w, \mathcal{V}.\text{cur})$  that the current thread-view has observed all atomic writes ( $\alpha_w$ );<sup>6</sup> and
- a fraction  $q$  of the race detector's non-atomic reads set  $\text{Read}_q^{\text{na}}(\ell, \alpha_r)$ , needed to extend the read set  $\alpha_r$  with the read to be performed.

The post-condition is simple: the singleton value  $v$  is returned, the history ownership and atomic writes set are unchanged, the non-atomic reads set is extended with a new action id  $r$  representing this read, and the thread arrives at a new thread-view  $\mathcal{V}'$  represented by  $\text{Seen}(\mathcal{V}')$ . We can make an abstraction here on how  $\mathcal{V}'$  is related to  $\mathcal{V}$  (like in the rule **BL-HOARE-REL-FENCE**), but the higher-level rules require a detailed relation between  $\mathcal{V}'$  and  $\mathcal{V}$ , so we simply keep the “raw” relation  $\mathcal{V} \xrightarrow{\text{R:na}, \ell, t, \perp, r} \mathcal{V}'$  from the operational semantics.<sup>7</sup> Note that we do know that  $\mathcal{V} \sqsubseteq \mathcal{V}'$ .

<sup>6</sup>Recall that observing the latest write (the previous part of the precondition) does not guarantee observation of all writes—it only guarantees observation of all non-atomic writes because non-atomic writes cannot race with one another, while atomic writes can. See also the discussion in **Definition 3.25**.

<sup>7</sup>see **OM-POST-READ-TVIEW**, §3.3



$$\begin{array}{c}
\text{BL-HOARE-READ-NA} \\
\frac{\text{Local}_A(\ell, [t \leftarrow (v, V^?)], \mathcal{V}.cur) \quad \text{Local}_W^{\exists \text{rlx}}(\ell, \alpha_w, \mathcal{V}.cur) \quad \text{Local}_R^{\text{na}}(\ell, \alpha_r, V_r)}{\left\{ \text{Seen}(\mathcal{V}) * \text{Hist}_q(\ell, [t \leftarrow (v, V^?)]) * \text{Write}_q^{\exists \text{rlx}}(\ell, \alpha_w) * \text{Read}_q^{\text{na}}(\ell, \alpha_r) \right\}} \\
(*^{\text{na}} \ell, \mathcal{V}) \\
\left\{ (w, \mathcal{V}'). w = v * \exists r. \mathcal{V} \xrightarrow{R:\text{na}, \ell, t, \perp, r} \mathcal{V}' * \text{Local}_R^{\text{na}}(\ell, \alpha_r \cup \{r\}, V_r \sqcup \mathcal{V}'.cur) * \right. \\
\left. \text{Seen}(\mathcal{V}') * \text{Hist}_q(\ell, [t \leftarrow (v, V^?)]) * \text{Write}_q^{\exists \text{rlx}}(\ell, \alpha_w) * \text{Read}_q^{\text{na}}(\ell, \alpha_r \cup \{r\}) \right\}_{\mathcal{E}}
\end{array}$$
  

$$\begin{array}{c}
\text{BL-HOARE-WRITE-NA} \\
\frac{\text{Local}_A(\ell, [t \leftarrow (v, V^?)], \mathcal{V}.cur) \quad \text{Local}_R^{\text{na}}(\ell, \alpha_1, \mathcal{V}.cur) \quad \text{Local}_R^{\exists \text{rlx}}(\ell, \alpha_2, \mathcal{V}.cur) \quad \text{Local}_W^{\exists \text{rlx}}(\ell, \alpha_w, \mathcal{V}.cur)}{\left\{ \text{Seen}(\mathcal{V}) * \text{Hist}(\ell, [t \leftarrow (v, V^?)]) * \text{Write}^{\exists \text{rlx}}(\ell, \alpha_w) * \text{Read}^{\text{na}}(\ell, \alpha_1) * \text{Read}^{\exists \text{rlx}}(\ell, \alpha_2) \right\}} \\
(\ell :=_{\text{na}} v', \mathcal{V}) \\
\left\{ (\otimes, \mathcal{V}'). \exists t' > t. \mathcal{V} \xrightarrow{W:\text{na}, \ell, t', \perp, \perp} \mathcal{V}' * \text{Local}_A(\ell, [t' \leftarrow (v', \perp)], \mathcal{V}'.cur) * \right. \\
\left. \text{Seen}(\mathcal{V}') * \text{Hist}(\ell, [t' \leftarrow (v', \perp)]) * \text{Write}^{\exists \text{rlx}}(\ell, \alpha_w) * \text{Read}^{\text{na}}(\ell, \alpha_1) * \text{Read}^{\exists \text{rlx}}(\ell, \alpha_2) \right\}_{\mathcal{E}}
\end{array}$$

FIGURE 7.4: The base logic's primitive Hoare rules for non-atomic reads and writes

Last but not least, we have an additional assertion  $\text{Local}_R^{\text{na}}(\ell, \alpha_r, V_r)$  in the pre-condition that is updated to  $\text{Local}_R^{\text{na}}(\ell, \alpha_r \cup \{r\}, V_r \sqcup \mathcal{V}'.cur)$  in the post-condition. This assertion is not needed for the rule per se. It is used to track the view  $V_r$  (which is  $V_r \sqcup \mathcal{V}'.cur$  after the step) that has observed the subset  $\alpha_r$  of all non-atomic reads performed so far with the fraction  $\text{Read}_q^{\text{na}}(\ell, \alpha_r)$ . The view  $V_r$  will only be needed later when a thread has recollected the full fraction  $\text{Read}^{\text{na}}(\ell, \alpha_r)$  where  $\alpha_r$  is then the complete set of  $\ell$ 's non-atomic reads. At that point, race-free operations would require that the executing thread has observed all reads in  $\alpha_r$ , which pins down to the thread's thread-view  $\mathcal{V}$  including  $V_r$ , i.e.,  $V_r \sqsubseteq \mathcal{V}.cur$ . We will see concretely how this view is used in [Chapter 9](#).

NON-ATOMIC WRITES. **BL-HOARE-WRITE-NA** is the most demanding rule, as a non-atomic write cannot race with any other memory accesses to the same location  $\ell$ .<sup>8</sup> The pre-condition therefore requires full ownership (the fraction  $q = 1$ ) of  $\ell$ 's current singleton history, and of the 3 sets of the race detector's state, and the knowledge that the current thread-view  $\mathcal{V}.cur$  has observed  $\ell$ 's allocation, latest write, and those sets of all reads and all atomic writes to  $\ell$ .<sup>9</sup>

The post-condition keeps most ownership unchanged, and only updates the singleton history ownership to  $\text{Hist}(\ell, [t' \leftarrow (v', \perp)])$ , where  $t'$  is the new timestamp for the new write message, with the value  $v'$  and no message view. The thread arrives at a new thread-view  $\mathcal{V}'$  computed from  $\mathcal{V}$  ( $\mathcal{V} \xrightarrow{W:\text{na}, \ell, t', \perp, \perp} \mathcal{V}'$ ),<sup>10</sup> and knows that the new current view  $\mathcal{V}'.cur$  has observed the new write ( $\text{Local}_A(\ell, [t' \leftarrow (v', \perp)], \mathcal{V}'.cur)$ ).

ATOMIC READS. The rule **BL-HOARE-READ-AT** for atomic reads is rather simple: it requires as pre-condition fractional ownership  $q$  of  $\ell$ 's current history, and of an atomic read subset of the race detector's state  $\text{Read}_q^{\exists \text{rlx}}(\ell, \alpha)$  to add the read to be performed. An atomic read only needs to avoid race with non-atomic writes,<sup>11</sup> so  $\text{Local}_A(\ell, h, \mathcal{V}.cur)$  is

<sup>8</sup>see [DRF-WRITE-NA](#), §3.4

<sup>9</sup>Recall that a fraction of  $\text{Read}_q^{\text{na}}(\ell, \alpha_1)$  or  $\text{Read}_q^{\exists \text{rlx}}(\ell, \alpha_2)$  only says that  $\alpha_1$  or  $\alpha_2$  is only a subset of the global set—we need a full fraction to be guaranteed that  $\alpha_1$  or  $\alpha_2$  is the global set.

<sup>10</sup>see [OM-POST-WRITE-TVIEW](#), §3.3

<sup>11</sup>see [DRF-READ-AT](#), §3.4



$$\begin{array}{c}
\text{BL-HOARE-READ-AT} \\
\frac{\mathbf{rlx} \sqsubseteq o \quad \text{Local}_A(\ell, h, \mathcal{V}.cur) \quad \text{Local}_R^{\exists \mathbf{rlx}}(\ell, \alpha, V_r)}{\left\{ \text{Seen}(\mathcal{V}) * \text{Hist}_q(\ell, h) * \text{Read}_q^{\exists \mathbf{rlx}}(\ell, \alpha) \right\}} \\
\left( {}^*o\ell, \mathcal{V} \right) \\
\left\{ \begin{array}{l} (v, \mathcal{V}'). \exists t, V^?, r. h(t) = (v, V^?) * \mathcal{V} \xrightarrow{R:o, \ell, t, V^?, r} \mathcal{V}' * \text{Local}_R^{\exists \mathbf{rlx}}(\ell, \alpha \cup \{r\}, V_r \sqcup \mathcal{V}'.cur) * \\ \text{Seen}(\mathcal{V}') * \text{Hist}_q(\ell, h) * \text{Read}_q^{\exists \mathbf{rlx}}(\ell, \alpha \cup \{r\}) \end{array} \right\}_{\mathcal{E}} \\
\\
\text{BL-HOARE-WRITE-AT} \\
\frac{\mathbf{rlx} \sqsubseteq o \quad \text{Local}_A(\ell, h, \mathcal{V}.cur) \quad \text{Local}_R^{\text{na}}(\ell, \alpha_r, \mathcal{V}.cur) \quad \text{Local}_W^{\exists \mathbf{rlx}}(\ell, \alpha_w, V_w)}{\left\{ \text{Seen}(\mathcal{V}) * \text{Hist}(\ell, h) * \text{Write}^{\exists \mathbf{rlx}}(\ell, \alpha_w) * \text{Read}^{\text{na}}(\ell, \alpha_r) \right\}} \\
\left( \ell :=_o v', \mathcal{V} \right) \\
\left\{ \begin{array}{l} (\mathcal{V}', \mathcal{V}'). \exists t' \notin \text{dom}(h), V'. \mathcal{V} \xrightarrow{W:o, \ell, t', \perp, V'} \mathcal{V}' * \text{Local}_W^{\exists \mathbf{rlx}}(\ell, \alpha_w \cup \{t'\}, V_w \sqcup \mathcal{V}'.cur) * \\ \text{Seen}(\mathcal{V}') * \text{Hist}(\ell, h[t' \leftarrow (v', V')]) * \text{Write}^{\exists \mathbf{rlx}}(\ell, \alpha_w \cup \{t'\}) * \text{Read}^{\text{na}}(\ell, \alpha_r) \end{array} \right\}_{\mathcal{E}}
\end{array}$$

FIGURE 7.5: The base logic's primitive Hoare rules for atomic reads and writes

sufficient, because every non-atomic write would reset the history  $h$  to a singleton, and  $\text{Local}_A(\ell, h, \mathcal{V}.cur)$  guarantees that the current thread-view  $\mathcal{V}.cur$  has observed some write in  $h$ .

In the post-condition, some value  $v$  in the history will be read and returned, and the atomic reads set is extended with the new action id for this read ( $\text{Read}_q^{\exists \mathbf{rlx}}(\ell, \alpha \cup \{r\})$ ), and the thread arrives at a new thread-view  $\mathcal{V}'$  computed from  $\mathcal{V}$  with  $\mathcal{V} \xrightarrow{R:o, \ell, t, V^?, r} \mathcal{V}'$ .<sup>12</sup> Note that the relaxed memory effects are contained within this relation for  $\mathcal{V}'$ , and will be abstracted later with higher-level rules.

<sup>12</sup>see [OM-POST-READ-TVVIEW](#), §3.3

The view  $V_r$  in  $\text{Local}_R^{\exists \mathbf{rlx}}(\ell, \alpha, V_r)$  plays the same role as its counterpart in [BL-HOARE-READ-NA](#).  $V_r$  is guaranteed to have observed the subset  $\alpha$  of atomic reads performed so far with the fraction  $q$  of  $\text{Read}_q^{\exists \mathbf{rlx}}(\ell, \alpha)$ . We will see concretely how  $V_r$  is used in [Chapter 10](#).

**ATOMIC WRITES.** An atomic write must not race with non-atomic accesses, both reads and writes,<sup>13</sup> so [BL-HOARE-WRITE-AT](#) requires as pre-condition full fractions of the current history ownership  $\text{Hist}(\ell, h)$  and of the non-atomic reads set  $\text{Read}^{\text{na}}(\ell, \alpha_r)$ , as well as the knowledge  $\text{Local}_A(\ell, h, \mathcal{V}.cur)$  and  $\text{Local}_R^{\text{na}}(\ell, \alpha_r, \mathcal{V}.cur)$  that the current thread-view  $\mathcal{V}.cur$  has observed all non-atomic writes and reads to  $\ell$ . Additionally, the full ownership  $\text{Write}^{\exists \mathbf{rlx}}(\ell, \alpha_w)$  of the race detector's atomic writes set for  $\ell$  is needed to extend the set  $\alpha_w$  with the write to be performed.

<sup>13</sup>see [DRF-WRITE-AT](#), §3.4

The post-condition keeps the non-atomic reads set unchanged, extends the atomic writes set with the new timestamp  $t'$  (fresh in  $h$ ), and updates the history and the atomic writes set to insert the new write ( $h[t' \leftarrow (v', V')]$  and  $\alpha_w \cup \{t'\}$ ). The new thread-view  $\mathcal{V}'$  is computed from  $\mathcal{V}$  accordingly.<sup>14</sup>

<sup>14</sup>see [OM-POST-WRITE-TVVIEW](#), §3.3

The view  $V_w$  in  $\text{Local}_W^{\exists \mathbf{rlx}}(\ell, \alpha_w, V_w)$  plays the same role as the view  $V_r$  in [BL-HOARE-READ-NA](#) and [BL-HOARE-READ-AT](#). It is the view that has observed all atomic writes  $\alpha_w$  done so far using  $\text{Write}^{\exists \mathbf{rlx}}(\ell, \alpha_w)$ . We will

BL-Hoare-CAS

$$\begin{array}{c}
\text{r1x} \sqsubseteq o_f, o_r, o_w \quad \text{Local}_A(\ell, h, \mathcal{V}.cur) \quad \text{Local}_R^{na}(\ell, \alpha_1, \mathcal{V}.cur) \\
\text{Local}_R^{\text{r1x}}(\ell, \alpha_2, V_r) \quad \text{Local}_W^{\text{r1x}}(\ell, \alpha_w, V_w) \quad \forall v_0 \in \text{Readable}(h, \mathcal{V}). \vdash v_0 =^? v_r \\
\hline
\left\{ \begin{array}{l}
\text{Seen}(\mathcal{V}) * \text{Hist}(\ell, h) * \text{Write}^{\text{r1x}}(\ell, \alpha_w) * \text{Read}^{na}(\ell, \alpha_1) * \text{Read}_q^{\text{r1x}}(\ell, \alpha_2) * \\
P_{\text{cmp}} * \Box((v_r = \ell_r) ? (P_{\text{cmp}} \multimap \Phi_{\text{cmp}}(\ell_r)) : \text{True})
\end{array} \right\} \\
(\text{CAS}^{o_f, o_r, o_w}(\ell, v_1, v_2), \mathcal{V}) \\
\left( \begin{array}{l}
(b, \mathcal{V}'). P_{\text{cmp}} * \exists h', t, t', v', V, V', r, \mathcal{V}_x. \mathcal{V} \sqsubseteq \mathcal{V}_x \sqsubseteq \mathcal{V}' * h(t') = (v', V') * \\
\text{Local}_W^{\text{r1x}}(\ell, (b) ? (\alpha_w \cup \{t\}) : \alpha_w, V_w \sqcup \mathcal{V}'.cur) * \text{Local}_R^{\text{r1x}}(\ell, \alpha_2 \cup \{r\}, V_r \sqcup \mathcal{V}'.cur) * \\
\text{Seen}(\mathcal{V}') * \text{Hist}(\ell, h') * \text{Write}^{\text{r1x}}(\ell, (b) ? (\alpha_w \cup \{t\}) : \alpha_w) * \text{Read}^{na}(\ell, \alpha_1) * \text{Read}_q^{\text{r1x}}(\ell, \alpha_2 \cup \{r\}) * \\
b = \text{false} * v_r \neq v' * h' = h * \mathcal{V} \xrightarrow{\text{R}:o_f, \ell, t', V', r} \mathcal{V}_x \\
\vee \quad b = \text{true} * v_r = v' * t \notin \text{dom}(h) * t = t' + 1 * V \sqsupseteq V' * h' = h[t \leftarrow (v_w, V)] * \\
\mathcal{V} \xrightarrow{\text{R}:o_r, \ell, t', V', r} \mathcal{V}_x \xrightarrow{\text{W}:o_w, \ell, t, V', V} \mathcal{V}'
\end{array} \right)_{\mathcal{E}}
\end{array}$$

where  $\Phi_{\text{cmp}}(\ell_r) ::= (\models_{\mathcal{E}} \exists q_r, h_r. \triangleright \text{Hist}_{q_r}(\ell_r, h_r)) \wedge (\forall \ell' \in \text{Readable}(h, \mathcal{V}) \setminus \{\ell_r\}. \models_{\mathcal{E}} \exists q', h'. \triangleright \text{Hist}_{q'}(\ell', h'))$

FIGURE 7.6: The base logic's primitive Hoare rule for CASes

also see concretely how  $V_w$  is used in [Chapter 10](#).

## 7.3.3 A Rule for CASes

We present a Hoare rule **BL-Hoare-CAS** for CASes in [Figure 7.6](#). It is a rather complicated rule, because a CAS is a combination of a read, a write, and a comparison that can be a pointer comparison.

<sup>15</sup>see [DRF-UPDATE](#), §3.4

First of all, a CAS cannot race with non-atomic accesses (reads and writes),<sup>15</sup> so the pre-condition requires the full fractions of the history ownership  $\text{Hist}(\ell, h)$ , and of the non-atomic reads set  $\text{Read}^{na}(\ell, \alpha_1)$ , and the knowledge  $\text{Local}_A(\ell, h, \mathcal{V}.cur)$  and  $\text{Local}_R^{na}(\ell, \alpha_1, \mathcal{V}.cur)$  that  $\mathcal{V}.cur$  has observed  $\ell$ 's allocation and all of its non-atomic reads and writes.

Second, the pre-condition needs the full fraction of the atomic writes set  $\text{Write}^{\text{r1x}}(\ell, \alpha_w)$  and a fraction of the atomic reads set  $\text{Read}_q^{\text{r1x}}(\ell, \alpha_2)$  in order to potentially extend those sets with a write event and a read event that are to be generated by this CAS operation. The views  $V_w$  and  $V_r$  have observed the sets  $\alpha_w$  and  $\alpha_2$  respectively, and play the similar role to those in the read and write rules, which we will see in [Chapter 10](#).

Third, the post-condition is a combination of read and write effects. The operation returns a boolean value  $b$  to indicate success or failure. In any case, a message  $(t', v', V')$  in  $h$  will be read, and the atomic reads set  $\alpha_2$  will be extended with a new action id  $r$  for that read. The non-atomic reads set remains unchanged.

<sup>16</sup>see [Definition 4.8](#), §4.2

- In case the CAS fails, i.e.,  $b = \text{false}$ , we know that the value read  $v'$  is not equal to the expected value  $v_r$  ( $\vdash v' \neq v_r$ ),<sup>16</sup> that the history  $h$  and the atomic writes set  $\alpha_w$  also remain unchanged, and that the thread-view effect is similar to that of a read with the mode  $o_f$  ( $\mathcal{V} \xrightarrow{\text{R}:o_f, \ell, t', V', r} \mathcal{V}_x$ ).<sup>17</sup>

<sup>17</sup>see [OM-POST-READ-TVIEW](#), §3.3

- In case the CAS succeeds, *i.e.*,  $b = \mathbf{false}$ , the read value  $v'$  is exactly the expected value  $v_r$ , and a new message  $(t, v_w, V)$  is inserted into  $h$  ( $h' = h[t \leftarrow (v_w, V)]$ ) right next to the read message ( $t = t' + 1$ ) which guarantees the atomicity of the read and the write generated by the CAS. The new thread-view  $\mathcal{V}'$  and the write message view  $V$  are computed from the old thread-view  $\mathcal{V}$  and the read message view  $V'$  accordingly.<sup>18</sup> The atomic writes set  $\alpha_w$  is also extended with the new timestamp  $t$ .

<sup>18</sup>see OM-UPDATE, §3.3

Finally, we look at the rule's components concerning (pointer) comparison. The rule requires safety in the comparison between the expected value  $v_r$  and any potential value  $v_0$  that the CAS may read:  $\forall v_0 \in \text{Readable}(h, \mathcal{V}). \vdash v_0 =^? v_r$ . The set of readable values  $\text{Readable}(h, \mathcal{V})$  is lifted for histories from Definition 4.6 for the global memory. The comparison is safe if the values are comparable (Definition 4.9).

**DETERMINISTIC POINTER COMPARISON.** If the comparison is between locations, *i.e.*, if  $v_r$  is a non-null location  $\ell_r$ , the pre-condition of **BL-HOARE-CAS** requires some extra resources  $P_{\text{cmp}}$  to learn that compared locations are alive, and thus to guarantee deterministic comparison.<sup>19</sup> Furthermore,  $P_{\text{cmp}}$  will only be used to derive facts and will not be consumed, so it is returned as-is in the post-condition. How  $P_{\text{cmp}}$  will be used is encoded in the predicate  $\Phi_{\text{cmp}}(\ell_r)$  which employs a *classical* conjunction. Intuitively, the persistent implication  $\Box(P_{\text{cmp}} \multimap \Phi_{\text{cmp}}(\ell_r))$  requires that the resources in  $P_{\text{cmp}}$  *simultaneously* support two goals:

<sup>19</sup>Comparability makes sure that we need not care about the case where the expected value  $v_r$  is an integer but the readable values can be locations. If that is the case,  $v_r$  must be 0, and the comparison always fails. See also Definition 4.8.

1. using  $P_{\text{cmp}}$  and potentially opening some invariant with the fancy update  $\models_{\mathcal{E}}$ , one gets some fraction of the ownership  $\text{Hist}_{q_r}(\ell_r, h_r)$  of the expected value  $\ell_r$ , which is sufficient to learn that  $\ell_r$  is alive.
2. for any location  $\ell'$  readable from  $h$  that is not  $\ell_r$ ,<sup>20</sup> using  $P_{\text{cmp}}$  and potentially opening some invariant, one also learns that  $\ell'$  is alive.

<sup>20</sup>If the read value  $\ell'$  is also  $\ell_r$ , then the first part of the classical conjunction is also the proof that  $\ell'$  is alive.

#### 7.3.4 A Stronger WP Rule for CASes

We present the rule **BL-WP-CAS** (Figure 7.7) for CASes that is stronger than **BL-HOARE-CAS**. Note that this rule is very technical and is only used to get stronger GPS rules that will be used in Part III. Readers are welcome to skip this rule and continue with the next section.

The rule is written in form of weakest pre-conditions—a general fashion that is common with Iris WPs, where the post-condition is universally quantified as  $\Phi$ . This style is not only more convenient to use in practice in Coq, but also important to make our CAS rule stronger.

**Notation 7.7** (Iris WP-style Rules). Recall Definition 6.7 where Hoare triples are derived from WPs. In practice (in Coq), Iris rules for Hoare triples and WPs are usually written with a universally quantified post-condition  $\Phi$ , so that they can be easily applied to a goal with an arbitrarily shaped WP. For example, if  $e$  is not a value,<sup>21</sup> a Hoare rule  $\vdash \{P\} e \{v. Q\}$  for  $e$  can instead be written as:

<sup>21</sup>If it is a value, we do not have a later modality.

$$\vdash \Box(P \multimap \forall \Phi. (\triangleright \forall v. Q \multimap \Phi(v)) \multimap \text{wp } e \{ \Phi \})$$

That is, the rule intuitively encodes that  $Q$  is the *strongest post-condition* for  $e$  under the pre-condition  $P$ . The later modality allows us to prove the post-condition only after the step, at which point our resources which are under a later before the step have been made available. This from of rule is more applicable to a goal of form  $\text{wp } K[e] \{ \Psi \}$ : we first apply the bind rule **WP-BIND** to focus on the expression  $e$  and push the continuation into the post-condition (i.e.,  $\text{wp}_{\mathcal{E}} e \{ v. \text{wp}_{\mathcal{E}} K[v] \{ \Psi \} \}$ ) which will then be used to instantiate the predicate  $\Phi$  of the rule.

We state our CAS rule in the following form:

$$\frac{R \quad R' \quad \triangleright \forall v, \mathcal{V}_v. Q \multimap \Phi(v, \mathcal{V}_v)}{P \vdash \text{wp } (e, \mathcal{V}) \{ \Phi \}}$$

where  $P$  is the pre-condition,  $Q$  is the post-condition, and  $R$  and  $R'$  are extra premises. The wand implication  $\triangleright \forall v, \mathcal{V}_v. Q \multimap \Phi(v, \mathcal{V}_v)$  is for the post-condition and is the right-most premise. A rule of this form can be read as the following Hoare rule:

$$\frac{R \quad R'}{\vdash \{P\} (e, \mathcal{V}) \{ (v, \mathcal{V}_v). Q \}}$$

Our CAS rule **BL-WP-CAS**, however, is strengthened by moving the later inside and adding fancy updates to the post-condition—a combination called *wand step viewshifts*.

**Notation 7.8** (Wand Step Viewshifts).

$$P \boxtimes_{\mathcal{E}}^{\mathcal{E}'} Q$$

The wand step viewshift is a balanced (potentially mask-changing) viewshift that has a later in between.

$$P \boxtimes_{\mathcal{E}}^{\mathcal{E}'} Q ::= P \multimap \mathcal{E} \boxtimes_{\mathcal{E}'}^{\mathcal{E}'} \triangleright \mathcal{E} \boxtimes_{\mathcal{E}'}^{\mathcal{E}'} Q$$

We can now look at **BL-WP-CAS** in **Figure 7.7**. The rule can be applied with an arbitrary post-condition  $\Psi$  which typically is the continuation after executing the CAS. The rule says that the client can go on proving  $\Psi$  *assuming* the return value  $(b, \mathcal{V}')$  (together with other variables) universally quantified in the right-most premise, as well as the resources on the left-hand side of the wand implications. Compared to the alternative WP-style reading (**Notation 7.7**) of the Hoare rule **BL-HOARE-CAS**, **BL-HOARE-CAS** are strengthened in several ways.

- The client of the rule does not need to specify and provide  $P_{\text{cmp}}$  in the pre-condition. Instead, the client can pick  $P_{\text{cmp}}$  *after* learning all information about the results of the CAS (e.g., the return value  $b$ , the read and write timestamps, the new history  $h'$ , the thread-views and views). In fact, the client only needs to provide (prove)  $P_{\text{cmp}}$  and how it is to be used ( $\Phi$ ) if the CAS succeeds ( $b = \mathbf{true}$ ).
- Note that the client however does not know that “ $v_r = v'$  in case  $b = \mathbf{true}$ ” *before* picking and proving  $P_{\text{cmp}}$ , because  $P_{\text{cmp}}$  is needed to achieve that deterministic comparison result.

BL-WP-CAS

$$\begin{array}{l}
\forall b, \mathcal{V}', h', t, t', v', V, V', r, \mathcal{V}_x. \\
\left( \begin{array}{l}
\mathcal{V} \sqsubseteq \mathcal{V}_x \sqsubseteq \mathcal{V}' * h(t') = (v', V') * \\
\text{Local}_w^{\neg \text{rlx}}(\ell, (b) ? \alpha_w \cup \{t\} : \alpha_w, V_w \sqcup \mathcal{V}'.\text{cur}) * \\
\text{Local}_r^{\neg \text{rlx}}(\ell, \alpha_2 \cup \{r\}, V_r \sqcup \mathcal{V}'.\text{cur}) * \\
b = \mathbf{false} * v_r \neq v' * h' = h * \mathcal{V} \xrightarrow{\text{R:} o_f, \ell, t', V', r} \mathcal{V}_x \\
\vee \quad b = \mathbf{true} * t \notin \text{dom}(h) * t = t' + 1 * V \sqsupseteq V' * \\
h' = h[t \leftarrow (v_w, V)] * \mathcal{V} \xrightarrow{\text{R:} o_r, \ell, t', V', r} \mathcal{V}_x \xrightarrow{\text{W:} o_w, \ell, t, V', V} \mathcal{V}'
\end{array} \right) \\
\vdash v_0 =^? v_r \quad \neg * P_{\text{cmp}}. P_{\text{cmp}} * \square((b \wedge v_r = \ell_r) ? \Phi_{\text{cmp}}(P_{\text{cmp}}, \ell_r, v') : \text{True}) * \\
\left( \begin{array}{l}
((b) ? v_r = v' : \text{True}) * P_{\text{cmp}} * \text{Seen}(\mathcal{V}') * \text{Hist}(\ell, h') * \\
\text{Write}^{\neg \text{rlx}}(\ell, (b) ? \alpha_w \cup \{t\} : \alpha_w) * \text{Read}^{\text{na}}(\ell, \alpha_1) * \text{Read}_q^{\neg \text{rlx}}(\ell, \alpha_2 \cup \{r\}) \\
\Rightarrow_{\mathcal{E}}^{\mathcal{E}'} \Psi(b, \mathcal{V}')
\end{array} \right) \\
\hline
\text{Seen}(\mathcal{V}) * \text{Hist}(\ell, h) * \text{Write}^{\neg \text{rlx}}(\ell, \alpha_w) * \text{Read}^{\text{na}}(\ell, \alpha_1) * \text{Read}_q^{\neg \text{rlx}}(\ell, \alpha_2) \vdash \text{wp}_{\mathcal{E}}(\text{CAS}^{o_f, o_r, o_w}(\ell, v_1, v_2), \mathcal{V}) \{ \Psi \}
\end{array}$$

where  $\Phi_{\text{cmp}}(P_{\text{cmp}}, \ell_r, v') ::= \wedge$

$$\begin{array}{l}
P_{\text{cmp}} \xRightarrow{\mathcal{E}}^{\mathcal{E}_r} \exists q_r, h_r. \triangleright \text{Hist}_{q_r}(\ell_r, h_r) \\
\forall \ell' = v' \neq \ell_r. P_{\text{cmp}} \xRightarrow{\mathcal{E}}^{\mathcal{E}'} \triangleright \xRightarrow{\mathcal{E}_0(\ell')} \exists q', h'. \triangleright \text{Hist}_{q'}(\ell', h')
\end{array}$$

FIGURE 7.7: The base logic's primitive WP rule for CASes

- After proving  $P_{\text{cmp}}$  and  $\Phi$ , the client does get the deterministic comparison result and  $P_{\text{cmp}}$  back. Recall that  $P_{\text{cmp}}$  is not consumed and is only needed to know that compared locations are alive.
- The client also acquires the returned resources (i.e., the history ownership and the ownership of the reads and writes sets), and then can prove the continuation with the wand step viewshift. The wand viewshift  $\Rightarrow_{\mathcal{E}}^{\mathcal{E}'}$  allows the client to make a mask-changing viewshift from the mask  $\mathcal{E}$  to the mask  $\mathcal{E}'$  to open invariants ( $\mathcal{E}'$  is of the client's choice), then to strip a later in any resources that the client owns at that point, and then to close the invariants and return to the mask  $\mathcal{E}$ , all in order to prove the continuation  $\Psi(b, \mathcal{V}')$ . Note that if the client uses **BL-HOARE-CAS**, they would not have a later at their disposal, because the results of the CAS operation are only available *after* the later is introduced.
- Finally, the client can also rely on a later and mask-changing viewshifts when proving  $\Phi_{\text{cmp}}$ , i.e., the proof that  $P_{\text{cmp}}$  implies that the compared locations are alive. In particular, the mask  $\mathcal{E}_r$  and the function  $\mathcal{E}_0$  from locations to masks are also of the client's choice. The client only needs to show that expected value  $\ell_r$  and the read value  $\ell'$  are alive. Interestingly, the client can do so by opening invariants (of the client's choice) without closing them.

## 7.4 Resource Algebras for Basic Local Assertions

We briefly explain the resource algebras needed to define our local assertions and tie them to the physical state. We will need 5 RAs.

**Definition 7.9** (Lattice RA for Seen Thread-view Observations). The *lattice* RA  $\text{LAT}(A)$  takes a join semi-lattice  $A$  and defines the composition as the lattice's join operation, the core function as the identity function (so that every element is the core of itself), and validity is trivial. That is,  $\text{LAT}(A) ::= (A, (\lambda_. \text{True}), \text{id}, \sqcup)$ . If the lattice  $A$  has a bottom element  $\perp \sqsubseteq a$  for any  $a \in A$ , then  $\perp$  is the unit of  $\text{LAT}(A)$ . Note that RA inclusion  $\preceq$  then coincides with the lattice order  $\sqsubseteq$ . Most importantly, the RA has the following properties.

$$\forall a, b. \text{valid}(\bullet a \cdot \circ b) \Rightarrow b \sqsubseteq a \quad (\text{AUTH-LAT-VALID})$$

$$\forall a, b. b \sqsubseteq a \Rightarrow \bullet b \rightsquigarrow \bullet a \cdot \circ a \quad (\text{AUTH-LAT-UPDATE})$$

For  $\text{Seen}(\mathcal{V})$ , we use the RA  $\text{SEENR} = \text{AUTH}(\text{LAT}(\text{View}))$ . It is an optimization that we only track a simple view with  $\text{LAT}(\text{View})$  and not a thread-view with  $\text{LAT}(\text{ThreadView})$ . Recall that the role of  $\text{Seen}(\mathcal{V})$  is to guarantee that  $\mathcal{V}$  is closed in the global memory, which can be done instead by just guaranteeing that  $\mathcal{V}.\text{acq}$  is closed in the global memory, because  $\mathcal{V}.\text{acq}$  includes all other components of a wellformed  $\mathcal{V}$ . The closedness condition also means that we need not track one view per thread: we simply track the upper bound  $V_{up}$  for all acquire components of all thread-views and require that  $V_{up}$  is closed in the global memory.

In particular, the authoritative element  $\bullet V_{up}$  guarantees that, for any other fragmentary element  $\circ \mathcal{V}'.\text{acq}$ , thanks to **AUTH-LAT-VALID**,  $\mathcal{V}'.\text{acq} \sqsubseteq V_{up}$ . Furthermore, due to **AUTH-LAT-UPDATE**,  $\bullet V_{up}$  can only be updated to a bigger view, mirroring the property that views only grow.

**Definition 7.10** (Fractional Agreement Map RA for History). We use the RA  $\text{HISTR} = \text{AUTH}(\text{MAP}(\text{Loc}, \text{FRAC} \times \text{AG}(\text{History}^?)))$  for  $\text{Hist}_q(\ell, h)$ .

The *agree* RA  $\text{AG}$  only provides valid composition between elements that are the same, and the *fractional* RA  $\text{FRAC}$  provides valid composition between non-negative quotients that sum up to no greater than 1 (i.e., they are in the range  $[0, 1)$ ). We use them together using the product RA (written here as  $\times$ ) which provides valid composition point-wise. The map RA  $\text{MAP}$  takes a key type and a value RA, and provides valid composition key-wise, using the valid composition of the value RA.

As such, our combined use of  $\text{MAP}$  with  $\text{FRAC}$  and  $\text{AG}$  gives per-location agreement between fractions of history ownership, and with the full fraction we can change the history. We use the option type  $\text{History}^?$  to support deallocation: when a location is deallocated, then its history will be  $\text{None}$ . We use  $\text{AUTH}$  to have the authoritative element be the complete memory, and the fragmentary elements of singleton maps will be used to define  $\text{Hist}_q(\ell, h)$ . More concretely, we have the following properties.

$$\forall m, \ell, q, h. \text{valid}(\bullet m \cdot \circ [\ell \leftarrow (q, \text{ag}(h))]) \Rightarrow m(\ell) = (1, \text{ag}(h))$$

$$\forall \ell, q, h, q', h'. \text{valid}(\circ [\ell \leftarrow (q, \text{ag}(h))] \cdot \circ [\ell \leftarrow (q', \text{ag}(h'))]) \Rightarrow h = h'$$

$$\forall m, \ell, h. \bullet m \cdot \circ [\ell \leftarrow (1, \text{ag}(h))] \rightsquigarrow \bullet m[\ell \leftarrow (1, \text{ag}(h'))] \cdot \circ [\ell \leftarrow (1, \text{ag}(h'))]$$

The injection  $\text{ag}$  is the constructor of the RA  $\text{AG}$ .

**Definition 7.11** (Fractional Map RA for Atomic Writes Sets). We use the RA  $\text{WRITER} = \text{AUTH}(\text{MAP}(\text{Loc}, \text{FRAC} \times \text{AG}(\text{ActIds})))$  for the fractional per-location ownership of atomic writes sets. This RA is similar to that for history ownership, but tracks a set of action ids instead.

**Definition 7.12** (Fractional Set Lattice RA for Reads Sets). We use a slightly different RA  $\text{READR} = \text{AUTH}(\text{MAP}(\text{Loc}, \text{FRAC} \times \text{LAT}(\text{ActIds})))$ . That is, we use  $\text{LAT}(\text{ActIds})$  in place of  $\text{AG}(\text{ActIds})$ . As such, we do not have agreement between the fractions of read sets. In exchange, a fraction is sufficient to grow the set: we can update the element  $\circ [\ell \leftarrow (q, \alpha)]$  (together with the authoritative element) to  $\circ [\ell \leftarrow (q, \alpha')]$  where  $\alpha \subseteq \alpha'$  without requiring  $q = 1$ . Note that because we use the lattice RA  $\text{LAT}$ , the sets can only grow. More concretely, we have the following properties.

$$\begin{aligned} \forall m, \ell, q, \alpha. \text{valid}(\bullet m \cdot \circ [\ell \leftarrow (q, \alpha)]) &\Rightarrow \exists \alpha'. m(\ell) = (1, \alpha') \wedge \alpha \subseteq \alpha' \\ \forall m, \ell, q, \alpha, \alpha', \alpha''. m(\ell) = (1, \alpha') &\Rightarrow \alpha \subseteq \alpha'' \Rightarrow \\ &\bullet m \cdot \circ [\ell \leftarrow (1, \alpha)] \rightsquigarrow \bullet m[\ell \leftarrow (q, \alpha' \cup \alpha'')] \cdot \circ [\ell \leftarrow (q, \alpha'')] \end{aligned}$$

**Definition 7.13** (Fractional Block RA for Block Ownership). We use the RA  $\text{BLOCKR} = \text{AUTH}(\text{MAP}(\mathbb{N}^+, \text{FRAC} \times \text{MAP}(\mathbb{Z}, \text{EX}(1))))$ . That is, we use a map from block indices (in  $\mathbb{N}^+$ ) to fractional maps from offsets (in  $\mathbb{Z}$ ) to exclusive tokens (of type unit 1). The outer map allows us to have per-block ownership with full fraction, and the inner map allows use to split that full fraction between the offsets in the same block. The ownership of every single offset in a block represents the block ownership of a location and, thanks to the exclusive RA  $\text{EX}$ , such per-location block ownership is unique.

## 7.5 State Interpretation

We now define the local assertions and the state interpretation  $S$  for our base logic. We first need a few *global* ghost locations to store the RAs defined in the previous section. They are  $\gamma_{\text{SEEN}}$ ,  $\gamma_{\text{HIST}}$ ,  $\gamma_{\text{NAR}}$ ,  $\gamma_{\text{ATW}}$ ,  $\gamma_{\text{ATR}}$ , and  $\gamma_{\text{BLK}}$ . These ghost locations will need to be allocated before any program runs (in the adequacy proof, see [Theorem 7.19](#)).

**Definition 7.14** (Ghost State Model of Local Assertions). We define our local assertions purely as ghost ownership of fragmentary elements.

$$\begin{aligned} \text{Seen}(\mathcal{V}) &::= [\circ \mathcal{V}.\text{acq} : \text{SEENR}]^{\gamma_{\text{SEEN}}} \\ \text{Hist}_q(\ell, h) &::= [\circ [\ell \leftarrow (q, \text{ag}(\text{Some}(h)))] : \text{HISTR}]^{\gamma_{\text{HIST}}} \\ \text{Write}_q^{\text{rlx}}(\ell, \alpha) &::= [\circ [\ell \leftarrow (q, \text{ag}(\alpha))] : \text{WRITER}]^{\gamma_{\text{ATW}}} \\ \text{Read}_q^{\text{na}}(\ell, \alpha) &::= [\circ [\ell \leftarrow (q, \alpha)] : \text{READR}]^{\gamma_{\text{NAR}}} \\ \text{Read}_q^{\text{rlx}}(\ell, \alpha) &::= [\circ [\ell \leftarrow (q, \alpha)] : \text{READR}]^{\gamma_{\text{ATR}}} \\ \dagger_q^n \ell &::= [\circ [\ell \leftarrow (q, [m \leftarrow \text{ex}() \mid m \in [0, n]])] : \text{BLOCKR}]^{\gamma_{\text{BLK}}} \end{aligned}$$

The injection  $\text{ex}$  is the constructor of the RA  $\text{EX}$ .



**Definition 7.15** (Ghost Ownership for the Global State). The ghost ownership `GlobalGhost` that mirrors the global physical state is defined with ownership of authoritative elements. It takes as inputs the physical state  $(\mathcal{M}, \mathcal{N})$  and the global upper bound  $V_{up}$  of all threads's thread-views. Additionally, to support truncating histories with **BL-HIST-DROP-SINGLETON**, it also takes as input a view that tracks for each location the timestamp of its latest write. We call this view the *cut view*  $V_{cut}$ .

$\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) ::=$

$$\begin{aligned} & \bullet V_{up} : \text{SEENR} \}^{\gamma_{\text{SEEN}}} * \\ & \bullet [\ell \leftarrow (1, \text{ag}(\text{trunc}(\mathcal{M}(\ell), V_{cut}(\ell).w))) \mid \ell \in \text{dom}(\mathcal{M})] : \text{HISTR} \}^{\gamma_{\text{HIST}}} * \\ & \bullet [\ell \leftarrow (1, \text{ag}(\mathcal{N}(\ell).aw)) \mid \ell \in \text{dom}(\mathcal{N})] : \text{WRITER} \}^{\gamma_{\text{ATW}}} * \\ & \bullet [\ell \leftarrow (1, \mathcal{N}(\ell).nr) \mid \ell \in \text{dom}(\mathcal{N})] : \text{READR} \}^{\gamma_{\text{NAR}}} * \\ & \bullet [\ell \leftarrow (1, \mathcal{N}(\ell).ar) \mid \ell \in \text{dom}(\mathcal{N})] : \text{READR} \}^{\gamma_{\text{ATR}}} * \\ & \bullet [i \leftarrow (1, [n \leftarrow \text{ex}() \mid (i, n) \in \text{dom}(\mathcal{M})]) \mid (i, \_) \in \text{dom}(\mathcal{M})] : \text{BLOCKR} \}^{\gamma_{\text{BLK}}} * \end{aligned}$$

where

$$\text{trunc}(h, t_0) ::= \begin{cases} \text{None} & \text{if } h \text{ is deallocated} \\ \text{Some}([t \leftarrow h(t) \mid t_0 \leq t \in \text{dom}(h)]) & \text{if } h \text{ is alive} \end{cases}$$

So  $\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut})$  contains:

- the authoritative ownership of the upper-bound view  $V_{up}$  for all thread-views; and
- the authoritative full fraction ownership of all histories in  $\mathcal{M}$ , truncated by the cut view  $V_{cut}$ ; <sup>22</sup> and
- the authoritative full fraction ownership of all atomic writes sets, non-atomic reads sets, and atomic reads sets in  $\mathcal{N}$ ; and
- the authoritative full fraction ownership of all blocks in  $\mathcal{M}$ .

We use the map insert notation, e.g.,  $[\ell \leftarrow (1, \mathcal{N}(\ell).nr) \mid \ell \in \text{dom}(\mathcal{N})]$  to convert the map  $\mathcal{N}$  to a map from locations to pairs of fractions and non-atomic reads sets that come from  $\mathcal{N}$ .

**Lemma 7.16** (Agreements between the Global Ghost State and Local Assertions). *The global ghost state ownership `GlobalGhost` and the local assertions satisfy several agreement properties given in Figure 7.8. They are all derived from validity of the corresponding RAs.*

**Lemma 7.17** (Updates of the Global Ghost State and Local Assertions). *GlobalGhost can be updated together the local assertions following the rules in Figure 7.9. They are all derived from frame-preserving updates of the corresponding RAs, and the properties of `trunc`.*

Most notably, **BL-GHOST-UPDATE-HIST-DROP-SINGLETON** demonstrates that shrinking a history to a singleton is simply a logical change (a *viewshift*). It is done by bumping the cut view  $V'_{cut}$  for  $\ell$  up to its latest timestamp  $t$ , which is the input to `trunc`. The rule is used to prove

<sup>22</sup>Note that we also need to convert memory values to values, following Definition 3.16.



## BL-GHOST-SEEN

$$\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \text{Seen}(\mathcal{V}) \vdash \mathcal{V}.acq \sqsubseteq V_{up} \wedge \mathcal{V} \in \mathcal{M}$$

## BL-GHOST-HIST

$$\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \text{Hist}_q(\ell, h) \vdash \text{trunc}(\mathcal{M}(\ell), V_{cut}(\ell)) = h \wedge \ell \notin \text{unalloc}(\mathcal{M})$$

## BL-GHOST-AT-WRITE

$$\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \text{Write}_q^{\neg \text{rlx}}(\ell, \alpha) \vdash \mathcal{N}(\ell).aw = \alpha$$

## BL-GHOST-NA-READ

$$\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \text{Read}_q^{\text{na}}(\ell, \alpha) \vdash \alpha \subseteq \mathcal{N}(\ell).nr \wedge (q = 1 \Rightarrow \mathcal{N}(\ell).nr = \alpha)$$

## BL-GHOST-AT-READ

$$\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \text{Read}_q^{\neg \text{rlx}}(\ell, \alpha) \vdash \alpha \subseteq \mathcal{N}(\ell).ar \wedge (q = 1 \Rightarrow \mathcal{N}(\ell).ar = \alpha)$$

## BL-GHOST-BLOCK-FULL

$$\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \dagger^n \ell \vdash 0 < n * \forall m. (\ell +_\ell m) \in \text{dom}(\mathcal{M}) \Leftrightarrow m \in [0, n)$$

FIGURE 7.8: Several agreements between the global ghost state and local assertions

## BL-GHOST-UPDATE-SEEN

$$\frac{V_{up} \sqsubseteq V'_{up} \quad \mathcal{V}.cur \sqsubseteq V'_{up} \quad V'_{up} \in \mathcal{M}}{\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) \dot{\Rightarrow} \text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V'_{up}, V_{cut}) * \text{Seen}(\mathcal{V})}$$

## BL-GHOST-UPDATE-HIST-DROP-SINGLETON

$$\frac{h(t) = (v, V) \quad t = \max(\text{dom}(h)) \quad V'_{cut} = V_{cut}[\ell \leftarrow \{V_{cut}(\ell) [\mathbf{w} := t]\}]}{\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \text{Hist}(\ell, h) \dot{\Rightarrow} \text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V'_{cut}) * \text{Hist}(\ell, [t \leftarrow (v, V)])}$$

## BL-GHOST-UPDATE-NA-WRITE

$$\frac{t > \max(\text{dom}(h)) \geq V_{cut}(\ell).w \quad \mathcal{M}' = \mathcal{M}[\ell \leftarrow \mathcal{M}(\ell)[t \leftarrow (v, V)]] \quad \mathcal{N}' = \mathcal{N}[\ell \leftarrow \{\mathcal{N}(\ell) [\mathbf{w} := t]\}] \quad V'_{cut} = V_{cut}[\ell \leftarrow \{V_{cut}(\ell) [\mathbf{w} := t]\}]}{\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \text{Hist}(\ell, h) \dot{\Rightarrow} \text{GlobalGhost}(\mathcal{M}', \mathcal{N}', V_{up}, V'_{cut}) * \text{Hist}(\ell, [t \leftarrow (v, V)])}$$

## BL-GHOST-UPDATE-AT-WRITE

$$\frac{t \geq V_{cut}(\ell).w \quad \mathcal{M}' = \mathcal{M}[\ell \leftarrow \mathcal{M}(\ell)[t \leftarrow (v, V)]] \quad \mathcal{N}' = \mathcal{N}[\ell \leftarrow \{\mathcal{N}(\ell) [\mathbf{aw} := \mathcal{N}(\ell).aw \cup \{t\}]\}] \quad V'_{cut} = V_{cut}[\ell \leftarrow \{V_{cut}(\ell) [\mathbf{aw} := V_{cut}(\ell).aw \cup \{t\}]\}]}{\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \text{Hist}(\ell, h) * \text{Write}_q^{\neg \text{rlx}}(\ell, \alpha) \dot{\Rightarrow} \text{GlobalGhost}(\mathcal{M}', \mathcal{N}', V_{up}, V'_{cut}) * \text{Hist}(\ell, h[t \leftarrow (v, V)]) * \text{Write}_q^{\neg \text{rlx}}(\ell, \alpha \cup \{t\})}$$

## BL-GHOST-UPDATE-NA-READ

$$\frac{\mathcal{N}' = \mathcal{N}[\ell \leftarrow \{\mathcal{N}(\ell) [\mathbf{nr} := \mathcal{N}(\ell).nr \cup \{r\}]\}]}{\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \text{Read}_q^{\text{na}}(\ell, \alpha) \dot{\Rightarrow} \text{GlobalGhost}(\mathcal{M}, \mathcal{N}', V_{up}, V_{cut}) * \text{Read}_q^{\text{na}}(\ell, \alpha \cup \{r\})}$$

## BL-GHOST-UPDATE-AT-READ

$$\frac{\mathcal{N}' = \mathcal{N}[\ell \leftarrow \{\mathcal{N}(\ell) [\mathbf{ar} := \mathcal{N}(\ell).ar \cup \{r\}]\}]}{\text{GlobalGhost}(\mathcal{M}, \mathcal{N}, V_{up}, V_{cut}) * \text{Read}_q^{\neg \text{rlx}}(\ell, \alpha) \dot{\Rightarrow} \text{GlobalGhost}(\mathcal{M}, \mathcal{N}', V_{up}, V_{cut}) * \text{Read}_q^{\neg \text{rlx}}(\ell, \alpha \cup \{r\})}$$

FIGURE 7.9: Several update rules for the global ghost state and local assertions

**BL-HIST-DROP-SINGLETON**, by hiding the global ghost state `GlobalGhost` in the state interpretation  $S$ , as we will see next.

**Definition 7.18** (State Interpretation for the Base Logic).

$$\begin{aligned} \text{GlobalInv}(\varsigma) &::= \exists V_{up}, V_{cut}. \text{GlobalGhost}(\varsigma.\mathcal{M}, \varsigma.\mathcal{N}, V_{up}, V_{cut}) * \\ &\quad \varsigma \text{ is wellformed} * V_{up} \in \varsigma.\mathcal{M} * \varsigma.\mathcal{N} \sqsubseteq V_{cut} \\ \mathcal{I}_{\text{HIST}} &::= \boxed{\exists \varsigma. [\bullet \text{ex}(\varsigma)]^{\gamma_{\text{STATE}}} * \text{GlobalInv}(\varsigma)}^{\mathcal{N}_{\text{HIST}}} \\ S(\varsigma) &::= [\bullet \text{ex}(\varsigma) : \text{AUTH}(\text{EX}(\text{GlobalState}))]^{\gamma_{\text{STATE}}} * \mathcal{I}_{\text{HIST}} \end{aligned}$$

The global physical state  $\varsigma$ , together with the logical states  $V_{up}$  and  $V_{cut}$ , need to satisfy some basic properties, as written in the *global invariant*  $\text{GlobalInv}(\varsigma)$ :  $\varsigma$  is wellformed (**Property 3.15**),  $V_{up}$  is closed in  $\varsigma.\mathcal{M}$  (**Property 3.12**), and the cut view  $V_{cut}$  must be at least the race detector view  $\mathcal{N}$  to guarantee that accessing some history  $h$  is not racy.

The definition of the state interpretation is a bit peculiar. We would simply want  $S = \text{GlobalInv}$ . But recall that  $S$  is only accessible with a weakest pre-condition (**Definition 6.14**), so if we let  $S = \text{GlobalInv}$ , we can only prove rules with WPs. This is usually the case: an update to the physical state needs to be done by some instruction, whose rule will come with a WP, by which we can access the state interpretation  $S$  and then the global ghost state `GlobalGhost`, with which we can perform a ghost update to keep the ghost state and the physical state in sync.

However, what if we simply want to change the ghost state without changing the physical state, *i.e.*, a “view shift”? For example, the rule **BL-HIST-DROP-SINGLETON** is simply a logical move that changes the “view”<sup>23</sup> of the logic from a history  $h$  to a singleton, without changing the physical memory for  $\ell$ . To support such rules with not just WPs but also viewshifts, we put  $\text{GlobalInv}$  inside an invariant with the fixed namespace  $\mathcal{N}_{\text{HIST}}$ , and employ extra ghost state to maintain that the state  $\varsigma$  existentially quantified in the invariant  $\mathcal{I}_{\text{HIST}}$  is always exactly the parameter  $\varsigma$  of  $S(\varsigma)$ , which is the actual physical state. The RA  $\text{AUTH}(\text{EX}(\text{GlobalState}))$  ensures that the states agree:  $\text{valid}(\bullet \text{ex}(\varsigma) \cdot \bullet \text{ex}(\varsigma')) \Rightarrow \varsigma = \varsigma'$ .

Note that we also need another global ghost location  $\gamma_{\text{STATE}}$ , and for every viewshift  $\models_{\mathcal{E}}$  that wants to access  $\text{GlobalInv}$ , we implicitly assume that  $\mathcal{N}_{\text{HIST}} \subseteq \mathcal{E}$  and the invariant  $\mathcal{I}_{\text{HIST}}$  is known.

## 7.6 Proofs of Some Primitive Rules and Adequacy

Now, we show proof sketches of some base-logic rules. All rules have been proven and checked by Coq.

*Proof sketch of **BL-HIST-DROP-SINGLETON** (§7.2).* Assuming  $\mathcal{N}_{\text{HIST}} \subseteq \mathcal{E}$ , we use **INV-ACC** (§6.3) to open  $\mathcal{I}_{\text{HIST}}$ . Note that since the contents of  $\mathcal{I}_{\text{HIST}}$  are all timeless, the later we get after opening  $\mathcal{I}_{\text{HIST}}$  can be stripped off right away. We then use **BL-GHOST-UPDATE-HIST-DROP-SINGLETON** to truncate the history with a new cut view  $V'_{cut} \sqsupseteq V_{cut} \sqsupseteq \varsigma.\mathcal{N}$ . The invariant contents only change in  $V'_{cut}$ . We therefore can easily re-establish invariant and close it using the closing wand viewshift we get earlier from **INV-ACC**.  $\square$

<sup>23</sup>Note how the “view” in viewshifts is different from views of the relaxed memory machine.

*Proof sketch of BL-HOARE-ACQ-FENCE (§7.3.1).* We start by unfolding the definition of Hoare triples (Definition 6.7 or Notation 7.7), and then of WPs (Definition 6.14). Our goal then looks as follow.

Context: Goal:

$$\text{Seen}(\mathcal{V}) * S(\varsigma) \quad \varepsilon \Vdash^{\emptyset} (\text{red}((\mathbf{fence}_{\text{acq}}, \mathcal{V}), \varsigma) * \forall(e', \mathcal{V}'). \dots)$$

Since we have a fancy update  $\varepsilon \Vdash^{\emptyset}$  in the goal, we can open  $\mathcal{I}_{\text{HIST}}$ :

$$\{\bullet \text{ex}(\varsigma)\}^{\gamma_{\text{STATE}}} * \{\circ \text{ex}(\varsigma)\}^{\gamma_{\text{STATE}}} * \text{GlobalInv}(\varsigma) * (\dots \emptyset \Rightarrow^{\varepsilon} \dots)$$

The obligation  $\text{red}((\mathbf{fence}_{\text{acq}}, \mathcal{V}), \varsigma)$  is easily discharged.

For the remaining goal, after introducing assumptions, we know

$$(i) \varsigma \mid (\mathbf{fence}_{\text{acq}}, \mathcal{V}) \xrightarrow{\text{F}^{\text{acq}}, \perp}_{\text{t}} \varsigma \mid (\mathbf{fence}, \mathcal{V}')$$

$$\text{and the goal is} \quad \triangleright \emptyset \Vdash^{\varepsilon} (S(\varsigma) * \mathcal{V}' \sqsubseteq \mathcal{V} * \text{Seen}(\mathcal{V}') * \mathcal{V}'.\text{cur} = \mathcal{V}'.\text{acq})$$

Using BL-GHOST-SEEN, we know  $\mathcal{V}.\text{acq} \sqsubseteq V_{\text{up}}$ , so we can use

BL-GHOST-UPDATE-SEEN to get  $\text{Seen}(\mathcal{V}')$  without updating  $V_{\text{up}}$ .

We can then use the closing wand viewshift  $(\dots \emptyset \Rightarrow^{\varepsilon} \dots)$  to

close the invariant without updating  $\varsigma$ . The goal is now:

$$S(\varsigma) * \text{Seen}(\mathcal{V}') \quad S(\varsigma) * \mathcal{V}' \sqsubseteq \mathcal{V} * \text{Seen}(\mathcal{V}') * \mathcal{V}'.\text{cur} = \mathcal{V}'.\text{acq}$$

This is easily done by looking at the reduction (i) for acquire fences.  $\square$

*Proof sketch of BL-HOARE-WRITE-AT (§7.3.2).* We have the following assumptions: (1)  $\mathbf{r1x} \sqsubseteq o$ , (2)  $\text{Local}_A(\ell, h, \mathcal{V}.\text{cur})$ , (3)  $\text{Local}_R^{\text{na}}(\ell, \alpha_r, \mathcal{V}.\text{cur})$ , and (4)  $\text{Local}_W^{\text{r1x}}(\ell, \alpha_w, V_w)$ . After unfolding Hoare triple and WP definitions, we have the following goal.

Context: Goal:

$$\text{Seen}(\mathcal{V}) * \text{Hist}(\ell, h) * \text{Write}^{\text{r1x}}(\ell, \alpha_w) * \text{Read}^{\text{na}}(\ell, \alpha_r)$$

$$S(\varsigma) \quad \varepsilon \Vdash^{\emptyset} (\text{red}((\ell :=_o v', \mathcal{V}), \varsigma) * \forall(e', \mathcal{V}'). \dots)$$

We unfold  $S$  and open the invariant  $\mathcal{I}_{\text{HIST}}$ :

$$\{\bullet \text{ex}(\varsigma)\}^{\gamma_{\text{STATE}}} * \{\circ \text{ex}(\varsigma)\}^{\gamma_{\text{STATE}}} * \text{GlobalInv}(\varsigma) * (\dots \emptyset \Rightarrow^{\varepsilon} \dots)$$

$$\text{We first show safety:} \quad \text{red}((\ell :=_o v', \mathcal{V}), \varsigma)$$

By BL-GHOST-HIST and BL-GHOST-NA-READ, we have

$$\text{trunc}(\varsigma.\mathcal{M}(\ell), V_{\text{cut}}(\ell)) = h \wedge \ell \notin \text{unalloc}(\varsigma.\mathcal{M}) \wedge \varsigma.\mathcal{N}(\ell).\text{nr} = \alpha_r.$$

Combining these with (2), (3), and  $\varsigma.\mathcal{N} \sqsubseteq V_{\text{cut}}$ , we have

$$\varsigma.\mathcal{N}(\ell).\text{w} \leq \mathcal{V}.\text{cur}(\ell).\text{w} \wedge \varsigma.\mathcal{N}(\ell).\text{nr} \sqsubseteq \mathcal{V}.\text{cur}(\ell).\text{nr}.$$

So we satisfy DRF-WRITE-AT (§3.4), i.e., we are race-free.

Consequently, we satisfy OC-MEM (§4.3), so we are done.

For the remaining goal, after introducing assumptions, we know

$$(i) \varsigma \mid (\ell :=_o v', \mathcal{V}) \xrightarrow{W^o(\ell, v'), \perp}_{\text{t}} \varsigma' \mid (\mathbf{fence}, \mathcal{V}')$$

and the goal is

$$\triangleright \emptyset \Vdash^{\varepsilon} \left( \begin{array}{l} S(\varsigma') * \exists t' \notin \text{dom}(h), V'. \mathcal{V} \xrightarrow{W: o, \ell, t', \perp, V'} \mathcal{V}' * \\ \text{Local}_W^{\text{r1x}}(\ell, \alpha_w \cup \{t'\}, V_w \sqcup \mathcal{V}'.\text{cur}) * \\ \text{Seen}(\mathcal{V}') * \text{Hist}(\ell, h[t' \leftarrow (v', V')]) * \\ \text{Write}^{\text{r1x}}(\ell, \alpha_w \cup \{t'\}) * \text{Read}^{\text{na}}(\ell, \alpha_r) \end{array} \right)$$

By looking at the reduction (i), we can get the new timestamp  $t'$  and the write message view  $V'$ , and the fact  $\mathcal{V} \xrightarrow{W:o,\ell,t',\perp,V'} \mathcal{V}'$ , and that  $\mathcal{N}' = \mathcal{N}[\ell \leftarrow \{\mathcal{N}(\ell) [\text{aw} := \mathcal{N}(\ell).\text{aw} \cup \{t\}]\}]$ .

Additionally, by **BL-GHOST-AT-WRITE** we have  $\varsigma.\mathcal{N}(\ell).\text{aw} = \alpha_w$ . So we can pick a new cut view  $V'_{\text{cut}}$ , and use **BL-GHOST-UPDATE-AT-WRITE** to update both the history ownership to  $\text{Hist}(\ell, h[t' \leftarrow (v', V')])$  and the atomic write ownership to  $\text{Write}^{\exists \text{rlx}}(\ell, \alpha_w \cup \{t'\})$ . By using (4), we also discharge the local observation  $\text{Local}^{\exists \text{rlx}}_w(\ell, \alpha_w \cup \{t'\}, V_w \sqcup \mathcal{V}'.\text{cur})$ . Using **BL-GHOST-UPDATE-SEEN**, we also get  $\text{Seen}(\mathcal{V}')$ . The non-atomic read ownership  $\text{Read}^{\text{na}}(\ell, \alpha_r)$  is returned as-is.

We eventually end up with the following goal.

$$\begin{aligned} & \left[ \bullet \text{ex}(\varsigma) \right]^{\gamma_{\text{STATE}}} * \left[ \circ \text{ex}(\varsigma) \right]^{\gamma_{\text{STATE}}} * \text{GlobalInv}(\varsigma') * (\dots \multimap^{\mathcal{E}} \dots) \\ & \multimap^{\mathcal{E}} S(\varsigma') \end{aligned}$$

This is easily done by first updating the ghost ownership  $\left[ \bullet \text{ex}(\varsigma) \right]^{\gamma_{\text{STATE}}} * \left[ \circ \text{ex}(\varsigma) \right]^{\gamma_{\text{STATE}}}$  to that of  $\varsigma'$ , and then use the closing wand viewshift to close the invariant  $\mathcal{I}_{\text{HIST}}$ .  $\square$

Finally, we show adequacy for our base logic, which is slightly difference from **Theorem 6.8** in that we do not fix the initial state.

**Theorem 7.19** (Base Logic Adequacy). *Assuming that the state  $\varsigma$  is wellformed and a thread-view  $\mathcal{V}$  is closed in  $\varsigma.\mathcal{M}$  ( $\mathcal{V} \in \varsigma.\mathcal{M}$ ), if  $\vdash \text{wp}_{\top}(e, \mathcal{V}) \{(v, \_). \phi(v)\}$  is derivable in the base logic for  $\lambda_{\text{Rust}} + \text{ORC11}$  where  $\phi(v)$  is a pure (meta-level) fact, then the following holds.*

$$\begin{aligned} & \forall \pi, \mathcal{T}', \varsigma'. ([\pi \mapsto (e, \mathcal{V})], \varsigma) \rightarrow^* (\mathcal{T}', \varsigma') \Rightarrow \\ & \quad \forall v. \mathcal{T}(\pi) = v \Rightarrow \phi(v) \quad (\text{BL-ADEQUACY-VAL}) \\ & \wedge \quad \forall \rho, e_{\rho}, \mathcal{V}_{\rho}. \mathcal{T}(\rho) = (e_{\rho}, \mathcal{V}_{\rho}) \Rightarrow (e_{\rho} \text{ is a value} \vee \text{red}((e_{\rho}, \mathcal{V}_{\rho}), \varsigma')) \quad (\text{BL-ADEQUACY-NO-STUCK}) \end{aligned}$$

*Proof.* The proof follows from a Iris-provided adequacy theorem (that also implies **Theorem 6.8**). All we need to do is to allocate the various global ghost locations with the correct RAs, and establish the global invariant  $\mathcal{I}_{\text{HIST}}$  of the state interpretation  $S(\varsigma)$ , which requires  $\varsigma$ 's wellformedness and that  $\mathcal{V} \in \varsigma.\mathcal{M}$ .  $\square$

**CHAPTER SUMMARY.** In this chapter, we demonstrated the instantiation of Iris with the  $\lambda_{\text{Rust}} + \text{ORC11}$  language to achieve a RMC base logic. The most important feature of the logic is the explicit use of thread-views in conjunction with various local assertions to achieve abstraction for the relaxed memory effects. In the next chapter, we provide the next abstractions for thread-views. In **Chapter 9** and **Chapter 10**, we will provide more abstractions for the local assertions.

# 8

## vProp: View-monotone Predicates

Following iGPS<sup>1</sup>, in this chapter we introduce an abstraction to hide thread-views in the base logic, and lift the base logic to a surface-level logic whose propositions have the type vProp, which stands for *view propositions*. We call this surface logic of vProp the iRC11 logic. We will, in this chapter as well as later ones, develop more reasoning principles for iRC11, within iRC11 itself or on top of the base logic.

The motivation for hiding thread-views and views is that most of the time, they do not have interesting behaviors, and when they do (in the relaxed memory operations), the effects are usually that the thread-views or views have grown in certain ways. If we can provide new assertions that abstract those ways that views change (*e.g.*, an observation of a value written or read, or that a thread-view’s current view has been upgraded to its acquire view), then we can achieve SC-like rules that have been developed in many previous logics.<sup>2</sup> In this chapter, we establish some of such core rules for iRC11.

Note, however, that hiding views is an abstraction that *weaken* the logic. While such abstraction is sufficient in many cases, views are inevitable in order to provide strong reasoning principles or specifications for very relaxed algorithms. This observation has been made by the RB<sub>r1x</sub> work (Part III), the Cosmo logic,<sup>3</sup> and the Compass specifications (Part IV), chronologically. §8.5 will introduce several modalities to restore explicit view reasoning in the logic of vProp.

<sup>1</sup>Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

<sup>2</sup>Vafeiadis and Narayan, “Relaxed separation logic: a program logic for C11 concurrency” [VN13]; Doko and Vafeiadis, “A Program Logic for C11 Memory Fences” [DV16]; Doko and Vafeiadis, “Tackling Real-Life Relaxed Concurrency with FSL++” [DV17]; Turon et al., “GPS: navigating weak memory with ghosts, protocols, and separation” [TVD14].

<sup>3</sup>Mével et al., “Cosmo: a concurrent separation logic for multicore OCaml” [MJP20].

### 8.1 View-monotone Predicates

We define vProp as the type of view-monotone predicates over iProp.

**Definition 8.1** (vProp).

$$\begin{aligned} \text{vProp} &::= \text{View} \xrightarrow{\text{mon}} \text{iProp} \\ \text{satisfying } \forall P : \text{vProp}. \forall V, V'. V \sqsubseteq V' \Rightarrow P(V) \Rightarrow P(V') \quad (\text{VPROP-MONO}) \end{aligned}$$

An assertion  $P : \text{vProp}$  is to be interpreted as some resource that holds at a simple view. This view usually is the current component  $\mathcal{V}.\text{cur}$  of a thread  $\pi$ ’s thread-view  $\mathcal{V}$  in case  $P$  is owned locally by the thread  $\pi$ ; or a view of some write message  $m$  in case we attach  $P$  to the message  $m$  in order to transfer  $P$  from  $m$ ’s writer to its readers.

Another choice is to define vProp as a predicate of thread-views ( $ThreadView \xrightarrow{\text{mon}} iProp$ ), but such a definition does not have an actual use. Thread-views are tied locally to threads, and so such a definition is not suitable to represent resources that are not tied to a thread, but instead, for example, are tied to a message, or are put inside a shared invariant. Furthermore, resources are typically not tied to a whole thread-view  $\mathcal{V}$ , but rather to one of its components (the release-fence, current, or acquire view). In short, simple view predicates are used pervasively, while thread-view predicates are not.

The monotonicity requirement is needed to maintain “stability” of the frame when the view grows. That is, more observations made by a thread’s step should not invalidate resources that are not relevant to the step. As a result, we explicitly monotinize vProp propositions when necessary (i.e., when they are not already monotone). Note that this is a source of weakening the surface logic compared to the base logic.

We lift the many logical connectives and modalities of the base logic (which we inherit from Iris) straightforwardly.

**Definition 8.2** (Model of vProp propositions). The function  $\llbracket \cdot \rrbracket$  provides a model of vProp propositions into predicates from views to iProp, embedding proofs that the predicates are monotone.<sup>4</sup>

<sup>4</sup>This model is also useful elsewhere, and has been generalized in Iris to *monotone predicates* over types that come with a partial order.

$$\begin{aligned}
\llbracket \phi \rrbracket &::= \lambda \_ . \phi \\
\llbracket \text{False} \rrbracket &::= \lambda \_ . \text{False} \\
\llbracket \text{True} \rrbracket &::= \lambda \_ . \text{True} \\
\llbracket P \Rightarrow Q \rrbracket &::= \lambda V . \forall V' \sqsupseteq V . \llbracket P \rrbracket(V') \Rightarrow \llbracket Q \rrbracket(V') \\
\llbracket P \wedge Q \rrbracket &::= \lambda V . \llbracket P \rrbracket(V) \wedge \llbracket Q \rrbracket(V) \\
\llbracket P \vee Q \rrbracket &::= \lambda V . \llbracket P \rrbracket(V) \vee \llbracket Q \rrbracket(V) \\
\llbracket P * Q \rrbracket &::= \lambda V . \llbracket P \rrbracket(V) * \llbracket Q \rrbracket(V) \\
\llbracket P \multimap Q \rrbracket &::= \lambda V . \forall V' \sqsupseteq V . \llbracket P \rrbracket(V') \multimap \llbracket Q \rrbracket(V') \\
\llbracket \exists x . P \rrbracket &::= \lambda V . \exists x . \llbracket P \rrbracket(V) \\
\llbracket \forall x . P \rrbracket &::= \lambda V . \forall x . \llbracket P \rrbracket(V) \\
\llbracket \triangleright P \rrbracket &::= \lambda V . \triangleright \llbracket P \rrbracket(V) \\
\llbracket \Box P \rrbracket &::= \lambda V . \Box \llbracket P \rrbracket(V) \\
\llbracket \bar{a}^\gamma \rrbracket &::= \lambda \_ . \{\bar{a}\}^\gamma \\
\llbracket \dot{\Rightarrow} P \rrbracket &::= \lambda V . \dot{\Rightarrow} \llbracket P \rrbracket(V) \\
\llbracket \varepsilon_1 \Rightarrow^{\varepsilon_2} P \rrbracket &::= \lambda V . \varepsilon_1 \Rightarrow^{\varepsilon_2} \llbracket P \rrbracket(V) \\
&\dots
\end{aligned}$$

Note that,  $\llbracket P * Q \rrbracket$ , for example, is view-monotone assuming  $P$  and  $Q$  are vProp and thus view-monotone. On the other hand,  $\llbracket P \multimap Q \rrbracket$  needs to be monotinized explicitly. The model of ghost state ownership  $\llbracket \bar{a}^\gamma \rrbracket$  interestingly simply ignores the input view.

**Lemma 8.3** (Properties of iRC11 connectives). *The properties in Figure 6.2, Figure 6.3, Figure 6.4, and Figure 6.7 are preserved for iRC11 connectives by the encoding of Definition 8.2.*

The most interesting encodings are those of weakest pre-conditions (from which Hoare triples are derived similarly as before), non-atomic and atomic points-to assertions, and invariants. We will discuss non-atomic points-to in [Chapter 9](#), atomic points-to in [Chapter 10](#), and invariants in [Chapter 11](#). In the remaining, we discuss the models of iRC11 (vProp) weakest pre-conditions and several RMC-specific modalities.

## 8.2 Model of iRC11 Weakest Pre-conditions

iRC11 WPs are a vProp proposition that is built upon the base logic WPs, and that hides away thread-views. Nevertheless, we need a way to refer to the thread-view, specifically to define the *release* and *acquire* modalities (§8.5) that provide the abstraction of fence behaviors. For this purpose, we expose thread-ids in the WP definition, which are used under the hood to associate with thread-views. Fortunately, these iRC11-level thread-ids need not be the same thread-ids of the threadpool, so we can simply store thread-views in ghost state—with the RA  $\text{TVIEWR} = \text{AUTH}(\text{LAT}(\text{ThreadView}))$ , and use ghost locations as thread-ids.

**Definition 8.4** (iRC11 Weakest Pre-conditions).

$$\begin{aligned} \llbracket \text{wp}_{\mathcal{E}} e \text{ in } \pi \{ \Phi \} \rrbracket &::= \\ \lambda V. \forall \mathcal{V}. V \sqsubseteq \mathcal{V}. \text{cur} \multimap & \left[ \bullet \mathcal{V} : \text{TVIEWR} \right]^{\pi} \multimap \text{Seen}(\mathcal{V}) \multimap \\ & \text{wp}_{\mathcal{E}}(e, \mathcal{V}) \left\{ (v, \mathcal{V}') . \left[ \bullet \mathcal{V}' \right]^{\pi} \multimap \llbracket \Phi(v) \rrbracket (\mathcal{V}'.\text{cur}) \right\} \end{aligned}$$

The WP definition takes care of several things.

- It makes sure that the definition is view-monotone explicitly, by requiring that the underlying base logic WP take a thread-view  $\mathcal{V}$  whose current component  $\mathcal{V}.\text{cur}$  is in the upward closure of the input view  $V$ .
- It threads through the authoritative ghost ownership  $\left[ \bullet \mathcal{V} \right]^{\pi}$  of the thread-view being executed with the expression  $e$ , in the pre- and post-conditions. This also allows for creating snapshots (fragmentary ownership)  $\left[ \bullet \mathcal{V} \right]^{\pi}$  for the lower bound of the executing thread  $\pi$ 's thread-view, which in turn will be used to define release and acquire modalities.
- By hiding thread-views, it also hides the assertion  $\text{Seen}(\mathcal{V})$ . Accordingly, it also provides  $\text{Seen}(\mathcal{V})$  as assumption to the base logic WP. It does not require  $\text{Seen}(\mathcal{V}')$  in the post-condition, because this can be easily obtained from the state interpretation  $S$  (hidden in the base logic WP) using [BL-GHOST-UPDATE-SEEN](#).

**Definition 8.5** (iRC11 Hoare triples). iRC11 Hoare triples are defined similarly as before.

$$\{ P \} e \text{ in } \pi \{ v. Q \}_{\mathcal{E}} ::= \Box (P \multimap \text{wp}_{\mathcal{E}} e \text{ in } \pi \{ v. Q \})$$

If we interpret this definition in iProp, we will arrive at the following.

$$\begin{aligned} \llbracket \{P\} e \text{ in } \pi \{v. Q\} \varepsilon \rrbracket &::= \\ \lambda V. \Box \forall V', \mathcal{V}. V \sqsubseteq V' \sqsubseteq \mathcal{V}. \text{cur} &\multimap \llbracket P \rrbracket(V) \multimap [\bullet \mathcal{V}]^\pi \multimap \text{Seen}(\mathcal{V}) \\ &\multimap \text{wp}_{\mathcal{E}}(e, \mathcal{V}) \left\{ (v, \mathcal{V}'). [\bullet \mathcal{V}']^\pi \multimap \llbracket Q \rrbracket(\mathcal{V}'.\text{cur}) \right\} \end{aligned}$$

As one can see, generally both the pre-condition  $P$  and post-condition  $Q$  are interpreted at the current components  $\mathcal{V}. \text{cur}$  and  $\mathcal{V}'. \text{cur}$ , respectively, of the executing thread's thread-view.

**Lemma 8.6** (Properties of iRC11 WPs and Hoare triples). *The properties in Figure 6.5 and Figure 6.6, except those concerning invariants which are not yet defined, also hold for iRC11 WPs and Hoare triples.*

**Theorem 8.7** (iRC11 Adequacy). *If  $\forall \pi. \vdash \text{wp}_{\top} e \text{ in } \pi \{v. \phi(v)\}$  is derivable in the iRC11 logic for  $\lambda_{\text{Rust}} + \text{ORC11}$  where  $\phi(v)$  is a pure (meta-level) fact, then the following holds.*

$$\begin{aligned} \forall \pi, \mathcal{T}', \varsigma'. ([\pi \mapsto (e, \mathcal{V}_{\text{init}})], \varsigma_{\text{init}}) &\rightarrow^* (\mathcal{T}', \varsigma') \Rightarrow \\ \forall v. \mathcal{T}(\pi) = v &\Rightarrow \phi(v) && (\text{ADEQUACY-VAL}) \\ \wedge \quad \forall \rho, e_{\rho}, \mathcal{V}_{\rho}. \mathcal{T}(\rho) = (e_{\rho}, \mathcal{V}_{\rho}) &\Rightarrow (e_{\rho} \text{ is a value} \vee \text{red}((e_{\rho}, \mathcal{V}_{\rho}), \varsigma')) && (\text{ADEQUACY-NO-STUCK}) \end{aligned}$$

where  $\mathcal{V}_{\text{init}} = (\emptyset, \emptyset, \emptyset, \emptyset)$  and  $\varsigma_{\text{init}} = (\emptyset, \emptyset)$ .

*Proof sketch.* The proof follows from the base logic adequacy (Theorem 7.19). Note that the initial thread-view  $\mathcal{V}_{\text{init}}$  and state  $\varsigma_{\text{init}}$  are wellformed and  $\mathcal{V}_{\text{init}} \in \varsigma_{\text{init}}$ . We then need to allocate the ghost state  $[\bullet \mathcal{V}]^\pi$  for some  $\pi$  and  $\mathcal{V}$ , and get  $\text{Seen}(\mathcal{V})$ , so that we can instantiate and apply our assumption  $\forall \pi. \vdash \text{wp}_{\top} e \text{ in } \pi \{v. \phi(v)\}$  and finish the proof.  $\square$

### 8.3 Fence Modalities

To model the effects of relaxed accesses and fences, iRC11 inherits two modalities from FSL<sup>5</sup>—the *release* modality  $\Delta$  and the *acquire* modality  $\nabla$ —which allow us to talk about ownership of resources at a thread's release-fence or acquire views. The assertion  $\Delta_{\pi} P$  represents ownership of  $P$  at thread  $\pi$ 's release-fence view, while the assertion  $\nabla_{\pi} P$  represents ownership of  $P$  at thread  $\pi$ 's acquire view.

The motivation for these modalities as follows. Recall the Message-Passing example using a pair of a relaxed write and a relaxed read, together with fences (Example 2.1(d), Figure 2.2(d)). We have some resource described by the proposition  $P$  that we want to transfer from the left-hand thread  $\pi$  to the right-hand thread  $\rho$ . However, when the “producer” thread  $\pi$  performs its relaxed write, the message view of that write is drawn from  $\pi$ 's release-fence view, not its current view. Hence, we need a way of insisting (in the precondition of the relaxed write) that the  $P$  that  $\pi$  is sending holds under its release-fence view—that is what is denoted by  $\Delta_{\pi} P$ . Dually, when the “consumer” thread  $\rho$  performs its relaxed read, the message view it reads will only be joined into its

<sup>5</sup>Doko and Vafeiadis, “A Program Logic for C11 Memory Fences” [DV16]; Doko and Vafeiadis, “Tackling Real-Life Relaxed Concurrency with FSL++” [DV17].



|                                                                                                                               |                                                                                                                      |                                                                                                                                                                                             |                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>HOARE-REL-FENCE</b><br>$\{P\} \text{ fence}_{\text{rel}} \text{ in } \pi \{ \Delta_{\pi} P \}$                             |                                                                                                                      | <b>HOARE-ACQ-FENCE</b><br>$\{ \nabla_{\pi} P \} \text{ fence}_{\text{acq}} \text{ in } \pi \{ P \}$                                                                                         |                                                                                               |
| <b>HOARE-REL-FENCE-ELIM</b><br>$\frac{\{P\} e \text{ in } \pi \{ \Phi \}}{\{ \Delta_{\pi} P \} e \text{ in } \pi \{ \Phi \}}$ |                                                                                                                      | <b>HOARE-ACQ-FENCE-INTRO</b><br>$\frac{\{ \nabla_{\pi} P \} e \text{ in } \pi \{ \Phi \}}{\{ P \} e \text{ in } \pi \{ \Phi \}}$                                                            |                                                                                               |
| <b>RELMOD-GHOST</b><br>$\Delta_{\pi} [\bar{a}]^{\gamma} \vdash [\bar{a}]^{\gamma}$                                            | <b>ACQMOD-GHOST</b><br>$\nabla_{\pi} [\bar{a}]^{\gamma} \vdash [\bar{a}]^{\gamma}$                                   | <b>GHOST-RELMOD</b><br>$[\bar{a}]^{\gamma} \dot{\Rightarrow} \Delta_{\pi} [\bar{a}]^{\gamma}$                                                                                               | <b>GHOST-ACQMOD</b><br>$[\bar{a}]^{\gamma} \dot{\Rightarrow} \nabla_{\pi} [\bar{a}]^{\gamma}$ |
| <b>RELMOD-MONO</b><br>$\frac{P \vdash Q}{\Delta_{\pi} P \vdash \Delta_{\pi} Q}$                                               | <b>RELMOD-PURE</b><br>$\Delta_{\pi} \phi \vdash \phi$                                                                | <b>RELMOD-AND</b><br>$\Delta_{\pi} (P \wedge Q) \vdash \Delta_{\pi} P \wedge \Delta_{\pi} Q$                                                                                                | <b>RELMOD-OR</b><br>$\Delta_{\pi} (P \vee Q) \dashv\vdash \Delta_{\pi} P \vee \Delta_{\pi} Q$ |
| <b>RELMOD-FORALL</b><br>$\Delta_{\pi} \forall x. P \vdash \forall x. \Delta_{\pi} P$                                          | <b>RELMOD-EXIST</b><br>$\Delta_{\pi} \exists x. P \dashv\vdash \exists x. \Delta_{\pi} P$                            | <b>RELMOD-SEP</b><br>$\Delta_{\pi} (P * Q) \dashv\vdash \Delta_{\pi} P * \Delta_{\pi} Q$                                                                                                    |                                                                                               |
| <b>RELMOD-WAND</b><br>$\Delta_{\pi} (P \multimap Q) \vdash \Delta_{\pi} P \multimap \Delta_{\pi} Q$                           | <b>RELMOD-LATER-INTRO</b><br>$\triangleright \Delta_{\pi} P \Rightarrow_{\mathcal{E}} \Delta_{\pi} \triangleright P$ | <b>RELMOD-UNOPS</b><br>$\frac{\cdot \in \{ \square, \triangleright, \dot{\Rightarrow}, \mathcal{E}_1 \dot{\Rightarrow} \mathcal{E}_2 \}}{\Delta_{\pi} \cdot P \vdash \cdot \Delta_{\pi} P}$ |                                                                                               |
| <b>ACQMOD-MONO</b><br>$\frac{P \vdash Q}{\nabla_{\pi} P \vdash \nabla_{\pi} Q}$                                               | <b>ACQMOD-PURE</b><br>$\nabla_{\pi} \phi \vdash \phi$                                                                | <b>ACQMOD-AND</b><br>$\nabla_{\pi} (P \wedge Q) \vdash \nabla_{\pi} P \wedge \nabla_{\pi} Q$                                                                                                | <b>ACQMOD-OR</b><br>$\nabla_{\pi} (P \vee Q) \dashv\vdash \nabla_{\pi} P \vee \nabla_{\pi} Q$ |
| <b>ACQMOD-FORALL</b><br>$\nabla_{\pi} \forall x. P \vdash \forall x. \nabla_{\pi} P$                                          | <b>ACQMOD-EXIST</b><br>$\nabla_{\pi} \exists x. P \dashv\vdash \exists x. \nabla_{\pi} P$                            | <b>ACQMOD-SEP</b><br>$\nabla_{\pi} (P * Q) \dashv\vdash \nabla_{\pi} P * \nabla_{\pi} Q$                                                                                                    |                                                                                               |
| <b>ACQMOD-WAND</b><br>$\nabla_{\pi} (P \multimap Q) \vdash \nabla_{\pi} P \multimap \nabla_{\pi} Q$                           | <b>ACQMOD-LATER-INTRO</b><br>$\triangleright \nabla_{\pi} P \Rightarrow_{\mathcal{E}} \nabla_{\pi} \triangleright P$ | <b>ACQMOD-UNOPS</b><br>$\frac{\cdot \in \{ \square, \triangleright, \dot{\Rightarrow}, \mathcal{E}_1 \dot{\Rightarrow} \mathcal{E}_2 \}}{\nabla_{\pi} \cdot P \vdash \cdot \nabla_{\pi} P}$ |                                                                                               |

FIGURE 8.1: iRC11 rules for fence modalities

acquire view, not its current view. Hence, we need a way of insisting (in the post-condition of the relaxed read) that  $\rho$  only receives ownership of  $P$  under its acquire view—that is what is denoted by  $\nabla_{\rho} P$ . We will see how this is materialized in the iRC11 rules for atomic operations in Chapter 10.

Of course, we need a way of actually *introducing*  $\Delta_{\pi} P$  and *eliminating*  $\nabla_{\rho} P$ . These steps are achieved by rules **HOARE-REL-FENCE** and **HOARE-ACQ-FENCE** (Figure 8.1), which allow one to transfer any proposition *into the release modality* at the point of a **rel** fence, or *out of the acquire modality* at the point of an **acq** fence, because those are the points where the current and release-fence/acquire views get synchronized.

**HOARE-REL-FENCE-ELIM** and **HOARE-ACQ-FENCE-INTRO** are the reverse of **HOARE-REL-FENCE** and **HOARE-ACQ-FENCE**, and demonstrate that the release-fence view is included in the current view, which in turn is included in the acquire view of a thread. So  $\Delta_{\pi} P$  can be easily turned into  $P$ , which can be turned into  $\nabla_{\pi} P$ . We note that we need a goal in form of a WP or a Hoare triple to perform these moves, but this is only an artifact of

our simple model of fence modalities (§8.3.1).

**RELMOD-GHOST**, **ACQMOD-GHOST**, **GHOST-RELMOD**, and **GHOST-ACQMOD** together state that the ghost ownership assertion  $\llbracket a \rrbracket^\gamma$  can move freely in and out of the fence modalities. Intuitively, ghost state belongs to the class of *view-agnostic* assertions, in the sense that their ownership interpretation is *not tied to any view at all!* Since  $\llbracket a \rrbracket^\gamma$  is view-agnostic and thus does not care at which view it is interpreted,<sup>6</sup> it is equivalent to  $\Delta_\pi \llbracket a \rrbracket^\gamma$  or  $\nabla_\pi \llbracket a \rrbracket^\gamma$ . As a result,  $\llbracket a \rrbracket^\gamma$  can be transferred from one thread to another *without the need for physical synchronization*—in particular, without the need for release/acquire fences.

The remaining rules in Figure 8.1 state various properties between the fence modalities and other modalities. Some of the rules only have one direction or need to use basic or fancy updates. This is due to our simple model of fence modalities—which we will see next—but fortunately they do not cause any problem in practice.

### 8.3.1 Model of the Fence Modalities

We rely on the extra ghost state of the RA TVIEWR that we have added to iRC11 WPs (Definition 8.4) to get access to the executing thread's hidden thread-view, so that we can give a model to our fence modalities.

**Definition 8.8** (Model of the Fence Modalities).

$$\llbracket \Delta_\pi P \rrbracket ::= \lambda_. \exists \mathcal{V}. \llbracket \bullet \mathcal{V} \rrbracket^\pi * \llbracket P \rrbracket(\mathcal{V}.frel) \quad (\text{RELMOD-MODEL})$$

$$\llbracket \nabla_\pi P \rrbracket ::= \lambda_. \exists \mathcal{V}. \llbracket \bullet \mathcal{V} \rrbracket^\pi * \llbracket P \rrbracket(\mathcal{V}.acq) \quad (\text{ACQMOD-MODEL})$$

Note that due to validity of TVIEWR, from  $\llbracket \bullet \mathcal{V}' \rrbracket^\pi * \llbracket \bullet \mathcal{V} \rrbracket^\pi$ , we know that  $\mathcal{V} \sqsubseteq \mathcal{V}'$ . Consequently, if we own  $\Delta_\pi P$  in a goal of a WP for the thread  $\pi$ , we know that  $P$  holds at the view  $\mathcal{V}.frel$  where  $\mathcal{V} \sqsubseteq \mathcal{V}'$  and  $\mathcal{V}'$  is  $\pi$ 's actual thread-view, and thus by view-monotonicity,  $P$  also holds at  $\mathcal{V}'.frel$ . In fact, let us sketch the proofs of some of the rules in Figure 8.1.

*Proof of HOARE-REL-FENCE.* We prove the rule in the base logic. After unfolding the Hoare triples (interpreting them in the base logic, as in Definition 8.5), we have the following goal.

|                                                                                                                                         |                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Context:                                                                                                                                | Goal:                                                                                                                                                                         |
| $V \sqsubseteq \mathcal{V}.cur * \llbracket P \rrbracket(V) * \llbracket \bullet \mathcal{V} \rrbracket^\pi * \text{Seen}(\mathcal{V})$ | $\text{wp}_E(e, \mathcal{V}) \left\{ (\bullet, \mathcal{V}'). \llbracket \bullet \mathcal{V}' \rrbracket^\pi * \llbracket \Delta_\pi P \rrbracket(\mathcal{V}'.cur) \right\}$ |

We then apply **WP-MONO** (§6.6) and **BL-HOARE-REL-FENCE** (§7.3.1).

The goal, after unfolding the model of the release modality, is now:

$$V \sqsubseteq \mathcal{V}.cur \wedge \mathcal{V} \sqsubseteq \mathcal{V}' \wedge \mathcal{V}'.frel = \mathcal{V}'.cur$$

$$\frac{\llbracket P \rrbracket(V) * \llbracket \bullet \mathcal{V} \rrbracket^\pi * \text{Seen}(\mathcal{V}')}{\llbracket \bullet \mathcal{V}' \rrbracket^\pi * \exists \mathcal{V}_0. \llbracket \bullet \mathcal{V}_0 \rrbracket^\pi * \llbracket P \rrbracket(\mathcal{V}_0.frel)}$$

We update the ghost thread-view of  $\pi$  using **AUTH-LAT-UPDATE** (§7.4).

We are then left with:

$$\llbracket P \rrbracket(V) * \llbracket \bullet \mathcal{V}' \rrbracket^\pi \quad \exists \mathcal{V}_0. \llbracket \bullet \mathcal{V}_0 \rrbracket^\pi * \llbracket P \rrbracket(\mathcal{V}_0.frel)$$

And then:

$$\llbracket P \rrbracket(V) \quad \llbracket P \rrbracket(\mathcal{V}'.frel)$$

<sup>6</sup>This can be seen clearly in the model of iRC11 ghost ownership, in Definition 8.2.

But we know that  $V \sqsubseteq \mathcal{V}.cur \sqsubseteq \mathcal{V}'.cur = \mathcal{V}'.frel$ . By view-monotonicity (**VPROP-MONO**), we are done.  $\square$

*Proof of **HOARE-ACQ-FENCE**.* The proof is similar to that of **HOARE-REL-FENCE**. Eventually we will arrive at the goal  $\llbracket P \rrbracket(\mathcal{V}_0.acq) \vdash \llbracket P \rrbracket(\mathcal{V}'.cur)$ . But by **AUTH-LAT-VALID** and **BL-HOARE-ACQ-FENCE** we know that  $\mathcal{V}_0.acq \sqsubseteq \mathcal{V}'.acq = \mathcal{V}'.cur$ . Again this is done by **VPROP-MONO**.  $\square$

*Proof of **RELMOD-GHOST**.* After unfolding, we arrive at the following base-logic goal:  $\exists \mathcal{V}. [\![\circ \mathcal{V}]\!]^\pi * [\![a]\!]^\gamma \vdash [\![a]\!]^\gamma$ . This is easily done.  $\square$

*Proof of **GHOST-RELMOD**.* After unfolding, we arrive at the following base-logic goal:  $[\![a]\!]^\gamma \Rightarrow \exists \mathcal{V}. [\![\circ \mathcal{V}]\!]^\pi * [\![a]\!]^\gamma$ . We only need to pick any  $\mathcal{V}$  for which we own  $[\![\circ \mathcal{V}]\!]^\pi$ , because the ghost ownership does not depend on  $\mathcal{V}$ . With a basic update, we can get ownership of an RA's unit element. In case of **TVIEWR**, the unit element is  $\circ \emptyset$  for the empty thread-view  $\emptyset$ . Therefore we can easily get  $[\![\circ \emptyset]\!]^\pi$  and we are done.  $\square$

*Proof of **RELMOD-FORALL**.* After unfolding, we arrive at the following base-logic goal:  $\exists \mathcal{V}. [\![\circ \mathcal{V}]\!]^\pi * \forall x. \llbracket P \rrbracket(\mathcal{V}.frel) \vdash \forall x. \exists \mathcal{V}. [\![\circ \mathcal{V}]\!]^\pi * \llbracket P \rrbracket(\mathcal{V}.frel)$ . This is easily done.

The result of the unfolding also demonstrates that the reverse direction **RELMOD-FORALL** is not provable for our simple model of the release modality: we would need to go from a  $\forall \exists$  assumption to a  $\exists \forall$  goal.  $\square$

## 8.4 Objective Propositions and The Objective Modality

We previously mentioned that ghost state ownership belongs to the class of view-agnostic propositions whose interpretations are not tied to any view at all. That is, relaxed memory has no effects on them. We formally call this class *objective* propositions, because they hold regardless of any subjective views of any threads in the program. They are thus important to establish global consensus among concurrent threads.

**Definition 8.9** (Objective Propositions). A proposition  $P : \text{vProp}$  is objective if its interpretation does not depend any view.<sup>7</sup>

$$\text{objective}(P) ::= \forall V, V'. \llbracket P \rrbracket(V) \vdash \llbracket P \rrbracket(V')$$

<sup>7</sup>Note that objectivity and the objective modality have also been generalized in Iris for monotone predicates, not just  $\text{vProp}$ .

**Definition 8.10** (The Objective Modality). The objective modality carries the proof that some resource  $P$  holds at any view.

$$\llbracket \langle \text{obj} \rangle P \rrbracket ::= \lambda \_ . \forall V. \llbracket P \rrbracket(V)$$

**Figure 8.2** presents many rules for objective propositions and the objective modality. Unsurprisingly, pure facts, True, False, ghost ownership, and a resource under the objective modality are all objective. Objectivity is maintained structurally, but it is not always so for the objective modality, due to our use of a universal quantifier ( $\forall$ ) in its model. **OBJMOD-INTRO** allows one to put objective propositions under the objective modality, so that one can store the meta-level objectivity

|                                                                                                                                                               |                                                                                                                                              |                                                                                                                                    |                                                                                                                                                                                             |                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>GHOST-OBJ</b><br>$\text{objective}(\ulcorner a \urcorner^\gamma)$                                                                                          | <b>PURE-OBJ</b><br>$\text{objective}(\phi)$                                                                                                  | <b>TRUE-OBJ</b><br>$\text{objective}(\text{True})$                                                                                 | <b>FALSE-OBJ</b><br>$\text{objective}(\text{False})$                                                                                                                                        | <b>OBJ-OBJ</b><br>$\text{objective}(\langle \text{obj} \rangle P)$ |
| <b>OBJ-BOPS</b><br>$\frac{\text{objective}(P) \quad \text{objective}(Q) \quad \cdot \in \{\wedge, \vee, \Rightarrow, *, \neg\}}{\text{objective}(P \cdot Q)}$ |                                                                                                                                              |                                                                                                                                    | <b>OBJ-UOPS</b><br>$\frac{\text{objective}(P) \quad \cdot \in \left\{ \square, \triangleright, \dot{\equiv}, \varepsilon_1 \dot{\equiv} \varepsilon_2 \right\}}{\text{objective}(\cdot P)}$ |                                                                    |
| <b>OBJ-FORALL</b><br>$\frac{\forall x. \text{objective}(P)}{\text{objective}(\forall x. P)}$                                                                  | <b>OBJ-EXIST</b><br>$\frac{\exists x. \text{objective}(P)}{\text{objective}(\exists x. P)}$                                                  | <b>OBJMOD-INTRO</b><br>$\frac{\text{objective}(P)}{P \vdash \langle \text{obj} \rangle P}$                                         | <b>OBJMOD-ELIM</b><br>$\langle \text{obj} \rangle P \vdash P$                                                                                                                               |                                                                    |
| <b>OBJMOD-MONO</b><br>$\frac{P \vdash Q}{\langle \text{obj} \rangle P \vdash \langle \text{obj} \rangle Q}$                                                   | <b>OBJMOD-AND</b><br>$\langle \text{obj} \rangle (P \wedge Q) \dashv\vdash \langle \text{obj} \rangle P \wedge \langle \text{obj} \rangle Q$ | <b>OBJMOD-OR</b><br>$\langle \text{obj} \rangle P \vee \langle \text{obj} \rangle Q \vdash \langle \text{obj} \rangle (P \vee Q)$  |                                                                                                                                                                                             |                                                                    |
| <b>OBJMOD-FORALL</b><br>$\langle \text{obj} \rangle \forall x. P \dashv\vdash \forall x. \langle \text{obj} \rangle P$                                        | <b>OBJMOD-EXIST</b><br>$\exists x. \langle \text{obj} \rangle P \vdash \langle \text{obj} \rangle \exists x. P$                              | <b>OBJMOD-SEP</b><br>$\langle \text{obj} \rangle (P * Q) \dashv\vdash \langle \text{obj} \rangle P * \langle \text{obj} \rangle Q$ |                                                                                                                                                                                             |                                                                    |
| <b>OBJMOD-RELMOD-INTRO</b><br>$\langle \text{obj} \rangle P \dot{\equiv} \Delta_\pi P$                                                                        | <b>RELMOD-OBJMOD-ELIM</b><br>$\Delta_\pi \langle \text{obj} \rangle P \vdash \langle \text{obj} \rangle P$                                   | <b>OBJMOD-ACQMOD-INTRO</b><br>$\langle \text{obj} \rangle P \dot{\equiv} \nabla_\pi P$                                             | <b>ACQMOD-OBJMOD-ELIM</b><br>$\nabla_\pi \langle \text{obj} \rangle P \vdash \langle \text{obj} \rangle P$                                                                                  |                                                                    |

FIGURE 8.2: iRC11 rules for objective propositions and the objective modality

fact in the logic. **OBJMOD-ELIM** says that a resource  $P$  under an objective modality can be used any time, because it holds at any view.

Last but not least, **OBJMOD-RELMOD-INTRO**, **RELMOD-OBJMOD-ELIM**, **OBJMOD-ACQMOD-INTRO**, and **ACQMOD-OBJMOD-ELIM** together state that resources under the objective modality move freely in and out of the fence modalities, because they do not depend on any view. In fact, the rules for ghost state interaction with fence modalities (**RELMOD-GHOST**, **ACQMOD-GHOST**, **GHOST-RELMOD**, and **GHOST-ACQMOD**) are derived from these rules, together with **GHOST-OBJ**, **OBJMOD-INTRO**, and **OBJMOD-ELIM**.

*Note 8.11* (On the objectivity of fence modalities). A resource  $P$  under a fence modality, e.g.,  $\Delta_\pi P$ , is objective, but that does not mean that  $P$  is objective.  $P$  is still interpreted at some snapshot view of the thread  $\pi$ 's thread-view.

## 8.5 View-explicit Modalities

As mentioned in the beginning of this chapter—and as will be demonstrated in later chapters, it is not always desirable to hide views. We therefore would like the ability to *briefly* perform explicit view reasoning *without* dropping back to the base logic. The solution is to introduce view-explicit modalities. This has been done on an ad hoc basis in the  $\text{RB}_{\text{r1x}}$  work (Part III), then developed more formally by the Cosmo logic,<sup>8</sup> and then used extensively in Compass (Part IV).

In the following, we present a formal account of these modalities, and their interaction with other modalities as well as among themselves. Note that this formalization can also be generalized further beyond vProp, to achieve modalities in a logic with thread-local state.

<sup>8</sup>Mével et al., “Cosmo: a concurrent separation logic for multicore OCaml” [MJP20].

**Definition 8.12** (The View-Seen Observations).

 $\boxed{\sqsupseteq V}$ 

The view-seen observation  $\sqsupseteq V$  asserts that the implicit view that is being used to interpret resources is at least  $V$ .

$$\llbracket \sqsupseteq V \rrbracket ::= \lambda V'. V \sqsubseteq V'$$

The view-seen observation is similar to the seen thread-view observation (Definition 7.9), but is a  $\text{vProp}$  proposition and is not limited to a view of some thread. It provides a lower bound of the view being used to interpret resources at the current point of a proof, regardless whether that proof is for a program execution (a WP) or not.

**Definition 8.13** (The View-At Modality).

 $\boxed{@_V P}$ 

The view-at modality  $@_V P$  asserts that  $P$  holds explicitly (at least) at the view  $V$ .

$$\llbracket @_V P \rrbracket ::= \lambda \_. \llbracket P \rrbracket(V)$$

When we progress through a proof—with or without a program execution (i.e., a WP)—in the iRC11 logic, either due to program execution or due to possible explicit monotonicization of  $\text{vProp}$  propositions, the view being used to interpret our resources may grow. The view-at modality  $@_V P$  allows us to keep some resource  $P$  *frozen* at some view  $V$  and not affected by the growth of the implicit interpreting view. This ability is needed in case the interpreting view grows too big, rendering our ownership of  $P$  useless.

**Definition 8.14** (The View-Join Modality).

 $\boxed{\sqcup_V P}$ 

The view-join modality  $\sqcup_V P$  asserts that  $P$  holds at the join of  $V$  and the implicit view that is being used to interpret resources.

$$\llbracket \sqcup_V P \rrbracket ::= \lambda V'. \llbracket P \rrbracket(V' \sqcup V)$$

The view-join modality is a compromise between a implicit view and a view-at modality: it remembers the difference between the implicit interpreting view and the view that justifies  $P$ . This allows the view that justifies  $P$  to still grow, but not too far away from the implicit view of the current proof.

Figure 8.3 lists several important properties of these new propositions.

- The seen-view observation is timeless and persistent. The observation for the empty view is always available (**VS-BOT**) and objective. The seen-view observation lifts the join operation of the view lattice to separation in the logic (**VS-JOIN**). Observations are also downward closed (**VS-MONO**).
- The view-at modality makes the interpreting view explicit and therefore is objective. It preserves timelessness and persistency, and is upward closed (**VA-MONO**), due to view monotonicity. The modality commutes with most connectives and modalities, almost in both directions (**VA-BOPS**, **VA-UNOPS**, **VA-IMPL**, and **VA-WAND**).

|                                                                                                                                                                                                              |                                                                                                      |                                                                                                                                        |                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VS-BOT<br>$\vdash \sqsubseteq \emptyset$                                                                                                                                                                     | objective( $\sqsubseteq \emptyset$ )<br>timeless( $\sqsubseteq V$ )<br>persistent( $\sqsubseteq V$ ) | VS-JOIN<br>$\sqsubseteq (V_1 \sqcup V_2) \dashv\vdash \sqsubseteq V_1 * \sqsubseteq V_2$                                               | VS-MONO<br>$\frac{V_1 \sqsubseteq V_2}{\sqsubseteq V_2 \vdash \sqsubseteq V_1}$                                                                                                            |
| objective( $@_V P$ )                                                                                                                                                                                         | $\frac{\text{timeless}(P)}{\text{timeless}(@_V P)}$                                                  | $\frac{\text{persistent}(P)}{\text{persistent}(@_V P)}$                                                                                | VA-MONO<br>$\frac{V_1 \sqsubseteq V_2 \quad P \vdash Q}{@_{V_1} P \vdash @_{V_2} Q}$                                                                                                       |
| $\frac{\text{objective}(P)}{\text{objective}(\sqcup_V P)}$                                                                                                                                                   | $\frac{\text{timeless}(P)}{\text{timeless}(\sqcup_V P)}$                                             | $\frac{\text{persistent}(P)}{\text{persistent}(\sqcup_V P)}$                                                                           | VJ-MONO<br>$\frac{V_1 \sqsubseteq V_2 \quad P \vdash Q}{\sqcup_{V_1} P \vdash \sqcup_{V_2} Q}$                                                                                             |
| VA-INTRO<br>$P \vdash \exists V. \sqsubseteq V * @_V P$                                                                                                                                                      | VA-INTRO-INCL<br>$P * \sqsubseteq V' \vdash \exists V \sqsubseteq V'. \sqsubseteq V * @_V P$         | VA-ELIM<br>$\sqsubseteq V * @_V P \vdash P$                                                                                            | VA-VS<br>$V_1 \sqsubseteq V_2 \dashv\vdash @_{V_2} \sqsubseteq V_1$                                                                                                                        |
| VA-OBJ<br>$\frac{\text{objective}(P)}{@_V P \dashv\vdash P}$                                                                                                                                                 | VA-INTRO-OBJ<br>$\langle \text{obj} \rangle P \vdash @_V P$                                          | VA-IDEMP<br>$@_{V_2} @_{V_1} P \dashv\vdash @_{V_1} P$                                                                                 | VA-BOPS<br>$\frac{\cdot \in \{\wedge, \vee, *\}}{@_V P \cdot Q \dashv\vdash @_V P \cdot @_V Q}$                                                                                            |
| VA-UNOPS<br>$\frac{\cdot \in \{\forall x., \exists x., \square, \triangleright, \dot{\equiv}, \varepsilon_1 \dot{\equiv} \varepsilon_2\}}{@_V \cdot P \dashv\vdash \cdot @_V P}$                             | VA-IMPL<br>$@_V P \Rightarrow Q \vdash @_V P \Rightarrow @_V Q$                                      | VA-WAND<br>$@_V P * Q \vdash @_V P * @_V Q$                                                                                            |                                                                                                                                                                                            |
| VJ-UNFOLD<br>$\sqcup_V P \dashv\vdash \sqsubseteq V \Rightarrow P$                                                                                                                                           | VJ-JOIN<br>$\sqcup_{V_1 \sqcup V_2} P \dashv\vdash \sqcup_{(V_1 \sqcup V_2)} P$                      | VA-VJ<br>$@_{V_1 \sqcup V_2} P \dashv\vdash @_{(V_1 \sqcup V_2)} P$                                                                    | VJ-VA<br>$\sqcup_{V_2} @_{V_1} P \dashv\vdash @_{V_1} P$                                                                                                                                   |
| VA-TO-VJ<br>$@_V P \vdash \sqcup_V P$                                                                                                                                                                        | VJ-INTRO-NOW<br>$P \vdash \sqcup_V P$                                                                | VJ-ELIM<br>$\sqsubseteq V * \sqcup_V P \vdash P$                                                                                       | VJ-ELIM-VA<br>$\sqsubseteq V_1 * \sqcup_{V_2} P \vdash \exists V' \sqsubseteq V_1. \sqsubseteq V' * @_{(V' \sqcup V_2)} P$                                                                 |
| VJ-VA-ACC<br>$\sqsubseteq V_1 * \sqcup_{V_2} P \vdash \exists V' \sqsubseteq V_1. \sqsubseteq V' * @_{(V' \sqcup V_2)} P * (\forall V''. \sqsubseteq V'' * @_{(V'' \sqcup V_2)} P \multimap \sqcup_{V_2} P)$ |                                                                                                      |                                                                                                                                        |                                                                                                                                                                                            |
| VJ-INTRO-OBJ<br>$\langle \text{obj} \rangle P \vdash \sqcup_V P$                                                                                                                                             | VJ-OBJ<br>$\frac{\text{objective}(P)}{\sqcup_V P \dashv\vdash P}$                                    | VJ-BOPS<br>$\frac{\cdot \in \{\wedge, \vee, \Rightarrow, *, \multimap\}}{\sqcup_V P \cdot Q \dashv\vdash \sqcup_V P \cdot \sqcup_V Q}$ | VJ-UNOPS<br>$\frac{\cdot \in \{\forall x., \exists x., \square, \triangleright, \dot{\equiv}, \varepsilon_1 \dot{\equiv} \varepsilon_2\}}{\sqcup_V \cdot P \dashv\vdash \cdot \sqcup_V P}$ |

FIGURE 8.3: iRC11 rules for view-explicit modalities

Objective propositions often ignore the view-at modality (VA-OBJ and VA-INTRO-OBJ). VA-IDEMP says that the inner-most view-at modality dominates. VA-VS says what it means for a view  $V_2$  to observe a view  $V_1$ : it is simply that  $V_1 \sqsubseteq V_2$ .

The two most important rules for the modality are its introduction and elimination rules. VA-INTRO allows us to freeze an owned resource  $P$  at some view  $V$  that we have observed ( $\sqsubseteq V$ ). As such, we can send  $@_V P$  and  $\sqsubseteq V$  away on different routes—a *separation of resources and observations*. A receiver once receives both parts can use VA-ELIM to regain  $P$ . VA-INTRO-INCL strengthens VA-INTRO to know more about the fixed view  $V$ .

- The view-join modality preserves timelessness, persistency, and

objectivity, and is also upward closed. It commutes with most connectives and modalities (**VJ-BOPS** and **VJ-UNOPS**). Again, objective propositions ignore the view-join modality (**VJ-OBJ** and **VJ-INTRO-OBJ**).

**VJ-UNFOLD** provides an alternative definition for the view-join modality, which states more clearly that  $P$  holds at view whose difference with the implicit view is  $V$ . **VA-VJ**, **VJ-VA**, and **VA-TO-VJ** provide important relations between the view-at and view-join modalities. **VJ-INTRO-NOW** allows us to move the owned  $P$  to a bigger view and introduce  $\sqcup_V P$ . **VJ-ELIM** allows us to eliminate the modality, in the same way as **VA-ELIM**. Finally, **VJ-ELIM-VA** allows us to go from the view-join modality to the view-at modality. Combining that with **VA-VJ** and **VA-ELIM**, we get the rule **VJ-VA-ACC** that allows us to switch between the two modalities.

**Definition 8.15** (Alternative Model for Fence Modalities). The fence modalities are in fact defined in  $\text{vProp}$  using the view-at modality.

$$\begin{aligned}\Delta_\pi P &::= \exists V. [\circ \mathcal{V}]^\pi * @_{V.\text{frel}} P \\ \nabla_\pi P &::= \exists V. [\circ \mathcal{V}]^\pi * @_{V.\text{acq}} P\end{aligned}$$

After unfolding into the base logic, this is exactly the same as **Definition 8.8**.

We note that the release modality and the view-at modality can interact through the following rule.

$$\frac{\text{RELMOD-VA-REVERT} \quad \forall V. \{ @_V P * \Delta_\pi \sqsupseteq V \} e \text{ in } \pi \{ \Phi \}_\mathcal{E}}{\{ \Delta_\pi P \} e \text{ in } \pi \{ \Phi \}_\mathcal{E}}$$

That is, with a goal in the form of a WP for the thread  $\pi$ , we can turn the assumption  $\Delta_\pi P$  into  $@_V P * \Delta_\pi \sqsupseteq V$  for some view  $V$ .

## 8.6 The Subjective Modality

Finally, we introduce the subjective modality, a derivation from the view-at modality.

**Definition 8.16** (The Subjective Modality).

$$\langle \text{subj} \rangle P ::= \exists V. @_V P$$

That is, the subjective modality asserts that  $P : \text{vProp}$  holds at *some* view that is hidden from others. The name “subjective” comes from the fact that  $P$  holds in someone’s subjective view.

The subjective modality satisfies the rules in **Figure 8.4**, which are derivable from the rules for the view-at modality. Some of these properties hold for general monotone predicates, but some (e.g., the reverse direction of **SUBJMOD-SEP**) only hold for monotone predicates on a lattice, which for  $\text{vProp}$  is the view lattice.

|                                                                                                                                                  |                                                                                                                                             |                                                                                                                          |                                                                                                                                                 |                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| $\text{objective}(\langle \text{subj} \rangle P)$                                                                                                | $\frac{\text{timeless}(P)}{\text{timeless}(\langle \text{subj} \rangle P)}$                                                                 | $\frac{\text{persistent}(P)}{\text{persistent}(\langle \text{subj} \rangle P)}$                                          | $\frac{\text{SUBJMOD-MONO} \quad P \vdash Q}{\langle \text{subj} \rangle P \vdash \langle \text{subj} \rangle Q}$                               | $\frac{\text{SUBJMOD-INTRO}}{P \vdash \langle \text{subj} \rangle P}$ |
| $\frac{\text{SUBJMOD-VA}}{\exists V. @_V P \dashv\vdash \langle \text{subj} \rangle P}$                                                          |                                                                                                                                             | $\frac{\text{SUBJMOD-ELIM-OBJ} \quad \text{objective}(P)}{\langle \text{subj} \rangle P \vdash P}$                       | $\frac{\text{SUBJMOD-AND}}{\langle \text{subj} \rangle (P \wedge Q) \vdash \langle \text{subj} \rangle P \wedge \langle \text{subj} \rangle Q}$ |                                                                       |
| $\frac{\text{SUBJMOD-OR}}{\langle \text{subj} \rangle P \vee \langle \text{subj} \rangle Q \dashv\vdash \langle \text{subj} \rangle (P \vee Q)}$ |                                                                                                                                             | $\frac{\text{SUBJMOD-FORALL}}{\langle \text{subj} \rangle \forall x. P \vdash \forall x. \langle \text{subj} \rangle P}$ | $\frac{\text{SUBJMOD-EXIST}}{\exists x. \langle \text{subj} \rangle P \dashv\vdash \langle \text{subj} \rangle \exists x. P}$                   |                                                                       |
|                                                                                                                                                  | $\frac{\text{SUBJMOD-SEP}}{\langle \text{subj} \rangle (P * Q) \dashv\vdash \langle \text{subj} \rangle P * \langle \text{subj} \rangle Q}$ |                                                                                                                          | $\frac{\text{SUBJMOD-LATER}}{\triangleright \langle \text{subj} \rangle P \dashv\vdash \langle \text{subj} \rangle \triangleright P}$           |                                                                       |

FIGURE 8.4: iRC11 rules for the subjective modality

CHAPTER SUMMARY. In this chapter, we presented view-monotone predicates vProp—the type of iRC11 propositions—and the lifting of many base-logic connectives and modalities to those of iRC11. We have defined iRC11 WPs also in terms of the base logic WPs, and showed iRC11 adequacy. We have also defined various iRC11 modalities: fence modalities and view-explicit modalities. In the next chapters, we follow the same approaches to derive more iRC11 assertions and their rules on top of the base logic: the non-atomic and atomic points-to assertions, and invariants.



# 9

## Non-Atomic Points-To

---

The points-to assertion  $\ell \mapsto v$  is a well-known feature of separation logics. It represents unique ownership of the location  $\ell$ , which allows for safe, non-racy operations on  $\ell$ . For concurrent reads, the assertion can be equipped with fractional permission, i.e.,  $\ell \mapsto^q v$ . Ownership of a fraction  $q \in (0, 1]$  is sufficient to prevent concurrent writes. We would like to have these features for non-atomic accesses: concretely, the full ownership of a points-to  $\ell \mapsto v$  should be sufficient to safely perform non-atomic writes (and thus also any atomic operations), while a fractional  $\ell \mapsto^q v$  should be sufficient to safely perform non-atomic reads (and thus also any atomic reads). In this chapter, we give a model for iRC11’s *non-atomic points-to* assertion that satisfies this interface, using the base logic local assertions defined in [Chapter 7](#). In [Chapter 10](#), we will also discuss iRC11’s ability to switch between non-atomic and atomic points-to assertions.

### 9.1 The Interface of Non-Atomic Points-To

The interface of iRC11 non-atomic points-to is rather standard, as given in [Figure 9.1](#). [NA-FRAC](#), [NA-FRAC-VALID](#), and [NA-FRAC-AGREE](#) together say that non-atomic points-to is fractional, and [NA-EXCL](#) says that full ownership of a non-atomic points-to is exclusive. [NA-READ](#) allows us to perform non-racy reads using *any* access mode  $o$  with a fraction  $\ell \mapsto^q v$ , and we are guaranteed the return value is  $v$ . [NA-WRITE](#) allows us to perform non-racy writes also using *any* access mode  $o$  with the full fraction  $\ell \mapsto \_$ , and we know afterwards  $\ell$  has the value just written. The support for an arbitrary access mode  $o$  reflects the fact that if the points-to ownership is sufficient to safely perform the most demanding mode (non-atomic, **na**), then it should also be sufficient for less demanding ones.

Furthermore, [NA-ALLOC](#) says that an allocation gives us the full block ownership ( $\dagger^n \ell$ —lifted from the base logic<sup>1</sup> to  $\text{vProp}$ —and the full non-atomic points-to ownership ( $\star_{m \in [0, n)} \ell + m \mapsto \star$ ) for all locations of the newly allocated block, whose base location is  $\ell$ . Conversely, [NA-DEALLOC](#) consumes the block ownership and the points-to ownership of all locations. We strengthen [NA-DEALLOC](#) slightly by only requiring a weaker points-to  $\ell \mapsto ?$ , which we call an *unsynchronized points-to*. Intuitively, the ownership of an unsynchronized points-to  $\ell \mapsto ?$  only guarantees that the owning thread has observed the latest write to  $\ell$ , but is not synchronized with that write, i.e., it has not observed the write’s message

<sup>1</sup>see [Definition 7.1](#)

$$\begin{array}{c}
\text{NA-FRAC} \quad \ell \mapsto^q v * \ell \mapsto^{q'} v \dashv\vdash \ell \mapsto^{q+q'} v \quad \text{NA-FRAC-VALID} \quad \ell \mapsto^q v \vdash q \in (0, 1] \quad \text{NA-FRAC-AGREE} \quad \ell \mapsto^q v * \ell \mapsto^{q'} v' \vdash v = v' \quad \text{NA-EXCL} \quad \ell \mapsto v * \ell \mapsto v' \vdash \text{False} \\
\\
\text{NA-READ} \quad \{\ell \mapsto^q v\} *^o \ell \text{ in } \pi \{w. w = v * \ell \mapsto^q v\}_{\mathcal{E}} \quad \text{NA-WRITE} \quad \{\ell \mapsto \_ \} \ell :=_o v \text{ in } \pi \{\star. \ell \mapsto v\}_{\mathcal{E}} \\
\\
\text{NA-ALLOC} \quad \frac{0 < n}{\{\text{True}\}} \quad \text{NA-DEALLOC} \quad \frac{0 < n}{\left\{ \uparrow^n \ell * \bigstar_{m \in [0, n)} \ell + m \mapsto ? \right\}} \quad \text{NA-UNSYNC} \quad \ell \mapsto \_ \vdash \ell \mapsto ? \\
\\
\text{alloc}(n) \text{ in } \pi \quad \left\{ \ell. \uparrow^n \ell * \bigstar_{m \in [0, n)} \ell + m \mapsto \star \right\}_{\mathcal{E}} \quad \text{free}(\ell, n) \text{ in } \pi \quad \{\star. \text{True}\}_{\mathcal{E}} \\
\\
\text{where } \ell \mapsto \_ ::= \exists v. \ell \mapsto^q v.
\end{array}$$

FIGURE 9.1: Rules for iRC11 non-atomic points-to

view. On the other hand, ownership of  $\ell \mapsto v$  guarantees that the thread is synchronized with that latest write, which is the write of  $v$ . The rule **NA-UNSYNC** demonstrates that the latter is stronger than the former.

## 9.2 The Model of Non-Atomic Points-To

In order to define the non-atomic points-to assertion purely within vProp, we first lift the base logic local assertions (either in iProp or in the meta-level logic, see **Definition 7.1**) to vProp as follows.

**Definition 9.1** (Lifting Local Assertions to vProp).

$$\begin{aligned}
\llbracket \text{Hist}(\ell, h) \rrbracket &::= \lambda \_. \text{Hist}(\ell, h) \\
\llbracket \text{Write}^{\exists \text{rlx}}(\ell, \alpha) \rrbracket &::= \lambda \_. \text{Write}^{\exists \text{rlx}}(\ell, \alpha) \\
\llbracket \text{Read}^{\text{na}}(\ell, \alpha) \rrbracket &::= \lambda \_. \text{Read}^{\text{na}}(\ell, \alpha) \\
\llbracket \text{Read}^{\exists \text{rlx}}(\ell, \alpha) \rrbracket &::= \lambda \_. \text{Read}^{\exists \text{rlx}}(\ell, \alpha) \\
\llbracket \text{Local}_A(\ell, h) \rrbracket &::= \lambda V. \text{Local}_A(\ell, h, V) \\
\llbracket \text{Local}_W^{\exists \text{rlx}}(\ell, \alpha) \rrbracket &::= \lambda V. \text{Local}_W^{\exists \text{rlx}}(\ell, \alpha, V) \\
\llbracket \text{Local}_R^{\exists \text{rlx}}(\ell, \alpha) \rrbracket &::= \lambda V. \text{Local}_R^{\exists \text{rlx}}(\ell, \alpha, V) \\
\llbracket \text{Local}_R^{\text{na}}(\ell, \alpha, V_{\text{na}}) \rrbracket &::= \lambda V. \text{Local}_R^{\text{na}}(\ell, \alpha, V_{\text{na}}) \wedge V_{\text{na}} \sqsubseteq V
\end{aligned}$$

The lifting is straightforward. Recall that the various local ownership for parts of the race-detector state are purely ghost state,<sup>2</sup> so in lifting them to vProp we simply ignore the interpreting view. For local observations, we use the interpreting view  $V$  as the last argument to the meta-level assertions. Recall **Property 7.5** that the local observations are all view-monotone.

**Remark 9.2** (The non-atomic view  $V_{\text{na}}$ ). Note that unlike the rest of iRC11 local observations, we do not hide the view  $V_{\text{na}}$  of the non-atomic local observation  $\text{Local}_R^{\text{na}}(\ell, \alpha, V_{\text{na}})$ . Instead, we require that the implicit interpreting view  $V$  includes  $V_{\text{na}}$ . The view  $V_{\text{na}}$  is called the *non-atomic view*, and we expose it to record the view of the most recent *non-atomic*

<sup>2</sup>see **Definition 7.14**

*access period*. Intuitively, safe accesses to a location  $\ell$  must alternate between periods of non-atomic accesses and periods of atomic accesses. Interestingly, the switch from a non-atomic access period to an atomic access period of  $\ell$  can happen (logically) much later than the most recent physical non-atomic operation to  $\ell$ . That is, *the end of a non-atomic access period may not logically coincide with the most recent non-atomic access*. Even so, any incoming atomic accesses of the new atomic access period must synchronize with not only the most recent non-atomic operation, but with the point of the switch itself. Therefore, we use the view  $V_{\text{na}}$  to track the view of the switch, so as to make more resources available to the incoming atomic accesses. In short, the non-atomic view  $V_{\text{na}}$  is needed to have strong reasoning principles for switching between non-atomic and atomic accesses, and its uses will be explained more clearly in [Chapter 10](#). In this chapter, we can simply ignore this view.

**Definition 9.3** (Model of  $\ell \mapsto v$ ). We define a *primitive* non-atomic points-to  $\ell \xrightarrow{q}_{\text{na}} h$  which represents fractional ownership of  $\ell$  with a history  $h$ , and then use it to define the unsynchronized non-atomic points-to  $\ell \xrightarrow{q} ?$  and the actual points-to  $\ell \xrightarrow{q} v$ .

$$\begin{aligned} \ell \xrightarrow{q}_{\text{na}} h &::= \exists \alpha_w, \alpha_1, \alpha_2. \text{Local}_{\text{A}}(\ell, h) * \text{Local}_{\text{W}}^{\exists \text{r1x}}(\ell, \alpha_w) * \\ &\quad \text{Local}_{\text{R}}^{\exists \text{r1x}}(\ell, \alpha_2) * (\exists V_{\text{na}}. \text{Local}_{\text{R}}^{\text{na}}(\ell, \alpha_1, V_{\text{na}})) * \\ &\quad \text{Hist}_q(\ell, h) * \text{Write}_q^{\exists \text{r1x}}(\ell, \alpha_w) * \\ &\quad \text{Read}_q^{\text{na}}(\ell, \alpha_1) * \text{Read}_q^{\exists \text{r1x}}(\ell, \alpha_2) \\ \ell \xrightarrow{q} ? &::= \exists t, v, V^?. \ell \xrightarrow{q}_{\text{na}} [t \leftarrow (v, V^?)] \\ \ell \xrightarrow{q} v &::= \exists t, V^?. \ell \xrightarrow{q}_{\text{na}} [t \leftarrow (v, V^?)] * \exists V^? \end{aligned}$$

A fraction  $q$  of the primitive non-atomic points-to for  $\ell$  contains the corresponding fractions for  $\ell$ 's history ownership of  $h$  and the parts of race-detector state. By  $\text{Local}_{\text{A}}(\ell, h)$ , the owner of  $\ell \xrightarrow{q}_{\text{na}} h$  has also observed the allocation of  $\ell$ . The sets  $\alpha_w$ ,  $\alpha_1$ , and  $\alpha_2$  of atomic writes, non-atomic and atomic reads, respectively, are existentially quantified, and, due to the local observations, all sets are also observed by the owner of  $\ell \xrightarrow{q}_{\text{na}} h$ .

The unsynchronized non-atomic points-to  $\ell \xrightarrow{q} ?$  then simply requires that the history be a singleton  $[t \leftarrow (v, V^?)]$  for  $\ell$ 's latest write event  $(t, v, V^?)$ . The non-atomic points-to  $\ell \xrightarrow{q} v$  additionally fixes the value to be  $v$ , and requires that the owner has observed the message view  $V^?$ .

The definitions clearly show that **NA-UNSYNC** holds. We sketch the proofs for the remaining rules.

*Proof sketch that  $\ell \mapsto v$  is fractional.* Proofs of **NA-FRAC**, **NA-FRAC-VALID**, and **NA-FRAC-AGREE** follow from the fact that the ownership history and the local assertions for parts of the race-detector state are all fractional—see [Figure 7.2](#). In proving **NA-FRAC**, we will need **BL-NAL-JOIN** and **BL-ATRL-JOIN** to join the local observations for reads.  $\square$

*Proof sketch of **NA-ALLOC**.* We perform the proof in the base logic. Note that we do not have a base logic rule for allocation and deallocation, so we will need to prove both **NA-ALLOC** and **NA-DEALLOC** by unfolding

the both WP definitions of iRC11 (Definition 8.4) and of the base logic (Definition 6.14), and work directly with the state interpretation (Definition 7.18), like for other base logic WP rules.

Fortunately, the pre-condition and the race-free condition for allocation is trivial. As the newly allocated block—whose base location is  $\ell$ —is fresh in the global memory,<sup>3</sup> we can update the global ghost state GlobalGhost to mirror the change in the global physical state, namely we allocate the block ownership  $\dagger^n \ell$ , and the history ownership as well as the local assertions for all locations in the newly allocated block, all of which are needed to construct the non-atomic points-to for them. Note that the allocated locations all have the allocated value  $\dagger$ , which is lifted to the poison value  $\text{poison}$  in iRC11.  $\square$

<sup>3</sup>see OM-ALLOC, Figure 3.3, §3.3

*Proof sketch of NA-DEALLOC.* After unfolding the WP definitions of both iRC11 and the base logic, we first need to show that the step is safe (it reduces). With the full fraction block ownership and the global ghost ownership GlobalGhost, we can use BL-GHOST-BLOCK-FULL (Figure 7.8, §7.5) to know that we have collected the ownership of all locations in the block. Furthermore, the unsynchronized non-atomic points-to of all the locations guarantee that they are still alive, and that the deallocation is race-free for all of them, as the deallocation acts like a non-atomic write. Consequently, we satisfy both OM-FREE (§3.3) and DRF-DEALLOC (§3.4), so the deallocation reduces. Since we do not need the caller to have synchronized with all the message views of the locations' latest writes, the unsynchronized non-atomic points-to's  $\ell + m \mapsto ?$  are sufficient.

After the step, we update the state interpretation to match the changed global state. Fortunately, the global ghost GlobalGhost are very loose on deallocated locations—we only need to update the ghost histories of the deallocated locations to None.  $\square$

*Proof sketch of NA-READ.* We only need to unfold the definitions of the non-atomic points-to (Definition 9.3) and the iRC11 Hoare triples and WPs (Definition 8.5 and Definition 8.4), and perform the proof in the base logic. Recall that by Definition 8.5, all of our resources are interpreted at the current component  $\mathcal{V}.\text{cur}$  of the thread-view  $\mathcal{V}$ .

- In case  $o = \text{na}$ , we apply BL-HOARE-READ-NA (§7.3.2). Note that  $V_r$  is instantiated to  $V_{\text{na}}$ . In the post-condition we only need to use the post-condition of BL-HOARE-READ-NA to address the only change by the read, which is the non-atomic reads set and its local observation.
- In case  $o \sqsubseteq \text{rlx}$ , we apply BL-HOARE-READ-AT (§7.3.2). Note that the history in the non-atomic points-to is a singleton, and  $V_r$  is instantiated to  $\mathcal{V}.\text{cur}$ , so the proof is straightforward.  $\square$

*Proof sketch of NA-WRITE.* The proof is similar to that of NA-READ. We use BL-HOARE-WRITE-NA in case  $o = \text{na}$ . Otherwise, if  $o \sqsubseteq \text{rlx}$ , we use BL-HOARE-WRITE-AT, and then use BL-HIST-DROP-SINGLETON (Figure 7.2, §7.2) to shrink the history back to a singleton.  $\square$

# 10

## Atomic Points-To

---

The *atomic points-to* assertion plays the similar role as the non-atomic points-to assertion, but for atomic accesses. It is iRC11’s first abstraction for the ownership needed to safely perform atomic accesses. It can be used directly to verify ORC11 code, but iRC11 also uses it to derive the higher-level GPS protocols<sup>1</sup> (Part III). Nevertheless, the atomic points-to assertion is more flexible than iRC11’s version of GPS protocols because they work more explicitly with views. Consequently, it is used pervasively in Compass, in conjunction with logical atomic triples (Part IV).

iRC11 atomic points-to assertion is inspired by Cosmo’s atomic points-to assertion<sup>2</sup> to work with explicit views. However, since Cosmo is sound for the stronger Multicore OCaml memory model, its atomic points-to assertion is fairly simple: the (potentially fractional) assertion  $\ell \mapsto_{\text{at}} (v, V)$  represents Cosmo’s ownership of an atomic location  $\ell$  with the value  $v$  and view  $V$  of the latest write. That is, Cosmo’s atomic points-to needs only to take care of the latest write, because atomic accesses in Multicore OCaml are much stronger than the different access modes supported by C11. In contrast, iRC11 atomic points-to assertion needs to carry around a history of multiple writes that are still visible to accessing threads, and to provide multiple rules for the different access modes.

In the presence of concurrent writes to the same location  $\ell$ , iRC11 rules for handling  $\ell$ ’s history are rather cumbersome and hard to use. In practice, if a client performs arbitrary concurrent writes to a location  $\ell$ , then the concurrent protocol for  $\ell$  is often trivial. That is because there would be no clear order between the writes: in the ORC11 semantics, we will see that the writes arrive in the history randomly, with **holes** in the history.<sup>3</sup> More specifically, this is the result of adapting C11’s support for *non-multi-copy-atomicity* (non-MCA), *i.e.*, the property where writes can arrive at different threads in different orders.

Fortunately, algorithms tend to avoid concurrent writes where interesting protocols are needed: they either have a single writer and multiple concurrent readers, or have all participants purely perform compare-and-swap (CASes) operations to resolve potential contention. In such fashion, the history has no holes, and the **mo** order becomes more meaningful and can be used to support some well-ordered protocol.

Consequently, iRC11 provides *multiple modes* for the atomic points-to assertion to cater to these common cases. In §10.1, we present these modes for the atomic points-to, the relations among them and with the

<sup>1</sup>Turon et al., “GPS: navigating weak memory with ghosts, protocols, and separation” [TVD14]; Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

<sup>2</sup>Mével et al., “Cosmo: a concurrent separation logic for multicore OCaml” [MJP20].

<sup>3</sup>see OM-WRITE, §3.3

non-atomic points-to, and iRC11 Hoare rules for atomic accesses with the atomic points-to. In §10.2, we give the model of the atomic points-to assertion, built atop the local assertions of the base logic (§7.2).

### 10.1 The Interface of the Atomic Points-To Assertion

We support 3 modes for the atomic points-to: arbitrarily concurrent (con), single-writer (sw), and CAS-only (cas).

$$\theta \in \text{AtomicMode} ::= \text{con} \mid \text{sw} \mid \text{cas}.$$

The atomic points-to with the arbitrarily concurrent mode con supports any access mode, but with a weak set of WP or Hoare rules. The other two modes enjoy stronger WP or Hoare rules. In the single-writer mode sw, all writes using the atomic points-to must be made sequential (synchronized), but reads can be arbitrarily concurrent. In the CAS-only mode cas, the atomic points-to only supports CASes to write, and reads can be arbitrarily concurrent.

**Definition 10.1** (Atomic Points-To Assertion). The atomic points-to assertion has the form  $\ell \xrightarrow{t_x, \gamma}_\theta h$  where (1)  $\theta$  is one of the 3 atomic modes; and (2)  $\emptyset \neq h \in \text{History}$  is  $\ell$ 's current history which contains write events still visible to accessing threads; and (3)  $\gamma$  is a ghost location used to uniquely identity an atomic period of the atomic points-to and can be ignored for now; and (4)  $t_x$  is the timestamp of the latest exclusive *single-writer* write, which will be needed for GPS protocols and can also be ignored for now.

We write  $\ell \mapsto_\theta h ::= \exists t. \ell \xrightarrow{t}_\theta h$  to ignore the exclusive single-writer timestamp.

**Definition 10.2** (Atomic Local Ownership and Observations). The atomic points-to assertion  $\ell \mapsto_\theta h$  is needed for every atomic access, and thus will be put inside an invariant for shared concurrent access. Therefore, we need to define several local ownership and observations to represent what a thread knows about the shared history  $h$  of the atomic points-to.

- The history-seen observation  $\ell \sqsubseteq_{\text{sn}} h$  asserts the observation of all  $\ell$ 's write events in the non-empty history  $h$ . This observation is the minimum requirement to perform an atomic read on  $\ell$ .
- The history-sync observation  $\ell \sqsubseteq_{\text{sy}} h$  asserts not only the observation of  $\ell$ 's write events of in  $h$ , but also the observation of those writes' message views.
- The single-writer ownership  $\ell \sqsubseteq_{\text{sw}} h$  asserts the exclusive permission to write (the single-writer) to  $\ell$ , and the history-sync observation of  $h$  (i.e.,  $\ell \sqsubseteq_{\text{sy}} h$ ). The single-writer ownership guarantees that  $h$  is the current history of  $\ell$ .
- The fractional CAS ownership  $\ell \sqsubseteq_{\text{cas}}^{t_x, q} h$  asserts the shared permission to CAS to  $\ell$ , and the history-seen observation of  $h$  (i.e.,  $\ell \sqsubseteq_{\text{sn}} h$ ). A fraction  $q$  of the CAS ownership only guarantees that  $h$

$$\begin{array}{c}
\text{persistent}(\ell \sqsubseteq_{\text{sn}} h) \\
\text{persistent}(\ell \sqsubseteq_{\text{sy}} h) \\
\text{timeless}(\ell \sqsubseteq_{\text{sn}} h) \\
\text{timeless}(\ell \sqsubseteq_{\text{sy}} h)
\end{array}
\quad
\begin{array}{c}
\text{timeless}(\ell \mapsto_{\theta} h) \\
\text{timeless}(\ell \sqsubseteq_{\text{sw}} h) \\
\text{timeless}(\ell \sqsubseteq_{\text{cas}}^{t,q} h)
\end{array}
\quad
\begin{array}{c}
\text{AT-EXCL} \\
\ell \mapsto_{\theta} h * \ell \mapsto_{\theta'} h' \vdash \text{False}
\end{array}
\quad
\begin{array}{c}
\text{AT-SW-EXCL} \\
\ell \sqsubseteq_{\text{sw}} h * \ell \sqsubseteq_{\text{sw}} h' \vdash \text{False}
\end{array}$$
  

$$\begin{array}{c}
\text{AT-SW-CAS-EXCL} \\
\ell \sqsubseteq_{\text{sw}} h * \ell \sqsubseteq_{\text{cas}}^q h' \vdash \text{False}
\end{array}
\quad
\begin{array}{c}
\text{AT-SW-AGREE} \\
\ell \mapsto_{\theta} h * \ell \sqsubseteq_{\text{sw}} h' \vdash \theta = \text{sw} \wedge h = h'
\end{array}$$
  

$$\begin{array}{c}
\text{AT-CAS-FRAC-AGREE} \\
\ell \mapsto_{\theta} h * \ell \sqsubseteq_{\text{cas}}^{t',q'} h' \vdash \theta \neq \text{con} \wedge t = t' \wedge h' \subseteq h
\end{array}
\quad
\begin{array}{c}
\text{AT-CAS-CAS-FRAC-AGREE} \\
\ell \sqsubseteq_{\text{cas}}^{t,q} h * \ell \sqsubseteq_{\text{cas}}^{t',q'} h' \vdash t = t' \wedge q + q' \in (0, 1]
\end{array}$$
  

$$\begin{array}{c}
\text{AT-CAS-JOIN} \\
\ell \sqsubseteq_{\text{cas}}^{t,q} h * \ell \sqsubseteq_{\text{cas}}^{t',q'} h' \vdash \ell \sqsubseteq_{\text{cas}}^{t,q+q'} (h \cup h')
\end{array}
\quad
\begin{array}{c}
\text{AT-CAS-SPLIT} \\
\ell \sqsubseteq_{\text{cas}}^{t,q+q'} h \vdash \ell \sqsubseteq_{\text{cas}}^{t,q} h * \ell \sqsubseteq_{\text{cas}}^{t',q'} h
\end{array}$$
  

$$\begin{array}{c}
\text{AT-SY} \\
\ell \mapsto_{\theta} h \vdash \ell \sqsubseteq_{\text{sy}} h
\end{array}
\quad
\begin{array}{c}
\text{AT-SY-SN} \\
\ell \sqsubseteq_{\text{sy}} h \vdash \ell \sqsubseteq_{\text{sn}} h
\end{array}
\quad
\begin{array}{c}
\text{AT-SW-SY} \\
\ell \sqsubseteq_{\text{sw}} h \vdash \ell \sqsubseteq_{\text{sy}} h
\end{array}
\quad
\begin{array}{c}
\text{AT-CAS-SN} \\
\ell \sqsubseteq_{\text{cas}}^q h \vdash \ell \sqsubseteq_{\text{sn}} h
\end{array}$$
  

$$\begin{array}{c}
\text{AT-SY-MONO} \\
\frac{h' \neq \emptyset \quad h' \subseteq h}{\ell \sqsubseteq_{\text{sy}} h \vdash \ell \sqsubseteq_{\text{sy}} h'}
\end{array}
\quad
\begin{array}{c}
\text{AT-SN-MONO} \\
\frac{h' \neq \emptyset \quad h' \subseteq h}{\ell \sqsubseteq_{\text{sn}} h \vdash \ell \sqsubseteq_{\text{sn}} h'}
\end{array}$$
  

$$\begin{array}{c}
\text{AT-SY-UNFOLD} \\
\frac{h(t) = (v, V)}{\ell \sqsubseteq_{\text{sy}} h \vdash \exists V * \exists [\ell \leftarrow \{\emptyset \mid \text{w} := t\}]}
\end{array}
\quad
\begin{array}{c}
\text{AT-SN-UNFOLD} \\
\frac{t \in \text{dom}(h)}{\ell \sqsubseteq_{\text{sn}} h \vdash \exists [\ell \leftarrow \{\emptyset \mid \text{w} := t\}]}
\end{array}$$
  

$$\begin{array}{c}
\text{AT-SY-JOIN} \\
\ell \sqsubseteq_{\text{sn}} h * \ell \sqsubseteq_{\text{sy}} h' \vdash \ell \sqsubseteq_{\text{sy}} (h \cup h')
\end{array}
\quad
\begin{array}{c}
\text{AT-SN-JOIN} \\
\ell \sqsubseteq_{\text{sy}} h * \ell \sqsubseteq_{\text{sn}} h' \vdash \ell \sqsubseteq_{\text{sn}} (h \cup h')
\end{array}
\quad
\begin{array}{c}
\text{AT-SN-VALID} \\
\ell \mapsto_{\theta} h * \ell \sqsubseteq_{\text{sn}} h' \vdash h' \subseteq h
\end{array}$$

FIGURE 10.1: Basic properties of assertions related to the atomic points-to

is the sub history of  $\ell$ 's current history. The timestamp  $t_x$  is of the latest exclusive single-writer write to  $\ell$ . As usual, we write  $\ell \sqsubseteq_{\text{cas}}^q h$  to ignore this timestamp, and write  $\ell \sqsubseteq_{\text{cas}} h$  for the full ownership where  $q = 1$ .

**Property 10.3** (Basic Properties of Assertions Related to Atomic Points-To).

Figure 10.1 presents several important basic properties of the atomic points-to assertions and its related assertions. All assertions are timeless, and the history-seen and history-sync observations are naturally persistent. The atomic points-to and the single-writer ownership are both exclusive (AT-EXCL and AT-SW-EXCL).

AT-SW-CAS-EXCL says that the single-writer ownership and the CAS ownership are incompatible, implying that the single-writer is indeed single. AT-SW-AGREE says that the atomic points-to and the single-writer ownership must agree on the history and the atomic mode. AT-CAS-FRAC-AGREE says that, on the other hand, the CAS ownership only guarantees that the history  $h'$  owned by the CAS ownership is a sub-history of the current history  $h$ . This is because CAS ownership are used for concurrent updates, so a fraction should not know the full history. AT-CAS-FRAC-AGREE additionally says that the CAS ownership guarantees that the latest exclusive single-writer timestamp is frozen in CAS-only mode

( $t = t'$ ). Interestingly, **AT-CAS-FRAC-AGREE** says that the CAS ownership only guarantees that the atomic mode  $\theta$  is not the concurrent mode  $\text{con}$ . We note that this is just a weakness in our model for atomic points-to, and this weakness does not affect us in practice. A better but slightly more complex model would give us  $\theta = \text{cas}$ .

**AT-CAS-CAS-FRAC-AGREE**, **AT-CAS-JOIN**, and **AT-CAS-SPLIT** encode the fractional nature of the CAS ownership.

**AT-SY**, **AT-SY-SN**, **AT-SW-SY**, and **AT-CAS-SN** together state the relations between the ownership assertions and the observations. **AT-SY** says that the atomic points-to naturally has observed and synchronized with all write events. **AT-SW-SY** says that this also applies to the single-writer ownership, because the writes are sequential. **AT-CAS-SN** on the other hand says that the CAS ownership does not guarantee synchronization. Recall that the history  $h$  in  $\ell \sqsubseteq_{\text{cas}}^q h$  is only a sub-history of the current one, and CAS ownership are used for concurrent updates.

**AT-SY-MONO** and **AT-SN-MONO** say that the observations on histories are downward-closed. **AT-SY-UNFOLD** and **AT-SN-UNFOLD** clearly state the difference between observing a write and synchronizing with that write, using the view-seen observation (**Definition 8.12**). **AT-SY-JOIN** and **AT-SN-JOIN** allow us to join observations. Finally, **AT-SN-VALID** says that an observation of  $h'$  guarantees that  $h'$  is a snapshot (a sub-history) of the current one  $h$ .

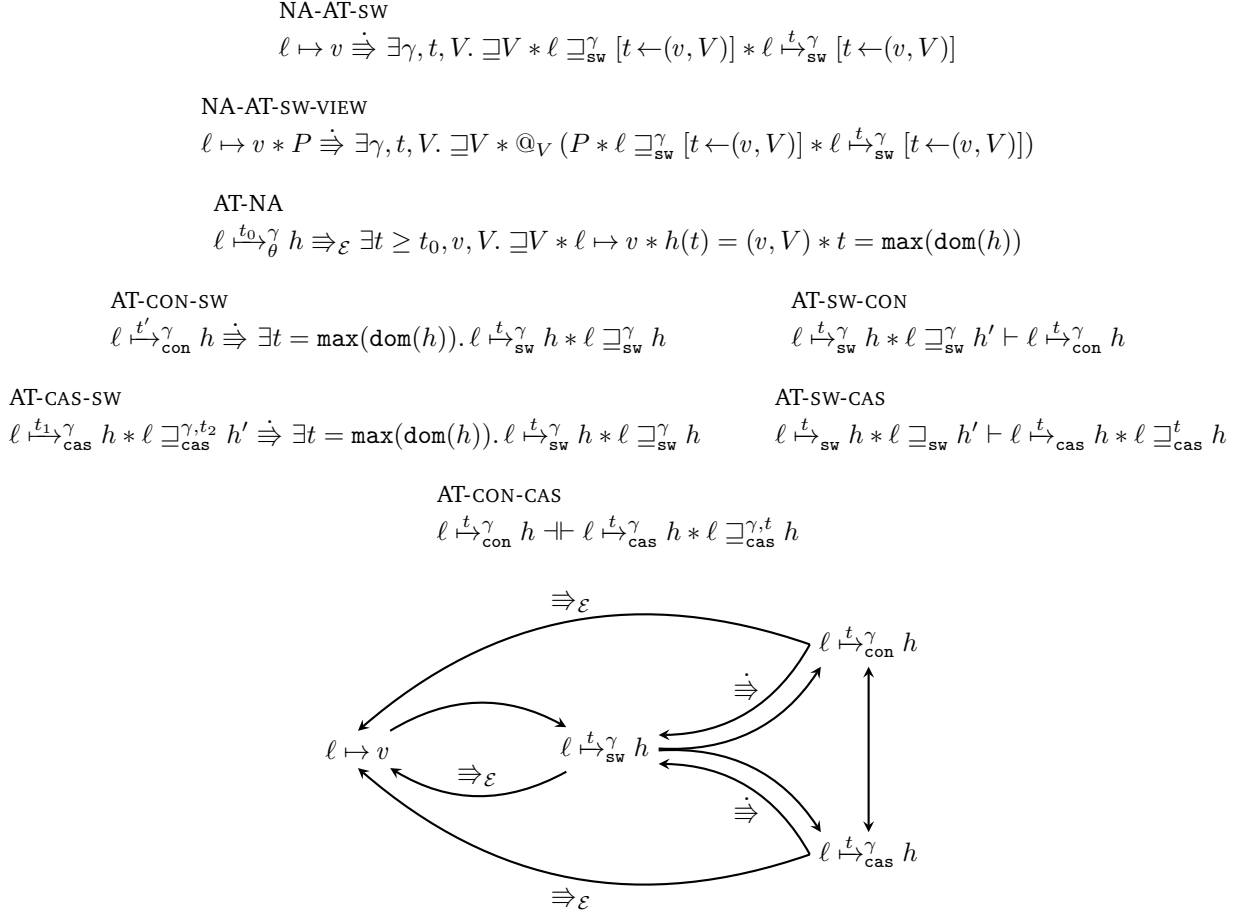
**Property 10.4** (Conversions Between Non-Atomic and Atomic Points-To). The top three rules of **Figure 10.2** present the rules for converting the non-atomic points-to ownership to the atomic one. The bottom of **Figure 10.2** visualizes the possible conversions between the points-to assertions.

**NA-AT-sw** says that we can go from the non-atomic points-to assertion to the single-writer atomic one and the single-writer ownership with a singleton history of the latest write  $(t, v, V)$ , knowing that we have observed the message view  $V$  ( $\sqsubseteq V$ ).

**NA-AT-sw-VIEW** strengthens **NA-AT-sw** by (1) freezing the atomic points-to and the single-writer ownership at the latest write message view  $V$  using the view-at modality (**Definition 8.13**); and (2) allowing the user to also freeze arbitrary local resource  $P$  at the same view. **NA-AT-sw-VIEW** demonstrates that the view  $V$  in fact is not the message view of the latest write in  $\ell$ 's history, because  $\ell$ 's latest write message view would not be able to justify  $P$ . Instead the view  $V$  is the view at which the switch (from non-atomic points-to to atomic points-to) happens, and the singleton history  $[t \leftarrow (v, V)]$  is not  $\ell$ 's actual history, but an abstraction of  $\ell$ 's actual history. This abstraction allows subsequent atomic accesses using the atomic points-to assertion to access the view  $V$ , and thus the resource  $P$  provided at the switch. In other words, **NA-AT-sw-VIEW** says that the atomic accesses to  $\ell$  after the switch are synchronized not only with the latest write to  $\ell$  before the switch, but also with the switch itself.

**AT-NA** allows us to go from an atomic points-to back to a non-atomic one, without knowing the atomic mode  $\theta$  nor having any other ownership (single-writer or CAS ownership). This demonstrates that the atomic points-to itself contains sufficient resources, and the single-write or CAS





Visualization of the conversions.

ownership is purely needed to enforce an access protocol (single-writer or CAS-only). We note that **NA-AT-sw** and **NA-AT-sw-view** only need a basic update to switch from non-atomic to atomic, while **AT-NA** requires a fancy update to go back. The reader may already have guessed correctly that the proof of **AT-NA** relies on **BL-HIST-DROP-SINGLETON** (§7.2), which justifies the fancy update. Consequently, the value  $v$  we get back for the non-atomic points-to is the latest write, regardless of how that write is made (with a CAS or a normal write using *any* access mode). Thanks to **AT-sy** and **AT-sy-unfold**, we know that we have observed the latest write message view  $V$  ( $\sqsubseteq V$ ).<sup>4</sup>

**CYCLES OF ALTERNATING NON-ATOMIC AND ATOMIC PERIODS.** We note that we have made the ghost location  $\gamma$  explicit in these rules, who signify its role. In the model of atomic points-to,  $\gamma$  is used to store the ghost state to define the protocols (concurrent, single-writer, or CAS-only) for the atomic points-to ownership assertions. But intuitively, the ghost location  $\gamma$  uniquely identifies an *atomic access period* of the location  $\ell$ . When we use the rule **NA-AT-sw** (or **NA-AT-sw-view**) to switch from non-atomic to atomic points-to, we receive a *fresh* location  $\gamma$  that identifies and enforces the atomic protocol for the *current* atomic period of  $\ell$ . As such,

FIGURE 10.2: Conversions between the non-atomic and atomic points-to assertion

<sup>4</sup>Recall that  $V$  is not just the latest write message view, it also includes the view of the switch.

the atomic local ownership and observations (Definition 10.2) with the ghost location  $\gamma$  are only meaningful when we still have access to the atomic points-to  $\ell \mapsto_{\theta}^{\gamma}$  with the same  $\gamma$ . Once we use **AT-NA** to turn  $\ell \mapsto_{\theta}^{\gamma}$  back to a non-atomic points-to, we can see that the atomic local ownership and observations with ghost location  $\gamma$  are not needed, and in fact  $\gamma$  is simply forgotten, and afterwards the atomic local ownership and observations of  $\gamma$  become meaningless. Later, when the non-atomic points-to is used again to switch to an atomic one, a new atomic period will be started with another fresh ghost location  $\gamma'$ . This life-cycle can probably be understood better if we look at the visualization graph in Figure 10.2 as an automaton.

We further note that all assertions appearing in a single rule in this chapter should be read with the same ghost location  $\gamma$ . It may not make sense to have interactions between ownership from different atomic periods, and we do not have rules for those cases anyway.

**Property 10.5** (Conversions Between Modes of Atomic Points-To). The rest of Figure 10.2 presents the rules for switching between different modes of the atomic points-to. **AT-CON-SW** and **AT-SW-CON** allow conversions between the concurrent mode and the single-writer mode of the atomic points-to, while **AT-CAS-SW** and **AT-SW-CAS** allow conversions between the CAS-only mode and the single-writer mode. Both **AT-CON-SW** and **AT-SW-CON** need a basic update simply to update the exclusive single-writer timestamp to the latest one. Finally **AT-CON-CAS** allows one to convert between the concurrent mode and the CAS-only mode.

**RULES FOR CONCURRENT ATOMIC ACCESSES.** We next look at the rules for atomic operations using the atomic points-to assertion. We note that these rules are meant for concurrent accesses. If we simply use atomic accesses sequentially,<sup>5</sup> we should locally own an atomic points-to  $\ell \mapsto_{\theta} h$ , which we can turn back to a non-atomic one using **AT-NA**, and then use the rules **NA-READ** and **NA-WRITE** (§9.1) which support atomic access modes.<sup>6</sup>

In the following, we look at atomic access rules where the atomic points-to  $\ell \mapsto_{\theta} h$  is meant to be shared for concurrent accesses. The rules do not enforce exactly how the points-to is shared—that can be done orthogonally with invariants (Chapter 11). Therefore, they only assume ownership of the atomic points-to under a view-at modality (Definition 8.13), i.e.,  $@_{V_b}(\ell \mapsto_{\theta} h)$  where the view  $V_b$  has no clear relation to what the executing thread has observed. In particular, the rules will have the following form.

$$\{\exists V * P * @_{V_b}(\ell \mapsto_{\theta} h)\} e \text{ in } \pi \{v. \exists V', h'. \exists V' * @_{V_b \sqcup V'}(\ell \mapsto_{\theta} h') * Q\}_{\mathcal{E}}$$

That is, the view  $V_b$  is meant to track all relaxed memory effects done by all participating threads to the shared points-to  $\ell \mapsto_{\theta} h$ , and hence the executing thread cannot know much about  $V_b$ , except that it also contributes to  $V_b$ . Most importantly, the view  $V_b$  will be used later to enable synchronized, safe switching back to non-atomic points-to (using **AT-NA**).

<sup>5</sup>which can be the case in C/C++ where mixing atomic and non-atomic accesses are forbidden. Instead, C/C++ make the distinction at the location level: there are non-atomic locations and atomic ones. Under this restriction, even though we know that there is no other thread racing with us, we may still have to use a **rlx** access for an atomic location.

<sup>6</sup>If one is to perform sequential accesses, then one would never need to use CASes.

$$\begin{array}{c}
\text{AT-READ-SN} \\
\frac{\mathbf{r1x} \sqsubseteq o}{\left\{ \sqsupseteq V_0 * \ell \sqsupseteq_{\text{sn}} h_0 * @_{V_b}(\ell \xrightarrow{t_x}_{\theta} h) \right\}^{*o} \ell \text{ in } \pi \left\{ \begin{array}{l} v. \exists h', t, V, V' \sqsupseteq V_0. h_0 \subseteq h' \subseteq h * \\ h'(t) = (v, V) * t \geq \max(\text{dom}(h_0)) * \\ (o \sqsupseteq \mathbf{acq}) ? V \sqsubseteq V' : \nabla_{\pi}(\sqsupseteq V) \\ \sqsupseteq V' * @_{V'}(\ell \sqsupseteq_{\text{sn}} h') * @_{V_b \sqcup V'}(\ell \xrightarrow{t_x}_{\theta} h) \end{array} \right\}^{\varepsilon}} \\
\\
\text{AT-READ-SN-ACQ} \\
\left\{ \sqsupseteq V_0 * \ell \sqsupseteq_{\text{sn}} h_0 * @_{V_b}(\ell \xrightarrow{t_x}_{\theta} h) \right\}^{*\mathbf{acq}} \ell \text{ in } \pi \left\{ \begin{array}{l} v. \exists h', t, V, V' \sqsupseteq V_0 \sqcup V. h_0 \subseteq h' \subseteq h * \\ h'(t) = (v, V) * t \geq \max(\text{dom}(h_0)) * \\ \sqsupseteq V' * @_{V'}(\ell \sqsupseteq_{\text{sn}} h') * @_{V_b \sqcup V'}(\ell \xrightarrow{t_x}_{\theta} h) \end{array} \right\}^{\varepsilon} \\
\\
\text{AT-READ-CAS} \\
\frac{\mathbf{r1x} \sqsubseteq o}{\left\{ \sqsupseteq V_0 * \ell \sqsupseteq_{\text{cas}}^q h_0 * @_{V_b}(\ell \xrightarrow{t_x}_{\theta} h) \right\}^{*o} \ell \text{ in } \pi \left\{ \begin{array}{l} v. \exists h', t, V, V' \sqsupseteq V_0. h_0 \subseteq h' \subseteq h * \\ h'(t) = (v, V) * t \geq \max(\text{dom}(h_0)) * \\ (o \sqsupseteq \mathbf{acq}) ? V \sqsubseteq V' : \nabla_{\pi}(\sqsupseteq V) \\ \sqsupseteq V' * @_{V'}(\ell \sqsupseteq_{\text{cas}}^q h') * @_{V_b \sqcup V'}(\ell \xrightarrow{t_x}_{\theta} h) \end{array} \right\}^{\varepsilon}} \\
\\
\text{AT-READ-SY} \\
\frac{\mathbf{r1x} \sqsubseteq o}{\left\{ \sqsupseteq V_0 * \ell \sqsupseteq_{\text{sy}} h * @_{V_b}(\ell \xrightarrow{t_x}_{\theta} h) \right\}^{*o} \ell \text{ in } \pi \left\{ \begin{array}{l} v. \exists t, V, V' \sqsupseteq V_0 \sqcup V. h'(t) = (v, V) * t = \max(\text{dom}(h)) * \\ \sqsupseteq V' * @_{V_b \sqcup V'}(\ell \xrightarrow{t_x}_{\theta} h) \end{array} \right\}^{\varepsilon}} \\
\\
\text{AT-READ-SW} \\
\frac{\mathbf{r1x} \sqsubseteq o}{\left\{ \sqsupseteq V_0 * \ell \sqsupseteq_{\text{sw}} h * @_{V_b}(\ell \xrightarrow{t_x}_{\theta} h) \right\}^{*o} \ell \text{ in } \pi \left\{ \begin{array}{l} v. \exists t, V, V' \sqsupseteq V_0 \sqcup V. h(t) = (v, V) * t = \max(\text{dom}(h)) * \\ \sqsupseteq V' * \ell \sqsupseteq_{\text{sw}} h * @_{V_b \sqcup V'}(\ell \xrightarrow{t_x}_{\theta} h) \end{array} \right\}^{\varepsilon}}
\end{array}$$

FIGURE 10.3: iRC11 read rules with the atomic points-to assertion

### 10.1.1 Atomic Read Rules

Several rules for atomic reads are given in Figure 10.3. **AT-READ-SN**, **AT-READ-SY**, **AT-READ-CAS**, and **AT-READ-SW** allow reading with a history-seen observation, a history-sync observation, a fractional CAS ownership, and a single-writer ownership, respectively, in addition to the shared atomic points-to.

**AT-READ-SN** is the most fundamental read rule for atomic points-to, as all other rules in Figure 10.3 are derived from it. The rule assumes in the pre-condition a local history-seen observation  $\ell \sqsupseteq_{\text{sn}} h_0$  for some snapshot history  $h_0$  of  $\ell$ , and the shared atomic points-to  $\ell \xrightarrow{t_x}_{\theta} h$  of the current history  $h$  at some view  $V_b$ . The pre-condition also includes a view-seen observation  $\sqsupseteq V_0$  for some view  $V_0$ . The post-condition says that the executing thread  $\pi$  will read a message  $(t, v, V)$  which is no earlier than what it has observed ( $t \geq \max(\text{dom}(h_0))$ ), and afterwards the thread will have observed a bigger snapshot history  $h'$  that contains the read message

$(\ell \sqsubseteq_{\text{sn}} h')$ . After the read, the current view of  $\pi$  will be at least  $V'$  ( $\sqsubseteq V'$ ), and if this is an acquire read then we know that the thread has observed the read message view  $V$ , due to  $V \sqsubseteq V'$  and **VS-MONO** (§8.5). We note that  $\sqsubseteq V' * @_{V'}(\ell \sqsubseteq_{\text{sn}} h')$  is stronger than  $\ell \sqsubseteq_{\text{sn}} h'$ , due to **VA-ELIM** (§8.5). If this is a relaxed read, then the message view  $V$  will only be available after an acquire fence, *i.e.*, its observation is under an acquire fence modality:  $\nabla_{\pi}(\sqsubseteq V)$ . Last but not least, the atomic points-to  $\ell \xrightarrow{t_x}_{\theta} h$  is returned unchanged, but at the view  $V_b$  extended with the view  $V'$  (*i.e.*,  $V_b \sqcup V'$ )—which is  $\pi$ 's current view after the read—to account for the observation of the read itself (the action id created by the read).

**AT-READ-SN-ACQ** is derived from **AT-READ-SN** simply by instantiating  $o$  with **acq**. Since it is an acquire read, we know that the thread's current view  $V'$  includes the view  $V$  of the read message, *i.e.*,  $V \sqsubseteq V'$ . **AT-READ-CAS** is derived from **AT-READ-SN**, simply by the rule **AT-CAS-SN** that the CAS ownership implies the history-seen observation. **AT-READ-SY** is derived from **AT-READ-SN** using **AT-SY-SN**: assuming that the thread has observed and synchronized with all write events in  $\ell$ 's current history  $h$ , the thread will read the latest write. **AT-READ-SW** is then derived from **AT-READ-SY** using **AT-SW-SY**.

### 10.1.2 Atomic Write Rules

Several rules for atomic writes are given in **Figure 10.4**. **AT-WRITE-SN**, **AT-WRITE-CAS**, and **AT-WRITE-SW** allow reading with a history-seen observation, a full fractional CAS ownership, and a single-writer ownership, respectively, in addition to the shared atomic points-to (in a corresponding atomic mode).

Again, **AT-WRITE-SN** is the basic rule from which the other rules are derived. In the pre-condition, it requires a view-seen observation  $\sqsubseteq V_0$  for some view  $V_0$ , a history-seen observation  $\ell \sqsubseteq_{\text{sn}} h_0$  for some snapshot history  $h_0$  of  $\ell$ , and the atomic points-to in the concurrent mode  $\ell \xrightarrow{t_x}_{\text{con}} h$ , shared at some view  $V_b$ . The pre-condition additionally requires, in case the write is a relaxed write, a view-seen observation  $\sqsubseteq V_{\text{rel}}$  of some view  $V_{\text{rel}}$  under the release modality, *i.e.*,  $\Delta_{\pi}(\sqsubseteq V_{\text{rel}})$ . This assertion ensures that the view  $V_{\text{rel}}$  has been observed by the thread  $\pi$  at its most recent release fence, so the message view of the relaxed write to be performed is guaranteed to include  $V_{\text{rel}}$ . In other words,  $V_{\text{rel}}$  is a lower bound for the message view of the relaxed write to be performed. If the write to be performed is at least a release one,  $\pi$ 's current view is a lower bound.

The post-condition of **AT-WRITE-SN** says that a new write message  $(t, v, V)$  will be inserted into the history  $h$ . That is, after the write, the ownership  $\ell \xrightarrow{t_x}_{\text{con}} h[t \leftarrow (v, V)]$  is returned, at the extended view  $V_b \sqcup V'$  where  $V'$  is the thread  $\pi$ 's current view after the write. Note that the timestamp  $t$  for the new write message must be fresh in  $h$  ( $t \notin \text{dom}(h)$ ), and must be **mo**-later than the events that the thread has observed for  $\ell$  ( $\max(\text{dom}(h_0)) < t$ ). Since this is the write, we know that the thread's current view  $V'$  after the step strictly extends the view  $V_0$  before:  $V_0 \sqsubset V'$ . Furthermore, the message view  $V$  cannot be smaller than the view  $V_0$  before the step ( $V_0 \not\sqsupseteq V$ ), because  $V$  contains at least the new timestamp

AT-WRITE-SN

$$\frac{\mathbf{rlx} \sqsubseteq o}{\left\{ \begin{array}{l} \exists V_0 * \ell \sqsubseteq_{\text{sn}} h_0 * @_{V_b}(\ell \xrightarrow{tx}_{\text{con}} h) * \\ (o = \mathbf{rlx}) ? \Delta_{\pi}(\exists V_{\text{rel}}) : \text{True} \end{array} \right\} \ell :=_o v \text{ in } \pi \left\{ \begin{array}{l} \text{✗}. \exists t, V, V' \sqsupset V_0 \not\sqsubseteq V. \max(\text{dom}(h_0)) < t \notin \text{dom}(h) * \\ (\mathbf{rel} \sqsubseteq o) ? V = V' : V_{\text{rel}} \sqsubseteq V \sqsubseteq V' * \\ \exists V' * @_V(\ell \sqsubseteq_{\text{sn}} [t \leftarrow (v, V)]) * \\ @_{V'}(\ell \sqsubseteq_{\text{sn}} h_0[t \leftarrow (v, V)]) * @_{V_b \sqcup V'}(\ell \xrightarrow{tx}_{\text{con}} h[t \leftarrow (v, V)]) \end{array} \right\} \varepsilon}$$

AT-WRITE-CAS

$$\frac{\mathbf{rlx} \sqsubseteq o}{\left\{ \begin{array}{l} \exists V_0 * \ell \sqsubseteq_{\text{cas}}^q h_0 * @_{V_b}(\ell \mapsto_{\text{cas}} h) * \\ (o = \mathbf{rlx}) ? \Delta_{\pi}(\exists V_{\text{rel}}) : \text{True} \end{array} \right\} \ell :=_o v \text{ in } \pi \left\{ \begin{array}{l} \text{✗}. \exists t, V, V' \sqsupset V_0 \not\sqsubseteq V. \max(\text{dom}(h_0)) < t \notin \text{dom}(h) * \\ (\mathbf{rel} \sqsubseteq o) ? V = V' : V_{\text{rel}} \sqsubseteq V \sqsubseteq V' * \\ \exists V' * @_V(\ell \sqsubseteq_{\text{sn}} [t \leftarrow (v, V)]) * \\ @_{V'}(\ell \sqsubseteq_{\text{cas}}^q h_0[t \leftarrow (v, V)]) * @_{V_b \sqcup V'}(\ell \mapsto_{\text{cas}} h[t \leftarrow (v, V)]) \end{array} \right\} \varepsilon}$$

AT-WRITE-SW

$$\frac{\mathbf{rlx} \sqsubseteq o}{\left\{ \begin{array}{l} \exists V_0 * \ell \sqsubseteq_{\text{sw}} h * @_{V_b}(\ell \xrightarrow{tx}_{\text{sw}} h) * \\ (o = \mathbf{rlx}) ? \Delta_{\pi}(\exists V_{\text{rel}}) : \text{True} \end{array} \right\} \ell :=_o v \text{ in } \pi \left\{ \begin{array}{l} \text{✗}. \exists t, V, V' \sqsupset V_0 \not\sqsubseteq V. \max(\text{dom}(h)) < t * \\ (\mathbf{rel} \sqsubseteq o) ? V = V' : V_{\text{rel}} \sqsubseteq V \sqsubseteq V' * \exists V' * \\ @_{V'}(\ell \sqsubseteq_{\text{sw}} h[t \leftarrow (v, V)]) * @_{V_b \sqcup V'}(\ell \xrightarrow{tx}_{\text{sw}} h[t \leftarrow (v, V)]) \end{array} \right\} \varepsilon}$$

AT-WRITE-SW-RLX

$$\left\{ \begin{array}{l} \exists V_0 * \ell \sqsubseteq_{\text{sw}} h * @_{V_b}(\ell \xrightarrow{tx}_{\text{sw}} h) * \\ @_{V_{\text{rel}}} P * \Delta_{\pi}(\exists V_{\text{rel}}) \end{array} \right\} \ell :=_{\mathbf{rlx}} v \text{ in } \pi \left\{ \begin{array}{l} \text{✗}. \exists t, V, V' \sqsupset V_0 \not\sqsubseteq V. \max(\text{dom}(h)) < t * \\ V_{\text{rel}} \sqsubseteq V \sqsubseteq V' * \exists V' * @_V P * \\ @_{V'}(\ell \sqsubseteq_{\text{sw}} h[t \leftarrow (v, V)]) * @_{V_b \sqcup V'}(\ell \xrightarrow{tx}_{\text{sw}} h[t \leftarrow (v, V)]) \end{array} \right\} \varepsilon}$$

AT-WRITE-SW-RLX-SIMPLE

$$\{ \ell \sqsubseteq_{\text{sw}} h * @_{V_b}(\ell \xrightarrow{tx}_{\text{sw}} h) * \Delta_{\pi} P \} \ell :=_{\mathbf{rlx}} v \text{ in } \pi \left\{ \begin{array}{l} \text{✗}. \exists t, V, V' \sqsupset V. \max(\text{dom}(h)) < t * \exists V' * @_V P * \\ \ell \sqsubseteq_{\text{sw}} h[t \leftarrow (v, V)] * @_{V_b \sqcup V'}(\ell \xrightarrow{tx}_{\text{sw}} h[t \leftarrow (v, V)]) \end{array} \right\} \varepsilon}$$

AT-WRITE-SW-REL

$$\{ \exists V_0 * \ell \sqsubseteq_{\text{sw}} h * @_{V_b}(\ell \xrightarrow{tx}_{\text{sw}} h) * P \} \ell :=_{\mathbf{rel}} v \text{ in } \pi \left\{ \begin{array}{l} \text{✗}. \exists t, V \sqsupset V_0. \max(\text{dom}(h)) < t * \exists V * @_V P * \\ @_V(\ell \sqsubseteq_{\text{sw}} h[t \leftarrow (v, V)]) * @_{V_b \sqcup V}(\ell \xrightarrow{tx}_{\text{sw}} h[t \leftarrow (v, V)]) \end{array} \right\} \varepsilon}$$

FIGURE 10.4: iRC11 write rules with the atomic points-to assertion

$t$  which  $V_0$  cannot have. In case this is at least a release write, then the message view  $V$  is exactly the view  $V'$  after the write. If it is only a relaxed write, we know the  $V_{\text{rel}}$  is a lower bound for  $V$  ( $V_{\text{rel}} \sqsubseteq V$ ). In any case the message view is no more than the thread's current view after the step ( $V \sqsubseteq V'$ ). Finally, after the step the thread extends its observation on  $\ell$ 's history by the write event it has just performed. That is, it owns  $\ell \sqsubseteq_{\text{sn}} h_0[t \leftarrow (v, V)]$ , with the history  $h_0$  it has observed before the step extended with  $(t, v, V)$ . The observation is strengthened by being put at the view  $V'$  which the thread has observed. Additionally, the thread also gets a history-seen observation for the singleton history of the write message  $(t, v, V)$ , at exactly its message view ( $@_V(\ell \sqsubseteq_{\text{sn}} [t \leftarrow (v, V)])$ ).

**AT-WRITE-CAS** is derived from **AT-WRITE-SN**, using **AT-CAS-SN** and an

extra ghost update to update the CAS ownership from  $h_0$  to  $h_0[t \leftarrow (v, V)]$ . This is also the case for **AT-WRITE-SW**, but using **AT-SW-SY** and **AT-SY-SN** instead. However, **AT-WRITE-SW** does update the latest exclusive single-writer timestamp to the new timestamp  $t$ .

<sup>7</sup>[VN13; TVD14; DV16; DV17; Kai+17; MJP20].

**RESOURCE TRANSFER.** **AT-WRITE-SW-RLX**, **AT-WRITE-SW-RLX-SIMPLE**, and **AT-WRITE-SW-REL** are all derived from **AT-WRITE-SW**, and demonstrate the support for transferring the resource  $P$  with a write, a typical pattern in GPS, RSL, FSL, and Cosmo.<sup>7</sup> **AT-WRITE-SW-RLX** specializes **AT-WRITE-SW** for the relaxed write case, and assumes  $P$  at the view  $V_{\text{rel}}$  which we know will be a lower bound the new write's message view. Consequently, after the write, by view-monotonicity  $P$  holds at the new write's message view  $V$ , i.e., we have  $@_V P$ . As such, we have released the resource  $P$  with the write, and we can attach it to the message  $(t, v, V)$  with the help of an invariant (see Chapter 11). Then, when another thread performs an acquire read (or a relaxed read and then an acquire fence) from  $(t, v, V)$ , by the rule **AT-READ-SN-ACQ**, the reading thread will obtain  $\sqsubseteq V$ , which it then can combine with  $@_V P$  (taken from the invariant) and apply the rule **VA-ELIM** (§8.5) to acquire the resource  $P$  locally, and thus conclude the resource transfer.

**AT-WRITE-SW-RLX-SIMPLE** simplifies **AT-WRITE-SW-RLX** further by dropping most of the views. It is derived from **AT-WRITE-SW-RLX** with the help of the rule **RELMOD-VA-REVERT** (§8.5) concerning the relation between the release modality and the view-at modality.

For a release write, **AT-WRITE-SW-REL** simply says that any local resource  $P$  before the step can be released with the write, by putting  $P$  at the new write's message view  $V$  after the step.

### 10.1.3 Atomic CAS Rules

The general rule **AT-CAS-SN-GEN** in Figure 10.5 allows us to perform CASes with the seen-history observation and the shared atomic points-to assertion in the CAS-only mode. The pre-condition is not so different from the pre-condition needed to perform a write, i.e., that of **AT-WRITE-CAS**. The extra premise  $\forall v_0, t_0 \geq \max(\text{dom}(h_0)). h(t_0) = (v_0, \_) \Rightarrow \vdash v_0 =? v_r$  is required to guarantee safe comparison between any readable value  $v_0$  and the expected value  $v_r$ , and the resources concern  $P_{\text{cmp}}$  are needed for deterministic pointer comparison. Please see also the explanation of the base logic rule **BL-HOARE-CAS** in §7.3.3.

In particular, if the expected value  $v_r$  is a location value  $\ell_r$ , to guarantee deterministic pointer comparison,  $P_{\text{cmp}}$  is required to simultaneously imply (with a basic update)  $\Phi_{\text{cmp}}(\ell_r, h)$  that (1) some primitive non-atomic points-to ownership of  $\ell_r$  at an arbitrary view, sufficient to show that  $\ell_r$  is alive; and (2) for any location value  $\ell'$  that the thread may read from  $h$ ,  $P_{\text{cmp}}$  is also sufficient to show that  $\ell'$  is also alive.

In the post-condition, a boolean value  $b$  signaling the success or failure of the CAS instruction is returned, and a message  $(t', v', V_r)$  will be read by the instruction. The timestamp  $t'$  cannot be earlier than what the thread has already observed through  $h_0$ . Regardless of success or failure,

AT-CAS-SN-GEN

$$\begin{array}{c}
\mathbf{rlx} \sqsubseteq o_f, o_r, o_w \quad \forall v_0, t_0 \geq \max(\text{dom}(h_0)). h(t_0) = (v_0, \_) \Rightarrow \vdash v_0 =? v_r \\
\hline
\left\{ \begin{array}{l} \sqsupseteq V_0 * \ell \sqsupseteq_{\text{sn}} h_0 * @_{V_b}(\ell \xrightarrow{t_x}_{\text{cas}} h) * (o = \mathbf{rlx}) ? \Delta_\pi(\sqsupseteq V_{\text{rel}}) : \text{True} * \\ P_{\text{cmp}} * \square((v_r = \ell_r) ? (P_{\text{cmp}} \dot{\Rightarrow} \Phi_{\text{cmp}}(\ell_r, h)) : \text{True}) \end{array} \right\} \\
\text{CAS}^{o_f, o_r, o_w}(\ell, v_r, v_w) \text{ in } \pi \\
\left\{ \begin{array}{l} b. \exists t', v', V_r, V' \sqsupseteq V_0, h'_0, h'. h_0 \subseteq h'_0 \subseteq h' * h'_0(t') = (v', V_r) * \max(\text{dom}(h_0)) \leq t' * \\ P_{\text{cmp}} * \sqsupseteq V' * @_{V'}(\ell \sqsupseteq_{\text{sn}} h'_0) * @_{V_b \sqcup V'}(\ell \xrightarrow{t_x}_{\text{cas}} h') * \\ \vee \left\{ \begin{array}{l} b = \mathbf{false} \quad * v_r \neq v' * h' = h * (o_f \sqsupseteq \mathbf{acq}) ? V_r \sqsubseteq V' : \nabla_\pi(\sqsupseteq V_r) \\ b = \mathbf{true} \quad * v_r = v' * \exists t \notin \text{dom}(h), V_w \sqsupset V_r \not\sqsubseteq V' \sqsupset V_0. t = t' + 1 \\ \quad * h' = h[t \leftarrow (v_w, V_w)] * h'_0(t) = (v_w, V_w) \\ \quad * (o_r \sqsupseteq \mathbf{acq}) ? V_w \sqsubseteq V' : \nabla_\pi(\sqsupseteq V_w) * (o_w \sqsupseteq \mathbf{rel}) ? V' \sqsubseteq V_w : V_{\text{rel}} \sqsubseteq V_w \end{array} \right. \end{array} \right\}_{\mathcal{E}}
\end{array}$$

where

$$\Phi_{\text{cmp}}(\ell_r, h) ::= (\exists q_r, h_r, V. \triangleright @_V(\ell_r \xrightarrow{q_r} h_r)) \wedge (\forall t' \geq \text{dom}(h), \ell'. h(t') = (\ell', \_) \multimap \exists q', h', V. \triangleright @_V(\ell' \xrightarrow{q'} h'))$$

FIGURE 10.5: An iRC11 CAS rule with the atomic points-to assertion

$P_{\text{cmp}}$  is returned unchanged, and we know that the thread's current view after the step is  $V'$ .

In case of failure, i.e.,  $b = \mathbf{false}$ , we know that the read value  $v'$  is definitely not equal to the expected value  $v_r$ , and that the atomic points-to ownership is returned unchanged but at the extended view  $V_b \sqcup V'$  ( $@_{V_b \sqcup V'}(\ell \xrightarrow{t_x}_{\text{cas}} h)$ ), and that the thread will have observed the new snapshot history  $h'_0$  ( $@_{V'}(\ell \sqsupseteq_{\text{sn}} h'_0)$ ). Furthermore, the read message view  $V_r$  is observed according to the failure read access mode  $o_f$ . If  $o_f$  is at least an acquire mode, then  $V_r$  is included in the view  $V'$  after the step ( $V_r \sqsubseteq V'$ ). Otherwise, the observation of  $V_r$  is only available after the next acquire fence ( $\nabla_\pi \sqsupseteq V_r$ ).

If the CAS succeeds, i.e.,  $b = \mathbf{true}$ , then  $v' = v_r$  and a new write message  $(t, v_w, V_w)$  will be inserted into the history  $h$  next to the read message  $(t = t' + 1)$  where  $t$  is fresh in  $h$  ( $t \notin \text{dom}(h)$ ).  $V_w$  strictly extends  $V_r$ , and  $V'$  strictly extends  $V_0$  and cannot be smaller than  $V_r$ , because they contain the new write of the timestamp  $t$ . The thread will have observed the new snapshot history  $h'_0$  that contains the new write event. The write message view  $V_w$  is observed according to the read access mode  $o_r$ : if  $o_r$  is at least an acquire mode, then  $V_w \sqsubseteq V'$ , otherwise the thread only has  $\nabla_\pi(\sqsupseteq V_w)$ . If the write access mode  $o_w$  is at least release, then the write message view includes the thread's current view after the step ( $V' \sqsubseteq V_w$ ), otherwise  $V_{\text{rel}}$  is a lower bound of  $V_w$ . Note that if this is a release-acquire CAS, then the write message view  $V_w$  is exactly the thread's current view after the step  $V'$ , i.e.,  $o_w \sqsupseteq \mathbf{rel} \wedge o_r \sqsupseteq \mathbf{acq} \Rightarrow V' = V_w$ .

Finally, we note that **AT-CAS-SN-GEN** can be used to derive CAS rules that use other kinds of ownership. Even though the rule requires the atomic points-to to be in the CAS-only mode (cas), we can apply **AT-CON-CAS** (Figure 10.2) to support CASes with the atomic points-to in the



$$\begin{array}{c}
\text{AT-CAS-SW-GEN} \\
\frac{\mathbf{rlx} \sqsubseteq o_f, o_r, o_w \quad \forall v_0, t_0 \geq \max(\text{dom}(h_0)). h(t_0) = (v_0, \_) \Rightarrow \vdash v_0 =^? v_r}{\left\{ \begin{array}{l} \exists V_0 * \ell \sqsubseteq_{\text{sn}} h_0 * @_{V_c}(\ell \sqsubseteq_{\text{sw}} h) * @_{V_b}(\ell \mapsto_{\text{sw}} h) * (o = \mathbf{rlx}) ? \Delta_\pi(\exists V_{\text{rel}}) : \text{True} \\ P_{\text{cmp}} * \square((v_r = \ell_r) ? (P_{\text{cmp}} \Rightarrow \Phi_{\text{cmp}}(\ell_r, h)) : \text{True}) \end{array} \right\}} \\
\text{CAS}^{o_f, o_r, o_w}(\ell, v_r, v_w) \text{ in } \pi \\
\left\{ \begin{array}{l} b. \exists t', v', V_r, V' \sqsubseteq V_0, h'_0, h'. h_0 \subseteq h'_0 \subseteq h' * h'_0(t') = (v', V_r) * \max(\text{dom}(h_0)) \leq t' * \\ P_{\text{cmp}} * \exists V' * @_{V'}(\ell \sqsubseteq_{\text{sn}} h'_0) * @_{V_b \sqcup V'}(\ell \sqsubseteq_{\text{sw}} h' * \ell \mapsto_{\text{sw}} h') * \\ \vee \left\{ \begin{array}{l} b = \mathbf{false} \quad * v_r \neq v' * h' = h * (o_f \sqsubseteq \mathbf{acq}) ? V_r \sqsubseteq V' : \nabla_\pi(\exists V_r) \\ b = \mathbf{true} \quad * v_r = v' * \exists t \notin \text{dom}(h), V_w \sqsupset V_r \not\sqsupseteq V' \sqsupset V_0. t = t' + 1 \\ \quad * h' = h[t \leftarrow (v_w, V_w)] * h'_0(t) = (v_w, V_w) \\ \quad * (o_r \sqsubseteq \mathbf{acq}) ? V_w \sqsubseteq V' : \nabla_\pi(\exists V_w) * (o_w \sqsubseteq \mathbf{rel}) ? V' \sqsubseteq V_w : V_{\text{rel}} \sqsubseteq V_w \end{array} \right\} \end{array} \right\}_\varepsilon
\end{array}$$

where  $\Phi_{\text{cmp}}(\ell_r, h)$  is defined as in Figure 10.5.

FIGURE 10.6: An iRC11 CAS rule with the atomic points-to in single-writer mode

concurrent mode *con.* With a fractional CAS ownership, we can also apply **AT-CAS-SN-GEN**, thanks to **AT-CAS-SN**.

If the atomic points-to is in the single-writer mode (*sw*), then, together with the single-writer ownership  $\ell \sqsubseteq_{\text{sw}} h$ , we can get to the atomic points-to in CAS-only mode thanks to **AT-SW-CAS** (also in Figure 10.2), then apply **AT-CAS-SN-GEN**, and then **AT-CAS-SW** to go back to the single-writer mode. The result is the CAS rule **AT-CAS-SW-GEN** for the single-writer mode, in Figure 10.6. Naturally, a single-writer owner never needs to perform a CAS, because it is not racing in writing with anyone. Furthermore, the rule is basically useless, as it requires and returns the single-writer ownership at some view  $V_c$  and  $V_b \sqcup V'$ , respectively. Nevertheless, the rule is a sanity check that shows that, in the single-writer mode, only the single-writer owner can actually perform a write (in this case, a CAS).

## 10.2 The Model of the Atomic Points-To Assertion

To give a model for the atomic points-to assertion, we rely on the base logic local assertions (§7.4), in a similar way to the model of non-atomic points-to assertion (§9.2). However, we need extra ghost state to manage the “switching” protocols among atomic modes and between the atomic points-to to the non-atomic points-to. The ghost location  $\gamma$  of the assertion  $\ell \mapsto_{\theta}^{\gamma}$ , which uniquely identifies an atomic period for  $\ell$ , will store this extra ghost state.

**Definition 10.6** (Extra RAs for Atomic Points-To). We need 3 RAs: one to allow creating snapshots of histories, one to store the latest non-atomic view—needed to switch between non-atomic and atomic points-to, and one to store the timestamp of the latest exclusive single-writer write—



needed to switch from other modes to single-writer mode.

$$\begin{aligned}
\text{ATHISTR} &::= \text{AUTH}(\text{MAP}(\text{Time}, \text{AG}(\text{Val} \times \text{View}))) \\
\text{NAWRITER} &::= \text{OPTION}(\text{AG}(\text{View})) \\
\text{EXWRITER} &::= \text{AUTH}(\text{OPTION}(\text{FRAC} \times \text{AG}(\text{Time}))) \\
\text{ATOMICR} &::= \text{ATHISTR} \times \text{EXWRITER} \times \text{NAWRITER}
\end{aligned}$$

The RA `ATHISTR` supports making snapshots of histories: the authoritative element is used to store the up-to-date history, while fragmentary elements are snapshots of that history. The authoritative element can only grow.

The RA `NAWRITER` allows storing permanently a view which is meant to be a thread's current view  $V_{\text{na}}$  at which the switch from non-atomic to atomic points-to is performed. All subsequent atomic accesses using the same atomic points-to identified by the ghost location  $\gamma$  will have synchronized with  $V_{\text{na}}$ .

The RA `EXWRITER` allows for fractional fragmentary elements that agree on the timestamp of the latest exclusive single-writer write, and only allows updating using the full fraction, together with the authoritative element.

Finally, the RA `ATOMICR` for the atomic points-to is just a product of the 3 RAs above.

**Definition 10.7** (Ghost Ownership Abstraction for Atomic RAs). We define the following abstractions for ghost ownership of the atomic RAs in `vProp`. All are timeless and objective.

$$\begin{aligned}
\text{atLastNA}^\gamma(V_{\text{na}}) &::= \llbracket (\varepsilon, \varepsilon, \text{ag}(V_{\text{na}})) \rrbracket^\gamma \\
\text{atExclTime}_q^\gamma(t_x) &::= \llbracket (\varepsilon, \circ(\text{Some}(q, \text{ag}(t_x))), \varepsilon) \rrbracket^\gamma \\
\text{atReader}^\gamma(h) &::= \llbracket (\circ h, \varepsilon, \varepsilon) \rrbracket^\gamma \\
\text{atWriter}^\gamma(h) &::= \llbracket (\bullet_{3/4} h \cdot \circ h, \varepsilon, \varepsilon) \rrbracket^\gamma \\
\text{atAuth}^\gamma(h, t_x, V_{\text{na}}) &::= \llbracket (\bullet_{1/4} h \cdot \circ h, \bullet(\text{Some}(1, \text{ag}(t_x))), \text{ag}(V_{\text{na}})) \rrbracket^\gamma
\end{aligned}$$

- $\text{atLastNA}^\gamma(V_{\text{na}})$  is persistent, and records the view at the point of the switch from a non-atomic points-to to the atomic period identified by  $\gamma$ .
- $\text{atExclTime}_q^\gamma(t_x)$  is fractional and records the timestamp  $t_x$  of the latest exclusive single-writer write in the atomic period identified by  $\gamma$ . The full fraction  $\text{atExclTime}^\gamma(t_x)$ , which a single-writer would own, is the *exclusive write permission* needed to update  $t_x$ .
- $\text{atReader}^\gamma(h)$  is persistent and records a snapshot history  $h$ , i.e., in the atomic period of  $\gamma$ ,  $h$  is a lower bound of the current history.
- $\text{atWriter}^\gamma(h)$  is exclusive and records the current history  $h$ . It is the (ghost) *writer permission* needed to perform a write (a change) to the history.
- $\text{atAuth}^\gamma(h, t_x, V_{\text{na}})$  is the authoritative state of the atomic points-to protocol. The other ghost ownership abstractions defined above

$$\begin{array}{l}
\text{persistent}(\text{atLastNA}^\gamma(V_{\text{na}})) \\
\text{persistent}(\text{atReader}^\gamma(h))
\end{array}
\quad
\text{atLastNA}^\gamma(V_{\text{na}}) * \text{atLastNA}^\gamma(V'_{\text{na}}) \vdash V_{\text{na}} = V'_{\text{na}}$$

$$\text{atExclTime}_q^\gamma(t_x) * \text{atExclTime}_{q'}^\gamma(t'_x) \vdash t_x = t'_x \wedge q + q' \in (0, 1]$$

$$\text{atExclTime}_q^\gamma(t_x) * \text{atExclTime}_{q'}^\gamma(t_x) \dashv\vdash \text{atExclTime}_{q+q'}^\gamma(t_x)$$

$$\text{atWriter}^\gamma(h) * \text{atReader}^\gamma(h') \vdash h' \subseteq h \quad \text{atWriter}^\gamma(h) * \text{atWriter}^\gamma(h') \vdash \text{False} \quad \text{atWriter}^\gamma(h) \vdash \text{atReader}^\gamma(h)$$

$$\text{atAuth}^\gamma(h, t_x, V_{\text{na}}) * \text{atLastNA}^\gamma(V'_{\text{na}}) \vdash V_{\text{na}} = V'_{\text{na}} \quad \text{atAuth}^\gamma(h, t_x, V_{\text{na}}) * \text{atExclTime}_q^\gamma(t'_x) \vdash t_x = t'_x$$

$$\text{atAuth}^\gamma(h, t_x, V_{\text{na}}) * \text{atReader}^\gamma(h') \vdash h' \subseteq h \quad \text{atAuth}^\gamma(h, t_x, V_{\text{na}}) * \text{atWriter}^\gamma(h') \vdash h' = h$$

FIGURE 10.7: Several properties of ghost abstractions for the atomic RA

must agree or be included in this authoritative state. We note that we use a setup where the authoritative element is also fractional ( $\bullet_{1/4}$  in  $\text{atAuth}^\gamma$ , or  $\bullet_{3/4}$  in  $\text{atWriter}^\gamma$ ). Fractions of the authoritative element enjoy agreement, and that is how we establish agreement between  $\text{atWriter}^\gamma$  and  $\text{atAuth}^\gamma$ .

Several properties of these ghost abstractions are given in [Figure 10.7](#).

We can now give the model of the atomic points-to assertion as well as its local ownership and observation assertions.

**Definition 10.8** (Model of Atomic Local Ownership and Observations). We first define what it means to locally observe a history  $h$  of  $\ell$ , and to locally synchronize with that history.

$$\text{Local}_{\text{sn}}(\ell, h) ::= \forall t, v, V^?. h(t) = (v, V^?) \Rightarrow \exists [\ell \leftarrow \{\emptyset [w := t]\}]$$

$$\text{Local}_{\text{sy}}(\ell, h) ::= \forall t, v, V^?. h(t) = (v, V^?) \Rightarrow \exists V^? * \exists [\ell \leftarrow \{\emptyset [w := t]\}]$$

That is, the owner of  $\text{Local}_{\text{sn}}(\ell, h)$  should have observed the timestamps of the writes in  $h$ , while the owner of  $\text{Local}_{\text{sy}}(\ell, h)$  should additionally have observed the message views of those writes. The model of the atomic local ownership and observations is then given within  $\text{vProp}$ , as follows.

$$\begin{aligned}
\ell \sqsubseteq_{\text{sn}}^\gamma h &::= \text{Local}_{\text{sn}}(\ell, h) * \text{atReader}^\gamma(h) * \exists V_{\text{na}}. \text{atLastNA}^\gamma(V_{\text{na}}) * \sqsubseteq V_{\text{na}} \\
\ell \sqsubseteq_{\text{sy}}^\gamma h &::= \text{Local}_{\text{sy}}(\ell, h) * \text{atReader}^\gamma(h) * \exists V_{\text{na}}. \text{atLastNA}^\gamma(V_{\text{na}}) * \sqsubseteq V_{\text{na}} \\
\ell \sqsubseteq_{\text{sw}}^{\gamma, t_x} h &::= \ell \sqsubseteq_{\text{sy}}^\gamma h * \text{atWriter}^\gamma(h) * \text{atExclTime}^\gamma(t_x) * t_x = \max(\text{dom}(h)) \\
\ell \sqsubseteq_{\text{cas}}^{\gamma, t_x, q} h &::= \ell \sqsubseteq_{\text{sn}}^\gamma h * \text{atExclTime}_q^\gamma(t_x) * t_x \in \text{dom}(h)
\end{aligned}$$

- The seen-history observation  $\ell \sqsubseteq_{\text{sn}}^\gamma h$  requires that the  $h$  is indeed a snapshot history of  $\ell$ 's atomic points-to identified by  $\gamma$  ( $\text{atReader}^\gamma(h)$ ), and that the owner has observed the writes in  $h$  ( $\text{Local}_{\text{sn}}(\ell, h)$ ). The remaining part  $\exists V_{\text{na}}. \text{atLastNA}^\gamma(V_{\text{na}}) * \sqsubseteq V_{\text{na}}$  says that the owner has observed the view  $V_{\text{na}}$  of the switch from the non-atomic points-to to the atomic points-to identified by  $\gamma$ .
- The sync-history observation  $\ell \sqsubseteq_{\text{sy}}^\gamma h$  is similar to the seen-history observation, but additionally requires the synchronization with all the message views in  $h$  ( $\text{Local}_{\text{sy}}(\ell, h)$ ).

- The single-writer ownership  $\ell \sqsupseteq_{\text{sw}}^{\gamma, t_x} h$  requires the sync-history observation  $\ell \sqsupseteq_{\text{sn}}^{\gamma} h$ , and holds the writer permission  $\text{atWriter}^{\gamma}(h)$  for the current history  $h$  as well as the exclusive writer permission  $\text{atExclTime}^{\gamma}(t_x)$  for the timestamp  $t_x$  of the latest exclusive single-writer write. That is, the single-writer ownership holds both the permissions to update  $h$  and  $t_x$ . Additionally, we know that  $t_x$  is the maximum timestamp in the current history  $h$ .
- The CAS ownership  $\ell \sqsupseteq_{\text{cas}}^{\gamma, t_x, q} h$  only requires the seen-history observation, and its fraction  $q$  is the fraction for exclusive single-writer timestamp  $\text{atExclTime}_q^{\gamma}(t_x)$ , which is sufficient to prevent others from updating  $t_x$  (and thus prevent any single-writer writes). Another requirement is that the owner has observed  $t_x$  ( $t_x \in \text{dom}(h)$ ).

**Definition 10.9** (Model of the Atomic Points-To). We now give the model of the atomic points-to assertions, also within  $\text{vProp}$ . It relies on a “lift-view” function  $\text{liftV}(h, V_{\text{na}})$  that lifts all  $h$ ’s message views to include the view  $V_{\text{na}}$  of the “non-atomic to atomic” switch.

$$\begin{aligned}
 \text{liftV}(h, V_{\text{na}}) &::= [t \leftarrow (v, V^? \sqcup V_{\text{na}}) \mid h(t) = (v, V^?)] \\
 \ell \xrightarrow[t]{t_x}^{\gamma} h &::= \exists h', \alpha_w, \alpha_1, \alpha_2, V_{\text{na}}. h = \text{liftV}(h', V_{\text{na}}) * t_x \in \text{dom}(h) \\
 &\quad * \text{Local}_{\text{sy}}(\ell, h) * \text{Local}_{\text{w}}^{\sqsupseteq \text{r1x}}(\ell, \alpha_w) \\
 &\quad * \text{Local}_{\text{R}}^{\sqsupseteq \text{r1x}}(\ell, \alpha_2) * \text{Local}_{\text{R}}^{\text{na}}(\ell, \alpha_1, V_{\text{na}}) \\
 &\quad * \text{Hist}(\ell, h') * \text{Write}^{\sqsupseteq \text{r1x}}(\ell, \alpha_w) \\
 &\quad * \text{Read}^{\text{na}}(\ell, \alpha_1) * \text{Read}^{\sqsupseteq \text{r1x}}(\ell, \alpha_2) \\
 &\quad * \text{atAuth}^{\gamma}(h, t_x, V_{\text{na}}) \\
 &\quad * \begin{cases} \theta = \text{sw} & * \text{True} \\ \theta = \text{cas} & * \text{atWriter}^{\gamma}(h) \\ \theta = \text{con} & * \text{atWriter}^{\gamma}(h) * \text{atExclTime}^{\gamma}(t_x) \end{cases}
 \end{aligned}$$

The model of the atomic points-to  $\ell \xrightarrow[t]{t_x}^{\gamma} h$  is very similar to that of the non-atomic points-to:<sup>8</sup> it requires *full* ownership of (1) the base logic assertions for history ownership ( $\text{Hist}(\ell, h')$ ) of the unlifted history  $h'$  ( $h = \text{liftV}(h', V_{\text{na}})$ ) and of (2) the difference parts of the race detector state (the atomic writes set  $\text{Write}^{\sqsupseteq \text{r1x}}(\ell, \alpha_w)$ , and the non-atomic and atomic reads sets  $\text{Read}^{\text{na}}(\ell, \alpha_1)$  and  $\text{Read}^{\sqsupseteq \text{r1x}}(\ell, \alpha_2)$ ).<sup>9</sup> Similarly, it requires the base logic’s local observations for all of those sets.<sup>10</sup> It also requires the observation of the non-atomic view  $V_{\text{na}}$  of the switch as well as that  $V_{\text{na}}$  has observed the non-atomic reads set, i.e.,  $\text{Local}_{\text{R}}^{\text{na}}(\ell, \alpha_1, V_{\text{na}})$ .

The main difference with the non-atomic points-to is the ghost ownership. The atomic points-to owns the authoritative state  $\text{atAuth}^{\gamma}(h, t_x, V_{\text{na}})$  to hold the authority over the local ownership and observations (given model in Definition 10.8). It further owns the remaining ghost permissions for different atomic modes: (i) for the single-writer mode  $\text{sw}$  it owns nothing more, because all permissions are owned by the single-writer ownership; (ii) for the CAS-only mode  $\text{cas}$  it owns the writer permission  $\text{atWriter}^{\gamma}(h)$  so as to allow concurrent CASes to update the history, but it does not own the exclusive write permission, which is owned by

<sup>8</sup>see Definition 9.3

<sup>9</sup>see also Definition 7.1 and Definition 9.1

<sup>10</sup>Note that  $\text{Local}_{\text{sy}}(\ell, h)$  implies the allocation observation  $\text{Local}_{\text{A}}(\ell, h)$ .

the fractions of CAS ownership themselves; and (iii) for the arbitrarily concurrent mode `con` it owns all the ghost permissions, as the clients of the `con` mode only have the seen-history or sync-history observations to work with  $\ell$ . Recall that the atomic points-to assertion is meant to be shared for concurrent accesses, and participants rely on their local atomic ownership and observations to relate themselves to the shared history  $h$  and thus to strengthen the behaviors of their own instructions.

We now sketch the proofs of several important rules.

### 10.2.1 Proof Sketches for Conversions between Non-Atomic and Atomic Points-To

*Proof sketch of NA-AT-SW-VIEW.* This proof is done entirely within `vProp`. We start by first freezing  $\ell \mapsto v * P$  at some view  $V_0$  using `VA-INTRO` (§8.5). Then we unfold the model of non-atomic points-to (`Definition 9.3`). We note that the local observations of the non-atomic points-to are not objective, while the history ownership and the race detector state ownership are objective, so the view-at modality can be easily eliminated using `VA-OBJ` (§8.5). Our goal then looks as follows.

| Context:                                                                                                                                                              | Goal: |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| $\exists V_0 * @_{V_0} P * @_{V_0} \exists V$                                                                                                                         |       |
| $@_{V_0}(\text{Local}_A(\ell, [t \leftarrow (v, V)]) * \text{Local}_W^{\exists \text{rlx}}(\ell, \alpha_w))$                                                          |       |
| $@_{V_0}(\text{Local}_R^{\exists \text{rlx}}(\ell, \alpha_2) * \text{Local}_R^{\text{na}}(\ell, \alpha_1, V_{\text{na}}))$                                            |       |
| $\text{Hist}(\ell, [t \leftarrow (v, V)]) * \text{Write}^{\exists \text{rlx}}(\ell, \alpha_w)$                                                                        |       |
| $\text{Read}^{\text{na}}(\ell, \alpha_1) * \text{Read}^{\exists \text{rlx}}(\ell, \alpha_2)$                                                                          |       |
| $\exists \gamma, t, V. \exists V * @_V (P * \ell \sqsubseteq_{\text{sw}}^\gamma [t \leftarrow (v, V)] * \ell \xrightarrow{\gamma}_{\text{sw}} [t \leftarrow (v, V)])$ |       |

From  $@_{V_0} \text{Local}_R^{\text{na}}(\ell, \alpha_1, V_{\text{na}})$  and the `vProp` definition of  $\text{Local}_R^{\text{na}}$  (`Definition 9.1`), we have  $V_{\text{na}} \sqsubseteq V_0$ .

We then allocate a new ghost location  $\gamma$  for the RA `ATOMICR`, using `GHOST-ALLOC` (§6.2), with the initial history  $h = [t \leftarrow (v, V \sqcup V_0)]$ , the exclusive write timestamp  $t$ , and the new non-atomic view  $V_0$ . This allocation will give us these following extra ownership  $\text{atAuth}^\gamma(h, t, V_0) * \text{atLastNA}^\gamma(V_0) * \text{atExclTime}^\gamma(t) * \text{atWriter}^\gamma(h)$ .

We note that from  $@_{V_0} \exists V$ , by `VA-VS` (§8.5), we have  $V \sqsubseteq V_0$ . Consequently,  $V \sqcup V_0 = V_0$ . We then instantiate the existential quantifiers respectively with  $\gamma$ ,  $t$ , and  $V_0$ . Note how  $V_0$ , the view of the switch, indeed becomes the new message view for  $t$ . We can easily discharge  $\exists V$  and  $@_{V_0} P$ , and then arrive at the following goal.

| Context:                                                                                                                   | Goal:                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| $\exists V_0 * @_{V_0} \exists V$                                                                                          |                                                                                            |
| $@_{V_0}(\text{Local}_A(\ell, [t \leftarrow (v, V)]) * \text{Local}_W^{\exists \text{rlx}}(\ell, \alpha_w))$               |                                                                                            |
| $@_{V_0}(\text{Local}_R^{\exists \text{rlx}}(\ell, \alpha_2) * \text{Local}_R^{\text{na}}(\ell, \alpha_1, V_{\text{na}}))$ |                                                                                            |
| $\text{Hist}(\ell, [t \leftarrow (v, V)]) * \text{Write}^{\exists \text{rlx}}(\ell, \alpha_w)$                             |                                                                                            |
| $\text{Read}^{\text{na}}(\ell, \alpha_1) * \text{Read}^{\exists \text{rlx}}(\ell, \alpha_2)$                               |                                                                                            |
| $\text{atAuth}^\gamma(h, t, V_0) * \text{atLastNA}^\gamma(V_0)$                                                            |                                                                                            |
| $\text{atExclTime}^\gamma(t) * \text{atWriter}^\gamma(h)$                                                                  | $@_{V_0}(\ell \sqsubseteq_{\text{sw}}^\gamma h * \ell \xrightarrow{\gamma}_{\text{sw}} h)$ |

By unfolding the definitions of the atomic points-to and the single-writer ownership, and then discharge all available assumptions, we arrive at the goal:

| Context:                                                                                                                                                                                                                                                                           | Goal: |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| $V \sqsubseteq V_0 * V_{\text{na}} \sqsubseteq V_0 * @_{V_0} \text{Local}_{\text{R}}^{\text{na}}(\ell, \alpha_1, V_{\text{na}})$ $@_{V_0}(\ell \sqsupseteq_{\text{sy}}^{\gamma} h * \text{Local}_{\text{sy}}(\ell, h) * \text{Local}_{\text{R}}^{\text{na}}(\ell, \alpha_1, V_0))$ |       |

This is easily done because  $h$  is the singleton  $[t \leftarrow (v, V_0)]$ , and  $\text{Local}_{\text{R}}^{\text{na}}$  is view monotone, so  $\text{Local}_{\text{R}}^{\text{na}}(\ell, \alpha_1, V_{\text{na}})$  implies  $\text{Local}_{\text{R}}^{\text{na}}(\ell, \alpha_1, V_0)$  knowing that  $V_{\text{na}} \sqsubseteq V_0$ .  $\square$

*Proof sketch of AT-NA.* The proof is rather straightforward: the model of the atomic points-to, after dropping the atomic ghost state abstractions, is almost the same as the model of the non-atomic points-to, except for the history  $h'$ . We then use **BL-HIST-DROP-SINGLETON** (which needs the fancy update, see Figure 7.2) to truncate  $h'$  to the singleton of its latest write. After sorting out the local observations, we are done.  $\square$

### 10.2.2 Proof Sketches for Conversions among Atomic Modes

*Proof sketch of AT-sw-CAS.* The proof is straightforward—its main proof step is to move the write permission  $\text{atWriter}^{\gamma}(h)$  from the single-writer ownership into the CAS-only atomic points-to.  $\square$

*Proof sketch of AT-CAS-sw.* The proof is also straightforward—it has 2 main proof steps. First, we move the write permission  $\text{atWriter}^{\gamma}(h)$  from the CAS-only atomic points-to out to construct the single-writer ownership. Second, we update the full fraction  $\text{atExclTime}^{\gamma}(t_2)$  from the CAS ownership, together with the authoritative ghost state in the atomic points-to, to the latest write timestamp  $t$  to complete the single-writer ownership.  $\square$

### 10.2.3 Proof Sketches for Atomic Operations

The proofs for atomic operations are done in the base logic, after unfolding  $\text{iRC11}$  Hoare triple and WP definitions (Definition 8.5 and Definition 8.4). We then rely on the base logic rules for atomic operations (§7.3) to proceed.

*Proof sketch of AT-READ-SN.* The basis of the proof is to apply **BL-HOARE-READ-AT** (Figure 7.5). In the pre-condition we have  $\ell \sqsupseteq_{\text{sn}} h_0 * @_{V_b}(\ell \xrightarrow{t_x}_{\theta} h)$ , and we need to satisfy the pre-condition of **BL-HOARE-READ-AT**. From  $\ell \sqsupseteq_{\text{sn}} h_0$  we get  $\text{Local}_{\text{A}}(\ell, h, \mathcal{V}.\text{cur})$  (after unfolding the models of Hoare triples and WPs, in the base logic). From  $@_{V_b}(\ell \xrightarrow{t_x}_{\theta} h)$  we get (1)  $\text{Hist}(\ell, h) * \text{Read}^{\sqsupseteq \text{rlx}}(\ell, \alpha_2)$ , because they are objective, and (2)  $\text{Local}_{\text{R}}^{\sqsupseteq \text{rlx}}(\ell, \alpha_2, V_b)$ . Therefore we can call **BL-HOARE-READ-AT**.

Afterwards, we prove the post-condition of **AT-READ-SN** from that of **BL-HOARE-READ-AT**. The most important new observation that we get is  $\text{Local}_{\text{R}}^{\sqsupseteq \text{rlx}}(\ell, \alpha_2 \cup \{r\}, V_b \sqcup \mathcal{V}'.\text{cur})$ , which fits perfectly in **AT-READ-SN**'s requirement of  $@_{V_b \sqcup V'}(\ell \xrightarrow{t_x}_{\theta} h)$ , where it is the case that  $V' = \mathcal{V}'.\text{cur}$ .

We are then left with the observations and facts about the read message. Note that due to  $\ell \sqsubseteq_{\text{sn}} h_0$ , the current view  $\mathcal{V}.\text{cur}$  before the step has observed all events in  $h_0$ , so by the post-condition  $\mathcal{V} \xrightarrow{\text{R:o}, \ell, t, V^?, r} \mathcal{V}'$  from **BL-HOARE-READ-AT**, we know that the read timestamp  $t$  is not earlier than the writes in  $h_0$ , i.e.,  $t \geq \max(\text{dom}(h_0))$ . Furthermore inspection of  $\mathcal{V} \xrightarrow{\text{R:o}, \ell, t, V^?, r} \mathcal{V}'$  is needed to prove the remaining observations, and while that is not trivial, it is rather routine so we elide it here.  $\square$

*Proof sketch of AT-WRITE-SN.* The proof is similar to that of **AT-READ-SN**, but relies on the rule **BL-HOARE-WRITE-AT**. The main difference is in the update of the local atomic writes observation, from  $\text{Local}_{\bar{w}}^{\text{r1x}}(\ell, \alpha_w, V_b)$  to  $\text{Local}_{\bar{w}}^{\text{r1x}}(\ell, \alpha_w \cup \{t\}, V_b \sqcup V')$ . Naturally, a careful inspection of the condition  $\mathcal{V} \xrightarrow{\text{W:o}, \ell, t, \perp, V} \mathcal{V}'$  is needed to establish the remaining observations.  $\square$

*Proof sketch of AT-CAS-SN-GEN.* The proof is similar to those of **AT-READ-SN** and **AT-WRITE-SN**, but relies on the rule **BL-HOARE-CAS**. The 3 main tasks are:

- show that the premise  $\forall v_0, t_0 \geq \max(\text{dom}(h_0)). h(t_0) = (v_0, \_) \Rightarrow \vdash v_0 =^? v_r$  implies  $\forall v_0 \in \text{Readable}(h, \mathcal{V}). \vdash v_0 =^? v_r$ ; and
- show that **AT-CAS-SN-GEN**'s  $(P_{\text{cmp}} \dot{\Rightarrow} \Phi_{\text{cmp}}(\ell_r, h))$  implies **BL-HOARE-CAS**'s  $(P_{\text{cmp}} \multimap \Phi_{\text{cmp}}(\ell_r))$ ; and
- show the observations of **AT-CAS-SN-GEN**'s post-condition from that of **BL-HOARE-CAS**.

The first two tasks are straightforward, mostly by unfolding definitions. Again, the last task requires careful inspection, but is routine and therefore elided here.  $\square$

**CHAPTER SUMMARY.** In this chapter, we presented the interface and the model of the atomic points-to assertion and its related local ownership and observations. The atomic points-to construction is designed to not only support the naturally desired concurrent atomic accesses, but also support strong reasoning principles in typical usage modes. It also supports switching between the different modes, as well as alternating between phases of non-atomic access and of atomic accesses. In the next chapters, we will see the flexibility of the atomic points-to assertion when combined with *invariants* in verifications.

# 11

## *Invariants in Relaxed Memory*

---

In §6.3, we have reviewed Iris invariants as the key tool to share resources for concurrent accesses. In the Iris program logic (and also Iris-derived SC logics), invariants can be used to build concurrent protocols of data structures, by putting *all* shared ownership into a data structure’s invariant. The invariant enforces a user-defined relation on the shared resources, so as to constrain how operations can access and change them. For example, in verifying a linked-list based concurrent queue, one can put the points-to ownership of the queue’s head and tail pointers, as well as all the nodes of the queue, into a single invariant, and state the FIFO protocol on those points-to assertions, within the same invariant.

In RMC separation logics, we also want the facilities of invariants to support concurrent resource sharing. The situation is a bit different, because when moving resources locally owned by a thread—and thus are being interpreted by that thread’s local views—into the “public domain” of invariants, we have to know the views used to interpret those resources, now that they are no longer tied to a thread. The SC-logic idea of putting all resources in a single invariant then appears intractable in RMC logics: those resources may be accessed separately and concurrently by multiple threads, so they may hold at separately different views. If we look at the concurrent queue example again, we see that an enqueueing thread would mostly work with the tail pointer, while a dequeuing one would separately work with the head pointer, so the ownership of the two pointers are likely to hold at different views. In other words, there is no single coherent history of updates to all locations shared in an invariant, and concurrent threads accessing the invariant cannot hope to agree on a consistent view of the *whole* invariant content.

As such, RSL, FSL, GPS, and their descendants<sup>1</sup> opted to restrict invariants to *single-location* invariants (or protocols) which restrict the evolution of a single, shared location. Intuitively, this is sound because in C11 all threads always share a consistent view on the single-location modification order `moℓ`: writes to a single location is always totally ordered. Single-location invariants have proved useful in practice, but they become cumbersome to use when one works with a set of closely related locations, such as in the concurrent queue example. Roughly speaking, one would have a single-location invariant for each of the queue’s head, tail, and nodes, and if one wants to enforce a property that spans multiple of those locations, one would have to invent mechanisms

<sup>1</sup>Vafeiadis and Narayan, “Relaxed separation logic: a program logic for C11 concurrency” [VN13]; Turon et al., “GPS: navigating weak memory with ghosts, protocols, and separation” [TVD14]; Tas-sarotti et al., “Verifying read-copy-update in a logic for weak memory” [TDV15]; Doko and Vafeiadis, “A Program Logic for C11 Memory Fences” [DV16]; Doko and Vafeiadis, “Tackling Real-Life Relaxed Concurrency with FSL+ +” [DV17]; Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17]; He et al., “GPS+: Reasoning About Fences and Relaxed Atom-ics” [He+18].

<sup>2</sup>Michael and Scott, “Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms” [MS96].

to tie those single-location invariants together.

In particular, the verifications done in GPS [TVD14] involve designing multiple extra ghost state to create permissions that can be owned by the protocol participants, and thus allow them to restrict interference by others. For example, the GPS verification for the linked-list based Michael-Scott concurrent queue<sup>2</sup> sets up the head’s single-location invariant to always hold a unique permission, say  $\diamond_0$  to access the first (0) element in the queue, and every node  $i$  holds also a unique permission  $\diamond_{i+1}$  to access the next node after it. A dequeue requires a CAS to update the head pointer, and if that is successful, the dequeue caller acquires  $\diamond_0$ , which then can be used to access the first (0) element resources from the first element’s single-location invariant, including the permission  $\diamond_1$ . The caller needs to put back  $\diamond_1$  into the head’s invariant, because that is the permission for the next dequeuer to access the element 1, which is now the next element to be dequeued. *This contrive setup with extra ghost state would not be needed—and in fact is not needed in SC logics—had we have all resources stored inside a single invariant*, because then we would have the relation between the nodes clearly stated in one place, instead of having the relation broken up in form of multiple permissions.

This long discussion is to motivate *general invariants for multiple locations* in RMC, whose variants will be presented in this chapter. In Part III, we will show how these invariants are used to derive GPS single-location invariants.

The key challenge of general invariants is to identify the views that justify the different parts of the invariant content. One solution is simple: let the clients (of invariants) pick those views, explicitly using the view-at modality (Definition 8.13, §8.5), and then the invariant itself has no extra work to do. This gives rise to iRC11 *objective* invariants, a direct lifting of Iris invariants from iProp to vProp, which we present in §11.1.

However, we also would like to support *cancelable invariants*, those that allow reclaiming the invariant content once the invariant is no longer in use. Instead of the clients, cancelable invariants themselves take care of the one view that justifies all parts of the invariant content, and thus guarantee that *the cancelation of the invariant is synchronized with all accesses to the invariant*. We present iRC11 cancelable invariants in §11.2.

In §11.3, we provide the interface of another invariant form, called *non-atomic invariants*. They are needed for RustBelt Relaxed in the model of Rust’s type system (Part III).

## 11.1 Objective Invariants

**Definition 11.1** (vProp Objective Invariants). The vProp objective invariants directly lift Iris iProp invariants as follows.

$$\llbracket I \rrbracket^{\mathcal{N}} ::= \lambda \_ . \llbracket \forall V . \llbracket I \rrbracket V \rrbracket^{\mathcal{N}}$$

That is, the invariant content  $I : \text{vProp}$  is put inside an Iris invariant at an arbitrary (universally quantified) view  $V$ . This resembles the model of the objective modality (Definition 8.10), hence the name.



$$\begin{array}{c}
\text{objective}(\boxed{I}^{\mathcal{N}}) \\
\text{persistent}(\boxed{I}^{\mathcal{N}})
\end{array}
\quad
\begin{array}{c}
\text{OINV-ALLOC} \\
\triangleright \langle \text{obj} \rangle I \vdash \Rightarrow_{\mathcal{E}} \boxed{I}^{\mathcal{N}}
\end{array}
\quad
\begin{array}{c}
\text{OINV-ACC} \\
\frac{\mathcal{N} \subseteq \mathcal{E}}{\boxed{I}^{\mathcal{N}} \vdash \Rightarrow^{\mathcal{E} \setminus \mathcal{N}} (\triangleright \langle \text{obj} \rangle I * (\triangleright \langle \text{obj} \rangle I \Rightarrow^{\mathcal{E} \setminus \mathcal{N}} *^{\mathcal{E}} \text{True}))}
\end{array}$$
  

$$\begin{array}{c}
\text{OINV-ALLOC-OBJ} \\
\frac{\text{objective}(I)}{\triangleright I \vdash \Rightarrow_{\mathcal{E}} \boxed{I}^{\mathcal{N}}}
\end{array}
\quad
\begin{array}{c}
\text{OINV-ACC-OBJ} \\
\frac{\text{objective}(I) \quad \mathcal{N} \subseteq \mathcal{E}}{\boxed{I}^{\mathcal{N}} \vdash \Rightarrow^{\mathcal{E} \setminus \mathcal{N}} (\triangleright I * (\triangleright I \Rightarrow^{\mathcal{E} \setminus \mathcal{N}} *^{\mathcal{E}} \text{True}))}
\end{array}$$

FIGURE 11.1: iRC11 rules for objective invariants

Objective invariants satisfy the rules in [Figure 11.1](#). That is, the rules are similar to those of Iris iProp invariants ([Figure 6.4](#)), except that they require the invariant content  $I$  to be objective, or be placed under an objective modality. More specifically, with [OINV-ALLOC](#), we can allocate a vProp objective invariant if we have the invariant content  $I$  under the objective modality, and with [OINV-ACC](#) we can access the invariant content atomically, but we only get  $I$  also under the objective modality. The rules [OINV-ALLOC-OBJ](#) and [OINV-ACC-OBJ](#) are derived from [OINV-ALLOC](#) and [OINV-ACC](#), respectively, using the rules [OBJMOD-INTRO](#) and [OBJMOD-ELIM](#) (§8.4).

Intuitively, if a client only stores objective resources, which include pure facts ( $\phi$ ) and ghost state ( $\{\bar{a}\}^{\gamma}$ ),<sup>3</sup> then objective invariants work the same as Iris traditional invariants. If the client, however, wishes to put points-to assertions into an objective invariants, then they should make those resources objective, by making their interpreting views explicit using the view-at modality ([Definition 8.13](#)). In particular, we can apply [VA-INTRO](#) ([Figure 8.3](#)) to turn an atomic points-to  $\ell \mapsto_{\theta}^{\gamma} h$  can be turn into the form  $@_V(\ell \mapsto_{\theta}^{\gamma} h)$  (knowing that  $\exists V$ ) which is objective (see [Figure 8.3](#)) and thus can be put inside objective invariants using [OINV-ALLOC](#). Fortunately, we have established in sections §10.1.1 to §10.1.3 that the atomic access rules only require and return the atomic points-to ownership at some arbitrary views  $V_b$  and  $V_b \sqcup V'$ , respectively.

<sup>3</sup>see [PURE-OBJ](#) and [GHOST-OBJ](#), §8.4

In other words, the atomic access rules using atomic points-to (§10.1) are compatible with objective invariants, and are sufficient to verify algorithms with concurrent protocols on shared atomic points-to assertions. We will see such examples in [Chapter 12](#) and in [Part IV](#).

We note that we can also use [VA-INTRO](#) to make *non-atomic* points-to assertions objective, and put them in invariants to transfer them to other threads. However, the non-atomic points-to can only be used without the view-at modality. Fortunately, the atomic access rules allow us to acquire the view-seen observations needed to remove the view-at modality from the non-atomic points-to, using [VA-ELIM](#). For more details, please recall the discussion about resource transfer at the end of §10.1.2, which uses the rules [AT-WRITE-SW-REL](#) and [AT-READ-SN-ACQ](#). Nevertheless, we will also see examples for resource transfer in [Chapter 12](#).

## 11.2 Cancelable Invariants

If we put some resources in objective invariants, the resources are in the public domain forever. But there are situations where we want to reclaim the resources in the invariants, after which point we know that the invariants are no longer needed any more. For example, we would like to have a per-object invariant to govern the protocol of a data structure, but when the data structure object is deallocated, the invariant should become obsolete.

Iris supports a kind of invariants called *cancelable invariants* where an invariant can be canceled to reclaim the invariant. We would like to support this kind of invariants for  $\nu\text{Prop}$ , but we need some extra work to make the returned resources useable: we need to track more carefully the view at which the resources are put inside the invariant, so that after the cancelation we can eliminate the view-at modality protecting those resources, and thus the client can own them locally.

### 11.2.1 The Interface of $\nu\text{Prop}$ Cancelable Invariants

<sup>4</sup>These are called *raw* cancelable invariants in the RustBelt Relaxed paper [Dan+20a].

<sup>5</sup>Please see Figure 14.1 for a few key rules of Iris traditional cancelable invariants. See also [Iri22, §10.1].

We present the interface of  $\nu\text{Prop}$  cancelable invariants in Figure 11.2.<sup>4</sup> Note that we did not present Iris  $\text{iProp}$  cancelable invariants, but they only differ from the  $\nu\text{Prop}$  ones in the parts concerning views.<sup>5</sup>

The interface of cancelable invariants involves two kinds of assertions: (1) a persistent, objective assertion  $\boxed{I}^{\gamma, \mathcal{N}}$  that an invariant with content  $I$  exists in the namespace  $\mathcal{N}$  with an identifier  $\gamma$ ; and (2) a fractional and timeless *invariant token* assertion  $\heartsuit_q^\gamma$  (also identified by  $\gamma$ ) that is needed to know that the invariant is not yet canceled.

**INVARIANT ALLOCATION.** With the invariant content  $I$ , we can allocate a cancelable invariant using **CINV-ALLOC**. Afterwards, we know that the invariant exists ( $\boxed{I}^{\gamma, \mathcal{N}}$ ), and we own the full fraction  $\heartsuit_1^\gamma$  of its invariant token. The invariant token is fractional, as shown in **CINV-TOK-FRAC-VALID** and **CINV-TOK-FRAC**, so that we can split the full fraction into pieces and give them to multiple threads and they can access the invariant concurrently. **CINV-TOK-OBJ-SPLIT** allows us to split a fraction of a token identified by  $\gamma$  into two parts, one of which is objective, which in turn can be easily put inside an invariant (which could possibly be the invariant with the same identifier  $\gamma$  as the token).

**INVARIANT ACCESS.** A fraction of the invariant token is needed to open the invariant with the same identifier  $\gamma$ , as can be seen in **CINV-ACC**. The opening is a mask-changing fancy update, after which we receive the invariant content  $I$  under a later. Once we are done, we must return  $I$  and close the invariant with a reverse mask-changing fancy update. Recall from §6.7 that mask-changing fancy updates are needed to prevent opening an invariant twice, and to limit the invariant accesses to a single, atomic step of computation. More concretely, recall that the rules **WP-INV** and **HOARE-INV** for opening traditional Iris invariants around an atomic step are derived from **WP-ATOMIC** and **INV-ACC**. We apply the same method

$$\begin{array}{c}
 \text{objective}(\boxed{I}^{\gamma, \mathcal{N}}) \\
 \text{persistent}(\boxed{I}^{\gamma, \mathcal{N}})
 \end{array}
 \quad
 \text{timeless}(\heartsuit_q^\gamma)
 \quad
 \text{CINV-ALLOC} \quad \triangleright I \vdash_{\mathcal{E}} \exists \gamma. \heartsuit_1^\gamma * \boxed{I}^{\gamma, \mathcal{N}}$$
  

$$\begin{array}{c}
 \text{CINV-ACC} \\
 \hline
 \boxed{I}^{\gamma, \mathcal{N}} * \heartsuit_q^\gamma \vdash_{\mathcal{E}} \exists \gamma. \heartsuit_1^\gamma * \boxed{I}^{\gamma, \mathcal{N}}
 \end{array}
 \quad
 \begin{array}{c}
 \mathcal{N} \subseteq \mathcal{E} \\
 \hline
 \left( \exists V_i. (\sqcup_{V_i} \triangleright I) * \left( (\sqcup_{V_i} \triangleright I) \stackrel{\mathcal{E} \setminus \mathcal{N}}{\multimap} \heartsuit_q^\gamma \right) \right)
 \end{array}$$
  

$$\begin{array}{c}
 \text{CINV-CANCEL} \\
 \hline
 \boxed{I}^{\gamma, \mathcal{N}} * \heartsuit_1^\gamma \vdash_{\mathcal{E}} \triangleright I
 \end{array}
 \quad
 \begin{array}{c}
 \mathcal{N} \subseteq \mathcal{E} \\
 \hline
 \boxed{I}^{\gamma, \mathcal{N}} * \heartsuit_1^\gamma \vdash_{\mathcal{E}} \triangleright I
 \end{array}$$
  

$$\begin{array}{c}
 \text{CINV-TOK-FRAC-VALID} \\
 \heartsuit_q^\gamma \vdash q \in (0, 1]
 \end{array}
 \quad
 \begin{array}{c}
 \text{CINV-TOK-FRAC} \\
 \heartsuit_q^\gamma * \heartsuit_{q'}^\gamma \dashv \vdash \heartsuit_{q+q'}^\gamma
 \end{array}
 \quad
 \begin{array}{c}
 \text{CINV-TOK-OBJ-SPLIT} \\
 \heartsuit_{q+q'}^\gamma \vdash \heartsuit_q^\gamma * \langle \text{obj} \rangle \heartsuit_{q'}^\gamma
 \end{array}$$
  

$$\begin{array}{c}
 \text{WP-CINV} \\
 \hline
 \text{atomic}(e) \quad \mathcal{N} \subseteq \mathcal{E} \\
 \forall V_i. \sqcup_{V_i} \triangleright I \vdash \text{wp}_{\mathcal{E} \setminus \mathcal{N}} e \text{ in } \pi \{v. \sqcup_{V_i} \triangleright I * Q\} \\
 \hline
 \boxed{I}^{\gamma, \mathcal{N}} * \heartsuit_q^\gamma \vdash \text{wp}_{\mathcal{E}} e \text{ in } \pi \{v. \heartsuit_q^\gamma * Q\}
 \end{array}
 \quad
 \begin{array}{c}
 \text{HOARE-CINV} \\
 \hline
 \text{atomic}(e) \quad \mathcal{N} \subseteq \mathcal{E} \\
 \{\sqcup_{V_i} \triangleright I * P\} e \text{ in } \pi \{\sqcup_{V_i} \triangleright I * Q\}_{\mathcal{E} \setminus \mathcal{N}} \\
 \hline
 \boxed{I}^{\gamma, \mathcal{N}} \vdash \{\heartsuit_q^\gamma * P\} e \text{ in } \pi \{v. \heartsuit_q^\gamma * Q\}_{\mathcal{E}}
 \end{array}$$

FIGURE 11.2: iRC11 rules for cancelable invariants

with **WP-ATOMIC** and **CINV-ACC** to derive the explicit rules **WP-CINV** and **HOARE-CINV** for opening vProp cancelable invariants.

The invariant content  $I$  we receive from **CINV-ACC**, however, unlike that from the rule **INV-ACC** for traditional Iris invariants, are protected under a *view-join* modality (**Definition 8.14**), *i.e.*, what we get from opening the invariant and what we have to return to close the invariant are both  $\sqcup_{V_i} \triangleright I$ . Recall the intuition of  $\sqcup_{V_i} P$ : it asserts the ownership of the resource  $P$  which hold at a view whose difference with the implicit interpreting view  $V$  is  $V_i$ . That is,  $P$  holds at  $V_i \sqcup V$ . This means that a client of the cancelable invariant only gains access to the invariant content  $I$  at an *arbitrary larger* view  $V_i \sqcup V$  ( $@_{V_i \sqcup V} \triangleright I$ ), where in practice  $V_i$  represents the view at which  $I$  is currently justified. During its access to  $I$ , the client can update the current view from  $V$  to some larger view  $V'$  ( $V \sqsubseteq V'$ ), so long as it returns the invariant content  $I$  at the view  $V_i \sqcup V'$  ( $@_{V_i \sqcup V'} \triangleright I$ ), *i.e.*, it returns  $\sqcup_{V_i} \triangleright I$ .

The use of the view-join modality in **CINV-ACC** therefore enforces two requirements on the clients of cancelable invariants. The first requirement is that clients of cancelable invariants should be able to work with the content  $I$  justified at some arbitrary view  $V_i \sqcup V$  (where  $V$  is the client's current view at the opening of the invariant). This requirement is the same as that for clients of objective invariants, and therefore can be mitigated in the same ways as discussed in the previous section (§11.1): objective resources do not care about views, and atomic access rules can work with the atomic points-to ownership at some arbitrary view, while views for other resources like non-atomic points-to need to be tracked more carefully and rely on the seen-view observations from the atomic operations. Again, we will see this point worked out more concretely in the example verifications in **Chapter 12**.

The second requirement is that the client cannot return  $I$  at a too big view: the view  $V_i \sqcup V'$  must be sufficient to justify  $I$  (where  $V'$  is the client's current view at the closing of the invariant).<sup>6</sup> We note that this requirement is only enforced on non-objective resources in  $I$ .

<sup>6</sup>Note that if the client returns  $I$  at a view smaller than  $V_i \sqcup V'$ , then it can always return  $I$  at  $V_i \sqcup V'$ , thanks to view monotonicity of  $I$ .

Fortunately, the atomic access rules using atomic points-to (in §10.1.1 to §10.1.3) are also designed to be compatible with this requirement: if we provides an atomic points-to  $@_{V_i \sqcup V} \ell \xrightarrow{t}_\theta h$  to the pre-condition of one of the rules, we will receive some  $@_{V_i \sqcup V \sqcup V'} \ell \xrightarrow{t}_\theta h'$  in the post-condition, which is indeed  $@_{V_i \sqcup V'} \ell \xrightarrow{t}_\theta h$  because  $V \sqsubseteq V'$ . We will see this point more concretely also in Chapter 12.

Note that we can switch between the view-join and view-at modalities easily using **VJ-VA-ACC**. In fact, with **VJ-VA-ACC** and the rules in §10.1.1 to §10.1.3, we can derive atomic access rules that take an atomic points-to in the form  $\sqcup_{V_i} \ell \xrightarrow{t}_\theta h$  in the pre-condition, and return an updated atomic points-to in the form  $\sqcup_{V_i} \ell \xrightarrow{t}_\theta h'$  in the post-condition. We will see the derivations of those rules also in §12.2 (see Figure 12.5).

**INVARIANT CANCELATION.** The second requirement by the access rule **CINV-ACC** is what guarantees the soundness of the cancelation rule **CINV-CANCEL**. Cancelation needs to maintain the following safety guarantee.

**Property 11.2** (Cancelation Safety).

*An invariant's cancellation must happen-after all accesses to it.*

(CANCEL-SAFE)

Nevertheless, **CINV-CANCEL** simply says that with we can trade in the full fraction  $\heartsuit_1^\gamma$  of the invariant token for the invariant  $\boxed{I}^{\gamma, \mathcal{N}}$  to cancel it and get back the invariant content  $I$  locally without any view-explicit modality (albeit under a later, as usual). As such, except for the uses of the view-join modality in **CINV-ACC**, the core interface of iRC11 cancelable invariants (**CINV-ALLOC**, **CINV-ACC**, and **CINV-CANCEL**) is exactly the same as that of traditional Iris cancel invariants that are sound only for SC logics.<sup>7</sup> The reason why the cancelation rule maintains **CANCEL-SAFE** (i.e., race-free and safe) for RMC, and why the relaxed memory effects can be localized in just the view-join modality used in **CINV-ACC**, is rather hard to explain intuitively, without looking into the model of invariant tokens. We therefore delay this explanation until §11.2.2.

<sup>7</sup>see Figure 14.1 and [Iri22, §10.1]

**STRONGER ALLOCATION RULES.** Figure 11.3 provides several stronger rules for cancelable allocation and access. **CINV-ALLOC-OPEN** strengthens **CINV-ALLOC** by not requiring the invariant content  $I$  upfront. Instead the client is first given a fresh identifier  $\gamma$  for the invariant token, and so the client can pick the invariant content  $I$  that may depend on  $\gamma$ . The client then receives the invariant assertion  $\boxed{I}^{\gamma, \mathcal{N}}$  but the invariant does not hold yet (hence the mask does not include  $\mathcal{N}$ ). Once the client provides  $\triangleright I$ , the invariant is established and the client receives the full invariant token  $\heartsuit_1^\gamma$ .

**CINV-ALLOC-FRAC** strengthens **CINV-ALLOC** in a slightly different way. The client first receives some fraction  $q$  of the invariant token with a fresh identifier  $\gamma$ , then the client can pick and provide the invariant content  $I$  that may contain  $\heartsuit_q^\gamma$  itself. After the invariant is established the client receives the remaining fraction  $\heartsuit_{q'}^\gamma$ , i.e.,  $q + q' = 1$ . Note that  $q$  and  $q'$  are picked by the client.

$$\begin{array}{c}
 \text{CINV-ALLOC-OPEN} \qquad \qquad \qquad \mathcal{N} \subseteq \mathcal{E} \\
 \hline
 \vdash \models_{\mathcal{E}} \exists \gamma. \forall I. \mathcal{E} \models^{\mathcal{E} \setminus \mathcal{N}} \boxed{I}^{\gamma, \mathcal{N}} * (\triangleright I \mathcal{E} \setminus \mathcal{N} \multimap^{\mathcal{E}} \heartsuit_1^{\gamma})
 \\
 \\
 \text{CINV-ACC-GEN} \qquad \qquad \qquad \mathcal{N} \subseteq \mathcal{E} \\
 \hline
 \boxed{I}^{\gamma, \mathcal{N}} * \heartsuit_q^{\gamma} \vdash \mathcal{E} \models^{\mathcal{E} \setminus \mathcal{N}} \heartsuit_q^{\gamma} * \exists V_i. (\sqcup_{V_i} \triangleright I) * \wedge \left\{ \begin{array}{l}
 (\heartsuit_q^{\gamma} * (\sqcup_{V_i} \triangleright I)) \mathcal{E} \setminus \mathcal{N} \multimap^{\mathcal{E}} \heartsuit_q^{\gamma} \\
 \forall V', P. (@_{V'} \heartsuit_q^{\gamma} * (@_{V'} \heartsuit_q^{\gamma} \multimap^{\mathcal{E} \setminus \mathcal{N}} (\sqcup_{V_i} \triangleright I) * P)) \mathcal{E} \setminus \mathcal{N} \multimap^{\mathcal{E}} P \\
 \heartsuit_1^{\gamma} \multimap (\exists V_i * \mathcal{E} \setminus \mathcal{N} \models^{\mathcal{E}} \text{True})
 \end{array} \right.
 \\
 \\
 \text{CINV-ALLOC-FRAC} \qquad \qquad \qquad q + q' = 1 \\
 \hline
 \vdash \models_{\mathcal{E}} \exists \gamma. \heartsuit_q^{\gamma} * (\forall I. \triangleright I \mathcal{E} \setminus \mathcal{N} \multimap^{\mathcal{E}} \heartsuit_{q'}^{\gamma} * \boxed{I}^{\gamma, \mathcal{N}})
 \end{array}$$

FIGURE 11.3: Stronger iRC11 rules for cancelable invariants

A STRONGER ACCESS RULE. **CINV-ACC-GEN** (Figure 11.3) generalizes the access rule **CINV-ACC** in several non-trivial ways. First, one does not need to trade in the invariant token  $\heartsuit_q^{\gamma}$  to access the invariant: one does need to provide it, but then receives it back immediately together with the invariant content  $\sqcup_{V_i} \triangleright I$ . Second, there are 3 options to close the invariant.

The first option is similar to the closing viewshift of **CINV-ACC**: to close the invariant, the client gives back the invariant content  $\sqcup_{V_i} \triangleright I$ . As the client did not trade in the invariant token, they need to provide the token at the closing, which they will get back immediately afterwards.<sup>8</sup>

The second closing option strengthens the first one further: to close the invariant, the client provides (1) the invariant token  $\heartsuit_q^{\gamma}$  at some view  $V'$  of its choice, and (2) a wand viewshift  $(@_{V'} \heartsuit_q^{\gamma} \multimap^{\mathcal{E} \setminus \mathcal{N}} (\sqcup_{V_i} \triangleright I) * P)$ . Intuitively, the invariant needs (1) to update its internal tracking of the current invariant view. After that it feeds (1) into (2), a logical *continuation* picked by the client that, when receiving the same token, will produce the invariant content  $\sqcup_{V_i} \triangleright I$  and some remaining resource  $P$ . The invariant then consumes  $\sqcup_{V_i} \triangleright I$  to re-establish and close the invariant, and returns the remaining resource  $P$  to the client.

The last closing option embeds the cancelation rule **CINV-CANCEL**: instead of returning the invariant content  $I$ , the client can trade the full fraction  $\heartsuit_1^{\gamma}$  of the invariant token for (1) a seen-view observation  $\sqsubseteq V_i$  of the same view  $V_i$  that the client has received at the invariant opening, so that they can eliminate the view-join modality from  $I$ ; and (2) a reverse mask-changing fancy update  $\mathcal{E} \setminus \mathcal{N} \models^{\mathcal{E}} \text{True}$  without additional requirement to close the invariant. We note that **CINV-CANCEL** is derivable from **CINV-ACC-GEN**.

Finally, we note that the stronger rules (**CINV-ALLOC-OPEN**, **CINV-ALLOC-FRAC**, and **CINV-ACC-GEN**) *without* the view-join modality are all sound in traditional Iris SC logics. In those logics, the second closing option of **CINV-ACC-GEN** also coincides with the rule's first option.

### 11.2.2 The Model of Cancelable Invariants

To model iRC11 cancelable invariants, we need more ghost state to encode the access and cancelation protocol, which will be stored in the

<sup>8</sup>Another possible generalization is that the fraction  $q$  needed at closing does not need to be the same fraction used for opening the invariant.

ghost location  $\gamma$ —the invariant identifier.

**Definition 11.3** (RA for iRC11 Cancelable Invariants). We need the fractional view-lattice RA  $\text{FRACVIEWR} = \text{AUTH}(\text{OPTION}(\text{FRAC} \times \text{LAT}(\text{View})))$ . We define notations for two kinds of elements of the RA.

$$\begin{aligned}\text{PartialV}_q(V_p) &::= \circ(\text{Some}(q, V_p)) \\ \text{FullV}(V_f) &::= \bullet(\text{Some}(1, V_f))\end{aligned}$$

**Definition 11.4** (Model of iRC11 Cancelable Invariants). The model of invariant tokens and cancelable invariants is given directly in  $\text{vProp}$ , using the RA and objective invariants, as follows.

$$\begin{aligned}\heartsuit_q^\gamma &::= \exists V_{\text{tok}}. [\text{PartialV}_q(V_{\text{tok}}) : \text{FRACVIEWR}]^\gamma * \exists V_{\text{tok}} \\ &\quad (\text{CINV-MODEL-TOK}) \\ [I]^{\gamma, \mathcal{N}} &::= \boxed{\exists V_i. @_{V_i} ([\text{PartialV}_1(V_{\text{tok}})]^\gamma \vee [\text{FullV}(V_i)]^\gamma * I)}^\mathcal{N} \\ &\quad (\text{CINV-MODEL})\end{aligned}$$

**INVARIANT TOKENS.** First of all, invariant tokens  $\heartsuit_q^\gamma$  are *view-dependent* assertions: even though owning a token  $\heartsuit_q^\gamma$  means owning only the ghost element  $\text{PartialV}_q(V_{\text{tok}})$ , this ghost ownership is tied to the current implicit view  $V$  at which the assertion is interpreted through the *token view*  $V_{\text{tok}}$ . In particular, the ghost element  $\text{PartialV}_q(V_{\text{tok}})$  records both the fraction  $q$ , which represents how much of the invariant this token owns, and the token view  $V_{\text{tok}}$ , which represents what this particular fractional token has *observed*, i.e., what invariant accesses this fractional token has participated in. The model requires that  $V$ —the current implicit view at which the token is interpreted—has also at least observed what  $\heartsuit_q^\gamma$  has observed:  $\exists V_{\text{tok}}$  (the seen-view observation, see [Definition 8.12](#)).

**INVARIANT ASSERTIONS.** The model of invariant assertions  $[I]^{\gamma, \mathcal{N}}$  simply encodes the two possible states of the invariant: “active” or “canceled”. Thus it is an *objective invariant* (§11.1) of a disjunction (see [CINV-MODEL](#)). The right-hand side of the disjunction encodes the active state, where the content  $I$  is still available in the invariant at some *content view*  $V_i$ . In the active state the underlying invariant also owns the authoritative ghost element  $\text{FullV}(V_i)$  that records the view  $V_i$  in the ghost location  $\gamma$ . The left-hand side of the disjunction encodes the canceled state, which asserts ownership of the full fractional element  $\text{PartialV}_1(V_i)$ . Recall that the invariant assertion  $[I]^{\gamma, \mathcal{N}}$  itself is objective.<sup>9</sup> The underlying invariant’s content is also objective: it is wrapped under a view-at modality of the content view  $V_i$ .<sup>10</sup> The relation between the content view  $V_i$  and the token views  $V_{\text{tok}}$ ’s is managed entirely by the ghost elements  $\text{FullV}(V_i)$  and  $\text{PartialV}_q(V_{\text{tok}})$ .

**Concept 11.5** (Synchronized Ghost State). In essence, by its model in [CINV-MODEL-TOK](#), cancelable invariant tokens are just ghost state. However, unlike the vanilla ghost state ownership which is objective,<sup>11</sup> invariant tokens are not objective as they are tied to their owner’s observations. We generally call these “synchronized ghost state”. The RA

<sup>9</sup>see [Figure 11.1](#)

<sup>10</sup>see [Figure 8.3](#)

<sup>11</sup>see [GHOST-OBJ](#), §8.4

FRACVIEWR has two interesting kinds of elements that help us implement the idea of “synchronicity”: (1) the unique element  $\text{FullV}(V_f)$  that is used to record the *full view*  $V_f$ , and (2) fractional elements  $\text{PartialV}_q(V_p)$  that are used to associate some *partial view*  $V_p$  with some fraction  $q$ . FRACVIEWR is built to maintain the following property:

*The join of all partial views (the  $V_p$ ’s from all  $\text{PartialV}_q(V_p)$ ’s) is always equal to the full view  $V_f$  in  $\text{FullV}(V_f)$ .*  
(SYNC-GHOST)

This property guarantees that the partial view  $V_p$  of the full fractional element  $\text{PartialV}_1(V_p)$  is actually equal to the full view  $V_f$  of  $\text{FullV}(V_f)$ :  $V_p = V_f$ . The **SYNC-GHOST** property is what we require for view-dependent ghost state to be synchronized ghost state. By synchronized ghost state we mean any ghost construction that is built on the notion of *fractional observations*. That is, the ghost state has fractional elements that track the subjective observations of the threads the elements are tied to, and, most importantly, the full fractional element is guaranteed to have tracked all observations.

In the case of cancelable invariants, the observations are the views around which threads access and update the invariant content  $I$ . Intuitively, we record the view  $V_i$  of the invariant content  $I$  as the full view in  $\text{FullV}(V_i)$  (see **CINV-MODEL**). The token view  $V_{\text{tok}}$  in the ghost element  $\text{PartialV}_q(V_{\text{tok}})$  of some token  $\heartsuit_q^\gamma$  tracks the changes to  $I$  made by each access that  $\heartsuit_q^\gamma$  participated in. By **SYNC-GHOST**, the full token view  $V_{\text{tok\_full}}$  of the full token  $\heartsuit_1^\gamma$  will thus be equal to the content view  $V_i$ . Consequently a thread owning  $\heartsuit_1^\gamma$  must have observed all changes to the invariant content  $I$ , i.e., it must have  $\sqsupseteq V_i$ . Effectively, **CANCEL-SAFE** is maintained and **CINV-CANCEL** can safely eliminate the view-at modality protecting  $I$  (using **VA-ELIM**, §8.5) and return  $\triangleright I$  at the canceling thread’s current view.

**FORMAL PROPERTIES OF FRACVIEWR**. To maintain **SYNC-GHOST**, the RA FRACVIEWR admits the rules in Figure 11.4. **CINV-MODEL-SYNC** says that any token view  $V_{\text{tok}}$  is included in the content view  $V_i$ , and the full token view  $V_{\text{tok\_full}}$  of  $\heartsuit_1^\gamma$  is exactly  $V_i$ . **CINV-MODEL-JOIN** requires that the fractions consistently cannot sum up to more than 1, and also allows us to join together partial token views of the fractions when we are recollecting them. **CINV-MODEL-UPDATE** formalizes a restriction on how the ghost state can grow: we can update a token view  $V_{\text{tok}}$  by extending it with some  $V'$  *only if* we simultaneously update the content view  $V_i$  in the same way. This makes sure that every change in the full view  $V_i$  is accounted for by some token view  $V_{\text{tok}}$ , and thus **SYNC-GHOST** is maintained.

Formally, **CINV-MODEL-SYNC** comes from validity of FRACVIEWR. If we own both  $\text{PartialV}_1(V_p)$  and  $\text{FullV}(V_f)$ , by **GHOST-OP** and **GHOST-VALID** (§6.2), we have  $\text{valid}(\text{FullV}(V_f) \cdot \text{PartialV}_1(V_p)) = \text{valid}(\bullet(\text{Some}(1, V_f)) \circ (\text{Some}(1, V_p)))$ . By **AUTH-BOTH-VALID** (§6.9), we have that  $\text{Some}(1, V_p) \preceq \text{Some}(1, V_f)$ . By the definition of RA inclusion (**RA-INCL**, §6.2), it must be the case that  $\text{Some}(1, V_p) = \text{Some}(1, V_f)$ , i.e.,  $V_p = V_f$ .



$$\begin{array}{l}
\text{CINV-MODEL-SYNC} \\
\boxed{\text{FullV}(V_i)}^\gamma * \boxed{\text{PartialV}_q(V_{\text{tok}})}^\gamma \vdash V_{\text{tok}} \sqsubseteq V_i \wedge (q = 1 \Rightarrow V_{\text{tok}} = V_i) \\
\\
\text{CINV-MODEL-UPDATE} \\
\boxed{\text{FullV}(V_i)}^\gamma * \boxed{\text{PartialV}_q(V_{\text{tok}})}^\gamma \Rightarrow \boxed{\text{FullV}(V_i \sqcup V')}^\gamma * \boxed{\text{PartialV}_q(V_{\text{tok}} \sqcup V')}^\gamma \\
\\
\text{CINV-MODEL-JOIN} \\
\boxed{\text{PartialV}_q(V)}^\gamma * \boxed{\text{PartialV}_{q'}(V')}^\gamma \vdash \boxed{\text{PartialV}_{q+q'}(V \sqcup V')}^\gamma * q + q' \in (0, 1]
\end{array}$$

FIGURE 11.4: Properties of the RA FRACVIEWR for cancelable invariants

PROOF SKETCHES. To understand how the model works, we briefly present the proofs of **CINV-CANCEL** and **CINV-ACC**.

*Proof sketch of CINV-CANCEL.* We prove the rule in vProp. After unfolding the model (**Definition 11.4**), we have the following goal.

| Context:                                                                                                                                                                                                                                           | Goal:                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| $ \begin{array}{l} \boxed{\text{PartialV}_1(V_{\text{tok}})}^\gamma * \sqsupseteq V_{\text{tok}} \\ \boxed{\exists V_i. @_{V_i} (\boxed{\text{PartialV}_1(V_{\text{tok}})}^\gamma \vee \boxed{\text{FullV}(V_i)}^\gamma * I)}^\gamma \end{array} $ | $ \begin{array}{l} \mathcal{N} \\ \Rightarrow_{\mathcal{E}} \triangleright I \end{array} $ |

We then open the underlying objective invariant using **OINV-ACC-OBJ** and find a content view  $V_i$  and the two possibilities for the invariant state. If the invariant were in the cancelled state (the left disjunct), we would have *two* full fractional  $\text{PartialV}_1(\_)$  and **CINV-MODEL-JOIN** would give us contradiction from  $1 + 1 \leq 1$ . Thus the underlying invariant must be in the active state (the right disjunct).

By owning the full fraction, with **CINV-MODEL-SYNC** we know that  $V_{\text{tok}} = V_i$ , so by owning  $\sqsupseteq V_{\text{tok}}$ , the thread must have observed all changes to the invariant content:  $\sqsupseteq V_i$ . With that, we now can take the content  $@_{V_i} V_i$  out of the invariant and eliminate the view-at modality with **VA-ELIM**, and return  $\triangleright I$  for the user. To finish the proof, we put  $\boxed{\text{PartialV}_1(V_{\text{tok}})}^\gamma$  in to switch the underlying invariant to the cancelled state and close it.  $\square$

*Proof sketch of CINV-ACC.* We also prove the rule in vProp. As in cancellation, we unfold the model, then open the underlying invariant with **OINV-ACC-OBJ** and deduce that it must be in the active state. Our goal then looks as follows.

| Context:                                                                                                                                                                                                                                                                                     | Goal:                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $ \begin{array}{l} (\triangleright \exists V_i. \dots) \mathcal{E} \setminus \mathcal{N} \Rightarrow^{\mathcal{E}} \text{True} \\ \boxed{\text{PartialV}_q(V_{\text{tok}})}^\gamma * \sqsupseteq V_{\text{tok}} \\ \boxed{\text{FullV}(V_i)}^\gamma * @_{V_i} \triangleright I \end{array} $ | $ \begin{array}{l} \exists V_i. \sqcup V_i \triangleright I * ((\sqcup V_i \triangleright I) \mathcal{E} \setminus \mathcal{N} \Rightarrow^{\mathcal{E}} \heartsuit_q^\gamma) \end{array} $ |

We instantiate the existential quantification with  $V_i$ , and then use **VA-TO-VJ** (**Figure 8.3**) to upgrade the invariant content  $I$  from the view-at modality to the view-join modality, so that we can discharge the left-hand side of the goal. We are then left to prove the closing wand viewshift.



After introduction, our goal looks as follows.

| Context:                                                                                                                                                                                                                                                                    | Goal:                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $(\triangleright \exists V_i. \dots) \mathcal{E} \backslash \mathcal{N} \not\Rightarrow^{\mathcal{E}} \text{True}$<br>$\boxed{\text{PartialV}_q(V_{\text{tok}})}^\gamma * \sqsupseteq V_{\text{tok}}$<br>$\boxed{\text{FullV}(V_i)}^\gamma * \sqcup_{V_i} \triangleright I$ | $\mathcal{E} \backslash \mathcal{N} \Rightarrow^{\mathcal{E}} \exists V'_{\text{tok}}. \boxed{\text{PartialV}_q(V'_{\text{tok}})}^\gamma * \sqsupseteq V'_{\text{tok}}$ |

We now use **VJ-ELIM-VA** (also [Figure 8.3](#)) to turn  $\sqcup_{V_i} \triangleright I$  and  $\sqsupseteq V_{\text{tok}}$  into  $@_{V_i \sqcup V'} \triangleright I$  for some  $V' \sqsupseteq V_{\text{tok}}$  that we know  $\sqsupseteq V'$ . We then use **CINV-MODEL-UPDATE** to update  $\boxed{\text{FullV}(V_i)}^\gamma * \boxed{\text{PartialV}_q(V_{\text{tok}})}^\gamma$  to  $\boxed{\text{FullV}(V_i \sqcup V')}^\gamma * \boxed{\text{PartialV}_q(V_{\text{tok}} \sqcup V')}^\gamma$ .

We then use the closing viewshift  $(\triangleright \exists V_i. \dots) \mathcal{E} \backslash \mathcal{N} \not\Rightarrow^{\mathcal{E}} \text{True}$  with the view  $V_i \sqcup V'$  and the resources  $\boxed{\text{FullV}(V_i \sqcup V')}^\gamma$  and  $@_{V_i \sqcup V'} \triangleright I$  to re-establish and close the invariant. We are left with the goal for the invariant token.

| Context:                                                                      | Goal:                                                                                                      |
|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| $\boxed{\text{PartialV}_q(V_{\text{tok}} \sqcup V')}^\gamma * \sqsupseteq V'$ | $\exists V'_{\text{tok}}. \boxed{\text{PartialV}_q(V'_{\text{tok}})}^\gamma * \sqsupseteq V'_{\text{tok}}$ |

From  $V' \sqsupseteq V_{\text{tok}}$ , we know that  $V_{\text{tok}} \sqcup V' = V'$ , so this is easily done.  $\square$

### 11.3 Non-Atomic Invariants

Iris additionally provides a derived form of invariants where the access can be non-atomic, *i.e.*, it can span multiple steps of execution. The catch is that each such access can only be done by one thread at a time. This form of invariants, called non-atomic invariants, is needed to model unique reference types in RustBelt.<sup>12</sup> We will thus also need non-atomic invariants for our RustBelt Relaxed work ([Part III](#)).

Fortunately, Iris non-atomic invariants can be proven sound in relaxed memory without any change in the interface! Naturally, the model of iRC11 non-atomic invariants still needs to handle relaxed memory effects, but it manages to encapsulate them within the interface. This is sound, intuitively because non-atomic invariants are meant to be thread-local—*i.e.* being accessed by only the current thread—so the thread is always synchronized with the invariant content. The model carefully tracks the view of the invariant content, employing an RA similar to that of cancelable invariants. The model and the proofs of iRC11 non-atomic invariants were constructed by Jacques-Henri Jourdan, as thus are not considered part of this dissertation and will not be presented here. The exact definitions are available in the Coq development of iRC11.<sup>13</sup>

Nevertheless, we present the interface of iRC11 non-atomic invariants in [Figure 11.5](#), which, again, is exactly the same as that of Iris-SC.<sup>14</sup> Like cancelable invariants, non-atomic invariants also have two kinds of assertions: (1) a persistent, objective assertion  $\text{Nalnv}^{p, \mathcal{N}}(I)$  that a non-atomic invariant with content  $I$  exists in the namespace  $\mathcal{N}$  with of invariant pool  $p$ ; and (2) a timeless *invariant token* assertion  $[\text{Na} : p. \mathcal{E}]$  that is needed to access the invariants in the set  $\mathcal{E}$  under the pool  $p$ .

Invariant pools allow us to have separate pools of invariants with their own namespaces and tokens. Intuitively, we can think of pools as

<sup>12</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [[Jun+18a](#)].

<sup>13</sup>[https://gitlab.mpi-sws.org/iris/gpfs1/-/blob/master/gpfs1/logic/na\\_invariants.v](https://gitlab.mpi-sws.org/iris/gpfs1/-/blob/master/gpfs1/logic/na_invariants.v)

<sup>14</sup>Iris Team (The), *The Iris 3.6 Technical Appendix* [[Iri22](#)], §10.3.

$$\begin{array}{c}
\text{objective}(\text{Nalnv}^{p.\mathcal{N}}(I)) \\
\text{persistent}(\text{Nalnv}^{p.\mathcal{N}}(I))
\end{array}
\quad
\text{timeless}([\text{Na} : p.\mathcal{E}])
\quad
\begin{array}{c}
\text{NAINV-NEW-POOL} \\
\vdash \dot{\models} \exists p. [\text{Na} : p.\top]
\end{array}$$

$$\begin{array}{c}
\text{NAINV-TOK-SPLIT} \\
[\text{Na} : p.\mathcal{E}_1 \uplus \mathcal{E}_2] \dashv\vdash [\text{Na} : p.\mathcal{E}_1] * [\text{Na} : p.\mathcal{E}_2]
\end{array}
\quad
\begin{array}{c}
\text{NAINV-ALLOC} \\
\triangleright I \vdash \models_{\mathcal{E}} \text{Nalnv}^{p.\mathcal{N}}(I)
\end{array}$$

$$\begin{array}{c}
\text{NAINV-ACC} \\
\hline
\text{Nalnv}^{p.\mathcal{N}}(I) * [\text{Na} : p.\mathcal{E}'] \vdash \models_{\mathcal{E}} \triangleright I * [\text{Na} : p.\mathcal{E}' \setminus \mathcal{N}] * (\triangleright I * [\text{Na} : p.\mathcal{E}' \setminus \mathcal{N}] \Rightarrow_{\mathcal{E}}^* [\text{Na} : p.\mathcal{E}'])
\end{array}$$

FIGURE 11.5: The interface of non-atomic invariants

threads, and non-atomic invariants as *thread-local* invariants, where each thread has its own, local pool of invariants. Every thread also has its own invariant tokens  $[\text{Na} : p.\mathcal{E}]$ , which can be “threaded through” its execution to access its own invariant pools, without having to worry about other threads’ interference. Thus the accesses are really sequential, and can span multiple (non-atomic) instructions.

A fresh invariant pool  $p$  can be allocated with **NAINV-NEW-POOL**, where we obtain the invariant token  $[\text{Na} : p.\top]$  for  $p$  with the full set of invariant names  $(p.\top)$ . The token can be split and joined using **NAINV-TOK-SPLIT**, which supports accessing disjoint sets of invariants. With some invariant content  $I$ , we can allocate a non-atomic invariant in some namespace  $\mathcal{N}$  of the pool  $p$  using **NAINV-ALLOC**.

Finally, the most important rule is the access rule **NAINV-ACC**: with a token  $[\text{Na} : p.\mathcal{E}']$  for some mask  $p.\mathcal{E}'$  that includes  $p.\mathcal{N}$ , we can open the invariant  $\text{Nalnv}^{p.\mathcal{N}}(I)$  to access the content  $\triangleright I$ . During the access, we lose the ownership of the invariant names in  $p.\mathcal{N}$ , so we only have the remaining token  $[\text{Na} : p.\mathcal{E}' \setminus \mathcal{N}]$  that allows us to open more invariants *except* those that have already been opened. Once we return the invariant content  $I$ , we can regain the original token  $[\text{Na} : p.\mathcal{E}']$ . We note that the access is non-atomic because we are not forced to have *mask-changing* fancy updates/viewshifts: through out the access the usual *atomic* invariants in  $\mathcal{E}$  still hold. Recall that the masks in fancy updates ( $\models_{\mathcal{E}}$ ) are used to maintain non-reentrancy for *atomic* invariants, while the masks in non-atomic invariant tokens ( $[\text{Na} : p.\mathcal{E}']$ ) are used to maintain non-reentrancy for *non-atomic* invariants.

**CHAPTER SUMMARY.** In this chapter we introduce 3 forms of invariants: (1) objective invariants that are useful to share general, multi-location resources for concurrent accesses; and (2) cancelable invariants that support reclaiming concurrently shared resources; and (3) non-atomic thread-local invariants that allow non-atomic accesses to invariant contents. We will see example uses of (1) and (2) in **Chapter 12**, and more of them in the rest of this dissertation. We will see the application of (3) in **Part III**, where it is used for the lifetime logic’s non-atomic persistent borrows (§16.2.3).

# 12

## Example Verifications with iRC11

---

In this chapter we demonstrate various features of iRC11 that have been presented so far, using several simple example verifications concerning the message-passing idiom. In §12.1 we sketch some verifications of the message-passing examples that we have seen in Chapter 2 (Figure 2.1) and demonstrate the uses of non-atomic (Chapter 9) and atomic points-to (Chapter 10), objective invariants (§11.1), view-explicit modalities (§8.5) and fence modalities (§8.3). In §12.2 we verify the message-passing but with resource reclamation (deallocation), demonstrating cancelable invariants (§11.2) and the switching from atomic back to non-atomic points-to (§10.5).<sup>1</sup> In §12.3, we verify a slightly more complex *spawn-and-join* library, which allows spawning a computation as a child thread and then waiting for its completion to receive the computation result. The transfer of the result is implemented using message-passing.<sup>2</sup> Finally, in §12.4, we verify a release-acquire implementation of the linked-list based Treiber stack<sup>3</sup> against a simple “bag” specification, which demonstrates the atomic points-to CAS rule with pointer comparison.<sup>4</sup> We will revisit the Treiber stack with a stronger specification in Compass (Part IV).

<sup>1</sup>Coq proofs of these MP examples are in [https://gitlab.mpi-sws.org/iris/gpfs1/-/blob/master/gpfs1-examples/mp/proof\\_gen\\_inv.v](https://gitlab.mpi-sws.org/iris/gpfs1/-/blob/master/gpfs1-examples/mp/proof_gen_inv.v)

<sup>2</sup>Coq proof in [https://gitlab.mpi-sws.org/iris/lambda-rust/-/blob/masters/weak\\_mem/theories/lang/spawn.v](https://gitlab.mpi-sws.org/iris/lambda-rust/-/blob/masters/weak_mem/theories/lang/spawn.v)

<sup>3</sup>Treiber, *Systems Programming: Coping with Parallelism* [Tre86].

<sup>4</sup>Coq proof in [https://gitlab.mpi-sws.org/iris/gpfs1/-/blob/master/gpfs1-examples/stack/proof\\_treiber\\_at.v](https://gitlab.mpi-sws.org/iris/gpfs1/-/blob/master/gpfs1-examples/stack/proof_treiber_at.v)

### 12.1 Release-Acquire Message-Passing

In Figure 12.1(b), we provide two  $\lambda_{\text{Rust}} + \text{ORC11}$  implementations of the message passing example, together with the desired specification. Figure 12.1(a) presents the implementation using release-acquire accesses, and corresponds to Example 2.1(c). The `mp` program runs on the main thread  $\pi$ . It allocates a block of size 2 with the base location  $\ell$ , and non-atomically initializes both locations to 0. It then forks a child thread  $\rho$  which will *non-atomically* write the message 42 to  $\ell + 1$ , and signal the message by *atomically* writing 1 to  $\ell$ . The main thread  $\pi$  waits for the signal by a loop of acquire reads of  $\ell$ . `repeat(e)` is implemented as a recursive function, which keeps executing  $e$  until it returns `true`. Once the loop ends,  $\pi$  should be able to get the message 42 from  $\rho$ , safely using a non-atomic read on  $\ell + 1$ . The program `mp_acq_fence` (Figure 12.1(b)) optimizes `mp` by using only relaxed reads in the loop, and then an acquire fence after the loop finishes.

Both programs should satisfy the simple specification in **MP-SPEC**: the returned value is the message 42. We note that both programs should satisfy the stronger specification **MP-SPEC-STRONG**, where the ownership

|                                                                                                          |                                                                                                          |
|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>mp</b> ::=                                                                                            | <b>mp_acq_fence</b> ::=                                                                                  |
| $\pi 1$ : <b>let</b> $\ell := \text{alloc}(2)$ <b>in</b>                                                 | $\pi 1$ : <b>let</b> $\ell := \text{alloc}(2)$ <b>in</b>                                                 |
| $\pi 2$ : $\ell :=_{\text{na}} 0; (\ell + 1) :=_{\text{na}} 0;$                                          | $\pi 2$ : $\ell :=_{\text{na}} 0; (\ell + 1) :=_{\text{na}} 0;$                                          |
| $\pi 3$ : <b>fork</b> { $\rho 1$ : $(\ell + 1) :=_{\text{na}} 42; \rho 2$ : $\ell :=_{\text{rel}} 1; \}$ | $\pi 3$ : <b>fork</b> { $\rho 1$ : $(\ell + 1) :=_{\text{na}} 42; \rho 2$ : $\ell :=_{\text{rel}} 1; \}$ |
| $\pi 4$ : <b>repeat</b> ( $*_{\text{acq}} \ell \neq 0$ );                                                | $\pi 4$ : <b>repeat</b> ( $*_{\text{rlx}} \ell \neq 0$ );                                                |
| $\pi 5$ : $*_{\text{na}}(\ell + 1) // 42$                                                                | $\pi 5$ : <b>fence</b> <sub>acq</sub> ;                                                                  |
|                                                                                                          | $\pi 6$ : $*_{\text{na}}(\ell + 1) // 42$                                                                |
| ((a)) MP with release-acquire accesses.                                                                  | ((b)) MP with an acquire fence.                                                                          |

**repeat** ( $e$ ) ::= (**rec**  $f(\square) := \text{let } v := e \text{ in}$   
**if**  $v == \text{false}$  **then**  $f(\square)$  **else**  $v$ )( $\square$ )

MP-SPEC

 $\{\text{True}\} \text{mp in } \pi \{v.v = 42\}_{\top}$ 

MP-SPEC-STRONG

 $\{\text{True}\} \text{mp in } \pi \{v.v = 42 * \exists \ell. \ell \mapsto [1, 42] * \dagger^2 \ell\}_{\top}$ 

FIGURE 12.1: Message-Passing with Loops

of the allocated block is returned, to prevent memory leaks. Note that the notation  $\ell \mapsto [1, 42]$  stands for  $\ell \mapsto 1 * \ell + 1 \mapsto 42$ . We will look at a similar proof to that of **MP-SPEC-STRONG** in §12.2.

*A high-level proof sketch of mp.* We start in line  $\pi 1$  using **NA-ALLOC** (§9.1), from which we get the block ownership  $\dagger^2 \ell$  and two non-atomic points-to assertions for  $\ell$  and  $\ell + 1$ . The two points-to are sufficient for initialization in line  $\pi 2$ , using **NA-WRITE**. We then use **NA-AT-SW** (§10.1) to turn the non-atomic points-to  $\ell \mapsto 0$  of  $\ell$  to an atomic one with a single-writer permission ( $\ell \mapsto_{\text{sw}}^{\gamma_{\ell}} \_ * \ell \sqsubseteq_{\text{sw}}^{\gamma_{\ell}} \_$ ). We then use **OINV-ALLOC-OBJ** (§11.1) to allocate an objective invariant that contains that atomic points-to  $\ell \mapsto_{\text{sw}}^{\gamma_{\ell}} \_$ . In line  $\pi 3$  when forking  $\rho$ , we give the non-atomic points-to  $\ell + 1 \mapsto 0$  of  $\ell + 1$  and the single-writer ownership  $\ell \sqsubseteq_{\text{sw}}^{\gamma_{\ell}} \_$  of  $\ell$  to  $\rho$ , as well as the fact that the invariant has been established.

In thread  $\rho$ : in line  $\rho 1$ , with the non-atomic points-to of  $\ell + 1$ , we can write the message 42 using **NA-WRITE**, and then get  $\ell + 1 \mapsto 42$ . In line  $\rho 2$ , we can open the invariant with **OINV-ACC** to access the atomic points-to  $\ell \mapsto_{\text{sw}}^{\gamma_{\ell}} \_$  of  $\ell$  so that we can perform the release write of 1 to it, using **AT-WRITE-SW-REL** (§10.1.2) and the single-writer ownership  $\ell \sqsubseteq_{\text{sw}}^{\gamma_{\ell}} \_$  of  $\ell$ . When closing the invariant we return to the invariant not only  $\ell$ 's atomic points-to but also  $\ell + 1 \mapsto 42$  at the view of the release write, so that  $\pi$  can regain it.

Back to thread  $\pi$ : in line  $\pi 4$ , in the **repeat** loop we can open the invariant and access  $\ell$ 's atomic points-to and keep reading  $\ell$ , using **AT-READ-SN-ACQ** (§10.1.1). One we read that  $\ell$  is non-zero and the loop ends, we know that  $\ell + 1 \mapsto 42$  must be inside the invariant, and thread  $\pi$  has observed the view of  $\ell + 1 \mapsto 42$ . We need a unique token  $\diamond$  to say that  $\pi$  is the only one who can acquire  $\ell + 1 \mapsto 42$ . This must be prepared in the beginning before allocating the invariant, and the token  $\diamond$  is given to

thread  $\pi$ . At this point, we trade  $\diamond$  for  $\ell + 1 \mapsto 42$  in the invariant, and use  $\pi$ 's view observation of the release write to acquire  $\ell + 1 \mapsto 42$  locally. With that, in line  $\pi 5$ , we use **NA-READ** to read and return 42.  $\square$

To write out the proof more formally, we need to define the exclusive ghost token  $\diamond$  and the objective invariant for **mp**.

**Definition 12.1** (Exclusive Tokens). We use the exclusive RA of unit  $\text{Ex}(1)$  to define exclusive tokens.

$$\diamond^\gamma ::= [\text{ex}() : \text{Ex}(1)]^\gamma$$

Exclusive tokens satisfy the following rules.

|                                |                                          |                                                         |
|--------------------------------|------------------------------------------|---------------------------------------------------------|
| timeless( $\diamond^\gamma$ )  | EXCL-TOK-ALLOC                           | EXCL-TOK-EXCL                                           |
| objective( $\diamond^\gamma$ ) | $\vdash \exists \gamma. \diamond^\gamma$ | $\diamond^\gamma * \diamond^\gamma \vdash \text{False}$ |

**Definition 12.2** (Invariant for **mp**). The invariant of **mp** needs to be objective, and contains the atomic points-to ownership of  $\ell$  in single-writer mode, since only the thread  $\rho$  is writing.

$$\begin{aligned} \text{mpl}(\ell, \gamma_\ell, \gamma) : \text{vProp} ::= & \\ & \exists h, b, t_0, V_0, V_\ell. @_{V_\ell}(\ell \mapsto_{\text{sw}}^{\gamma_\ell} h) * \text{let } h_0 := [t_0 \leftarrow (0, V_0)] \text{ in} \\ & \text{if } b = \text{false} \text{ then } h = h_0 \text{ else} \\ & \exists t_1 > t_0, V_1. h = [t_0 \leftarrow (0, V_0)][t_1 \leftarrow (1, V_1)] * (\diamond^\gamma \vee @_{V_1}(\ell + 1 \mapsto 42)) \end{aligned}$$

More concretely, the invariant **mpl** owns  $@_{V_\ell}(\ell \mapsto_{\text{sw}}^{\gamma_\ell} h)$  for some atomic period identifier  $\gamma_\ell$  and some history  $h$ , at some view  $V_\ell$ . The history  $h$  of  $\ell$  can be in two states, dictated by the existential quantified boolean  $b$ . If  $b$  is **false**, then  $\ell$  is still in its initialized state, i.e., it has a singleton history  $h_0$  with the write message  $(t_0, 0, V_0)$ . Once  $b$  is **true**,  $\ell$  is in its “signaled” state, where its history  $h$  has one extra write message  $(t_1, 1, V_1)$ . When  $\ell$  is in the signaled state, **mpl** either owns  $\diamond^\gamma$ , or owns  $\ell + 1 \mapsto 42$  at the view  $V_1$  (of the signaling write). If **mpl** owns  $\ell + 1 \mapsto 42$ , it means that  $\rho$  has released the non-atomic of  $\ell + 1$  but  $\pi$  has not acquired it yet. Once  $\pi$  has acquired  $\ell + 1$ 's non-atomic points-to, **mpl** will own  $\diamond^\gamma$ .

**mpl** is clearly objective, because it only contains pure facts, ghost state, and points-to assertions that are under the view-at modality.

*Proof sketch of **mp**.* We present the detailed proof sketch of **mp** using *Hoare proof outlines*, in **Figure 12.2**. Note that the post-condition of the proof is almost satisfying the stronger specification **MP-SPEC-STRONG**: it is only missing the points-to  $\ell \mapsto 1$ .  $\square$

*Proof sketch of **mp\_acq\_fence**.* The proof of **mp** uses the same invariant **mpl**, and follows the proof of **mp** closely. The main difference is that in thread  $\rho$ 's read of  $\ell$ , which is a relaxed read instead of an acquire read, we use the rule **AT-READ-SN**. Consequently, in the case where  $\rho$  reads 1 from  $\ell$ , we will acquire  $\nabla_\pi \sqsupseteq V' * @_{V_1} \ell + 1 \mapsto 42$ , where  $V' \sqsupseteq V_1$ . Then, with the acquire fence, we apply **HOARE-ACQ-FENCE** (§8.3) to get  $\sqsupseteq V'$ .

$\{\text{True}\}$   
 $\pi 1: \text{let } \ell := \text{alloc}(2) \text{ in } \{ \ell \mapsto \text{?} * \ell + 1 \mapsto \text{?} * \uparrow^2 \ell \} \quad // \text{NA-ALLOC}$   
 $\pi 2: \ell :=_{\text{na}} 0; (\ell + 1) :=_{\text{na}} 0; \{ \ell \mapsto 0 * \ell + 1 \mapsto 0 * \uparrow^2 \ell \} \quad // \text{NA-WRITE}$   
 $\{ \ell + 1 \mapsto 0 * \uparrow^2 \ell * \diamond \gamma * \exists \gamma, \gamma_\ell, t_0, V_0. \diamond \gamma * \sqsubseteq V_0 * \ell \mapsto_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] \}$   
 Context:  $\sqsubseteq V_0 \quad // \text{EXCL-TOK-ALLOC and NA-AT-SW}$   
 $\{ \ell + 1 \mapsto 0 * \uparrow^2 \ell * \diamond \gamma * \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * \exists V_\ell. \sqsubseteq V_\ell * @_{V_\ell} \ell \mapsto_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] \} \quad // \text{VA-INTRO}$   
 $\{ \ell + 1 \mapsto 0 * \uparrow^2 \ell * \diamond \gamma * \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * \ell \sqsubseteq_{\text{sn}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * @_{V_\ell} \ell \mapsto_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] \}$   
 Context:  $\ell \sqsubseteq_{\text{sn}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] \quad // \text{AT-SW-SY and AT-SY-SN}$   
 $\{ \ell + 1 \mapsto 0 * \uparrow^2 \ell * \diamond \gamma * \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * \text{mpl}(\ell, \gamma_\ell, \gamma) \}$   
 $\{ \ell + 1 \mapsto 0 * \uparrow^2 \ell * \diamond \gamma * \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * \boxed{\text{mpl}(\ell, \gamma_\ell, \gamma)}^{\mathcal{N}} \} \quad // \text{OINV-ALLOC-OBJ}$   
 Context:  $\boxed{\text{mpl}(\ell, \gamma_\ell, \gamma)}^{\mathcal{N}}$   
 $\pi 3: \text{fork } \{ \dots \} \quad // \text{HOARE-FORK}$

Thread  $\rho$

$\{ \ell + 1 \mapsto 0 * \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] \}_\top$   
 $\rho 1: (\ell + 1) :=_{\text{na}} 42; \{ \ell + 1 \mapsto 42 * \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] \}_\top \quad // \text{NA-WRITE}$   
 $\{ \ell + 1 \mapsto 42 * \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * \triangleright \text{mpl}(\ell, \gamma_\ell, \gamma) \}_{\top \setminus \mathcal{N}} \quad // \text{OINV-ACC-OBJ}$   
 $\{ \ell + 1 \mapsto 42 * \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * @_{V'_\ell} (\ell \mapsto_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)]) \}_{\top \setminus \mathcal{N}}$   
 $// \text{By unfolding mpl and using a stronger version of AT-SW-AGREE}$   
 $\rho 2: \ell :=_{\text{rel}} 1;$   
 $\left\{ \begin{array}{l} \exists t_1, V_1 \sqsubseteq V_0. \sqsubseteq V_1 * @_{V_1} (\ell + 1 \mapsto 42) * \\ @_{V_1} \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] [t_1 \leftarrow (1, V_1)] * @_{V'_\ell \sqcup V_1} (\ell \mapsto_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] [t_1 \leftarrow (1, V_1)]) \end{array} \right\}_{\top \setminus \mathcal{N}}$   
 $// \text{By applying AT-WRITE-SW-REL with } \sqsubseteq V_0$   
 $\{ \sqsubseteq V_1 * @_{V_1} \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} - * \text{mpl}(\ell, \gamma_\ell, \gamma) \}_{\top \setminus \mathcal{N}} \quad // \text{By picking the "signaled" state for mpl}$   
 $\{ \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} - \}_\top \quad // \text{VA-ELIM}$   
 $\{\text{True}\}_\top$

$\{ \uparrow^2 \ell * \diamond \gamma \}$   
 $\pi 4: \text{repeat } (*\text{acq } \ell \neq 0);$

Loop body

$\{ \diamond \gamma \}_\top$   
 $\{ \diamond \gamma * \triangleright \text{mpl}(\ell, \gamma_\ell, \gamma) \}_{\top \setminus \mathcal{N}} \quad // \text{OINV-ACC-OBJ}$   
 $\left\{ \diamond \gamma * \sqsubseteq V_0 * \ell \sqsubseteq_{\text{sn}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * @_{V'_\ell} (\ell \mapsto_{\text{sw}}^{\gamma_\ell} h) * \triangleright \text{if } \dots \text{ then } \dots \text{ else } \dots \right\}_{\top \setminus \mathcal{N}}$   
 $*\text{acq } \ell$   
 $\left\{ \begin{array}{l} v. \exists h' \subseteq h, t, V, V' \sqsubseteq V_0 \sqcup V. h'(t) = (v, V) * \sqsubseteq V' * @_{V'_\ell \sqcup V'} (\ell \mapsto_{\text{sw}}^{\gamma_\ell} h) \\ * \diamond \gamma * \text{if } b = \text{false then } \dots \text{ else } (\diamond \gamma \vee @_{V_1} (\ell + 1 \mapsto 42)) \end{array} \right\}_{\top \setminus \mathcal{N}} \quad // \text{AT-READ-SN-ACQ}$   

We have  $v \neq 0 \Rightarrow b = \text{true} \wedge V_1 = V$ . If  $v = 0$ , we return the invariant content unchanged.  
 If  $v \neq 0 \wedge b = \text{true}$ , by **EXCL-TOK-EXCL** we have  $@_{V_1} (\ell + 1 \mapsto 42)$ . With  $V' \sqsubseteq V_0 \sqcup V$  and  $\sqsubseteq V'$ , by **VS-MONO** we have  $\sqsubseteq V_1$ . With **VA-ELIM**, we get  $\ell + 1 \mapsto 42$ . We put back  $\diamond \gamma$  to re-establish **mpl**.  
 $\{ ((v = 0 * \diamond \gamma) \vee (v \neq 0 * \ell + 1 \mapsto 42)) * \text{mpl}(\ell, \gamma_\ell, \gamma) \}_{\top \setminus \mathcal{N}}$

 $\{ (v = 0 * \diamond \gamma) \vee (v \neq 0 * \ell + 1 \mapsto 42) \}_\top \quad // \text{"loop invariant", we use L\"ob induction before the loop}$

$\{ \uparrow^2 \ell * \ell + 1 \mapsto 42 \} \quad // \uparrow^2 \ell \text{ was framed}$   
 $\pi 5: *\text{na}(\ell + 1) \{ v. v = 42 * \uparrow^2 \ell * \ell + 1 \mapsto 42 \} \quad // \text{NA-READ}$   
 $\{ v. v = 42 \}$

FIGURE 12.2: Hoare proof outlines for **mp**

Then we use **VS-MONO** to get  $\sqsupseteq_{V_1}$ , which allows us to use **VA-ELIM** and get  $\ell + 1 \mapsto 42$ .

We give the Hoare proof outlines for the part that changes below.

```

 $\{\dagger^2 \ell * \diamond^\gamma\}$ 
 $\pi 4$ : repeat ( $*_{\text{acq}} \ell \neq 0$ );
  Loop body  $\left| \begin{array}{l} \{\diamond^\gamma\}_\top \\ *_{\text{rlx}} \ell \text{ // OINV-ACC-OBJ and AT-READ-SN} \\ \{(v = 0 * \diamond^\gamma) \vee (v \neq 0 * \exists V' \sqsupseteq V_1. \nabla_\pi \sqsupseteq V' * @_{V_1}(\ell + 1 \mapsto 42))\}_\top \end{array} \right.$ 
 $\{\dagger^2 \ell * \nabla_\pi \sqsupseteq V' * @_{V_1}(\ell + 1 \mapsto 42)\}$ 
 $\pi 5$ : fenceacq;  $\{\dagger^2 \ell * \sqsupseteq V' * @_{V_1}(\ell + 1 \mapsto 42)\}$  // HOARE-ACQ-FENCE
 $\{\dagger^2 \ell * \ell + 1 \mapsto 42\}$  // VS-MONO then VA-ELIM
 $\pi 6$ :  $*_{\text{na}}(\ell + 1) \{v.v = 42 * \dagger^2 \ell * \ell + 1 \mapsto 42\}$  // NA-READ
 $\{v.v = 42\}$ 

```

□

COMPARISON WITH IGPS PROOFS. The iGPS paper<sup>5</sup> also presents similar proofs of the Message-Passing example, once in its base logic and once in its surface logic. The proof presented in Figure 12.2 is slightly less complicated than the iGPS base-level proof where views are fully explicit, but is still more complicated than the iGPS surface-level proof, where views are fully hidden. The proof in Figure 12.2 only hides views around sequential (non-atomic) steps, but works with explicit views around atomic access steps, because the proof employs objective invariants and atomic points-to assertions. The iGPS surface-level proof for **mp** employs GPS single-location protocols in place of objective invariants and atomic points-to, and so views are hidden completely around atomic accesses. That proof is thus simpler than our proof here, because the invariant **mpl** is indeed a single-location invariant that governs the atomic accesses of  $\ell$ .

<sup>5</sup>Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17], §3.2.2, §4.1.

This demonstrates that working with explicit views using atomic points-to in situations where we only have a single atomic location (or multiple atomic locations that are unrelated) are counterproductive, compared to single-location protocols. In §17 (Part III), we will show how to derive the higher-level abstraction of GPS single-location protocols from our atomic points-to and general invariants. Atomic points-to and general invariants, however, significantly simplify the process of stating a relation that spans multiple atomic locations, as we will see in Compass (Part IV). Furthermore, as we will see, working with explicit views is not just a trade-off (for multi-location invariants), but becomes a necessity to achieve the stronger specifications of Compass.

## 12.2 Release-Acquire Message-Passing with Reclamation

We now look at the verification of the **mp\_reclaim**, whose implementation and specification are given in Figure 12.3. The **mp\_reclaim** program extends **mp** simply by cleaning up the memory needed to perform the message passing, in line  $\pi 6$  after reading the message and before returning it. The main difference of the verification is now to show that the



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <pre> <b>mp_reclaim</b> ::=   <math>\pi 1</math>: <b>let</b> <math>\ell := \text{alloc}(2)</math> <b>in</b>   <math>\pi 2</math>: <math>\ell :=_{\text{na}} 0; (\ell + 1) :=_{\text{na}} 0;</math>   <math>\pi 3</math>: <b>fork</b> { <math>\rho 1</math>: <math>(\ell + 1) :=_{\text{na}} 42;</math> <math>\rho 2</math>: <math>\ell :=_{\text{rel}} 1;</math> }   <math>\pi 4</math>: <b>repeat</b> (<math>*_{\text{acq}} \ell \neq 0</math>);   <math>\pi 5</math>: <b>let</b> <math>v := *_{\text{na}} (\ell + 1)</math> <b>in</b>   <math>\pi 6</math>: <b>free</b>(<math>\ell, 2</math>);   <math>\pi 7</math>: <math>v // 42</math> </pre> | <pre> MP-RECLAIM-SPEC {True}  <b>mp_reclaim</b> <b>in</b> <math>\pi</math> {<math>v.v = 42</math>}_<math>\top</math> </pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|

FIGURE 12.3: Message-Passing with Reclamation

call of **free** in line  $\pi 6$  is safe. We note that verifying **mp\_reclaim** against **MP-RECLAIM-SPEC** is the same as verifying **mp** against **MP-SPEC-STRONG**.

Recall that by the end the proof for **mp** (Figure 12.2), we are missing only the non-atomic points-to  $\ell \mapsto 1$  of the location  $\ell$ , which has been put in the invariant for shared atomic accesses, in the form of an atomic points-to. To reclaim the atomic points-to of  $\ell$  from the invariant, we need to be able to cancel the invariant once thread  $\pi$  receives the message after line  $\pi 4$ . Interestingly, at that point the invariant does not hold anymore, so we do not need the exclusive token  $\diamond^\gamma$  and the disjunction as in **mpl** (see Definition 12.2). On the other hand, we now need to deal with the invariant token  $\heartsuit_q^\gamma$  of cancelable invariants (§11.2).

**Definition 12.3** (Invariant for **mp\_reclaim**). The invariant of **mp\_reclaim** does not need to be objective, so it contains the atomic points-to ownership of  $\ell$  in single-writer mode, without being under a view-at modality.

$$\begin{aligned}
 \text{mpcl}(\ell, \gamma_i, \gamma) : \text{vProp} ::= & \\
 & \exists h, b, t_0, V_0. \ell \mapsto_{\text{sw}}^{\gamma_\ell} h * \text{let } h_0 := [t_0 \leftarrow (0, V_0)] \text{ in} \\
 & \text{if } b = \text{false then } h = h_0 \text{ else} \\
 & \exists t_1 > t_0, V_1. h = [t_0 \leftarrow (0, V_0)][t_1 \leftarrow (1, V_1)] * @_{V_1}(\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42)
 \end{aligned}$$

The invariant also has two states (“initialized” and “signaled”) like the invariant of **mp**. However, in the signaled state, instead of having a disjunction between the exclusive token  $\diamond^\gamma$  and the non-atomic points-to of  $\ell + 1$ , the invariant just holds the points-to together with half of the cancelable invariant token  $\heartsuit_{1/2}^\gamma$ , at the message view  $V_1$ .

Intuitively, at the beginning the invariant token  $\heartsuit_{1/2}^\gamma$  is split into two halves, and each participant (threads  $\pi$  and  $\rho$ ) will keep one half to access the invariant. Once thread  $\rho$  is done, it sends back its half  $\heartsuit_{1/2}^\gamma$  together with  $\ell + 1 \mapsto 42$  (by putting them in **mpcl** so that thread  $\pi$  can acquire both, reconstruct the full token  $\heartsuit_1^\gamma$ , and cancel the invariant to get back  $\ell \mapsto_{\text{sw}}^{\gamma_\ell} h$ ).

*Proof sketch for mp\_reclaim.* The Hoare proof outlines for **mp\_reclaim** are given in Figure 12.4. We note several important points.

- Between line  $\pi 2$  and  $\pi 3$ , we allocate a cancelable invariant for **mpcl**. But instead of using **CINV-ALLOC**, we use the stronger rule



$\{\text{True}\}$   
 $\pi 1$ : **let**  $\ell := \text{alloc}(2)$  **in**  
 $\pi 2$ :  $\ell :=_{\text{na}} 0$ ;  $(\ell + 1) :=_{\text{na}} 0$ ;  $\{\ell \mapsto 0 * \ell + 1 \mapsto 0 * \dagger^2 \ell\}$  // NA-ALLOC and NA-WRITE  
 $\{\ell + 1 \mapsto 0 * \dagger^2 \ell * \exists \gamma_\ell, t_0, V_0. \sqsupseteq V_0 * \ell \mapsto_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)] * \ell \sqsupseteq_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)]\}$  // NA-AT-SW  
 Context:  $\sqsupseteq V_0 * \ell \sqsupseteq_{\text{sn}}^\gamma [t_0 \leftarrow (0, V_0)]$  // AT-SW-SY and AT-SY-SN  
 $\{\ell + 1 \mapsto 0 * \dagger^2 \ell * \ell \sqsupseteq_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)] * \ell \mapsto_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)] * \exists \gamma. \forall I. \top \Vdash^{\top \setminus \mathcal{N}} \dots\}$  // CINV-ALLOC-OPEN  
 $\{\ell + 1 \mapsto 0 * \dagger^2 \ell * \ell \sqsupseteq_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)] * \heartsuit_1^\gamma * \boxed{\text{mpcl}(\ell, \gamma_\ell, \gamma)}^{\gamma, \mathcal{N}}\}$   
 Context:  $\boxed{\text{mpcl}(\ell, \gamma_\ell, \gamma)}^{\gamma, \mathcal{N}}$   
 $\pi 3$ : **fork**  $\{\dots\}$  // HOARE-FORK  

Thread  $\rho$

Accessing mpcl

$\{\ell + 1 \mapsto 0 * \ell \sqsupseteq_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)] * \heartsuit_{1/2}^\gamma\}_\top$   
 $\rho 1$ :  $(\ell + 1) :=_{\text{na}} 42$ ;  $\{\ell + 1 \mapsto 42 * \ell \sqsupseteq_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)] * \heartsuit_{1/2}^\gamma\}_\top$  // NA-WRITE  

$\{\ell + 1 \mapsto 42 * \ell \sqsupseteq_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)] * \heartsuit_{1/2}^\gamma * \exists V_i. \sqcup_{V_i} (\triangleright \text{mpcl}(\ell, \gamma_\ell, \gamma))\}$   
 $\{\dots (* \text{closing viewshifts} *)\}$   
 $\{\ell + 1 \mapsto 42 * \ell \sqsupseteq_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)] * \heartsuit_{1/2}^\gamma * \sqcup_{V_i} (\ell \mapsto_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)]) * \dots\}_{\top \setminus \mathcal{N}}$

// CINV-ACC-GEN

 $\rho 2$ :  $\ell :=_{\text{rel}} 1$ ;  
 $\{\exists t_1, V_1 \sqsupseteq V_0. \sqsupseteq V_1 * @_{V_1} (\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42)$   
 $\{\dots (* @_{V_1} \ell \sqsupseteq_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)] [t_1 \leftarrow (1, V_1)] * \sqcup_{V_i} (\ell \mapsto_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_0)] [t_1 \leftarrow (1, V_1)]) * \dots\}_{\top \setminus \mathcal{N}}$   
// By applying AT-WRITE-SW-REL-VJ with  $\sqsupseteq V_0$   
We pick the “signaled” state for mpcl. We use the second closing viewshift option to close and release  $@_{V_1} \heartsuit_{1/2}^\gamma$  to the invariant at the same time, in addition to  $@_{V_1} (\ell + 1 \mapsto 42)$ .  
 $\{\ell \sqsupseteq_{\text{sw}}^\gamma \_ \}_\top$  // VA-ELIM  
 $\{\text{True}\}_\top$

Loop body

Accessing mpcl

$\{\dagger^2 \ell * \heartsuit_{1/2}^\gamma\}$   
 $\pi 4$ : **repeat**  $(* \text{acq} \ell \neq 0)$ ;  

$\{\heartsuit_{1/2}^\gamma\}_\top$   
 $\{\heartsuit_{1/2}^\gamma * \exists V_i. \sqcup_{V_i} \triangleright \text{mpcl}(\ell, \gamma_\ell, \gamma) * \dots (* \text{closing viewshifts} *)\}_{\top \setminus \mathcal{N}}$  // CINV-ACC-GEN  
 $\{\heartsuit_{1/2}^\gamma * \sqsupseteq V_0 * \ell \sqsupseteq_{\text{sn}}^\gamma [t_0 \leftarrow (0, V_0)] * \sqcup_{V_i} (\ell \mapsto_{\text{sw}}^\gamma h) * \triangleright (\text{if } \dots \text{ then } \dots \text{ else } \dots) * \dots\}_{\top \setminus \mathcal{N}}$   
 $* \text{acq} \ell \left\{ \begin{array}{l} v. \exists h' \sqsubseteq h, t, V, V' \sqsupseteq V_0 \sqcup V. h'(t) = (v, V) * \sqsupseteq V' * \sqcup_{V_i} (\ell \mapsto_{\text{sw}}^\gamma h) \\ * \heartsuit_{1/2}^\gamma * (\text{if } b = \text{false then } \dots \text{ else } @_{V_1} (\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42)) * \dots \end{array} \right\}_{\top \setminus \mathcal{N}}$   
// AT-READ-SN-ACQ-VJ  
We have  $(v \neq 0 \Rightarrow b = \text{true} \wedge V_1 = V)$ . If  $v = 0$ , we return the invariant content unchanged.  
If  $b = \text{true}$ , we have  $@_{V_1} (\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42)$ . With VS-MONO and VA-ELIM, we get  $\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42$ . Combining with  $\pi$ ’s own  $\heartsuit_{1/2}^\gamma$  we get  $\heartsuit_1^\gamma$ .  
 $\{(v = 0 * \heartsuit_{1/2}^\gamma * \sqcup_{V_i} \text{mpcl}(\ell, \gamma_\ell, \gamma)) \vee (v \neq 0 * \heartsuit_1^\gamma * \ell + 1 \mapsto 42 * \sqcup_{V_i} (\ell \mapsto_{\text{sw}}^\gamma h))\}$   
 $\{\dots (* \text{closing viewshift} *)\}_{\top \setminus \mathcal{N}}$   
If  $v = 0$ , we use the first closing viewshift option to close the invariant. If  $v \neq 0$ , we use the *third* closing viewshift option to cancel the invariant. By the cancelation (CINV-ACC-GEN), we get  $\sqsupseteq V_i$ , which can be used with VJ-ELIM to get  $\ell \mapsto_{\text{sw}}^\gamma h$ . We then use AT-NA to get  $\ell \mapsto 1$ .  
 $\{(v = 0 * \top \setminus \mathcal{N} \Vdash^\top \heartsuit_{1/2}^\gamma) \vee (v \neq 0 * \ell \mapsto [1, 42] * \top \setminus \mathcal{N} \Vdash^\top \text{True})\}_{\top \setminus \mathcal{N}}$   
 $\{(v = 0 * \heartsuit_{1/2}^\gamma) \vee (v \neq 0 * \ell \mapsto [1, 42])\}_\top$  // “loop invariant”, we use LÖB induction before the loop  
 $\{\dagger^2 \ell * \ell \mapsto [1, 42]\}$  //  $\dagger^2 \ell$  was framed  
 $\pi 5$ : **let**  $v :=_{\text{na}} (\ell + 1)$  **in**  $\{v = 42 * \dagger^2 \ell * \ell \mapsto [1, 42]\}$  // NA-READ  
 $\pi 6$ : **free**  $(\ell, 2)$ ;  $\{v = 42\}$  // NA-UNSYNC and NA-DEALLOC  
 $\pi 7$ :  $v \{v.v = 42\}$

FIGURE 12.4: Hoare proof outlines for mp\_reclaim

**CINV-ALLOC-OPEN** (Figure 11.3), because our invariant  $\text{mpcl}$  depends on the invariant identifier  $\gamma$  itself. **CINV-ALLOC-OPEN** allows us to get the identifier  $\gamma$  before picking the invariant content which can depend on  $\gamma$ . In our case, that is  $\text{mpcl}(\ell, \gamma_\ell, \gamma)$ .

- After the allocation of  $\boxed{\text{mpcl}(\ell, \gamma_\ell, \gamma)}^{\gamma, \mathcal{N}}$ , we also get the full invariant token  $\heartsuit_1^\gamma$ , which we split into two halves of  $\heartsuit_{1/2}^\gamma$ , using **CINV-TOK-FRAC**. We then give one half to the thread  $\rho$ , using **HOARE-FORK**. The thread  $\pi$  retains the other half.
- In the proof of thread  $\rho$ , around the atomic access in line  $\rho 2$ , we also do not use the simple rule **CINV-ACC**, but instead use **CINV-ACC-GEN** to access the invariant. The latter gives us several options when closing the invariant, of which we will use the second option (see Figure 11.3). The second option allows us to use the half token  $\heartsuit_{1/2}^\gamma$  (which we have used to open the invariant) to close the invariant and simultaneously release it into the invariant content  $\text{mpcl}$ . **CINV-ACC-GEN** indeed allows us to keep  $\heartsuit_{1/2}^\gamma$  around after opening (and does not consume  $\heartsuit_{1/2}^\gamma$  like **CINV-ACC**), so that when performing the release write of 1 to  $\ell$ , we also release  $\heartsuit_{1/2}^\gamma$  together with  $\ell + 1 \mapsto 42$ . That is, after the write, we have  $@_{V_1}(\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42)$  where  $V_1$  is the message view of the write.

The second closing option looks as follows.

$$\forall V', P. \left( @_{V'} \heartsuit_{1/2}^\gamma * \left( @_{V'} \heartsuit_{1/2}^\gamma \Rightarrow_{\mathcal{E} \setminus \mathcal{N}}^* (\sqcup_{V_i} \triangleright \text{mpcl}(\ell, \gamma_\ell, \gamma)) * P \right) \right) \\ \mathcal{E} \setminus \mathcal{N} \Rightarrow_{\mathcal{E}}^* P$$

With  $@_{V_1}(\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42)$  and the atomic points-to

$$\sqcup_{V_i}(\ell \mapsto_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)][t_1 \leftarrow (1, V_1)])$$

after the write, we instantiate the second closing option with  $V' := V_1$  and  $P := \text{True}$ . We give up  $@_{V_1} \heartsuit_{1/2}^\gamma$  for the left-hand side of the separating conjunction before the wand viewshift ( $\mathcal{E} \setminus \mathcal{N} \Rightarrow_{\mathcal{E}}^*$ ). For the right-hand side, it is easy to show that

$$@_{V_1}(\ell + 1 \mapsto 42) * \sqcup_{V_i}(\ell \mapsto_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)][t_1 \leftarrow (1, V_1)]) \\ \vdash @_{V_1} \heartsuit_{1/2}^\gamma \Rightarrow_{\mathcal{E} \setminus \mathcal{N}}^* \sqcup_{V_i} \triangleright \text{mpcl}(\ell, \gamma_\ell, \gamma)$$

by picking the signaled state ( $b = \text{true}$ ) for  $\text{mpcl}(\ell, \gamma_\ell, \gamma)$ . We note that  $@_{V_1}(\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42) \dashv\vdash \sqcup_{V_i} @_{V_1}(\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42)$ , due to **VJ-VA** (Figure 8.3).

After the instantiation, we get  $\mathcal{E} \setminus \mathcal{N} \models_{\mathcal{E}}^* \text{True}$  that we use to close the invariant and complete the access.

Note that **AT-WRITE-SW-REL** allows us the option to also release the single-writer ownership to the invariant, because we have  $@_{V_1} \ell \sqsupseteq_{\text{sw}}^{\gamma_\ell} \_$ . We do not need this feature here, but it can be useful elsewhere (see §17).

- The same situation applies for thread  $\pi$ 's atomic access in line  $\pi 4$ : we need to use **CINV-ACC-GEN** to open the invariant, and in case the

AT-WRITE-SW-REL-VJ

$$\{\sqsupseteq V_0 * \ell \sqsupseteq_{\text{sw}} h * \sqcup_{V_b}(\ell \xrightarrow{t_x}_{\text{sw}} h) * P\} \ell :=_{\text{rel}} v \text{ in } \pi \left\{ \begin{array}{l} \exists t, V \sqsupseteq V_0. \max(\text{dom}(h)) < t * \sqsupseteq V * @_V P * \\ @_V(\ell \sqsupseteq_{\text{sw}} h[t \leftarrow (v, V)]) * \sqcup_{V_b}(\ell \xrightarrow{t_x}_{\text{sw}} h[t \leftarrow (v, V)]) \end{array} \right\} \varepsilon$$

AT-READ-SN-ACQ-VJ

$$\{\sqsupseteq V_0 * \ell \sqsupseteq_{\text{sn}} h_0 * \sqcup_{V_b}(\ell \xrightarrow{t_x}_{\theta} h)\} *_{\text{acq}} \ell \text{ in } \pi \left\{ \begin{array}{l} v. \exists h', t, V, V' \sqsupseteq V_0 \sqcup V. h_0 \subseteq h' \subseteq h * \\ h'(t) = (v, V) * t \geq \max(\text{dom}(h_0)) * \\ \sqsupseteq V' * @_{V'}(\ell \sqsupseteq_{\text{sn}} h') * \sqcup_{V_b}(\ell \xrightarrow{t_x}_{\theta} h) \end{array} \right\} \varepsilon$$

FIGURE 12.5: Derived iRC11 atomic access rules with the view-join modality

invariant is in the signaled state ( $b = \mathbf{true}$ ), we need to use **VJ-VA** to get  $@_{V_1}(\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42)$ . If the invariant is in the signaled state, we acquire  $@_{V_1}(\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42)$  from the invariant, which we can use **VA-ELIM** to get  $\heartsuit_{1/2}^\gamma * \ell + 1 \mapsto 42$ , because we also have  $\sqsupseteq V_1$  thanks to the acquire read. Note that we also have  $\pi$ 's half token  $\heartsuit_{1/2}^\gamma$  locally, so together we have the full token  $\heartsuit_1^\gamma$  (using **CINV-TOK-FRAC**). We then use the third closing option from **CINV-ACC-GEN** to cancel the invariant, from which we receive  $\sqsupseteq V_i * \top \mathcal{N} \Rightarrow^\top \text{True}$ . We use the fancy update to conclude the access. We combine  $\sqsupseteq V_i$  with  $\sqcup_{V_i}(\ell \mapsto_{\text{sw}}^\gamma h)$  to get  $\ell \mapsto_{\text{sw}}^\gamma h$ , using **VJ-ELIM**. We then use **AT-NA** to turn  $\ell$ 's atomic points-to to the non-atomic points-to  $\ell \mapsto 1$ , knowing that 1 is the latest write in  $h$ .

- Last but not least, we note that the atomic access rules **AT-WRITE-SW-REL** and **AT-READ-SN-ACQ** are not directly applicable to this proof, because the rules require an atomic points-to under a view-at modality, i.e.,  $@_{V_b} \ell \mapsto_\theta h$ , while we get a points-to under a view-join modality from the cancelable invariant access rule, i.e.,  $\sqcup_{V_b} \ell \mapsto_\theta h$ . Fortunately, in general, rules with the view-join modality can be derived from those with the view-at modality. We show the derived versions **AT-WRITE-SW-REL-VJ** and **AT-READ-SN-ACQ-VJ** in Figure 12.5. We demonstrate the derivation of **AT-WRITE-SW-REL-VJ** below.

□

*Proof sketch of AT-WRITE-SW-REL-VJ.* With  $\sqsupseteq V_0$  and  $\sqcup_{V_b}(\ell \xrightarrow{t_x}_{\text{sw}} h)$ , we use **VJ-ELIM-VA** (Figure 8.3) to get

$$\sqsupseteq V' * @_{(V' \sqcup V_b)}(\ell \xrightarrow{t_x}_{\text{sw}} h)$$

for some  $V' \sqsupseteq V_0$ . We apply **AT-WRITE-SW-REL** with  $@_{(V' \sqcup V_b)}(\ell \xrightarrow{t_x}_{\text{sw}} h)$ . In the post-condition we get back the atomic points-to in the form

$$@_{V' \sqcup V_b \sqcup V}(\ell \xrightarrow{t_x}_{\text{sw}} h[t \leftarrow (v, V)])$$

for some  $V \sqsupseteq V_0$  and  $\sqsupseteq V$ . We can rewrite it using **VA-VJ** to the form

$$@_{V' \sqcup V}(\sqcup_{V_b}(\ell \xrightarrow{t_x}_{\text{sw}} h[t \leftarrow (v, V)]))$$

We now use **VA-ELIM** to get back  $\sqcup_{V_b}(\ell \xrightarrow{t_x}_{\text{sw}} h[t \leftarrow (v, V)])$ . Note that we have  $\sqsupseteq(V' \sqcup V)$  from  $\sqsupseteq V'$  and  $\sqsupseteq V$  using **VS-JOIN**. □

|                                                                                                                                                                                                                                                                                                                         |                                                                                                                                              |                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>spawn</b> ::=<br>$\lambda[f].$<br>1: <b>let</b> $\ell := \text{alloc}(2)$ <b>in</b><br>2: $\ell :=_{\text{na}} 0$ ; // init to 0<br>3: <b>fork</b> $\{f([\ell])\}$ ; // spawn $f$<br>4: $\ell$                                                                                                                       | <b>finish</b> ::=<br>$\lambda[\ell, v].$<br>1: $(\ell + 1) :=_{\text{na}} v$ ; // write result<br>2: $\ell :=_{\text{rel}} 1$ // signal done | <b>join</b> ::=<br>$\lambda[\ell].$<br>1: <b>repeat</b> $(^*\text{acq} \ell \neq 0)$ ;<br>2: <b>let</b> $v := ^*\text{na}(\ell + 1)$ <b>in</b><br>3: <b>free</b> $(\ell, 2)$ ; // clean up<br>4: $v$ // return result |
| <b>SPAWN-SPEC</b><br>$\left\{ \left( \forall \ell, \rho. \text{FinishHandle}^{\mathcal{N}}(\ell, \Phi) \multimap \text{wp } f([\ell]) \text{ in } \rho \{ \_ . \text{True} \} [\top] \right) \right\} \text{spawn}([f]) \text{ in } \pi \{ \ell. \exists \gamma. \text{JoinHandle}^{\mathcal{N}}(\ell, \Phi) \}_{\top}$ |                                                                                                                                              |                                                                                                                                                                                                                       |
| <b>FINISH-SPEC</b><br>$\{ \text{FinishHandle}^{\mathcal{N}}(\ell, \Phi) * \Phi(v) \} \text{finish}([\ell, v]) \text{ in } \pi \{ \_ . \text{True} \}_{\top}$                                                                                                                                                            |                                                                                                                                              | <b>JOIN-SPEC</b><br>$\{ \text{JoinHandle}^{\mathcal{N}}(\ell, \Phi) \} \text{join}([\ell]) \text{ in } \pi \{ v. \Phi(v) \}_{\top}$                                                                                   |

FIGURE 12.6: A Spawn-and-Join library

### 12.3 Spawn and Join

We now look at the verification of a spawn-and-join library, which allows us to spawn an arbitrary computation on a child thread, and wait for the child thread to receive the computation result. The implementation and specification are given in [Figure 12.6](#).

The library provides 3 functions: **spawn**, **finish**, and **join**. **finish** and **join** are meant to be used—and indeed are implemented exactly—as message passing: once the child thread is done with the computation, it calls **finish** $([\ell, v])$ , which writes the computation result  $v$  non-atomically to  $\ell + 1$ , and signals the completion with a release write of 1 to  $\ell$ . A waiting thread calls **join** $([\ell])$ , which waits with a repeat loop reading  $\ell$ , and once the loop finishes it reads the result from  $\ell + 1$ , and cleans up the memory block of  $\ell$ . The child thread can be spawned with **spawn** $([f])$ , where  $f$  is the computation that is to be executed in parallel. **spawn** simply allocates the block of size 2. The allocated block with the base location  $\ell$  is initialized with 0 for  $\ell$ —we do not really need to initialize  $\ell + 1$  (this applies to **mp** too). **spawn** then forks a new thread with the computation  $f$ , which takes in the location  $\ell$  to report the result. It is assumed that  $f$  will call **finish** $([\ell, v])$  at the end. If this is not the case, then  $f$  does not need to take  $\ell$  as an argument but we assume  $f$  returns the computation result, and we can then change **spawn**'s line 3 to **fork**  $\{ \text{finish}([\ell, f()]) \}$ .

The specifications of the library functions assume a predicate  $\Phi : \text{Val} \rightarrow \text{vProp}$ , where the computation is expected to produce not just some value  $v$ , but also the resource  $\Phi(v)$ . The specifications assume a namespace  $\mathcal{N}$  that will be used to allocate the library invariant. The specifications then involve two assertions: the finish handle  $\text{FinishHandle}^{\mathcal{N}}(\ell, \Phi)$  and the join handle  $\text{JoinHandle}^{\mathcal{N}}(\ell, \Phi)$ . Intuitively, both handles will be generated by **spawn**, and then the finish handle will be given to the computation  $f$ , which eventually must call **finish**, while the join handle will be given to the caller of **join**. As such, **SPAWN-SPEC** says that **spawn** (1) assumes for the computation  $f$  a weakest pre-condition that will

consume the finish handle that **spawn** generates, and (2) returns the join handle in the post-condition. **FINISH-SPEC** says that once  $f$  produces the result  $v$  and finally calls **finish**, it consumes the finish handle and release the resource  $\Phi(v)$ . **JOIN-SPEC** says that **join** consumes the join handle generated by **spawn** and once it is done the resulting resource  $\Phi(v)$  is returned.

Note that we could have stated the pre-condition of **SPAWN-SPEC** with a Hoare triple for  $f$ , i.e.,

$$\forall \ell, \rho. \{ \text{FinishHandle}^{\mathcal{N}}(\ell, \Phi) \} f([\ell]) \text{ in } \rho \{ \_ . \text{True} \}_\top$$

However, stating as it in in **SPAWN-SPEC**, we allow the client to use extra resources to verify their  $f$ . Recall that a Hoare triple is defined with a persistent modality ( $\Box$ ), so if we use a Hoare triple the client can only use  $\text{FinishHandle}^{\mathcal{N}}(\ell, \Phi)$  for the proof of  $f$ .

**Definition 12.4** (Invariant for Spawn-and-Join). The invariant of Spawn-and-Join is almost the same as that for **mp\_reclaim** (Definition 12.3), except that the message to be sent is now some value  $v$  together with the resource  $\Phi(v)$ .

$$\begin{aligned} \text{spawnJoin}(\ell, \gamma_L, \gamma) : \text{vProp} ::= & \\ & \exists h, b, t_0, V_0. \ell \mapsto_{\text{sw}}^{\gamma_L} h * \text{let } h_0 := [t_0 \leftarrow (0, V_0)] \text{ in} \\ & \text{if } b = \text{false then } h = h_0 \text{ else} \\ & \exists t_1 > t_0, V_1. h = [t_0 \leftarrow (0, V_0)][t_1 \leftarrow (1, V_1)] \\ & * @_{V_1}(\heartsuit_{1/2}^{\gamma} * \exists v. \ell + 1 \mapsto v * \Phi(v)) \end{aligned}$$

**Definition 12.5** (Model of Handles).

$$\begin{aligned} \text{FinishHandle}^{\mathcal{N}}(\ell, \Phi) ::= & \exists \gamma_L, \gamma, t, V. \ell + 1 \mapsto \heartsuit * \ell \sqsupseteq_{\text{sw}}^{\gamma_L} [t \leftarrow (0, V)] \\ & * \heartsuit_{1/2}^{\gamma} * \boxed{\text{spawnJoin}(\ell, \gamma_L, \gamma)}^{\gamma, \mathcal{N}} \\ \text{JoinHandle}^{\mathcal{N}}(\ell, \Phi) ::= & \exists \gamma_L, \gamma, t, V. \ell \sqsupseteq_{\text{sn}}^{\gamma_L} [t \leftarrow (0, V)] * \dagger^2 \ell \\ & * \heartsuit_{1/2}^{\gamma} * \boxed{\text{spawnJoin}(\ell, \gamma_L, \gamma)}^{\gamma, \mathcal{N}} \end{aligned}$$

The model for the finish and join handles mirror exactly what we have seen in the proof of **mp\_reclaim** (Figure 12.4). In particular, the finish handle  $\text{FinishHandle}^{\mathcal{N}}(\ell, \Phi)$  carries the resources needed to send the message (but without  $\Phi(v)$ ), mirroring the resources owned by the thread  $\rho$  of **mp\_reclaim**: (1) the non-atomic points-to of the location  $\ell + 1$  that will be used to store the result, (2) the single-writer ownership of the flag  $\ell$ , and (3) the half token  $\heartsuit_{1/2}^{\gamma}$  and the knowledge of the cancelable invariant  $\boxed{\text{spawnJoin}(\ell, \gamma_L, \gamma)}^{\gamma, \mathcal{N}}$ . The join handle  $\text{JoinHandle}^{\mathcal{N}}(\ell, \Phi)$  carries the resources needed to receive the message and to clean up the memory locations, mirroring the resources owned by the thread  $\pi$  of **mp\_reclaim** from line  $\pi 4$ : (1) the seen-history observation of the flag  $\ell$ , (2) the block ownership of  $\ell$ , and (3) the other half token  $\heartsuit_{1/2}^{\gamma}$  and the knowledge of the same cancelable invariant.

The proofs of **FINISH-SPEC** and **JOIN-SPEC** then follow almost the same proof of **mp\_reclaim** in Figure 12.4. The main differences is the extra

Context:  $\text{let } P := \left( \forall \ell, \rho. \text{FinishHandle}^{\mathcal{N}}(\ell, \Phi) \multimap \text{wp } f([\ell]) \text{ in } \rho \{ \_ . \text{True} \} [\top] \right)$   
 $\{P\}$   
**1:**  $\text{let } \ell := \text{alloc}(2) \text{ in}$   
**2:**  $\ell :=_{\text{na}} 0; \{ \ell \mapsto 0 * \ell + 1 \mapsto \text{⊗} * \dagger^2 \ell \} \text{ // NA-ALLOC and NA-WRITE}$   
 $\{ P * \ell + 1 \mapsto \text{⊗} * \dagger^2 \ell * \exists \gamma_\ell, t_0, V_0. \exists V_0 * \ell \mapsto_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * \ell \sqsupseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] \} \text{ // NA-AT-SW}$   
 $\{ P * \ell + 1 \mapsto \text{⊗} * \dagger^2 \ell * \ell \sqsupseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * \heartsuit_1^\gamma * \boxed{\text{spawnJoin}(\ell, \gamma_\ell, \gamma)}^{\gamma, \mathcal{N}} \} \text{ // CINV-ALLOC-OPEN}$   
Context:  $\boxed{\text{spawnJoin}(\ell, \gamma_\ell, \gamma)}^{\gamma, \mathcal{N}}$   
 $\{ P * \ell + 1 \mapsto \text{⊗} * \ell \sqsupseteq_{\text{sw}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * \heartsuit_{1/2}^\gamma * \ell \sqsupseteq_{\text{sn}}^{\gamma_\ell} [t_0 \leftarrow (0, V_0)] * \dagger^2 \ell * \heartsuit_{1/2}^\gamma \}$   
// AT-SW-SY and AT-SY-SN and CINV-TOK-FRAC  
 $\{ P * \text{FinishHandle}^{\mathcal{N}}(\ell, \Phi) * \text{JoinHandle}^{\mathcal{N}}(\ell, \Phi) \}$   
**3:**  $\text{fork} \{ \dots \} \text{ // HOARE-FORK}$   

|              |                                                                                                                                                 |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Child thread | $\{ P * \text{FinishHandle}^{\mathcal{N}}(\ell, \Phi) \}$<br>$f([\ell]) \text{ in } \rho$<br>$\{ \_ . \text{True} \} \text{ // By applying } P$ |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------|

 $\{ \text{JoinHandle}^{\mathcal{N}}(\ell, \Phi) \}$   
**4:**  $\ell \{ \ell. \text{JoinHandle}^{\mathcal{N}}(\ell, \Phi) \}$

FIGURE 12.7: Hoare proof outlines for **SPAWN-SPEC**

releasing and acquiring of the resulting resource  $\Phi(v)$ . In the proof of **FINISH-SPEC**, corresponding to line  $\rho 2$  of **mp\_reclaim**, when invoking **AT-WRITE-SW-REL-VJ** we also release  $\Phi(v)$  to the view  $V_1$ , i.e., we get  $@_{V_1} \Phi(v)$  in the post-condition, which we then put into the invariant. In the proof of **JOIN-SPEC**, at the end of the loop in line  $\pi 4$  of **mp\_reclaim**, we acquire also  $@_{V_1} \Phi(v)$ , which again can be used with **VA-ELIM** to get  $\Phi(v)$ .

The proof of **SPAWN-SPEC** also follows the proof of **mp\_reclaim**. We give its Hoare proof outlines in **Figure 12.7**.

## 12.4 A Release-Acquire Treiber Stack

We now look at the verification of a linked-list based Treiber stack implementation using release-acquire and relaxed accesses, given in **Figure 12.8**. The interface includes 5 functions:

1. **new\_stack**( $[]$ ) allocates and returns a new, empty stack handle  $s$ ;
2. **push**( $[s, v]$ ) pushes a new element  $v$  into the stack  $s$ ;
3. **pop**( $[s]$ ) pops and returns an element in  $s$ , unless  $s$  is empty then it returns 0;
4. **try\_push**( $[s, v]$ ) is a *try* version of **push**, which returns true if it have successfully pushed to the stack  $s$ , or returns false if it fails to do so. **try\_push** can fail due to contention by concurrent pushes/pops. Unlike **try\_push**, **push** would try again until it succeeds.
5. **try\_pop**( $[s, v]$ ) is a *try* version of **pop**. It returns an element from the stack  $s$ , or returns empty (0), or returns  $-1$  in case it fails due to

|                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> <b>new_stack</b> ::=   λ[].   1: <b>let</b> <math>s := \text{alloc}(1)</math> <b>in</b>   2: <math>s :=_{\text{na}} 0</math>;   3: <math>s</math>  <b>try_push_swap</b> ::=   λ[<math>s, n</math>].   1: <b>let</b> <math>s_h := *_{\text{rlx}} s</math> <b>in</b>   2: <math>n :=_{\text{na}} s_h</math>;   3: <math>\text{CAS}^{\text{rel}}(s, s_h, n)</math> </pre> | <pre> <b>try_push</b> ::=   λ[<math>s, v</math>].   1: <b>let</b> <math>n := \text{alloc}(2)</math> <b>in</b>   2: <math>(n + 1) :=_{\text{na}} v</math>; // write elem   3: <b>if</b> <b>try_push_swap</b>([<math>s, n</math>])   4: <b>then true</b>   5: <b>else free</b>(<math>n, 2</math>); <b>false</b>  <b>push</b> ::=   λ[<math>s, v</math>].   1: <b>let</b> <math>n := \text{alloc}(2)</math> <b>in</b>   2: <math>(n + 1) :=_{\text{na}} v</math>; // write elem   3: <b>repeat</b> (<b>try_push_swap</b>([<math>s, n</math>])) </pre> | <pre> <b>try_pop</b> ::=   λ[<math>s</math>].   1: <b>let</b> <math>s_h := *_{\text{acq}} s</math> <b>in</b>   2: <b>if</b> <math>s_h == 0</math> // null   3: <b>then</b> 0 // EMPTY   4: <b>else</b>   5:   <b>let</b> <math>n := *_{\text{na}} s_h</math>   6:   <b>if</b> <math>\text{CAS}^{\text{acq}}(s, s_h, n)</math> // swap   7:   <b>then</b> <math>*_{\text{na}}(s_h + 1)</math> // read elem   8:   <b>else</b> -1 // FAIL  <b>pop</b> ::=   <b>rec pop</b>([<math>s</math>]) :=   1: <b>let</b> <math>v := \text{try\_pop}([s])</math> <b>in</b>   2: <b>if</b> <math>v == -1</math>   3: <b>then pop</b>([<math>s</math>]) <b>else</b> <math>v</math> </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

FIGURE 12.8: A simple release-acquire implementation for Treiber stacks

contention by concurrent pushes/pops. Unlike **try\_pop**, **pop** would try again until it succeeds or until it finds that the stack is empty.

#### 12.4.1 A Release-Acquire Implementation of the Treiber Stack

The implementation employs a list-linked based representation of the stack, which only requires *non-atomic* (**na**) accesses, because once a node is initialized it is never changed. The only atomic location is the location for the head of the stack. We note that the implementation does not take care of resource reclamation, and nor do we have an interface for that. The implementation includes an extra internal function **try\_push\_swap**.

- **new\_stack**([]) allocates a location  $s$  that will store the pointer to the *head* node, i.e., the top of the stack.  $s$  is initialized to null, i.e., the value 0 (line 2).
- **try\_push**([ $s, v$ ]) starts with allocating a *node*  $n$  of size 2 to store the to-be-pushed element  $v$ . The value  $v$  is stored at the location  $n + 1$  (line 2), while the location  $n$  will be used to store the pointer to the *next* node in the linked list of the stack. **try\_push** then calls **try\_push\_swap**([ $s, n$ ]) to try to swap  $n$  in as the new head (line 3). The function returns **true** if the try succeeds. Otherwise, it cleans up the node  $n$  and returns **false** (line 5).
- **try\_push\_swap**([ $s, n$ ]) tries to read the pointer  $s_h$  to the (potentially current) head of the stack  $s$ , and then stores  $s_h$  in the *next* “field” of the new node  $n$  (which is  $n$ ) in line 2. Then in 3 it uses a release compare-and-swap (**CAS**) instruction to try to swap  $n$  for  $s_h$  in the location  $s$ . The function returns the value returned by the **CAS**. If

the **CAS** succeeds, it returns **true**, and the stack pointer  $s$  points-to the new head node  $n$ , which in turn points-to the old head node  $s_h$ .

If the **CAS** fails, it returns **false** and nothing changes but the node  $n$ 's next field. The **CAS** may fail if  $s_h$  was not really the current head of  $s$ . This could be because the relaxed in line 1 may read a stale value, or because there are concurrent push or pop operations that have updated the head of the stack while the function was running between lines 1 and 3.

Note that the **CAS** is a release (**rel**) **CAS**, *i.e.*, it uses only the release write access mode for the successful write, and it only uses the relaxed access mode for the reads in both success and failure cases. The release write mode allows a successful push to release anything that happens before it to the matching successful pop.

- **push**( $[s, v]$ ) is similar to **try\_push**: it allocates and initializes a new node  $n$ , but instead of calling **try\_push\_swap** only once, it will keep calling **try\_push\_swap** until it succeeds.
- **try\_pop**( $[s]$ ) tries to read the top (first) node of the stack—which the head pointer points to—and the second node, and swaps the head pointer to point to the second node. More specifically, in line 1, it reads the potentially current head of the stack into  $s_h$ . If  $s_h$  is null (0), it returns empty (0) immediately, in line 3. If  $s_h$  is not null, then in line 5 the function reads the pointer to  $s_h$ 's next node, stored in the location  $s_h$ , into  $n$ . The function then uses an acquire **CAS** to swap  $n$  for  $s_h$  in the location  $s$  (line 6). If the **CAS** succeeds, then  $n$  will be the new top of the stack, and the function can read and return the element value of the old top, from the location  $s_h + 1$ , in line 7.

If the **CAS** fails, then **try\_pop** returns  $-1$  to signal failure due to possible contention by concurrent operations. The **CAS** is an acquire (**acq**) **CAS**, *i.e.*, it uses the acquire mode for the successful read, but uses the relaxed mode for the successful write and the failure read. This is because a pop does not try to release anything. Instead, it acquires what the matching push that it reads from was releasing. In other words, the happens-before **hb** relation is only established between matching pairs of a successful push and a successful pop.

- **pop**( $[s]$ ) simply calls **try\_pop**. It only returns if **try\_pop** returns an actual element or the empty value 0. If **try\_pop** fails due to contention, **pop** tries again.

We note again the all accesses to the link-list's nodes are non-atomic. Furthermore, only the successful write of a push operation uses the release access mode, and in a pop operation, only the read of the head in 1 and the successful read by the **CAS** in 6 use the acquire access mode. Other memory accesses use relaxed access mode.



$$\begin{array}{l}
\text{STACK-BAG-NEW} \\
\{\text{True}\} \text{new\_stack}([\ ])\text{ in } \pi \{s. \text{isStack}^{\mathcal{N}}(s, \Phi)\}_{\top} \\
\\
\text{STACK-BAG-TRY-PUSH} \\
\{\text{isStack}^{\mathcal{N}}(s, \Phi) * \Phi(v)\} \text{try\_push}([s, v])\text{ in } \pi \{b. (b = \text{false} * \Phi(v)) \vee b = \text{true}\}_{\top} \\
\\
\text{STACK-BAG-TRY-POP} \\
\{\text{isStack}^{\mathcal{N}}(s, \Phi)\} \text{try\_pop}([s])\text{ in } \pi \{v. v = -1 \vee v = 0 \vee \Phi(v)\}_{\top} \\
\\
\text{STACK-BAG-PUSH} \qquad \qquad \qquad \text{STACK-BAG-POP} \\
\{\text{isStack}^{\mathcal{N}}(s, \Phi) * \Phi(v)\} \text{push}([s, v])\text{ in } \pi \{\text{True}\}_{\top} \qquad \{\text{isStack}^{\mathcal{N}}(s, \Phi)\} \text{pop}([s])\text{ in } \pi \{v. v = 0 \vee \Phi(v)\}_{\top} \\
\\
\text{STACK-BAG-TRY-PUSH-SWAP} \\
\{\text{isStack}^{\mathcal{N}}(s, \Phi) * \Phi(v) * n \mapsto \_ * (n + 1) \mapsto v * \dagger^2 n\} \\
\text{try\_push\_swap}([s, v])\text{ in } \pi \\
\{b. (b = \text{false} * \Phi(v) * n \mapsto \_ * (n + 1) \mapsto v * \dagger^2 n) \vee b = \text{true}\}_{\top}
\end{array}$$

FIGURE 12.9: Bag or per-element specifications for Treiber stacks

#### 12.4.2 Bag or Per-element Specifications for Stacks

We will verify the implementation in Figure 12.8 against the so-called *bag* or *per-element* specifications, given in Figure 12.9. These specifications are rather weak: they only establish a connection between matching pairs of successful push and pop operations. We will verify the implementation against stronger specifications that establish stack properties, e.g., last-in-first-out (LIFO), in Part IV. Nevertheless, the bag specifications are sufficient as a good demonstration for iRC11—they were also used in GPS/iGPS.<sup>6</sup>

The specifications involve a persistent assertion  $\text{isStack}^{\mathcal{N}}(s, \Phi)$  which says that the location  $s$  is a stack tied to a predicate  $\Phi : \text{Val} \rightarrow \text{vProp}$ . **STACK-BAG-NEW** says that `new_stack`( $[\ ]$ ) returns a new stack  $s$  that satisfies  $\text{isStack}^{\mathcal{N}}(s, \Phi)$  for the user-chosen  $\Phi$  and  $\mathcal{N}$ . The namespace  $\mathcal{N}$  is needed to store the underlying invariant for the stack  $s$ . The predicate  $\Phi(v)$  dictates the per-element resource that a push operation of  $v$  will release. As can be seen in **STACK-BAG-TRY-PUSH** and **STACK-BAG-PUSH**, a successful push of  $v$  to the stack  $s$  will consume  $\Phi(v)$ , while a failed `try_push` will not. On the other side, **STACK-BAG-TRY-POP** and **STACK-BAG-POP** says that a successful non-empty pop of some element  $v$  from the stack  $s$  will acquire the corresponding resource  $\Phi(v)$  that has been released by the push of  $v$ . As such, the specifications at least recognize the synchronization between matching pairs of push and pop operations, by which a user-chosen resource  $\Phi(v)$  can flow from one thread to another. Under the hood, this synchronization is established by the release-acquire synchronization through the stack location  $s$  (between line 3 of `try_push_swap` and line 1 of `try_pop`).

We also give the specification **STACK-BAG-TRY-PUSH-SWAP** for the internal function `try_push_swap`. The function assumes the non-atomic ownership of the locations  $n$  and  $n + 1$ , and requires that  $n + 1$  has been set to the to-be-pushed value  $v$ . Additionally it also assumes the

<sup>6</sup>Turon et al., “GPS: navigating weak memory with ghosts, protocols, and separation” [TVD14]; Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

per-element resource  $\Phi(v)$ . All of these resources will be consumed if the function succeeds.

### 12.4.3 Verification of the Treiber Stack against the Bag Specifications

From the specification **STACK-BAG-TRY-PUSH-SWAP** for **try\_push\_swap**, we can easily verify **try\_push** and **push** against **STACK-BAG-TRY-PUSH** and **STACK-BAG-PUSH**. Furthermore, we also can easily verify **pop** against **STACK-BAG-POP** assuming **STACK-BAG-TRY-POP** for **try\_pop**. We therefore present the verifications of **new\_stack**, **try\_push\_swap**, and **try\_pop** below. As usual, we start with defining the invariant for the stack implementation.

**Definition 12.6** (Invariant for Per-element Treiber Stack).

$$\begin{aligned}
\text{null0}(v) &::= (v = \text{Some}(\ell)) ? \ell : 0 \\
\text{AllNodes}(vs) &::= \bigstar_{(\text{Some}(n), V) \in vs} \exists q, v. @_V(n \xrightarrow{q} \text{null0}(v)) \\
\text{InNodes}(S) &::= \bigstar_{n=S(i)} \exists v. n + 1 \mapsto v * \Phi(v) * \dagger^2 n * \exists q. n \xrightarrow{q} \text{null0}(S(i + 1)) \\
\text{TreiberBI}(s, \gamma) &::= \\
&\quad \exists V_s, t_0, V_h, vs, S. \\
&\quad \text{let } v_h : \text{Loc}^? := \text{hd}(S); vs' : \overline{(\text{Loc}^?, \text{View})} := vs ++ [(v_h, V_h)] \text{ in} \\
&\quad \text{let } h := [t_0 + i \leftarrow (\text{null0}(v), V) \mid vs'(i) = (v, V)] \text{ in} \\
&\quad @_{V_s}(s \mapsto_{\text{con}}^{\gamma} h) * @_{V_h} \text{InNodes}(S) * \text{AllNodes}(vs') * \dagger^1 s \\
\text{isStack}^{\mathcal{N}}(s) &::= \exists \gamma, h. s \sqsubseteq_{\text{sn}}^{\gamma} h * \boxed{\text{TreiberBI}(s, \gamma)}^{\mathcal{N}}
\end{aligned}$$

The invariant content  $\text{TreiberBI}(s, \gamma)$  for some location  $s$  and its atomic period identifier  $\gamma$  (used by its atomic points-to) is objective, as we will put it in an objective invariant (§11.1) because we do not care to reclaim the stack. Nevertheless, we leave the block ownership (**free**(1,  $s$ ) for the stack pointer and  $\dagger^2 n$  for each node) in the definition as a reminder that we may want to extend the invariant to support reclamation of the stack.  $\text{isStack}^{\mathcal{N}}(s, \Phi)$  simply asserts the existence of the objective invariant for  $\text{TreiberBI}(s, \gamma)$  in  $\mathcal{N}$ , and a seen-history observation for  $s$ , needed to perform atomic operations on  $s$ .

The definitions rely on a  $\text{null0}$  function that turns an optional location ( $v \in \text{Loc}^?$ ) into a nullable location value, where  $\text{null}$  is the value 0. The invariant content  $\text{TreiberBI}(s, \gamma)$  is composed of 3 parts: the atomic points-to of  $s$ , the ownership of nodes (in  $S$ ) that are currently in the stack, and the ownership of all nodes (in  $vs'$ ) that have ever been in the stack.

- The atomic points-to ownership  $s \mapsto_{\text{con}}^{\gamma} h$  of the stack pointer is put in the concurrent mode (**con**) and at some view  $V_s$  (to make it objective). The history  $h$  is a contiguous block of writes because we only use **CAS**'s on  $s$ .<sup>7</sup> This contiguous block of writes starts at the timestamp  $t_0$ , and contains the writes from the list

<sup>7</sup>Note that we do not use the CAS-only mode (**cas**) atomic points-to yet—that mode was developed mainly for GPS protocols in Part III. Nevertheless, the contiguous block-of-writes abstraction can also be a useful feature for CAS-only atomic points-to, but it has not been incorporated into atomic points-to.

$vs' \in \overline{(Loc^?, View)}$  of pairs of optional locations and views. That is, the history of  $s$  only contains nullable location values.

The top write of the block, which is the latest write to  $s$  and the pointer to the current top node of the stack, is the message  $(t_0 + |vs|, \text{null0}(v_h), V_h)$ ,<sup>8</sup> where  $vs$  is the list of non-current writes to  $s$ .  $v_h$  is the head ( $\text{hd}$ ) of the stack's *abstract state*  $S \in \overline{Loc}$ , which is a list of pointers to nodes in the stack. We model the abstract stack as a list whose top element is at index 0 and bottom element is at index  $(|S| - 1)$ . If the abstract stack is empty, *i.e.*,  $S = []$ , then  $\text{hd}(S) = \text{None}$ , and the latest write to  $s$  has the value  $\text{null0}(\text{None}) = 0$ , *i.e.*, the null value.

<sup>8</sup>Indices  $i$  to a list start at 0.

- $\text{InNodes}(S)$  contains ownership of all nodes that are currently in the stack. It constructs a singly-linked list starting from the top node of the abstract stack  $S$ , which is at index 0, and ending at the bottom node at index  $(|S| - 1)$ . The ownership of each node is grouped together using the big separating conjunction  $\ast$ .

For each node  $S(i) = n \in Loc$  that is currently in the stack, we have: (1) the full non-atomic points-to ownership  $n + 1 \mapsto v$  of the *data* field with a pushed value  $v$ , and (2) the corresponding released resource  $\Phi(v)$  of the push, and (3) a fractional non-atomic points-ownership  $n \xrightarrow{q} \text{null0}(S(i + 1))$  of  $n$ 's *next* field that points-to the next node in the stack. If  $(i + 1)$  is out-of-bound, then  $S(i + 1)$  is  $\text{None}$  and the next field of the bottom node will store the null value (0).

- $\text{AllNodes}(vs')$  contains fractional ownership of next fields of all nodes  $(vs')$  that have ever been in the stack. For every write to  $s$  with a non-null location  $n$  (a node) and message view  $V$ , it owns a fraction of the non-atomic points-to  $n \xrightarrow{q} \text{null0}(v)$  for  $n$ 's next field, which has some nullable location value  $v$ . The fractional points-to is put at the view  $V$  of the write message to the stack location  $s$ .  $\text{AllNodes}(vs')$  is needed because a pop operation can read the next field of *any* node that has ever been in the stack (line 5 of `try_pop`).

**Remark 12.7** (On Reclaiming Stack Resources). Looking at the definition of `TreiberBl`, we can see that the main problem with reclaiming the stack's resources is in the non-atomic points-to ownership of the nodes' next fields: they are split into fractions that are not carefully tracked (existentially quantified), and there are overlapping ownership between  $\text{InNodes}(S)$  and  $\text{AllNodes}(vs')$ .  $\text{AllNodes}(vs')$  is needed because of the non-atomic read of a node's next field in `try_pop`'s line 5. More specifically, we need to acquire some fraction  $q$  of  $n$ 's non-atomic in line 1, so that at line 5 we can read  $n$ . To enable reclamation, we either have to recollect this fraction  $q$ , or avoid giving out a fraction at all. With the latter choice, we keep the full ownership of  $n$  in the invariant, and turn the non-atomic read in 5 into a relaxed read of  $n$ , which now can open the invariant to access  $n$ . Naturally, we have to avoid the overlaps between  $\text{InNodes}(S)$  and  $\text{AllNodes}(vs')$ , by carefully split the nodes into

those currently in the stack, and those that have been but are no longer in the stack. Once all of that is set up, we use a cancelable invariant (§11.2) to be able to reclaim all resources that have ever been put into the stack.

We now look at the proof sketches of the most 3 important functions.

*Proof sketch of new\_stack.* The proof is easy. We use **NA-ALLOC** and **NA-WRITE** (§9.1) respectively for allocation and initialization of  $s$ . Then we use **NA-AT-sw** (Figure 10.2) to turn the non-atomic points-to to an atomic points-to in single-writer mode (sw) with some atomic period identifier  $\gamma$ , and **AT-sw-con** to switch the mode to concurrent (con). We then use **VA-INTRO** to put the atomic points-to of  $s$  at some view  $V_s$ . From the allocated resources we can easily construct  $\text{TreiberBl}(s, \gamma)$ , because the history of  $s$  is a singleton with a null location value. We use **OINV-ALLOC-OBJ** (§11.1) to allocate the invariant, and then we are done.

```

{True}
1: let  $s := \text{alloc}(1)$  in
  { $s. \uparrow^1 s * s \mapsto \text{null}$ } // NA-ALLOC
2:  $s :=_{\text{na}} 0; \{\uparrow^1 s * s \mapsto 0\}$  // NA-WRITE
  { $\uparrow^1 s * \exists \gamma, t_0, V_h. s \mapsto_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_h)] * s \sqsubseteq_{\text{sw}}^\gamma [t_0 \leftarrow (0, V_h)]$ }
  { $\uparrow^1 s * s \sqsubseteq_{\text{sn}}^\gamma [t_0 \leftarrow (0, V_h)] * \exists V_s. @_{V_s}(s \mapsto_{\text{con}}^\gamma [t_0 \leftarrow (0, V_h)])$ }
  // NA-AT-sw and AT-sw-con and AT-sw-sy and AT-sy-sn and VA-INTRO
  { $s \sqsubseteq_{\text{sn}}^\gamma [t_0 \leftarrow (0, V_h)] * \text{TreiberBl}(s, \gamma)$ }
3:  $s \{s. \text{isStack}^N(s, \Phi)\}$  // OINV-ALLOC-OBJ

```

□

*Proof sketch of try\_push\_swap.* The proof is also easy. We open the invariant twice using **OINV-ACC-OBJ**, in lines 1 and 3 to get access to the atomic points-to of  $s$  to read or **CAS** on it. In line 1 we do not change the invariant. In line 3, if the **CAS** succeeds, then we extend the abstract state  $S$  with our new node  $n$  into  $[n] \uparrow\uparrow S$  and put all of the resources in the pre-condition to that node in the invariant. The proof outlines are given in Figure 12.10.

The most important point is that we can easily establish deterministic pointer comparison (the conditions involving  $P_{\text{cmp}}$ ) when performing the **CAS**, because the location  $s$  only stores nullable locations, whose points-to ownership (if non null) are kept inside the invariant  $\text{TreiberBl}$  and thus we know that they are all alive. □

*Proof sketch of try\_pop.* The proof is not so complicated. We again open the invariant twice using **OINV-ACC-OBJ**, in lines 1 and 6 to get access to the atomic points-to of  $s$  to read or **CAS** on it. In line 1, we do not change the invariant, but if we read  $s_h$  to be non-null, we also acquire from  $\text{AllNodes}(vs')$  some fraction  $q$  of  $s_h$ 's non-atomic points-to for its next field, i.e.,  $s_h \xrightarrow{q} \text{null0}(n)$  for some  $n$ . This fraction will be used to read  $s_h$ 's next field in line 4.<sup>9</sup>

In line 6, if the **CAS** succeeds, we know that  $s_h$  is indeed the top of the current abstraction  $S$ , i.e.,  $S = [s_h] \uparrow\uparrow S'$  for some  $S'$ . We then

<sup>9</sup>And it is also the source of leaking ownership of nodes in this proof.

$$\begin{array}{l}
\{ \text{isStack}^{\mathcal{N}}(s, \Phi) * \Phi(v) * n \mapsto \_ * (n+1) \mapsto v * \dagger^2 n \} \\
\text{Context: } \boxed{\text{TreiberBl}(s, \gamma)}^{\mathcal{N}} \\
\{ s \sqsubseteq_{\text{sn}}^{\gamma} h_0 * \Phi(v) * n \mapsto \_ * (n+1) \mapsto v * \dagger^2 n \}_{\top} \\
\text{O[INV-ACC-OBJ]} \left\{ \begin{array}{l}
\{ s \sqsubseteq_{\text{sn}}^{\gamma} h_0 * \sqsubseteq \emptyset * \triangleright \text{TreiberBl}(s, \Phi) \}_{\top \setminus \mathcal{N}} \text{ // VS-BOT} \\
\text{1: let } s_h := * \text{rlx } s \text{ in} \\
\{ s_h. \exists h'_0, t_h. h'_0(t_h) = (s_h, \_) * s \sqsubseteq_{\text{sn}}^{\gamma} h'_0 * \text{TreiberBl}(s, \Phi) \}_{\top \setminus \mathcal{N}} \text{ // AT-READ-SN} \\
\{ s \sqsubseteq_{\text{sn}}^{\gamma} h'_0 * \Phi(v) * n \mapsto \_ * (n+1) \mapsto v * \dagger^2 n \}_{\top} \\
\text{2: } n :=_{\text{na}} s_h; \{ \Phi(v) * n \mapsto s_h * (n+1) \mapsto v * \dagger^2 n \}_{\top} \text{ // NA-WRITE} \\
\left\{ \begin{array}{l}
\{ s \sqsubseteq_{\text{sn}}^{\gamma} h'_0 * \exists V_0. \sqsubseteq V_0 * @_{V_0}(\Phi(v) * n \mapsto s_h * (n+1) \mapsto v) * \dagger^2 n * \triangleright \text{TreiberBl}(s, \gamma) \}_{\top \setminus \mathcal{N}} \text{ // VA-INTRO} \\
\left\{ \begin{array}{l}
s \sqsubseteq_{\text{sn}}^{\gamma} h'_0 * \sqsubseteq V_0 * @_{V_0}(\Phi(v) * n \mapsto s_h * (n+1) \mapsto v) * \dagger^2 n * \\
@_{V_s}(s \mapsto_{\text{con}}^{\gamma} h) * @_{V_h} \text{InNodes}(S) * \text{AllNodes}(vs')
\end{array} \right\} \text{ // Unfolding TreiberBl}
\end{array} \right\} \\
\text{We know } s_h \text{ is a value written to } h \text{ at } t_h, \text{ because } h'_0 \subseteq h. \\
\text{3: CAS}^{\text{rel}}(s, s_h, n) \\
\left\{ \begin{array}{l}
b. \exists h', V' \sqsubseteq V_0, v'. \sqsubseteq V' * @_{V_s \sqcup V'}(s \mapsto_{\text{con}}^{\gamma} h') * \\
@_{V'}(\Phi(v) * n \mapsto s_h * (n+1) \mapsto v) * \dagger^2 n * @_{V_h} \text{InNodes}(S) * \text{AllNodes}(vs') * \dagger^1 s * \\
((b = \text{false} * s_h \neq v' * h' = h) \vee (b = \text{true} * s_h = v' * \exists V_w \sqsubseteq V' \sqcup V_h. h' = h[t_h + 1 \leftarrow (n, V_w)]))
\end{array} \right\}_{\top \setminus \mathcal{N}} \\
\text{// AT-CAS-SN-GEN Note that any location values readable from } s \text{ are alive because we store their} \\
\text{fractional non-atomic points-to ownership in AllNodes}(vs'). \\
\left\{ \begin{array}{l}
(b = \text{false} * \Phi(v) * n \mapsto \_ * (n+1) \mapsto v * \dagger^2 n * \\
@_{V_s \sqcup V'}(s \mapsto_{\text{con}}^{\gamma} h) * @_{V_h} \text{InNodes}(S) * \text{AllNodes}(vs') * \dagger^1 s * \\
\vee (b = \text{true} * @_{V_s \sqcup V'}(s \mapsto_{\text{con}}^{\gamma} h') * @_{V_w} \text{InNodes}([n] \uparrow \uparrow S) * \text{AllNodes}(vs' \uparrow \uparrow [(Some(n), V_w)]) * \dagger^1 s)
\end{array} \right\} \\
\text{In the failure case } (b = \text{false}), \text{ we use VA-ELIM to get back the original resources without the view-at} \\
\text{modality.} \\
\text{In the successful case } (b = \text{true}), \text{ the resources are put inside InNodes and AllNodes of the invariant.} \\
\left\{ \begin{array}{l}
(b = \text{false} * \Phi(v) * n \mapsto \_ * (n+1) \mapsto v * \dagger^2 n * \text{TreiberBl}(s, \gamma)) \\
\vee (b = \text{true} * \text{TreiberBl}(s, \gamma))
\end{array} \right\}_{\top \setminus \mathcal{N}} \\
\{ b. (b = \text{false} * \Phi(v) * n \mapsto \_ * (n+1) \mapsto v * \dagger^2 n) \vee b = \text{true} \}_{\top}
\end{array}
\right.$$

FIGURE 12.10: Hoare proof outlines for `try_push_swap`

can acquire the resources for  $s_h$  in  $\text{InNodes}(S)$ , and leave the rest as  $\text{InNodes}(S')$  for the new abstract state  $S'$ .

We give the Hoare proof outlines in Figure 12.11. We note again that we always have deterministic pointer comparison with  $\text{CAS}$ 's, because ownership of all reachable locations in the stack are stored in the invariant  $\text{TreiberBl}$ , so we know they are all alive.  $\square$

**Remark 12.8** (On the Use of Acquire  $\text{CAS}$  in `try_pop`). We note that we do not really need the  $\text{CAS}$  of `try_pop` (line 6) to use the acquire access mode. In fact, the acquire access mode used in the read of the stack pointer in line 1 alone should be sufficient, and such implementation should also be verifiable in iRC11. Intuitively, by reading acquire in `try_pop`'s line 1, we have been synchronized with the write of  $n$  to  $s$ , whose view is  $V_n$ , i.e., we acquire  $\sqsubseteq V_n$ . The view  $V_n$  should be bigger

|              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|              | $\{ \text{isStack}^{\mathcal{N}}(s, \Phi) \}_{\top}$                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|              | Context: $\boxed{\text{TreiberBl}(s, \gamma)}^{\mathcal{N}}$                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|              | $\{ s \sqsubseteq_{\text{sn}}^{\gamma} h_0 \}_{\top}$                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|              | $\{ s \sqsubseteq_{\text{sn}}^{\gamma} h_0 * \sqsupseteq \emptyset * \triangleright \text{TreiberBl}(s, \Phi) \}_{\top \setminus \mathcal{N}} \quad // \text{VS-BOT}$                                                                                                                                                                                                                                                                                                                             |
| OINV-ACC-OBJ | 1: <b>let</b> $s_h := \text{*acq} s$ <b>in</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|              | $\{ s_h. \exists h'_0, t_h, V_0. h'_0(t_h) = (s_h, V_0) * \sqsupseteq V_0 * s \sqsubseteq_{\text{sn}}^{\gamma} h'_0 * \text{TreiberBl}(s, \Phi) * (s_h = 0 \vee s_h \neq 0 * \exists q, n. @_{V_0} s_h \xrightarrow{q} n) \}_{\top \setminus \mathcal{N}}$                                                                                                                                                                                                                                        |
|              | // AT-READ-SN-ACQ                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|              | Context: $h'_0(t_h) = (s_h, V_0) * \sqsupseteq V_0 * s \sqsubseteq_{\text{sn}}^{\gamma} h'_0$                                                                                                                                                                                                                                                                                                                                                                                                     |
|              | $\{ \text{TreiberBl}(s, \Phi) * (s_h = 0 \vee s_h \neq 0 * s_h \xrightarrow{q} n) \}_{\top \setminus \mathcal{N}} \quad // \text{VA-ELIM}$                                                                                                                                                                                                                                                                                                                                                        |
|              | $\{ s \sqsubseteq_{\text{sn}}^{\gamma} h'_0 * (s_h = 0 \vee s_h \neq 0 * s_h \xrightarrow{q} n) \}_{\top}$                                                                                                                                                                                                                                                                                                                                                                                        |
|              | 2: <b>if</b> $s_h == 0$ // null                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Empty        | $\{ s_h = 0 \}$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|              | 3: <b>then</b> 0 $\{ v.v = 0 \}_{\top}$                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|              | 4: <b>else</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|              | $\{ s_h \xrightarrow{q} n \}$                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|              | 5: <b>let</b> $n := \text{*na} s_h \{ s_h \xrightarrow{q} n \}_{\top} \quad // \text{NA-READ}$                                                                                                                                                                                                                                                                                                                                                                                                    |
|              | $\{ s_h \xrightarrow{q} n * \triangleright \text{TreiberBl}(s, \gamma) \}_{\top \setminus \mathcal{N}}$                                                                                                                                                                                                                                                                                                                                                                                           |
|              | $\{ s_h \xrightarrow{q} n * \sqsupseteq V_0 * s \sqsubseteq_{\text{sn}}^{\gamma} h'_0 * s \mapsto_{\text{con}}^{\gamma} h * @_{V_h} \text{InNodes}(S) * \text{AllNodes}(vs') * \dagger^1 s \}_{\top \setminus \mathcal{N}}$                                                                                                                                                                                                                                                                       |
|              | // Unfolding TreiberBl                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|              | We know $s_h$ is a value written to $h$ at $t_h$ , because $h'_0 \subseteq h$ . specifically, $h(t_h) = (s_h, V_0)$                                                                                                                                                                                                                                                                                                                                                                               |
|              | 6: <b>if</b> $\text{CAS}^{\text{acq}}(s, s_h, n)$                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Non-empty    | $\left\{ \begin{array}{l} b. s_h \xrightarrow{q} n * \exists h', V' \sqsupseteq V_0, v'. \sqsupseteq V' * @_{V_s \sqcup V'}(s \mapsto_{\text{con}}^{\gamma} h') * @_{V_h} \text{InNodes}(S) * \text{AllNodes}(vs') * \dagger^1 s * \\ (b = \text{false} * s_h \neq v' * h' = h) \\ \vee (b = \text{true} * s_h = v' * V_0 = V_h \sqsubseteq V' * t_h = \max(\text{dom}(h)) * \exists V_w \sqsupseteq V_h. h' = h[t_h + 1 \leftarrow (n, V_w)]) \end{array} \right\}_{\top \setminus \mathcal{N}}$ |
|              | // AT-CAS-SN-GEN Noe that any location values readable from $s$ are alive because we store their                                                                                                                                                                                                                                                                                                                                                                                                  |
|              | fractional non-atomic points-to ownership in $\text{AllNodes}(vs')$ .                                                                                                                                                                                                                                                                                                                                                                                                                             |
|              | $\left\{ \begin{array}{l} b. s_h \xrightarrow{q} n * (b = \text{false} * \text{TreiberBl}(s, \gamma)) \\ \vee (b = \text{true} * S = [(s_h, V_h)] ++ S' * @_{V_s \sqcup V'}(s \mapsto_{\text{con}}^{\gamma} h') * \\ @_{V_w} \text{InNodes}(S') * \text{AllNodes}(vs' ++ [(n, V_w)]) * \dagger^1 s * \\ @_{V_h} (\exists v, q'. \Phi(v) * s_h + 1 \mapsto v * \dagger^2 s_h * s_h \xrightarrow{q'} n) \end{array} \right\}_{\top \setminus \mathcal{N}}$                                          |
|              | $\{ b. s_h \xrightarrow{q} n * \text{TreiberBl}(s, \gamma) * (b = \text{false} \vee (b = \text{true} * \exists v, q'. \Phi(v) * s_h + 1 \mapsto v * \dagger^2 s_h * s_h \xrightarrow{q'} n)) \}_{\top \setminus \mathcal{N}}$                                                                                                                                                                                                                                                                     |
|              | // VA-ELIM since we have $\sqsupseteq V'$ and $V_h \sqsubseteq V'$                                                                                                                                                                                                                                                                                                                                                                                                                                |
|              | 7: <b>then</b> $\text{*na}(s_h + 1) \{ v. \Phi(v) * s_h + 1 \mapsto v * \dagger^2 s_h * s_h \xrightarrow{q+q'} n \}_{\top}$                                                                                                                                                                                                                                                                                                                                                                       |
|              | 8: <b>else</b> $-1 \{ v.v = -1 * s_h \xrightarrow{q} n \}_{\top}$                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|              | $\{ v.v = -1 \vee \Phi(v) \}_{\top} \quad // \text{Dropping points-to and block ownership of } s_h$                                                                                                                                                                                                                                                                                                                                                                                               |
|              | $\{ v.v = -1 \vee v = 0 \vee \Phi(v) \}_{\top}$                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

FIGURE 12.11: Hoare proof outlines for `try_pop`

than the view at which the node  $n$  was pushed to the stack, so the seen-view observation  $\sqsubseteq V_n$  should allow us to access the resources released together with the push of  $n$ , i.e.,  $(\exists v. n + 1 \mapsto v * \Phi(v) * \dots)$  in  $\text{InNodes}(S)$ .

Unfortunately, the twist is that  $n$  can be written multiple times to  $s$ , each time with an increasingly bigger view  $V_n$ . This can happen when  $n$  keeps coming back as the top of the stack, because some other nodes are pushed on top of  $n$  and then popped. Each time  $n$  comes back as the top of the stack, it is written to  $s$  with a bigger view  $V_n$ , because we only **CAS** on  $s$ . As such, when we read  $n$  from  $s$  with the acquire mode in line 1, we may have read the  $i$ -th write of  $n$  to  $s$  and acquire some observation of  $V_n^i$ , i.e.,  $\sqsubseteq V_n^i$ . Meanwhile, the resources  $(\exists v. n + 1 \mapsto v * \Phi(v) * \dots)$  we want to acquire (when the **CAS** on  $s$  line 6 succeeds) hold at the view  $V_n^0$  of the very *first* write of  $n$  to  $s$ —the release write of the push of  $n$  to  $s$ . That is, we should have  $@_{V_n^0}(\exists v. n + 1 \mapsto v * \Phi(v) * \dots)$  in the invariant. So if we want to acquire those resources with a fully relaxed **CAS** in line 6, we need to track the fact that  $V_n^0 \sqsubseteq V_n^i$  for *all*  $i$ . With that, we can use  $\sqsubseteq V_n^i$  we acquired in line 1 to eliminate the view-at modality and acquire  $(\exists v. n + 1 \mapsto v * \Phi(v) * \dots)$ . In short, to support the fully relaxed **CAS** in line 6 of **try\_pop**, we need to adjust our invariant so that (1) for each node  $n$ , we keep the  $\text{InNodes}$  resources at the view  $V_n^0$  of the first write of  $n$  to  $s$  (the write of the push of  $n$  to  $s$ ), and (2) we know that any later write of  $n$  to  $s$  has a view  $V_n^i$  that is bigger than  $V_n^0$ .

The invariant  $\text{TreiberBI}(s, \gamma)$  (Definition 12.6) on the other hand is rather simple: we keep all  $\text{InNodes}$  resources at the view  $V_h$  of the latest write to  $s$ . As such, we need an acquire **CAS** to synchronize with that view  $V_h$  in order to acquire the resources of the top node in  $\text{InNodes}$ . To keep the proof simple, we have decided to present this invariant for the **try\_pop** version with the acquire **CAS** (Figure 12.8).

**CHAPTER SUMMARY.** In this chapter we have demonstrated multiple features of iRC11, using several examples. We show how to switch from non-atomic to atomic points-to to share locations, how to build concurrent protocols using atomic and non-atomic points-to, how to perform atomic accesses with resources under view-explicit modalities, how to cancel invariants and eliminate the view-explicit modalities to regain shared resources, and how to convert atomic points-to back to non-atomic ones to deallocate or reuse them.

We have also seen that working with explicit views and histories are quite cumbersome, and in Part III we will show how to abstract them further with GPS protocols, and we will see more complex verifications of concurrent Rust libraries against their assigned Rust types in  $\text{RB}_{\text{rlx}}$ . Nevertheless, the GPS abstraction in Part III is not so convenient to work with when we have multiple atomic locations, and hiding views indeed weakens the logic. Furthermore, GPS protocols would not be sufficient to verify the **try\_pop** version that uses a fully relaxed **CAS** (see Remark 12.8). In Part IV we will need explicit views and general invariants that can store and relate multiple atomic locations, in order to verify libraries against stronger logically-atomic specifications of Compass.





# 13

## Related Work

---

He et al.<sup>1</sup> present GPS+, a extension to GPS that supports relaxed accesses and release/acquire fences. The logic, however, lacks support for relaxed RMW operations *e.g.*, CAS/FAA operations. Additionally, their logic is based on the axiomatic semantics of C11 which cannot be used together with Iris and thus would not allow us to extend the existing Rust-Belt development. GPS+ also does not support mechanized verification of programs.

iGPS<sup>2</sup> supports a mechanism called *fractional protocols*, which is closely related to cancelable invariants. However, iGPS's fractional protocols are not as powerful as iRC11's cancelable invariants in that they cannot reclaim the resources governed by the protocol at the thread's local view. This is because the protocol tokens they use are modeled with unsynchronized ghost state. By using synchronized ghost state, we can support full reclamation of all resources governed by either iRC11 cancelable invariants or borrow propositions.

Tassarotti et al.<sup>3</sup> use GPS to verify an implementation of the Read-Copy-Update (RCU) technique. With GPS, they are able verify the reclamation of clients' non-atomic locations, but not RCU's internal atomic locations because they are governed by GPS's *non-cancelable* protocols. Kaiser et al. fixed this problem by re-verifying RCU in iGPS with their fractional protocols. However, as mentioned above, fractional protocols are not a general solution.

FSL and FSL+<sup>4</sup> are the first RMC logics to support fences. iRC11 directly inherits the idea of fence modalities from them, but provide a model for the modalities on top of the operational semantics of ORC11,<sup>5</sup> whereas FSL and FSL++ are proven directly on top of the axiomatic semantics of (R)C11. iRC11 also provides better support for mechanized verification of programs, thanks to the Iris proofmode.

Gotsman et al.<sup>6</sup> provide an SC-based logic where ownership of a location can be turned into a lock with fractional permissions for shared accesses, and later when the full permission of the lock is collected, the ownership of the location can be reclaimed. Hobor et al.<sup>7</sup> provide a similar mechanism that additionally allows attaching “invariant resources” to locks. Our cancelable invariants are more general than these mechanisms in that our cancelable invariants are not specifically tied to locks and are proven sound with respect to a much weaker memory model.

Svendsen et al.<sup>8</sup> introduce the first program logic for the promising

<sup>1</sup>He et al., “GPS+: Reasoning About Fences and Relaxed Atomics” [He+18].

<sup>2</sup>Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

<sup>3</sup>Tassarotti et al., “Verifying read-copy-update in a logic for weak memory” [TDV15].

<sup>4</sup>Doko and Vafeiadis, “A Program Logic for C11 Memory Fences” [DV16]; Doko and Vafeiadis, “Tackling Real-Life Relaxed Concurrency with FSL++” [DV17].

<sup>5</sup>see Section 8.3

<sup>6</sup>Gotsman et al., “Local Reasoning for Storable Locks and Threads” [Got+07].

<sup>7</sup>Hobor et al., “Oracle Semantics for Concurrent Separation Logic” [HAN08].

<sup>8</sup>Svendsen et al., “A Separation Logic for a Promising Semantics” [Sve+18].

semantics. Their logic is based on RSL and supports relaxed accesses but not fences. Moreover, unlike FSL, it disallows the transfer of ownership through relaxed accesses, among other reasoning principles that have proven useful in  $\text{RB}_{\text{rlx}}$ . Extending iRC11 to account for promises is a very interesting avenue for future work.

It is worth re-iterating that ORC11 and iRC11 currently do not support SC accesses and SC fences. In fact, we are not aware of any existing RMC separation logic that does.<sup>9</sup> Adding support for SC would enable us to verify some interesting and challenging fine-grained concurrent algorithms, such as the work-stealing queue,<sup>10</sup> as well as epoch-based resource reclamation schemes such as that implemented by Rust’s `crossbeam` library.<sup>11</sup>

Very recently, Hammond et al.<sup>12</sup> introduces the first CSL for the ARMv8 “user” concurrency memory model,<sup>13</sup> named AxSL. AxSL provides rules similar to those of RSL, where resources can be transferred among threads following a per-location protocol  $\Phi$ : a read of value  $v$  from the location  $\ell$  should acquire the resource  $\Phi(\ell, v)$ , and a write of  $v$  to  $\ell$  should release the resource  $\Phi(\ell, v)$ . However, unlike RSL which does not support ghost state as a resource and only allows resource transfer over release-acquire synchronization, AxSL is built in Iris (so resources can include arbitrary ghost state) and allows resource transfer over weaker accesses, such as relaxed ones, although not arbitrary. To be sound with respect to the ARMv8 RMC memory model, AxSL does not allow resources to flow along  $\text{po} \cup \text{rf}$  which can be cyclic. Recall that all RSL, FSL,  $\text{FSL}++$ , GPS, iGPS, iRC11 assume  $\text{po} \cup \text{rf}$  to be acyclic, and, as a result, resources in these logic can flow implicitly along  $\text{po} \cup \text{rf}$ , and we can get simple reasoning principles following program syntax. On the other hand, in the ARMv8 memory model, intra-thread concurrency can happen, as independent instructions can be executed and speculated concurrently. Consequently, while AxSL’s reasoning principles still follow program syntax, the logic maintains that resources can only flow along dependencies that prevent intra-thread concurrent execution of instructions. Concretely, the logic supports explicit reasoning about memory events generated by the program and dependencies among them (edges among events in candidate execution graphs), and only allows resources to flow along the memory model’s *ordered-before* relation  $\text{ob}$  (which includes the intra-thread locally-ordered-before relation  $\text{lob}$  and the inter-thread observed-by relation  $\text{obs}$ ). The restriction is enforced by tying resources to events, e.g.,  $a \vdash \Phi(\ell, v)$ , and by allowing an event  $b$  to acquire  $\Phi(\ell, v)$  only if  $a$  is ordered before  $b$ . This construction is similar to FSL modalities, or our view-explicit modalities (§8.5), but at a finer-grained level of events.

Another impressive achievement of AxSL is that it is proven by instantiating Iris with a simple operationalisation of the axiomatic semantics of [Pul+18]. The operationalised semantics is very close to the original axiomatic one. We have conjectured that we can do the same with OGS (§3.6), but we have not put in the effort to do so. The benefit of this approach is that we can cut down the trusted code base (TCB), and avoid doing a correspondence proof as in §3.6. On the other hand, the model of predicates, such as that of the view-explicit ones or of weakest

<sup>9</sup>The abstract of the RSL paper ([VN13]) claims that it supports reasoning about SC accesses, but according to Vafeiadis [personal communication], this is a mistake, and indeed the body of the paper does not.

<sup>10</sup>Chase and Lev, “Dynamic circular work-stealing deque” [CL05].

<sup>11</sup>*Crossbeam: Tools for concurrent programming in Rust* [Www].

<sup>12</sup>Hammond et al., “An Axiomatic Basis for Computer Programming on the Relaxed Arm-A Architecture: The AxSL Logic” [Ham+24].

<sup>13</sup>Pulte et al., “Simplifying ARM concurrency: multicopy-atomic axiomatic and operational models for ARMv8” [Pul+18].

preconditions, would become more complex. In fact, the authors of AxSL report a non-standard and challenging model and adequacy proof for their logic, which expectedly should be more complex than those done by Vafeiadis et al. for RSL and FSL.



### Part III

## RUSTBELT MEETS RELAXED MEMORY



# 14

## Challenge: RustBelt and Relaxed Memory

---

The Rust programming languages<sup>1</sup> strike a delicate balance between safety and control using a *substructural* type system, in which types not only classify data but also represent *ownership* of resources, such as the right to read, write, or deallocate a piece of memory. By tracking ownership in the types, Rust is able to prohibit dangerous combinations of mutation and aliasing, a well-known source of programming pitfalls and security vulnerabilities in both C/C++ and Java. And yet, the type system is expressive enough to type-check many common systems programming idioms. Nonetheless, certain kinds of functionality (e.g., some pointer-based data structures, synchronization abstractions, garbage collection mechanisms) cannot be implemented within the strictures of Rust’s type system. Rust provides these abstractions instead via *libraries* whose implementations internally utilize *unsafe features* (e.g., unchecked type casts, array accesses without bounds checks, or accesses of “raw” pointers whose aliasing is untracked by the type system). These libraries are *claimed* to be safe extensions to Rust because they encapsulate their uses of unsafe features in “safe APIs”. However, given that the set of such extensions is far from fixed—new and surprising “safe APIs” are being developed all the time—there is a pressing need to understand what property an internally-unsafe library ought to satisfy to be deemed a safe extension to Rust.

To formalize Rust’s “extensible” notion of safety, RustBelt<sup>2</sup> follows prior work on Foundational Proof-Carrying Code<sup>3</sup> by employing a *semantic soundness* proof. First, it defines a *semantic model* of Rust types: a mapping from types  $\mathsf{T}$  to logical predicates on terms  $\Phi(e)$ , which asserts what it means for the term  $e$  to *behave* safely at type  $\mathsf{T}$  (even if internally  $e$  uses unsafe features). Then, the RustBelt proof breaks into two main parts:

1. *Safety of libraries that use unsafe features*: For any library that makes use of unsafe features, the implementation of the library is proven to satisfy the semantic model of its API, thus establishing that it is safe for clients to make use of the library. RustBelt proved safety for a number of widely-used Rust libraries, including `Arc`, `Rc`, `Cell`, `RefCell`, `Mutex`, and `RwLock`.
2. *Safety of the  $\lambda_{\text{Rust}}$  type system*: The syntactic typing rules of  $\lambda_{\text{Rust}}$

<sup>1</sup>Klabnik and Nichols, *The Rust Programming Language* [KN18].

<sup>2</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [Jun+18a].

<sup>3</sup>Ahmed et al., “Semantic foundations for typed assembly languages” [Ahm+10].

are proven to respect the semantic model, thus establishing that code written in the “safe” fragment of Rust is in fact observably safe—*i.e.*, its behavior is well-defined.

Put together, these imply that if a program  $\mathcal{P}$  is well-typed, and its only uses of unsafe features appear within the libraries that have been verified safe (in part 1), then  $\mathcal{P}$  is observably safe.

In carrying out their semantic soundness proof for RustBelt, Jung et al. relied on the higher-order concurrent separation logic framework Iris.<sup>4</sup> Separation logic is a good fit for modeling Rust because it is designed around the same notion of *ownership* as Rust’s type system, and thus provides built-in support for ownership-based reasoning. One benefit of using Iris is that it was designed to support the derivation of new separation logics with domain-specific reasoning principles. Jung et al. exploited this facility to derive a new logic called the *lifetime logic*, which they used extensively in their proofs in order to reason about Rust’s “lifetimes” and “borrowing” mechanisms at a higher level of abstraction.<sup>5</sup> A second benefit of using Iris is that it comes with tactical support for developing *machine-checked* proofs interactively in Coq;<sup>6</sup> this support made it possible for RustBelt to be fully mechanized in Coq.

**RUSTBELT FOR RELAXED MEMORY.** In the original RustBelt work, Iris was instantiated with a sequentially consistent (SC) semantics for  $\lambda_{\text{Rust}}$ . This SC instantiation of Iris (call it “Iris-SC”) provides a variety of proof rules that are valid only under SC semantics and not under relaxed-memory semantics. To adapt RustBelt to relaxed memory, we would like to “port” RustBelt so that it is built on top of iRC11, which is sound for the  $\lambda_{\text{Rust}} + \text{ORC11}$  semantics, rather than Iris-SC. Following the structure of RustBelt, this porting effort breaks down into two major tasks:

**Task 1:** Re-prove the safety of the Rust libraries considered by RustBelt, this time verifying their real, relaxed-memory implementations in iRC11.

**Task 2:** Re-prove the safety of the  $\lambda_{\text{Rust}}$  type system, this time relying only on proof rules that are sound in iRC11.

**KEY CHALLENGE.** As it turns out, both of these tasks require us to overcome a technical challenge that is relevant not just to Rust but to relaxed-memory verification in general: namely, that **existing previous work on separation logic does not provide an adequate foundation for reasoning about resource reclamation under relaxed memory**. We will first explain this challenge in the context of Task 1, before briefly describing how it also informs Task 2.

## 14.1 Task 1: Re-prove the Safety of Rust Libraries under RMC

One of the main motivations for using a “systems programming” language like Rust or C/C++ (as opposed to a garbage-collected language like Java) is to have more precise control over limited resources such as memory. In particular, the Rust programmer can be assured that when an

<sup>4</sup>Jung et al., “Iris from the ground up: A modular foundation for higher-order concurrent separation logic” [Jun+18b].

<sup>5</sup>Klabnik and Nichols, *The Rust Programming Language* [KN18], §4.2, §10.3.

<sup>6</sup>Krebbers et al., “Interactive Proofs in Higher-Order Concurrent Separation Logic” [KTB17]; Krebbers et al., “MoSeL: A General, Extensible Modal Framework for Interactive Proofs in Separation Logic” [Kre+18].



$$\begin{array}{c}
\text{SC-CINV-ACC} \\
\frac{\mathcal{N} \subseteq \mathcal{E}}{\boxed{I}^{\gamma, \mathcal{N}} * \heartsuit_q^\gamma \vdash \varepsilon \Vdash^{\mathcal{E} \setminus \mathcal{N}} \left( \triangleright I * \left( \triangleright I \varepsilon \setminus \mathcal{N} \multimap^{\mathcal{E}} \heartsuit_q^\gamma \right) \right)}
\end{array}
\qquad
\begin{array}{c}
\text{SC-CINV-TOK} \\
\heartsuit_{q+q'}^\gamma \dashv\vdash \heartsuit_q^\gamma * \heartsuit_{q'}^\gamma
\end{array}
\qquad
\begin{array}{c}
\text{SC-CINV-CANCEL} \\
\boxed{I}^{\gamma, \mathcal{N}} * \heartsuit_1^\gamma \vdash \varepsilon \Vdash_{\mathcal{E}} \triangleright I
\end{array}$$

FIGURE 14.1: Key rules for cancelable invariants in Iris-SC

object goes out of scope, the destructor (**drop** method) associated with its type will be invoked and any resources it owns will be reclaimed. Yet the safety of destructors is often quite subtle because objects can contain references to resources that are shared with other objects. For example, objects of type `Arc<T>` are simply aliases to a shared **struct** containing an object of type `T` along with a *reference counter*, which keeps track of the current number of active aliases to the object. Consequently, the destructor for `Arc<T>` cannot simply reclaim the shared **struct** that it points to: rather, it decrements the shared reference counter, and only if it observes that it was the last remaining alias can it safely reclaim the memory for the reference counter and invoke the destructor for the object of type `T`.

RustBelt showed how to put this subtle kind of resource reclamation on a sound formal footing using Iris-SC’s mechanism of *cancelable invariants* (Figure 14.1), a generalization of Gotsman et al.<sup>7</sup> and Hobor et al.<sup>8</sup>’s “storable locks”. A cancelable invariant  $\boxed{I}^{\gamma, \mathcal{N}}$  is an invariant governing a shared resource  $I$  which is only “active” for a certain period of time, after which point it is “cancelled”. To access the shared resource during an atomic step of computation (**SC-CINV-ACC**), a thread must prove that the invariant is still active by exhibiting ownership of a fractional *invariant token*  $\heartsuit_q^\gamma$ , where  $q$  is a fraction in  $(0, 1]$ . If a thread  $\pi$  can assert ownership of  $\heartsuit_1^\gamma$  (i.e., the “full”  $\gamma$  token), it knows that no other thread can assert that the invariant is active; thus it is safe for  $\pi$  to cancel the invariant and reclaim full ownership of  $I$  (**SC-CINV-CANCEL**), after which it can free the memory governed by  $I$  if it wants to. In RustBelt, cancelable invariants played a crucial role in verifying the safety of destructors such as `Arc`’s.

However, adapting cancelable invariants to the relaxed-memory setting turns out to be quite tricky—tricky enough that no existing relaxed-memory separation logic supports them.<sup>9</sup> The main problem arises in how to model the cancelable invariant *tokens*. Under SC, one can simply model invariant tokens as a form of ghost state. But in existing relaxed-memory separation logics, ghost state is *view-agnostic*, meaning that ownership of it can be transferred between threads without the need for any physical synchronization. On the one hand (see §19), view-agnostic ghost state is indispensable for representing *globally consistent* state, such as (in the case of `Arc`) the number of `Arc` aliases currently in existence. On the other hand, if invariant tokens are modeled naively as view-agnostic ghost state, the logic of cancelable invariants becomes unsound! In particular, the access rule **SC-CINV-ACC** is not sound in relaxed memory.

Our solution is to instead model invariant tokens using a novel notion of *synchronized ghost state*: ghost state that implicitly tracks the

<sup>7</sup>Gotsman et al., “Local Reasoning for Storable Locks and Threads” [Got+07].

<sup>8</sup>Hobor et al., “Oracle Semantics for Concurrent Separation Logic” [HAN08].

<sup>9</sup>iGPS supports a related notion of “fractional protocol”, but it is not nearly as powerful as cancelable invariants and is thus not general enough to account for resource reclamation in Rust.

subjective view of the thread that owns it, and that therefore can only be transferred between threads using physical synchronization. **Using synchronized ghost state, iRC11 offers the first general account of resource reclamation in relaxed-memory separation logic.** The resulting interface and model of iRC11 cancelable invariants have been provided in §11.2. In the subsequent chapters in this part, we will demonstrate its effectiveness on a number of real Rust libraries.

## 14.2 Task 2: Re-prove the Safety of the $\lambda_{\text{Rust}}$ Type System under RMC

In contrast to RustBelt’s proofs of safety for libraries, its proof of safety for the  $\lambda_{\text{Rust}}$  type system did not rely directly on cancelable invariants or any other SC-specific features of Iris-SC. Rather, as mentioned above, the safety proof for the type system made essential use of a Rust-oriented logic called the *lifetime logic*, which was a domain-specific logic derived within Iris-SC. Thus, if we are able to show that the lifetime logic remains sound under relaxed memory—by instead deriving its soundness in iRC11—then  $\text{RB}_{\text{rlx}}$  can inherit RustBelt’s safety proof for the  $\lambda_{\text{Rust}}$  type system without modification!

Synchronized ghost state is the key to making this modular porting strategy possible. Specifically, the lifetime logic is centered around a mechanism called *borrow propositions*, describing resources that are borrowed for the duration of a Rust “lifetime” and that can be reclaimed once the lifetime is over. Borrow propositions are similar in many ways to cancelable invariants, but also more flexible and more complex in terms of the protocols they support for sharing and reclamation of resources. Just as synchronized ghost state enables us to adapt cancelable invariants to relaxed memory, it plays an analogously central role in adapting borrow propositions to relaxed memory as well.

## 14.3 Contributions of RustBelt Relaxed

RustBelt Relaxed, or  $\text{RB}_{\text{rlx}}$  for short, is an adaptation of RustBelt to ORC11 and, like its predecessor, is fully mechanized in Coq. We use iRC11 both to re-verify the standard libraries that internally use unsafe features, and to re-prove the soundness of RustBelt’s lifetime logic. The safety proof of  $\lambda_{\text{Rust}}$ ’s type system, by virtue of being built atop the lifetime logic, did not need to be changed at all.

$\text{RB}_{\text{rlx}}$  has ported all verifications done in RustBelt, including the following concurrency libraries: `thread::spawn`, `rayon::join`,<sup>10</sup> `Mutex`, `RwLock`, and `Arc`. For the most part, we were able to verify Rust’s uses of relaxed-memory operations in these concurrent libraries as is. Only in the implementation of `Arc` did we need to strengthen the consistency level of two memory reads (from relaxed to acquire) in order to make our verification go through. And in one of these cases, our attempt to verify the original (more relaxed) access led us to expose it as the source of a previously undetected data race in the library. Our fix for this race has

<sup>10</sup>whose verification we have seen in §12.3.

since been merged into the Rust codebase.<sup>11</sup> Meanwhile, the verifications of sequential libraries—`Rc`, `Cell`, `RefCell`—remain largely unchanged from RustBelt.

The structure of the remaining chapters in this part is as follows. Please also refer to [Figure 1.1](#) for their dependency graph. [Chapter 15](#) briefly reviews the *lifetime logic*, the core abstraction needed for the original RustBelt’s soundness proof the Rust’s type system, and [Chapter 16](#) discusses, at a high level, the changes to the interfaces as well as the model of the lifetime logic, so as to be sound on top of iRC11. [Chapter 17](#) discusses how to construct *GPS single-location protocols*<sup>12</sup> using atomic points-to (§10), and how to combine them with cancelable invariants (§11.2) to build *cancelable* GPS protocols. [Chapter 18](#) presents how to combine cancelable GPS protocols with the lifetime logic to verify safety of Rust’s `RwLock` library. [Chapter 19](#) presents how to use iRC11 cancelable GPS protocols to verify safety of Rust’s `Arc` library. It also discusses a bit of history of how the bug in `Arc` was found.

<sup>11</sup>Jourdan, *Insufficient synchronization in `Arc::get_mut`* [[Jou18](#)].

<sup>12</sup>Turon et al., “GPS: navigating weak memory with ghosts, protocols, and separation” [[TVD14](#)]; Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [[Kai+17](#)].



# 15

## *The Lifetime Logic of SC RustBelt*

---

Working with separation logics, one may have become used to being able to transfer ownership of resources from one piece of a program (e.g., one thread) to another. In [Chapter 12](#), we have seen this in action, in relaxed memory even, where ownership of resources are transferred through synchronization between threads, either by message-passing, or by joining, or by matching push and pop operations of a stack. These are all examples of what we call the traditional *direct* style of ownership transfer, where it is clear what is being transferred when.

Although direct ownership transfer is fairly simple, it is unfortunately not sufficient to explain a key feature of the Rust language, namely its *borrowing* mechanism. In this chapter, we review what borrowing is, and why direct ownership transfer does not easily account for it (§15.1). We will then review in §15.2 how RustBelt’s original lifetime logic (in SC) comes to the rescue. In [Chapter 16](#), we will discuss how to port the lifetime logic to relaxed memory.

### 15.1 Borrowing in Rust

The central tenet of Rust is that the most insidious source of safety vulnerabilities in systems programming is the unrestricted combination of mutation and aliasing—when one part of a program mutates some state in such a way that it corrupts the view of other parts of the program that have aliases to (i.e., references to) that state. Consequently, Rust’s type system enforces the discipline of *aliasing XOR mutability* (AXM, for short): a value of type  $T$  may *either* have multiple aliases (called *shared references*), of type  $\&T$ , or it may be mutated via a unique, *mutable reference*, of type  $\&\text{mut } T$ , but it may not be both aliased and mutable at the same time.

To create a mutable reference to an object  $o : T$  in Rust, one *borrow*s  $o$  for the duration of some *lifetime*  $'a$ , with the result being a reference value  $r$  of type  $\&a \text{ mut } T$ . Borrowing causes the ownership of  $o$  to be *split in time*: while the lifetime  $'a$  is alive, the borrower controls the object and can use  $r$  to mutate it; but once  $'a$  is dead, the original owner of  $o$  can reclaim ownership of it. The reclamation that occurs once the lifetime  $'a$  is over is essentially a form of ownership transfer from the borrower to the original owner of  $o$ . And the natural question that arises when proving the safety of Rust is: how do we know that this reclamation

is sound?

One might think that there is an obvious way of modeling this reclamation using direct ownership transfer: when the lifetime `'a` is over, the borrower just needs to hand ownership of the borrowed reference back to the original owner. Unfortunately, it is not always that straightforward.

**Example 15.1** (Indirect Reclamation of Resources). Consider the following example taken from the RustBelt paper.<sup>1</sup> The example makes use of the function `index_mut` from the `Vec` library.

<sup>1</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [Jun+18a].

```

1 let mut v = vec![21, 57];
2 { let mut head = v.index_mut(0);
3   // head : &'a mut i32
4   // Cannot access v: v.push(42) rejected
5   *head = 23; }           Lifetime 'a
6 v.push(42);

```

In this example, we start with a vector `r` of two elements 21 and 57. Then, in lines 2-5, using `index_mut`, we get a mutable deep pointer `head` into the head element of `r` and update its contents to 23. Such a deep aliasing into the vector’s internal is dangerous, because if the pointer outlives the internal storage, the pointer will be a dangling one and dereferencing it is undefined behavior. This is why Rust will reject `v.push(42)` in line 6, as `push` may reallocate the internal storage.

But what does this have to do with indirect ownership transfer? What is happening is that the code block in lines 2-5 *borrow*s the whole vector `r` from its parent block. Borrowing the whole vector is due to `index_mut`, whose type is

```
fn(&'a mut Vec<i32>, usize) -> &'a mut i32
```

This function takes a mutable reference `r` to an integer vector, along with an index `n`, and returns an interior mutable reference `e` to the `n`-th element of the vector. Crucially, thanks to Rust’s substructural type system, the caller of this function *gives up* ownership of the argument `r` in exchange for the result `e`. The surrendering of `r` is quite important here because otherwise `r` could be used to subsequently mutate the object in a way that would invalidate the interior pointer `e`. On the other hand, the lifetime `'a` ensures that `e` can only be accessed during the lifetime at which the original `r` was accessible.

In our example, `index_mut` borrows the whole vector `v` and returns a single mutable reference into the first index as `head`, restricted to the duration of the lifetime `'a`. Firstly, the inner block does not need to care about transferring the vector `v` back to the parent block once it is done. The inner block simply knows that it will finish using `v` before the lifetime `'a` ends—the moment after which the vector will be given back to the parent block indirectly.

Secondly, the inner block effectively declares that it only cares about one index in the vector and only wants to borrow that single index. But the life of the index is tied to the life of the whole vector, therefore, to prevent dangerous aliasing, it is necessary to borrow the whole vector with the same lifetime `'a`. In other words, even when the borrower

forgets about the borrow `&'a mut Vec<i32>`, it is still indirectly tied to the borrow `&'a mut i32`, which, in our case, is the `head` pointer. When the lifetime `'a` ends, the ownership of not just the `head` index but the whole vector will be indirectly transferred back to the parent block. Regaining ownership of the whole vector, the parent block can `push` again in line 6.

In summary, in addition to direct ownership transfer, Rust also employs an indirect transfer scheme which (1) uses lifetimes to indirectly specify *when* the return transfer happens: all borrowed resources will be returned at the end of the lifetime which does not need to be directly agreed on up front; and (2) uses borrows and their relations to indirectly specify *what* needs to be returned without bookkeeping every bit of borrowed resources. Translating this indirect transfer scheme to separation logics is no easy task, but have been accomplished by Jung et al. [Jun+18a] with the *lifetime logic*.

## 15.2 The Lifetime Logic Primer, in SC

At a high level, the idea of the lifetime logic is to formalize the intuition mentioned above: borrowing an object `o` for a lifetime `'a` splits ownership of `o` in time, between a “borrow” assertion, which the borrower can use to access `o` while `'a` is alive, and an “inheritance” assertion, which the original owner can use to reclaim ownership of `o` once `'a` is dead. Although “splitting ownership in time” is not a standard notion in separation logic, the Iris framework is designed to enable one to derive such non-standard notions of separation and embed them in the separating conjunction connective, and that is precisely what Jung et al. did.

The lifetime logic introduces several *abstract predicates* representing a variety of capabilities and permissions related to lifetimes and borrowing, together with axioms (proven sound in Iris-SC) for manipulating them, as shown in Figure 15.1. Let us begin with an overview of the new predicates:

- The *full borrow*  $\&_{\text{full}}^{\kappa} P$  asserts *temporary* ownership of resource  $P$ , while the lifetime  $\kappa$  is alive. It provides a direct means of modeling the semantics of Rust’s mutable reference types.
- The timeless *lifetime token*  $[\kappa]_q$  serves as a witness that the lifetime  $\kappa$  is still alive. Here,  $q$  is a fraction in  $(0, 1]$ . If  $q = 1$ , we say that this is the *full token* for  $\kappa$ . The use of fractions allows one to share the knowledge that a lifetime is alive with multiple parties.
- The *killer permission*  $\text{Kill}(\kappa)$  is a unique permission needed to kill the lifetime  $\kappa$ .
- The timeless and persistent *dead token*  $[\dagger\kappa]$  is used to witness the knowledge that lifetime  $\kappa$  is dead.
- The *inheritance*  $\text{Inh}(\kappa, P)$  asserts the right to reclaim the ownership of borrowed resource  $P$  once  $\kappa$  is dead.

|                                                                                                                                |                                                                                                                                              |                                                                                                            |                                                           |
|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| timeless( $[\kappa]_q$ )                                                                                                       | LFTL-BEGIN                                                                                                                                   | LFTL-KILL                                                                                                  | LFTL-TOK-FRAC                                             |
| timeless( $[\dagger\kappa]$ )                                                                                                  | $\text{True} \Rightarrow_{\mathcal{N}_{\text{lift}}} \exists \kappa. [\kappa]_1 * \text{Kill}(\kappa)$                                       | $[\kappa]_1 * \text{Kill}(\kappa) \not\Rightarrow_{\mathcal{N}_{\text{lift}}}^{\emptyset} [\dagger\kappa]$ | $[\kappa]_{q+q'} \dashv\vdash [\kappa]_q * [\kappa]_{q'}$ |
| persistent( $[\dagger\kappa]$ )                                                                                                |                                                                                                                                              |                                                                                                            |                                                           |
| LFTL-TOK-NOT-DEAD                                                                                                              | LFTL-FULL-BOR                                                                                                                                | LFTL-FULL-INH                                                                                              |                                                           |
| $[\kappa]_q * [\dagger\kappa] \vdash \text{False}$                                                                             | $\triangleright P \Rightarrow_{\mathcal{N}_{\text{lift}}} \&_{\text{full}}^{\kappa} P * \text{Inh}(\kappa, P)$                               | $[\dagger\kappa] * \text{Inh}(\kappa, P) \Rightarrow_{\mathcal{N}_{\text{lift}}} \triangleright P$         |                                                           |
| LFTL-FULL-ACC                                                                                                                  | LFTL-FULL-RET                                                                                                                                |                                                                                                            |                                                           |
| $\&_{\text{full}}^{\kappa} P * [\kappa]_q \Rightarrow_{\mathcal{N}_{\text{lift}}} \triangleright P * \text{Ret}(\kappa, P, q)$ | $\triangleright P * \text{Ret}(\kappa, P, q) \Rightarrow_{\mathcal{N}_{\text{lift}}} \&_{\text{full}}^{\kappa} P * [\kappa]_q$               |                                                                                                            |                                                           |
|                                                                                                                                | LFTL-FULL-SEP                                                                                                                                |                                                                                                            |                                                           |
|                                                                                                                                | $\&_{\text{full}}^{\kappa}(P * Q) \Longleftrightarrow_{\mathcal{N}_{\text{lift}}} \&_{\text{full}}^{\kappa} P * \&_{\text{full}}^{\kappa} Q$ |                                                                                                            |                                                           |

FIGURE 15.1: Selected rules of SC Rust-Belt’s lifetime logic

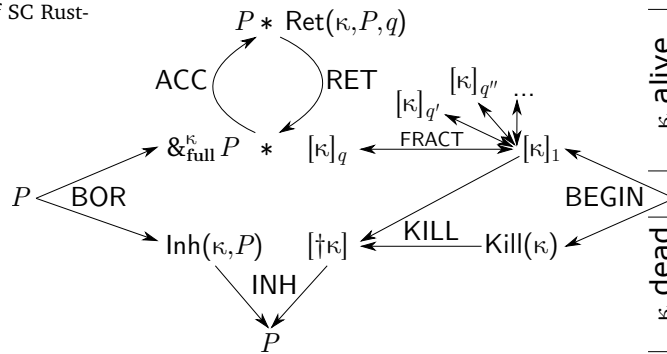


FIGURE 15.2: The life cycle of borrows and lifetimes

- The *return policy*  $\text{Ret}(\kappa, P, q)$  is used as part of the protocol for accessing the contents of a full borrow.

We briefly explain the rules in [Figure 15.1](#) with the help of [Figure 15.2](#), which depicts the life cycle of a lifetime and a full borrow. We start from the right of [Figure 15.2](#), where we create a new lifetime using `LFTL-BEGIN` (`BEGIN` in [Figure 15.2](#)). This yields the full token  $[\kappa]_1$  for a new lifetime  $\kappa$ , as well as the corresponding Kill( $\kappa$ ) permission. Lifetime tokens are *fractional* (`LFTL-TOK-FRAC`, `FRACT`), so that they can be split into (and joined back from) smaller pieces which enable multiple threads to simultaneously witness that  $\kappa$  is still alive.

Next, on the left of [Figure 15.2](#), we see the “flagship” rule of the lifetime logic: given ownership of any assertion  $P$ , and any lifetime  $\kappa$ , we can use the *borrowing rule* **LFTL-FULL-BOR** (BOR in [Figure 15.2](#)) to create a borrow of  $P$  for  $\kappa$ . The rule splits ownership of  $P$  in time between two separately ownable assertions: (1) a full borrow  $\&_{\text{full}}^{\kappa} P$  that represents ownership of  $P$  while  $\kappa$  is alive; and (2) an inheritance  $\text{Inh}(\kappa, P)$  that can be used to reclaim  $P$  after  $\kappa$  dies. Intuitively, this rule directly models what happens when an object is borrowed in Rust, with the full borrow then being given to the borrower and the inheritance given to the object’s original owner.

A thread owning *both* the full borrow  $\&\kappa_{\text{full}} P$  and a token  $[\kappa]_q$  (proving  $\kappa$  is alive) can trade them to obtain  $P$  using the *accessing rule* **LFTL-FULL-ACC** (ACC in Figure 15.2). As part of the trade, the thread is also given the return policy  $\text{Ret}(\kappa, P, q)$ . Once the thread is done using  $P$ , it trades



$P$  and  $\text{Ret}(\kappa, P, q)$  to get back  $\&_{\text{full}}^{\kappa} P$  and  $[\kappa]_q$  (**LFTL-FULL-RET**, **RET** in Figure 15.2).

Once all accesses to borrows at lifetime  $\kappa$  are done, we can recollect the full token  $[\kappa]_1$  and use the killer permission  $\text{Kill}(\kappa)$  with **LFTL-KILL** (**KILL**) to end the lifetime. This yields the dead token  $[\dagger\kappa]$ . Since  $\kappa$  is now dead, the content  $P$  in  $\&_{\text{full}}^{\kappa} P$  cannot be accessed any more and can thus be reclaimed. Anyone owning  $[\dagger\kappa]$  and the inheritance  $\text{Inh}(\kappa, P)$  can use **LFTL-FULL-INH** (**INH** in Figure 15.2) to reclaim  $P$ . **LFTL-TOK-NOT-DEAD** says that a fraction  $[\kappa]_q$  of the lifetime token indeed proves that the lifetime is alive: it is disjoint from the dead token  $[\dagger\kappa]$ .

Note that **LFTL-KILL** uses a wand step viewshift (**Notation 7.8**), i.e., its user needs an actually step to discharge the later in the viewshift. As such, we use **LFTL-KILL** around a so-called ghost instruction **endlft** (**Notation 4.3**, Figure 4.2). In fact,  $\lambda_{\text{Rust}}$  code will be populated with ghost instructions **newlft** and **endlft** to mark the beginning and the end of some lifetime. One can image that these ghost instructions are inserted by Rust's type inference, or explicitly by programmers.

In short, borrows are tied to lifetimes' life cycles. After the lifetime ended, the inheritor (one who owns the inheritance) can reclaim the borrowed resources. Note that the inheritance does not need to be used immediately after the lifetime dies. The inheritor may only receive the dead token a long time after the lifetime dies, and even then it does not need to inherit immediately. This supports a part of Rust's indirect transfer: participants do not need to agree up front on when the return transfer happens.

Although not depicted in Figure 15.2, another crucial rule of the lifetime logic is **LFTL-FULL-SEP**, which lets one go back and forth between a borrow of  $P * Q$  and separate borrows of  $P$  and  $Q$ . This rule is essential in verifying the soundness of Rust functions like **index\_mut** (§15.1) that split a reference to an object into references to its sub-components.

**Example 15.2** (MP in the Lifetime Logic). Let us now quickly demonstrate how the lifetime logic can support a somewhat different verification of the MP example, albeit with the SC semantics, in Figure 15.3. Here, instead of transferring the location  $\ell_x$  from thread 1 to thread 2 directly, we transfer a lifetime token, which thread 2 then uses to reclaim ownership of  $\ell_x$ . The use of the lifetime logic here is clearly overkill since direct ownership transfer of  $\ell_x$  already suffices, but it will nonetheless give the reader a concrete feel for the lifetime logic in action.

In Figure 15.3(a), we start by creating a lifetime  $\kappa$  (**LFTL-BEGIN**). Then, with the ownership of  $\ell_x \mapsto 0$ , we create a borrow  $\&_{\text{full}}^{\kappa}(\ell_x \mapsto \_)$  using **LFTL-FULL-BOR**. We assume a *send-receive* protocol **SENDRECV** for  $\ell_y$  that satisfies the rules **SENDRECV-CREATE**, **SC-SEND**, and **SC-RECV** (in the top of Figure 15.3). We instantiate this protocol with  $[\kappa]_{1/2}$  as the content to be sent. Again, we could have instantiated the protocol with  $\ell_x \mapsto \_$  and be done with it. Instead, here we want to demonstrate the use of borrows.

When spawning two threads, we give a half token  $[\kappa]_{1/2}$ , the borrow  $\&_{\text{full}}^{\kappa}(\ell_x \mapsto \_)$ , and **Send** to the thread  $\pi$ , and give the other half of the  $\kappa$  token, the killer, the inheritance, and **Recv** to thread  $\rho$ .

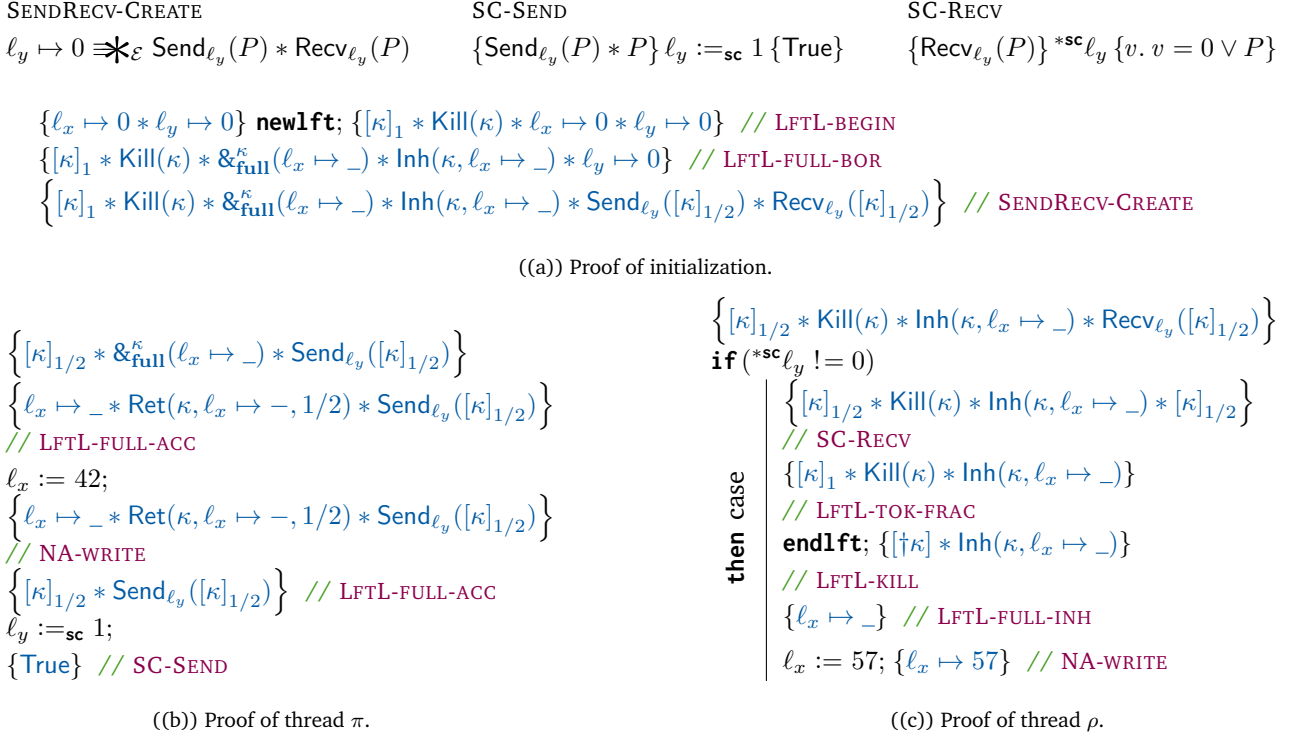


FIGURE 15.3: MP verified with the lifetime logic in Iris-SC.

In Figure 15.3(b), thread  $\pi$  trades the token and the borrow to access  $\ell_x \mapsto \_$  with **LFTL-FULL-ACC** and writes to  $\ell_x$ . After that, with **LFTL-FULL-RET**, thread  $\pi$  trades the return policy and  $\ell_x \mapsto \_$  to get back the token and the borrow. Finally, thread  $\pi$  writes to  $\ell_y$  and sends the token  $[\kappa]_{1/2}$  to thread  $\rho$ .

In Figure 15.3(c), thread  $\rho$  uses **Recv** to get back the full token. Owning **Kill**( $\kappa$ ), it ends the lifetime and earns the dead token  $[\dagger\kappa]$  (**LFTL-KILL**). Combining that with the inheritance, thread  $\rho$  reclaims the ownership of  $\ell_x \mapsto \_$  (**LFTL-FULL-INH**) and can safely write (non-atomically) to  $\ell_x$ .  $\square$

**Remark 15.3** (Safety of Inheritance). Let us note an important safety property of the lifetime logic: *the inheritance of a borrow can only be used after all accesses to the borrowed content have finished*. The key to ensuring this is that, during an access of the borrow  $\&_{\text{full}}^{\kappa} P$  via the accessing rule **LFTL-FULL-ACC**, the lifetime token  $[\kappa]_q$  and the borrow assertion are “kept” by the return policy and are only returned in exchange for the borrowed content  $P$ . By withholding  $[\kappa]_q$  and only returning it after the access finishes, the rule ensures that no party can have the full token  $[\kappa]_1$  needed to kill  $\kappa$  while others are still accessing borrows associated with  $\kappa$ . Consequently, the inheritance can only be used *after* all accesses have finished.

This safety property is no difference from the safety property **CANCEL-SAFE** (Property 11.2) of cancelable invariants, both in Iris-SC (Figure 14.1) and in iRC11 (§11.2). Indeed, we can see that borrows and cancelable invariants share many similarities in their interfaces: a fractional token is needed to access some protected resources, and a full fraction of the

token is sufficient to reclaim those protected resources. In this aspect, we can in fact see borrows as a generalization of cancelable invariants, where the generalization is in the flexibility of reclamation, which is now tied to lifetimes. This allows multiple borrows to be managed by a single lifetime, and while the borrows become “canceled” when the lifetime is killed, their reclamation can be done much later than that.

However, when looking at the aspect of concurrent accesses, cancelable invariants are more accommodating than full borrows. Full borrows only allow *sequential* accesses, by the fact that the full borrow assertion  $\&_{\text{full}}^{\kappa} P$  is unique and cannot be shared. As such, if multiple threads want to access a full borrow, they need extra synchronization to pass on the ownership of the full borrow assertion. (In [Example 15.2](#), only the thread  $\pi$  accesses the full borrow, while the thread  $\rho$  simply kills the lifetime and thus the borrow.) Meanwhile, cancelable invariants support multiple threads accessing shared resources atomically.

The SC lifetime logic does support other forms of borrows, including atomic borrows which do allow concurrent atomic accesses. Atomic borrows then are a true generalization of cancelable invariants. In [Chapter 16](#), we will see that porting atomic borrows to RMC faces the same challenge as porting cancelable invariants to RMC—we need to maintain **CANCEL-SAFE** in the presence of accesses that do not establish synchronization. Fortunately, the solution is also the same, and we needed to only change the interface of the access rule for atomic borrows. Other kinds of borrows, including full borrows, maintain the same interfaces when being ported from SC to RMC. Naturally, we needed to update the model of the lifetime logic so as to be sound in  $\lambda_{\text{Rust}} + \text{ORC11}$ .

*Note 15.4 (Rules with Viewshifts).* Finally, let us note that the rules given in [Figure 15.1](#) have been streamlined for better representation, with the additional concepts of the killer permission  $\text{Kill}(\kappa)$ , the inheritance  $\text{Inh}(\kappa, P)$ , and the return policy  $\text{Ret}(\kappa, P, q)$ . In practice, these concepts are actually just wand viewshifts, and we simply have them bundled in the rules. We will see the rules in [Figure 16.1](#) (§16.1).

Furthermore, note that the namespace  $\mathcal{N}_{\text{lf}}$  is a global, public namespace that is needed to establish the invariants for the internal model of the lifetime logic.

**CHAPTER SUMMARY.** In this chapter, we have reviewed borrows in Rust and how RustBelt accounts for them by developing the lifetime logic, which provides separation logic principles for borrows. We have seen the rules for managing lifetimes and full borrows. In the next chapter, we will see a more complete interface of the lifetime logic in RMC, and how to adapt the logic’s model on top of iRC11.



# 16

## *Lifetime Logic Meets Relaxed Memory*

---

In this chapter, we present a more complete interface of the lifetime logic *after* being ported from Iris-SC to iRC11. Fortunately, almost all proof rules of the lifetime logic are sound in RMC. The only change in the proof rules is in **LFTL-AT-ACC**—the access rule for *atomic borrows*—which allows access to the borrowed resource only under the view-join modality. This is very much similar to how the access rule of cancelable invariants (**CINV-ACC**, §11.2.1) needed to be changed in iRC11.

In this adaptation, the models for other borrows as well as the model of lifetime tokens are extended with instances of *synchronized ghost state* (**Concept 11.5**) to account for synchronization that always exists but needs to be witnessed explicitly under RMC. Despite these changes, the borrows enjoy the same proof rules as in SC. In particular, *the SC rules in Figure 15.1 for lifetimes and full borrows still hold in iRC11*.

We discuss more rules for lifetimes and full borrows in §16.1, and the interface of other borrow forms in §16.2. We simultaneously review these constructs (which are already developed in the original lifetime logic)<sup>1</sup> and describe the changes due to adaptation as they appear. Readers interested in more details of the origin lifetime logic are recommended to refer to the original paper and its technical appendix. In §16.3, we will discuss how to adapt the lifetime logic’s model on top of iRC11.

<sup>1</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [Jun+18a].

### 16.1 More Rules for the Lifetime Logic

In addition to the rules in Figure 15.1, the relaxed lifetime logic built atop iRC11 also admits stronger rules, some of which are given in Figure 16.1. The rule **LFTL-BEGIN-BD** bundles **LFTL-BEGIN** and **LFTL-KILL** together, and similarly **LFTL-FULL-BOR-BD** bundles **LFTL-FULL-BOR** and **LFTL-FULL-INH**, and **LFTL-FULL-ACC-BD** bundles **LFTL-FULL-ACC** and **LFTL-FULL-RET**. The rest of Figure 16.1 presents stronger rules that are originally from SC lifetime logic, but now adapted to relaxed memory.

#### 16.1.1 Lifetime Tokens Track Observations

**LFTL-TOK-OBJ-SPLIT** strengthens **LFTL-TOK-FRAC** by allowing splitting into two bits where one bit can be objective and thus can be put in invariant. This mirrors the same rule **CINV-TOK-OBJ-SPLIT** (§11.2.1) for iRC11 cancelable invariant tokens  $\heartsuit_q$ . Intuitively, both types of tokens are now

<sup>2</sup>of a borrow for a lifetime token, or of a cancelable invariant for an invariant token

synchronized ghost state, because they are witnesses not only for the liveness of a lifetime or a cancelable invariant, but also for what has happened during accesses<sup>2</sup> that the tokens have been used for. As such, the tokens are view-dependency, so sending them away (by *e.g.*, putting them in invariants) would require some synchronization. **LFTL-TOK-OBJ-SPLIT** helps in this regard: since the part  $[\kappa]_q$  of  $[\kappa]_{q+q'}$  is sufficient to bare witness for what  $[\kappa]_{q+q'}$  itself has observed, the other part  $[\kappa]_{q'}$  can start afresh with zero observation, *i.e.*, be placed under the objective modality (**Definition 8.10**), so that it can be sent to other threads without extra synchronization. The same situation applies for **CINV-TOK-OBJ-SPLIT** and cancelable invariant tokens.

The rule **LFTL-TOK-NOT-DEAD-SUBJ** strengthens **LFTL-TOK-NOT-DEAD**: the fact  $[\dagger\kappa]$  that the lifetime is dead is useful globally without the need of synchronization. That is, it is still sufficient if we have  $[\dagger\kappa]$  under a subjective modality (§8.6). In fact, it is often the case that we only need  $\langle \text{subj} \rangle [\dagger\kappa]$  (see also *faking*, below). We would need  $[\dagger\kappa]$  locally in inheritance to ensure the safety of inheritance (see **LFTL-FULL-BOR-BD** or **LFTL-FULL-INH**, and **Remark 15.3**).

### 16.1.2 *Faking*

**LFTL-BOR-FAKE** witnesses the fact that, once a lifetime has ended, the borrows tied to it have also ended and the owners of the inheritances can freely reclaim the borrowed resources *without* owning the borrow assertions  $\&_{\text{full}}^{\kappa} P$ . In other words, the borrow assertions  $\&_{\text{full}}^{\kappa} P$  become meaningless then. **LFTL-BOR-FAKE** thus allows us to create fake borrows if we know that the associated lifetime is dead, even only at the view the lifetime killer, without synchronization (*i.e.*, with only  $\langle \text{subj} \rangle [\dagger\kappa]$ ).

### 16.1.3 *Lifetime Inclusion*

The Rust's type system involves subtyping rules that rely on an inclusion between lifetime. Intuitively, a lifetime  $\kappa$  is included in a lifetime  $\kappa'$  if  $\kappa$  is *shorter* than  $\kappa'$ —in other words,  $\kappa'$  *outlives*  $\kappa$ . The subtyping rule with respect to lifetime (**T-BOR-LFT** in [Jun+18a, §3.3]) then allows a (type-level) borrow associated with  $\kappa$  to be a subtype of the borrow associated with the longer  $\kappa'$ .

In RustBelt, the (reflexive, transitive) lifetime inclusion relation  $\sqsubseteq$  gives rise to a meet semi-lattice for lifetimes, where the meet composition ( $\sqcap$ ) is commutative and associative. In other words, lifetimes  $\kappa$  form a *partial commutative monoid*.  $\sqcap$  is also called the *intersection* of lifetimes. Intersection is useful to create fresh sub-lifetimes: the meet composition respects lifetime inclusion, following **LFTL-INCL-INTER** and **LFTL-INCL-GLB**. **LFTL-FULL-SHORTEN** says that a borrow with a lifetime  $\kappa'$  can be turned into a borrow with a shorter lifetime  $\kappa$ . This lifetime-logic-level rule is the model for the type-level subtyping rule **T-BOR-LFT**.

**LFTL-TOK-INTER** and **LFTL-DEAD-INTER** show the interactions between lifetime intersection and liveness. To know that the intersected lifetime  $\kappa \sqcap \kappa'$  is alive, we need to know that both component lifetimes are alive. Reversely, the intersected lifetime is dead, then either  $\kappa$  or  $\kappa'$  is dead. This

**Bundled Rules.**

LFTL-BEGIN-BD

$$\text{True} \Rightarrow_{\mathcal{E}} \exists \kappa. [\kappa]_1 * \square([\kappa]_1 \Rightarrow_{\mathcal{N}_{\text{lf}}}^{\emptyset} [\dagger \kappa])$$

LFTL-FULL-BOR-BD

$$\triangleright P \Rightarrow_{\mathcal{N}_{\text{lf}}} \&_{\text{full}}^{\kappa} P * ([\dagger \kappa] \Rightarrow_{\mathcal{N}_{\text{lf}}}^* \triangleright P)$$

LFTL-FULL-ACC-BD

$$\&_{\text{full}}^{\kappa} P * [\kappa]_q \Rightarrow_{\mathcal{N}_{\text{lf}}} \triangleright P * (\triangleright P \Rightarrow_{\mathcal{N}_{\text{lf}}}^* \&_{\text{full}}^{\kappa} P * [\kappa]_q)$$

**Lifetime Liveness in Relaxed Memory.**

LFTL-TOK-OBJ-SPLIT

$$[\kappa]_{q+q'} \vdash [\kappa]_q * \langle \text{obj} \rangle [\kappa]_{q'}$$

LFTL-TOK-NOT-DEAD-SUBJ

$$[\kappa]_q * \langle \text{subj} \rangle [\dagger \kappa] \vdash \text{False}$$

LFTL-BOR-FAKE

$$\langle \text{subj} \rangle [\dagger \kappa] \Rightarrow_{\mathcal{N}_{\text{lf}}} \&_{\text{full}}^{\kappa} P$$

**Lifetime Inclusion.**

timeless( $\kappa \sqsubseteq \kappa'$ )  
persistent( $\kappa \sqsubseteq \kappa'$ )

LFTL-INCL-INTER

$$\kappa \sqcap \kappa' \sqsubseteq \kappa$$

LFTL-INCL-GLB

$$\frac{\kappa \sqsubseteq \kappa' \quad \kappa \sqsubseteq \kappa''}{\kappa \sqsubseteq \kappa' \sqcap \kappa''}$$

LFTL-FULL-SHORTEN

$$\kappa' \sqsubseteq \kappa \vdash \&_{\text{full}}^{\kappa} P \Rightarrow \&_{\text{full}}^{\kappa'} P$$

LFTL-TOK-INTER

$$[\kappa]_q * [\kappa']_{q'} \vdash \exists q''. [\kappa \sqcap \kappa']_{q''} * ([\kappa \sqcap \kappa']_{q''} \multimap [\kappa]_q * [\kappa']_{q'})$$

LFTL-DEAD-INTER

$$[\dagger \kappa \sqcap \kappa'] \dashv \vdash [\dagger \kappa] \vee [\dagger \kappa']$$

LFTL-UNIT-STATIC

$$\kappa \sqcap \varepsilon = \kappa$$

LFTL-INCL-STATIC

$$\kappa \sqsubseteq \varepsilon$$

LFTL-STATIC-NOT-DEAD

$$[\dagger \varepsilon] \vdash \text{False}$$

LFTL-TOK-STATIC

$$\vdash [\varepsilon]_q$$

**Reborrowing.**

LFTL-REBORROW

$$\kappa' \sqsubseteq \kappa \vdash \&_{\text{full}}^{\kappa} P \Rightarrow_{\mathcal{N}_{\text{lf}}}^* \&_{\text{full}}^{\kappa'} P * ([\dagger \kappa'] \Rightarrow_{\mathcal{N}_{\text{lf}}}^* \&_{\text{full}}^{\kappa} P)$$

LFTL-BOR-UNNEST

$$\&_{\text{full}}^{\kappa'} (\&_{\text{full}}^{\kappa} P) \Rightarrow_{\mathcal{N}_{\text{lf}}}^* \&_{\text{full}}^{\kappa \sqcap \kappa'} P$$

**Stronger Access Rules.**

LFTL-FULL-ACC-STRONG

$$\&_{\text{full}}^{\kappa} P * [\kappa]_q \Rightarrow_{\mathcal{N}_{\text{lf}}} \exists \kappa'. \kappa \sqsubseteq \kappa' * \triangleright P * (\forall Q. \triangleright (\triangleright Q * \langle \text{subj} \rangle [\dagger \kappa'] \Rightarrow_{\mathcal{N}_{\text{lf}}}^* \triangleright P) * \triangleright Q \Rightarrow_{\mathcal{N}_{\text{lf}}}^* \&_{\text{full}}^{\kappa'} Q * [\kappa]_q)$$

LFTL-FULL-ACC-ATOMIC-STRONG

$$\&_{\text{full}}^{\kappa} P \xrightarrow{\mathcal{N}_{\text{lf}}} \emptyset \vee \left\{ \begin{array}{l} \exists P', \kappa'. \kappa \sqsubseteq \kappa' * \triangleright ([P'] \wedge P) * \left( \begin{array}{l} \forall Q. \triangleright (\triangleright Q * \langle \text{subj} \rangle [\dagger \kappa'] \Rightarrow_{\mathcal{N}_{\text{lf}}}^* \triangleright P) * \triangleright ([P'] \wedge Q) \\ \emptyset \Rightarrow_{\mathcal{N}_{\text{lf}}}^* \&_{\text{full}}^{\kappa'} Q \end{array} \right) \\ \exists \kappa'. \kappa \sqsubseteq \kappa' * \langle \text{subj} \rangle [\dagger \kappa'] * \emptyset \xrightarrow{\mathcal{N}_{\text{lf}}} \text{True} \end{array} \right.$$

FIGURE 16.1: More selected rules for lifetimes and full borrows, ported to  $\lambda_{\text{Rust}} + \text{ORC11}$

implies that if the shorter lifetime  $\kappa$  ( $\kappa \sqsubseteq \kappa'$ ) is alive, the  $\kappa'$  is also alive, and if  $\kappa'$  is dead, then  $\kappa$  must also be dead.

The semi-lattice has a unit  $\varepsilon$  (**LFTL-UNIT-STATIC**), which is used to model the static lifetime ('**static**' in Rust). Intuitively, the static lifetime is the global lifetime that includes all lifetimes (**LFTL-INCL-STATIC**) and is never dead (**LFTL-STATIC-NOT-DEAD**).<sup>3</sup> Consequently, we can always acquire a fraction of the static lifetime token (**LFTL-TOK-STATIC**).

<sup>3</sup>Intuitively, if the static lifetime is already dead, then the program has ended.

**Definition 16.1** (Dynamic Lifetime Inclusion). Lifetime inclusion in RustBelt is in fact defined *semantically* or *dynamically*, using a token trading scheme. A lifetime  $\kappa$  is included in  $\kappa'$  if, given a fraction of the token for  $\kappa$ , we can produce some fraction of the token for  $\kappa'$ .

$$\kappa \sqsubseteq \kappa' ::= \Box \forall q. [\kappa]_q \Rightarrow_{\mathcal{N}_{\text{lt}}} \left( \exists q'. [\kappa']_{q'} * ([\kappa']_{q'} \Rightarrow_{\mathcal{N}_{\text{lt}}} [\kappa]_q) \right)$$

#### 16.1.4 Reborrowing

The rule **LFTL-REBORROW** strengthens **LFTL-FULL-SHORTEN**: it lets us *re-borrow* a  $\&_{\text{full}}^{\kappa} P$  into a borrow  $\&_{\text{full}}^{\kappa'} P$  where  $\kappa' \sqsubseteq \kappa$ . And when the shorter lifetime  $\kappa'$  ends, we get our original full borrow back. As such, while **LFTL-FULL-SHORTEN** can also be seen as a reborrow, it simply forgets the difference between  $\kappa'$  and  $\kappa$ . On the other hand, **LFTL-REBORROW** gives us an inheritance, effectively allowing us to regain and use the original borrow when  $\kappa'$  is already dead but  $\kappa$  is still alive. Note that the inheritance requires the observation of  $\kappa'$ 's end locally, *i.e.*, not under a subjective ( $\langle \text{subj} \rangle$ ) modality. **LFTL-REBORROW** justifies the RustBelt's type system rule **C-REBORROW** (in [Jun+18a, §3.3]).

The related rule **LFTL-BOR-UNNEST** allows us to turn a full borrow of a full borrow ( $\&_{\text{full}}^{\kappa'} \&_{\text{full}}^{\kappa} P$ ) into a full borrow of the intersected lifetime  $\&_{\text{full}}^{\kappa' \sqcap \kappa}$ . The catch is that we need a wand step viewshift (**Notation 7.8**) to strip off an extra later modality that appears between the two borrows.

#### 16.1.5 Stronger Access Rules

The rule **LFTL-FULL-ACC-STRONG** generalizes **LFTL-FULL-ACC-BD**. It allows us to close an access by giving back not just the original resource  $\triangleright P$ , but some  $\triangleright Q$  if we can show that  $\triangleright Q$  entails  $\triangleright P$  through a view shift. That view shift is only needed when the lifetime ends, *i.e.*, we can assume  $\langle \text{subj} \rangle [\dagger \kappa]$  when proving that  $Q$  entails  $P$ . In exchange, we get back a full borrow  $\&_{\text{full}}^{\kappa'} Q$  of  $Q$ . Intuitively, the rule allows us to turn a full borrow  $\&_{\text{full}}^{\kappa} P$  into  $\&_{\text{full}}^{\kappa'} Q$  with the access, as long as we can still guarantee that the inheritance will get back the original resource  $P$ , and hence the viewshift is only needed at inheritance, once the lifetime  $\kappa'$  is dead ( $[\dagger \kappa']$ ). If **LFTL-FULL-ACC-STRONG** is invoked multiple times, then the borrow's internal invariant will collect as many such viewshifts, and will apply all of them together at the inheritance to reclaim the original resource with which the very first borrow was created.

Furthermore, the rule exposes the fact that  $\kappa \sqsubseteq \kappa'$ , which comes from a part of RustBelt's model for the lifetime logic's borrows. That is, the model of a full borrow  $\&_{\text{full}}^{\kappa} P$  actually ties the resource  $P$  to a bigger lifetime  $\kappa'$ , and hence the borrow assertion is downward-closed



with respect to lifetime inclusion, and renders the proofs of rules like **LFTL-FULL-SHORTEN** easy. This technique is employed quite frequently in the lifetime logic’s model.

Finally, the rule **LFTL-FULL-ACC-ATOMIC-STRONG** provides a way to access a full borrow *without* having a proof that the lifetime is still ongoing. As such, with the access, we will find a disjunction, corresponding to the two cases where the lifetime  $\kappa$  is still alive or is already dead. If it is already dead, we get that fact subjectively, *i.e.*,  $\langle \text{subj} \rangle [\dagger \kappa']$ . If the lifetime is still alive, we get access to the resource  $P$ , and we can close the access in the same way as that of **LFTL-FULL-ACC-STRONG**. Since we do not need to provide a lifetime token  $[\kappa]_q$ , the access cannot be non-atomic, because the lifetime  $\kappa$  may be killed during such a non-atomic access. The atomicity of the access is enforced by the mask-changing viewshifts.

Note that in addition to acquiring  $P$  at opening and returning  $Q$  at closing, we also know and then have to show—at opening and closing of the access—that some resource  $P' : \text{iProp}$  holds simultaneously (*i.e.*, with a classical conjunction) with  $P$  or  $Q$ . The “embed” modality  $[\cdot]$  embeds  $\text{iProp}$  propositions into  $\text{vProp}$ . This extra requirement is also due to potential relaxed memory effects. Since we do not have a lifetime token at hand to witness to possible changes from  $P$  to  $Q$ , we instead require that the underlying resource  $P'$  that may be tied to some view (hence the use of  $\text{iProp}$ ) needs to be maintained at the same view.

**LFTL-FULL-ACC-ATOMIC-STRONG** is needed to prove rules for full borrows that should not require a lifetime token, for examples, to prove commutativity with the later modality or the existential quantifier, or to convert a full borrow into a *fractured* borrow—which we will see next.

## 16.2 Other Forms of Borrows

Full borrows are perfect for modeling mutable references that can only have one user at a time. This is because accesses to full borrows are always sequential: at any moment in time, there can be only one ongoing access to a full borrow. For this reason, however, full borrows are not suitable for modeling types that are meant to be accessed concurrently by multiple threads, *e.g.*, shared references. This motivates two alternatives of full borrows: *fractured* borrows and *atomic* borrows.

Furthermore, full borrows are not persistent—they are unique resources and are withheld during an access so as to ensure at most one access to the underlying protected resource at a time. Non-persistency makes it difficult to build more complex protocols using full borrows. To make the borrows persistent but still maintain unique accesses, one can restrict accesses to a single-threaded manner, by employing non-atomic thread-local invariants (§11.3). This gives rise to *non-atomic* persistent borrows.

All of these different borrows are originally developed in the SC RustBelt work.<sup>4</sup> Table 16.1 compares several of their properties. These differences already exist in RustBelt except for the last column, which is unique to the relaxed-memory setting. (We will come back to that column

<sup>4</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [Jun+18a].

| Borrow type                                                  | Access type            | Access amount | Persistent | Communication among accesses | Access at local view |
|--------------------------------------------------------------|------------------------|---------------|------------|------------------------------|----------------------|
| Full borrows $\&_{\text{full}}^{\kappa} P$                   | sequential, non-atomic | full          | no         | yes                          | yes                  |
| Non-atomic borrows $\&_{\text{na}}^{\kappa/p.\mathcal{N}} P$ | sequential, non-atomic | full          | yes        | yes                          | yes                  |
| Fractured borrows $\&_{\text{frac}}^{\kappa} \Phi$           | concurrent, non-atomic | fractions     | yes        | no                           | yes                  |
| Atomic borrows $\&_{\text{at}}^{\kappa/\mathcal{N}} P$       | concurrent, atomic     | full          | yes        | yes                          | no                   |

TABLE 16.1: Comparison of borrow types

in §16.3.) All three forms of borrows— $\&_{\text{frac}}^{\kappa} \Phi$ ,  $\&_{\text{at}}^{\kappa/\mathcal{N}} P$ , and  $\&_{\text{na}}^{\kappa/p.\mathcal{N}} P$  for fractured, atomic and non-atomic borrows, respectively—are created from a full borrow  $\&_{\text{full}}^{\kappa} P$ . All of their borrow assertions are persistent, so that the same borrow can be referred to and accessed by multiple parties. Fractured and atomic borrows are in fact accessible concurrently by multiple threads, but fractured borrows allow non-atomic accesses to only a *fraction* of the protected resource—ensuring that enough fractions remain for all participants at all times, whereas atomic borrows enforce a strict turn-taking scheme, allowing access to the *full* resource but only for a single atomic step of execution. On the other hand, non-atomic borrows allow non-atomic accesses to the full resource—like full borrows—but the unique access restriction is instead enforced through the non-atomic invariant token  $[\text{Na} : p.\mathcal{N}]$  that ties the borrow to the invariant pool  $p.\mathcal{N}$ .

We now look more closely at each form of borrows, with their respective rules given in Figure 16.2.

### 16.2.1 Fractured Borrows

To meaningfully talk about fractions of resources, fractured borrows assume a *predicate*  $\Phi$  over fractions that is compatible with fraction addition:  $\Phi(q_1 + q_2) \Leftrightarrow \Phi(q_1) * \Phi(q_2)$ . With that, **LFTL-FULL-FRACTURE** (Figure 16.2) allows converting a full borrow  $\&_{\text{full}}^{\kappa} \Phi(1)$  of the full resource  $\Phi(1)$  into a fracture borrow  $\&_{\text{frac}}^{\kappa} \Phi$ . Note that  $\Phi$ 's compatibility with fraction addition can be proven as a persistent  $\text{vProp}$  fact under a later, as it will only be used once the resource  $\Phi(1)$  is stored inside the internal invariant of the fracture borrow (*i.e.*, stored under a later). As mentioned earlier, the creation of a fracture borrow (and similarly, of an atomic or non-atomic borrow) does not need to know whether  $\kappa$  is alive or not. This demonstrates again the flexibility of the indirect resource reclamation scheme with lifetimes: the inheritance will receive the resource after its associated lifetime ends, regardless of how the original full borrow may have been transformed or used.

With a lifetime token  $[\kappa]_q$ , the access rule **LFTL-FRACT-ACC** gives access to  $\Phi(q')$  for some fraction  $q'$ . Once that exact  $q'$  of  $\Phi$  is returned, we regain the lifetime token that we started the access with.

**LFTL-FRACT-SHORTEN** allows shortening the lifetime of a fractured bor-

**Fractured borrows.**

$$\frac{\text{LFTL-FULL-FRACTURE} \quad \triangleright \Box(\forall q_1, q_2. \Phi(q_1 + q_2) \Leftrightarrow \Phi(q_1) * \Phi(q_2))}{\&_{\text{full}}^{\kappa} \Phi(1) \Rightarrow_{\mathcal{N}_{\text{ft}}} \&_{\text{frac}}^{\kappa} \Phi}$$

$$\text{LFTL-FRACT-ACC} \quad \&_{\text{frac}}^{\kappa} \Phi \vdash [\kappa]_q \Rightarrow_{\mathcal{N}_{\text{ft}}} \exists q'. \triangleright \Phi(q') * \left( \triangleright \Phi(q') \Rightarrow_{\mathcal{N}_{\text{ft}}} [\kappa]_q \right)$$

$$\text{LFTL-FRACT-SHORTEN} \quad \kappa' \sqsubseteq \kappa \vdash \&_{\text{frac}}^{\kappa} \Phi \Rightarrow \&_{\text{frac}}^{\kappa'} \Phi$$

$$\text{LFTL-FRACT-IFF} \quad \triangleright \Box(\forall q. \Phi(q) \Leftrightarrow \Psi(q)) \vdash \&_{\text{frac}}^{\kappa} \Phi \Rightarrow \&_{\text{frac}}^{\kappa} \Psi$$

**Atomic persistent borrows.**

$$\frac{\text{LFTL-FULL-AT} \quad \mathcal{N} \# \mathcal{N}_{\text{ft}}}{\&_{\text{full}}^{\kappa} P \Rightarrow_{\mathcal{N}_{\text{ft}}} \&_{\text{at}}^{\kappa/\mathcal{N}} P}$$

$$\text{LFTL-AT-ACC} \quad \&_{\text{at}}^{\kappa/\mathcal{N}} P \vdash [\kappa]_q \quad \mathcal{N}_{\text{ft}} \uplus \mathcal{N} \Rightarrow_{\mathcal{N}_{\text{ft}}} \exists V_b. \sqcup_{V_b} \triangleright P * \left( \sqcup_{V_b} \triangleright P \Rightarrow_{\mathcal{N}_{\text{ft}}} [\kappa]_q \right)$$

$$\text{LFTL-AT-SHORTEN} \quad \kappa' \sqsubseteq \kappa \vdash \&_{\text{at}}^{\kappa/\mathcal{N}} P \Rightarrow \&_{\text{at}}^{\kappa'/\mathcal{N}} P$$

$$\text{LFTL-AT-IFF} \quad \triangleright \Box(P \Leftrightarrow Q) \vdash \&_{\text{at}}^{\kappa/\mathcal{N}} P \Rightarrow \&_{\text{at}}^{\kappa/\mathcal{N}} Q$$

**Non-atomic persistent borrows.**

$$\text{LFTL-FULL-NA} \quad \&_{\text{full}}^{\kappa} P \Rightarrow_{\mathcal{N}_{\text{ft}}} \&_{\text{na}}^{\kappa/p.\mathcal{N}} P$$

$$\text{LFTL-NA-ACC} \quad \&_{\text{na}}^{\kappa/p.\mathcal{N}} P \vdash [\kappa]_q * [\text{Na} : p.\mathcal{N}] \Rightarrow_{\mathcal{N}_{\text{ft}} \cup \mathcal{N}} \triangleright P * \left( \triangleright P \Rightarrow_{\mathcal{N}_{\text{ft}} \cup \mathcal{N}} [\kappa]_q * [\text{Na} : p.\mathcal{N}] \right)$$

$$\text{LFTL-NA-SHORTEN} \quad \kappa' \sqsubseteq \kappa \vdash \&_{\text{na}}^{\kappa/p.\mathcal{N}} P \Rightarrow \&_{\text{na}}^{\kappa'/p.\mathcal{N}} P$$

$$\text{LFTL-NA-IFF} \quad \triangleright \Box(P \Leftrightarrow Q) \vdash \&_{\text{na}}^{\kappa/p.\mathcal{N}} P \Rightarrow \&_{\text{na}}^{\kappa/p.\mathcal{N}} Q$$

row, like **LFTL-FULL-SHORTEN** for full borrows. **LFTL-FRACT-IFF** says that fractured borrows are closed under the equivalence of the fractional predicate.

FIGURE 16.2: Selected rules for other borrow alternatives, sound in  $\lambda_{\text{Rust}} + \text{ORC11}$

### 16.2.2 Atomic Persistent Borrows

In contrast to fractured borrows, atomic borrows do provide full access to the resources contained within, but only for a single, atomic instruction. This restriction of atomic borrows is encoded in its access rule **LFTL-AT-ACC** using mask-changing viewshifts, like **INV-ACC** or **CINV-ACC**. Recall that the namespace  $\mathcal{N}_{\text{ft}}$  is used for the internal invariant of the lifetime logic's model. The client of atomic borrows picks a namespace  $\mathcal{N}$  *disjoint* from  $\mathcal{N}_{\text{ft}}$ , at the creation of a atomic borrow from a full borrow using **LFTL-FULL-AT**, to allocate the underlying invariant that will store the resource  $P$  for concurrent atomic accesses. Of course, an atomic borrow is still tied to a lifetime specifying its period of validity, so that a lifetime token  $[\kappa]_q$  is still required to guarantee that the lifetime is alive during the access.

Atomicity is crucial, for example, when several threads need to modify a shared variable, such as a reference counter for shared pointers. Therefore, while fractured borrows are designed to model *immutable* shared resources, atomic borrows are designed to model *mutable* shared resources in concurrent libraries. Naturally, these libraries use atomic memory accesses whose rules<sup>5</sup> are compatible with **LFTL-AT-ACC**.

Most importantly, **LFTL-AT-ACC** only allows accesses to the borrowed resource  $P$  under a view-join modality. That is, the access gives us

<sup>5</sup>e.g., those in §10

$\sqcup_{V_b} \triangleright P$  and requires us to return the same  $\sqcup_{V_b} \triangleright P$  at the end of the access. This is exactly the same requirement as that of the access rule **CINV-ACC** for iRC11 cancelable invariants (§11.2.1), for the same reason: we need to maintain soundness of the rules when porting them from Iris-SC to iRC11. The view-join modality helps us prevent unsound accidental synchronization between concurrent accesses.

This leads us back to the last two columns of Table 16.1. Each access of a fractured borrow does not communicate with another, because each access obtains an independent fraction of the borrowed contents, and thus can access that fraction at the accessing thread’s local view. Meanwhile, each access of an atomic (or non-atomic or full) borrow obtains the full contents and can modify them and thus can communicate with other accesses. For atomic borrows, it means that without the view-join modality, concurrent accesses would transfer resources from one thread to another without actual physical synchronization, rendering our logic *unsound*! Imagine that if **LFTL-AT-ACC** without the view-join modality were sound, a thread  $\pi$  would access and modify  $P$  (i.e., after the access  $P$  holds at  $\pi$ ’s local view), and immediately in the next step a different thread  $\rho$  would get synchronized access to  $P$  at its own local view, including the modifications made by  $\pi$ . In that case, the logic would allow synchronization from  $\pi$  to  $\rho$  without there being any physical synchronization to support that.

To avoid this unsoundness due to accidental synchronization between concurrent accesses, it is sufficient to protect  $P$  with a view-at modality, i.e.,  $@_{V_b} \triangleright P$ , and this would make atomic borrows roughly related to *objective invariants* (§11.1). However, as we also want to reclaim the borrowed resources, we need to employ the view-join modality to enable and maintain the safety of inheritance (Remark 15.3), in the same exact way as how iRC11 cancelable invariants maintains **CANCEL-SAFE** (§11.2).

Finally, **LFTL-AT-SHORTEN** and **LFTL-AT-IFF** say that atomic borrows are closed with respect to lifetime inclusion and predicate equivalence.

### 16.2.3 Non-Atomic Persistent Borrows

A non-atomic borrow  $\&_{\text{na}}^{\kappa/p.\mathcal{N}} P$  can be created from a full borrow  $\&_{\text{full}}^{\kappa} P$  using **LFTL-FULL-NA** (Figure 16.2). Non-atomic borrows offer the same sequential, non-atomic access style as that of full borrows, but manages the unique access restriction through the non-atomic invariant token  $[\text{Na} : p.\mathcal{N}]$  (for the namespace  $\mathcal{N}$  under the pool  $p$ ),<sup>6</sup> instead of the borrow assertion  $\&_{\text{full}}^{\kappa} P$  itself like in the case of full borrows. As such, non-atomic borrow assertions  $\&_{\text{na}}^{\kappa/p.\mathcal{N}} P$  are persistent and can be owned by multiple parties. Non-atomic borrows are thus useful to model single-threaded “smart pointer” types, e.g., **Rc** or **RefCell**.

The access rule **LFTL-NA-ACC** thus requires the invariant token  $[\text{Na} : p.\mathcal{N}]$  in addition to the lifetime token  $[\kappa]_q$ . Under the hood, the rule is supported by the non-atomic invariant access rule **NAINV-ACC** and the access rules for borrows.

**LFTL-NA-SHORTEN** and **LFTL-NA-IFF** say that non-atomic borrows are also closed with respect to lifetime inclusion and predicate equivalence.

<sup>6</sup>see §11.3

### 16.3 Adaption of the Lifetime Logic's Model in iRC11

In this section, we briefly discuss the adaptation of the lifetime logic's model on top of iRC11. This work was spearheaded by Jacques-Henri Jourdan, one of the designers of the original Iris-SC lifetime logic. Therefore, the discussion here is not a contribution of this dissertation, and is only provided for the sake of completeness.

**Concept 16.2** (The Invariant for Lifetimes and Borrows). The model of the lifetime logic sets up a global invariant that governs the protocols for all lifetimes and their associated borrows. At a very high-level, the global invariant  $\text{LftInv}(\kappa)$  for each lifetime  $\kappa$  is roughly as follows.

- $\kappa$  is either alive or dead:  $\text{LftInv}(\kappa) = \text{LftAlive}(\kappa) \vee \text{LftDead}(\kappa)$ .
- For each borrow  $\&^\kappa P$  associated with  $\kappa$ , when  $\kappa$  is already dead,  $\text{LftDead}(\kappa)$  tracks if the inheritance  $\text{Inh}(\kappa, P)$ <sup>7</sup> has been used. For borrows whose inheritance have not been used,  $\text{LftDead}(\kappa)$  carries the resources of those borrows. This part of the protocol is needed for **LFTL-FULL-INH**.<sup>8</sup>
- When  $\kappa$  is still alive,  $\text{LftAlive}(\kappa)$  tracks the states of each borrow  $\&^\kappa P$  associated with  $\kappa$ . The borrow can be in one of three states:
  - The borrow is not being accessed, and the resource  $P$  is still being hold by  $\text{LftAlive}(\kappa)$ .
  - The borrow is being opened by someone, so the resource  $P$  has been taken out of  $\text{LftAlive}(\kappa)$ . On the other hand, as part of the borrowing process, some fraction  $q$  of the lifetime token  $[\kappa]_q$  must have been put in  $\text{LftAlive}(\kappa)$  as a deposit. This corresponds to the rules **LFTL-FULL-ACC** and **LFTL-FULL-RET**.<sup>9</sup>
  - The borrow has been reborrowed at a strictly shorter lifetime  $\kappa'$ .  $\text{LftAlive}(\kappa)$  then needs to track how to reclaim resources for  $\&^\kappa P$  from the reborrows when  $\kappa$  is killed. This part of the protocol models the reborrowing rule **LFTL-REBORROW**.<sup>10</sup>

<sup>7</sup>see §15.2

<sup>8</sup>see Figure 15.1

<sup>9</sup>see Figure 15.1

<sup>10</sup>see Figure 16.1

As typical for logics built in Iris, the lifetime invariant  $\text{LftInv}(\kappa)$  together with the lifetime and borrow assertions (e.g., the lifetime token  $[\kappa]_q$  or the borrow assertion  $\&_{\text{full}}^\kappa P$ , as listed in §15.2) are modeled with user-defined ghost state that encodes the desirable properties needed by the protocols. For example, in the Iris-SC model of the lifetime logic,  $[\kappa]_q$  and  $[\dagger\kappa]$  are defined purely with disjoint ghost elements (so as to satisfy **LFTL-TOK-NOT-DEAD**),<sup>11</sup> and  $\&_{\text{full}}^\kappa P$  is defined with an exclusive ghost element, together with some invariant that ties  $P$  to the global invariant  $\text{LftInv}(\kappa)$ .

<sup>11</sup>see Figure 15.1

The Iris-SC lifetime logic, however, models the lifetime token  $[\kappa]_q$  as a view-agnostic assertion, simply asserting the ghost ownership of some fraction  $q$  of the ghost location  $\kappa$ :

$$\llbracket [\kappa]_q \rrbracket ::= \left[ \left[ \frac{\kappa}{q} \right] \right] \quad (\text{LFT-TOK-SC-MODEL})$$

This model is insufficient for the RMC lifetime logic, because it does not guarantee the safety of inheritance ([Remark 15.3](#)) in the presence of concurrent borrow accesses that do not establish synchronization. In  $\text{RB}_{\text{rlx}}$ , we instead need to enrich this model so that it depends on the view at which  $[\kappa]_q$  is asserted:

$$\llbracket [\kappa]_q \rrbracket ::= \exists V_{\text{tok}}. [\overline{(q, V_{\text{tok}})}]^\kappa * \sqsupseteq V_{\text{tok}} \quad (\text{LFT-TOK-RLX-MODEL})$$

In this model, the ghost element is no longer just a fraction  $q$ , but a pair of the fraction and the *token view*  $V_{\text{tok}}$ . The token view  $V_{\text{tok}}$  represents what this particular fraction of the token has *observed*, i.e., what borrow accesses the token has participated in. The model requires that the local view at which the token is interpreted has also at least observed what  $[\kappa]_q$  has observed:  $\sqsupseteq V_{\text{tok}}$ .

<sup>12</sup>see [Definition 11.4](#), §11.2.2

Note that this model of lifetime token is exactly the same as the model of cancelable invariant token  $\heartsuit_q^\gamma$ , given by [CINV-MODEL-TOK](#),<sup>12</sup> because they share the same goal: guaranteeing the safety of inheritance/cancelation. The protocol for borrow inheritance, however, is quite more elaborate than invariant cancelation, because a lifetime can be associated with multiple borrows, who in turn can be reborrowed. In the following, we sketch the proofs for the inheritance and access rules, for full, fractured, and atomic borrows in iRC11. As we will see shortly, the key idea of the proofs is to associate views not only with the lifetime token assertions, but with all the other assertions that play a role in the lifetime logic.

### 16.3.1 Full Borrows

**PROVING INHERITANCE.** To prove [LFTL-FULL-INH](#) ([Figure 15.1](#)) sound for  $\text{RB}_{\text{rlx}}$ , we get to assume  $[\dagger\kappa]$  and  $\text{Inh}(\kappa, P)$  at the thread's current local view—call it  $V$ , and we need to produce  $\triangleright P$  at that same view  $V$ . Now, the lifetime logic is responsible for controlling ownership of the content of the borrow,  $P$ ; so let us assume that, when no threads are accessing the borrow,  $P$  is maintained at a view  $V_b$ , which we call the *content view*. Furthermore, by owning  $[\dagger\kappa]$ , we know that the global lifetime invariant  $\text{LftInv}(\kappa)$  is in the  $\text{LftDead}(\kappa)$  disjunct, and by owning  $\text{Inh}(\kappa, P)$ , we know that the inheritance has not been applied yet (we are the one to apply it), so  $\text{LftDead}(\kappa)$  is still holding on to  $P$  at the view  $V_b$ . In other words,  $\text{LftDead}(\kappa)$  is holding  $@_{V_b} \triangleright P$ .

When we apply the inheritance, we will therefore be trading  $\text{Inh}(\kappa, P)$  for  $@_{V_b} \triangleright P$  from  $\text{LftDead}(\kappa)$ . If we can show that  $\sqsupseteq V_b$ , i.e., the current thread has locally observed  $V_b$ , we can use [VA-ELIM](#) ([Figure 8.3](#)) to obtain  $\triangleright P$  and finish the proof.

The key to proving the goal  $\sqsupseteq V_b$  is to:

- associate with each lifetime logic assertion a view that represents what the assertion has observed, i.e., what activities it has been involved in; and then
- establish and maintain for those associated views sufficiently strong invariants in  $\text{LftInv}(\kappa)$  so that we can prove our goal.

Below, we summarize a list of some associated views for assertions that interact with a full borrow  $\&_{\text{full}}^{\kappa} P$ :

- the *content view*  $V_b$  at which the lifetime invariant keeps  $P$ ;
- the *token views*  $V_{\text{tok}}$ , one for every  $[\kappa]_q$ ;
- the *full token view*  $V_{\kappa}$  of the full token  $[\kappa]_1$ , defined as the join of all token views  $V_{\text{tok}}$ ;
- the *dead token view*  $V_{\dagger} \sqsupseteq V_{\kappa}$ ; and
- the *borrow view*  $V_B$  of the borrow assertion  $\&_{\text{full}}^{\kappa} P$ .

In order to prove  $\sqsupseteq V_b$  for **LFTL-FULL-INH**, we enforce the following property in  $\text{LftInv}(\kappa)$  for the borrow in question:

$$V_b \sqsubseteq V_{\kappa} \quad (\text{LFTL-FULL-BOR-INV-1})$$

By owning  $[\dagger\kappa]$ , we have for the current thread  $\sqsupseteq V_{\dagger}$ . By view-monotonicity (i.e., **VS-MONO**, **Figure 8.3**), the current thread must have observed  $\sqsupseteq V_{\kappa}$  and  $\sqsupseteq V_b$ .  $\square$

This shows that **LFTL-FULL-BOR-INV-1** allows us to prove **LFTL-FULL-INH**. But what is the intuition for this invariant? As hinted at before, each piece of a lifetime token is intended to bear “witness” to any access to the borrow that the token is used for. Ultimately, the full token  $[\kappa]_1$ —which is the join of all tokens—must have witnessed *all* accesses to the borrow ( $V_b \sqsubseteq V_{\kappa}$ ). Therefore, a thread owning the full token should have observed all modifications made to  $P$  by all of those accesses, so it can safely kill the lifetime and produce the dead token  $[\dagger\kappa]$ . Effectively, a thread owning  $[\dagger\kappa]$  must have observed all modifications made to  $P$  ( $V_b \sqsubseteq V_{\kappa} \sqsubseteq V_{\dagger}$ ), so it can use the inheritance to reclaim  $P$  at its local view.

How do we maintain **LFTL-FULL-BOR-INV-1**?  $V_b \sqsubseteq V_{\kappa}$  is maintained by the rule **LFTL-FULL-RET**. Note that the lifetime token  $[\kappa]_q$  is withheld during the access. When the access finishes and  $P$  is returned to the borrow at an updated view  $V'_C$ , the rule uses  $V'_C$  to update the view of the withheld token  $[\kappa]_q$  from  $V_{\text{tok}}$  to  $V_{\text{tok}} \sqcup V'_C$  before returning it to the user. Since  $V_{\kappa}$  is the join of all lifetime tokens, this effectively updates  $V_{\kappa}$  to  $V_{\kappa} \sqcup V'_C \sqsupseteq V'_C$ . With this, the invariant is re-established. Note that this line of reasoning mirrors that of the proof of the cancelable invariant access rule **CINV-ACC** in §11.2.2.

**PROVING ACCESSES.** To prove the rule for accessing full borrows, **LFTL-FULL-ACC**, we assume the lifetime token and the borrow assertion, and we need to provide the user with synchronized access to  $P$  at the thread's local view. Assuming that we can prove the return policy  $\text{Ret}(\kappa, P, q)$ , we still need to ensure that  $P$  holds locally, i.e., we have  $\sqsupseteq V_b$ . For this, we rely on the following invariant:

$$V_b \sqsubseteq V_B \quad (\text{LFTL-FULL-BOR-INV-2})$$

That is, the content view  $V_b$  of  $P$  is always included in the borrow view  $V_B$  of  $\&_{\text{full}}^{\kappa} P$ . In other words, owning  $\&_{\text{full}}^{\kappa} P$  should imply  $\sqsupseteq V_B$ , which



in turn implies  $\sqsubseteq V_b$ . Like **LFTL-FULL-BOR-INV-1**, **LFTL-FULL-BOR-INV-2** is straightforward to maintain: the borrow assertion  $\&_{\text{full}}^{\kappa} P$  is withheld during the access; and when an access is returned, we update the borrow assertion's  $V_B$  with the new content view  $V'_C$ .  $\square$

### 16.3.2 Fractured Borrows

Fractured borrows, like all borrows, need to guarantee that all accesses happen-before the inheritance is applied. However, unlike full borrows where all accesses are ordered, accesses to a fractured borrow can happen independently from one another. Thus, for fractured borrows, we need to maintain that independent changes to fractions of  $\Phi$  made by independent accesses are all observed by the thread performing the inheritance. The key to achieve this is to recognize that the content view of  $\Phi$  is no longer a single view, but consists of two views:

- (1) the view  $V_{\text{YTBA}}$  of the *yet-to-be-accessed* portion of  $\Phi$ ,
- (2) the view  $V_{\text{AA}}$  of the *already-accessed* portion of  $\Phi$ .

$V_{\text{YTBA}}$  is the view of the chunk of  $\Phi$  that has not been given out to any access, as well as the view at which the fractured borrow was created. Meanwhile,  $V_{\text{AA}}$  tracks all the changes made by the accesses to all the chunks that have been given out to those accesses.

With that in mind, we repeat what we did for full borrows: defining the invariants for the inheritance and the accesses of fractured borrows.

**PROVING INHERITANCE.** We enforce an invariant on the two views of a fractured borrow  $\&_{\text{frac}}^{\kappa} \Phi$ :

$$V_{\text{YTBA}} \sqcup V_{\text{AA}} \sqsubseteq V_{\kappa} \quad (\text{LFTL-FRACT-BOR-INV-1})$$

That is, instead of using a single content view  $V_b$  like in **LFTL-FULL-BOR-INV-1**, we use the view  $V_{\text{YTBA}} \sqcup V_{\text{AA}}$  which is the view of the full resource  $\Phi(1)$ , and require that it is always included in the full token view  $\kappa$ . Thus, by similar reasoning to **LFTL-FULL-BOR-INV-1**, any changes to the fractured borrow's contents are guaranteed to happen-before the moment the inheritance is applied.

**PROVING ACCESSES.** We maintain a second invariant:

$$V_{\text{YTBA}} \sqsubseteq V_B \quad (\text{LFTL-FRACT-BOR-INV-2})$$

This invariant enables the synchronized access to a fraction of  $\Phi$  in the accessing rule **LFTL-FRACT-ACC**: when the rule is applied with a thread owning a borrow assertion  $\&_{\text{frac}}^{\kappa} \Phi$  with some borrow view  $V_B$ , we have that the thread has observed  $V_B$ , i.e.,  $\sqsubseteq V_B$ . Consequently, the thread has  $\sqsubseteq V_{\text{YTBA}}$ , and it can then obtain some portion  $\Phi(q')$  from the yet-to-be-accessed chunk at its local view.

### 16.3.3 Atomic Persistent Borrows

The challenge of porting atomic borrows is the same one of porting cancelable invariants to RMC: atomic borrows allow concurrent accesses



to the full contents of the borrow, where each access can modify the contents and thus can communicate with other accesses. While concurrent accesses are also possible with fractured borrows, this situation does not apply to them because each access of a fractured borrow obtains an independent fraction of the underlying contents.

The content view  $V_b$  of the borrow content  $P$  can be constantly changed by different threads with atomic accesses to the borrow  $\&_{\text{at}}^{\kappa/\mathcal{N}} P$ , and there is in general no relationship between threads' local views and the content view  $V_b$ . This is why, like the cancelable invariant access rule **CINV-ACC**, the atomic borrow access rule **LFTL-AT-ACC** only provides the borrow content protected under a view-join modality, in the form  $\sqcup_{V_b} \triangleright P$ . It is then the obligation of the clients of the atomic borrow to eliminate the view-join modality in order to get actual access to  $P$ . The main role of the view-join modality is to maintain safety of inheritance: we still have that  $V_b \sqsubseteq V_{\dagger}$ , so with  $\sqcup_{V_b} \triangleright P$  and  $\sqsubseteq V_{\dagger}$ , we can use **VJ-ELIM** (Figure 8.3) to safely inherit  $\triangleright P$ . The proofs of inheritance and accesses for atomic borrows are therefore very similar to the proofs for cancelable invariants (see §11.2.2).

**CHAPTER SUMMARY.** In this chapter, we have reviewed the adaptation of the RustBelt lifetime logic to RMC, on top of our logic iRC11. This is sufficient to establish “Task 2: re-proving the safety of the  $\lambda_{\text{Rust}}$  type system under RMC”, due to the fact that the lifetime logic interface only changes in the atomic borrow access rule. In this part's remaining chapters, we demonstrate some of the results in “Task 1”, where we rely on the facilities of iRC11 as well as atomic persistent borrows to re-verify the safety of several Rust libraries, considering their real, relaxed-memory implementations.



# 17

## GPS Single-Location Protocols

---

In this chapter, we present how to encode GPS single-location protocols<sup>1</sup> in iRC11, using atomics points-to (Chapter 10). From an organizational perspective, this encoding should have been introduced as part of iRC11. However, the encoding was mainly developed to perform verifications of relaxed memory concurrent libraries for  $\text{RB}_{\text{rlx}}$  (i.e., for Task 2, see §14.2). In Chapter 18 and Chapter 19, we will demonstrate how GPS protocols are used to verify respectively, the reader-writer lock and the atomic reference counting against their RustBelt’s semantic types.

We will start in §17.1 with the interfaces of several different kinds of GPS protocols in iRC11. The kinds of GPS protocols differ on (1) how long they can be accessed, and on (2) how much concurrency they allow. For (2), there are *concurrent* protocols which allow *arbitrary* concurrent accesses, and *single-writer* protocols which allow either single-writer or CAS-only writers. For (1), there are *persistent* protocols which stay alive forever, *cancelable* protocols which stay alive as long as the cancelable invariant token is still available, and *atomic-borrows-based* protocols whose lifecycle is tied to a lifetime. Atomic-borrows-based protocols will be used in the verification of the reader-writer lock (Chapter 18), while cancelable protocols will be used in the verification of the atomic reference counting (Chapter 19).

In §17.2, we provide intermediate-level interfaces of GPS protocols, which we call the *middleware*. We show how middleware GPS protocols are a common interface that can be combined with different types of invariants to derive the surface-level GPS protocols. Specifically, we define, as examples, the models of persistent/concurrent protocols, of cancelable/single-writer protocols, and of atomic-borrows-based/single-writer protocols using the middleware protocols respectively together with objective invariants (§11.1), cancelable invariants (§11.2), and atomic borrows (§16.2).

Finally, in §17.3, we briefly explain the model of middleware protocols, which supports both concurrent and single-writer protocols, and which is built upon the atomic points-to assertion.

<sup>1</sup>Turon et al., “GPS: navigating weak memory with ghosts, protocols, and separation” [TVD14]; Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

### 17.1 Surface-level GPS Protocols in iRC11

**Definition 17.1** (GPS Protocol Type). A GPS protocol for a single location  $\ell$  restricts how  $\ell$ ’s history can grow. The setup therefore lets the user

pick a type of *protocol states*, and then pick a state for every write in the history. The restriction is that, as the history grows, the states picked can only grow following a *pre-order* (i.e., it is reflexive and transitive) that is also picked up front by the user. More concretely, the user needs to pick  $S \in \text{ProtoState} ::= (T_S, \sqsubseteq)$  where  $T_S$  is the state type that enjoys the pre-order  $\sqsubseteq$ .

**Example 17.2** (GPS Protocol Types). We provide a list of common protocol types.

$$\begin{aligned} S_{()} &::= ((), \lambda_. \text{True}) \\ S_{\mathbb{B}} &::= (\mathbb{B}, \lambda s_1, s_2. s_2 = \text{true} \vee s_1 = \text{false}) \\ S_{\mathbb{N}} &::= (\mathbb{N}, \leq) \\ S_{\wp(A)} &::= (\wp(A), \subseteq) \\ S_{\text{List}(A)} &::= (\text{List}(A), \text{prefix}) \end{aligned}$$

The unit protocol  $S_{()}$  has a single state and a trivial order. The boolean protocol  $S_{\mathbb{B}}$  has two states **false** and **true** and can only grow from the former to the latter. The natural protocol  $S_{\mathbb{N}}$  has states as natural numbers and can only grow following the natural number order. The set protocol  $S_{\wp(A)}$  has states as sets of elements from the type  $A$ , and states can only grow by adding more elements. The list protocol  $S_{\text{List}(A)}$  has states as lists of elements from  $A$ , and lists can only grow by appending.

#### 17.1.1 Persistent Concurrent Protocols

The interface of persistent concurrent GPS protocols is given in [Figure 17.1](#) and [Figure 17.2](#).

**Definition 17.3** (Persistent Concurrent Protocol Assertion). The protocol assertion  $\boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}}$  says that the location  $\ell$  is persistently (permanently) governed by a GPS protocol *interpretation*  $\mathcal{I}$ , under the namespace  $\mathcal{N}$  and the ghost location  $\gamma$ .

- The namespace  $\mathcal{N}$  is picked by the creator (user) of the protocol to house the protocol invariant. As GPS protocols are built with Iris invariants, namespaces allow the user to atomically access multiple invariants (housed in disjoint namespaces). This ability is particularly important for deterministic pointer comparison in the semantics of ORC11: when performing a compare-and-swap of location values,<sup>2</sup> if the two compared values are locations governed by persistent concurrent protocols in disjoint namespaces, we know atomically that the two locations are both alive, and hence their comparison is deterministic.
- The ghost location  $\gamma$  is needed to store the ghost state of the protocol's model. It is important in the case the protocol is cancelable: multiple protocols can be tied to one location (but only at most one can be alive at any moment in time), the ghost location  $\gamma$  uniquely identifies a protocol instance. For persistent concurrent protocols that are not cancelable,  $\gamma$  never changes and can be safely ignored.

<sup>2</sup>for example, see [GPS-CON-CAS-LOC](#) in [Figure 17.2](#)

- The protocol interpretation  $\mathcal{I}$  is a pair of predicates  $(\mathcal{I}_r, \mathcal{I}_w)$  called the *read* and *write* interpretations, respectively. Both predicates are of the type  $(Loc \times GName \times Time \times T_S \times Val) \rightarrow \text{vProp}$ , where  $S$  is the protocol type picked by the user.  $\mathcal{I}_r(\ell, \gamma, t, s, v)$  represents the resources one would get from the protocol when reading the location's write message of timestamp  $t$  and value  $v$ . Meanwhile,  $\mathcal{I}_w(\ell, \gamma, t, s, v)$  represents the resources one needs to give up to the protocol when writing a message of timestamp  $t$  and value  $v$  to  $\ell$  and tying the state  $s$  to that message.
- In addition to asserting that the protocol exists persistently, the assertion  $\boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}}$  also says that the current thread has observed the write message of timestamp  $t$ , value  $v$ , and state  $s$ .

We can now look more closely at some of the rules for persistent concurrent protocols, first in [Figure 17.1](#) and then in [Figure 17.2](#). We explain first some simple rules.

- **GPS-CON-AGREE** says that the pre-order of protocol states are *total per protocol*, as it follows the timestamp order—which encodes the modification order **mo**—of a location's history.
- **GPS-CON-ATOM-PTSTO** says that, by knowing that a persistent protocol exists for  $\ell$ , we can *atomically*—due to the mask-changing fancy update—access the primitive atomic points-to  $\ell \mapsto h$  of  $\ell$  subjectively. This is sufficient to deduce that  $\ell$  is still alive, and can be useful in deterministic pointer comparison.
- **GPS-CON-INIT** allows one to allocate a new GPS protocol from a non-atomic points-to  $\ell \mapsto v$  and the write interpretation  $\mathcal{I}_w$  of the latest write (with value  $v$ ).

The rule **GPS-CON-READ** shows how persistent GPS protocols allow us to read the location  $\ell$  with the access mode  $o$ . Albeit looking simple, the rule requires a bit of explanation.

- The read must use an atomic access mode for read, *i.e.*,  $o \in \{\mathbf{rlx}, \mathbf{acq}\}$ .  $o$  can also be **sc**, but we do not provide a stronger rule for SC accesses in iRC11.
- The rule can be used with the mask  $\mathcal{E}$  that includes the protocol's namespace  $\mathcal{N}$ , *i.e.*, when the protocol's invariant is enabled.
- If the reading thread  $\pi$ 's observation for  $\ell$  was at least the message  $(t, s, v)$ , then the read will return a message  $(t', s', v')$  (and its observation) that is no-**mo**-earlier than  $t$ .
- Additionally, if the user can prove a fancy update  $\text{Extract}_{\mathcal{I}}$  that can extract the resource  $R(t', s', v')$  from either the read ( $\mathcal{I}_r$ ) or write ( $\mathcal{I}_w$ ) interpretation of the protocol for the read message  $t'$ , then the user can get back  $R$  in the post-condition.

$$\begin{array}{c}
\text{persistent}(\mathcal{R}(\ell, \gamma, t, s, v)) \\
\text{timeless}(\mathcal{R}(\ell, \gamma, t, s, v)) \\
\text{persistent}(\boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}})
\end{array}
\quad
\begin{array}{c}
\text{GPS-CON-RDR} \\
\boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} \vdash \mathcal{R}(\ell, \gamma, t, s, v)
\end{array}$$

$$\begin{array}{c}
\text{GPS-CON-AGREE} \\
\boxed{(\ell, \gamma) : (t_1, s_1, v_1) \mid \mathcal{I}}^{\mathcal{N}} * \boxed{(\ell, \gamma) : (t_2, s_2, v_2) \mid \mathcal{I}}^{\mathcal{N}} \Rightarrow_{\mathcal{E}} (t_1 \leq t_2 \Rightarrow s_1 \sqsubseteq s_2) \wedge (t_2 \leq t_1 \Rightarrow s_2 \sqsubseteq s_1)
\end{array}$$

$$\begin{array}{c}
\text{GPS-CON-ATOM-PTSTO} \\
\boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} \xRightarrow{\mathcal{E} \Rightarrow \mathcal{E} \uparrow \mathcal{N}} \exists h. \triangleright \langle \text{subj} \rangle \ell \mapsto h
\end{array}$$

$$\begin{array}{c}
\text{GPS-CON-INIT} \\
\ell \mapsto v * (\forall \gamma, t. \triangleright \mathcal{I}_w(\ell, \gamma, t, s, v)) \Rightarrow_{\mathcal{E}} \exists \gamma, t. \boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}}
\end{array}$$

$$\begin{array}{l}
\nabla_{\pi}^{o? \mathbf{rlx}} P ::= \mathbf{if} (o = \mathbf{rlx}) \mathbf{then} \nabla_{\pi} P \mathbf{else} P \\
\Delta_{\pi}^{o? \mathbf{rlx}} P ::= \mathbf{if} (o = \mathbf{rlx}) \mathbf{then} \Delta_{\pi} P \mathbf{else} P
\end{array}$$

$$\text{Extract}_{\mathcal{I}}(\ell, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N}) ::= \langle \text{obj} \rangle \wedge \begin{cases} \mathcal{I}_r(\ell, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \uparrow \mathcal{N}} \mathcal{I}_r(\ell, \gamma, t', s', v') * R(t', s', v') \\ \mathcal{I}_w(\ell, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \uparrow \mathcal{N}} \mathcal{I}_w(\ell, \gamma, t', s', v') * R(t', s', v') \end{cases}$$

$$\begin{array}{c}
\text{GPS-CON-READ} \\
\frac{\mathbf{rlx} \sqsubseteq o \quad \uparrow \mathcal{N} \subseteq \mathcal{E}}{\left\{ \boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} * \forall t' \geq t, s' \sqsupseteq s, v'. \text{Extract}_{\mathcal{I}}(\ell, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N}) \right\}} \\
*^o \ell \text{ in } \pi \\
\left\{ v'. \exists t' \geq t, s' \sqsupseteq s. \boxed{(\ell, \gamma) : (t', s', v') \mid \mathcal{I}}^{\mathcal{N}} * \nabla_{\pi}^{o? \mathbf{rlx}} R(t', s', v') \right\}_{\mathcal{E}}
\end{array}$$

$$\begin{array}{c}
\text{GPS-CON-WRITE} \\
\frac{\mathbf{rlx} \sqsubseteq o \quad \uparrow \mathcal{N} \subseteq \mathcal{E} \quad \forall s. s \sqsubseteq s'}{\left\{ \boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} * \Delta_{\pi}^{o? \mathbf{rlx}} (\forall t' > t. \mathcal{R}(\ell, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \uparrow \mathcal{N}} \mathcal{I}_w(\ell, \gamma, t', s', v')) \right\}} \\
\ell :=_o v' \text{ in } \pi \\
\left\{ \star. \exists t' > t. \boxed{(\ell, \gamma) : (t', s', v') \mid \mathcal{I}}^{\mathcal{N}} \right\}_{\mathcal{E}}
\end{array}$$

FIGURE 17.1: Rules for GPS Persistent Concurrent Protocols

- The extraction must be proven *objectively*,<sup>3</sup> without changing the interpretations.
- If the read access mode is at least **acq**, then  $R$  is acquired as-is. If the read is only relaxed (**rlx**),  $R$  will be returned protected by the acquire modality  $\nabla$ ,<sup>4</sup> which can later be eliminated by with an acquire fence. To succinctly encapsulate both cases in the rule, we use the conditional notation  $\nabla_{\pi}^{o?rlx}$  for the acquire modality.

<sup>3</sup>see §8.4<sup>4</sup>see §8.3

**GPS-CON-WRITE** allows us to write a value  $v'$  to  $\ell$  and tie the state  $s'$  to that write.

- The write access mode  $o$  should be **{rlx, rel}**.
- The value  $v'$  will be tied to the timestamp  $t'$  strictly greater than the writing thread's observation of timestamp  $t$  for  $\ell$ .
- As the protocol states need to grow with respect to the pre-order  $\sqsubseteq$ , the user needs to guarantee that the new state  $s'$  is greater than the protocol's current state. However, as the protocol allows concurrent writes, it is difficult to know what the current state is. Consequently, the rule requires the user to prove that  $s'$  is greater than any state ( $\forall s. s \sqsubseteq s'$ ). This is quite a strong requirement, and expectedly, protocols for arbitrarily concurrent locations are often trivial.
- Most importantly, the user has to provide the *write interpretation*  $\mathcal{I}_w$  for the new write message  $(t', s', v')$ . If the write access mode is at least **rel**, then the user only has to provide  $\mathcal{I}_w$  right before the write. If the mode is relaxed, then the user has to provide  $\mathcal{I}_w$  at the most recent release fence, i.e., provide  $\mathcal{I}_w$  under the release modality  $\Delta$ .<sup>5</sup> Similarly to the read rule, we use the conditional notation  $\Delta_{\pi}^{o?rlx}$  here to combine the two cases.
- Additionally, in proving  $\mathcal{I}_w$ , the user can assume the *pure-ghost* observation  $\mathcal{R}(\ell, \gamma, t', s', v')$  of the new message  $v'$  with the state  $s'$ . That is, the write interpretation can include the fact that the write has been registered in the protocol. Intuitively, the observation  $\mathcal{R}(\ell, \gamma, t', s', v')$  is simply the assertion  $\boxed{(\ell, \gamma) : (t', s', v') \mid \mathcal{I}}^{\mathcal{N}}$  without asserting the existence of the protocol invariant. As such,  $\mathcal{R}$  is both persistent and timeless. The relation between the two observations is shown in the rule **GPS-CON-RDR**.

<sup>5</sup>see §8.3

We now look at the rule **GPS-CON-CAS-INT** in Figure 17.2, which allows us to perform a compare-exchange from an *integer* value  $v_r$  to a value  $v_w$ .

- Recall that  $o_r$  and  $o_w$  are respectively the read and write access modes in the successful exchange case. Meanwhile,  $o_f$  is the read access mode in the failure case. Like the read and write rules, these access modes must be at least **rlx**, and dictate whether the resources the user needs to provide or can acquire will be protected by the release or acquire modality, respectively. As in the read and

## GPS-CON-CAS-INT

$$\begin{array}{c}
\mathbf{rlx} \sqsubseteq o_f, o_r, o_w \quad \uparrow \mathcal{N} \subseteq \mathcal{E} \quad v_r \in \mathbb{Z} \\
\hline
\left[ \begin{array}{l}
\boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} * \\
\triangleright \langle \text{obj} \rangle (\forall t' \geq t, s' \sqsupseteq s, v'. (\mathcal{I}_r(\ell, \gamma, t', s', v') \vee \mathcal{I}_w(\ell, \gamma, t', s', v')) \multimap \vdash v' =^? v_r) * \\
\Delta_{\pi}^{o_w ? \mathbf{rlx}} P * \\
\Delta_{\pi}^{o_w ? \mathbf{rlx}} \forall t' \geq t, s' \sqsupseteq s. \wedge \left\{ \begin{array}{l}
\triangleright (\forall v' \neq v_r. \text{Extract}_{\mathcal{I}}(\ell, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N})) \\
\langle \text{obj} \rangle (\triangleright \mathcal{I}_w(\ell, \gamma, t', s', v_r) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} (\triangleright Q_1(t', s') * \triangleright Q_2(t', s'))) \\
(P * \triangleright Q_2(t', s')) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \exists s'' \sqsupseteq s'. \forall t'' > t'. \\
\triangleright \mathcal{R}(\ell, \gamma, t'', s'', v_w) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \\
* \left\{ \begin{array}{l}
\langle \text{obj} \rangle (\triangleright Q_1(t', s') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_r(\ell, \gamma, t', s', v')) \\
\vdash_{\mathcal{E} \setminus \uparrow \mathcal{N}} \triangleright \vdash_{\mathcal{E} \setminus \uparrow \mathcal{N}} (Q(t'', s'') * \mathcal{I}_w(\ell, \gamma, t'', s'', v_w))
\end{array} \right.
\end{array} \right. \\
\text{CAS}^{o_f, o_r, o_w}(\ell, v_r, v_w) \text{ in } \pi
\end{array} \right. \\
\left\{ b. \exists t', v', s' \sqsupseteq s. \vee \left\{ \begin{array}{l}
v_r \neq v' * t \leq t' * \boxed{(\ell, \gamma) : (t', s', v')}^{\mathcal{N}} * \nabla_{\pi}^{o_f ? \mathbf{rlx}} R(t', s', v') * \nabla_{\pi}^{o_w ? \mathbf{rlx}} P \quad b = \text{false} \\
v_r = v' * t < t' * \boxed{(\ell, \gamma) : (t', s', v_w)}^{\mathcal{N}} * \nabla_{\pi}^{o_r ? \mathbf{rlx}} Q(t', s') \quad b = \text{true}
\end{array} \right\} \right\}_{\mathcal{E}}
\end{array}$$

## GPS-CON-CAS-LOC

$$\begin{array}{c}
\mathbf{rlx} \sqsubseteq o_f, o_r, o_w \quad \uparrow \mathcal{N} \subseteq \mathcal{E} \quad \ell_r \in \text{Loc} \\
\hline
\left[ \begin{array}{l}
\boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} * \\
P_{\text{cmp}} * \square (P_{\text{cmp}} \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \exists q, h_r. \triangleright \langle \text{subj} \rangle \ell_r \stackrel{q}{\mapsto} h_r) * \\
\triangleright \langle \text{obj} \rangle (\forall t' \geq t, s' \sqsupseteq s, v'. (\mathcal{I}_r(\ell, \gamma, t', s', v') \vee \mathcal{I}_w(\ell, \gamma, t', s', v')) \multimap \vdash v' =^? \ell_r) * \\
\Delta_{\pi}^{o_w ? \mathbf{rlx}} P * \\
\Delta_{\pi}^{o_w ? \mathbf{rlx}} \forall t' \geq t, s' \sqsupseteq s. \wedge \left\{ \begin{array}{l}
\triangleright (\forall v' \neq \ell_r. \text{Extract}_{\mathcal{I}}(\ell, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N})) \\
\langle \text{obj} \rangle (\forall \ell' \neq \ell_r. \triangleright \mathcal{I}_w(\ell, \gamma, t', s', \ell') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \exists q, h'. \triangleright \ell' \stackrel{q}{\mapsto} h') \\
\langle \text{obj} \rangle (\triangleright \mathcal{I}_w(\ell, \gamma, t', s', \ell_r) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} (\triangleright Q_1(t', s') * \triangleright Q_2(t', s'))) \\
(P * \triangleright Q_2(t', s')) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \exists s'' \sqsupseteq s'. \forall t'' > t'. \\
\triangleright \mathcal{R}(\ell, \gamma, t'', s'', v_w) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \\
* \left\{ \begin{array}{l}
\langle \text{obj} \rangle (\triangleright Q_1(t', s') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_r(\ell, \gamma, t', s', v')) \\
\vdash_{\mathcal{E} \setminus \uparrow \mathcal{N}} \triangleright \vdash_{\mathcal{E} \setminus \uparrow \mathcal{N}} (Q(t'', s'') * \mathcal{I}_w(\ell, \gamma, t'', s'', v_w))
\end{array} \right.
\end{array} \right. \\
\text{CAS}^{o_f, o_r, o_w}(\ell, \ell_r, v_w) \text{ in } \pi
\end{array} \right. \\
\left\{ b. \exists t', v', s' \sqsupseteq s. P_{\text{cmp}} * \vee \left\{ \begin{array}{l}
\ell_r \neq v' * t \leq t' * \boxed{(\ell, \gamma) : (t', s', v')}^{\mathcal{N}} * \nabla_{\pi}^{o_f ? \mathbf{rlx}} R(t', s', v') * \Delta_{\pi}^{o_w ? \mathbf{rlx}} P \quad b = \text{false} \\
\ell_r = v' * t < t' * \boxed{(\ell, \gamma) : (t', s', v_w)}^{\mathcal{N}} * \nabla_{\pi}^{o_r ? \mathbf{rlx}} Q(t', s') \quad b = \text{true}
\end{array} \right\} \right\}_{\mathcal{E}}
\end{array}$$

FIGURE 17.2: CAS Rules for GPS Persistent Concurrent Protocols



write rules, we use the conditional notations  $\Delta_{\pi}^{o?rlx}$  and  $\nabla_{\pi}^{o?rlx}$  to denote these requirements.

- Putting the usual pre-condition of the protocol assertion aside, the first pre-condition requires the user to show, objectively, that any message  $(t', s', v')$  the CAS can read from would have the value  $v'$  that is *comparable* with the expected integer  $v_r$ , i.e.,  $\vdash v' =? v_r$ .<sup>6</sup> In proving comparability, the user can assume more resources, specifically the read or write interpretation about the read message  $(t', s', v')$ .

<sup>6</sup>see Definition 4.9

- As the second pre-condition, the user is required to pick and prove some predicate  $P$ —either now or at the most recent release fence—that will be released with the write effect if the CAS succeeds ( $b = \mathbf{true}$  in the post-condition). In case the CAS fails ( $b = \mathbf{false}$ ),  $P$  is returned unchanged in the post-condition.
- The last pre-condition, also needed to be proven either now or at the most recent release fence depending on  $o_w$ , requires that the user can resolve two obligations that represent the failure and success cases of the CAS, and that consequently are tied together by the classical conjunction  $\wedge$ .

(i) If the CAS fails, the situation is similar to that of a read with the mode  $o_f$ : the user should prove a fancy update  $\text{Extract}_{\mathcal{I}}$  that can extract the resource  $R(t', s', v')$  from either the read ( $\mathcal{I}_r$ ) or write ( $\mathcal{I}_w$ ) interpretation for the read message  $t'$ , whose value  $v'$  is definitely not the expected integer  $v_r$ .

(ii) If the CAS succeeds, it has a combined effect of both a read of the message  $(t', s', v_r)$  and a write of the message  $(t'', s'', v_w)$ . In this obligation, the user can consume the write interpretation  $\mathcal{I}_w$  of the *read* message to produce the write interpretation  $\mathcal{I}_w$  of the new write message. The obligation is split into several steps:

- first, the user shows, objectively, that the write interpretation  $\mathcal{I}_w$  of the read message  $(t', s', v_r)$  can be split into two resources  $Q_1$  and  $Q_2$ ;
- second, the user shows that, with the pre-condition  $P$  and the 2nd part  $Q_2$ , the user can *pick* the new state  $s''$  that extends  $s'$  for the new write of timestamp  $t''$ ;
- finally; knowing the observation  $\mathcal{R}(\ell, \gamma, t'', s'', v_w)$  that the protocol has been extended with the new state, the user proves both (a) the read interpretation  $\mathcal{I}_r$  for the read message, assuming  $Q_1$ , and (b) the write interpretation  $\mathcal{I}_w$  for the new write message  $(t'', s'', v_w)$ , assuming the rest of the resources. If there are still resources left, the user can choose to keep them in  $Q$ , which will be returned in the post-condition when the CAS succeeds ( $b = \mathbf{true}$ ).

- The later and fancy updates are put in positions that provide maximum flexibility for the user in accessing invariants, performing ghost updates, and eliminating later.
- The timestamp  $t'$  and state  $s'$  returned in the post-condition correspond, in the CAS failure case, to the read message's timestamp  $t'$  and  $s'$ , and in the CAS success case, to the write message's timestamp  $t''$  and state  $s''$ .

The rule **GPS-CON-CAS-LOC** (Figure 17.2) is very similar to **GPS-CON-CAS-INT**, except that it allows to use a *pointer* (location)  $\ell_r$  as the expected value for the comparison. Consequently, the rule has extra obligations to achieve deterministic pointer comparison. Specifically, the rule now requires, as the first condition, some resource  $P_{\text{cmp}}$  that can be used to show that  $\ell_r$  is alive, by showing its primitive atomic points-to  $\ell_r \mapsto h_r$ , subjectively. The resource  $P_{\text{cmp}}$ , however, is only used to for this purpose and will not be consumed, and will be returned unchanged in the post-condition. Furthermore, the last pre-condition requires the user to show that the location value  $\ell'$  that is to be compared with  $\ell_r$  is also alive, also by showing its primitive atomic points-to. Note that, interestingly, this requirement only applies for  $\ell'$  and  $\ell_r$  that are *definitionally* unequal, because definitionally equal locations are always compared equal and never unequal,<sup>7</sup> regardless of their liveness status.

<sup>7</sup>see Definition 4.7 and Definition 4.8

### 17.1.2 Cancelable Single-Writer Protocols

As can be seen from **GPS-CON-WRITE**, allowing arbitrarily concurrent writes results in a weak write rule which is often only applicable to a trivial protocol order, because it is impossible to know what the current up-to-date state of the protocol is. In more interesting protocols, one needs more control on how states can change, *i.e.*, how writes can happen. To provide stronger reasoning principles for some of such interesting scenarios, iGPS<sup>8</sup> introduces *single-writer* protocols, where only one thread can write to the location, while other threads can concurrently read from it. Intuitively, the setup is that writes can only be performed with the unique *writer permission*, which must be transferred explicitly among multiple threads to write to it, one at a time. As such, the writer always knows exactly what the current state of the protocol is.

iRC11's single-writer protocols build upon those of iGPS, but extends the notion of single-writer to also include *CAS-only* accesses: when multiple threads concurrently try to write to the same location  $\ell$ , if they can resolve their contentions by using only CASes, then the protocol still maintains a single writer, albeit only *atomically*. In the following, we present the *cancelable* variant of iRC11 single-writer protocols, *i.e.*, they also allow the user to switch protocols or switch to non-atomic reasoning if need be.

**Definition 17.4** (Cancelable Single-Writer Protocol Assertions). There are four kinds of assertions.

<sup>8</sup>Kaiser et al., “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris” [Kai+17].

- The reader assertion  $\boxed{(\ell, \gamma_i, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}}$  says that a cancelable protocol instance  $\gamma$  with the interpretation  $\mathcal{I}$  has been established for the location  $\ell$ , under the namespace  $\mathcal{N}$ . It is similar to the assertion  $\boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}}$  of persistent concurrent protocols,<sup>9</sup> as it also says that the current thread has observed the write message of timestamp  $t$ , value  $v$ , and state  $s$  of the protocol instance  $\gamma$ . However, it says nothing about the *liveness* of that instance.
- The cancelable token  $\heartsuit_q^{\gamma_i}$  asserts that the protocol has not been canceled, *i.e.*, it is still alive. The token is tied to other protocol assertions by the ghost location  $\gamma_i$ , and like the cancelable invariant token,<sup>10</sup> it is needed for any access to the location  $\ell$  through the protocol instance  $\gamma$  that is tied to  $\gamma_i$ .<sup>11</sup>
- The single-writer assertion  $\boxed{(\ell, \gamma_i, \gamma) : (t, s, v) \mid \mathcal{I}}_W^{\mathcal{N}}$  asserts the unique permission to perform writes on  $\ell$ . It additionally says that the latest write to  $\ell$  is exactly the message  $(t, s, v)$ , and that the owner thread has observed that write.
- The CAS-only assertion  $\boxed{(\ell, \gamma_i, \gamma) : (t, s, v) \mid \mathcal{I}}_q^{\mathcal{N}}$  asserts the permission to perform CASes on  $\ell$ . The assertion is fractional, as it is indexed by a fraction  $q$ . Therefore it can be split and joined, so as to allow multiple threads to concurrently perform CASes on  $\ell$ . However, the CAS-only assertion cannot co-exist with the single-writer assertion, enforcing that the protocol is in the CAS-only mode, where writes can only be done with CASes.
- Last but not least, in order to support a strong single-writer rule, we need to extend the protocol interpretation  $\mathcal{I}$  into a triple of predicates  $(\mathcal{I}_r, \mathcal{I}_w, \mathcal{I}_m)$  which are the read, write, and *moved* interpretations, respectively. The new moved interpretation  $\mathcal{I}_m$  denotes the *leftover* resources of a write message that has been overwritten by a single-writer write. We will see the use of this new interpretation in the single-writer write rule **GPS-SW-WRITE-REL** in Figure 17.4.

Some of the relations among the assertions can be found in Figure 17.3. For example, **GPS-SW-W** and **GPS-SW-RSHR** respectively show that the single-writer assertion and the CAS-only assertion both imply the reader assertion, so they can both be used to read from the location  $\ell$ . **GPS-SW-W-WSHR-RSHR** shows how to convert the single-writer assertion into the full fraction of the CAS-only assertion, effectively switching the protocol from the single-writer mode to the CAS-only mode. Dually, **GPS-SW-W-REVERT** shows how to convert the full fraction of the CAS-only assertion back to the single-writer assertion. To fully understand the rules in Figure 17.3, we need to introduce some new auxillary assertions.

**Definition 17.5** (Auxillary Single-Writer Protocol Assertions). The auxillary assertions are all pure ghost-state assertions, so they are *timeless* and can be put into invariants and can bypass all step-indexing restrictions (manifested in the later modality). In fact, they are only needed specifically for this purpose.

<sup>9</sup>see Definition 17.3

<sup>10</sup>see §11.2.1

<sup>11</sup>Consequently, the pair  $(\gamma_i, \gamma)$  uniquely identifies the protocol instance, and we should always use the pair as a single name for the instance. However, here we choose to state the pair explicitly, to easily show the similarity with the persistent concurrent protocol assertion (through  $\gamma$ ) and the cancelable invariant assertions (through  $\gamma_i$ ).

$$\begin{array}{c}
\text{persistent}(\mathcal{R}(\ell, \gamma, t, s, v)) \quad \text{timeless}(\mathcal{W}(\ell, \gamma, t, s, v)) \\
\text{timeless}(\mathcal{R}(\ell, \gamma, t, s, v)) \quad \text{timeless}(\mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v)) \\
\text{persistent}\left(\left[\frac{(\ell, \gamma_i, \gamma) : (t, s, v)}{\mathcal{I}}\right]^{\mathcal{N}}\right) \quad \text{timeless}(\mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v))
\end{array}$$

$$\begin{array}{c}
\text{GPS-SW-W} \\
\left[\frac{(\ell, \gamma_i, \gamma) : (t, s, v)}{\mathcal{I}}\right]^{\mathcal{N}}_W \dashv\vdash \mathcal{W}(\ell, \gamma, t, s, v) * \left[\frac{(\ell, \gamma_i, \gamma) : (t, s, v)}{\mathcal{I}}\right]^{\mathcal{N}}
\end{array}$$

$$\begin{array}{c}
\text{GPS-SW-RSHR} \\
\left[\frac{(\ell, \gamma_i, \gamma) : (t, s, v)}{\mathcal{I}}\right]^{\mathcal{N}}_q \dashv\vdash \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v) * \left[\frac{(\ell, \gamma_i, \gamma) : (t, s, v)}{\mathcal{I}}\right]^{\mathcal{N}}
\end{array}$$

$$\begin{array}{c}
\text{GPS-SW-R} \\
\left[\frac{(\ell, \gamma_i, \gamma) : (t, s, v)}{\mathcal{I}}\right]^{\mathcal{N}} \vdash \mathcal{R}(\ell, \gamma, t, s, v)
\end{array}$$

$$\begin{array}{ccc}
\text{GPS-W-R} & \text{GPS-WSHR-R} & \text{GPS-RSHR-R} \\
\mathcal{W}(\ell, \gamma, t, s, v) \vdash \mathcal{R}(\ell, \gamma, t, s, v) & \mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v) \vdash \mathcal{R}(\ell, \gamma, t, s, v) & \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v) \vdash \mathcal{R}(\ell, \gamma, t, s, v)
\end{array}$$

$$\begin{array}{ccc}
\text{GPS-W-EXCL} & & \text{GPS-WSHR-EXCL} \\
\mathcal{W}(\ell, \gamma, t, s, v) * \mathcal{W}(\ell, \gamma, t', s', v') \vdash \text{False} & & \mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v) * \mathcal{W}_{\text{shr}}(\ell, \gamma, t', s', v') \vdash \text{False}
\end{array}$$

$$\begin{array}{ccc}
\text{GPS-W-WSHR-EXCL} & & \text{GPS-W-RSHR-EXCL} \\
\mathcal{W}(\ell, \gamma, t, s, v) * \mathcal{W}_{\text{shr}}(\ell, \gamma, t', s', v') \vdash \text{False} & & \mathcal{W}(\ell, \gamma, t, s, v) * \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t', s', v') \vdash \text{False}
\end{array}$$

$$\begin{array}{c}
\text{GPS-RSHR-FRAC-1} \\
\mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v) * \langle \text{subj} \rangle \mathcal{R}_{\text{shr}}^{q'}(\ell, \gamma, t', s', v') \vdash \mathcal{R}_{\text{shr}}^{q+q'}(\ell, \gamma, t, s, v)
\end{array}$$

$$\begin{array}{c}
\text{GPS-RSHR-FRAC-2} \\
\mathcal{R}_{\text{shr}}^{q+q'}(\ell, \gamma, t, s, v) \vdash \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v) * \mathcal{R}_{\text{shr}}^{q'}(\ell, \gamma, t, s, v)
\end{array}$$

$$\begin{array}{c}
\text{GPS-R-RSHR-JOIN} \\
\mathcal{R}(\ell, \gamma, t, s, v) * \langle \text{subj} \rangle \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t', s', v') \vdash \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v)
\end{array}$$

$$\begin{array}{c}
\text{GPS-WSHR-RSHR-UPDATE} \\
\mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v) * \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t', s', v') \vdash \mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v) * \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v)
\end{array}$$

$$\begin{array}{c}
\text{GPS-W-WSHR-RSHR} \\
\mathcal{W}(\ell, \gamma, t, s, v) \vdash \mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v) * \mathcal{R}_{\text{shr}}^1(\ell, \gamma, t, s, v)
\end{array}$$

$$\begin{array}{c}
\text{GPS-SW-W-WSHR-RSHR} \\
\left[\frac{(\ell, \gamma_i, \gamma) : (t, s, v)}{\mathcal{I}}\right]^{\mathcal{N}}_W \vdash \mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v) * \left[\frac{(\ell, \gamma_i, \gamma) : (t, s, v)}{\mathcal{I}}\right]^{\mathcal{N}}_1
\end{array}$$

$$\begin{array}{c}
\text{GPS-SW-W-REVERT} \\
\frac{\uparrow \mathcal{N} \subseteq \mathcal{E}}{\heartsuit_q^{\gamma_i} * \mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v) * \left[\frac{(\ell, \gamma_i, \gamma) : (t', s', v')}{\mathcal{I}}\right]^{\mathcal{N}}_1 \Rightarrow_{\mathcal{E}} \heartsuit_q^{\gamma_i} * \left[\frac{(\ell, \gamma_i, \gamma) : (t, s, v)}{\mathcal{I}}\right]^{\mathcal{N}}_W}
\end{array}$$

FIGURE 17.3: Rules for auxillary assertions of GPS Single-Writer Protocols

- The read observation  $\mathcal{R}(\ell, \gamma, t, s, v)$ , which we already see in the rule **GPS-CON-WRITE**, is persistent and asserts that the message  $(t, s, v)$  has been registered in the protocol instance  $\gamma$ .
- The single-writer permission  $\mathcal{W}(\ell, \gamma, t, s, v)$  is the unique ghost permission needed to perform writes when the protocol is in single-writer mode.
- The shared-writer permission  $\mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v)$  and the shared-reader permission  $\mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v)$  are needed to perform CASes when the protocol is in CAS-only mode. Intuitively, every participant locally owns a fraction of the shared-reader permission and globally shares the shared-writer permission in order to perform CASes.

We can now explain the rules in **Figure 17.3** in more details. **GPS-SW-W** says explicitly that the single-writer assertion is in fact the ghost single-writer permission combined with the reader assertion—the ghost single-writer permission maintains that the protocol is in the single-writer mode and that the observed message  $(t, s, v)$  is the latest write. Meanwhile **GPS-SW-RSHR** says that the CAS-only assertion is the ghost shared-reader permission combined with the reader assertion—the ghost shared-reader permission maintains that the protocol is still in CAS-only mode. Furthermore, **GPS-SW-R** says that the reader assertion simply implies the read observation (in addition to the fact that an invariant exists for the protocol). The rules **GPS-W-R**, **GPS-WSHR-R**, and **GPS-RSHR-R** say that all ghost permissions always include the read observation of their message  $(t, s, v)$ .

**GPS-W-EXCL** and **GPS-WSHR-EXCL** respectively say that the ghost single-writer permission and shared-writer permission are exclusive. **GPS-W-WSHR-EXCL** and **GPS-W-RSHR-EXCL** say that the single-writer permission cannot co-exist with the shared-writer permission or the shared-reader permission, because the protocol can only be either in the single-writer mode or in the CAS-only mode at a time. **GPS-RSHR-FRAC-1** and **GPS-RSHR-FRAC-2** say that the shared-reader permission can be split and join fractionally. **GPS-R-RSHR-JOIN** says that we can update the shared-reader permission  $\mathcal{R}_{\text{shr}}^q(\ell, \gamma, t', s', v')$  with the local observation  $\mathcal{R}q(\ell, \gamma, t, s, v)$ . Meanwhile, **GPS-WSHR-RSHR-UPDATE** says that we can update the observation of the shared-reader permission with that of the shared-writer permission, which always carries the latest write to the protocol.

The rule **GPS-W-WSHR-RSHR** crucially shows how we can convert the single-writer permission into the shared-writer permission and the full fraction of the shared-reader permission. This is the key rule to switch the protocol from the single-writer mode to the CAS-only mode, and in fact it establishes the soundness of **GPS-SW-W-WSHR-RSHR**, which applies the switch at the level of protocol assertions. The reverse switch—from CAS-only mode back to single-writer mode—is stated in **GPS-SW-W-REVERT** and additionally requires a fancy update as well as a fraction of cancelable token  $\heartsuit_q^{\gamma_i}$ . We need these two facilities because the reverse switch needs to access the internal invariant that supports the model of cancelable

single-writer protocols. We will explain this in more details when we look at the model of cancelable single-writer protocols in §17.2.

We now turn our eyes to an excerpt of the *operational* rules for cancelable single-writer protocols in Figure 17.4.

- **GPS-SW-INIT** allows us to allocate a new cancelable single-writer GPS protocol for  $\ell$  provided the non-atomic points-to  $\ell \mapsto v$  and the write interpretation  $\mathcal{I}_w$  for the latest message  $(t, s, v)$ . The rule is very similar to the rule **GPS-CON-INIT** of persistent concurrent protocols, but in this case we get back the single-writer assertion as well as the *full* cancelable token  $\heartsuit_1^{\gamma_i}$ .
- **GPS-SW-READ** shows how we can read from the location using only the reader assertion. Again, this is almost the same as **GPS-CON-READ**, except that we need a fraction  $\heartsuit_q^{\gamma_i}$  of the token to know that the protocol instance is still alive. **GPS-SW-WRITER-READ** shows a stronger read rule with the single-writer assertion: we always read the latest write message.
- **GPS-SW-WRITE-REL** is a release write rule with the single-writer assertion. The rule is much stronger than the concurrent write rule **GPS-CON-WRITE**,<sup>12</sup> because it gets access to the latest write. More specifically, we can use the write interpretation  $\mathcal{I}_w$  of the latest write  $(t, s, v)$  to establish the write interpretation  $\mathcal{I}_w$  of the *next* latest write  $(t', s', v')$ , as follows.
  - The protocol state must grow, i.e.,  $s \sqsubseteq s'$ .
  - $\mathcal{I}_w$  of  $(t, s, v)$  can be split into  $Q_1$  and  $Q_2$ , where  $Q_1$  is the *leftover* resource that can be used to establish the new *moved* interpretation  $\mathcal{I}_m$  of the overwritten message  $(t, s, v)$ .
  - $Q_2$  can be used to establish the write interpretation  $\mathcal{I}_w$  of the next latest write  $(t', s', v')$ .
  - Additionally, the ghost writer permission  $\mathcal{W}(\ell, \gamma, t', s', v')$  (updated with the next latest write) as well as the cancelable token  $\heartsuit_q^{\gamma_i}$  (needed to know that the protocol is still alive) can also be used for  $\mathcal{I}_w$  of the next write  $(t', s', v')$ .
  - Any unused resources can be returned in  $Q(t')$ . For example, if the ghost writer permission  $\mathcal{W}(\ell, \gamma, t', s', v')$  is returned in  $Q(t')$ , we can get back the writer assertion with the updated latest write, i.e.,  $\boxed{(\ell, \gamma_i, \gamma) : (t', s', v') \mid \mathcal{I}}_w^{\mathcal{N}}$ . The same also applies to the cancelable fraction  $\heartsuit_q^{\gamma_i}$ .
- **GPS-SW-DEALLOC** is the cancellation rule for the single-writer protocol. It is derived from **CINV-CANCEL**,<sup>13</sup> so it naturally requires the *full* cancelable token. The rule then will return the full non-atomic points-to of the location, as well as the write interpretation  $\mathcal{I}_w$  for the latest write. Note that it is sufficient to have only the reader assertion of the protocol to cancel it. However, we also have a variant of the rule (elided here) with the writer assertion, where we know that we will get back the non-atomic points-to with  $v$

<sup>12</sup>see Figure 17.1. In fact, it has the flavor of a CAS rule, like **GPS-CON-CAS-INT** (in Figure 17.2).

<sup>13</sup>see Chapter 11

GPS-SW-INIT

$$\ell \mapsto v * (\forall \gamma_i, \gamma, t. \triangleright \mathcal{I}_w(\ell, \gamma_i, \gamma, t, s, v)) \Rightarrow_{\mathcal{E}} \exists \gamma_i, \gamma, t. \heartsuit_1^{\gamma_i} * \boxed{(\ell, \gamma_i, \gamma) : (t, s, v)} \boxed{\mathcal{I}}_W^{\mathcal{N}}$$

$$\text{Extract}_{\mathcal{I}}(\ell, \gamma_i, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N}) ::= \langle \text{obj} \rangle \wedge \begin{cases} \mathcal{I}_r(\ell, \gamma_i, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_r(\ell, \gamma_i, \gamma, t', s', v') * R(t', s', v') \\ \mathcal{I}_w(\ell, \gamma_i, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_w(\ell, \gamma_i, \gamma, t', s', v') * R(t', s', v') \\ \mathcal{I}_m(\ell, \gamma_i, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_m(\ell, \gamma_i, \gamma, t', s', v') * R(t', s', v') \end{cases}$$

GPS-SW-READ

$$\frac{\mathbf{rlx} \sqsubseteq o \quad \uparrow \mathcal{N} \subseteq \mathcal{E}}{\left\{ \heartsuit_q^{\gamma_i} * \boxed{(\ell, \gamma_i, \gamma) : (t, s, v)} \boxed{\mathcal{I}}_W^{\mathcal{N}} * (\forall t' \geq t, s' \sqsupseteq s, v'. \text{Extract}_{\mathcal{I}}(\ell, \gamma_i, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N})) \right\}} \\ *o \ell \text{ in } \pi \\ \left\{ v'. \exists t' \geq t, s' \sqsupseteq s. \heartsuit_q^{\gamma_i} * \boxed{(\ell, \gamma_i, \gamma) : (t', s', v')} \boxed{\mathcal{I}}_W^{\mathcal{N}} * \nabla_{\pi}^{o? \mathbf{rlx}} R(t', s', v') \right\}_{\mathcal{E}}$$

GPS-SW-WRITER-READ

$$\frac{\mathbf{rlx} \sqsubseteq o \quad \uparrow \mathcal{N} \subseteq \mathcal{E}}{\left\{ \heartsuit_q^{\gamma_i} * \boxed{(\ell, \gamma_i, \gamma) : (t, s, v)} \boxed{\mathcal{I}}_W^{\mathcal{N}} * \langle \text{obj} \rangle (\mathcal{I}_w(\ell, \gamma_i, \gamma, t, s, v) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_w(\ell, \gamma_i, \gamma, t, s, v) * R) \right\}} \\ *o \ell \text{ in } \pi \\ \left\{ v. \heartsuit_q^{\gamma_i} * \boxed{(\ell, \gamma_i, \gamma) : (t, s, v)} \boxed{\mathcal{I}}_W^{\mathcal{N}} * R \right\}_{\mathcal{E}}$$

GPS-SW-WRITE-REL

$$\frac{\uparrow \mathcal{N} \subseteq \mathcal{E} \quad s \sqsubseteq s'}{\left\{ \heartsuit_q^{\gamma_i} * \boxed{(\ell, \gamma_i, \gamma) : (t, s, v)} \boxed{\mathcal{I}}_W^{\mathcal{N}} * \triangleright \langle \text{obj} \rangle (\mathcal{I}_w(\ell, \gamma_i, \gamma, t, s, v) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} Q_1 * Q_2) * \right.} \\ \left. \forall t' > t. (\mathcal{W}(\ell, \gamma, t', s', v') * Q_2 * \heartsuit_q^{\gamma_i}) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} * \left\{ \langle \text{obj} \rangle Q_1 \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_m(\ell, \gamma_i, \gamma, t, s, v) \right. \right.} \\ \left. \left. \models_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_w(\ell, \gamma_i, \gamma, t', s', v') * Q(t') \right\} \right\} \\ \ell :=_{\mathbf{rel}} v' \text{ in } \pi \\ \left\{ \heartsuit_q^{\gamma_i} * \boxed{(\ell, \gamma_i, \gamma) : (t', s', v')} \boxed{\mathcal{I}}_W^{\mathcal{N}} * Q(t') \right\}_{\mathcal{E}}$$

GPS-SW-DEALLOC

$$\frac{\uparrow \mathcal{N} \subseteq \mathcal{E}}{\heartsuit_1^{\gamma_i} * \boxed{(\ell, \gamma_i, \gamma) : (t', s', v')} \boxed{\mathcal{I}}_W^{\mathcal{N}} \Rightarrow_{\mathcal{E}} \exists t, s, v. \ell \mapsto v * \triangleright \mathcal{I}_w(\ell, \gamma_i, \gamma, t, s, v)}$$

GPS-SW-READ-ACQ-DEALLOC

$$\frac{\mathbf{rlx} \sqsubseteq o \quad \uparrow \mathcal{N} \subseteq \mathcal{E}}{\left\{ \heartsuit_q^{\gamma_i} * \boxed{(\ell, \gamma_i, \gamma) : (t, s, v)} \boxed{\mathcal{I}}_W^{\mathcal{N}} * (\forall t' \geq t, s' \sqsupseteq s, v'. \text{Extract}_{\mathcal{I}}(\ell, \gamma_i, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N})) * \right.} \\ \left. P * (\forall t' \geq t, s' \sqsupseteq s, v'. (P * \heartsuit_q^{\gamma_i} * R(t', s', v_d)) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \heartsuit_1^{\gamma_i} * Q(t', s')) \right\} \\ * \mathbf{acq} \ell \text{ in } \pi \\ \left\{ v'. \exists t' \geq t, s' \sqsupseteq s. \boxed{(\ell, \gamma_i, \gamma) : (t', s', v')} \boxed{\mathcal{I}}_W^{\mathcal{N}} * \begin{cases} Q(t', s') * \exists t, s, v. \ell \mapsto v * \triangleright \mathcal{I}_w(\ell, \gamma_i, \gamma, t, s, v) & v' = v_d \\ \heartsuit_q^{\gamma_i} * R(t', s', v') * P & v' \neq v_d \end{cases} \right\}_{\mathcal{E}}$$

FIGURE 17.4: Selected rules for Cancellable Single-Writer GPS Protocols

being the latest write from the writer assertion (instead of being existentially quantified here).

- **GPS-SW-READ-ACQ-DEALLOC** is a very specific read rule that extends **GPS-SW-READ** with the ability to cancel the protocol if it happens to be the case that the reader is the *only* party accessing the protocol. To do so, the user first picks a special value  $v_d$  that signals the end-of-life of the protocol. That is, the reader should prepare to cancel the protocol if it happens to read  $v_d$ . Specifically, the reader should provide as extra pre-conditions (1) some resource  $P$  and (2) a proof that  $P$  together with the cancelable fraction  $\heartsuit_q^{\gamma_i}$  (used for the read) *and* the acquired resource  $R(t', s', v_d)$  can be used to reconstruct the full cancelable token  $\heartsuit_1^{\gamma_i}$ . In that case, the cancelation is performed in the same way as **GPS-SW-DEALLOC**: we get back the full non-atomic points-to of the location and the latest write interpretation  $\mathcal{I}_w$ . Furthermore, we also get out whatever leftover resource  $Q(t', s')$  that comes out of (2). In case that we do not read the signal value  $v_d$ , we simply get back the acquired resource  $R(t', s', v')$  with the original  $P$  and the cancelable fraction.

### 17.1.3 Atomic-Borrows-based Protocols

Finally, we introduce a variant of GPS single-writer protocols that is tied to a lifetime  $\kappa$ , who effectively is the upper-bound of the protocol's lifetime.

**Definition 17.6** (Atomic-Borrows-based Single-Writer Protocol Assertions). There are three kinds of assertions that mirror those of cancelable single-writer protocols. They all simply tie the protocol to a lifetime  $\kappa$ , instead of to a ghost location  $\gamma_i$  for the cancelable token.

- The reader assertion  $\boxed{\&^\kappa(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}}$
- The single-writer assertion  $\boxed{\&^\kappa(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}_w^{\mathcal{N}}$
- The CAS-only assertion  $\boxed{\&^\kappa(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}_q^{\mathcal{N}}$

We present only a small selection of rules for atomic-borrows-based protocols in [Figure 17.5](#) and [Figure 17.6](#).

- **GPS-AtBOR-R**, **GPS-AtBOR-W**, and **GPS-AtBOR-RSHR** are similar to the rules **GPS-SW-R**, **GPS-SW-W**, and **GPS-SW-RSHR** in [Figure 17.3](#), respectively. They show the relations among the protocol assertions and the ghost permissions (introduced in [Definition 17.5](#)).
- **GPS-AtBOR-W-WSHR-RSHR** is the atomic-borrow-based variant of **GPS-SW-W-WSHR-RSHR**, which shows how to convert the single-writer assertion into the full fraction of the CAS-only assertion, effectively switching the protocol from the single-writer mode to the CAS-only mode.



$$\begin{array}{c}
\text{persistent} \left( \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} \right) \qquad \text{GPS-AtBOR-R} \quad \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} \vdash \mathcal{R}(\ell, \gamma, t, s, v) \\
\\
\text{GPS-AtBOR-W} \quad \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} \dashv\vdash \mathcal{W}(\ell, \gamma, t, s, v) * \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} \\
\\
\text{GPS-AtBOR-RSHR} \quad \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}}_q \dashv\vdash \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v) * \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} \\
\\
\text{GPS-AtBOR-W-WSHR-RSHR} \quad \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}}_W \vdash \mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v) * \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}}_1 \\
\\
\text{GPS-AtBOR-W-REVERT} \quad \frac{\uparrow \mathcal{N} \subseteq \mathcal{E}}{\boxed{[\kappa]_q} * \mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v) * \boxed{\&^{\kappa}(\ell, \gamma) : (t', s', v') \mid \mathcal{I}}^{\mathcal{N}}_1 \Rightarrow_{\mathcal{E}} \boxed{[\kappa]_q} * \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}}_W} \\
\\
\text{GPS-AtBOR-INIT} \quad \left\{ \begin{array}{l} \boxed{[\kappa]_q} * \&_{\text{full}}^{\kappa}(\exists v. \ell \mapsto v * P(v)) \\ * \forall \gamma, t, v. \triangleright P(v) * \mathcal{W}(\ell, \gamma, t, s, v) \Rightarrow_{\mathcal{E}} \triangleright \mathcal{I}_w(\ell, \gamma, t, s, v) * Q(\gamma, t, v) \\ \square \forall \gamma, t, s, v. \mathcal{I}_w(\ell, \gamma, t, s, v) \Rightarrow \triangleright P(v) \end{array} \right. \\
\Rightarrow_{\mathcal{E}} \boxed{[\kappa]_q} * \exists \gamma, t, v. \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} * Q(\gamma, t, v)
\end{array}$$

FIGURE 17.5: Selected basic rules for Atomic-Borrows-based GPS Protocols

- The reverse rule **GPS-AtBOR-W-REVERT**, like **GPS-SW-W-REVERT**, shows how to switch the protocol from CAS-only mode back to single-writer mode. Similar to that of cancelable single-writer protocols, this reverse switch needs to know the internal invariant that supports the model of GPS protocols is still alive, so it needs the fancy update and a fraction of the lifetime token. (If the lifetime is still alive, then the invariant is also still alive.)
- **GPS-AtBOR-INIT** allows us to initialize a atomic-borrows-based protocol for  $\ell$ . The rule is similar to the rule **GPS-SW-INIT** of cancelable protocols,<sup>14</sup> but here we cannot simply allocate a new cancelable invariant (and so a cancelable token) for every new protocol instance. Instead, we need to tie the new protocol instance to some existing lifetime  $\kappa$ , and therefore we need to integrate the more complex (but also more general) management of lifetimes and borrows into the rule.

<sup>14</sup>see Figure 17.4

- First, we need to know that the lifetime  $\kappa$  is alive, so we require a fraction of the lifetime token  $[\kappa]_q$ , which will we return after the initialization.
- Second, like in **GPS-SW-INIT**, we need the non-atomic points-to  $\ell \mapsto v$  and the write interpretation  $\mathcal{I}_w$  for the write message  $(t, s, v)$ . However, unlike **GPS-SW-INIT**, we make one general-

ization, and we must establish an atomic borrow tied to  $\kappa$  for the new protocol instance.

- The generalization is the first fancy update where we allow the write interpretation  $\mathcal{I}_w$  to use the single-writer ghost permission  $\mathcal{W}(\ell, \gamma, t, s, v)$  immediately at initialization point. Any unused resources, which can possibly include the writer permission, is returned in  $Q(\gamma, t, v)$  after the initialization, together with the read assertion  $\boxed{\&^\kappa(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^N$ . If the writer permission  $\mathcal{W}(\ell, \gamma, t, s, v)$  is indeed unused for  $\mathcal{I}_w$  and returned in  $Q(\gamma, t, v)$ , then after the initialization the user can get the single-writer assertion  $\boxed{\&^\kappa(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^N_{\mathcal{W}}$ , using **GPS-AtBOR-W**.<sup>15</sup>
- To create an atomic borrow for our protocol, we need to use **LFTL-FULL-AT**<sup>16</sup> to turn a *full* borrow into an atomic borrow. As such, our rule here requires that the needed resources have been put into a full borrow, i.e.,  $\&^\kappa_{\text{full}}(\exists v. \ell \mapsto v * P(v))$ . However, before applying **LFTL-FULL-AT** to create an atomic borrow, we need to transform the non-atomic points-to into the protocol's resources *under* a full borrow. For that purpose, we need to apply the strong access rule **LFTL-FULL-ACC-STRONG** and then show that the transformation can be reversed, i.e., from the protocol resources we can get back the original resources. Since the rule itself takes care of giving back the non-atomic points-to, it only requires the user to show the remaining obligation, in the second fancy update, that the write interpretation  $\mathcal{I}_w(\ell, \gamma, t, s, v)$  can be used to reconstruct the original resource  $P(v)$ .

Intuitively, this requirement in **LFTL-FULL-ACC-STRONG** is to ensure the lifetime inheritance rule **LFTL-FULL-INH**,<sup>17</sup> where we need to maintain that once the lifetime is dead, we can reclaim the original resource picked when the borrow is created with **LFTL-FULL-BOR**. As such, any update to a borrow's resource needs to show that we can go back to the original resource. Effectively, we see that **GPS-AtBOR-INIT** has the flavor of both an initialization rule (like **GPS-SW-INIT**) and a cancelation rule (like **GPS-SW-DEALLOC**). Specifically, atomic-borrows-based protocols do not have an actual cancelation rule. Instead, once the lifetime  $\kappa$  is dead, the user will reclaim the resource  $\exists v. \ell \mapsto v * P(v)$  that they originally put into the full borrow, regardless of whether that full borrow has ever been used to construct a GPS protocol.

- **GPS-AtBOR-READ** and **GPS-AtBOR-WRITE-REL** are essentially the same as their cancelable protocol counterparts **GPS-SW-READ** and **GPS-SW-WRITE-REL** in Figure 17.4, respectively. The only difference is that they require a fraction of the lifetime token  $[\kappa]_q$  instead of a fraction of the cancelable token.

<sup>15</sup>This generalization has also been applied in **GPS-SW-WRITE-REL** in Figure 17.4, and will also be applied in **GPS-AtBOR-WRITE-REL** in Figure 17.6.

<sup>16</sup>see Figure 16.2

<sup>17</sup>see Figure 15.1

$$\text{Extract}_{\mathcal{I}}(\ell, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N}) ::= \langle \text{obj} \rangle \wedge \begin{cases} \mathcal{I}_r(\ell, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_r(\ell, \gamma, t', s', v') * R(t', s', v') \\ \mathcal{I}_w(\ell, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_w(\ell, \gamma, t', s', v') * R(t', s', v') \\ \mathcal{I}_m(\ell, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_m(\ell, \gamma, t', s', v') * R(t', s', v') \end{cases}$$

GPS-ATBOR-READ

$$\frac{\mathbf{rlx} \sqsubseteq o \quad \uparrow \mathcal{N} \subseteq \mathcal{E}}{\left\{ [\kappa]_q * \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v)} \mid \mathcal{I} \right\}_q^{\mathcal{N}} * (\forall t' \geq t, s' \sqsupseteq s, v'. \text{Extract}_{\mathcal{I}}(\ell, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N})) \right\}} \\ *^o \ell \text{ in } \pi \\ \left\{ v'. \exists t' \geq t, s' \sqsupseteq s. [\kappa]_q * \boxed{\&^{\kappa}(\ell, \gamma) : (t', s', v')} \mid \mathcal{I} \right\}_q^{\mathcal{N}} * \nabla_{\pi}^{o? \mathbf{rlx}} R(t', s', v') \right\}_{\mathcal{E}}$$

GPS-ATBOR-WRITE-REL

$$\frac{\uparrow \mathcal{N} \subseteq \mathcal{E} \quad s \sqsubseteq s'}{\left\{ [\kappa]_q * \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v)} \mid \mathcal{I} \right\}_q^{\mathcal{N}} * \triangleright \langle \text{obj} \rangle (\mathcal{I}_w(\ell, \gamma, t, s, v) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} Q_1 * Q_2) * \\ \forall t' > t. (\mathcal{W}(\ell, \gamma, t', s', v') * Q_2) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} * \left\{ \langle \text{obj} \rangle Q_1 \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_m(\ell, \gamma, t, s, v) \right. \\ \left. \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_w(\ell, \gamma, t', s', v') * Q(t') \right\} \right\}} \\ \ell :=_{\mathbf{rel}} v' \text{ in } \pi \\ \left\{ \text{fail}. \exists t' > t. [\kappa]_q * \boxed{\&^{\kappa}(\ell, \gamma) : (t', s', v')} \mid \mathcal{I} \right\}_q^{\mathcal{N}} * Q(t') \right\}_{\mathcal{E}}$$

GPS-ATBOR-WRITE

$$\frac{\uparrow \mathcal{N} \subseteq \mathcal{E} \quad s \sqsubseteq s'}{\left\{ [\kappa]_q * \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v)} \mid \mathcal{I} \right\}_q^{\mathcal{N}} * \triangleright \Delta_{\pi}^{o? \mathbf{rlx}} \left( \forall t' > t. \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_w(\ell, \gamma, t', s', v') \right) * \\ \triangleright \langle \text{obj} \rangle (\mathcal{I}_w(\ell, \gamma, t, s, v) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_m(\ell, \gamma, t, s, v) * R) \right\}} \\ \ell :=_o v' \text{ in } \pi \\ \left\{ \text{fail}. \exists t' > t. [\kappa]_q * \boxed{\&^{\kappa}(\ell, \gamma) : (t', s', v')} \mid \mathcal{I} \right\}_q^{\mathcal{N}} * R \right\}_{\mathcal{E}}$$

FIGURE 17.6: Selected read and write rules for Atomic-Borrows-based GPS Protocols

- **GPS-ATBOR-WRITE** is a strong write rule that supports also relaxed access mode, using the single-writer assertion.

Finally, Figure 17.7 gives a CAS rule for the atomic-borrows-based single-writer protocol, which can only be used when the protocol is in CAS-only mode. **GPS-ATBOR-WSHR-CAS-INT** is similar to the persistent concurrent protocol CAS rule **GPS-CON-CAS-INT**, except for the following points.

- The rule requires a fraction  $[\kappa]_{q'}$  to know that that the lifetime  $\kappa$  is still alive, and a fraction  $\boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v)} \mid \mathcal{I} \right\}_q^{\mathcal{N}}$  of the CAS-only assertion. These resources are returned (and potentially updated) in the post-condition.
- In case of a failing CAS, the extraction  $\text{Extract}_{\mathcal{I}}$  does not require a

$$\begin{aligned}
& \text{Extract}_{\mathcal{I}}(\ell, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N}) ::= \langle \text{obj} \rangle \wedge \begin{cases} \mathcal{I}_r(\ell, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_r(\ell, \gamma, t', s', v') * R(t', s', v') \\ \mathcal{I}_w(\ell, \gamma, t', s', v') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_w(\ell, \gamma, t', s', v') * R(t', s', v') \end{cases} \\
& \text{GPS-ATBOR-WSHR-CAS-INT} \\
& \text{rlx} \sqsubseteq o_f, o_r, o_w \quad \uparrow \mathcal{N} \subseteq \mathcal{E} \quad v_r \in \mathbb{Z} \\
& \left[ \begin{array}{l} [\kappa]_{q'} * \boxed{\&^{\kappa}(\ell, \gamma) : (t, s, v)} \boxed{\mathcal{I}}_q^{\mathcal{N}} * \\ \triangleright \langle \text{obj} \rangle (\forall t' \geq t, s' \sqsupseteq s, v'. (\mathcal{I}_r(\ell, \gamma, t', s', v') \vee \mathcal{I}_w(\ell, \gamma, t', s', v'))) \multimap \vdash v' =^? v_r) * \\ \Delta_{\pi}^{o_w ? \text{rlx}} P * \\ \Delta_{\pi}^{o_w ? \text{rlx}} \forall t' \geq t, s' \sqsupseteq s. \wedge \left\{ \begin{array}{l} \triangleright (\forall v' \neq v_r. \text{Extract}_{\mathcal{I}}(\ell, \gamma, t', s', v', R, \mathcal{E}, \mathcal{N})) \\ \langle \text{obj} \rangle (\triangleright \mathcal{I}_w(\ell, \gamma, t', s', v_r) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} (\triangleright Q_1(t', s') * \triangleright Q_2(t', s'))) \\ (P * \triangleright Q_2(t', s')) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{W}_{\text{shr}}(\ell, \gamma, t', s', v_r) * \exists s'' \sqsupseteq s'. \forall t'' > t'. \\ \quad \triangleright \mathcal{W}_{\text{shr}}(\ell, \gamma, t'', s'', v_w) \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \\ * \left\{ \begin{array}{l} \langle \text{obj} \rangle (\triangleright Q_1(t', s') \Rightarrow_{\mathcal{E} \setminus \uparrow \mathcal{N}} \mathcal{I}_r(\ell, \gamma, t', s', v')) \\ \vdash_{\mathcal{E} \setminus \uparrow \mathcal{N}} \triangleright \vdash_{\mathcal{E} \setminus \uparrow \mathcal{N}} (Q(t'', s'') * \mathcal{I}_w(\ell, \gamma, t'', s'', v_w)) \end{array} \right. \end{array} \right\} \\ \text{CAS}^{o_f, o_r, o_w}(\ell, v_r, v_w) \text{ in } \pi \end{array} \right] \\
& \left\{ b. [\kappa]_{q'} * \exists t', v', s' \sqsupseteq s. \vee \left\{ \begin{array}{l} b = \text{false} * v_r \neq v' * t \leq t' * \boxed{\&^{\kappa}(\ell, \gamma) : (t', s', v')} \boxed{\mathcal{I}}_q^{\mathcal{N}} * \nabla_{\pi}^{o_f ? \text{rlx}} R(t', s', v') * \nabla_{\pi}^{o_w ? \text{rlx}} P \\ b = \text{true} * v_r = v' * t < t' * \boxed{\&^{\kappa}(\ell, \gamma) : (t', s', v_w)} \boxed{\mathcal{I}}_q^{\mathcal{N}} * \nabla_{\pi}^{o_r ? \text{rlx}} Q(t', s') \end{array} \right\} \right\}_{\mathcal{E}}
\end{aligned}$$

FIGURE 17.7: A CAS rule for Atomic-Borrows-based GPS Protocols

case for the moved interpretation  $\mathcal{I}_m$ , because there cannot be a single-writer write in the CAS-only mode.

- In case of a successful CAS, the user needs to provide the shared-writer permission  $\mathcal{W}_{\text{shr}}(\ell, \gamma, t', s', v_r)$  for the read message, and will receive the updated shared-writer permission  $\mathcal{W}_{\text{shr}}(\ell, \gamma, t'', s'', v_w)$  for the new write message afterwards.

Intuitively, when in CAS-only mode, every party who wants to perform a CAS with the protocol needs a fraction of the CAS-only assertion, and all parties need to share the shared-writer permission  $\mathcal{W}_{\text{shr}}$  that maintains the total order of updates to the location's history.

## 17.2 Middleware GPS Protocols in iRC11

In this section, we introduce middleware GPS protocols as a common interface that can be combined with different types of invariants to derive the various versions of surface-level GPS protocols, some of which we have seen in the previous section. We first will introduce the assertions of middleware GPS protocols, and then show how the surface-level protocol assertions are modeled using those middleware assertions.

**Definition 17.7** (Middleware GPS Protocol Assertions). The middleware assertions include all auxillary ghost permissions in [Definition 17.5](#),

i.e., the read observation  $\mathcal{R}(\ell, \gamma, t, s, v)$ , the single-writer permission  $\mathcal{W}(\ell, \gamma, t, s, v)$ , the shared-writer permission  $\mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v)$  and the shared-reader permission  $\mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v)$ .

Most importantly, these ghost permissions are tied together to the *core* GPS invariant construction  $\text{GPS}^\theta(\ell, \gamma, \mathcal{I})$  where  $\theta \in \{\text{con}, \text{sw}\}$ . Intuitively, the core GPS construction  $\text{GPS}^\theta(\ell, \gamma, \mathcal{I})$  owns the atomic points-to of  $\ell$ <sup>18</sup> and enforces that  $\ell$ 's history can only change within a GPS protocol's restrictions, who are manifested in the auxillary ghost permissions. The core construction will then be put in an invariant or an atomic borrow, so that it can be accessed concurrently by threads that own the ghost permissions.

<sup>18</sup>where  $\gamma$  is the atomic points-to's atomic period identifier—see also Definition 10.1.

**Definition 17.8** (Assertion Model of GPS Persistent Concurrent Protocols). Using the middleware assertions, the model of GPS persistent concurrent protocol assertions simply (1) picks the construction that supports arbitrarily concurrent accesses, i.e.,  $\text{GPS}^{\text{con}}(\ell, \gamma, \mathcal{I})$ ; and (2) shares the construction in a persistent, *objective invariant*.<sup>19</sup> The subjective modality  $\langle \text{subj} \rangle$ <sup>20</sup> turns the core construction into an objective assertion such that it can be put inside the invariant.

<sup>19</sup>see §11.1

<sup>20</sup>see §8.6

$$\boxed{(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} ::= \mathcal{R}(\ell, \gamma, t, s, v) * \boxed{\langle \text{subj} \rangle \text{GPS}^{\text{con}}(\ell, \gamma, \mathcal{I})}^{\mathcal{N}}$$

**Definition 17.9** (Assertion Model of Cancelable Single-Writer GPS Protocols). Similarly, the model of GPS cancelable single-writer protocol assertions (1) picks the construction  $\text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I})$  that supports single-writer accesses; and (2) shares the construction in a cancelable invariants;<sup>21</sup> and (3) pairs up the invariant assertion with the corresponding ghost permissions.

<sup>21</sup>see §11.2

$$\begin{aligned} \boxed{(\ell, \gamma_i, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} &::= \mathcal{R}(\ell, \gamma, t, s, v) * \boxed{\text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I})}^{\gamma_i, \mathcal{N}} \\ \boxed{(\ell, \gamma_i, \gamma) : (t, s, v) \mid \mathcal{I}}_W^{\mathcal{N}} &::= \mathcal{W}(\ell, \gamma, t, s, v) * \boxed{\text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I})}^{\gamma_i, \mathcal{N}} \\ \boxed{(\ell, \gamma_i, \gamma) : (t, s, v) \mid \mathcal{I}}_q^{\mathcal{N}} &::= \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v) * \boxed{\text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I})}^{\gamma_i, \mathcal{N}} \end{aligned}$$

**Definition 17.10** (Assertion Model of Atomic-Borrows-based Single-Writer GPS Protocols). The model of GPS atomic-borrows-based single-writer protocol assertions is also similar to that of cancelable single-writer protocol assertions, but instead of a cancelable invariant, we put the core construction  $\text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I})$  in an atomic borrow.<sup>22</sup>

<sup>22</sup>see §16.2.2

$$\begin{aligned} \boxed{\&^\kappa(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}^{\mathcal{N}} &::= \mathcal{R}(\ell, \gamma, t, s, v) * \&_{\text{at}}^{\kappa/\mathcal{N}} \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I}) \\ \boxed{\&^\kappa(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}_W^{\mathcal{N}} &::= \mathcal{W}(\ell, \gamma, t, s, v) * \&_{\text{at}}^{\kappa/\mathcal{N}} \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I}) \\ \boxed{\&^\kappa(\ell, \gamma) : (t, s, v) \mid \mathcal{I}}_q^{\mathcal{N}} &::= \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v) * \&_{\text{at}}^{\kappa/\mathcal{N}} \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I}) \end{aligned}$$

From these models, it is easy to see the soundness of rules that relate the various assertions and the ghost permissions, that is, the rules **GPS-SW-R**, **GPS-SW-W**, **GPS-SW-RSHR**, **GPS-SW-W-WSHR-RSHR**, **GPS-AtBOR-R**, **GPS-AtBOR-W**, **GPS-AtBOR-RSHR**, and **GPS-AtBOR-W-WSHR-RSHR**.

$$\begin{array}{c}
\text{GPS-MID-HIST} \\
\text{GPS}^\theta(\ell, \gamma, \mathcal{I}) \vdash \exists h. \ell \mapsto_\theta h \\
\\
\text{GPS-MID-CON-INIT} \\
\ell \mapsto v * (\forall \gamma, t. \triangleright \mathcal{I}_w(\ell, \gamma, t, s, v)) \Rightarrow_{\mathcal{E}} \exists \gamma, t. \triangleright \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I}) * \mathcal{R}(\ell, \gamma, t, s, v) \\
\\
\text{GPS-MID-SW-INIT} \\
\ell \mapsto v * (\forall \gamma, t. \triangleright \mathcal{I}_w(\ell, \gamma, t, s, v)) \Rightarrow_{\mathcal{E}} \exists \gamma, t. \triangleright \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I}) * \mathcal{W}(\ell, \gamma, t, s, v) \\
\\
\text{GPS-MID-SW-INIT-STRONG} \\
\ell \mapsto v * (\forall \gamma, t. \mathcal{W}(\ell, \gamma, t, s, v) \Rightarrow_{\mathcal{E}} \triangleright \mathcal{I}_w(\ell, \gamma, t, s, v) * Q(\gamma, t)) \Rightarrow_{\mathcal{E}} \exists \gamma, t. \triangleright \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I}) * Q(\gamma, t) \\
\\
\text{GPS-MID-DEALLOC} \\
\triangleright \text{GPS}^-(\ell, \gamma, \mathcal{I}) \Rightarrow_{\mathcal{E}} \exists t, s, v. \ell \mapsto v * \triangleright \mathcal{I}_w(\ell, \gamma, t, s, v) \\
\\
\text{GPS-MID-SW-DEALLOC} \\
\triangleright \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I}) * \mathcal{W}(\ell, \gamma, t, s, v) \Rightarrow_{\mathcal{E}} \ell \mapsto v * \triangleright \mathcal{I}_w(\ell, \gamma, t, s, v) \\
\\
\\
\frac{P}{Q} [R] ::= P * R \vdash Q * R \\
\\
\text{GPS-R-AGREE} \\
\frac{\mathcal{R}(\ell, \gamma, t_1, s_1, v_1) * \mathcal{R}(\ell, \gamma, t_2, s_2, v_2)}{\Rightarrow_{\mathcal{E}} (t_1 \leq t_2 \Rightarrow s_1 \sqsubseteq s_2) \wedge (t_2 \leq t_1 \Rightarrow s_2 \sqsubseteq s_1)} [\sqcup_V \triangleright \text{GPS}^-(\ell, \gamma, \mathcal{I})] \\
\\
\text{GPS-RSHR-AGREE} \\
\frac{\mathcal{R}_{\text{shr}}^{q_1}(\ell, \gamma, t_1, s_1, v_1) * \mathcal{R}_{\text{shr}}^{q_2}(\ell, \gamma, t_2, s_2, v_2)}{\Rightarrow_{\mathcal{E}} (t_1 \leq t_2 \Rightarrow s_1 \sqsubseteq s_2) \wedge (t_2 \leq t_1 \Rightarrow s_2 \sqsubseteq s_1)} [\sqcup_V \triangleright \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I})] \\
\\
\text{GPS-R-RSHR-AGREE} \\
\frac{\mathcal{R}(\ell, \gamma, t_1, s_1, v_1) * \mathcal{R}_{\text{shr}}^{q_2}(\ell, \gamma, t_2, s_2, v_2)}{\Rightarrow_{\mathcal{E}} (t_1 \leq t_2 \Rightarrow s_1 \sqsubseteq s_2) \wedge (t_2 \leq t_1 \Rightarrow s_2 \sqsubseteq s_1)} [\sqcup_V \triangleright \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I})] \\
\\
\text{GPS-W-LATEST} \\
\frac{\mathcal{R}(\ell, \gamma, t_2, s_2, v_2)}{\Rightarrow_{\mathcal{E}} (t_2 \leq t_1 \wedge s_2 \sqsubseteq s_1)} [\mathcal{W}(\ell, \gamma, t_1, s_1, v_1) * \sqcup_V \triangleright \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I})] \\
\\
\text{GPS-MID-SW-W-REVERT} \\
\frac{\mathcal{W}_{\text{shr}}(\ell, \gamma, t_1, s_1, v_1) * \langle \text{subj} \rangle \mathcal{R}_{\text{shr}}^1(\ell, \gamma, t_2, s_2, v_2)}{\Rightarrow_{\mathcal{E}} \mathcal{W}(\ell, \gamma, t_1, s_1, v_1)} [\sqcup_V \triangleright \text{GPS}^{\text{sw}}(\ell, \gamma, \mathcal{I})]
\end{array}$$

FIGURE 17.8: Selected rules for assertions of middleware GPS protocols

We now look at several structural rules for middleware GPS assertions in Figure 17.8 to informally justify the soundness of the other structural rules for surface-level assertions. *We will not attempt to justify the soundness of operational rules here, due to their complexity.* (Their soundness proofs are checked in Coq.)

- **GPS-MID-HIST** says that the core GPS construction for  $\ell$  owns the atomic points-to of  $\ell$ . Therefore the core construction is naturally shared to allow multiple threads to use the protocol. Furthermore,

the rule justifies the rule **GPS-CON-ATOM-PTSTO** (and similar rules for other protocol variants) that allows us to atomically peek at the atomic points-to of  $\ell$  and subsequently learn that  $\ell$  is still alive. Such liveness fact is needed for deterministic pointer comparison (such as in CASes).

- **GPS-MID-CON-INIT** intuitively performs the creation of the core GPS construction for concurrent protocols, by allocating the GPS ghost state (which will be explained in §17.3) at the ghost location  $\gamma$ . The rule also suggests that the core construction contains all interpretations ( $\mathcal{I}_w$ ,  $\mathcal{I}_r$ , and  $\mathcal{I}_m$ ). The rule justifies **GPS-CON-INIT**.
- **GPS-MID-SW-INIT** and **GPS-MID-SW-INIT-STRONG** similarly perform the creation of the core GPS construction for single-writer protocols. The rules respectively justifies **GPS-SW-INIT** and **GPS-AtBOR-INIT**.
- **GPS-MID-DEALLOC** and **GPS-MID-SW-DEALLOC** justify surface-level deallocation rules, such as **GPS-SW-DEALLOC**, **GPS-SW-READ-ACQ-DEALLOC**, and **GPS-AtBOR-INIT**.<sup>23</sup>
- **GPS-R-AGREE** says that two read observations are in a total order, *provided* that we get access to the core construction GPS in the frame. This is because the two read observations are related through the ghost state stored in the core construction. The core construction GPS only needs to be provided under a later modality and a view-join modality<sup>24</sup>  $\sqcup_V$  for some view  $V$ , which makes the rule compatible with cancelable invariant access rules, e.g., **CINV-ACC**.<sup>25</sup> The fancy update is needed to eliminate the later modality on the ghost state stored in GPS. **GPS-R-AGREE** justifies the rule **GPS-CON-AGREE** and its variants.
- **GPS-RSHR-AGREE**, **GPS-R-RSHR-AGREE**, and **GPS-W-LATEST** work similarly to **GPS-R-AGREE**, and respectively justify surface-level rules that relate different ghost permissions.<sup>26</sup>
- **GPS-MID-SW-W-REVERT** shows how to switch the GPS ghost state from the shared-writer and shared-reader permissions to the single-writer permission, effectively switching from CAS-only mode to single-writer mode. The rule justifies **GPS-SW-W-REVERT** and **GPS-AtBOR-W-REVERT**.

<sup>23</sup>Recall that **GPS-AtBOR-INIT** incorporates both an initialization rule and a deallocation rule—see the explanation in §17.1.3.

<sup>24</sup>see §8.5

<sup>25</sup>see §11.2

<sup>26</sup>Those rules are elided in this chapter.

### 17.3 The Model of GPS Protocols

In this section, we briefly give the model for GPS middleware protocol assertions (**Definition 17.7**). More specifically, we give the model of the ghost reader, writer, shared-writer, and shared-reader permissions, as well as the model of the core GPS construction GPS. We start with the resource algebra needed for the GPS ghost state.

**Definition 17.11** (Resource Algebra for GPS Protocol States). Assuming the protocol state type  $T_S$  we need the following RA to records how a

write event—identified by a timestamp—is tied to a protocol state.

$$\text{GPSR} ::= \text{AUTH}(\text{MAP}(\text{Time}, \text{AG}(T_S)))$$

The fragmentary elements of GPSR will be used to model the ghost permissions, while the authoritative element will be used in the model of the core GPS construction GPS.

**Definition 17.12** (Model of GPS Ghost Permissions).

$$\begin{aligned} \mathcal{R}(\ell, \gamma, t, s, v) &::= \exists \gamma_i, \gamma_\ell. \gamma = (\gamma_i, \gamma_\ell) * [\text{ghost} \text{ state } t \leftarrow s]^{\gamma_i} * \\ &\quad \exists V. \ell \sqsubseteq_{\text{sn}}^{\gamma_\ell} [t \leftarrow (v, V)] \\ \mathcal{W}(\ell, \gamma, t, s, v) &::= \exists \gamma_i, \gamma_\ell. \gamma = (\gamma_i, \gamma_\ell) * [\text{ghost} \text{ state } t \leftarrow s]^{\gamma_i} * \\ &\quad \exists h. \ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} h * h(t) = (v, \_) * \max(\text{dom}(h)) = t \\ \mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v) &::= \exists \gamma_i, \gamma_\ell. \gamma = (\gamma_i, \gamma_\ell) * [\text{ghost} \text{ state } t \leftarrow s]^{\gamma_i} * \\ &\quad \exists h. \text{atWriter}^{\gamma_\ell}(h) * \ell \sqsubseteq_{\text{sy}}^{\gamma_\ell} h * \\ &\quad h(t) = (v, \_) * \max(\text{dom}(h)) = t \\ \mathcal{R}_{\text{shr}}^q(\ell, \gamma, t, s, v) &::= \exists \gamma_i, \gamma_\ell. \gamma = (\gamma_i, \gamma_\ell) * [\text{ghost} \text{ state } t \leftarrow s]^{\gamma_i} * \\ &\quad \exists V. \ell \sqsubseteq_{\text{sn}}^{\gamma_\ell} [t \leftarrow (v, V)] * \\ &\quad \exists h, t_x \leq t, V'. @_{V'} \ell \sqsubseteq_{\text{cas}}^{\gamma_\ell, t_x, q} h \end{aligned}$$

The model of GPS middleware assertions for a protocol instance  $\gamma$  has the following pattern:

- The instance identifier  $\gamma$  uniquely identifies (1) the ghost location  $\gamma_i$  needed to store the GPS ghost state and (2) the current atomic period identifier  $\gamma_\ell$  for  $\ell$ 's atomic points-to.<sup>27</sup>
- Each assertion owns a fragmentary element of the GPS ghost state  $[\text{ghost} \text{ state } t \leftarrow s]^{\gamma_i}$  that tracks that the timestamp  $t$  is assigned the protocol state  $s$ .
- Each assertion owns the appropriate assertion for  $\ell$ 's atomic points-to.

More specifically:

- The reader observation  $\mathcal{R}(\ell, \gamma, t, s, v)$  additionally only carries a history-seen observation<sup>28</sup>  $\ell \sqsubseteq_{\text{sn}}^{\gamma_\ell} [t \leftarrow (v, V)]$  of the write  $(t, v, V)$ .
- Meanwhile, the single-writer permission carries the single-writer ownership  $\ell \sqsubseteq_{\text{sw}}^{\gamma_\ell} h$  of  $\ell$ 's atomic points-to, with  $(t, v, \_)$  being the latest write in  $\ell$ 's history  $h$ .
- The shared-writer permission  $\mathcal{W}_{\text{shr}}(\ell, \gamma, t, s, v)$  owns the atomic points-to's *ghost writer permission*  $\text{atWriter}^{\gamma_\ell}(h)$ ,<sup>29</sup> which is the exclusive permission needed to change  $\ell$ 's current history  $h$ . The shared-writer permission also requires the history-sync observation of  $h$ , and requires that  $t$  is indeed the latest write.

<sup>27</sup>see Definition 10.1

<sup>28</sup>see Definition 10.2

<sup>29</sup>see Definition 10.7



- The shared-reader permission is the reader observation extended with the fractional CAS ownership  $\ell \sqsupseteq_{\text{cas}}^{\gamma_\ell, t, q} h$  asserted at some arbitrary view  $V'$ . That the view  $V'$  can be arbitrary is because we do not need the fractional CAS ownership assertion's observation—we already have the observation  $\ell \sqsupseteq_{\text{sn}}^{\gamma_\ell} [t \leftarrow (v, V)]$  locally. On the other hand, we need the fractional CAS ownership to maintain that the most recent exclusive write with the timestamp  $t_x$  is frozen, and that our observation  $t$  is at least  $t_x$ . This is to ensure that the shared-reader permission prevents any single-writer writes when the protocol is in CAS-only mode. And, as we have seen in the rule **GPS-ATBOR-Wshr-CAS-INT** in Figure 17.7, the shared-reader permission can only be used in conjunction with the shared-writer permission to perform CASes in the GPS single-writer protocol setup.

We elide the proofs that the model supports various rules on the relations among the ghost permissions, e.g., **GPS-W-R** or **GPS-Wshr-R**.<sup>30</sup>

<sup>30</sup>see Figure 17.3

**Definition 17.13** (Model of GPS Core Construction).

$$\begin{aligned}
 \text{GPS}^\theta(\ell, \gamma, \mathcal{I}) ::= & \\
 & \exists \gamma_i, \gamma_\ell. \gamma = (\gamma_i, \gamma_\ell) * \exists \mu. \bullet \mu \cdot \gamma_i * \\
 & \exists h, t_x. \ell \stackrel{t_x}{\rightarrow}_\theta^{\gamma_\ell} h * \\
 & \bigstar_{(t, v, V) \in \text{block\_ends}(h)} @_V((t_x \leq t) ? \mathcal{I}_w : \mathcal{I}_m)(\ell, \gamma, t, s, v) * \\
 & \bigstar_{(t, v, V) \in (h \setminus \text{block\_ends}(h))} @_V((t_x \leq t) ? \mathcal{I}_{w|r} : \mathcal{I}_{w|r|m})(\ell, \gamma, t, s, v) * \\
 & \text{dom}(\mu) = \text{dom}(h) * \text{state\_sorted}(\mu) * \quad (\text{GPS-CORE-WF}) \\
 & \forall t \in \text{disconnected\_from}(\mu, t_x). \text{final}(\mu(t)) * \quad (\text{GPS-CORE-FIN}) \\
 & \left\{ \begin{array}{ll} \theta = \text{sw} & \forall t \in \text{dom}(h). t_x \leq t \Rightarrow \\ & t \notin \text{disconnected\_from}(\mu, t_x) \quad (\text{GPS-CORE-SW-CON}) \\ \theta = \text{con} & \forall t \in \text{dom}(h). t_x \leq t \end{array} \right.
 \end{aligned}$$

where

$$\begin{aligned}
 \text{block\_ends}(h) &::= \{(t, v, V) \in h \mid t + 1 \notin \text{dom}(h)\} \\
 \mathcal{I}_{w|r}(\ell, \gamma, t, s, v) &::= \mathcal{I}_w(\ell, \gamma, t, s, v) \vee \mathcal{I}_r(\ell, \gamma, t, s, v) \\
 \mathcal{I}_{w|r|m}(\ell, \gamma, t, s, v) &::= \mathcal{I}_{w|r}(\ell, \gamma, t, s, v) \vee \mathcal{I}_m(\ell, \gamma, t, s, v) \\
 \text{state\_sorted}(\mu) &::= \forall t_1, t_2. t_1 \leq t_2 \Rightarrow \mu(t_1) \sqsubseteq \mu(t_2) \\
 \text{final}(s) &::= \forall s' \in T_S. s' \sqsubseteq s \\
 \text{disconnected\_from}(\mu, t_x) &::= \{t \in \text{dom}(\mu) \mid \exists t_d \notin \text{dom}(\mu). t_x \leq t_d \leq t\}
 \end{aligned}$$

The core  $\text{GPS}^\theta(\ell, \gamma, \mathcal{I})$  construction is rather simple—it is composed of several components that are tied together to establish a protocol instance  $\gamma$  on the location  $\ell$  with the interpretation  $\mathcal{I}$ .

- the authoritative GPS ghost state  $\bullet \mu$  at the ghost location  $\gamma_i$ ;

- the atomic points-to  $\ell \xrightarrow[t]{t_x} \gamma_\ell h$ ;
- the interpretation resources ( $\mathcal{I}_w$ ,  $\mathcal{I}_r$ , and  $\mathcal{I}_m$ ) for the write messages in  $\ell$ 's history  $h$ ;
- the various properties (**GPS-CORE-WF**, **GPS-CORE-FIN**, and **GPS-CORE-SW-CON**) that ties  $\mu$  and  $h$  together.

More specifically,

<sup>31</sup>see [Definition 6.11](#)

<sup>32</sup>see [Definition 17.13](#)

- Thanks to the authoritative algebra,<sup>31</sup> the map  $\mu$  from timestamps to protocol states is the upper bound for all ghost permissions.<sup>32</sup>
- The atomic points-to has the same mode  $\theta$  as the protocol's mode, and its most recent exclusive write  $t_x$  dictates which interpretation resources are available for a write message in  $h$ .
- The write messages in  $h$  are split into *block-ends* and *non block-ends*. Intuitively, the messages in  $h$  are grouped into *contiguous blocks*: a write can always initiate a new block, while a CAS always extends an existing block. As a CAS can take the write interpretation  $\mathcal{I}_w$  of the message it reads from in order to establish  $\mathcal{I}_w$  for its write message,  $\mathcal{I}_w$  is guaranteed for block ends, *unless* it has been taken away ("moved") by an exclusive single-writer write.<sup>33</sup> In other words, for block-ends that are later than the most recent exclusive write  $t_x$  we have  $\mathcal{I}_w$ , and for block-ends that are earlier we only have the moved interpretation  $\mathcal{I}_m$ . Meanwhile, for non block-ends that are later than  $t_x$  we can have either  $\mathcal{I}_w$  or  $\mathcal{I}_r$  (in case it has been CAS-ed on), and for non block-ends that are earlier than  $t_x$  we can have any case of  $\mathcal{I}_w$ ,  $\mathcal{I}_r$ , or  $\mathcal{I}_m$ .
- **GPS-CORE-WF** requires that  $\mu$  and  $h$  must agree on the current set of timestamps, and that  $\mu$  enforces monotonicity on states.
- **GPS-CORE-FIN** requires that any write *disconnected from*—i.e., later than and not from the same block with— $t_x$  must be *final*. This property is needed for state monotonicity, as can be seen in the concurrent write rule **GPS-CON-WRITE**, but it is not very restrictive, so as to support single-write rules, e.g., **GPS-SW-WRITE-REL**.
- **GPS-CORE-SW-CON** requires that, in a single-writer protocol, any writes later than  $t_x$  are not disconnected from  $t_x$ —effectively enforces that only single-writer writes or CASes can be performed. Meanwhile, a concurrent protocol forbids single-writer writes: any writes are later than the most recent exclusive write  $t_x$ .

<sup>33</sup>see, e.g., **GPS-SW-WRITE-REL** in [Figure 17.4](#)

**CHAPTER SUMMARY.** In this chapter, we presented the interfaces and the models of various GPS protocols. A GPS protocol is built atop the atomic points-to, and provides some abstraction of write messages into protocol states that can only grow monotonically. The construction of a GPS protocol model is done in multiple layers, allowing us to attach it with different invariant types, including atomic borrows. In the next chapters, we will see how GPS protocols can be used in  $\text{RB}_{\text{rlx}}$  to verify concurrent Rust libraries that use **unsafe** blocks.

# 18

## Verification of `RwLock`

---

In this chapter, we demonstrate a concrete verification of the Rust’s reader-writer lock library `RwLock` as part of `RBrlx`’s Task 1—proving the safety of Rust libraries under relaxed memory concurrency. As with the original RustBelt work,<sup>1</sup> we perform the verification with a hand-translated  $\lambda_{\text{Rust}}$  version of `RwLock`. In the framework of RustBelt, the obligations of verifying that a type  $\tau$  satisfies the Rust type system pin down to (1) giving a separation-logic *semantic interpretation* of  $\tau$  that satisfies certain requirements, and (2) proving that the type’s operations *maintain* the semantic interpretation. More specifically, the semantic interpretation  $\llbracket \tau \rrbracket$  for  $\tau$  is a triple  $(\llbracket \tau \rrbracket.\text{size}, \llbracket \tau \rrbracket.\text{own}, \llbracket \tau \rrbracket.\text{shr})$  that respectively represents (1) the memory size of  $\tau$ , (2) the separation-logic unique ownership of an *owned* object or a *mutable reference* to an object of type  $\tau$ , and (3) the separation-logic shared ownership of an *immutable reference* to an object of type  $\tau$ . We will give more details on the requirements of the predicates when we get to define them for `RwLock`. However, we would like to note that for concurrently libraries (which are our focus here), the most relevant predicate is the shared predicate  $\llbracket \tau \rrbracket.\text{shr}$  which arbitrates concurrent accesses to the underlying resources of the type, and which encodes the *interior mutability* of the library through immutable references. For `RwLock`, we will use atomic-borrows-based single-writer GPS protocols<sup>2</sup> to define  $\llbracket \text{RwLock} \rrbracket.\text{shr}$ .

<sup>1</sup>Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language” [Jun+18a].

<sup>2</sup>see §17.1.3

In §18.1, we will present the relaxed memory  $\lambda_{\text{Rust}}$  version of `RwLock<T>`, which uses the same relaxed memory operations as that of the Rust’s library. We then give the library the semantic model  $\llbracket \text{RwLock} \rrbracket$  in §18.2 and sketch how the operations maintain the model in §18.3.

### 18.1 RMC Implementation of a Reader-Writer Lock

The RMC  $\lambda_{\text{Rust}}$  implementation of `RwLock<T>` and its related types—*lock guards*—`RwLockReadGuard<T>` and `RwLockWriteGuard<T>` are given in Figure 18.1. Intuitively, the reader-writer lock `RwLock<T>` protects an underlying object of type `T`. When acquiring a reader lock or a writer lock, a corresponding lock guard of type `RwLockReadGuard<T>` or `RwLockWriteGuard<T>` respectively is returned. The lock guard follows the “Resource Acquisition is initialization” (RAII) idiom, where the lifetime of the lock guard is the duration that a client holds on to the lock: when the lock is acquired, the lock guard object is created, and when the

|                                          |                           |                                                                          |
|------------------------------------------|---------------------------|--------------------------------------------------------------------------|
| <code>RwLock&lt;T&gt;::</code>           | <code>new :</code>        | <code>fn(T) -&gt; RwLock&lt;T&gt;</code>                                 |
| <code>RwLock&lt;T&gt;::</code>           | <code>into_inner :</code> | <code>fn(self) -&gt; T</code>                                            |
| <code>RwLock&lt;T&gt;::</code>           | <code>get_mut :</code>    | <code>fn(&amp;mut self) -&gt; &amp;mut T</code>                          |
| <code>RwLock&lt;T&gt;::</code>           | <code>try_read :</code>   | <code>fn(&amp;self) -&gt; Result&lt;RwLockReadGuard&lt;T&gt;&gt;</code>  |
| <code>RwLock&lt;T&gt;::</code>           | <code>try_write :</code>  | <code>fn(&amp;self) -&gt; Result&lt;RwLockWriteGuard&lt;T&gt;&gt;</code> |
| <code>RwLockReadGuard&lt;T&gt;::</code>  | <code>deref :</code>      | <code>fn(&amp;self) -&gt; &amp;T</code>                                  |
| <code>RwLockReadGuard&lt;T&gt;::</code>  | <code>drop :</code>       | <code>fn(&amp;mut self)</code>                                           |
| <code>RwLockWriteGuard&lt;T&gt;::</code> | <code>deref :</code>      | <code>fn(&amp;self) -&gt; &amp;T</code>                                  |
| <code>RwLockWriteGuard&lt;T&gt;::</code> | <code>deref_mut :</code>  | <code>fn(&amp;mut self) -&gt; &amp;mut T</code>                          |
| <code>RwLockWriteGuard&lt;T&gt;::</code> | <code>drop :</code>       | <code>fn(&amp;mut self)</code>                                           |

TABLE 18.1: A summary of Rust types for `RwLock<T>` and its lock guards

lock guard object is destroyed, the lock is released.

Table 18.1 gives a summary of the Rust types for the functions in Figure 18.1.

- `new_lock( $\tau$ )` is the  $\lambda_{\text{Rust}}$  version of `fn new(x: T) -> RwLock<T>` which consumes an object of type `T` and returns a reader-writer lock for `T`. The type  $\tau$  is the  $\lambda_{\text{Rust}}$  version of `T`.
- `into_inner( $\tau$ )` consumes the reader-writer lock and returns the underlying object of type `T`. `self` is the Rust idiom to mention the current object, which, in this case, is of type `RwLock<T>`.
- `get_mut` borrows a mutable reference to the underlying object from a mutable reference to the lock. There is no need to acquire the lock, because the mutable reference to the lock should guarantee that no one holds the lock.
- `try_read` tries to acquire a reader lock from an *immutable* reference to the lock. The `Result` type has two cases: a `None` means the function fails to acquire the reader lock, and a `Some` means the reader lock has been acquired through a reader lock guard. In the Rust version, this function's body uses `unsafe`.
- `try_write` tries to acquire a writer lock from an *immutable* reference to the lock. If the writer has been acquired then a writer lock guard is returned. In the Rust version, this function's body uses `unsafe`.
- `rdguard_deref` borrows an *immutable* reference to the underlying object `T` of the lock from a reader lock guard. (`self` is of type `RwLockReadGuard<T>` here.) In the Rust version, this function's body uses `unsafe`.
- `rdguard_drop` is the destructor of `RwLockReadGuard<T>` that also releases a reader lock. In the Rust version, this function's body uses `unsafe`.

- `wrguard_deref` and `wrguard_deref_mut` respectively borrow an immutable or mutable reference to the underlying object `T` of the lock from a writer lock guard. (`self` is of type `RwLockWriteGuard<T>` here.) In the Rust version, this function's body uses `unsafe`.
- `wrguard_drop` is the destructor of `RwLockWriteGuard<T>` that also releases a writer lock. In the Rust version, this function's body uses `unsafe`.

We now look more closely at the  $\lambda_{\text{Rust}}$  implementations in Figure 18.1.

- `new_lock`( $\tau$ ) shows that the memory layout of a `RwLock<T>` is a counter at offset 0 and the memory of  $\tau$  itself at offset 1. The counter represents the lock state: if it is  $-1$ , the writer lock is being held, while if it is positive, the number is the number of reader locks being held. The lock is unlocked when the counter is 0.

Since  $\lambda_{\text{Rust}}$  is call-by-value, it models consumption of the object of type  $\tau$  (addressed by the value of  $x$ ) as mem-copying (non-atomically) the contents of  $\tau$ , and then deleting the original input object.<sup>3</sup>

- Similarly, `into_inner`( $\tau$ ) non-atomically mem-copies the underlying object  $\tau$  from offset 1 of the lock (addressed by the value of  $x$ ) into a fresh memory region (of size  $\tau.\text{size}$ ), and then deletes the whole lock object.
- `get_mut` reads the address  $y$  to the lock stored in the argument  $x$ , and updates (non-atomically writes to)  $x$  with the address to the underlying  $\tau$  (offset by 1 from the lock's address).
- `try_read` returns  $r$  as the address to a memory region of size 2 for the type `Result<RwLockReadGuard<T>>`. The first memory cell of the memory region stores the variant of the `Result` type (None or Some), and the second memory cell in the Some case stores the value of the type `RwLockReadGuard<T>`, which is simply the address  $y$  to the lock (stored in the argument  $x$ ). A reader lock can only be acquired using an acquire CAS if either the lock is unlocked (the counter is 0) or there are reader locks (the counter is positive). The function only retries in case the CAS fails due to contention with other threads trying to acquire reader locks.<sup>4</sup>
- `try_write` does not retry at all. A writer lock can only be acquired when the lock is unlocked, also by using an acquire CAS.
- `rdguard_deref` simply returns  $r$  that stores the address to the underlying object of type  $\tau$ .
- `rdguard_drop` keeps trying until successful decrement the counter.
- `wrguard_deref` and `wrguard_deref_mut` are exactly the same as `rdguard_deref`—they only differ by types.
- `wrguard_drop` simply resets the counter to 0 with a release write and thus releases the writer lock.

<sup>3</sup>The implementations of `new`, `delete`, and mem-copy have been provided in Figure 4.2.

<sup>4</sup>The implementation of `letcont` is also provided in provided in Figure 4.2.

```

new_lock( $\tau$ ) ::=
   $\lambda[x].$  1: let  $r := \text{new}[\tau.\text{size} + 1]$  in
    2:  $r + 0 := 0$ ;
    3:  $r + 1 :=_{\tau.\text{size}} *x$ ;
    3: delete $[\tau.\text{size}, x]$ ;
    4:  $r$ 

into_inner( $\tau$ ) ::=
   $\lambda[x].$  1: let  $r := \text{new}[\tau.\text{size}]$  in
    2:  $r :=_{\tau.\text{size}} *(x + 1)$ ;
    3: delete $[\tau.\text{size} + 1, x]$ ;
    4:  $r$ 

get_mut ::=
   $\lambda[x].$  1: let  $y := *x$  in
    2:  $x := y + 1$ ;
    3:  $x$ 

rdguard_deref ::=
   $\lambda[x].$  1: let  $y := *x$  in
    2: let  $r := \text{new}[1]$  in
    3:  $r := *(y + 1)$ ;
    4: delete $[1, x]$ ;
    5:  $r$ 

rdguard_drop ::=
   $\lambda[x].$  1: let  $y := *x$  in
    2: letcont  $\text{loop}([]) :=$ 
    3:   let  $n := *r1x y$  in
    4:   if  $\text{CAS}^{\text{rel}}(y, n, n - 1)$ 
    5:   then
    6:     delete $[1, x]$ 
    7:   else  $\text{loop}([])$ 
    8: in  $\text{loop}([])$ 

try_read ::=
   $\lambda[x].$  1: let  $r := \text{new}[2]$  in
    2: let  $y := *x$  in
    3: letcont  $k([]) := \text{delete}[1, x]; r$  in
    4: letcont  $\text{loop}([]) :=$ 
    5:   let  $n := *r1x y$  in
    6:   if  $n \leq -1$ 
    7:   then  $r :=_{\text{inj } 0} ()$ ;  $k([])$  // returns None
    8:   else
    9:     if  $\text{CAS}^{\text{acq}}(y, n, n + 1)$ 
    10:    then  $r :=_{\text{inj } 1} y$ ;  $k([])$  // returns Some
    11:    else  $\text{loop}([])$ 
    12: in  $\text{loop}([])$ 

try_write ::=
   $\lambda[s].$  1: letcont  $k([r]) := \text{delete}[1, x]; r$  in
    2: let  $r := \text{new}[2]$  in
    3: let  $y := *x$  in
    4: if  $\text{CAS}^{\text{acq}}(y, 0, -1)$ 
    5: then  $r :=_{\text{inj } 1} y$ ;  $k([r])$  // returns Some
    6: else  $r :=_{\text{inj } 0} ()$ ;  $k([r])$  // returns None

wrguard_deref ::=
   $\lambda[x].$  1: let  $y := *x$  in
    2: let  $r := \text{new}[1]$  in
    3:  $r := *(y + 1)$ ;
    4: delete $[1, x]$ ;
    5:  $r$ 

wrguard_deref_mut ::=
   $\lambda[x].$  1: let  $y := *x$  in
    2: let  $r := \text{new}[1]$  in
    3:  $r := *(y + 1)$ ;
    4: delete $[1, x]$ ;
    5:  $r$ 

wrguard_drop ::=
   $\lambda[x].$  1: let  $y := *x$  in
    2:  $y :=_{\text{rel}} 0$ ;
    3: delete $[1, x]$ 

```

FIGURE 18.1: A  $\lambda_{\text{Rust}}$  version of Rust's RMC `RwLock<T>`

## 18.2 The Semantic Model of the Reader-Writer Lock Type

The general formula of semantically type checking an expression  $e$  to have the type  $\tau$  is to show that, under an assignment of semantically well-typed values for all free variables in  $e$ , the substituted expression of  $e$  executes without errors, and if the execution terminates, it terminates with a value that satisfies the semantic interpretation of  $\tau$ . More formally, a semantic typing judgment  $\llbracket \Gamma \vdash e : \tau \rrbracket$  is typically defined as:

$$\llbracket \Gamma \vdash e : \tau \rrbracket ::= \forall \gamma \in \llbracket \Gamma \rrbracket. \mathbf{wp} \, \gamma e \left\{ v. \llbracket \tau \rrbracket_{\gamma}(v) \right\}$$

where  $\gamma$  is the assignment satisfying the context  $\Gamma$  for free variables in  $e$ , and  $\gamma e$  is the substitution of  $e$  with  $\gamma$ . The semantic typing for the function type may also read as:

$$\begin{aligned} \llbracket \Gamma \vdash \lambda x. e : \tau_1 \rightarrow \tau_2 \rrbracket ::= \\ \forall \gamma \in \llbracket \Gamma \rrbracket. v. \llbracket \tau_1 \rrbracket_{\gamma}(v) \multimap \mathbf{wp} \, \gamma((\lambda x. e)v) \left\{ v'. \llbracket \tau_2 \rrbracket_{\gamma}(v') \right\} \end{aligned}$$

That is, for  $\lambda x. e$  to be semantically well-typed if when provided with the interpretation of the arguments  $\llbracket \tau_1 \rrbracket_{\gamma}(v)$ , the resulting function application is also semantically well-typed.

Therefore, to show that the reader-writer lock library is semantically well-typed, we first need to provide the semantic interpretations for `RwLock<T>`, `RwLockReadGuard<T>`, and `RwLockWriteGuard<T>`. Then we need to show that the library's functions are also semantically well-typed, which, as explained above, translates to verifications of function bodies in our separation logic.

The semantic typing judgment of RustBelt is more complicated than the general intuition above: the type system needs to account for lifetimes and continuations. However, in the following, we focus on the type interpretations, and only sketch the verifications of the function bodies, and avoid going into the details of the semantic typing judgment.

**Definition 18.1** (RA for `RwLock`). We use the unit protocol  $S_{\text{()}}$  for `RwLock`. That is, the protocol is trivial, so we always ignore the state (which is only unit). Instead, we will use the richer ghost state of the algebra `RWLOCKR` to model the lock states.

$$\text{RWLOCKR} ::= \text{AUTH}(\text{OPTION}(\text{SUM}(\text{EX}(1), \text{AG}(\text{Lft}) \times \text{FRAC} \times \text{POS})))$$

This algebra is already used in the original RustBelt work. The lock's ghost state is an option of a sum of two types of sub states. The `None` case models the unlocked state. The `Some` case with the left-hand side of the sum models the writer lock state, while a `Some` case with the right-hand side of the sum models a reader lock state. The writer lock state needs to be exclusive (`EX`), while a reader lock state needs to track the lifetimes (using agreement `AG`) of all reader locks, the fractions (`FRAC`) of the lock being held by all reader locks, and the number (`POS`—positive numbers) of reader locks.

$$\text{Writing} ::= \circ \text{Some}(\text{left}())$$

$$\text{Reading}(q, \nu) ::= \circ \text{Some}(\text{right}(\text{ag}(\nu), q, 1))$$

The writer state `Writing` will be owned by the one holding the writer lock. Every piece of reader state  $\text{Reading}(q, \nu)$  will be held by one (1) reader where  $q$  is the fraction of the lock held by the reader, and  $\nu$  is the lifetime for the reader lock guard.

**Definition 18.2** (GPS Protocol for `RwLock`). We will not rely on the timestamps exposed by our version of GPS protocols. As we are working with a concurrent Rust type, it is natural to use an atomic-borrows-based variant of GPS protocols. We use the single-writer variant introduced in §17.1.3—the writer lock relies on single-writer writes, while reader locks rely on CAS-only writes.

$$\begin{aligned}
\mathcal{I} &::= (\mathcal{I}_w, \mathcal{I}_r, \mathcal{I}_m) \\
\mathcal{I}_r &::= \lambda \ell, \gamma, \_, \_, v. \text{True} \\
\mathcal{I}_m &::= \lambda \ell, \gamma, \_, \_, v. v = -1 \\
\mathcal{I}_w(\tau, \alpha, \gamma_s, t) &::= \lambda \ell, \gamma, \_, \_, v. \exists a. \left[ \bullet a \right]_{\gamma_s}^{\tau} * \\
&\quad \left\{ \begin{array}{l} a = \text{None} * v = 0 \\ \quad * \&_{\text{full}}^{\alpha}(\ell + 1 \mapsto \llbracket \tau \rrbracket.\text{own}(t)) \\ \quad * \mathcal{W}_{\text{shr}}(\ell, \gamma, \_, \_, 0) * \mathcal{R}_{\text{shr}}^1(\ell, \gamma, \_, \_, 0) \\ a = \text{Some}(\text{right}(\nu, q, n)) * v = n \\ \quad * \mathcal{W}_{\text{shr}}(\ell, \gamma, \_, \_, n) \\ \quad * \llbracket \tau \rrbracket.\text{shr}(\alpha \sqcap \nu, t, \ell + 1) \\ \quad * \Box \text{Kill}(\nu) \\ \quad * \text{Inh}(\nu, \&_{\text{full}}^{\alpha}(\ell + 1 \mapsto \llbracket \tau \rrbracket.\text{own}(t))) \\ \quad * \exists q'. q + q' = 1 * [\nu]_{q'} * \mathcal{R}_{\text{shr}}^{q'}(\ell, \gamma, \_, \_, n) \\ a = \text{Some}(\text{left}()) * v = -1 \end{array} \right.
\end{aligned}$$

where

$$\ell \mapsto \Phi ::= \exists \bar{v}. \ell \mapsto \bar{v} * \Phi(\bar{v})$$

Recall that we need to define the read, write, and move interpretations  $(\mathcal{I}_r, \mathcal{I}_w, \mathcal{I}_m)$ .<sup>5</sup> Since we do not have any interesting read operation for the lock itself (acquiring and releasing locks are writes),  $\mathcal{I}_r$  is trivial. The move interpretation  $\mathcal{I}_m$  encodes the obligation for single-writer writes, and as we only use single-writer writes to release the writer lock—switching the lock from  $-1$  to  $0$ —we only require that only writer lock writes ( $-1$ ) can be overwritten by single-writer writes that release the writer lock.

The write interpretation  $\mathcal{I}_w$  is the most important. It is parameterized by the underlying type  $\tau$ , the lifetime upper-bound  $\alpha$  of the lock guards (the upper-bound of e.g., `'a` in `RwLockReadGuard<'a, T>`), the ghost location  $\gamma_s$  for the lock ghost state (whose type is the `RWLOCKR` algebra), and the owner's thread id  $t$ .  $\mathcal{I}_w$  always holds on to the current authoritative lock state  $\bullet a$  (while the clients of the lock hold the fragmentary parts, in `Reading` or `Writing`).  $\mathcal{I}_w$  enforces that the *physical* lock state  $v$  stored at  $\ell$  must agree with the ghost state  $a$ : if  $a$  is `None`, the lock is unlocked and  $v = 0$ ; if the lock is write-locked,  $a$  is in the left-hand side of the sum and

<sup>5</sup>See Definition 17.3



$v = -1$ ; if the lock is read-locked by  $n$  readers,  $a$  is in the right-hand side of the sum and  $v = n$ .

Furthermore, when the lock is unlocked, the invariant holds (1) the resources for the underlying type  $\tau$ , in form of a full borrow<sup>6</sup> with the *own* part  $\llbracket \tau \rrbracket.\text{own}$  of the semantic interpretation of  $\tau$  (see more below); (2) the shared-writer and shared-reader permissions of the protocol,<sup>7</sup> both with value 0. These resources will be given to those acquiring a lock, so that they can access the underlying type  $\tau$  as well as permissions to release the lock.

<sup>6</sup>see §15.2

<sup>7</sup>see Definition 17.5

The notation  $\ell \mapsto \Phi$  is short for  $\exists \bar{v}. \ell \mapsto \bar{v} * \Phi(\bar{v})$ , which represents the resources that  $\Phi$  holds for the list of values  $v$  that are stored at  $\ell$ . Effectively, when the lock is unlocked, the invariant holds the points-to of  $\tau$  with values  $\bar{v}$  as well as  $\llbracket \tau \rrbracket.\text{own}(t, \bar{v})$ —the resources of  $\tau$  that hold at the owner's thread  $t$  and  $\bar{v}$ .

When the lock is write-locked, all of the resources in (1) and (2) will be held by the one holding the lock, so the invariant has nothing left. The one holding the writer lock will have access to  $\llbracket \tau \rrbracket.\text{own}$ , which is the full resources of the underlying type  $\tau$ .

When the lock is read-locked, the invariant holds (1) the shared-writer permission  $\mathcal{W}_{\text{shr}}(\ell, \gamma, -, -, n)$  needed to perform CASes (to acquire reader locks), with  $n$  being the current number of reader locks; (2) the *share* part  $\llbracket \tau \rrbracket.\text{shr}$  of  $\tau$ 's semantic interpretation, which will be given to any one acquiring a reader lock, so that they can access  $\tau$ 's contents in *shared* mode; (3) the ability to kill the shared lifetime  $\nu$  ( $\text{Kill}(\nu)$ ) and to inherit the own resources of  $\tau$ , which will be used to switch the lock from sharing (read-lock) mode to unlocked mode; (4) the remaining available resources that will be given to new reader locks, which include a fraction  $q'$  of the lifetime token for  $\nu$  ( $[\nu]_{q'}$ ) and the same fraction  $q'$  for the shared-reader permission ( $\mathcal{R}_{\text{shr}}^{q'}(\ell, \gamma, -, -, n)$ ) needed to release a reader lock (performing a CAS).  $q'$  together with  $q$ —the fraction of those resources that have been given out to existing reader lock holders—must sum up to 1.

The semantic type interpretations for **RwLock** and its lock guards are provided by Definition 18.3. In the RustBelt framework, an interpretation of a type  $\tau$  is in the form  $\text{Type} \{ \text{size} := \dots; \text{own} := \dots; \text{shr} := \dots \}$ .  $\llbracket \tau \rrbracket.\text{size}$  is the number of values that  $\tau$  has, i.e., the length of a list of values  $\bar{v}$  of  $\tau$ .  $\llbracket \tau \rrbracket.\text{own}$  is a predicate of type  $\text{Thread} \times \text{Val}^* \rightarrow \text{vProp}$ , where  $\llbracket \tau \rrbracket.\text{own}(t, \bar{v})$  should encode what it means for the thread  $t$  to *uniquely own* an object of type  $\tau$  (i.e., owning a **T**) or a *mutable* reference to an object of type  $\tau$  (i.e., own a **&mut T**), and the object has the values  $\bar{v}$ .  $\llbracket \tau \rrbracket.\text{shr}$  is a predicate of type  $\text{Lft} \times \text{Thread} \times \text{Loc} \rightarrow \text{vProp}$ , where  $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$  should encode what it means for the thread  $t$  to own, at the location  $\ell$ , a *shared* reference, bounded by the lifetime  $\kappa$ , to an object of type  $\tau$  (i.e., own a **&T**). Indeed, the predicate is used for the own predicate of a shared reference:

$$\begin{aligned} \llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{size} &::= 1 \\ \llbracket \&_{\text{shr}}^{\kappa} \tau \rrbracket.\text{own}(t, \bar{v}) &::= \exists \ell. \bar{v} = [\ell] * \llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell) \end{aligned}$$

$\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell)$  needs to satisfy several properties: it needs to be persistent and it needs to be closed with respect to shorter lifetimes, because shared references should be easily copied (shared) for smaller lifetimes. But most importantly, the predicate needs to satisfy the property that shared references can be created from mutable references:

$$\left( \&_{\text{full}}^{\kappa} (\exists \bar{w}. \ell \mapsto \bar{w} * \llbracket \tau \rrbracket.\text{own}(t, \bar{w})) * [\kappa]_q \right) \Rightarrow * \left( \llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell) * [\kappa]_q \right) \quad (\text{TY-SHARE})$$

**Definition 18.3** (Type Interpretations).

$\llbracket \text{rwlock}(\tau) \rrbracket ::=$

Type {size := 1 +  $\llbracket \tau \rrbracket$ .size;

own :=  $\lambda t, \bar{v}. \exists n \geq -1, \bar{w}. \bar{v} = [n] ++ \bar{w} * \llbracket \tau \rrbracket.\text{own}(t, \bar{w})$ ;

shr :=  $\lambda \kappa, t, \ell. \exists \alpha \sqsupseteq \kappa, \gamma, \gamma_s. \boxed{\&^{\alpha}(\ell, \gamma) : \_ \mid \mathcal{I}(\tau, \alpha, \gamma_s, t)}^{\mathcal{N}} \}$

$\llbracket \text{rwlockreadguard}(\alpha, \tau) \rrbracket ::=$

Type {size := 1;

own :=  $\lambda t, \bar{v}. \exists \ell. \bar{v} = [\ell] * \exists \nu, q, \beta \sqsupseteq \alpha, t_o, \gamma, \gamma_s.$

$\llbracket \tau \rrbracket.\text{shr}(\alpha \sqcap \nu, t, \ell + 1) * [\nu]_q * \Box \text{Kill}(\nu) *$

$\boxed{\text{Reading}(q, \nu)}^{\gamma_s} * \boxed{\&^{\beta}(\ell, \gamma) : \_ \mid \mathcal{I}(\tau, \beta, \gamma_s, t)}^{\mathcal{N}}_q$ ;

shr :=  $\lambda \kappa, t, \ell. \exists \ell'. \&_{\text{frac}}^{\kappa}(\lambda q. \ell \mapsto^q [\ell']) * \triangleright \llbracket \tau \rrbracket.\text{shr}(\alpha \sqcap \kappa, t, \ell' + 1)\}$

$\llbracket \text{rwlockwriteguard}(\alpha, \tau) \rrbracket ::=$

Type {size := 1;

own :=  $\lambda t, \bar{v}. \exists \ell. \bar{v} = [\ell] * \exists \beta \sqsupseteq \alpha, \gamma, \gamma_s.$

$\&_{\text{full}}^{\beta}(\ell + 1 \mapsto \llbracket \tau \rrbracket.\text{own}(t)) *$

$\boxed{\text{Writing}}^{\gamma_s} * \boxed{\&^{\beta}(\ell, \gamma) : (\_, \_, -1) \mid \mathcal{I}(\tau, \beta, \gamma_s, t)}^{\mathcal{N}}_W$ ;

shr :=  $\lambda \kappa, t, \ell. \exists \ell'. \&_{\text{frac}}^{\kappa}(\lambda q. \ell \mapsto^q [\ell']) * \triangleright \llbracket \tau \rrbracket.\text{shr}(\alpha \sqcap \kappa, t, \ell' + 1)\}$

The interpretations of reader-writer lock types are as follows.

- For  $\text{rwlock}(\tau)$ , its size is the size of the underlying type  $\tau$  plus one for the counter. Its own interpretation says that the current list of values  $\bar{v}$  (of length  $\llbracket \tau \rrbracket$ .size) starts with the first value being the number of locks  $n$ , which must always be at least  $-1$ . (Recall that  $-1$  stands for the writer-locked state,  $0$  for the unlocked state, and  $n > 0$  for the reader-locked state with  $n$  readers.) The rest of the list  $\bar{w}$  satisfies the underlying type  $\tau$ 's own interpretation ( $\llbracket \tau \rrbracket.\text{own}(t, \bar{w})$ ).

$\text{rwlock}(\tau)$ 's shared interpretation for a shared reference to the lock bound by the lifetime  $\kappa$  simply says that the lock state must be governed by an atomic-borrowed-based GPS protocol—with the protocol given by [Definition 18.2](#)—bound by a lifetime  $\alpha$  that is at least  $\kappa$ . The lifetime inclusion relation  $\alpha \sqsupseteq \kappa$  is typically used to make sure that the interpretation is closed under smaller lifetimes, so that it can allow for reborrowing. In case of  $\llbracket \text{rwlock}(\tau) \rrbracket.\text{shr}$ ,

this means that the lock is already governed by a protocol bound by  $\alpha$ , and is then available for any smaller lifetime  $\kappa$ . Note that when proving **TY-SHARE** for **rwlock**( $\tau$ ), we perform the allocation of the GPS protocol. Reversely, when the sharing ends with the end of the lifetime  $\kappa$ , we perform the *deallocation* of the GPS protocol.

- For **rwlockreadguard**( $\alpha, \tau$ ), the size is simply one because it only records the base location  $\ell$  of the lock. The own interpretation simply owns the resources that are acquired by a reader lock, as already mentioned in **Definition 18.2**: a reader lock owns (1) the shared interpretation  $\llbracket \tau \rrbracket.\text{shr}$  at the intersected lifetime  $\alpha \sqcap \nu$ , so that one can access  $\tau$ 's contents in shared mode; (2) a fraction  $q$  of the lifetime token for  $\nu$  ( $[\nu]_q$ ) to maintain that  $\nu$  is alive; and (3) the ability to kill the shared lifetime  $\nu$  ( $\text{Kill}(\nu)$ ) and to inherit the own resources of  $\tau$ , which will be used to switch the lock from sharing (read-lock) mode to unlocked mode if this is the only existing reader lock; (4) the same fraction  $q$  for the reader state  $\llbracket \text{Reading}(q, \nu) \rrbracket^{\gamma_s}$  as well as the shared-reader permission of the GPS protocol bound by the lifetime  $\beta$  that is at least the lifetime  $\alpha$  that has computed statically by the type system for the guard.

The shared interpretation of the lock guard itself is just sharing the ownership of the storage to the base location of the lock (now quantified as  $\ell'$ ) and the shared interpretation of the underlying type  $\tau$  bound by both the static lifetime  $\alpha$  of the lock guard and the lifetime  $\kappa$  of the shared reference (to the lock guard).

- For **rwlockwriteguard**( $\alpha, \tau$ ), the size and the shared interpretation are similar to those of reader guards.

The own interpretation is rather simple compared to that of the reader lock guard. In addition to owning the writer state  $\llbracket \text{Writing} \rrbracket^{\gamma_s}$  as well as the writer permission of the GPS protocol—bound by  $\beta$ , and with the writer-lock state value  $-1$ , the interpretation also holds the own interpretation of the underlying type  $\tau$ , but under a full borrow bound by  $\beta$ . Intuitively, the resources  $\ell+1 \mapsto \llbracket \tau \rrbracket.\text{own}(t)$  is under a borrow at  $\beta$  because it is indeed borrowed from the own interpretation of **rwlock**( $\tau$ ) for the writer lock guard when the lock is acquired (when the writer lock guard is created).

### 18.3 Proof Sketches of the Library's Operations

In this section we sketch the proofs that the implementation of the reader-writer lock's functions given in **Figure 18.1**, which uses **unsafe** code, is semantically well-typed. That is, we show that the implementation satisfies the semantic interpretations for their corresponding Rust types given in **Table 18.1**, using the core interpretations of **Definition 18.3**. More specifically, the proof obligation for a function is typically that, given the interpretations of the function's arguments, we need to produce the interpretation of the function's return value, while maintaining all

invariants. Below, we only sketch the proofs for the most interesting functions.

*Proof sketch of `new_lock`( $\tau$ ).* The type is `fn(T) -> RwLock<T>`, so we get to assume  $\llbracket \tau \rrbracket.\text{own}(t, \bar{w})$  where  $t$  is the calling thread and  $w$  are the values of the argument of type  $\tau$ , and we need to produce  $\llbracket \text{rwlock}(\tau) \rrbracket.\text{own}(t, \bar{v})$ . This is easy because  $\llbracket \text{rwlock}(\tau) \rrbracket.\text{own}(t, \bar{v})$  contains  $\llbracket \tau \rrbracket.\text{own}(t, \bar{w})$ , and the implementation only needs to maintain that the lock counter  $n$  is at least  $-1$ . In fact, `new_lock`( $\tau$ ) initializes the lock counter to 0 (the unlocked state). Recall that the allocation of the GPS protocol is done only when one gets a shared reference to the lock, and we have performed that obligation in `TY-SHARE`.  $\square$

*Proof sketch of `try_read`.* `try_read` takes in a `&self`, a shared reference of `rwlock`( $\tau$ ), and returns a `Result<RwLockReadGuard<T>>`. We only consider the successful case, where the `Result` is a `Some`. In that case, we get to assume  $\llbracket \text{rwlock}(\tau) \rrbracket.\text{shr}(\kappa, t, \ell)$  where  $\kappa$  is the lifetime of the shared reference and  $\ell$  is the base location of the lock, and we need to produce  $\llbracket \text{rwlockreadguard}(\alpha, \tau) \rrbracket.\text{own}(t, [\ell])$ , knowing that  $\alpha \sqsubseteq \kappa$ . We therefore can pick  $\kappa$  to instantiate  $\beta$  in  $\llbracket \text{rwlockreadguard}(\alpha, \tau) \rrbracket.\text{own}(t, [\ell])$ .

The success case means that the CAS in line 9 of `try_read` (see Figure 18.1) has successfully increased the lock counter by 1. In the proof, we rely on the GPS protocol assertion  $\boxed{\&^\alpha(\ell, \gamma) : \_ \mid \mathcal{I}(\tau, \alpha, \gamma_s, t)}$  from  $\llbracket \text{rwlock}(\tau) \rrbracket.\text{shr}(\kappa, t, \ell)$  to verify the CAS (using e.g., `GPS-ATBOR-WSHR-CAS-INT` in Figure 17.7). While doing so, we interact with the write interpretation  $\mathcal{I}_w(\tau, \alpha, \gamma_s, t)$  given in Definition 18.2, knowing that the state  $a$  is `Some(right( $\nu, q, n$ ))`. We can then acquire all of the resources needed for  $\llbracket \text{rwlockreadguard}(\alpha, \tau) \rrbracket.\text{shr}(\kappa, t, \ell)$ , by carving out some fraction  $q''$  from the remaining fraction  $q'$  in the write interpretation, and then updating the state  $a$  to `Some(right( $\nu, q + q'', n + 1$ ))`.  $\square$

*Proof sketch of `try_write`.* The proof is similar to that of `try_read`: when performing the CAS from 0 to  $-1$  in 4 of `try_write`, we also interact with the write interpretation  $\mathcal{I}_w(\tau, \alpha, \gamma_s, t)$ , but this time we know that  $a = \text{None}$ . We then update the state  $a$  to `Some(left())` and acquire  $\llbracket \text{Writing} \rrbracket^{\gamma_s}$ . Together with the remaining resources from  $\mathcal{I}_w(\tau, \alpha, \gamma_s, t)$ , we can establish  $\llbracket \text{rwlockwriteguard}(\alpha, \tau) \rrbracket.\text{own}(t, [\ell])$ .  $\square$

*Proof sketch of `rdguard_deref`.* With the type `fn(&self) -> &T`, we get to assume  $\llbracket \text{rwlockreadguard}(\alpha, \tau) \rrbracket.\text{shr}(\kappa, t, \ell)$  and we need to produce  $\llbracket \tau \rrbracket.\text{shr}(\kappa, t, \ell' + 1)$ . This is straightforward because the latter is contained in the former.  $\square$

*Proof sketch of `wrguard_deref_mut`.* With the argument `&mut self`, we get to assume  $\llbracket \text{rwlockwriteguard}(\alpha, \tau) \rrbracket.\text{own}(t, [\ell])$  under a full borrow bounded by some lifetime  $\kappa$ , and we need to produce, for `&mut T`,  $\llbracket \tau \rrbracket.\text{own}(t, \ell + 1)$  also under a  $\kappa$ -bounded full borrow. Recall that in RustBelt,  $\llbracket \&_{\text{mut}}^\kappa \tau \rrbracket.\text{own}(t, \bar{v}) ::= \exists \ell. \bar{v} = [\ell] * \&_{\text{full}}^\kappa (\ell \mapsto \llbracket \tau \rrbracket.\text{own}(t))$ .

We know that  $\kappa \sqsubseteq_1 \alpha$ . Looking at the own interpretation for the writer lock guard, we have also  $\alpha \sqsubseteq_1 \beta$  and  $\&_{\text{full}}^\beta (\ell + 1 \mapsto \llbracket \tau \rrbracket.\text{own}(t))$ . So

we can *reborrow* that into  $\&_{\text{full}}^{\kappa} (\ell + 1 \mapsto \llbracket \tau \rrbracket.\text{own}(t))$ . The remaining resources of the writer lock guard will be withheld by the inheritance of the reborrow, so that when the reborrow ends, those resources are returned so that we can reconstruct the own interpretation of the writer lock guard. (The RustBelt framework requires us to establish such reconstruction proof when performing the reborrow.)  $\square$

*Proof sketch of `rdguard_drop`.* We get to consume the reader lock guard, that is  $\llbracket \text{rwlockreadguard}(\alpha, \tau) \rrbracket.\text{own}(t, [\ell])$ . This is the reverse of the `try_read` proof: we use the GPS protocol resources in the own interpretation for the lock guard to perform the CAS that decrements the lock counter, and we then return all resources from the lock guard's own interpretation to the protocol's write interpretation  $\mathcal{I}_w(\tau, \alpha, \gamma_s, t)$ . Additionally, we need to update the ghost state to decrement the ghost counter. In doing so, we deallocate our reader state  $\llbracket \text{Reading}(q, \nu) \rrbracket^{\gamma_s}$ .

In case we are the only existing reader lock guard, we kill the lifetime  $\nu$  (using  $\square \text{Kill}(\nu)$ ), so that we can inherit  $\&_{\text{full}}^{\alpha} (\ell + 1 \mapsto \llbracket \tau \rrbracket.\text{own}(t))$ , i.e., switching the access mode of the underlying type  $\tau$  from shared to owned. Note that in this case we also regain the full read and write permissions of the GPS protocol. All of those resources allow us to set the protocol ghost state back to  $a = \text{None}$ , i.e., to unlock the lock.  $\square$

*Proof sketch of `wrguard_drop`.* We get to consume the writer lock guard,  $\llbracket \text{rwlockwriteguard}(\alpha, \tau) \rrbracket.\text{own}(t, [\ell])$ . The GPS protocol writer permission allows us to know that the protocol is in state  $a = \text{Some}(\text{left}())$ , as well as to simply perform a release write of 0 to release the lock. From the writer lock guard's own interpretation, we have all the resources needed to set the protocol ghost state back to  $a = \text{None}$ . When performing the ghost update, we also deallocate our writer state  $\llbracket \text{Writing} \rrbracket^{\gamma_s}$ .  $\square$

CHAPTER SUMMARY. In this chapter, we sketched the semantic type checking of a  $\lambda_{\text{Rust}}$  version of Rust's reader-writer lock library (which employs `unsafe` blocks), using a combination of the relaxed lifetime logic and iRC11 GPS protocols.



# 19

## Verification of `Arc`

---

The verification of the `Arc` library is by far the most challenging library verification in `RBr1x`. In the original RustBelt work, the verification of `Arc` is split into 2 steps: first, the implementation is verified against a sufficiently strong abstract specification, and then the semantic type checking of `Arc`'s functions is done using only that abstract specification. While the latter task is no less challenging, we managed to reuse that part of proofs entirely for `RBr1x`. Therefore, in this chapter, we focus on the first task: reverifying the now-relaxed implementation against the strong abstract specification. Fortunately, the specification is reasonably balanced to also admit the relaxed implementation, so we did not need to change it—we only need to redo the proofs in `iRC11`.

Regardless, to make the verification go through, we needed to strengthen two atomic reads from `r1x` to `acq` in the implementations of `Arc::get_mut` and `Arc::make_mut`. We conjecture that the relaxed access in `Arc::make_mut` is indeed sound but verifying this would have required a significantly more complex invariant. The relaxed access in `Arc::get_mut` turned out to be a bug. We provide more details about this bug in §19.4.

In §19.1, we present the core `Arc` implementation in  $\lambda_{\text{Rust}} + \text{ORC11}$  and sketch how to verify it using cancelable GPS protocols in §19.2. In §19.3, we present the *full* APIs of `Arc`, which involves the associate type `Weak`, and sketch how to verify the full APIs. The synchronization bug and a bit of history how it was found is mentioned in §19.4.

### 19.1 The Core `Arc` library

`Arc<T>`, short for *Atomically Reference Counted*, is used to share atomically an object of type `T`, whose mutation is disabled by default. To mutate `T`, one needs `T` to support thread-safe mutability, for example with `T` being an atomic type, or with `T` wrapped inside a lock (e.g., `Mutex<T>`).

**Example 19.1** (A Client of `Arc`). The following Rust example instantiates `Arc` with an atomic integer `AtomicUsize` and demonstrates how `Arc` is typically used:

```
1 let arc1 = Arc::new(AtomicUsize::new(5)); // create the first Arc
2 let arc2 = Arc::clone(&arc1); // clone the second Arc
3 thread::spawn(move || { // give arc2 to child thread
4     println!("child: {:?}", arc2.fetch_add(1, Ordering::Relaxed));
```

```

5    // drop(arc2);
6  });
7  println!("main: {:?}", arc1.fetch_add(2, Ordering::Relaxed));
8  // drop(arc1);

```

In line 1 in the main thread, a new `Arc` pointer `arc1` is created to govern an atomic integer allocated in shared memory. The `Arc`'s internal `counter` field for the number of references to the content is set to 1. An `Arc` pointer acts almost like its underlying content, so in line 4 we can call `fetch_add` on `arc1` as if on the atomic integer itself. To share the content with the child thread, we create another `arc2` by `clone`-ing `arc1` (line 2), which effectively increments the internal counter to 2: there are now 2 pointers sharing the atomic integer. Unsurprisingly, to allow concurrent updates by multiple threads, the internal `counter` field is implemented with an atomic integer.

When the `Arc` pointers go out of scope, in lines 5 and 8, their destructors—the `drop` function—are called and the `counter` field is decremented accordingly. The last call of `drop` will deallocate the underlying content *and* the `counter` field.

The  $\lambda_{\text{Rust}} + \text{ORC11}$  implementation of Core `Arc`'s functions is given in Figure 19.1. The `new` function allocates two locations, one for the `counter` field and one for the `data` field, then initializes them. Note that we present Core `Arc` as a simplification of the actual implementation verified in our Coq development: we assume here that the type `T` only has size 1, but the actual implementation allocates  $\llbracket \tau \rrbracket.\text{size}$  locations (memory cells) to be able to store `T`. Furthermore, the actual implementation needs to consume all of those memory cells (copying them into the allocated memory and freeing the memory for the argument of type `T`), instead of just assuming that the argument is just a simple value  $v$ .

The `deref` function provides access to the `data` field, effectively allowing an `Arc<T>` to behave like its content `T`. The `clone` function does a relaxed (**rlx**) *fetch-and-add* (**FAA**) by 1 to increment `counter` and then return a copy of `a`.

Finally, the `drop` function does a *release* (**rel**) **FAA** to decrement `counter`. If the value of `counter` was 1 before the decrement (*i.e.*, this is the last `drop`), `drop` additionally does an acquire (**acq**) fence before deallocating both the `counter` and `data` fields. The acquire fence is needed because the release **FAA**, although being a release write, is only a relaxed read.

**CORRECTNESS.** Intuitively, the main correctness guarantee of Core `Arc` is that the deallocation of its `data` and `counter` fields is synchronized with (happens-after) *all* accesses to those fields. Those accesses happen between (and including) the construction of an `Arc` pointer, either by `new` or `clone`, and its destruction by `drop`. Therefore, the correctness guarantee translates to making sure that the deallocation done by the last `drop` is synchronized with all previous `drop`'s. In this case, that synchronization is established between the release **FAA**'s of all previous `drop`'s and the acquire fence of the last `drop`.



|                                                                                                                                                                        |                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> new(v) ::= let a := alloc(2) in           a.counter :=<sub>na</sub> 1;           a.data :=<sub>na</sub> v;           a deref(a) ::= *<sub>na</sub> a.data </pre> | <pre> drop(a) ::= if FAA<sup>rel</sup>(a.counter, -1) == 1             fence<sub>acq</sub>;             free(a, 2) clone(a) ::= FAA<sup>rlx</sup>(a.counter, 1);             a </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

FIGURE 19.1: Implementation of Core Arc

## 19.2 Verification of Core Arc with Cancelable GPS Protocols

We demonstrate the verification of the most important functions of Core Arc: `new`, `clone` and `drop`. For `clone`, we need to guarantee that any newly-created pointer `arc` to an object `a` can *non-atomically read* its `data` field `a.data` (so that the `deref` function can be called on `arc`), and perform *atomic FAA*'s on its `counter` field `a.counter` (so that `clone` and `drop` can be called on `arc`). This means that both fields must be *shared* for concurrent accesses by multiple threads.

For `drop`, we instead show that this sharing of the fields must have been finished before the deallocation is called. To deallocate a block `a` of two locations using `free(a, 2)`, the deallocation rule **NA-DEALLOC** (Figure 9.1, §9.1) requires us to have the full ownership of the whole block *i.e.*, both  $a.data \mapsto \_$  and  $a.counter \mapsto \_$ .

In short, we start out with the full ownership  $a.data \mapsto v$  and  $a.counter \mapsto 1$  in the `new` function, then we share both  $a.data$  and  $a.counter$  for concurrent accesses, and at the end reclaim both  $a.data \mapsto \_$  and  $a.counter \mapsto \_$  in the last `drop` for deallocation. Our job is to set up the sharing to satisfy this scheme. Because the `data` field only needs concurrent *read* accesses, we employ *fractional ownership*<sup>1</sup> on the points-to assertion  $a.data \mapsto v$ .<sup>2</sup> That is, we start out with the full fraction  $a.data \mapsto v = a.data \mapsto v$  and for every newly-created pointer we give it a small fraction  $a.data \xrightarrow{q} v$ , which is sufficient to perform concurrent reads. When a pointer goes out of scope, its small fraction  $a.data \xrightarrow{q} v$  is recollected. Before the very end, we recollect all the small fractions into the full fraction  $a.data \mapsto v = a.data \mapsto v$ . Then we are ready for deallocating `a.data`.

The `counter` field, on the other hand, needs concurrent **FAA** accesses, so we will use a cancelable GPS protocol (Definition 17.4) to share it. The cancelable protocol is also used for recollecting the small fractions of the `data` field.

<sup>1</sup>Boyland, “Checking interference with fractional permissions” [Boy03].

<sup>2</sup>To allow concurrent atomic writes to the underlying object of type **T** with only some fractional ownership, the type needs to support interior mutability. This can be achieved if we can *e.g.*, also use a cancelable GPS protocol for **T**.

### 19.2.1 The Abstract Specifications for Core Arc

We want to prove the following simple specification for Core Arc:

$$\begin{aligned}
& \{\text{True}\} \text{new}(v) \{a. \exists \gamma_i, \gamma. \text{ARC}^{\gamma_i, \gamma}(a, v, \mathcal{I})\} \quad (\text{ARC-NEW-SPEC}) \\
& \{\text{ARC}^{\gamma_i, \gamma}(a, v, \mathcal{I})\} \text{clone}(a) \{ \text{ARC}^{\gamma_i, \gamma}(a, v, \mathcal{I}) * \text{ARC}^{\gamma_i, \gamma}(a, v, \mathcal{I}) \} \\
& \hspace{15em} (\text{ARC-CLONE-SPEC}) \\
& \{\text{ARC}^{\gamma_i, \gamma}(a, v, \mathcal{I})\} \text{drop}(a) \{\text{True}\} \quad (\text{ARC-DROP-SPEC})
\end{aligned}$$

where the abstract predicate  $\text{ARC}^{\gamma_i, \gamma}(\mathbf{a}, v, \mathcal{I})$  represents the logical ownership of an Arc pointer to some object  $\mathbf{a}$  whose data is  $v$ . The arguments  $\gamma_i$ ,  $\gamma$ , and  $\mathcal{I}$  are used for the cancelable GPS protocol.

**ARC-NEW-SPEC** says that `new`( $v$ ) allocates a new Arc pointer, and **ARC-CLONE-SPEC** says that `clone` can create two pointers from one. **ARC-DROP-SPEC** simply says that a `drop` is always safe with just a Arc pointer ownership.

### 19.2.2 The GPS Protocol for Core Arc

**Definition 19.2** (Model of the ARC Assertion). We define the abstract predicate  $\text{ARC}^{\gamma_i, \gamma}(\mathbf{a}, v, \mathcal{I})$  as follows:

$$\text{ARC}^{\gamma_i, \gamma}(\mathbf{a}, v, \mathcal{I}) ::= \exists q. \mathbf{a}.\text{data} \xrightarrow{q} v * \boxed{(\mathbf{a}.\text{counter}, \gamma_i, \_): \_ \mid \mathcal{I}^{\gamma, v}}_q * \heartsuit_q^{\gamma_i} * [\text{Count}(q)]^\gamma$$

Owning an Arc pointer  $\text{ARC}^{\gamma_i, \gamma}(\mathbf{a}, v, \mathcal{I})$  means that we own: (1) some small fraction  $q$  of the `data` field  $\mathbf{a}.\text{data} \xrightarrow{q} v$  at the value  $v$ , which allows us to safely read  $\mathbf{a}.\text{data}$  for the value  $v$ ;<sup>3</sup> (2) the CAS-only assertion  $\boxed{(\mathbf{a}.\text{counter}, \gamma_i, \_): \_ \mid \mathcal{I}^{\gamma, v}}_q$  that the `counter` field is governed by a protocol  $\mathcal{I}$  protected by  $\gamma_i$ , as well as the cancelable invariant token  $\heartsuit_q^{\gamma_i}$ —with the *same* fraction  $q$ —to know that the protocol is still alive, which allows us to concurrently access  $\mathbf{a}.\text{counter}$ ; and finally (3) an *unsynchronized* ghost element  $[\text{Count}(q)]^\gamma$  that represent the 1 *single* count of this pointer in the *total* count (see below).

**Definition 19.3** (GPS Protocol for  $\mathbf{a}.\text{counter}$ ). The protocol for the location  $\mathbf{a}.\text{counter}$  is defined as follows:

$$\begin{aligned} \mathcal{I}^{\gamma, v} &::= (\mathcal{I}_w^{\gamma, v}, \mathcal{I}_r, \mathcal{I}_m) \\ \mathcal{I}_r(\mathbf{a}, \gamma_{\mathbf{a}}, \_, \_, n) &::= n \geq 0 \\ \mathcal{I}_m(\mathbf{a}, \gamma_{\mathbf{a}}, \_, \_, n) &::= n = 0 \\ \mathcal{I}_w^{\gamma, v}(\mathbf{a}, \gamma_{\mathbf{a}}, \_, \_, n) &::= \begin{cases} \text{False} & n < 0 \\ [\text{TotalCount}(0, 0)]^\gamma & n = 0 \\ \exists q_{\text{in}}, q_{\text{out}} \in (0, 1). \mathbf{a}.\text{data} \xrightarrow{q_{\text{in}}} v * \heartsuit_{q_{\text{in}}}^{\gamma_i} * \mathcal{W}_{\text{shr}}(\mathbf{a}, \gamma_{\mathbf{a}}, \_, \_, n) * \mathcal{R}_{\text{shr}}^{q_{\text{in}}}(\mathbf{a}, \gamma_{\mathbf{a}}, \_, \_, n) & n > 0 \\ * q_{\text{in}} + q_{\text{out}} = 1 * [\text{TotalCount}(n, q_{\text{out}})]^\gamma \end{cases} \end{aligned}$$

(ARC-INV)

First of all,  $\mathcal{I}$  requires that the value  $v$  of the `counter` field to be non-negative. When it is positive *i.e.*, when there is some Arc pointers, the number of pointers is  $n$  and the write interpretation  $\mathcal{I}_w^{\gamma, v}$  owns the unsynchronized ghost element  $[\text{TotalCount}(v, q_{\text{out}})]^\gamma$ . We use this ghost element to track the protocol state, and so have no need for the GPS protocol type, which we pick to be the trivial unit protocol  $S_\emptyset$ .

The element  $[\text{TotalCount}(v, q_{\text{out}})]^\gamma$  tracks the globally-consistent knowledge that there are currently  $n$  pointers and the *sum* of all fractional permissions owned by those pointers is  $q_{\text{out}}$ .<sup>4</sup> The write interpretation further requires that the remaining fraction  $q_{\text{in}} = 1 - q_{\text{out}}$  must be owned

<sup>3</sup>In the general case, this would be fractional ownership of the underlying type  $\tau$ 's interpretation, which may allow more than read access.

<sup>4</sup>Here, *out* means ownership of the fractions outside of  $\mathcal{I}_w^{\gamma, v}$ .

$$\begin{array}{l}
\text{COUNTING-START} \\
q \in (0, 1] \vdash \exists \gamma. [\text{TotalCount}(1, q)]^\gamma * [\text{Count}(q)]^\gamma \\
\\
\text{COUNTING-NEW} \\
q + q' \leq 1 \vdash [\text{TotalCount}(n, q)]^\gamma \dot{\Rightarrow} [\text{TotalCount}(n+1, q+q')]^\gamma * [\text{Count}(q')]^\gamma \\
\\
\text{COUNTING-AGREE} \\
[\text{TotalCount}(n, q)]^\gamma * [\text{Count}(q')]^\gamma \vdash n \geq 1 \wedge q' \leq q \leq 1 \\
\\
\text{COUNTING-DROP} \\
[\text{TotalCount}(n+1, q+q')]^\gamma * [\text{Count}(q')]^\gamma \dot{\Rightarrow} [\text{TotalCount}(n, q)]^\gamma \wedge (n = 0 \Rightarrow q = 0)
\end{array}$$

FIGURE 19.2: Counting permissions for Core Arc

by the protocol. This includes the fractional ownership of `a.data` and the cancelable token  $\heartsuit_{q_{in}}^{\gamma_i}$  of `a.counter`. The fraction  $q_{in}$  in fact includes the *used* fraction of pointers that have been *dropped*. Thus the protocol makes sure that any fractions of the `a.data` and  $\gamma_i$  are all accounted for.

Finally, when the *counter* reaches 0, the protocol is simply trivial with the ghost element also being 0. Note that the move interpretation  $\mathcal{I}_m$  allows only for this case, which means that we can perform a single-writer write once the counter reaches 0. This is simply because there is no more sharing in this case. Such a single-writer write can be useful in resetting and reusing the reference counting, which will be needed in Full Arc (§19.3).

The ghost elements  $[\text{TotalCount}(n, q)]^\gamma$  and  $[\text{Count}(q)]^\gamma$  is an instance of *counting permissions*,<sup>5</sup> used here to track the outside fractions associated with each single count. They satisfy the axioms in Figure 19.2. **COUNTING-START** creates a ghost location  $\gamma$  for the first count and gives us the total count  $[\text{TotalCount}(1, q)]^\gamma$  as well as a single count  $[\text{Count}(q)]^\gamma$ . With **COUNTING-NEW** we can increase the total count and produce more single counts. With **COUNTING-DROP** we can decrease the total count by consuming single counts. **COUNTING-AGREE** ensures that every single count is always included in the total count. How this ghost construction comes into play will be revealed next section.

After this long setup, we are finally ready to explain the verification of Core Arc.

<sup>5</sup>Bornat et al., “Permission accounting in separation logic” [Bor+05].

### 19.2.3 Verifying new

In the proof of **ARC-NEW-SPEC**, we elide the standard allocation and initialization of the *data* and *counter* fields. Our main obligation here is to transform the two full ownership  $\text{a.data} \mapsto v$  and  $\text{a.counter} \mapsto 1$  to the abstract permission  $\text{ARC}^{\gamma_i, \gamma}(\text{a}, v, \mathcal{I})$  for some  $\gamma_i$  and  $\gamma$ . That is, turning our unique ownership into sharing mode.

To do so, we initialize an iRC11 cancelable single-writer protocol for `a.counter`, using the rule **GPS-SW-INIT**.<sup>6</sup> The rule gives us fresh ghost locations  $\gamma_i$  and  $\gamma_a$  and the full cancelable token  $\heartsuit_1^{\gamma_i}$ . What we need to provide are the points-to  $\text{a.counter} \mapsto 1$ , which we have, and the write

<sup>6</sup>see Figure 17.4

interpretation  $\mathcal{I}_w^{\gamma,v}(\mathbf{a}, \gamma_{\mathbf{a}}, -, -, 1)$  for some  $\gamma$ . Below, we write  $\mathcal{I}_w^{\gamma,v}(1)$  as a short-hand notation for  $\mathcal{I}_w^{\gamma,v}(\mathbf{a}, \gamma_{\mathbf{a}}, -, -, 1)$ .

For  $\gamma$ , we use **COUNTING-START** to create the total count and the first single count with  $q ::= 1/2$ . That is, we get  $\boxed{\text{TotalCount}(1, 1/2)}^\gamma$  and  $\boxed{\text{Count}(1/2)}^\gamma$ . We use  $\boxed{\text{TotalCount}(1, 1/2)}^\gamma$  for  $\mathcal{I}_w$  and  $\boxed{\text{Count}(1/2)}^\gamma$  for ARC. Similarly, we split  $\mathbf{a.data} \mapsto v$  into two halves  $\mathbf{a.data} \xrightarrow{1/2} v$ 's and use each for  $\mathcal{I}$  and ARC.

With  $\mathbf{a.data} \xrightarrow{1/2} v$  and  $\boxed{\text{TotalCount}(1, 1/2)}^\gamma$ , to establish  $\mathcal{I}_w^{\gamma,v}(1)$ , we need  $\heartsuit_{1/2}^{\gamma_i}$ , and  $\mathcal{W}_{\text{shr}}(\mathbf{a}, \gamma_{\mathbf{a}}, -, -, 1)$ , and  $\mathcal{R}_{\text{shr}}^{1/2}(\mathbf{a}, \gamma_{\mathbf{a}}, -, -, 1)$ . Unfortunately, **GPS-SW-INIT** does not allow us to use some of the cancelable token nor the single-writer permission  $\mathcal{W}(\ell, \gamma, t, s, v)$  to establish  $\mathcal{I}_w$ . Therefore, we need to use a more general initialization rule that has the flavors of both **CINV-ALLOC-FRAC**<sup>7</sup> and **GPS-MID-SW-INIT-STRONG**<sup>8</sup> combined. Concretely, we want keep half of cancelable token ( $\heartsuit_{1/2}^{\gamma_i}$ ) locally, and use the other half for  $\mathcal{I}_w^{\gamma,v}(1)$ . Furthermore, we use the rule **GPS-W-WSHR-RSHR**<sup>9</sup> to turn the single-writer permission  $\mathcal{W}(\mathbf{a}, \gamma_{\mathbf{a}}, -, -, 1)$  into the shared-writer permission  $\mathcal{W}_{\text{shr}}(\mathbf{a}, \gamma_{\mathbf{a}}, -, -, 1)$  and the full shared-reader permission  $\mathcal{R}_{\text{shr}}^1(\mathbf{a}, \gamma_{\mathbf{a}}, -, -, 1)$ . We use the former and *half* ( $1/2$ ) of the latter to complete  $\mathcal{I}_w^{\gamma,v}(1)$ .

As a result, we get  $\heartsuit_{1/2}^{\gamma_i} * \boxed{(\mathbf{a.counter}, \gamma_i, \gamma_{\mathbf{a}}) : - \mid \mathcal{I}_w^{\gamma,v}}_{1/2}$ . We combine these with the remaining  $\mathbf{a.data} \xrightarrow{1/2} v$  and  $\boxed{\text{Count}(1/2)}^\gamma$  to complete the first  $\text{ARC}^{\gamma_i, \gamma}(\mathbf{a}, v, \mathcal{I})$  permission.  $\square$

#### 19.2.4 Verifying clone

In proving **ARC-CLONE-SPEC**, we need to split one  $\text{ARC}^{\gamma_i, \gamma}(\mathbf{a}, v, \mathcal{I})$  into two. Unfolding the definition of  $\text{ARC}^{\gamma_i, \gamma}(\mathbf{a}, v, \mathcal{I})$ ,<sup>10</sup> we see that the fractions  $\mathbf{a.data} \xrightarrow{q} v$ ,  $\boxed{(\mathbf{a.counter}, \gamma_i, -) : - \mid \mathcal{I}_w^{\gamma,v}}_q$ , and  $\heartsuit_q^{\gamma_i}$  (for some  $q$ ) can be split into halves i.e.,  $\mathbf{a.data} \xrightarrow{q/2} v$ ,  $\boxed{(\mathbf{a.counter}, \gamma_i, -) : - \mid \mathcal{I}_w^{\gamma,v}}_{q/2}$ , and  $\heartsuit_{q/2}^{\gamma_i}$ , each for one new ARC. So we only need to transform one single count  $\boxed{\text{Count}(q)}^\gamma$  into two. To match the fraction  $q/2$ , we actually need two single counts of the form  $\boxed{\text{Count}(q/2)}^\gamma$ . Unfortunately,  $\boxed{\text{Count}(q)}^\gamma$  is *not* splittable into two  $\boxed{\text{Count}(q/2)}^\gamma$ 's. So we can only get two  $\boxed{\text{Count}(q/2)}^\gamma$ 's with the help of the total count  $\boxed{\text{TotalCount}(-, -)}^\gamma$ , which is inside the GPS protocol. To do so, at the relaxed **FAA** made by **clone** (Figure 19.1), we invoke the CAS rule for the single-writer protocol in the *CAS-only* mode. We did not present this rule, but it is very much the same as the rule **GPS-ATBOR-WSHR-CAS-INT**<sup>11</sup> for the Atomic-Borrows-based single-writer variant of GPS protocols, where we only need to replace the lifetime token fraction  $[\kappa]_q$  with the cancelable token  $\heartsuit_q^{\gamma_i}$ —which we do have from  $\text{ARC}^{\gamma_i, \gamma}(\mathbf{a}, v, \mathcal{I})$ .

Note that, in **clone**, we use a *relaxed FAA* which is a relaxed read and a relaxed write. Therefore, in the successful case of the **FAA**, we only get to access the write interpretation  $\mathcal{I}_w^{\gamma,v}$  under the release modality  $\Delta$  and we can only get some resource out under the acquire modality  $\nabla$ .<sup>12</sup> But remember that, if our resource is, however, view-agnostic—for example, if they are unsynchronized ghost state—then the fence modalities can be bypassed. We exploit this in our invocation of the CAS rule for **clone**.

<sup>7</sup>see Figure 11.3

<sup>8</sup>see Figure 17.8

<sup>9</sup>see Figure 17.3

<sup>10</sup>see Definition 19.2

<sup>11</sup>see Figure 17.7

<sup>12</sup>see §8.3

In particular, as we have  $\{\text{Count}(q)\}^\gamma$ , we use **GHOST-RELMOD**<sup>13</sup> to get  $\Delta\{\text{Count}(q)\}^\gamma$ .

<sup>13</sup>see Figure 8.1

Then, we now have to show that

$$\mathcal{I}_w^{\gamma,v}(n) * \{\text{Count}(q)\}^\gamma \Rightarrow \mathcal{I}_w^{\gamma,v}(n+1) * \{\text{Count}(q/2)\}^\gamma * \{\text{Count}(q/2)\}^\gamma,$$

where  $n$  is the value the **FAA** reads from `a.counter`.

First, by the definition of  $\mathcal{I}_w^{\gamma,v}(n)$  (see **ARC-INV**), we know that  $n \geq 0$ . By owning  $\{\text{Count}(q)\}^\gamma$ , we also know that  $n$  cannot be 0, because if  $n = 0$ , we can combine  $\{\text{TotalCount}(0, 0)\}^\gamma$  with  $\{\text{Count}(q)\}^\gamma$  and use the rule **COUNTING-AGREE** to derive the contradiction that  $0 \geq 1$ . Thus  $n > 0$ .

Now, we are not going to change the fractions ( $q_{\text{in/out}}$ ) and the fractional ownerships: we will keep them the same (i.e., *framing*) for  $\mathcal{I}_w^{\gamma,v}(n+1)$ . Therefore our job is simply transform  $\{\text{TotalCount}(n, q_{\text{out}})\}^\gamma * \{\text{Count}(q)\}^\gamma$  to  $\{\text{TotalCount}(n+1, q_{\text{out}})\}^\gamma * \{\text{Count}(q/2)\}^\gamma * \{\text{Count}(q/2)\}^\gamma$ . This is simple: we first use **COUNTING-DROP** to drop the single count  $\{\text{Count}(q)\}^\gamma$  associated with  $q$  and get  $\{\text{TotalCount}(n-1, q_{\text{out}}-q)\}^\gamma$ . We then call **COUNTING-NEW** twice on  $\{\text{TotalCount}(n-1, q_{\text{out}}-q)\}^\gamma$ , each time creating a new single count  $\{\text{Count}(q/2)\}^\gamma$  and in the end we get back  $\{\text{TotalCount}(n+1, q_{\text{out}})\}^\gamma$ . Note that we always satisfy the side condition of **COUNTING-NEW** because  $q_{\text{out}} \leq 1$ .

Finally, after the access, we get back the access token  $\heartsuit_q^{\gamma_i}$  and two single counts under the acquire modality:

$$\nabla \left( \{\text{Count}(q/2)\}^\gamma * \{\text{Count}(q/2)\}^\gamma \right)$$

Since the single counts are unsynchronized ghost state, we use **ACQMOD-GHOST**<sup>14</sup> to get  $\{\text{Count}(q/2)\}^\gamma * \{\text{Count}(q/2)\}^\gamma$ . Now we can split the token  $\heartsuit_q^{\gamma_i}$  and the fraction ownership  $\text{a.data} \xrightarrow{q} v$  into two halves and gain two  $\text{ARC}^{\gamma_i, \gamma}(\text{a}, v, \mathcal{I})$ 's.  $\square$

<sup>14</sup>see Figure 8.1

### 19.2.5 Verifying drop

The first intuition in the proof of **drop** is that, if the **drop** is not the last drop, we will return all the resources of the current pointer  $\text{ARC}^{\gamma_i, \gamma}(\text{a}, v, \mathcal{I})$  to the protocol. This includes the fractional ownership  $\text{a.data} \xrightarrow{q} v$ , the shared-reader permission  $\boxed{(\text{a.counter}, \gamma_i, -) : -} \mathcal{I}_w^{\gamma,v}$ , the cancelable token  $\heartsuit_q^{\gamma_i}$  and the single count element  $\{\text{Count}(q)\}^\gamma$ . The former three will be stored in the protocol and will be transferred to the last **drop** for deallocation. The single count element will be used to decrease the total count by 1.

The second intuition is that, in the case of the last **drop**, we know from the **ARC** permission and the protocol's write interpretation that the local fractions and the fractions stored in the protocol sum up to 1, so we can recollect the full fraction for deallocation.

In both cases, we need a stronger rule for *release* **FAA** that allow us to use the token  $\heartsuit_q^{\gamma_i}$  to access the protocol and *simultaneously* use the token to establish the write interpretation. Consequently, we would not regain  $\heartsuit_q^{\gamma_i}$  in the post-condition of such a rule. While we do not write down this rule concretely, it has the same flavor as the release single-writer rule **GPS-SW-WRITE-REL**.<sup>15</sup>

<sup>15</sup>see Figure 17.4

Now, at the release **FAA** of **drop**, using  $\heartsuit_q^{\gamma_i}$ , we have the following goal  $\heartsuit_q^{\gamma_i} * P * \mathcal{I}^{\gamma,v}(n) \Rightarrow^* \mathcal{I}^{\gamma,v}(n-1) * Q(n)$  where  $n$  is the old value of `a.counter` and

$$P ::= \text{a.data} \vdash^q v * \boxed{(\text{a.counter}, \gamma_i, \_) : \_} \boxed{\mathcal{I}^{\gamma,v}}_q * \boxed{\text{Count}(q)}^\gamma$$

$$Q(n) ::= \begin{cases} \text{True} & n \neq 1 \\ \text{a.data} \mapsto v * \heartsuit_1^{\gamma_i} & n = 1 \\ * \mathcal{W}_{\text{shr}}(\text{a}, \gamma_{\text{a}}, \_, \_, 0) & \\ * \boxed{(\text{a.counter}, \gamma_i, \_) : \_} \boxed{\mathcal{I}^{\gamma,v}}_1 & \end{cases}$$

That is,  $P$  are the resources we have locally in our pre-condition, and  $Q$  are the resources we will have in our post-condition of the rule.

Similarly to the reasoning in **clone**, with  $\boxed{\text{Count}(q)}^\gamma$  from  $P$ , we know that  $n > 0$  and the invariant has some fractional permissions  $\heartsuit_{q_{\text{in}}}^{\gamma_i}$  and  $\text{a.data} \vdash^{q_{\text{in}}} v$  for some  $q_{\text{in}}$  (see **ARC-INV**).

Now, if this is not the last **drop** i.e.,  $n-1 > 0$ , we need to re-establish  $\mathcal{I}^{\gamma,v}(n-1)$  with some new fractions  $q'_{\text{in/out}}$ . We pick them as follows:  $q'_{\text{in}} := q_{\text{in}} + q$  and  $q'_{\text{out}} := q_{\text{out}} - q$ . From  $\heartsuit_q^{\gamma_i} * P * \mathcal{I}^{\gamma,v}(n)$ , we can easily get  $\heartsuit_{q_{\text{in}}}^{\gamma_i} = \heartsuit_{q_{\text{in}}}^{\gamma_i} * \heartsuit_q^{\gamma_i}$  and  $\text{a.data} \vdash^{q_{\text{in}}} v = \text{a.data} \vdash^{q_{\text{in}}} v * \text{a.data} \vdash^q v$ , as well as  $\boxed{(\text{a.counter}, \gamma_i, \_) : \_} \boxed{\mathcal{I}^{\gamma,v}}_{q'_{\text{in}}}$ , which are needed for  $\mathcal{I}^{\gamma,v}(n-1)$ .

Our remaining work is to transform  $\boxed{\text{TotalCount}(n, q_{\text{out}})}^\gamma * \boxed{\text{Count}(q)}^\gamma$  to  $\boxed{\text{TotalCount}(n-1, q'_{\text{out}})}^\gamma$ . Fortunately, this is but a simple application of **COUNTING-DROP**. Then we are done because  $n \neq 1$ .

In the case where this is the last **drop's FAA**, we have  $n = 1$  and we must prove  $\heartsuit_q^{\gamma_i} * P * \mathcal{I}^{\gamma,v}(1) \Rightarrow^* \mathcal{I}^{\gamma,v}(0) * Q(1)$ . From  $\mathcal{I}^{\gamma,v}(1)$  we have  $\boxed{\text{TotalCount}(1, q_{\text{out}})}^\gamma$  and from  $P$  we have  $\boxed{\text{Count}(q)}^\gamma$ . By an application of **COUNTING-DROP**, we have  $\boxed{\text{TotalCount}(0, 0)}^\gamma$ , which is exactly  $\mathcal{I}^{\gamma,v}(0)$ , and additionally the fact that  $q_{\text{out}} = q$ . From  $\mathcal{I}^{\gamma,v}(1)$  we also know that  $q_{\text{in}} + q_{\text{out}} = q_{\text{in}} + q = 1$ . Thus combining what we have left from our assumption  $\heartsuit_q^{\gamma_i} * P * \mathcal{I}^{\gamma,v}(1)$ , we have exactly  $Q(1)$ . So we finish the last **drop's FAA** and gain  $\nabla Q(1)$ .

As the return value is  $n = 1$ , we perform an acquire fence (see the code of **drop** in Figure 19.1). Thanks to the acquire fence rule **HOARE-ACQ-FENCE**,<sup>16</sup> we remove the modality and regain  $Q(1)$ . We are almost done: we only need to get back the points-to ownership of `a.counter`. For this, we *cancel* the GPS single-writer protocol for `a.counter` using the cancellation rule **GPS-SW-DEALLOC**.<sup>17</sup> The rule requires the full token  $\heartsuit_1^{\gamma_i}$ , which we do have, to ensure that the cancellation happens after all accesses to the protocol. At long last, after the cancellation we now have the full ownership of both fields and can safely use **NA-DEALLOC** to free them.  $\square$

Note that in this proof, before applying **GPS-SW-DEALLOC**, we can first use the rule **GPS-SW-W-REVERT**<sup>18</sup> to regain the single-writer assertion  $\boxed{(\text{a.counter}, \gamma_i, \_) : \_} \boxed{\mathcal{I}^{\gamma,v}}_W$ . With that and a *stronger* variant of **GPS-SW-DEALLOC**,<sup>19</sup> we can regain exactly  $\mathcal{I}_w^{\gamma,v}(\text{a}, \gamma_{\text{a}}, \_, \_, 0)$  which will allow us to reuse the counting permissions at the ghost location  $\gamma$ , if needed.

<sup>16</sup>see Figure 8.1

<sup>17</sup>see Figure 17.4

<sup>18</sup>see Figure 17.3

<sup>19</sup>see the discussion for **GPS-SW-DEALLOC** in §17.1.2

| Arc                                                                            | Weak                                                                          |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>new: fn(T) -&gt; Arc&lt;T&gt;</code>                                     | <code>new: fn() -&gt; Weak&lt;T&gt;</code>                                    |
| <code>deref: fn(&amp;Arc&lt;T&gt;) -&gt; &amp;T</code>                         |                                                                               |
| <code>clone: fn(&amp;Arc&lt;T&gt;) -&gt; Arc&lt;T&gt;</code>                   | <code>clone: fn(&amp;Weak&lt;T&gt;) -&gt; Weak&lt;T&gt;</code>                |
| <code>downgrade: fn(&amp;Arc&lt;T&gt;) -&gt; Weak&lt;T&gt;</code>              | <code>upgrade: fn(&amp;Weak&lt;T&gt;) -&gt; Option&lt;Arc&lt;T&gt;&gt;</code> |
| <code>drop: fn(Arc&lt;T&gt;) -&gt; ()</code>                                   | <code>drop: fn(Weak&lt;T&gt;) -&gt; ()</code>                                 |
| <code>get_mut: fn(&amp;mut Arc&lt;T&gt;) -&gt; Option&lt;&amp;mut T&gt;</code> |                                                                               |
| <code>make_mut: fn(&amp;mut Arc&lt;T&gt;) -&gt; &amp;mut T</code>              |                                                                               |

TABLE 19.1: An excerpt of Rust’s `Arc<T>` and `Weak<T>` APIs

### 19.3 Verification of Arc’s Full APIs

We discuss the verification of an extended version of `Arc`, which is also the version we have verified in `RBr1x`. Its most interesting APIs are given in Table 19.1. Here we need to tackle two extra sets of behaviors, presented as two following challenges.

`Arc<T>` HAS A SUBORDINATE TYPE `Weak<T>`. The first challenge involves a type called `Weak<T>`. `Weak` itself is a variant of `Arc`: it has a counter to count how many `Weak` pointers are in existence, and also has the similar `clone` and `drop` functions.<sup>20</sup> However, `Weak` does not guarantee access to the underlying object of type `T`: while owning an `Arc` guarantees that the object is still available, owning a `Weak` does not prevent the object to be reclaimed. In order to access the object with a `Weak` pointer, one first calls `Weak::upgrade` to obtain an `Arc` pointer. `upgrade` can fail when the object has already been reclaimed, that is when there is no `Arc` pointer left. A `Weak` pointer are typically created by calling `Arc::downgrade` on a shared reference of `Arc`.

<sup>20</sup>see Table 19.1

The challenge for verifying `Arc` and `Weak` in the relaxed memory setting is that they involve two tightly coupled atomic locations—one for each counter. As multi-location RMC invariants (as presented in Chapter 11) are consolidated at a much later time after the `RBr1x` work, we needed to use separate GPS protocols for each counter and at the same time maintain their relation. This was a known challenge, as had been observed earlier by GPS.<sup>21</sup> The general solution is to construct ghost state to encode the relation between the locations and prevent their protocols from breaking the relation. We were able to set up several unsynchronized ghost state constructions to encode the relation, but those, unfortunately, are not enough.

<sup>21</sup>Turon et al., “GPS: navigating weak memory with ghosts, protocols, and separation” [TVD14].

`Arc<T>` SUPPORTS TEMPORARY BORROWS OF THE UNDERLYING CONTENT. The second challenge involves the support to temporarily reclaim full ownership of the underlying content when the thread knows it owns the last unique `Arc` and `Weak` pointers. The functions `Arc::get_mut` and `Arc::make_mut` provide these capabilities: they return a mutable reference `&mut T` to the underlying content. The reclamation is temporary because when the reference goes out of scope (when the lifetime of the mutable reference ends), the content is returned and the original `Arc`



```

1  fn is_unique(&mut self) -> bool {
2      // lock the weak pointer count if we appear to be the sole weak pointer holder.
3      if self.inner().weak.compare_exchange(1, usize::MAX, Acquire, Relaxed).is_ok() {
4          let unique = self.inner().strong.load(Relaxed) == 1;
5
6          self.inner().weak.store(1, Release); // release the lock
7          unique
8      } else { false }
9  }
10 fn get_mut(this: &mut Self) -> Option<&mut T> {
11     if this.is_unique() {
12         unsafe { Some(&mut this.ptr.as_mut().data) }
13     } else { None }
14 }
15 fn drop(&mut self) {
16     if self.inner().strong.fetch_sub(1, Release) != 1 {
17         return;
18     } ...
19 }

```

TABLE 19.2: Rust’s implementation (excerpt) of `Arc::get_mut` and `Arc::drop`

pointer can be used again.

The challenge here is to guarantee that if the temporary reclamation is successful, it is synchronized with *all* accesses to the content of type `T`. Again, note that those accesses can only happen between the construction and the destruction of an `Arc` pointer. How an `Arc` pointer can be constructed is now more complicated than that of Core `Arc`: an `Arc` pointer can now additionally be created by *upgrade*-ing from a `Weak` pointer. Therefore, to establish the synchronization guarantee, we now need to handle the intertwined life-cycles of `Arc` and `Weak` pointers.

To be more concrete, let us look at the implementation of `get_mut` in Table 19.2. To return temporary full ownership of the `data` field, the function checks that the thread owns the unique `Arc` and `Weak` pointers in two steps, using `is_unique`.

First, it acquires a “lock” on the `Weak` counter—a `weak`—to make sure that there is no other `Weak` pointers. This is done by an acquire compare-and-swap (**CAS**) from 1 to `MAX`. The function uses `MAX` as the “locked” value to resolve conflicts with other contentious `Arc::get_mut` or `Arc::downgrade` calls. If the **CAS** succeeds, the thread knows that there is no `Weak` pointers left, but there may exist still some `Arc` pointers. This comes from the agreed contract between the counters: the `Weak` counter *implicitly* counts 1 for *all* `Arc` pointers. So when the thread still owns an `Arc` pointer, and the value of the `Weak` counter is exactly 1, that 1 must be accountable for the remaining `Arc` pointers, and there is no `Weak` pointers left.

Second, it does an acquire read<sup>22</sup> on the `Arc` counter—a `strong`—and then checks if the value read is 1. If that value is 1, `is_unique` succeeds and `get_mut` concludes that thread owns the unique `Arc` pointer, and gives the thread temporary full access to the underlying content with

<sup>22</sup>A relaxed read in the original code in line 4, Table 19.2. We need to fix it to an acquire read—see §19.4.



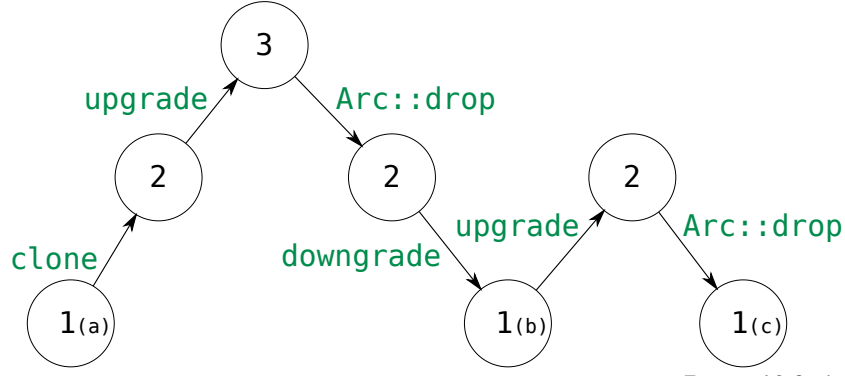


FIGURE 19.3: A truncated history of the Arc counter

type `&mut T`.<sup>23</sup> No matter if the second check fails or not, `is_unique` will release the lock on the `Weak` counter with a release write of value 1.

**CORRECTNESS OF `is_unique`.** We wish to verify `is_unique` against the following abstract specification.

$$\begin{aligned} & \{ \text{ARC}_q^{\gamma_i, \gamma}(\mathbf{a}, \mathcal{I}) * \Phi(q) \} \\ & \quad \text{is\_unique}(\mathbf{a}) \\ & \left\{ \begin{array}{l} b. \text{ if } b \text{ then } \mathbf{a}.\text{strong} \mapsto 1 * \mathbf{a}.\text{weak} \mapsto 1 * \Phi(1) \\ \text{ else } \text{ARC}_q^{\gamma_i, \gamma}(\mathbf{a}, \mathcal{I}) * \Phi(q) \end{array} \right\} \quad (\text{ARC-ISUNIQUE-SPEC}) \end{aligned}$$

where  $\Phi$  is a predicate on fractions that may be instantiated with the interpretation of the underlying type `T`. The specification says that if the function returns `true`, then we should have access to the full fraction of  $\Phi(1)$ . Therefore, if we have instantiated  $\Phi$  as the ownership of `T`, we should have full access to `T` (e.g., `&mut T`) in that case.

The key idea to the correctness of `is_unique` is that its two checks ensure the synchronization guarantee for temporary reclamation. The second check ensures that the thread is synchronized with all *other* `Arc::drop` calls. This means that it is synchronized with all accesses to the content made by all other `Arc` pointers. The thread, of course, must have synchronized with all accesses made by the current `Arc` pointer that it owns. Consequently, the thread must have synchronized with all accesses to the underlying content.

The problem, however, is that the second check uses an acquire *read*, instead of a **CAS**. If it were a **CAS**, then we would be guaranteed to read the latest value of the `Arc` counter, and thus synchronizing with all other `Arc::drop`'s. However, an acquire read does not guarantee reading the latest value: it can read a stale one. Consider a truncated history of the `Arc` counter in Figure 19.3, where our call to `get_mut` was initiated somewhere before the latest write 1(c) to the counter. Since we do not know exactly when `get_mut` was initiated, the second check by `is_unique` may read 1 from any events 1(a), 1(b) or 1(c). Had it read from 1(a), we would not have synchronized with the `Arc::drop`'s or `downgrade`'s after that. Our obligation here is to show that if the second check read 1, it must have read from 1(c).

By contradiction, we show that it is impossible to read 1 from 1(a), 1(b) or any stale 1 values. Put it another way, we show that the thread

<sup>23</sup>The `Arc::make_mut` function also follows the similar pattern, but the targets are reversed: it first acquires a “lock” on the `Arc` counter and then reads the `Weak` counter.

has observed all updates to the Arc counter from a stale 1 to 2, denoted as  $\text{stale}(1 \rightsquigarrow 2)$ , and therefore cannot read those stale 1's again. This is where the first check comes into play: it gives us the guarantee that the thread has observed all  $\text{stale}(1 \rightsquigarrow 2)$  updates. Note that these updates come either from an `Arc::clone` or from a `Weak::upgrade`. If the update is from an `Arc::clone`, like in 1(a), the thread must have observed it because that update must have been performed by some Arc pointer—unique at that time—of which the current Arc pointer (which this thread owns) is a *descendant*.

The remaining case is when the update is from a `Weak::upgrade`, like in 1(b). By the first check the thread is synchronized with all `Weak::drop`'s by all Weak pointers. Note that `Weak::drop`, similar to Core Arc::drop,<sup>24</sup> does a release FAA to decrement the Weak counter. However, unlike in Core Arc, the last `Weak::drop` decrements the counter to 1 (instead of 0). Therefore, when the first check did a successful acquire CAS for value 1 on the Weak counter, it knows that there is no Weak pointers left and it is synchronized with all `Weak::drop`'s.

If an update  $\text{stale}(1 \rightsquigarrow 2)$  is from an `Weak::upgrade`, it must happen before the `Weak::drop` of the same Weak pointer. Thus, by synchronizing with all `Weak::drop`'s, the thread is guaranteed to synchronize with all  $\text{stale}(1 \rightsquigarrow 2)$  updates from `Weak::upgrade`'s. It follows that the thread must have read the latest write to the Arc counter.

ANOTHER INSTANCE OF SYNCHRONIZED GHOST STATE. Thus, our challenge here pins down to formalizing the observations of  $\text{stale}(1 \rightsquigarrow 2)$  and the two sources of those observations. Furthermore, the observations are tied to the ownership of some Arc or Weak pointer, and when such ownership is transferred the observations must also be transferred in a *synchronized* way.

For this purpose, we use an instance of synchronized ghost state for those observations. Similar to the ghost state for RMC cancelable invariants,<sup>25</sup> we use the ghost state of form  $\llbracket \circ(q, O) \rrbracket^\gamma$  where  $O$  is a set of observations. In the particular case of Arc, an observation is simply a unique identifier *id* for each  $\text{stale}(1 \rightsquigarrow 2)$  update event on the Arc counter. We therefore rely on the iRC11 logic to provide unique identifiers for update events. Fortunately, we can simply use the *timestamp* of an update/write event as the identifier. This is indeed a motivation to expose timestamps in the logic, and subsequently in GPS protocol assertions. We thus can tie the logical ghost state to the write events, making the observations actually *physical* and so can only be transferred with physical synchronization.

In the verification of Arc, we use two different constructions: one,  $\llbracket \circ(q, O_u) \rrbracket^\mu$ , to track the observations coming from `Weak::upgrade`, and another,  $\llbracket \circ(q, O_c) \rrbracket^\gamma$ , to track those coming from `Arc::clone`. The former construction  $\llbracket \circ(q, O_u) \rrbracket^\mu$  enjoys similar properties to that of RMC cancelable invariants. That is, the observations can be joined (using set union), and if we own the full fraction  $\llbracket \circ(1, O_u) \rrbracket^\mu$ , then we are guaranteed that  $O_u$  contains all possible `Weak::upgrade`'s  $\text{stale}(1 \rightsquigarrow 2)$  events and we have *physically* seen them all. Additionally, each owner of each fraction  $q$

<sup>24</sup>see Figure 19.1

<sup>25</sup>see §11.2.2

can concurrently add observations to its local set  $O$ . This is to reflect the fact that any `Weak` pointer can always perform a  $\text{stale}(1 \rightsquigarrow 2)$  event.

The latter construction  $\llbracket \circ(q, O_c) \rrbracket^\gamma$  is a bit different. Even if we only own a fraction  $\llbracket \circ(q, O_c) \rrbracket^\gamma$ , we need to know that  $O_c$  contains all possible `Arc::clone`'s  $\text{stale}(1 \rightsquigarrow 2)$  events and we have *physically* seen all of them. Furthermore, we can only add observations to  $O_c$  if we have the full fraction  $\llbracket \circ(1, O_c) \rrbracket^\gamma$ . This reflects the fact that any `Arc` pointer must have seen all `Arc::clone`'s  $1 \rightsquigarrow 2$  updates, and that any `Arc::clone`'s  $1 \rightsquigarrow 2$  update can only be done by the one `Arc` pointer that was unique and should own the full fraction at the time of the update.

We then set up that the abstract predicate `ARC` for ownership of `Arc` pointers also contains a fraction  $\llbracket \circ(q, O_c) \rrbracket^\gamma$  for some  $q$ <sup>26</sup> and  $O_c$  (because only `Arc` pointers can do `Arc::clone`), and that the abstract predicate `WEAK` for ownership of `Weak` pointers contains a fraction  $\llbracket \circ(q, O_u) \rrbracket^\mu$  for some  $q$  and  $O_u$  (because only `Weak` pointers can do `Weak::upgrade`). We further require that `Arc::drop` also releases the fraction  $\llbracket \circ(q, O_c) \rrbracket^\gamma$  like releasing the other fractions, and similarly that `Weak::drop` releases  $\llbracket \circ(q, O_u) \rrbracket^\mu$ .

With that setup, we are ready to show that when the two checks of `is_unique` succeed, the thread must have observed all  $\text{stale}(1 \rightsquigarrow 2)$  updates. First, when acquiring the “lock” on the `Weak` counter, the thread also acquire the full fraction  $\llbracket \circ(1, O_1) \rrbracket^\mu$  from the `Weak` counter protocol. The full fraction is available in the protocol because all `Weak` pointers have been `dropped`. With  $\llbracket \circ(1, O_1) \rrbracket^\mu$ , the thread is guaranteed to have seen all `Weak::upgrade`'s  $\text{stale}(1 \rightsquigarrow 2)$  updates. Second, since the thread owns an `Arc` pointer, it owns a fraction  $\llbracket \circ(q, O_2) \rrbracket^\gamma$ , which guarantees that the thread has seen all `Arc::clone`'s  $\text{stale}(1 \rightsquigarrow 2)$  updates. Consequently, the thread must have read 1 from the latest write to the `Arc` counter, and thus is synchronized with all previous accesses to the underlying content `T`.  $\square$

<sup>26</sup>the same  $q$  in e.g.,  $\heartsuit_q^{\gamma_i}$  and  $\text{a.data} \mapsto^q v$ —see [Definition 19.2](#)

## 19.4 Insufficient Synchronization in `get_mut`

Unfortunately, our setup was not strong enough to verify `Arc` and `Weak` without change. The two reads of the counters in the second check of `get_mut` and `make_mut` were `rlx` in the original code (line 4, [Table 19.2](#)), and we had to strengthen them both to `acq` in order to make the verification go through. The reason is that, while we managed to temporarily get the full resources out by a read, the `rlx` reads do not give us those resources in the current view: they are under an acquire modality  $\nabla$ , and we could not move them out of the modality, and got stuck in the proof.

While we conjecture that a `rlx` read in `make_mut` is in fact sufficient, a `rlx` read in `get_mut` turned out to be insufficient and we have reported the bug and the fix has been merged into Rust codebase. The following example invokes a data race when using `get_mut`:

```
1 let mut arc1 = Arc::new(0);
2 let arc2 = Arc::clone(&arc1);
3 thread::spawn(move || {
```

```

4   let _ : u32 = *arc2; /* drop(arc2); */
5   });
6   loop {
7       match Arc::get_mut(&mut arc1) {
8           None => {}
9           Some(m) => { *m = 1u32; return; }
10      }
11  }

```

In this example there are two non-atomic operations: the read of the underlying integer in line 4 (child thread) and the write to the same integer in line 9 (parent thread). The read should be safe because the child thread owns `arc2`, thus the underlying integer is shared and *immutable*. The write should be safe because `get_mut` guarantees that the parent thread owns the unique `Arc` pointer (`arc1`) and should temporarily gain full access to the non-atomic integer. This can only happen after the child thread finishes and `arc2` has been dropped. However, the two non-atomic operations constitute a data race by C11 standard, because neither one happens-before the other. More specifically, in line 4 of the child thread, when `arc2` goes out of scope, it will be destructed by `Arc::drop`, which uses a release (**rel**) RMW (see the code at line 16, Table 19.2). This release RMW will be read by `get_mut` (line 4, Table 19.2) in the parent thread (line 7). If this read had been **acq**, then there would have been a release-acquire synchronization between the release RMW of `drop` and the acquire read of `get_mut`, and the non-atomic read of the child thread would have been guaranteed to happen-before the non-atomic write of the parent thread. However, the read was **rlx**, thus no happen-before relationship can be established between the two non-atomic operations.

CHAPTER SUMMARY. In this chapter, we sketched the verification of  $\text{RB}_{\text{rlx}}$  most substantial example: `Arc<T>` and `Weak<T>` libraries from Rust. We followed the original RustBelt work to verified the libraries against a sufficiently strong abstract specification, which is then used to perform the semantic type checking of those libraries. The semantic type checking proofs are reused as-is from the original RustBelt work, and we have only discussed the RMC verification of the libraries against the abstract specification. We reported a synchronization bug in `Arc::get_mut`.

# 20

## *Related Work*

---

Doko and Vafeiadis<sup>1</sup> verify a subset of the `Arc` library with FSL++. We improve on their results by (1) enlarging the scope of the verification to include important parts of the API such as the `make_mut` and `get_mut` functions (the latter of which we found to contain a data race) as well as the `Weak` reference type, (2) allowing full resource reclamation of both the contents and the reference-count fields of the `Arc` data structure, and (3) embedding our verification effort in the RustBelt framework, so that we can establish the soundness of `Arc` when linked with unknown well-typed  $\lambda_{\text{Rust}}$  code.

Cancelable single-writer protocols given in §17.1 are more general and support stronger reclamation schemes than any variants from previous RMC logics, such as FSL/FSL++, GPS, and iGPS. For example, previous logics would only be able to verify `RwLock`’s writer-lock release function where the lock is released with a `CAS`—but not with a `release write` as in the Rust’s implementation. More importantly, previous variants were not designed to reclaim resources in the fashion of Rust’s lifetimes.

<sup>1</sup>Doko and Vafeiadis, “Tackling Real-Life Relaxed Concurrency with FSL++” [DV17].



## Part IV

### COMPASS





**Part IV** presents the Compass specification framework. **Chapter 21** starts by reviewing specifications with logical atomicity in both SC and RMC settings. **Chapter 22** discusses how to encode Yacovet specs in iRC11 with logical atomicity. **Chapter 23** present the proofs of RMC stacks and queues against the Compass specs, relying on general *multi-location invariants* (**Chapter 11**) and atomic points-to (**Chapter 10**). **Chapter 24** discusses the composition of the stack spec and the exchanger spec to verify the elimination stack. In doing so, it discusses the role of *helping* with logical atomicity in the specs of exchangers. The top of **Figure 1.1** presents the dependency between these chapters and with previous chapters.



# 21

## Background: Strong Specifications with Logical Atomicity

---

Strong memory models provide strong guarantees about the ordering of memory operations, making it easier to write clearly correct library implementations. More relaxed memory consistency models offer more opportunities for more efficient implementations, which, on the other hand, may provide weaker guarantees to clients. In this chapter, using the Queue data structure as an example, we review existing *logically atomic* specifications (from now on, *specs* for short) in stronger memory models (Figure 21.1), and in Chapter 22 we will present several of our logically atomic specs in the RC11 model. We review, in §21.1, the traditional Hoare-triple-based specs for sequential queues; in §21.2, *logical atomicity*<sup>1</sup> and its uses to give strong specs for concurrent SC queues; and in §21.3, how Cosmo<sup>2</sup> extends those specs for RMC with *thread views*.

<sup>1</sup>Rocha Pinto et al., “TaDA: A Logic for Time and Data Abstraction” [RPDG14]; Svendsen and Birkedal, “Impredicative Concurrent Abstract Predicates” [SB14]; Jung et al., “The future is ours: prophecy variables in separation logic” [Jun+20].

<sup>2</sup>Mével and Jourdan, “Formal verification of a concurrent bounded queue in a weak memory model” [MJ21].

### 21.1 Sequential Specifications for Queues

The separation logic sequential specs for queues are given as **SEQ-ENQ** and **SEQ-DEQ** (Figure 21.1). **SEQ-ENQ** specifies that an enqueue function call **enq**([ $q, v$ ]) can run safely as long as it has  $\text{Queue}(q, vs)$ , an abstract separation logic assertion that represents full ownership of the queue object  $q$  (an instance of the data structure). An implementation can define  $\text{Queue}(q, vs)$  as arbitrary resources that it specifically needs. But from the perspective of clients,  $\text{Queue}(q, vs)$  is abstract because it asserts that  $q$ ’s current state can be seen *abstractly* as a list of values  $vs$ —that is, the queue’s elements are currently  $vs$ , ordered by the list order.

**SEQ-ENQ** then says that **enq**([ $q, v$ ]) requires and consumes  $q$ ’s ownership at the beginning of the call, and at the end of the call it returns the ownership with the updated abstract state  $vs ++ [v]$ , reflecting the operation’s effects:  $v$  has been enqueued to the end of  $q$ . Conversely, by **SEQ-DEQ**, a dequeue **deq**([ $q$ ]) also consumes  $q$ ’s ownership and, if the queue is not empty, returns the head value  $v$  of  $vs$  and gives back the ownership with only its tail  $vs'$ . (The notation  $\{v.Q\}$  denotes the post-condition with a returned value  $v$ .) Otherwise, if  $q$  is empty, **deq**([ $q$ ]) returns empty ( $\epsilon$ ) and the fact that the abstract state is also empty ( $[]$ ).

That an operation is allowed to consume the queue ownership for

the *whole* duration of its execution is what makes the specs *sequential*: a group of threads cannot access the ownership  $\text{Queue}(q, vs)$  concurrently in order to perform concurrent enqueues and/or dequeues. To have strong specs for such *fine-grained concurrency*, we need logical atomicity.

## 21.2 SC Specifications with Logical Atomicity

In fine-grained concurrency, a concurrent object’s ownership is shared for concurrent accesses, and contention is most commonly resolved by *atomic* read-modify-write (RMW) instructions, such as compare-and-swap (CAS). In this case, even if a concurrent object’s operation may not be atomic (because it is implemented with multiple instructions), its effects are published by a single atomic instruction. This is the intuition of *logical atomicity*: from the perspective of clients, the operation *appears* to be atomically updating the object exactly around a single atomic instruction—often called the *commit* or *linearization* point of the operation.

As such, a client only needs to provide ownership of the concurrent object at the operation’s commit point, and can expect the update to happen right after the commit point. This idea is encoded in *logically atomic triples* (LATs),<sup>3</sup> of the form  $\langle P \rangle e \langle Q \rangle$ , with angle brackets  $\langle \rangle$  instead of curly braces. The intuitive interpretation is also a bit more subtle than normal Hoare triples:  $\langle P \rangle e \langle Q \rangle$  means that there exists a commit point (instruction)  $c$  by which  $e$  *atomically* consumes  $P$ , transforms it, and returns  $Q$ .

Using LATs, we can give strong specs like **SC-ENQ** and **SC-DEQ** (Figure 21.1) to fine-grained concurrent SC queues. Here we use **red** font-face to denote the gradual changes in the specs. One obvious change is the aforementioned angle brackets  $\langle \rangle$ . Less obvious is the quantification of **vs** in the precondition  $\langle vs. \text{Queue}(q, vs) \rangle$ : this is a special form of universal quantification that signifies the possibility that the queue may be modified concurrently. Specifically, it signifies that during the specified enqueue/dequeue operation, other threads may be changing the state  $vs$  of the queue *arbitrarily*, up until the commit point of the operation, when it atomically updates the state to what is described in the post-condition. For example, **SC-ENQ** says that **enq**([ $q, v$ ]) can withstand arbitrary concurrent updates to the state  $vs$  of  $q$ , up until the commit point when it atomically transforms  $\text{Queue}(q, vs)$  (where  $vs$  is the state at that instant) to the new state  $\text{Queue}(q, vs ++ [v])$ . In contrast, the sequential spec **SEQ-ENQ** implicitly quantifies over  $vs$  with a normal universal quantifier ( $\forall vs$ ) at the outside: this allows the implementation to assume exclusive ownership of  $\text{Queue}(q, vs)$  for an arbitrary but *unchanging*  $vs$ , thereby prohibiting concurrent interference.

Last but not least, we add a *local* precondition **isQueue**( $q$ ), another abstract assertion that encodes persistent separation logic facts about the queue, e.g., facts about its head and tail pointers. Thus, they are freely duplicable, and they are local in the sense that they are to be provided at the beginning of a call, so that operations can use them for the whole execution, more conveniently than  $\text{Queue}(q, vs)$  which is

<sup>3</sup>Rocha Pinto et al., “TaDA: A Logic for Time and Data Abstraction” [RPDG14]; Svendsen and Birkedal, “Impredicative Concurrent Abstract Predicates” [SB14]; Jung et al., “Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning” [Jun+15]; Jung et al., “The future is ours: prophecy variables in separation logic” [Jun+20].

$$\begin{array}{c}
\text{SEQ-ENQ} \\
\{ \text{Queue}(q, vs) \} \text{enq}([q, v]) \{ (). \text{Queue}(q, vs ++ [v]) \} \\
\\
\text{SEQ-DEQ} \\
\{ \text{Queue}(q, vs) \} \text{deq}([q]) \{ v. (vs = [] * v = \epsilon * \text{Queue}(q, [])) \vee (\exists vs'. vs = v :: vs' * \text{Queue}(q, vs')) \} \\
\\
\text{SC-ENQ} \\
\text{isQueue}(q) \vdash \langle vs. \text{Queue}(q, vs) \rangle \text{enq}([q, v]) \langle (). \text{Queue}(q, vs ++ [v]) \rangle \\
\\
\text{SC-DEQ} \\
\text{isQueue}(q) \vdash \langle vs. \text{Queue}(q, vs) \rangle \text{deq}([q]) \langle v. (vs = [] * v = \epsilon * \text{Queue}(q, [])) \vee (\exists vs'. vs = v :: vs' * \text{Queue}(q, vs')) \rangle \\
\\
\text{ABS-SO-ENQ} \\
\text{isQueue}(q) * \exists V \vdash \langle vs. \text{Queue}(q, vs) \rangle \text{enq}([q, v]) \langle (). \text{Queue}(q, vs ++ [(v, V)]) \rangle \\
\\
\text{ABS-SO-DEQ} \\
\text{isQueue}(q) * \exists V \vdash \\
\langle vs. \text{Queue}(q, vs) \rangle \text{deq}([q]) \langle v. (v = \epsilon * \text{Queue}(q, vs)) \vee (\exists vs', V'. vs = (v, V') :: vs' * \text{Queue}(q, vs') * \exists V') \rangle
\end{array}$$

FIGURE 21.1: Specifications of Queue operations, from sequential, to SC concurrency and strong RMC

neither duplicable nor local.

Intuitively, it should be clear that  $\langle P \rangle e \langle Q \rangle_{\mathcal{E}}$  is a stronger spec than  $\{P\} e \{Q\}_{\mathcal{E}}$ , seeing as the former permits concurrent interference whereas the latter does not. Intuitively, if  $e$  only needs  $P$  and  $Q$  around its commit point  $c$ , then it can also work with having  $P$  and  $Q$  around the whole execution, which includes  $c$ . But how does a client actually make use of these LATs to arbitrate concurrent accesses to a shared resource like  $\text{Queue}(q, vs)$ ? To that end, we need to see the interaction between LATs and *invariants*, which can formally explain how LATs are strictly stronger than normal Hoare triples.

**LOGICAL ATOMICITY AND INVARIANTS.** We recall the standard access rule for invariants **HOARE-INV** given in **Figure 6.6**: a physically atomic instruction  $e$  can access and rely on  $I$ , in addition to  $P$ , for its execution, as long as it gives  $I$  back right afterwards. The LAT invariant access rule **LAT-INV** *strengthens* **HOARE-INV**, as it relaxes the restriction of “accessing around atomic instructions” ( $\text{atomic}(e)$ ) to “accessing around *logically* atomic expressions”.

$$\frac{\text{LAT-INV} \quad \langle \triangleright I * P \rangle e \langle \triangleright I * Q \rangle_{\mathcal{E} \setminus \mathcal{N}} \quad \mathcal{N} \subseteq \mathcal{E}}{\boxed{I}^{\mathcal{N}} \vdash \langle P \rangle e \langle Q \rangle_{\mathcal{E}}}$$

With this rule, clients can build protocols to use and combine libraries with LAT specs. For example, we can allocate an invariant

$$\boxed{\exists vs_1, vs_2. \text{Queue}(q_1, vs_1) * \text{Queue}(q_2, vs_2) * R(vs_1, vs_2)}^{\mathcal{N}}$$

that ties together two queues by a relation  $R$ , and then use **LAT-INV** with **SC-ENQ** and **SC-DEQ** to verify clients that use the two queues and adhere to the “protocol”  $R$ . For example,  $R$  may require that  $vs_1$  and  $vs_2$  are disjoint, or even more specifically, that one queue only contains odd numbers and the other only contains even numbers.

<sup>4</sup>Birkedal et al., “Theorems for free from separation logic specifications” [Bir+21].

In summary, with logical atomicity and invariants, one can give stronger modular specs for fine-grained concurrent libraries. Furthermore, LAT specs can be seen as giving *abstract operational semantics* to a library’s operations. As such, the library should be linearizable, *i.e.*, there is a total order of its operations according to which the concurrent object appears to behave sequentially. In fact, Birkedal et al.<sup>4</sup> recently showed formally that, in SC, logical atomicity implies linearizability. It is therefore an important tool to achieve full functional correctness *and* modular client reasoning.

### 21.3 Logically Atomic Specifications in RMC with Views

However, linearizability and logical atomicity do not directly extend to relaxed memory. In RMC, a total order of operations (the linearization) might not exist, or if it does exist, it may not be very useful. In contrast to the SC model where every atomic instruction is *synchronized with* each other instruction, in RMC an atomic instruction may only be synchronized with *some* other instructions. It is the partially-ordered synchronizations—formally defined as the *happens-before* (**hb**) relation—between operations that really matter for their correctness, not the total order. In the terms of logical atomicity, this means that an update to the state by the commit of an operation  $o$  may only be meaningful to operations that are synchronized with  $o$ . Consequently, LAT specs for RMC libraries have to additionally account for **hb**.

#### 21.3.1 Cosmo Specs for Queues

**ABS-SO-ENQ** and **ABS-SO-DEQ** (in **Figure 21.1**) are a simplified version of Cosmo specs for multi-producer multi-consumer queues. They differ from the SC specs in the extra tracking of views (in **red** in **Figure 21.1**): (1) the specs take the “seen view” assertion  $\sqsupseteq V$  as a *local precondition* (that is, outside of the LAT precondition and needed at the beginning of the call); and (2) the abstract state is no longer just a list of values, but a list of value-view pairs, where the view component of a pair is the view of the enqueue operation (after its commit point). Similar to the release-acquire rules, the views in the abstract state support view transfers between matching enqueue-dequeue pairs: by **ABS-SO-ENQ**, an enqueue releases its local view  $V$  at its commit point, and by **ABS-SO-DEQ**, the matching dequeue acquires  $V$  into its local view, also at its respective commit point. Effectively, they expose the **so** relation between matching enqueue-dequeue pairs via views in the abstract state. This is why we call them  $\text{LAT}_{\text{so}}^{\text{abs}}$  style. (The complete Cosmo specs also track **so** among enqueues and among dequeues.)

#### 21.3.2 Abstract State and Read-Only Operations

However, by using just the abstract state, the specs do not specify behaviors of *read-only* operations that do not modify the abstract state. For example, in **ABS-SO-DEQ**, a failing empty dequeue is a read-only operation, and the  $\text{LAT}_{\text{so}}^{\text{abs}}$  specs do not give us any new facts about *vs*. This is

|                                                                                     |             |                   |             |                                                                                                            |
|-------------------------------------------------------------------------------------|-------------|-------------------|-------------|------------------------------------------------------------------------------------------------------------|
| $\text{enq}([q, 41]);$<br>$\text{enq}([q, 42]);$<br>$\text{flag} :=_{\text{rel}} 1$ | $\parallel$ | $\text{deq}([q])$ | $\parallel$ | $\text{repeat } (*\text{acqflag} \neq 0);$<br>$\text{deq}([q])$<br>$// \text{ return 41 or 42, not empty}$ |
|-------------------------------------------------------------------------------------|-------------|-------------------|-------------|------------------------------------------------------------------------------------------------------------|

FIGURE 21.2: A Message-Passing (MP) client with Queues

weaker than in the SC model, where **SC-DEQ** says that dequeues fail with  $\epsilon$  only if the state  $vs$  is truly empty (at the commit point).

Realistically, an RMC spec cannot be quite as strong as the SC spec: recall that in RMC effects can appear to threads differently, so it may be that the thread  $\pi$  sees the queue as empty and returns  $\epsilon$ , but the queue is in fact not empty, because a fresh enqueue by another thread  $\rho$  has not become visible to  $\pi$  yet. But we can do better than the empty case of **ABS-SO-DEQ**, which gives the client no useful information.

More concretely, the Cosmo spec's **ABS-SO-DEQ** cannot be used to verify the Message-Passing client with queues, with the expected behavior given in Figure 21.2. Here, the queue is accessed concurrently by 3 threads: the left-most thread performs 2 enqueues (**enq**), the middle one performs a dequeue (**deq**), and the right-most thread waits for the signal by the left-most thread through `flag` and then performs a dequeue. A weak implementation of dequeue can return *empty* even though the queue is not empty, due to contention. However, in this example, the right-most thread *cannot* get an empty dequeue result, because (1) at most one enqueue could have been consumed concurrently by the middle thread, and (2) due to the release-acquire synchronization through `flag`, the thread has synchronized with the two enqueues.

Unfortunately, the Cosmo spec only exposes *internal* (to the implementation) synchronizations among operations, without taking into account how additional *external* synchronizations created by the client (such as the synchronization through `flag`) can affect the behaviors of dequeues. It therefore cannot exclude the possibility that the right-most thread's dequeue returns empty.

In the next Chapter 22, we present specs that expose more of the **hb** relation, enough to cover read-only operations such as failing dequeues. Using those specs, we can verify the MP client in Figure 21.2: by combining the queue's richer **hb** relation with the client's *external* **hb** relation, we prove that the right-most thread's dequeue cannot return empty.





# 22

## Strong Compass Specifications with Richer Partial Orders

---

In this chapter, we present several of our logically atomic specs that, by exposing richer partial orders that can be combined with *external synchronizations*, can stay reasonably strong and yet still satisfiable by more relaxed implementations in the weaker ORC11 memory model. In §22.1 we present the  $\text{LAT}_{\text{hb}}^{\text{abs}}$  style which generalizes the  $\text{LAT}_{\text{so}}^{\text{abs}}$  style, and its instance for queues, which suffices to verify the MP client in Figure 21.2. In §22.2, we present the  $\text{LAT}_{\text{hb}}$  spec style, a weakening of  $\text{LAT}_{\text{hb}}^{\text{abs}}$ . In §22.3, we show how to encode these logically atomic specs in iRC11.

### 22.1 Graph-Based Specs to Encode Partial Orders

The  $\text{LAT}_{\text{hb}}^{\text{abs}}$  style extends the  $\text{LAT}_{\text{so}}^{\text{abs}}$  style by exposing a greater part of  $\text{hb}$ . An instance for queues is given in **ABS-HB-ENQ**, **ABS-HB-DEQ**, and **ABS-HB-QUEUE-CONSISTENCY** (Figure 22.1). That these specs are stronger than those of Cosmo can be seen easily by ignoring the added **red** parts. The main improvement of this instance is in **ABS-HB-DEQ**’s failure case, where the caller sees the queue as empty. Here, the spec provides more information about how the resulting read-only *empty dequeue* operation is ordered with other operations in  $\text{hb}$ .

As read-only operations have no effects on the abstract state, we need a new component  $G$  to identify and relate them to other operations. The component  $G \in \text{Graph}$  is a general construction inspired by the declarative specs of Yacovet.<sup>1</sup> Yacovet works on *whole-program execution graphs*, and abstracts them into per-library *event graphs* of operations, where every operation is uniquely identified by an event. A Yacovet spec for a library encodes the ordering between events in a graph as partial orders that must satisfy some *library-specific consistency conditions*. Here, we encode Yacovet specs with the event graph component  $G$ . The main differences with Yacovet are that (1)  $G$  records only the library events that have happened so far, not complete executions; and (2) our specs are stated as separation logic LATs, so each operation can access the current, up-to-date event graph  $G$  and only needs to extend  $G$  with the operation’s event and to maintain the graph’s consistency.

The (simplified) types of event graphs are given at the top of Fig-

<sup>1</sup>Raad et al., “On library correctness under weak memory consistency: specifying and verifying concurrent libraries under declarative consistency models” [Raa+19].

$$\begin{aligned}
V &\in \text{View} ::= \text{Loc} \rightarrow \text{Time} \\
e &\in \text{EventId} ::= \mathbb{N} \\
\text{QueueEvent} &::= \text{Enq}(v) \mid \text{Deq}(v) \mid \text{Deq}(\epsilon) \\
M &\in \text{LogView} ::= \wp(\text{EventId}) \\
\text{Event} &::= \text{QueueEvent} \times \text{View} \times \text{LogView} \\
G &\in \text{Graph} ::= (\text{EventId} \rightarrow \text{Event}, \wp(\text{EventId} \times \text{EventId})) \\
\text{QueueConsistent}(vs, G) &:= \\
&\wedge \left\{ \begin{array}{l} \forall (e, d) \in G.\text{so}. \exists v. G(e).\text{type} = \text{Enq}(v) \wedge G(d).\text{type} = \text{Deq}(v) \wedge \dots \\ \hspace{15em} (\text{QUEUE-MATCHES}) \\ \forall (e, d) \in G.\text{so}. e'. G(e').\text{type} = \text{Enq}(\_) \rightarrow (e', e) \in G.\text{lhb} \rightarrow \\ \hspace{15em} \exists d'. (e', d') \in G.\text{so} \wedge (d, d') \notin G.\text{lhb} \hspace{10em} (\text{QUEUE-FIFO}) \\ \forall d, e. G(d).\text{type} = \text{Deq}(\epsilon) \rightarrow G(e).\text{type} = \text{Enq}(\_) \rightarrow \\ \hspace{15em} (e, \_) \notin G.\text{so} \rightarrow (e, d) \notin G.\text{lhb} \hspace{10em} (\text{QUEUE-EMPDEQ}) \\ \dots \end{array} \right.
\end{aligned}$$

$$\begin{aligned}
&\text{ABS-HB-QUEUE-CONSISTENCY} \\
&\text{Queue}(q, vs, G) \vdash \text{QueueConsistent}(vs, G)
\end{aligned}$$

ABS-HB-ENQ

$$\text{SeenQueue}(q, G_0, M_0) * \sqsupseteq V \vdash$$

$$\langle G, vs. \text{Queue}(q, vs, G) \rangle$$

$$\text{enq}([q, v])$$

$$\left\langle \begin{array}{l} (). \exists G' \sqsupseteq G, M' \sqsupseteq M_0, V' \sqsupseteq V. \text{Queue}(q, vs ++ [(v, V')], G') \\ * \text{SeenQueue}(q, G', M') * \sqsupseteq V' * \exists e \notin G. e \in M' \wedge G' = G[e \mapsto (\text{Enq}(v), V', M')] \end{array} \right\rangle$$

ABS-HB-DEQ

$$\text{SeenQueue}(q, G_0, M_0) * \sqsupseteq V \vdash$$

$$\langle G, vs. \text{Queue}(q, vs, G) \rangle$$

$$\text{deq}([q])$$

$$\left\langle \begin{array}{l} v. \exists vs', G' \sqsupseteq G, M' \sqsupseteq M_0, V' \sqsupseteq V. \text{Queue}(q, vs', G') * \text{SeenQueue}(q, G', M') * \sqsupseteq V' \\ * \vee \left\{ \begin{array}{l} (v = \epsilon \wedge vs' = vs \wedge \exists d \notin G. d \in M' \wedge G' = G[d \mapsto (\text{Deq}(\epsilon), V', M')]) \\ (\exists V_e. vs = (v, V_e) :: vs' \wedge \exists e, M_e, d \notin G. G(e) = (\text{Enq}(v), V_e, M_e) \wedge (e, \_) \notin G.\text{so} \wedge V_e \sqsubseteq V' \\ \wedge M_e \cup \{e, d\} \subseteq M' \wedge G' = G[d \mapsto (\text{Deq}(v), V', M')] \wedge G'.\text{so} = \{(e, d)\} \cup G.\text{so}) \end{array} \right\} \end{array} \right\rangle$$

FIGURE 22.1: Compass Specs for Queues

**ure 22.1.** A graph  $G$  is a pair of (1) a function that maps each event id  $e \in \text{EventId}$  to event data of type  $\text{Event}$ , and (2) a set of event id pairs that encodes the **so** relation. We use  $G(e)$  to denote the event data for  $e$  in  $G$ , and  $G.\text{so}$  to denote the **so** relation of  $G$ .

The type  $\text{Event}$  is a tuple of (1) an event type (type), (2) a *physical* view (view), and (3) a *logical* view (logview). In **Figure 22.1** we give an instance of the event type for queues: the events can be an *enqueue* event of  $v$  ( $\text{Enq}(v)$ ), a *successful dequeue* event of  $v$  ( $\text{Deq}(v)$ ), or a *failing (empty) dequeue* event ( $\text{Deq}(\epsilon)$ ). An event's physical view is the view at the commit point of the operation that the event represents, and is needed in the logic to interact with other memory instructions. The event's logical view is also recorded at the commit point of its operation, and is a set of events for all library operations that *happen-before* the operation in question. If an event  $e$  is in the logical view of another event  $d$ , i.e.,  $e \in G(d).\text{logview}$ , we say that  $e$  happens before  $d$ . Technically, it is the commit instruction of  $e$ 's operation that happens before the commit instruction of  $d$ 's operation.

Intuitively, we use the logical view construction as an approximation of the **hb** relation between library operations, just as the physical view construction is an approximation of **hb** between memory instructions. The difference is that while physical views approximate **hb** *globally* between memory instructions, logical views only approximate **hb** *locally* for the library in question. As such, our logical views correspond to the *local happens-before* **lhb** relation of a library object introduced by Yacovet. Henceforth we use  $e \in G(d).\text{logview}$  and  $(e, d) \in G.\text{lhb}$  interchangeably.

The  $\text{LAT}_{\text{hb}}^{\text{abs}}$  style extends  $\text{LAT}_{\text{so}}^{\text{abs}}$  following a simple pattern: (1) the abstract state is accompanied by the graph that tracks all operations committed so far, and (2) at each operation's commit point, in addition to a potential update of the abstract state, a fresh event  $e$  representing the operation is added to the graph. For example, in **ABS-HB-ENQ**, when an enqueue of  $v$  commits, the current graph  $G$  of  $q$  is extended atomically with a fresh event  $e$  whose type is  $\text{Enq}(v)$ , into  $G'$ :  $G \sqsubseteq G'$ .

**LOCAL ASSERTIONS FOR LOGICAL VIEWS.** The partial orders are also extended at  $e$ 's commit point to relate it to other operations. In **ABS-HB-ENQ**,  $G'.\text{lhb}$  extends  $G.\text{lhb}$  by setting  $G'(e).\text{logview} = M'$ , the set containing all operations that happen before  $e$ .  $M'$  includes  $M_0$ —the *local logical view* of the calling thread, which tracks the operations that happen-before the **enq** call. This tracking of thread-local logical views is done by a new *persistent* assertion **SeenQueue**( $q, G_0, M_0$ ), where  $G_0$  is a *snapshot* of the current  $G$  ( $G_0 \sqsubseteq G$ ), and together with  $M_0$  they accumulate (a lower bound on) the information about operations that the thread has synchronized with. For instance, after the call, the thread receives **SeenQueue**( $q, G', M'$ ) with the latest snapshot  $G'$  and a new logical view  $M'$ , reflecting that the thread has synchronized with more operations ( $M_0 \sqsubseteq M'$ ), including the operation  $e$  that it has just executed ( $e \in M'$ ). By taking **SeenQueue** as a local pre-condition, the specs can specify that the operation's behavior can depend on what has happened before it—we will shortly see how that allows us to use **ABS-HB-DEQ** to verify the MP

client in Figure 21.2.

Compared to the  $\text{LAT}_{\text{so}}^{\text{abs}}$  style, in  $\text{LAT}_{\text{hb}}^{\text{abs}}$  each library type has a local logical view assertion like `SeenQueue` that plays a double role: (1) to track the thread-local logical view (as explained above) and also (2) to track persistent facts about the object like the `isQueue( $q$ )` assertion in **ABS-SO-ENQ**. The logical view assertion plays the same role for logical views as the “seen view” assertion  $\sqsubseteq V$  does for physical views: the tracked current local view can be published into the “public domain” (i.e., the shared graph for logical views, the shared location history or abstract state for physical views) so that it can be consumed by other threads.

**CONSISTENCY CONDITIONS.** The  $\text{LAT}_{\text{hb}}^{\text{abs}}$  style specifies properties of the abstract state and the partial orders through the library’s *consistency conditions*. The consistency conditions are invariant, i.e., should be maintained by all operations, and are specific to each library type.

For example, an excerpt of `QueueConsistent`, the consistency conditions for the queue library type, is given at the bottom right of Figure 22.1. It requires, among other things, that enqueues and dequeues must follow the first-in-first-out principle (FIFO, **QUEUE-FIFO**), stated in a fashion that is not too strong for RMC (more about that below). The fact that `QueueConsistent` is maintained by all operations is encoded in **ABS-HB-QUEUE-CONSISTENCY**: the queue ownership assertion `Queue( $q, vs, G$ )`, which is consumed and reproduced around the commit point, always implies consistency. So when **ABS-HB-ENQ** and **ABS-HB-DEQ** extend  $(vs, G)$  to new state  $(vs', G')$ , the operations can assume `QueueConsistent( $vs, G$ )` and must then re-establish `QueueConsistent( $vs', G'$ )`.

More specifically, if **deq** succeeds with a value  $v$ , **ABS-HB-DEQ** tells the client that  $G'.\text{so}$  extends  $G.\text{so}$  with a new pair  $(e, d)$  where  $d$  is the new successful event added by the dequeue operation and  $e$  is an existing enqueue event that  $d$  dequeues from. Therefore, through **ABS-HB-QUEUE-CONSISTENCY**, the spec additionally says that  $(e, d)$  satisfies, among other things,<sup>2</sup> (1) **QUEUE-MATCHES**: the return value  $v$  of the dequeue  $d$  must match the value enqueued by  $e$ ; and (2) **QUEUE-FIFO**: if there is another enqueue event  $e'$  that happens before  $e$ , then  $e'$  must already have been dequeued by some  $d'$  ( $(e', d') \in G.\text{so}$ ), and our  $d$  cannot happen before  $d'$  ( $(d, d') \notin G.\text{hb}$ ). (The consistency conditions on enqueue events are elided, so we will not discuss them.)

**WEAKER BUT FLEXIBLE.** The **QUEUE-FIFO** condition appears weaker than what one might expect, i.e.,  $(d', d) \in G.\text{hb}$ , but such a condition only works for strongly synchronized (e.g., SC) implementations. As stated, **QUEUE-FIFO** is also satisfiable by implementations that have little synchronization between dequeues. In fact, we have verified that **QUEUE-FIFO** is satisfiable by a fairly relaxed implementation (similar to the weak version in [Raa+19]) of the Herlihy-Wing queue.<sup>3</sup> The implementation only ensures **hb** between matching enqueue-dequeue pairs, but not among enqueues or among dequeues. (As one might guess, enqueues only use release operations, and dequeues only use acquire ones.)

Nonetheless, **QUEUE-FIFO** is still flexible enough that, for example,

<sup>2</sup>For example, an element can only be dequeued once.

<sup>3</sup>Herlihy and Wing, “Linearizability: A Correctness Condition for Concurrent Objects” [HW90].

$$\begin{array}{c}
\text{Invariant: } \boxed{\exists vs, G. \text{Queue}(q, vs, G) * \text{deqPerm}(\text{size}(G.\text{so})) * \text{size}(G.\text{so}) \leq 2 * \dots}^{\mathcal{N}} \\
\\
\begin{array}{|l|l|l|}
\hline
\begin{array}{l}
\{ \text{SeenQueue}(q, \emptyset, \emptyset) * \sqsupseteq V_1 \} \\
\langle \text{Queue}(q, \_) * \dots \rangle \\
\text{enq}([q, 41]); \\
\langle \text{Queue}(q, \_) * \dots \rangle \\
\{ \text{SeenQueue}(q, G_1, \{e_1\}) * \dots \} \\
\langle \text{Queue}(q, \_) * \dots \rangle \\
\text{enq}([q, 42]); \\
\langle \text{Queue}(q, \_) * \dots \rangle \\
\{ \text{SeenQueue}(q, G_2, \{e_1, e_2\}) * \dots \} \\
\text{flag} :=_{\text{rel}} 1
\end{array}
&
\begin{array}{l}
\{ \text{SeenQueue}(q, \emptyset, \emptyset) * \\
\text{deqPerm}(1) * \sqsupseteq V_2 \} \\
\langle \text{Queue}(q, \_) * \dots \rangle \\
\text{deq}([q]) \\
\langle \text{Queue}(q, \_) * \dots \rangle \\
\{ \text{SeenQueue}(q, G'_1, \{d_1\}) * \dots \}
\end{array}
&
\begin{array}{l}
\{ \text{SeenQueue}(q, \emptyset, \emptyset) * \\
\text{deqPerm}(1) * \sqsupseteq V_3 \} \\
\text{while} (*\text{acqflag} == 0) \{ \\
\{ \text{SeenQueue}(q, G_2, \{e_1, e_2\}) * \\
\text{deqPerm}(1) * \sqsupseteq V_3 \} \\
\langle \text{Queue}(q, \_) * \dots \rangle \\
\text{deq}([q]) \\
\langle \text{Queue}(q, \_) * \dots \rangle \\
\{ v. \text{SeenQueue}(q, G_3, \{e_1, e_2, d_2\}) * \\
v \in \{41, 42\} \}
\end{array}
\end{array}
\end{array}$$

FIGURE 22.2: A proof sketch of Message Passing with queues

if a client decides to use the queue in an SC fashion by adding sufficient external synchronization, the client can know that **lhb** is total, i.e.,  $(d', d) \in G.\text{lhb} \vee (d, d') \in G.\text{lhb}$ , and can thus exclude the right-hand side of the disjunction and regain the stronger FIFO condition with  $(d', d) \in G.\text{lhb}$ . This demonstrates the benefits of more detailed partial orders: by specifying ordering between operations with more complex but seemingly weaker conditions, we can (1) require only minimal ordering from implementations, and at the same time (2) allow clients the flexibility to strengthen the specs by combining the library's exposed internal ordering with the client-generated external ordering.

**MESSAGE-PASSING CLIENT VERIFICATION** When a call to **deq** returns empty ( $\epsilon$ ), consistency demands that the added empty dequeue event  $d$  satisfies **QUEUE-EMPDEQ**, which is sufficient to verify the MP client (Figure 21.2). Intuitively, **QUEUE-EMPDEQ** says that there cannot be another enqueue  $e$  which happens before  $d$  but has not been dequeued in  $G$ —if there were, then the dequeue would have successfully returned some element from the queue. The verification of MP depends on the fact that both enqueue events  $e_1$  and  $e_2$  done by the left-most thread, of which at most one can be consumed by the middle thread, happen before the dequeue of the right-most thread. By **QUEUE-EMPDEQ** the dequeue cannot be an empty one and must dequeue from  $e_1$  or  $e_2$  and return either 41 or 42.

The proof sketch of this example in Compass is given in Figure 22.2. Following the pattern mentioned at the end of §21.2, we put the ownership  $\text{Queue}(q, \_)$  in an invariant to enforce a concurrent protocol on the queue, using a *dequeue permission* called **deqPerm** that can be defined with Iris ghost state. One dequeue permission  $\text{deqPerm}(1)$  is needed to perform one successful dequeue. This requirement can be seen in the invariant:  $\text{deqPerm}(\text{size}(G.\text{so}))$  counts the number of successful dequeues, and a successful dequeue will extend  $G.\text{so}$  by 1, so anyone who successfully dequeues needs to put in a  $\text{deqPerm}(1)$  to re-establish the

invariant. For our particular example, we also implement `deqPerm` such that there are only *two* `deqPerm(1)`'s (i.e., `deqPerm(2)`) in the whole system. We then give one permission to each consumer thread before they run. Initially the queue is set to be empty, and all threads are given a persistent observation  $\text{SeenQueue}(q, \emptyset, \emptyset)$  of the initial empty state.

The verification of the left-most thread is straightforward: for each enqueue, we use **LAT-INV** to open the invariant and then use **ABS-HB-ENQ**. Afterwards the thread has two enqueue events  $\{e_1, e_2\}$  in its logical view, and the write to flag releases  $\text{SeenQueue}(q, G_1, \{e_1, e_2\})$  to the right-most thread. The verification of the middle thread uses **LAT-INV** and **ABS-HB-DEQ**, and if the dequeue succeeds, `deqPerm(1)` can be given up to re-establish the client invariant.

Finally, in the verification of the right-most thread, the acquire read of 1 from flag receives  $\text{SeenQueue}(q, G_1, \{e_1, e_2\})$  from the left-most thread. We then use **LAT-INV** and **ABS-HB-DEQ** to perform the dequeue, with  $M_0 := \{e_1, e_2\}$ . Before re-establishing the invariant, we inspect the resulting dequeue  $d_3$ . If it is a successful dequeue, we can put `deqPerm(1)` in the invariant and finish. If  $d_3$  is an empty dequeue, we derive a contradiction. As there are only two `deqPerm(1)` permissions in the whole system, of which one is owned by the current (right-most) thread, when we open the invariant we know that the most up-to-date (right before  $d_3$ ) graph  $G$  can have at most one dequeue:  $\text{size}(G.\text{so}) \leq 1$ . Furthermore, the thread has observed two enqueues, so in  $G$  there must be at least one enqueue that is not dequeued yet, which must be in  $\{e_1, e_2\}$ . Due to  $\text{SeenQueue}(q, G_1, \{e_1, e_2\})$ , both  $e_1$  and  $e_2$  happen before  $d_3$ . By **QUEUE-EMPDEQ**, we have our contradiction.  $\square$

## 22.2 Weaker Specs by Abandoning Abstract States

The  $\text{LAT}_{\text{hb}}^{\text{abs}}$  specs are particularly strong and only satisfiable by strong implementations, because one must be able to construct the abstract state at commit points. For example, we have verified that a purely release-acquire implementation of the Michael-Scott queue<sup>4</sup> satisfies the  $\text{LAT}_{\text{hb}}^{\text{abs}}$  specs for queues (and therefore transitively the  $\text{LAT}_{\text{so}}^{\text{abs}}$  specs). The release-acquire memory model, though not as strong as the SC or Multicore OCaml model, still provides sufficient synchronization to construct the list of values *vs* in the queue.

However, it is extremely difficult to construct the abstract state for the relaxed Herlihy-Wing queue implementation mentioned above: it would require delicate reordering of commit points on the fly, and sometimes require *future-dependent* knowledge about dequeue operations. In fact, the verification of the LAT specs in the SC memory model for Herlihy-Wing queue relied on *prophecy variables*,<sup>5</sup> whose application in RMC is still an open research problem. In this work we instead verify the relaxed Herlihy-Wing implementation against  $\text{LAT}_{\text{hb}}$  specs, a weakening of the  $\text{LAT}_{\text{hb}}^{\text{abs}}$  specs where the abstract state is abandoned. In particular, our instance of the  $\text{LAT}_{\text{hb}}$  specs for queues is exactly the specs **ABS-HB-ENQ** and **ABS-HB-DEQ** (Figure 22.1) *without* *vs*.

<sup>4</sup>Michael and Scott, “Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms” [MS96].

<sup>5</sup>Jung et al., “The future is ours: prophecy variables in separation logic” [Jun+20].

$\text{LAT}_{\text{hb}}$  specs may appear weak, but they can still take advantage of external synchronization information, *i.e.*, the argument in §22.1 about flexibility of the partial orders still applies. Practically, they are sufficient to verify the MP client in Figure 21.2.

We can also use them to verify the following *single-producer single-consumer* (SPSC) client of a queue:

$$\left\{ \begin{array}{l} \text{SeenQueue}(q, \_, \_) * \\ a_p \mapsto [a_0, \dots, a_{n-1}] * \dots \\ \text{produce}(q, a_p, 0, n) \\ \{a_p \mapsto [a_0, \dots, a_{n-1}] * \dots\} \end{array} \right\} \parallel \left\{ \begin{array}{l} \text{SeenQueue}(q, \_, \_) * \\ a_c \mapsto [0, \dots, 0] * \dots \\ \text{consume}(q, a_c, 0, n) \\ \{a_c \mapsto [a_0, \dots, a_{n-1}] * \dots\} \end{array} \right\}$$

Here, there is only one thread performing enqueues—the **producer**—and only one thread performing dequeues—the **consumer**. The **producer** reads the array  $a_p$  for elements with the indices in  $[0, n)$  and enqueues them in that order, while the **consumer** keeps dequeuing for  $n$  elements and writes them in the indices  $[0, n)$  of the array  $a_c$  in the dequeuing order. The expected behavior is FIFO: in the end the array  $a_c$  should have the same elements as  $a_p$ .

To verify this example, we use the  $\text{LAT}_{\text{hb}}$  specs for queues (*i.e.*, **ABS-HB-ENQ** and **ABS-HB-DEQ** without abstract states) to derive *stronger*  $\text{LAT}_{\text{hb}}$ -style specs for SPSC queues, simply by building a concurrent SPSC client protocol. In this derivation, thanks to logical atomicity, at every commit point of a successful dequeue we can easily match it up with the right enqueue and thus prove FIFO. With the SPSC  $\text{LAT}_{\text{hb}}$  specs, the example’s verification is straightforward.

## 22.3 Implementing Compass Specs in iRC11

Surprisingly, it is rather straightforward to implement logical atomicity specs in iRC11: we reuse the general definition of *atomic update* that Iris provides (see Definition 22.1), and RMC effects can be fully accounted for using the various view modalities introduced in §8.5.

**Definition 22.1** (General Iris Atomic Update).

$$\langle x. P \mid y. Q \Rightarrow \Phi \rangle_{\mathcal{E}'}^{\mathcal{E}}$$

iRC11 reuses the general logical atomicity setup provided by Iris for all bunched-implication logics. The key concept in this setup is the atomic update which encodes the obligations to update some state around an atomic commit point.

$$\langle x. P \mid y. Q \Rightarrow \Phi \rangle_{\mathcal{E}'}^{\mathcal{E}} ::= \mathcal{E} \Vdash^{\mathcal{E}'} \exists x. P * \wedge \left\{ \begin{array}{l} P \xrightarrow{\mathcal{E}'}^{\mathcal{E}} \langle x. P \mid y. Q \Rightarrow \Phi \rangle_{\mathcal{E}'}^{\mathcal{E}} \\ \forall y. Q \xrightarrow{\mathcal{E}'}^{\mathcal{E}} \Phi \end{array} \right.$$

The atomic update  $\langle x. P \mid y. Q \Rightarrow \Phi \rangle_{\mathcal{E}'}^{\mathcal{E}}$  encodes the obligation that one has to prove—potentially using some invariants in  $\mathcal{E} \setminus \mathcal{E}'$  thanks to the mask-changing update  $\mathcal{E} \Vdash^{\mathcal{E}'}$ —an *atomic pre-condition*  $P$  that may depend on some existentially quantified value  $x$ . One then will have a choice—thanks to the classical conjunction  $\wedge$ —either to *abort* the update by returning  $P$  and get back the atomic update, or to *commit* the update



by assuming the *atomic post-condition*  $Q$  (which can depend on some new value  $y$ ) to re-establish the invariants in  $\mathcal{E} \setminus \mathcal{E}'$  (due to  $\mathcal{E}' \models^{\mathcal{E}}$ ) as well as the *private post-condition*  $\Phi$ . Note that both  $Q$  and  $\Phi$  may depend on both  $x$  and  $y$ .

Intuitively, the client of a logically atomic triple will prove this atomic update *i.e.*, they will have to produce  $x$  and the atomic pre-condition  $P$ , as well as proving the transformation of  $Q$  to  $\Phi$ . Meanwhile, the prover of that logically atomic triple will consume this atomic update *i.e.*, they will consume  $P$  and produce  $y$  and the atomic post condition  $Q$ .

**Definition 22.2** (Logically Atomic Triples). A logically atomic triple in iRC11 is encoded using the atomic update instantiation for vProp (*i.e.*,  $P$ ,  $Q$ ,  $\Phi$  are predicates on vProp) and the vProp weakest pre-condition.<sup>6</sup>

<sup>6</sup>see Definition 8.4

$$\langle x. P \rangle e \text{ in } \pi \langle y. Q \rangle_{\mathcal{E}} \{R\} ::= \\ \forall \Phi. \left( \langle x. P \mid y. Q \Rightarrow (R \multimap \Phi) \rangle_{\mathcal{N}_{\text{HIST}}}^{\top \setminus \mathcal{E}} \multimap \text{wp}_{\top} e \text{ in } \pi \{ \Phi \} \right)$$

To say that  $e$  satisfies an atomic triple is to say that an atomic update for some post-condition  $(R \multimap \Phi)$  is at least the weakest pre-condition needed for  $e$  to result in a post-condition  $\Phi$ .

$\mathcal{E}$  is the mask used by the prover of the triple to host *internal* invariants that are needed to verify the implementation. The client of the triple therefore can use any invariants outside of  $\mathcal{E}$  *i.e.*, any invariants in  $(\top \setminus \mathcal{E}) \setminus \mathcal{N}_{\text{HIST}}$ . Recall that  $\mathcal{N}_{\text{HIST}}$  is the mask needed by the model of the base logic of iRC11, so it cannot be used by the client either.<sup>7</sup>

<sup>7</sup>see more in Definition 7.18

Furthermore, the atomic triple also supports the *private post-condition*  $R$  which is produced by the prover of the triple and is only returned to the client *after* the invariants are established. This can be seen by unfolding the atomic update definition:<sup>8</sup>  $R$  appears after the mask-changing update  $\mathcal{N}_{\text{HIST}} \models^{\top \setminus \mathcal{E}}$ . The private post-condition support will be needed in the specs of exchangers (see Chapter 24).

<sup>8</sup>see Definition 22.1

Recall the  $\text{LAT}_{\text{hb}}^{\text{abs}}$  spec **ABS-HB-ENQ** for queues in Figure 22.1, we have

$$\begin{aligned} x &::= G, vs \\ y &::= (), G', M', V' \\ P &::= \text{Queue}(q, vs, G) \\ Q &::= \text{Queue}(q, vs \uparrow\uparrow [(v, V')], G') * \dots \\ R &::= \text{True} \end{aligned}$$

Therefore, **ABS-HB-ENQ** intuitively says that, given an atomic update that can atomically provide the queue state  $\text{Queue}(q, vs, G)$  and in return receive the updated state  $\text{Queue}(q, vs \uparrow\uparrow [(v, V')], G')$  at the commit point, the implementation of **enq**  $([q, v])$  can be verified.

Note that we always want to share a library's state ownership, *e.g.*,  $\text{Queue}(q, vs, G)$ , so our specs always require it to be objective.<sup>9</sup> Recall that we can always make some resource objective by using the view-at modality.<sup>10</sup> We will then have to maintain that the local assertions, *e.g.*,  $\text{SeenQueue}(q, G, M)$  are tied to the views of the view-at modality used in the state ownership, *e.g.*,  $\text{Queue}(q, vs, G)$ .

<sup>9</sup>see Definition 8.9

<sup>10</sup>see Definition 8.13



Last but not least, unsurprisingly an iRC11 logically atomic triple does imply a normal iRC11 Hoare triple,<sup>11</sup> that is

<sup>11</sup>see [Definition 8.5](#)

$$\langle x. P \rangle e \text{ in } \pi \langle y. Q \rangle_{\mathcal{E}} \{R\} \vdash \forall x. \{P\} e \text{ in } \pi \{y. Q * R\}_{\mathcal{E}}$$

This allows use to use iRC11 adequacy<sup>12</sup> to extract adequacy for a logically atomic triple.

<sup>12</sup>see [Theorem 8.7](#)

CHAPTER SUMMARY. In this chapter we have presented a set of concrete Compass specs for queues, and we have showed how to encode them with atomic updates in Iris. In the next chapter we will see how to perform verifications against these specs.



# 23

## Verifications of Stacks and Queues

---

In this chapter, we demonstrate in more details the verifications of stack and queue implementations against Compass strong specs. In §23.1 we present a more complete version of the  $\text{LAT}_{\text{hb}}^{\text{abs}}$  specs for queues given in Figure 22.1, and sketch the verification of a release-acquire implementation of the Michael-Scott queue.<sup>1</sup> We will also show how the specs imply more strongly consistent specs. In §23.2, we present a complete set of  $\text{LAT}_{\text{hb}}$  specs for stacks, and sketch the verification of the release-acquire implementation of the Treiber stack<sup>2</sup> given in Figure 12.8.

<sup>1</sup>Michael and Scott, “Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms” [MS96].

<sup>2</sup>Treiber, *Systems Programming: Coping with Parallelism* [Tre86].

### 23.1 Queue Specs and Verification of the Michael-Scott Queue

#### 23.1.1 Queue Specifications

We present the remaining parts of our  $\text{LAT}_{\text{hb}}^{\text{abs}}$  specs for queues in Figure 23.1, which complete those in Figure 22.1. We add the specs for the try versions who can fail due to contention. We also present the components of QueueConsistent. Again, the  $\text{LAT}_{\text{hb}}$  specs for queues are simply the  $\text{LAT}_{\text{hb}}^{\text{abs}}$  specs without the abstract state *vs.* **ABS-HB-QUEUE-OBJ** says that the queue’s state assertion is objective, so it can be easily accessed from an objective invariant. We make explicit the namespace  $\mathcal{N}_{\text{QUEUE}}$  which is needed to house an implementation’s internal invariants, and which can be picked by clients.

#### 23.1.2 RMC Implementations of Queues

As mentioned in §22.2, we have verified two RMC queue implementations against Compass specs. We have verified a release-acquire implementation of the Michael-Scott queue<sup>3</sup> against the  $\text{LAT}_{\text{hb}}^{\text{abs}}$  spec (see Figure 23.1), and therefore transitively against the  $\text{LAT}_{\text{hb}}$  spec. We have verified an RMC implementation of the Herlihy-Wing queue<sup>4</sup> against the  $\text{LAT}_{\text{hb}}$  spec.

THE RELEASE-ACQUIRE IMPLEMENTATION OF THE MICHAEL-SCOTT QUEUE is given in Figure 23.2. This implementation follows the implementation given in the GPS logic.<sup>5</sup> The queue is implemented with a singly-linked list whose node contains a next field and a data field. Elements are enqueued at the tail and are dequeued from the head of the list.

`new_queue` allocates a new queue with a sentinel node *s* without a data field. Both the head and the tail of the list initially points to the

<sup>3</sup>Michael and Scott, “Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms” [MS96].

<sup>4</sup>Herlihy and Wing, “Linearizability: A Correctness Condition for Concurrent Objects” [HW90].

<sup>5</sup>Turon et al., “GPS: navigating weak memory with ghosts, protocols, and separation” [TVD14].

ABS-HB-NEW-QUEUE

$$\{\text{True}\} \text{new\_queue}() \{q. \text{SeenQueue}^{\mathcal{N}_{\text{QUEUE}}}(q, \emptyset, \emptyset) * \text{Queue}(q, [], \emptyset)\} \mathcal{N}_{\text{QUEUE}}$$

ABS-HB-TRY-ENQ

$$\text{SeenQueue}(q, G_0, M_0) * \exists V \vdash$$

$$\langle G, vs. \text{Queue}(q, vs, G) \rangle$$

**try\_enq**([q, v])

$$\left| \begin{array}{l} b. \exists vs', G' \sqsupseteq G, M' \sqsupseteq M_0, V' \sqsupseteq V. \text{Queue}(q, vs', G') * \text{SeenQueue}(q, G', M') * \exists V' \\ * \vee \begin{cases} b = \text{false} & \wedge vs' = vs \wedge G' = G \\ b = \text{true} & \wedge vs' = vs \mathrel{++} [(v, V')] \wedge \exists e \notin G. e \in M' \wedge G' = G[e \mapsto (\text{Enq}(v), V', M')] \end{cases} \end{array} \right| \mathcal{N}_{\text{QUEUE}}$$

ABS-HB-TRY-DEQ

$$\text{SeenQueue}(q, G_0, M_0) * \exists V \vdash$$

$$\langle G, vs. \text{Queue}(q, vs, G) \rangle$$

**try\_deq**([q])

$$\left| \begin{array}{l} v. \exists vs', G' \sqsupseteq G, M' \sqsupseteq M_0, V' \sqsupseteq V. \text{Queue}(q, vs', G') * \text{SeenQueue}(q, G', M') * \exists V' \\ * \vee \begin{cases} v = \perp & \wedge vs' = vs \wedge G' = G \\ v = \epsilon & \wedge vs' = vs \wedge \exists d \notin G. d \in M' \wedge G' = G[d \mapsto (\text{Deq}(\epsilon), V', M')] \\ v \notin \{\perp, \epsilon\} & \wedge \exists e, M_e, d \notin G, V_e. vs = (v, V_e) :: vs' \wedge G(e) = (\text{Enq}(v), V_e, M_e) \\ & \wedge (e, \_) \notin G.\text{so} \wedge V_e \sqsubseteq V' \wedge M_e \cup \{e, d\} \sqsubseteq M' \\ & \wedge G' = G[d \mapsto (\text{Deq}(v), V', M')] \wedge G'.\text{so} = \{(e, d)\} \cup G.\text{so} \end{cases} \end{array} \right| \mathcal{N}_{\text{QUEUE}}$$

ABS-HB-QUEUE-OBJ

$$\text{objective}(\text{Queue}(q, vs, G))$$

$$\text{QueueConsistent}(vs, G) :=$$

$$\wedge \left\{ \begin{array}{ll} \forall (e, d) \in G.\text{so}. \exists v. G(e).\text{type} = \text{Enq}(v) \wedge G(d).\text{type} = \text{Deq}(v) & (\text{QUEUE-MATCHES-FULL}) \\ \wedge G(e).\text{view} \sqsubseteq G(d).\text{view} \wedge G(e).\text{logview} \sqsubseteq G(d).\text{logview} & \\ G.\text{so} \text{ and } G.\text{so}^{-1} \text{ are functional.} & (\text{QUEUE-SO-FUNCTIONAL}) \\ \forall d. G(d).\text{type} = \text{Deq}(\_) \rightarrow (\_, d) \notin G.\text{so} \rightarrow G(d).\text{type} = \text{Deq}(\epsilon) & (\text{QUEUE-UNMATCHED-EMPDEQ}) \\ \forall (e, d) \in G.\text{so}, e'. G(e').\text{type} = \text{Enq}(\_) \rightarrow (e', e) \in G.\text{lhb} & \\ \rightarrow \exists d'. (e', d') \in G.\text{so} \wedge (e' \neq e \rightarrow (d, d') \notin G.\text{lhb}) & (\text{QUEUE-FIFO-FULL}) \\ \forall d, e. G(d).\text{type} = \text{Deq}(\epsilon) \rightarrow G(e).\text{type} = \text{Enq}(\_) \rightarrow (e, d) \in G.\text{lhb} & \\ \rightarrow \exists d'. (e, d') \in G.\text{so} \wedge (d', d) \in G.\text{lhb} & (\text{QUEUE-EMPDEQ-FULL}) \end{array} \right.$$

FIGURE 23.1: Full  $\text{LAT}_{\text{hb}}^{\text{abs}}$  specs for queue

|                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> next := 0  data := 1 head := 0  tail := 1 new_queue ::=   λ[].   1: let s := alloc(1) in   2: s + next :=<sub>na</sub> 0;   3: let q := alloc(2) in   4: q + head :=<sub>na</sub> s;   5: q + tail :=<sub>na</sub> s;   6: q  find_tail ::=   λ[q].   1: let n := *acq(q + tail) in   2: let n' := *acq(n + next) in   3: if n' = 0   4: then n   5: else   6:   q + next :=<sub>rel</sub> n';   7:   false </pre> | <pre> try_enq ::=   λ[q, v].   1: let n := alloc(2) in   2: n + next :=<sub>na</sub> 0;   3: n + data :=<sub>na</sub> v;   4: let t :=   5:   repeat (find_tail([q])) in   6:   if CAS<sup>relacq</sup>(t + next, 0, n)   7:   then   8:     q + tail :=<sub>rel</sub> n; true   9:   else free(n, 2); false  enq ::=   rec try([q, v]) :=   1: if try_enq([q, v])   2: then ✕   3: else try([q, v]) </pre> | <pre> try_deq ::=   λ[q].   1: let h := *acq(q + head) in   2: let n := *acq(h + next) in   3: if n == 0   4: then 0 // EMPTY   5: else   6:   if CAS<sup>relacq</sup>(q + head, h, n)   7:   then *na(n + 1)   8:   else -1 // FAIL  deq ::=   rec try([q]) :=   1: let v := try_deq([q]) in   2: if 0 ≤ v   3: then v else try([q]) </pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

FIGURE 23.2: A release-acquire Michael-Scott queue

sentinel node.

**try\_enq**([q, v]) first allocates a new node  $n$  with the data field storing the to-be-enqueued value  $v$ , and with the next field null (0). It then uses **find\_tail**([q]) to try to find the “real” tail of the list: in the presence of concurrent enqueues, the value  $t$  one reads from the tail field is not guaranteed to be the most up-to-date tail. **try\_enq** therefore uses a release-acquire **CAS** on  $t + \text{next}$  to make sure that  $t$  is the real tail—the invariant is that the real tail’s next field is always null. If the **CAS** succeeds,  $n$  is the new tail. Otherwise,  $n$  is deallocated. **enq**([q, v]) simply calls **try\_enq**([q, v]) until it succeeds. One can consider optimizing **enq**([q, v]) by inlining **try\_enq** to avoid allocating and deallocating  $n$  multiple times.

Reversely, **try\_deq**([q]) reads the value  $n$  from the next field of  $q$ ’s head, and uses a release-acquire **CAS** on  $q + \text{head}$  to make sure that  $n$  is the first node at the head. If the **CAS** succeeds,  $n$  is dequeued and the function reads  $n$ ’s data field non-atomically and returns the read value.  $n$  then becomes the new sentinel node. If the **CAS** fails, the function returns  $-1$ . Note that when  $n$  is null ( $n == 0$ ), the function *subjectively* sees that the queue is empty and returns 0 (empty) immediately. **deq**([q]) simply calls **try\_deq**([q]) until it succeeds or returns empty. (Technically, this means that we can only put positive numbers into the queue.)

<sup>6</sup>Raad et al., “On library correctness under weak memory consistency: specifying and verifying concurrent libraries under declarative consistency models” [Raa+19].

THE RMC IMPLEMENTATION OF THE HERLIHY-WING QUEUE is given in Figure 23.3. This implementation follows the “weak” implementation from Raad et al.<sup>6</sup>. The queue has a backing store as an array of size  $N$ . When the size  $N$  is reached, i.e., the queue is full, any enqueue will simply diverge. In practice, we may use a circular buffer as the backing store.

`new_queue` allocates the array with size  $N + 1$ , and uses the first element of the array as the back counter to indicate the array slot that will be used for the next-to-be-enqueued element. `back` is initially set to 0. The implementation stores the elements from the index `buff` of the array. `new_queue` initialize the element array to all 0's (conveniently using `back` as a loop counter). We use 0 as the sentinel value to indicate that the array slot is not in used. Therefore the implementation only supports non-zero queue elements.

`enq`( $[q, v]$ ) uses a release **FAA** to increase the back counter by 1 and to reserve the slot  $i$  (the returned value of the **FAA**) for its enqueue (line 1). If the previous value  $i$  is still in bound (of  $N$ ), the reservation succeeded, and the function puts the value  $v$  in the slot  $i$  using a release write (line 3). Otherwise, the queue is full and the function diverges.

`deq`( $[q]$ ) traverses the array from the index 0 to the minimum of  $N$  and the value  $b$  read from `back`. That is, in line 4 of `deq_loop`,  $j$  starts from  $q + \text{buff}$  and ends with  $q + \text{buff} + r$ . The function then uses an acquire exchange to swap the element at  $j$  with 0. If the swapped value  $x$  from  $j$  was 0, it means that the slot  $j$  has been either reserved or dequeued by someone else, and the function continues with the next slot in the array. If  $x$  is non-zero, the dequeue succeeds, and  $x$  is returned. If the function has traversed the whole range  $r$ —that is when  $i$  is 0 in line 1 of `deq_loop`, the function uses the recursion `dequeue` to restart the traversal from the index 0.

As we can see, `deq` commits its effect in line 5 (of `deq_loop`) with a successful *acquire* exchange, and `enq` in line 3 with a *release* write. These are very relaxed operations: while they guarantee that matching enqueue-dequeue pairs are synchronized through the release-acquire idiom, they provide no guaranteed synchronization among enqueues nor among dequeues. Regardless, the queue still enjoys the FIFO property (**QUEUE-FIFO-FULL**), thanks to the combination of (1) the reservation order through `back` by enqueues and (2) the traversal order from index 0 by dequeues.

### 23.1.3 The Verification of the Michael-Scott Queue

We sketch the proof of the Michael-Scott queue implementation in Figure 23.2 against the  $\text{LAT}_{\text{hb}}^{\text{abs}}$  spec in Figure 23.1.<sup>7</sup> The gist of the proofs is the definitions of the queue's state ownership  $\text{Queue}(q, vs, G)$  (Definition 23.7) and the queue's local observation  $\text{SeenQueue}(q, G, M)$  (Definition 23.9).

**Definition 23.1** (Extra ghost state for Michael-Scott queue). We use the authoritative RA to create the master-snapshot ghost state for 4 types of data:

<sup>7</sup>The proof of the Herlihy-Wing will be elided, but is available in the Coq development.

|                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> back := 0  buff := 1 new_queue ::=   λ[].   1: let q := alloc(N + 1) in   2: q + back :=<sub>na</sub> 0;   3: repeat (   4:   let c := *(q + back) in   5:   q + buff + c := 0;   6:   q + back := c + 1;   7:   c + 1 == N   8: );   9: q + back := 0; q  enq ::=   λ[q, v].   1: let i := FAA<sup>rel</sup>(q + back, 1) in   2: if i &lt; N   3: then q + buff + i :=<sub>rel</sub> v   4: else diverge([⊥]) </pre> | <pre> deq_loop ::=   λ[dequeue, q, r].   rec loop([i]) :=   1: if i == 0   2: then dequeue([q])   3: else   4:   let j := q + buff + r - i in   5:   let x := xchg<sup>acq</sup>(j, 0) in   6:   if x == 0   7:   then loop([i - 1])   8:   else x  deq ::=   rec dequeue([q]) :=   1: let b := *acq(q + back) in   2: let r := min(b, N) in   3: deq_loop([dequeue, q, r])([r]) </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

FIGURE 23.3: An RMC Herlihy-Wing Queue

- We need  $\text{master}^\gamma(G)$  to record the current graph  $G$  for some ghost location  $\gamma$ , and  $\text{snap}^\gamma(G')$  for some snapshot  $G'$  of  $G$  ( $G' \sqsubseteq G$ ). This type of ghost state is needed generally in the graph-based specs of Compass.
- We need  $\text{master}^\gamma(\mathcal{T})$  to store the persistent mapping from a queue's node identified by some ghost location  $\eta$  to some enqueue event  $e$ . That is,  $\mathcal{T} \in G\text{Name} \xrightarrow{\text{fin}} \text{EventId}$ . The ghost state  $\text{snap}^\gamma(\mathcal{T}')$  records that  $\mathcal{T}'$  is a snapshot of  $\mathcal{T}$  ( $\mathcal{T}' \sqsubseteq \mathcal{T}$ ).
- We need  $\text{master}^\gamma(\mathcal{L})$  to store the current list of enqueued nodes, where  $\mathcal{L} \in (G\text{Name} \times \text{Loc})^*$  is a list of pairs of a node identifier and the node's physical location  $(\eta, \ell)$ . The ghost state  $\text{snap}^\gamma(\mathcal{L}')$  records that  $\mathcal{L}'$  is a snapshot of  $\mathcal{L}$  ( $\mathcal{L}' \sqsubseteq \mathcal{L}$ ).
- We need  $\text{master}^\gamma(\mathcal{D})$  to store the current list of dequeued nodes, where  $\mathcal{D} \in (G\text{Name} \times \text{Loc})^*$ . The ghost state  $\text{snap}^\gamma(\mathcal{D}')$  records that  $\mathcal{D}'$  is a snapshot of  $\mathcal{D}$  ( $\mathcal{D}' \sqsubseteq \mathcal{D}$ ). Note that we maintain that  $\mathcal{D} \sqsubseteq \mathcal{L}$ .

**Definition 23.2** (Ownership of a node). We sketch the ownership of a node of the queue. Owning a node with identifier  $\eta$  and physical location  $\ell$  means the atomic ownership of  $\ell + \text{next}$ 's field with the atomic period  $\eta$ . The history  $h$  of  $\eta$  can only be in two states (represented by  $n$ ): (1) the initial state where  $\ell + \text{next}$  is null (0), or (2) the “linked” state where

$\ell + \text{next}$  points to the next node  $(\eta', \ell')$ .

$$\begin{aligned} \text{ownNode}^{G, \mathcal{T}}(\eta, \ell, n) ::= & \\ & \exists t_0, V_0, h. \ell + \text{next} \xrightarrow{\eta}_{\text{con}} h * \\ & \left\{ \begin{array}{ll} n = \text{None} & h = [t_0 \leftarrow (0, V_0)] \\ n = \text{Some}(\eta', \ell, d') & \exists v', V', e'. \mathcal{T}(\eta') = e' \\ & * h = [t_0 \leftarrow (0, V_0)][t_0 + 1 \leftarrow (\ell', V')] \\ & * @_{V'} \left( \text{seenEnq}(G, e', v') * \ell' \sqsubseteq_{\text{sy}}^{\eta'} [- \leftarrow (0, -)] \right) \\ & * \text{if } d' \text{ then True else } @_{V'} (\ell' + \text{data} \mapsto v') \end{array} \right. \end{aligned}$$

where

$$\begin{aligned} \text{seenEnq}(G, e', v') ::= & \\ & G(e').\text{type} = \text{Enq}(v') * \text{seenEvt}(G, e') * \\ & \text{syncEnqLogView}(G, G(e').\text{logview}) \\ \text{seenEvt}(G, e') ::= & \\ & \sqsubseteq G(e').\text{view} * @_{G(e').\text{view}}(\text{seenLogView}(G, G(e').\text{logview})) \\ \text{seenLogView}(G, M) ::= & \forall e \in M. \sqsubseteq G(e).\text{view} \\ \text{syncEnqLogView}(G, M) ::= & \\ & \forall e \in M. G(e) = \text{Enq}(\_) \Rightarrow G(e).\text{logview} \subseteq M \end{aligned}$$

(QUEUE-MS-SYNC-ENQS)

In the latter case,  $\text{ownNode}$  of  $\eta$  also owns  $\text{seenEnq}(G, e', v')$ , the observation that the next node  $(\eta', \ell')$  has been enqueued by the enqueue event  $e'$  in a snapshot graph  $G$ , with the value  $v'$ , as well as the observation  $\ell' + \text{next} \sqsubseteq_{\text{sy}}^{\eta'} [- \leftarrow (0, -)]$  that  $\ell' + \text{next}$  field has been initialized to null. Furthermore, if the next node  $(\eta', \ell')$  has *not* been dequeued—represented by  $d' == \text{false}$ , then the ownership of  $\eta$  holds on to the data field of  $\eta'$ , i.e.,  $\ell' + \text{data} \mapsto v'$ .

The reason we put the data field of  $\eta'$  in the ownership of its previous node  $\eta$  is that, when dequeuing  $\eta'$ , the implementation simply updates the head pointer from  $\ell$  (of  $\eta$ ) to  $\ell'$  (of  $\eta'$ ). Before doing that, the implementation would have read  $\ell + \text{next}$  and would have acquired only the value  $\ell'$  and the view  $V'$ . That is, the dequeue can only acquire the ownership  $\ell' + \text{data} \mapsto v'$  safely after having acquired  $V'$  from the history  $h$  of  $\ell + \text{next}$ . It is therefore necessary to tie  $\ell' + \text{data}$  field to  $\ell + \text{next}$  field.

**Remark 23.3** (Logical happens-before vs. physical happens-before). Note that the observation  $\text{seenEnq}(G, e', v')$  enforces strong synchronization properties that are satisfiable by the release-acquire Michael-Scott queue implementation in [Figure 23.2](#), but not necessarily all implementations. It requires  $\text{seenEvt}(G, e')$  that says that the observation also observes not only the physical view of the event  $e'$  ( $\sqsubseteq G(e').\text{view}$ ), but also transitively (through the definition of  $\text{seenLogView}(G, M)$ ) all physical views of all events  $e$ 's that *logically happen before*  $e'$ . We note that



the fact that  $e \in G(e').\text{logview}$ — $e$  logically happens before  $e'$ —does not necessarily always means that  $e$  *physically* happens before  $e'$ —it is the  $\text{seenLogView}(G, M)$  condition that enforces such interpretation. Furthermore, logical happens-before among graph events is not necessarily transitive:  $e_1 \in G(e_2).\text{logview}$  and  $e_2 \in G(e_3).\text{logview}$  does not need to imply  $e_1 \in G(e_3).\text{logview}$ . In fact, we only require **QUEUE-MS-SYNC-ENQS** (see the definition of  $\text{syncEnqLogView}(G, M)$ ) that says that logical happens-before is transitive at least for enqueue events. This generality of allowing logical happens-before to deviate from physical happens-before is not really needed for our release-acquire implementation, but can become useful to allow for more relaxed implementations (e.g., the Herlihy-Wing queue or the Treiber stack) which interpret logical happens-before more “relaxedly”.

**Definition 23.4** (Ownership of all nodes). All nodes of a queue include the sentinel node  $(\eta_0, s_0)$ , the dequeued nodes  $\mathcal{D}$ , and the nodes that are still in the queue  $\mathcal{L}_{\text{in}}$

$$\begin{aligned} \text{ownNodes}^{G, \mathcal{T}}(\eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}}) ::= & \\ & \text{let } \mathcal{L} := (\eta_0, s_0) ++ \mathcal{D} ++ \mathcal{L}_{\text{in}} \text{ in} \\ & \text{let } \mathcal{L}' := \text{nextNodes}(\eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}}) \text{ in} \\ & \bigstar_{(\eta, \ell, n) \in (\mathcal{L}, \mathcal{L}')} \text{ownNode}^{G, \mathcal{T}}(\eta, \ell, n) \end{aligned}$$

The ownership  $\text{ownNodes}$  simply collects all  $\text{ownNode}$ . The definition relies on the function  $\text{nextNodes}$  that computes the correct next node information  $n$  for a node. For all but the last node in  $\mathcal{L}$ ,  $n$  will be Some of the next node in the list  $\mathcal{L}$ . For all but the last node in  $(\eta_0, s_0) ++ \mathcal{D}$ ,  $n$  will have  $d' == \text{true}$ .

We note that all nodes in  $\mathcal{L}$  are unique, because each is identified by a logical name  $\eta$  which in turn is uniquely identified by an enqueue event.

**Definition 23.5** (Observations of node links). We define the observations of node links.

$$\begin{aligned} \text{seenLink}^{G, \mathcal{T}, \mathcal{L}}(\eta, \ell, \eta', \ell', e', v') ::= & \\ & \mathcal{T}(\eta') = e' * \text{seenEnq}(G, e', v') * \ell' \sqsupseteq_{\text{sy}}^{\eta'} [\_ \leftarrow (0, \_)] * \\ & (\eta, \ell) ++ (\eta', \ell') \sqsubseteq \mathcal{L} * \ell \sqsupseteq_{\text{sy}}^{\eta} [\_ \leftarrow (\ell', \_)] \\ \text{seenLinks}^{G, \mathcal{T}, \mathcal{L}}(M) ::= & \\ & \forall e' \in M \cap G. G(e').\text{type} = \text{Enq}(v') \Rightarrow \\ & \exists \eta, \ell, \eta', \ell'. \text{seenLink}^{G, \mathcal{T}, \mathcal{L}}(\eta, \ell, \eta', \ell', e', v') \\ \text{headLinks}^{G, \mathcal{T}, \mathcal{D}}(\bar{V}) ::= & \\ & \bigstar_{(\eta', \ell', V) \in (\mathcal{D}, \bar{V})} @_V(\exists e'. \text{seenLink}^{G, \mathcal{T}, \mathcal{D}}(\_, \_, \eta', \ell', e', \_) * (e', \_) \in G.\text{so} * \dots) \\ \text{tailLinks}^{G, \mathcal{T}, \mathcal{L}}(h) ::= & \\ & \bigstar_{(t, v, V) \in h} @_V(\exists \eta', \ell', e'. v = \ell' * \text{seenLink}^{G, \mathcal{T}, \mathcal{L}}(\_, \_, \eta', \ell', e', \_)) \\ & \bullet \text{ The observation } \text{seenLink}^{G, \mathcal{T}, \mathcal{L}}(\eta, \ell, \eta', \ell', e', v') \text{ says that the node } \\ & \quad (\eta', \ell') \text{ has been enqueued to the graph } G \text{ with the event } e' \text{ (} \eta' \end{aligned}$$

is mapped to  $e'$  in  $\mathcal{T}$ ) and with the value  $v'$ . We have seen this information tracked also in `ownNode`. Additionally, `seenLink` also observes that  $(\eta', \ell')$  is linked to the node  $(\eta, \ell)$ , because it immediately follows  $(\eta, \ell)$  in the node list  $\mathcal{L}$ .

- The observation  $\text{seenLinks}^{G, \mathcal{T}, \mathcal{L}}(M)$  collects all of those single-link observations for all enqueue events in the logical view  $M$ .
- The observation  $\text{headLinks}^{G, \mathcal{T}, \mathcal{D}}(\bar{V})$  collects the single-link observations for all nodes that the head has seen, i.e., all nodes that have been *dequeued*, as signaled by the meta-variable  $\mathcal{D}$ . The list of views  $\bar{V}$  will come from the **CASes** to head. This tracks the fact that every **CAS** to the head—which dequeues a node  $\eta$ —is also synchronized with the enqueue event of  $\eta$ .
- The observation  $\text{tailLinks}^{G, \mathcal{T}, \mathcal{L}}(h)$  collects the single-link observations for all nodes that the tail has seen, i.e., all nodes that have been written to the history  $h$  of the tail. This will allow the tail to *release* those observations to whoever reads the tail with an acquire read.

**Definition 23.6** (Ownership of a queue’s physical locations). The physical locations of a queue include all nodes and the head and the tail.

$$\begin{aligned} \text{ownQueueLocs}_{\gamma_h, \gamma_t}^{G, \mathcal{T}}(\eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}}) ::= & \\ \text{ownNodes}^{G, \mathcal{T}}(\eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}}) * & \\ \exists \bar{V}. q + \text{head} \mapsto_{\text{con}}^{\gamma_h} (\_, s_0 ++ \mathcal{D}.2, \bar{V}) * \text{headLinks}^{G, \mathcal{T}, (\eta_0, s_0) ++ \mathcal{D}}(\bar{V}) * & \\ \exists h_t. q + \text{tail} \mapsto_{\text{con}}^{\gamma_t} h_t * \text{tailLinks}^{G, \mathcal{T}, (\eta_0, s_0) ++ \mathcal{D} ++ \mathcal{L}_{\text{in}}}(h_t) & \end{aligned}$$

**Definition 23.7** (Ownership of a queue’s state). The ownership of a queue’s state  $\text{Queue}^{\gamma_g}(q, vs, G)$  is implemented purely with ghost state. Since we are verifying the implementation against the  $\text{LAT}_{\text{hb}}^{\text{abs}}$  spec with an abstract queue  $vs$ , we additionally employ a master-snapshot instance for that abstract state. We expose halves of the master ghost state for  $G$  and  $vs$ . The other halves will be governed by the queue’s internal invariant.

$$\begin{aligned} \text{Queue}^{\gamma_g}(q, vs, G) ::= & \\ \text{QueueConsistent}(vs, G) * \text{master}_{1/2}^{\gamma_g}(G) * \text{master}_{1/2}(vs) & \end{aligned}$$

$\text{Queue}^{\gamma_g}(q, vs, G)$  is trivially objective.

**Definition 23.8** (The queue’s internal invariant). We define the internal invariant  $\boxed{\text{queueel}(q, \gamma, \gamma_g, \gamma_h, \gamma_t)}^{\mathcal{N}_{\text{QUEUE}}}$  that is needed by verification and that is not exposed to clients of the queue specs.

$$\begin{aligned} \text{queueel}(q, \gamma, \gamma_g, \gamma_h, \gamma_t) ::= & \\ \exists vs, G. \text{Queue}^{\gamma_g}(q, vs, G) * \exists \mathcal{T}, \eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}}. & \\ \text{let } \mathcal{L} := (\eta_0, s_0) ++ \mathcal{D} ++ \mathcal{L}_{\text{in}} \text{ in} & \\ \text{master}^{\gamma.1}(\mathcal{L}) * \text{master}^{\gamma.2}(\mathcal{D}) * \text{master}^{\gamma.3}(\mathcal{T}) * & \\ \exists V_b. @_{V_b} \text{ownQueueLocs}_{\gamma_h, \gamma_t}^{G, \mathcal{T}}(\eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}}) * & \\ \text{queueViews}(G, \mathcal{T}, \mathcal{L}) * \text{queueProps}(vs, G, \mathcal{T}, \eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}}) & \end{aligned}$$

where

$$\begin{aligned}
& \text{queueViews}(G, \mathcal{T}, \mathcal{L}) ::= \\
& \quad \forall e \in G. @_{G(e).\text{view}} \left( \text{seenLinks}^{G, \mathcal{T}, \mathcal{L}}(G(e).\text{logview}) \right) \\
& \text{queueProps}(vs, G, \mathcal{T}, \eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}}) ::= \\
& \quad \left\{ \begin{array}{l}
((\eta_0, s_0) ++ \mathcal{D} ++ \mathcal{L}_{\text{in}}).1 \text{ does not have duplicated node ids} \\
\hspace{15em} (\text{QUEUE-MS-NO-DUP}) \\
\text{dom}(\mathcal{T}) = (\mathcal{D} ++ \mathcal{L}_{\text{in}}).1 \hspace{10em} (\text{QUEUE-MS-ENQ-ONLY-1}) \\
\text{codom}(\mathcal{T}) = \{e \in G \mid G(e).\text{type} = \text{Enq}(\_)\} \\
\hspace{15em} (\text{QUEUE-MS-ENQ-ONLY-2}) \\
\mathcal{T} \text{ is injective} \hspace{15em} (\text{QUEUE-MS-MAP-INJ}) \\
\{e \mid \mathcal{T}(\eta) = e \wedge (\eta, \_) \in \mathcal{D}\} = G.\text{so}.1 \\
\hspace{15em} (\text{QUEUE-MS-MAP-DEQ-GRAPH}) \\
\forall e_1, e_2, \eta_1, \eta_2. \mathcal{T}(\eta_1) = e_1 \Rightarrow \mathcal{T}(\eta_2) = e_2 \Rightarrow \\
\quad \eta_1 \text{ precedes } \eta_2 \text{ in } \mathcal{D} ++ \mathcal{L}_{\text{in}} \Rightarrow e_1 \leq e_2 \hspace{1em} (\text{QUEUE-MS-ENQ-MO}) \\
\forall e_1, e_2, \eta_1, \eta_2. \mathcal{T}(\eta_1) = e_1 \Rightarrow \mathcal{T}(\eta_2) = e_2 \Rightarrow \\
\quad \eta_1 \text{ precedes } \eta_2 \text{ in } \mathcal{D} ++ \mathcal{L}_{\text{in}} \Rightarrow e_1 \in G(e_2).\text{logview} \\
\hspace{15em} (\text{QUEUE-MS-ENQ-HB}) \\
vs = \{v \mid \eta \in \mathcal{L}_{\text{in}}.1 \wedge \mathcal{T}(\eta) = e \wedge G(e).\text{type} = \text{Enq}(v)\} \\
\hspace{15em} (\text{QUEUE-MS-ABS})
\end{array} \right.
\end{aligned}$$

The invariant contains:

- The other halves of the ghost state for the abstract queue  $vs$  and the graph  $G$ , collected in  $\text{Queue}^{\gamma_g}(q, vs, G)$ . This means that the invariant is always in agreement on the state with the client, who owns the “main” halves in their copy of  $\text{Queue}^{\gamma_g}(q, vs, G)$ .
- The master (authoritative) ghost state instances for the list  $\mathcal{L}$  of enqueued nodes ( $\text{master}^{\gamma \cdot 1}(\mathcal{L})$ ), the list  $\mathcal{D}$  of dequeued nodes ( $\text{master}^{\gamma \cdot 2}(\mathcal{D})$ ), and the mapping  $\mathcal{T}$  from node ids to the graph  $G$ ’s event ids ( $\text{master}^{\gamma \cdot 3}(\mathcal{T})$ ).
- The ownership  $\text{ownQueueLocs}_{\gamma_h, \gamma_t}^{G, \mathcal{T}}(\eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}})$  of all physical locations of the queue, put under a view-at modality to make sure that the invariant is objective.
- The observations  $\text{queueViews}(G, \mathcal{T}, \mathcal{L})$  of node links per graph event. Intuitively, we require that the physical view of an event  $e$  in  $G$  justifies all single-link observations for all enqueued events that happen before  $e$  (that is, all enqueued events observed by  $e$ ’s logical view).
- The properties  $\text{queueProps}$  connecting the abstract state  $vs$  with the graph  $G$  and the ghost state  $\mathcal{T}$  and  $\mathcal{L}$ .
  - **QUEUE-MS-NO-DUP** says that all nodes in a queue are unique.

- **QUEUE-MS-ENQ-ONLY-1** says that the map  $\mathcal{T}$  tracks all nodes in  $(\mathcal{D} ++ \mathcal{L}_{\text{in}})$  and **QUEUE-MS-ENQ-ONLY-2** says that those nodes are all of those that have ever been put into the queue.
- **QUEUE-MS-MAP-INJ** makes sure that each enqueue event in  $G$  is tied to exactly on node id.
- **QUEUE-MS-MAP-DEQ-GRAPH** makes sure that  $\mathcal{D}$  are all dequeued nodes.
- **QUEUE-MS-ENQ-MO** says that the order in the enqueued node list agrees with the insertion order of event ids into the graph  $G$ .
- **QUEUE-MS-ENQ-HB** requires that a node  $\eta_1$  enqueued earlier than  $\eta_2$  (earlier in the order in the enqueued node list) also happens before  $\eta_2$ . This is a very strong synchronization property that holds for the release-acquire Michael-Scott queue, but may *not* hold for all queue implementations.
- **QUEUE-MS-ABS** says that the node list  $\mathcal{L}_{\text{in}}$  indeed is the nodes that are in the queue and agrees with the abstract queue *vs*.

**Definition 23.9** (Local observation of a queue's state). The local observation of a queue's state  $\text{SeenQueue}(q, G, M)$  is implemented with the snapshots of the queue's ghost state instances, the queue internal invariant, the observations of the queue's node links, as well as the observations of the queue's head and tail physical locations.

$$\begin{aligned}
\text{SeenQueue}^{\gamma_g}(q, G, M) ::= & \\
& \text{snap}^{\gamma_g}(G) * \text{seenLogView}(G, M) * \\
& \exists \gamma, \gamma_h, \gamma_t. \boxed{\text{queue}(q, \gamma, \gamma_g, \gamma_h, \gamma_t)}^{\mathcal{N}_{\text{QUEUE}}} * \\
& \exists \mathcal{T}, \eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}}. \text{let } \mathcal{L} := (\eta_0, s_0) ++ \mathcal{D} ++ \mathcal{L}_{\text{in}} \text{ in} \\
& \text{snap}^{\gamma.1}(\mathcal{L}) * \text{snap}^{\gamma.2}(\mathcal{D}) * \text{snap}^{\gamma.3}(\mathcal{T}) * \\
& \text{syncEnqLogView}(G, M) * \\
& \text{seenLinks}^{G, \mathcal{T}, \mathcal{L}}(M) * \\
& q + \text{tail} \sqsubseteq_{\text{sn}}^{\gamma_t} \_ * \exists \mathcal{D}' \sqsubseteq \mathcal{D}. q + \text{head} \sqsubseteq_{\text{sn}}^{\gamma_h} (\_, s_0 ++ \mathcal{D}'.2, \_) * \dots
\end{aligned}$$

More concretely,

- $\text{snap}^{\gamma_g}(G)$  enforces that  $G$  is a snapshot of the current graph (who is stored in the master ghost state in the internal invariant).
- $\text{seenLogView}(G, M)$  enforces that the current thread has observed all physical views of all events in the logical view  $M$ . Additionally,  $\text{syncEnqLogView}(G, M)$  says that the thread observes all enqueue events that transitively happen-before the events in  $M$ .
- $\text{snap}^{\gamma.1}(\mathcal{L})$ ,  $\text{snap}^{\gamma.2}(\mathcal{D})$ , and  $\text{snap}^{\gamma.3}(\mathcal{T})$  together show that the current thread has extracted the snapshots  $\mathcal{L}$ ,  $\mathcal{D}$ , and  $\mathcal{T}$  of the corresponding ghost instances from the internal invariant.
- $\text{seenLinks}^{G, \mathcal{T}, \mathcal{L}}(M)$  says that the current thread observes all enqueueing links for all enqueue events in  $\eta$ , which must already be included in the snapshot  $\mathcal{L}$ .

- $q + \text{tail} \sqsupseteq_{\text{sn}}^{\gamma_t} \_$  says that the thread has some observation on the queue tail, which is enough to perform operations on the tail.
- $q + \text{head} \sqsupseteq_{\text{sn}}^{\gamma_h} (\_, s_0 ++ \mathcal{D}'.2, \_)$  also says that the thread has some observation on the queue head, and that observation must follow the dequeued list snapshot  $\mathcal{D}$  ( $\mathcal{D}' \sqsubseteq \mathcal{D}$ ). This is because dequeues are done exclusively on the head (using CASes).

Below, we sketch the proofs of **ABS-HB-TRY-ENQ** and **ABS-HB-TRY-DEQ**.

*Proof sketch of **try\_enq**([q, v]).* We start by unfolding the logically atomic triple (**Definition 22.2**), so we have in our context  $\text{SeenQueue}(q, G_0, M_0)$ ,  $\sqsupseteq V$ , and the atomic update  $\langle G, vs. \text{Queue}(q, vs, G) \mid b. \dots \Rightarrow \Phi \rangle_{\mathcal{N}_{\text{HIST}}^{\top} \setminus \mathcal{N}_{\text{QUEUE}}^{\top}}$ , and our goal is  $\text{wp}_{\top} \text{try\_enq}([q, v])$  in  $\pi \{\Phi\}$ .

Lines 1 to 3 (see **Figure 23.2**) are easy, as they allocate and initialize a fresh node non-atomically. Afterwards, we have  $n + \text{data} \mapsto v$  and  $n + \text{next} \mapsto 0$ .

Next, in lines 4 and 5, to make a call to **find\_tail**([q]), we apply its specs, given below (and proven separately).

$$\begin{aligned} & \text{ABS-HB-FIND-TAIL} \\ & \left\{ q + \text{tail} \sqsupseteq_{\text{sn}}^{\gamma_t} \_ * \boxed{\text{queue}(\gamma, \gamma_g, \gamma_h, \gamma_t)}^{\mathcal{N}_{\text{QUEUE}}} \right\} \\ & \text{find\_tail}([q]) \text{ in } \pi \\ & \left\{ \begin{array}{l} t.t = 0 \vee \\ \exists \ell_t. t = \ell_t \wedge \exists \eta_t, G_t, \mathcal{T}_t, \mathcal{L}_t. \text{snap}^{\gamma_g}(G_t) * \text{snap}^{\gamma_1}(\mathcal{L}_t) * \text{snap}^{\gamma_3}(\mathcal{T}_t) \\ * \text{seenLink}^{G_t, \mathcal{T}_t, \mathcal{L}_t}(\_, \_, \eta_t, \ell_t, \_, \_) \end{array} \right\} \end{aligned}$$

That is, when **find\_tail**([q]) returns a (non-null) location  $\ell_t$  (which the current thread sees as a tail because  $\ell + \text{next}$  is still 0), it also returns the observation that  $\ell_t$  has been enqueued as some node  $\eta'$  in the snapshots of  $G_t$ ,  $\mathcal{T}_t$ , and  $\mathcal{L}_t$ . The proof of this spec simply open the internal invariant  $\text{queue}(\dots)$  to access the atomic points-to of the tail.

After line 5, we have  $t = \ell_t$  and  $\text{seenLink}^{G_t, \mathcal{T}_t, \mathcal{L}_t}(\_, \_, \eta_t, \ell_t, \_, \_)$ . In line 6, we have a CAS that tries to set  $\ell_t + \text{next}$  from null (0) to  $n$ . To verify this CAS, we use an iRC11 rule with atomic points-to (like **AT-CAS-SN-GEN** in **Figure 10.5**). We therefore need to extract (1) the seen-history observation  $\ell_t \sqsupseteq_{\text{sn}}^{\eta_t} [\_ \leftarrow (0, \_)]$  from  $\ell_t \sqsupseteq_{\text{sy}}^{\eta_t} [\_ \leftarrow (0, \_)]$  in  $\text{seenLink}^{G_t, \mathcal{T}_t, \mathcal{L}_t}(\dots)$ , and (2) the atomic points-to  $\ell_t + \text{next} \mapsto_{\text{con}} \_$  from  $\text{ownQueueLocs}_{\gamma_h, \gamma_t}^{G, \mathcal{T}}(\dots)$  from the queue invariant  $\text{queue}(\gamma, \gamma_g, \gamma_h, \gamma_t)$ .

If the CAS fails, we simply return the points-to to the invariant—we do not update the ghost state instances. Nevertheless, we need to commit the atomic update  $\langle G, vs. \text{Queue}(q, vs, G) \mid b. \dots \Rightarrow \Phi \rangle_{\mathcal{N}_{\text{HIST}}^{\top} \setminus \mathcal{N}_{\text{QUEUE}}^{\top}}$  from our context, with the return value  $b = \mathbf{false}$  and  $G' = G$  and  $M' = M_0$ .

On the other hand, if the CAS succeeds, we have to

- (1) turn the non-atomic points-to of  $n$  into an atomic points-to with a new node id  $\eta$  which is also the atomic points-to's ghost location;
- (2) update the graph to  $G' = G[e \mapsto (\text{Enq}(v), V', M')]$  for some newly picked event id  $e \notin G$  and logical view  $M'$ ;

- (3) map the node id  $\eta$  for the newly enqueued node  $n$  to the current master  $\mathcal{T}$  (from  $\text{master}^{\gamma \cdot 3}(\mathcal{T})$  in the invariant) to  $e$ ;
- (4) append the current master enqueued list  $\mathcal{L}$  (from  $\text{master}^{\gamma \cdot 1}(\mathcal{L})$  in the invariant) with  $(\eta, n)$ ;
- (5) establish the updated observations  $\text{queueViews}(\dots)$  and properties  $\text{queueProps}(\dots)$  for the updated queue;
- (6) establish the queue consistency  $\text{QueueConsistent}(vs ++ [(v, V')], G')$ .

Here, we commit the atomic update with the return value  $b = \mathbf{true}$ . We update of the graph (step (2)) using both halves of the graph master ( $\text{master}_{1/2}^{\gamma g}(G)$ ) from the two pieces of  $\text{Queue}^{\gamma g}(q, vs, G)$ , one provided by the client through our committing of the atomic update and one through our opening of the invariant  $\text{queue}$ .

In step (3), we update  $\text{master}^{\gamma \cdot 3}(\mathcal{T})$  to  $\text{master}^{\gamma \cdot 3}(\mathcal{T}')$  where  $\mathcal{T}' = \mathcal{T}[\eta \mapsto e]$ . In step (4), we update  $\text{master}^{\gamma \cdot 1}(\mathcal{L})$  to  $\text{master}^{\gamma \cdot 1}(\mathcal{L}')$  where  $\mathcal{L}' = \mathcal{L} ++ [(\eta, n)]$ .

By accomplishing step (1), we get  $\text{ownNode}^{G', \mathcal{T}'}(\eta, n, \text{None})$  for the new node  $(\eta, n)$ . Furthermore, to update the previous tail  $(\eta_t, \ell_t)$  to link to the new node, we need to produce  $\text{ownNode}^{G', \mathcal{T}'}(\eta_t, \ell_t, \text{Some}(\eta, n))$ . To do so, we need to show  $@_{V'}\text{seenEnq}(G', e, v)$ , which in turn requires us to show  $\text{syncEnqLogView}(G', M')$ . In other words, we need to maintain **QUEUE-MS-SYNC-ENQS** for the new logical view. For that reason, we pick  $M' = M_0 \cup \{e\} \cup G(e_t).\text{logview}$  where  $\mathcal{T}(\eta_t) = e_t$ . That is, after the successful enqueue, the thread extends its logical view (logical happens-before set) to include not only the new enqueue event  $e$  but also all events from the previous tail  $\eta_t$ . This encodes the fact that, in this release-acquire implementation, enqueues are synchronized with one another.

In a related node, in step (5), to show  $\text{queueViews}(G', \mathcal{T}', \mathcal{L}')$ , we need to show

$$@_{V'}\text{seenLinks}^{G', \mathcal{T}', \mathcal{L}'}(M')$$

For all the nodes in the  $\eta$  part of  $M'$ , we get the observations from the original  $\text{SeenQueue}(q, G_0, M_0)$  assumption we have received at the very beginning. For all the nodes in the  $G(e_t).\text{logview}$  part of  $M'$ , we get the observations the old  $\text{queueViews}(G, \mathcal{T}, \mathcal{L})$  from the invariant. It remains to show

$$@_{V'}\text{seenLink}^{G', \mathcal{T}', \mathcal{L}'}(\eta_t, \ell_t, \eta, n, e, v)$$

which follows from  $@_{V'}\text{seenEnq}(G', e, v)$  that has been shown right above.

In step (5), we also need to establish

$$\text{queueProps}(vs ++ [(v, V')], G', \mathcal{T}', \eta_0, s_0, \mathcal{D}, \mathcal{L}_{\text{in}} ++ [(\eta, n)])$$

The most two important properties are **QUEUE-MS-ENQ-MO** and **QUEUE-MS-ENQ-HB** (see **Definition 23.8**). We achieve **QUEUE-MS-ENQ-MO** by maintaining an event id allocating scheme that follows the relation  $\leq$  among event

ids. We achieve **QUEUE-MS-ENQ-HB** through the way we pick  $G(e).logview$  for a new enqueue event  $e$ : as can be seen from  $M'$ ,  $G(e).logview$  always includes  $e$  itself, and  $G(e).logview$  also includes  $G(e_t).logview$  of the preceding enqueue event (the previous tail)  $e_t$ . Consequently,  $e_t \in G(e).logview$ .

Step (6) is rather straightforward, because the properties in the queue consistency mostly concern dequeues. With that, we achieve the two copies of the queue state  $Queue^{\gamma_g}(q, vs', G')$ , one needed to re-establish our internal invariant, and one needed to commit the atomic update (we have to give back the queue state to the client). This concludes the successful CAS sub-proof in line 6.

In line 7, we update the queue's tail to point to the newly enqueued node  $n$ . We can do this simply by opening the internal invariant `queuel(...)` again to access the atomic points-to of the tail. We also need a local history-seen observation of  $q + tail \sqsubseteq_{sn}^{\gamma_t} \_$ , which we do have from the assumption `SeenQueue`. In this step we do not update the queue's ghost state, so we can easily re-establish the invariant.

Lastly, before returning, we need to construct  $SeenQueue^{\gamma_g}(q, G', M')$ . Fortunately, this step only accumulates all of the observations we have proven previously in this proof.  $\square$

*Proof sketch of **try\_deq**([q]).* The proof of **try\_deq** is very similar to that of **try\_enq**, except that the latter works on the tail, while the former works on the head. In line 1 when reading  $q + head$ , we open the internal invariant `queuel` to access the atomic points-to of  $q + head$  (from `ownQueueLocs`, see Definition 23.6). We also get a local history-seen observation of  $q + head \sqsubseteq_{sn}^{\gamma_h} \_$  from the assumption `SeenQueue`. In this **acq** read, we acquire the observation  $seenLink(\_, \_, \eta_h, h, e_h, v_h)$  for the read sentinel node  $h$ . From that observation we can extract  $h \sqsubseteq_{sy}^{\eta_h} [\_ \leftarrow (0, \_)]$ , which we need for the read of  $h + next$  in line 2 (recall that  $next = 0$ ).

For the read in line 2, again, we need to open the invariant `queuel` to access the atomic points-to of  $h + next$ . We know that the atomic points-to is indeed in the invariant (in `ownNodes`, see Definition 23.4) thanks to the observation  $seenLink(\_, \_, \eta_h, h, e_h, v_h)$  acquired from line 1.

In line 3, we check if the read value  $n$  in line 2 is null (0). If it is indeed the case, the caller thread *subjectively* sees the queue as empty. We then commit the atomic update  $\langle G, vs. Queue(q, vs, G) \mid b. \dots \Rightarrow \Phi \rangle_{\mathcal{N}_{HIST}}^{\top \setminus \mathcal{N}_{QUEUE}}$  from our context, with the return value  $v = \epsilon = 0$  and  $vs' = vs$ , and  $G' = G[d \mapsto (Deq(\epsilon), V', M')]$ . We pick the new  $M' = M_0 \cup M_{e_h}^{prev} \cup \{d\}$ , where

- $d$  is a fresh event id for our empty dequeue event; and
- $M_0$  is the local logical view of the thread when it started the call to **try\_deq**; and
- $M_{e_h}^{prev}$  is the set of enqueue events that are enqueued before  $e_h$ , i.e.,  $M_{e_h}^{prev} = \{e' \in G(e_h).logview \mid G(e').type = \text{Enq}(\_)\}$ .

That is, the empty dequeue  $d$  is synchronized with all enqueues that come before (in  $\mathcal{L}$ ) the sentinel node  $h$  that the dequeue reads from. The

<sup>8</sup>see the pick of  $M'$  in the successful case of the `try_enq` proof

<sup>9</sup>recall that this is the meaning of  $(e, d) \in G.lhb$

synchronization comes from (1) all enqueues are synchronized with one another,<sup>8</sup> and (2) the write to the next field of a node is a **rel** write and the read in line 2 is an **acq** read.

This precise pick of  $M'$  is crucial to re-establish **QUEUE-EMPDEQ-FULL** for  $\text{QueueConsistent}(vs', G')$  (see Figure 23.1): we have to show that for any enqueue event  $e$  in  $G(d).\text{logview}$ <sup>9</sup> is already dequeued and that dequeue  $d'$  also happens before  $d$ . We can prove this property because

- (1) all enqueue events in  $M'$  must have come before  $e_h$  in  $\mathcal{L}$ ; and
- (2)  $e_h$  is the enqueue event of  $(\eta_h, h)$  which has been written to  $q + \text{head}$ , so  $(\eta_h, h)$  must have been dequeued, i.e.,  $(\eta_h, h) \in \mathcal{D}$ ; and
- (3) all dequeues (those in  $\mathcal{D}$ ) are synchronized with one another, due to the use of release-acquire synchronization to  $q + \text{head}$  in `try_deq`.

Other obligations for  $\text{QueueConsistent}(vs', G')$  are rather straightforward, so we elide them and conclude this case here.

If  $n$  is not null, line 6 uses a **CAS** on  $q + \text{head}$  to try to make  $n$  the new head, effectively dequeuing  $n$  from the queue. Here we open the invariant queue again to access the atomic points-to of  $q + \text{head}$ . If the **CAS** fails, someone already dequeued  $n$ , so we commit the atomic update with  $v = \perp = -1$  and without any update to the graph  $G$  or the abstract state  $vs$ , and with  $M' = M_0$ .

If the **CAS** succeeds, we have  $\text{ownNode}^{G, \mathcal{T}}(\eta_h, h, \text{Some}(\eta, n, d))$  of the node  $h$  where  $d = \text{false}$ , i.e.,  $(\eta, n)$  has not been dequeued in  $G$ . Here, we switch  $n$  to be dequeued in  $G' = G[d \mapsto (\text{Deq}(v), V', M')]$ , which means producing  $\text{ownNode}^{G', \mathcal{T}}(\eta_h, h, \text{Some}(\eta, n, \text{true}))$ . This means that we can take  $@_{V'}(n + \text{data} \mapsto \_)$  out of the invariant. As the acquire read in line 2 already gives us  $\sqsubseteq V'$ , later in line 7 we can safely read  $n + \text{data}$  non-atomically.

In the proof of line 6 we move  $(\eta, n)$  from the head of  $\mathcal{L}_{\text{in}}$  to the tail of  $\mathcal{D}$ , i.e.,  $\mathcal{L}_{\text{in}} = [(\eta, n)] \mathbin{++} \mathcal{L}'_{\text{in}}$  and  $\mathcal{D}' = \mathcal{D} \mathbin{++} [(\eta, n)]$ . We also commit the atomic update with  $G'.so = \{(e, d)\} \cup G.so$ , where  $e$  is the enqueue event of  $n$ , and with  $M' = M_0 \cup G(e).\text{logview} \cup \{d\}$ . That is, the dequeue  $d$  is also synchronized with all events that happen before the enqueue  $e$  that  $d$  is synchronized with.

Then, the gist of re-establishing  $\text{QueueConsistent}(vs', G')$  is to prove the FIFO property **QUEUE-FIFO-FULL** for  $G'$ . This can be achieved because we track that all enqueues are synchronized with one another, all dequeues are synchronized with one another, and dequeues are pairwise synchronized with enqueues.

Note that in all cases we have to establish  $\text{SeenQueue}(q, \_, M')$ . We elide these details and conclude the sketch here.  $\square$

**Remark 23.10** (Verification against specs as refinement proofs). We note that, as can be seen from the proof sketches, the verification is effectively showing a refinement between the operations of the  $\text{LAT}_{\text{hb}}^{\text{abs}}$  specs and the implementations: it proves that the abstract state and the graph are *refined* by the set of physical locations used by the implementation. In these proofs, we not only have to show that we maintain the invariant



of the abstract (e.g.,  $\text{QueueConsistent}(vs, G)$ ) as well as the invariant of the concrete (e.g.,  $\text{queue}$ ) but we also have show that every update in the state of the abstract (the commit of the atomic update) is always matched with the corresponding updates of the physical locations of the nodes and the head or the tail.

## 23.2 Stack Specs and Verification of the Treiber Stack

We present the  $\text{LAT}_{\text{hb}}$ -style specs for stacks, that is, those without an abstract state, in [Figure 23.4](#). These specs are very similar to those of queues: we have two basic assertions, the ownership of the stack state  $\text{Stack}(s, G)$ , which is now only a graph, and the local observation  $\text{SeenStack}(s, G_0, M_0)$  on some snapshot  $G_0$  and some logical view  $M_0$ . The stack consistency conditions are given in [Figure 23.5](#), which are also very similar to those of queues.<sup>10</sup> Similar to queues, the stack can have successful push events ( $\text{Push}(v)$ ), successful pop events ( $\text{Pop}(v)$ ), and empty pop events ( $\text{Pop}(\epsilon)$ ).

<sup>10</sup>see [Figure 23.1](#)

We briefly explain the specs of stacks.

- **HB-NEW-STACK** allocates a new stack  $s$  with the empty graph  $\emptyset$ . It returns the initial stack state  $\text{Stack}(s, \emptyset)$  and the initial observation  $\text{SeenStack}^{\mathcal{N}_{\text{STACK}}}(q, \emptyset, \emptyset)$ . The namespace  $\mathcal{N}_{\text{STACK}}$  is picked by the client to host the internal invariants needed by the implementation. Recall that  $\text{Stack}(s, G)$  is objective, so it can be easily put inside invariants. Just as we have seen in the proof of queues, this can be achieved by implementing  $\text{Stack}(s, G)$  with just ghost state.
- **HB-TRY-PUSH** specifies the try version of the **push** function. If the try fails, there is no interesting information about the failure. We note that, for stacks, queues, or any data structure, the “specifier” may also choose to include other events, such as failures due to contention, and specify the conditions on those events accordingly. If the try succeeds, a new push event is added to the current graph  $G$ , with appropriate physical view  $V'$  and logical view  $M'$ .
- **HB-TRY-POP** says that the try version of the **pop** function can return empty ( $\epsilon$ ), or fails due to contention (returning  $\perp$ ). If the try succeeds, it pops a value in the stack, which must have been pushed and has not been popped yet. An appropriate pop event is then added to the current graph  $G$ .
- **HB-PUSH** and **HB-POP** just rule out the failure case (due to contention) for a push and a pop, respectively.
- $\text{StackConsistent}(G)$  is very similar to the consistency conditions of queues: the only difference is the rule **STACK-LIFO** which requires that the push and pop order must follow the last-in-first-out principle, stated using the local happens-before **lhb** relation.

The verification of the Treiber stack implementation in [Figure 12.8](#) against the  $\text{LAT}_{\text{hb}}$  specs is also very similar to that of the Michael-Scott

$$\text{StackEvent} ::= \text{Push}(v) \mid \text{Pop}(v) \mid \text{Pop}(\epsilon)$$

$$\text{Event} ::= \text{StackEvent} \times \text{View} \times \text{LogView}$$

HB-NEW-STACK

$$\{\text{True}\} \text{new\_stack}() \{s. \text{SeenStack}^{\mathcal{N}_{\text{STACK}}}(q, \emptyset, \emptyset) * \text{Stack}(s, \emptyset)\}$$

HB-TRY-PUSH

$$\text{SeenStack}(s, G_0, M_0) * \sqsupseteq V \vdash$$

$$\langle G. \text{Stack}(s, G) \rangle$$

**try\_push**(s, v)

$$\left| \begin{array}{l} b. \exists G' \sqsupseteq G, M' \sqsupseteq M_0, V' \sqsupseteq V. \text{Stack}(s, G') * \text{SeenStack}(s, G', M') * \sqsupseteq V' \\ * \vee \begin{cases} b = \text{false} & \wedge G' = G \\ b = \text{true} & \wedge \exists e \notin G. e \in M' \wedge G' = G[e \mapsto (\text{Push}(v), V', M')] \end{cases} \end{array} \right|_{\mathcal{N}_{\text{STACK}}}$$

HB-TRY-POP

$$\text{SeenStack}(s, G_0, M_0) * \sqsupseteq V \vdash$$

$$\langle G. \text{Stack}(s, G) \rangle$$

**try\_pop**(s)

$$\left| \begin{array}{l} v. \exists G' \sqsupseteq G, M' \sqsupseteq M_0, V' \sqsupseteq V. \text{Stack}(s, G') * \text{SeenStack}(s, G', M') * \sqsupseteq V' \\ * \vee \begin{cases} v = \perp & \wedge G' = G \\ v = \epsilon & \wedge \exists d \notin G. d \in M' \wedge G' = G[d \mapsto (\text{Pop}(\epsilon), V', M')] \\ v \notin \{\perp, \epsilon\} & \wedge \exists e, V_e, M_e, d \notin G. G(e) = (\text{Enq}(v), V_e, M_e) \wedge (e, \_) \notin G.\text{so} \wedge V_e \sqsubseteq V' \\ & \wedge M_e \cup \{e, d\} \sqsubseteq M' \wedge G' = G[d \mapsto (\text{Pop}(v), V', M')] \wedge G'.\text{so} = \{(e, d)\} \cup G.\text{so} \end{cases} \end{array} \right|_{\mathcal{N}_{\text{STACK}}}$$

HB-PUSH

$$\text{SeenStack}(s, G_0, M_0) * \sqsupseteq V \vdash$$

$$\langle G. \text{Stack}(s, G) \rangle$$

**push**(s, v)

$$\left| \begin{array}{l} (). \exists G' \sqsupseteq G, M' \sqsupseteq M_0, V' \sqsupseteq V. \text{Stack}(s, G') * \text{SeenStack}(s, G', M') * \sqsupseteq V' \\ * \exists e \notin G. e \in M' \wedge G' = G[e \mapsto (\text{Push}(v), V', M')] \end{array} \right|_{\mathcal{N}_{\text{STACK}}}$$

HB-POP

$$\text{SeenStack}(s, G_0, M_0) * \sqsupseteq V \vdash$$

$$\langle G. \text{Stack}(s, G) \rangle$$

**pop**(s)

$$\left| \begin{array}{l} v. \exists G' \sqsupseteq G, M' \sqsupseteq M_0, V' \sqsupseteq V. \text{Stack}(s, G') * \text{SeenStack}(s, G', M') * \sqsupseteq V' \\ * \vee \begin{cases} v = \epsilon & \wedge \exists d \notin G. d \in M' \wedge G' = G[d \mapsto (\text{Pop}(\epsilon), V', M')] \\ v \notin \{\perp, \epsilon\} & \wedge \exists e, V_e, M_e, d \notin G. G(e) = (\text{Enq}(v), V_e, M_e) \wedge (e, \_) \notin G.\text{so} \wedge V_e \sqsubseteq V' \\ & \wedge M_e \cup \{e, d\} \sqsubseteq M' \wedge G' = G[d \mapsto (\text{Pop}(v), V', M')] \wedge G'.\text{so} = \{(e, d)\} \cup G.\text{so} \end{cases} \end{array} \right|_{\mathcal{N}_{\text{STACK}}}$$

HB-STK-OBJ

$$\text{objective}(\text{Stack}(s, G))$$

FIGURE 23.4: LAT<sub>hb</sub> specs for stack

$$\begin{array}{c}
\text{HB-STACK-CONSISTENCY} \\
\text{Stack}(s, G) \vdash \text{StackConsistent}(G) \\
\\
\text{StackConsistent}(G) := \\
\wedge \left\{ \begin{array}{ll}
\forall (e, d) \in G.\text{so}. \exists v. G(e).\text{type} = \text{Push}(v) \wedge G(d).\text{type} = \text{Pop}(v) & (\text{STACK-MATCHES}) \\
\wedge G(e).\text{view} \sqsubseteq G(d).\text{view} \wedge G(e).\text{logview} \sqsubseteq G(d).\text{logview} & (\text{STACK-SO-FUNCTIONAL}) \\
G.\text{so} \text{ and } G.\text{so}^{-1} \text{ are functional.} & \\
\forall d. G(d).\text{type} = \text{Pop}(\_) \rightarrow (\_, d) \notin G.\text{so} \rightarrow G(d).\text{type} = \text{Pop}(\epsilon) & (\text{STACK-UNMATCHED-EMPPop}) \\
\forall (e, d), (e', d') \in G.\text{so}, (e, e') \in G.\text{lhs} \rightarrow (d, d') \in G.\text{lhs} \rightarrow (e', d) \notin G.\text{lhs} & (\text{STACK-LIFO}) \\
\forall d, e. G(d).\text{type} = \text{Pop}(\epsilon) \rightarrow G(e).\text{type} = \text{Push}(\_) \rightarrow (e, \_) \notin G.\text{so} \rightarrow (e, d) \notin G.\text{lhs} & (\text{STACK-EMPPop})
\end{array} \right.
\end{array}$$

FIGURE 23.5: LAT<sub>hb</sub> Stack Consistency

queue (against the LAT<sub>hb</sub><sup>abs</sup> specs). In fact, the proof is simpler because we only work with the head of the stack, while for the queue, we have to coordinate between the head and the tail. As we already presented another proof of the Treiber stack against the simpler bag specs in §12.4.3, we elide the proof here. We note that the LAT<sub>hb</sub> specs presented here imply the bag specs in Figure 12.9.

**CHAPTER SUMMARY.** In this chapter we have demonstrated concrete Compass specs and verifications through queues and stacks, which, unsurprisingly, show the extra proof obligations required by the stronger logically-atomic specs.



# 24

## Exchangers and the Elimination Stack

In this chapter, we demonstrate the application of our specs to verify an RMC implementation of the elimination stack.<sup>1</sup> We show that the RMC elimination stack satisfies the  $\text{LAT}_{\text{hb}}$  specs for stacks given in §23.2. This verification is both a *client* verification and a *library* verification: the elimination stack is a client that composes an underlying base stack and an *exchanger*. In §24.1 we give an overview of the RMC implementation for the elimination stack. In §24.2 we present the strong specs with *helping* for exchangers. In §24.3 we sketch the verification of the elimination stack as a simulation proof using the specs of the base stack and the exchanger.

<sup>1</sup>Hendler et al., “A scalable lock-free stack algorithm” [HSY04].

### 24.1 The Elimination Stack

The idea for the elimination stack (ES) comes from a simple observation: if a **push** is immediately followed by a **pop**, then the stack appears unchanged, and that **push** and **pop** are said to *eliminate* each other. The elimination mechanism can be implemented with an exchanger (which in turn can be implemented as an array of exchangers) that supports concurrent exchanges of data with arbitrary matching. A thread simply calls **exchange**( $x, v_1$ ) on the exchanger object  $x$  with some value  $v_1 \neq \perp$ . If the return value is  $\perp$ , the exchange has failed, but if it is some  $v_2 \neq \perp$ , then the thread has successfully exchanged  $v_1$  for  $v_2$  with another thread. Additionally, the two threads *synchronize with each other*,<sup>2</sup> which from the separation logic perspective supports *resource exchanges* between the matching threads.

The ES *try* operations, which can fail due to contention, can be implemented simply by composing the two libraries without any extra synchronization, as follows:

```
try_push(s, v) ::= if try_push'(s.base, v) then true
                  else exchange(s.ex, v) == SENTINEL
try_pop(s) ::= let v = try_pop'(s.base) in
               if v != FAIL_RACE then v else
               let v' = exchange(s.ex, SENTINEL) in
               if v' ∉ {SENTINEL, ⊥} then v'
               else FAIL_RACE
```

<sup>2</sup>Technically, the two commit points of the matching exchanges are not both in **hb** with each other—it is counterintuitive to have cycles in **hb**—but it is the case that the *beginning* of one **exchange** call happens before the *end* of its matching **exchange** call. We needed to extend our specs to account for this subtlety, but, due to space constraints, we elide it from the specs and from the discussion here.

Each operation first tries the base stack's corresponding operation, and if that fails due to contention, it tries to use the exchanger to match another operation without going through the base stack. More specifically, `try_push(s, v)` calls the base stack's own `try_push'` and returns `true` (signifying success) if that succeeds. Otherwise, it calls `exchange` (on `s.ex`) and returns `true` only if its exchange is successfully matched with a pop operation, signified by the SENTINEL value. Similarly, `try_pop(s)` calls the base stack's `try_pop'` and returns `v` only if `try_pop'` did not fail due to contention (FAIL\_RACE). (`try_pop` returns empty `ε` if `try_pop'` does.) Otherwise, `try_pop` calls `exchange` with SENTINEL, and only succeeds with the returned value `v'` if it is matched ( $v' \neq \perp$ ) with a push ( $v' \neq \text{SENTINEL}$ ).

## 24.2 A Strong Spec for Exchangers

An exchanger event carries an additional physical view (`view_in`) for recording the *input view* of the event. It is used for describing the mutual synchronization of matching exchange events (EXCHANGER-MATCHES).

A simplified LAT<sub>hb</sub>-style spec for the `exchange` function is shown in HB-EXCHANGE (Figure 24.1). The spec involves a local logical view assertion `SeenExchanges(x, G0, M0)`, and an atomically shared ownership assertion `Exchanger(x, G)` for the exchanger object `x`. At the commit point, the current graph `G` is extended with a new event `e1` with type `Exchange(v1, v2)`, where `v2` is the returned value. If the exchange fails, the return value `v2` is  $\perp$  and the event type is `Exchange(v1,  $\perp$ )`. If the exchange succeeds, it can only succeed together with another exchange identified by `e2`, and the `G.so` relation is extended with the two events in both directions ( $\{(e_1, e_2), (e_2, e_1)\}$ ), signifying that they are synchronized with each other.

The remaining part of the spec is to maintain the perspective that a *matching pair of exchanges is committed atomically together*: it is important that there can be *no interference* between the two commits of the matching exchanges. In other words, no other thread should be able to observe an incomplete state of the exchanger where one successful exchange has been committed but its matching exchange has not. But how can two commit points be atomic? This conflicts with the intuitive interpretation of LATs that there exists a committing instruction *I* within each logical operation! To resolve this conundrum, we need *helping*.

**HELPING FOR ATOMICITY** Helping is a pattern where one operation—the helper—helps to perform the commit (the update to the shared state) of another operation—the helpee. This means that the commit point of the helpee is not within its own execution, but rather within the helper's execution. For the matching exchange pairs, the commit points coincide: at the helper exchange's commit point, it atomically performs the helpee exchange's commit and then its own commit. This is materialized in the successful case of HB-EXCHANGE with (1) a *commit order* ( $<$ ) of the events and (2) the addition of a *local postcondition* (in red,  $\{\dots\}$ ) that only holds once the function returns (rather than at the commit point).

$$\begin{aligned}
& \text{ExchangeEvent} ::= \text{Exchange}(v_1, v_2) \\
& \text{Event} ::= \text{ExchangeEvent} \times \text{View} \times \text{View} \times \text{LogView} \\
& \text{HB-NEW-EXCHANGER} \\
& \{ \text{True} \} \text{new\_exchanger}() \{ x. \text{SeenExchanges}^{\mathcal{N}_{\text{XCHG}}}(x, \emptyset, \emptyset) * \text{Exchanger}(x, \emptyset) \} \\
& \text{HB-EXCHANGE} \\
& \text{SeenExchanges}(x, G_0, M_0) * v_1 \neq \perp * \exists V_0 \vdash \\
& \langle b, G. \text{Exchanger}(x, G) \rangle \\
& \text{exchange}(x, v_1) \\
& \left[ \begin{array}{l}
v_2. \exists b', G' \sqsupseteq G, M' \sqsupseteq M_0, V_1 \sqsupseteq V_0, V_{21}, V_{22}, e_1 \notin G, e_2. \\
\text{Exchanger}(x, G') * \exists V_1 * e_1 \in M' * G' = G[e_1 \mapsto (\text{Exchange}(v_1, v_2), V_0, V_1, M')] \\
* \vee \left\{ \begin{array}{l}
v_2 = \perp \quad * \text{SeenExchanges}(x, G', M') * \text{ExchangerConsistent}(G') \\
v_2 \neq \perp \wedge e_1 < e_2 \quad * e_2 \in M' * \text{SeenExchanges}(x, G', M' \setminus \{e_2\}) \\
v_2 \neq \perp \wedge e_2 < e_1 \quad * e_2 \in M' * \text{SeenExchanges}(x, G', M') * \text{ExchangerConsistent}(G')
\end{array} \right. \\
* G(e_2) = (\text{Exchange}(v_2, v_1), V_{21}, V_{22}, M') * G'.\text{so} = \{(e_1, e_2), (e_2, e_1)\} \cup G.\text{so}
\end{array} \right]_{\mathcal{N}_{\text{XCHG}}} \\
& \left\{ \begin{array}{l}
(v_2 \neq \perp \wedge e_1 < e_2) \implies \exists G'' \sqsupseteq G'. \text{SeenExchanges}(x, G'', M') * \text{ExchangerConsistent}(G'') \\
* G''(e_2) = (\text{Exchange}(v_2, v_1), V_{21}, V_{22}, M') * G''.\text{so} = \{(e_1, e_2), (e_2, e_1)\} \cup G'.\text{so}
\end{array} \right\} \\
& \text{HB-EXCHANGER-CONSISTENCY} \\
& \text{Exchanger}(x, G) \multimap \text{SeenExchanges}(x, G', M') \Rightarrow_{\mathcal{N}_{\text{XCHG}}} \text{Exchanger}(x, G) * \text{ExchangerConsistent}(G) \\
& \text{ExchangerConsistent}(G) := \\
& \left\{ \begin{array}{l}
\forall (e_1, e_2) \in G.\text{so}. |e_1 - e_2| = 1 \\
\wedge \exists v_1 v_2. G(e_1).\text{type} = \text{Exchange}(v_1, v_2) \wedge G(e_2).\text{type} = \text{Exchange}(v_2, v_1) \\
\wedge G(e_1).\text{view\_in} \sqsubseteq G(e_2).\text{view} \wedge G(e_2).\text{view\_in} \sqsubseteq G(e_1).\text{view} \\
\text{G.so is symmetric and irreflexive.} \\
\text{G.so and G.so}^{-1} \text{ are functional.} \\
\forall e. (e, \_) \notin G.\text{so} \rightarrow G(e).\text{type} = \text{Exchange}(\_, \perp)
\end{array} \right. \quad \begin{array}{l}
(\text{EXCHANGER-MATCHES}) \\
(\text{EXCHANGER-SO-SYM-IRREFL}) \\
(\text{EXCHANGER-SO-FUNCTIONAL}) \\
(\text{EXCHANGER-UNMATCHED})
\end{array}
\end{aligned}$$

FIGURE 24.1:  $\text{LAT}_{\text{hb}}$  specs for exchangers (excerpt, simplified).

The commit order  $<$  on events is the logical order in which the events are committed to the shared graph  $G$ . In **HB-EXCHANGE**, the commit order between a matching exchange pair dictates who the helper is, and how each commit updates the shared graph  $G$ . If the *current* exchange  $e_1$  is committed before the *other* exchange  $e_2$ , i.e.,  $e_1 < e_2$ , then  $e_2$  is the helper. Otherwise, if  $e_2 < e_1$ , then  $e_1$  is the helper.

Since the helper atomically performs the helpee's update and then its own update, it always knows the result of the helpee's update, while the helpee will only learn about the helper's update *after* both commits have been completed. This is the asymmetry in **HB-EXCHANGE**: if  $e_1$  is the helpee, it only adds itself to the current graph  $G$ :  $G' = G[e_1 \mapsto (\text{Exchange}(v_1, v_2), V_1, M')]$ ; but if  $e_1$  is the helper, it knows that the helpee's event  $e_2$  must already be in the current graph:  $G(e_2) = (\text{Exchange}(v_2, v_1), V_2, M')$ , and the helper not only adds itself to the current graph  $G$ , but also extends  $G$ .so with the pairs  $\{(e_1, e_2), (e_2, e_1)\}$ . The client thread of the helper learns all of this information about the updated  $G'$  atomically right after the helper's commit, by which point it has also locally observed both  $e_1$  and  $e_2$ , via  $\text{SeenExchanges}(x, G', M')$  and  $\{e_1, e_2\} \subseteq M'$ . The client thread of the helpee, on the other hand, right after its own commit has only locally observed its own event  $e_1$ , via  $\text{SeenExchanges}(x, G', M' \setminus \{e_2\})$ , because the helper commit has not been performed yet and  $e_2$  has not been added to  $G'$ . Only in the local postcondition (in  $\{\dots\}$ ), after both commits have been performed, can the helpee learn about the new graph  $G''$  ( $e_2$ 's  $G'$ ) that completes  $e_1$ 's  $G'$  ( $e_2$ 's  $G$ ) with  $e_2$  and the so pairs, and locally observe both events, via  $\text{SeenExchanges}(x, G'', M')$ .

**INTERMEDIATE STATES** That matching exchange pairs are committed atomically together is also reflected by the fact that we do not always have consistency: the ownership  $\text{Exchanger}(x, G)$  does *not* imply  $\text{ExchangerConsistent}(G)$ . Instead, we have  $\text{ExchangerConsistent}(G')$  only with a completed graph  $G'$ , i.e., after the failure case or after the helper's commit. Between the helpee's commit and the helper's commit, the exchanger is in an incomplete intermediate state.

As such, those intermediate states can appear in a client invariant. However, it is important that the client needs to handle such states *only when* it uses the exchanger, and that other non-exchanger-related operations will never observe those states. For example, the invariant of the elimination stack needs to consider the intermediate state where a push event created by a successful exchange is inserted into the graph, but the matching pop event by the matching exchange is not. A successful push using the base stack and running concurrently with the exchange pair should *not* observe the client invariant in such an intermediate state, because it would not be able to prove LIFO then.

Our full exchanger spec (in Coq) supports this form of intermediate state reasoning: when using the exchanger, the client need not maintain its invariant for the intermediate state between the two commits; it only needs to re-establish its invariant after both commits. When *not* using the exchanger, the client invariant is never in such intermediate states.



**STRENGTH OF THE SPECS** To the best of our knowledge, the full exchanger spec is the first ever proposed CSL spec for RMC exchangers. It is strong enough for the proof of the elimination stack (§24.1), and we have also used it to derive a spec that supports resource exchanges, where each **exchange** call needs to provide the resources to be exchanged only at its commit point, and only if the exchange succeeds.

### 24.3 Verifying the Elimination Stack

**VERIFICATION RESULTS.** Assuming the  $\text{LAT}_{\text{hb}}$ -style specs for the base stack and the exchanger (Figure 24.1), we have verified that our relaxed ES implementation satisfies the same  $\text{LAT}_{\text{hb}}$  specs as the base stack. Since we have verified, *separately*, that our RMC Treiber stack and exchanger implementations satisfy their  $\text{LAT}_{\text{hb}}$  specs, we easily get a closed-proof verification of an ES built from those two implementations. We assumed the  $\text{LAT}_{\text{hb}}$  specs for the base stack to demonstrate that the verification does not rely on very strong properties, but, since the proofs are modular enough, we conjecture that the same proofs can be applied (with minor modifications) to show that, if the base stack satisfies the stronger  $\text{LAT}_{\text{hb}}^{\text{abs}}$  specs, then the ES implementation also satisfies the same stronger specs.

**COMPOSITIONAL VERIFICATION.** The ES verification does not involve RMC reasoning, because the implementation does not add any new atomic instructions. The core work is composing the base stack's events and the exchanger's events into the ES events in a way that satisfies the consistency conditions for stacks. This can be seen as a simulation proof with a simple simulation relation: every base stack operation is simulated by a corresponding ES operation, and successful matching exchange pairs between a non-SENTINEL value and a SENTINEL value are simulated by an ES push and an ES pop respectively. (Other exchange events are ignored by the simulation.)

The non-trivial parts of the proof are where the simulation needs to simulate commit points and maintain consistency: whenever a base operation commits, the ES operation needs to commit accordingly, and needs to re-establish the ES consistency conditions using the consistency conditions of the base stack and the exchanger. The re-establishment of consistency relies crucially on the fact that eliminations are *atomic*: the commits of ES push and pop events that originate from a pair of matching exchanges need to be performed together at once, so that the pushed element is popped immediately, and no (commit points of) other concurrent ES operations can observe the intermediate state where the ES push has already been committed but the ES pop has not. This atomicity property of the exchange-based ES event pairs is crucially needed for LIFO.



# 25

## Related Work

---

Our specification styles build on extensive prior work in relaxed correctness conditions, and in program logics for fine-grained concurrent SC and RMC programs.

**RELAXED CORRECTNESS CONDITIONS** Various alternative correctness conditions to linearizability have been developed,<sup>1</sup> particularly for distributed systems<sup>2</sup> and relaxed memory.<sup>3</sup> Most of these were developed outside a program logic, directly on complex low-level concurrency semantics, and with little support for client reasoning or mechanization. As discussed in §1.3, we believe the Yacovet approach<sup>4</sup> is the most general of these. By enhancing Yacovet specs in Compass with logical atomicity, we demonstrate that existing relaxed correctness conditions can be used in combination with separation logic to achieve stronger and better modular client reasoning as well as more foundational (mechanized) verifications. We consider it future work to encode more of these relaxed correctness conditions in Compass.

Recently, Singh and Lahav<sup>5</sup> use abstract programs as specifications for RMC libraries, and define the correctness condition as contextual refinement<sup>6</sup> between the implementation and the specification programs. To be sound, the refinement is defined as trace refinement between histories (event graphs) who additionally track the propagation of method calls and returns among thread (propagation of observations of library actions, similar to that of Compass’ logical views). It is interesting to encode such a correctness condition in CSL, which corresponds to showing contextual refinements for RMC programs in separation logic.

**SC PROGRAM LOGICS** Logical atomicity is just one CSL alternative to linearizability. Another is to avoid identifying commit points and instead reason directly about *refinements* between a sequential “specification” program and the concurrent implementation program.<sup>7</sup> However, sequential specs are not always suitable as correctness conditions (e.g., for exchangers), and *non-sequential* refinement is still an open problem for RMC logics. Our work demonstrates the usefulness of logical atomicity in RMC. As future work, one can consider adapting *prophecy variables*<sup>8</sup> to our framework, as they may help simplify our specs.

FCSL<sup>9</sup> and the rely-guarantee-based Hoare logics by Hemed et al.<sup>10</sup> and Khyzha et al.<sup>11</sup> support specifying non-linearizable SC data structures

<sup>1</sup>[Hen+13; JR14; Der+14; Haa+16; ON22].

<sup>2</sup>[Nei94; AKY10; Bur+14; CRR15].

<sup>3</sup>[Bur+12; BDG13; Jag+13; Doh+18; Don+18; Raa+19; EE19; Kri+20].

<sup>4</sup>Raad et al., “On library correctness under weak memory consistency: specifying and verifying concurrent libraries under declarative consistency models” [Raa+19].

<sup>5</sup>Singh and Lahav, “An Operational Approach to Library Abstraction under Relaxed Memory Concurrency” [SL23].

<sup>6</sup>where the context must adhere to the library’s calling policy

<sup>7</sup>[LF13; TDB13; FKB18; KSB17].

<sup>8</sup>[AL88; AL91; Jun+20].

<sup>9</sup>[SNB15; Ser+16; Del+17].

<sup>10</sup>[HRV15].

<sup>11</sup>[Khy+17].

with histories by encoding histories as auxiliary (ghost) state and by exposing partial, subjective views of the histories to clients. This is similar to our construction of graphs or linear histories. Compass can be seen as extending these logics with logical atomicity and RMC.

<sup>12</sup>[DD21b; DD21a].

<sup>13</sup>Dalvandi et al., “Owicki-Gries Reasoning for C11 RAR” [Dal+20].

**RMC PROGRAM LOGICS** Dalvandi and Dongol,<sup>12</sup> in parallel work, try to achieve the same goal of providing compositional specs and modular client reasoning for RMC data structures. Their approach uses an Owicki-Gries-style Hoare logic<sup>13</sup> for a more limited fragment of RC11 called RAR (which only has release-acquire and relaxed accesses, not non-atomics or fences). They specify libraries with view-based, atomic *abstract object semantics* for the library’s operations, treating the object methods as primitives of the language. Client verifications rely on Hoare-triple specs derived directly from the abstract object semantics. To verify an implementation against a spec, they prove refinement, showing that synchronizations (in the view semantics) of the abstract object are simulated by synchronizations in the implementation. Their approach therefore shares similar ideas to ours. The main limitation is in their simulation method: it applies only to *synchronization-free* clients, *i.e.*, those who synchronize only through the library in question. This is because it is non-trivial to characterize how external synchronizations affect the simulation relation. Consequently, they cannot obtain an end-to-end proof for clients that use external synchronizations, *e.g.*, the MP client in Figure 21.2. Furthermore, the use of Owicki-Gries-style logic means that they have to deal with additional *interference freedom* proofs. They report only one mechanized library verification, for the Treiber stack, with 12KLOC in Isabelle. In comparison, our mechanization results are more extensive, and our Treiber stack verification takes only 2.2KLOC in Iris, in Coq.

<sup>14</sup>[Kai+17; Dan+20a; MJP20].

Several CSLs for RMC have been developed within Iris.<sup>14</sup> Compass extends iRC11 and follows Cosmo in exposing more view information in specs. Our key innovation is the use of logical views on library operations, allowing us to give stronger specifications that can describe interactions with external synchronization. In retrospect, we believe that views are a concise, compositional, and user-friendly tool to describe the different kinds of synchronization that may occur in and around a data structure, and thus are useful for formulating full functional correctness specs under RMC.

<sup>15</sup>Park et al., “A Proof Recipe for Linearizability in Relaxed Memory Separation Logic (to appear in PLDI 2024)” [Par+24].

Very recently, Park et al.<sup>15</sup> improve RMC linearizability proofs with Compass-style specifications by (1) introducing a general *object modification order* (omo) for libraries, (2) introducing a general *commit-with* relation to help with the simulation relation between commit points of the specification and the implementation, and (3) applying proof automation with Diaframe<sup>16</sup> to infer the logical event graph state. The result is that, in some cases, proofs are significantly shortened by an order of magnitude. This demonstrates the extensibility of Diaframe, and signifies the benefits of more general relations as well as automation in scaling RMC verification efforts further—a topic that this thesis did not discuss.

Finally, it would be interesting to apply the Compass approach to

<sup>16</sup>Mulder et al., “Diaframe: automated verification of fine-grained concurrent programs in Iris” [MKG22]; Mulder and Krebbers, “Proof Automation for Linearizability in Separation Logic” [MK23].

more sophisticated RMC libraries such as work-stealing queues<sup>17</sup> and safe memory reclamation schemes for lock-free data structures.<sup>18</sup>

<sup>17</sup>Chase and Lev, “Dynamic circular work-stealing deque” [CL05]; Lê et al., “Correct and efficient work-stealing for weak memory models” [Lê+13].

<sup>18</sup>Michael, “Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects” [Mic04]; Fraser, “Practical lock-freedom” [Fra04]; Jung et al., “Modular Verification of Safe Memory Reclamation in Concurrent Separation Logic” [Jun+23].



# 26

## Conclusion

---

This thesis tries to sketch a recipe to formally build and apply separation logics for verifying safety and functional correctness of programs with relaxed memory concurrency (RMC). Undoubtedly, building and applying such logics require substantial efforts, and are composed of several steps, which can partly be seen from the thesis structure in [Figure 1.1](#). These steps can be grouped into the following 4 general tasks.

- (1) selecting a trusted semantic model of RMC, combined with other language features;
- (2) constructing a low-level logic that provides core separation reasoning principles;
- (3) deriving high-level reasoning principles that may be application-specific;
- (4) applying the high-level logic to the target programs.

**SELECTING A SEMANTIC MODEL OF RMC.** Multiple RMC semantic models have been proposed, in both axiomatic and operational style. We proposed to use an operational version of RC11, called ORC11, in [Chapter 3](#), where we also include non-atomics whose semantics can invoke undefined behaviors (UBs). Triggering UBs is enforced by a race detector on non-atomics. In [Chapter 4](#), we incorporated the ORC11 memory model with the semantics of  $\lambda_{\text{Rust}}$ , which has other pure expressions, and which also includes a conservative pointer comparison scheme. The pure fragment of language can trigger more UBs (such as incompatible comparison).

The integration of a race detector and a conservative pointer comparison scheme shows a serious challenge in formal verifications with realistic languages. Modern languages have become so complicated that individual research works on formal semantics have mostly been contended with only *fragments* of a language. It is unclear if we can keep the semantics of those fragments modularly unentangled, or how much work would be needed to compose their semantics so that we can consider programs that exploit multiple language features all at once. In [Chapter 4](#), we saw that the interaction of our conservative pointer comparison scheme and the RMC semantics resulted in a rather global semantic of **CAS** (RMW) operations, whose complication also bubbles up into the logics.

We have chosen to use the operational-style semantics of ORC11, and in §3.6 only provided an on-paper correspondence proof between RC11 and  $\text{ORC11} + \lambda_{\text{Rust}}$ , partly because the resulting semantics encompasses more features and more UB-triggering behaviors. But the main driver of such decision is that it is easier to build separation logics with ORC11 (as well as other views-based semantics), because views restructure the semantic in a more modular way (an effect focuses on a thread’s view) and therefore are easier to “separated”. Nevertheless, it should still be possible to build separation logics in Iris using an operational model that is much closer to the axiomatic model than ORC11. We consider it future work to build separation logics in Iris directly on the Operational Graph Semantics (OGS) mentioned in §3.6, which means putting a substantial part of the correspondence proof inside Iris, and thus cutting down the TCB.

<sup>1</sup>Lee et al., “Putting Weak Memory in Order via a Promising Intermediate Representation” [Lee+23].

<sup>2</sup>see also Remark 2.21

Finally, very recent work by Lee et al.<sup>1</sup> suggests that one could forbid load-buffering (LB), and therefore the out-of-thin-air (OOTA) problem, at least in the source language memory model.<sup>2</sup> That would mean that ORC11 would be more suitable to model C and C++ RMC programs.

CONSTRUCTING A CORE LOGIC. With an operational RMC model, we then can start constructing the core separation logic iRC11 for our RMC semantics. In Chapter 7, we demonstrated how to use Iris to acquire a base logic that provides basic separation on the RMC states. Most importantly, we introduced thread-local assertions such as the seen thread-view observation  $\text{Seen}(\mathcal{V})$  to account for views, the *thread-local state* of our RMC semantics. Then, in Chapter 8, we introduced a way to hide views in the logic, because most of the time when performing *intra*-thread reasoning, we do not need to care about views. The interaction with views only becomes interesting when we have to reason about synchronizations between threads, *i.e.*, when we need *inter*-thread reasoning. For inter-thread reasoning, we developed several modalities that allow us to temporarily expose views again, which we can then use with general invariants (Chapter 11) to establish inter-thread communications or resource transfer. We note that this approach is inspired by the Cosmo logic,<sup>3</sup> and is applicable not only to views-based semantics,<sup>4</sup> but can also be generalized to handle other *thread-local states*, such as call stacks or physical-virtual address translations.

<sup>3</sup>Mével et al., “Cosmo: a concurrent separation logic for multicore OCaml” [MJP20].

<sup>4</sup>for example, see a separation logic for non-volatile memory by Vindum and Birkedal ([VB23]).

Finally, respectively in Chapter 9 and Chapter 10, we have developed abstractions for non-atomic and atomic accesses. The atomic points-to is general enough to support most usage patterns of atomics, and is also convertible to non-atomic points-to. This setup therefore should be applicable to stronger memory models. The key feature of the atomic points-to is that it interacts well with general invariants (with explicit views), which makes it easier to state protocols spanning multiple locations.

DERIVING HIGH-LEVEL REASONING PRINCIPLES. With a solid set of fundamental low-level abstractions, we started building higher-level abstractions that can be application-specific. In Chapter 16, we have integrated



the SC model of the lifetime logic with our new abstractions to make the lifetime logic sound in RMC. Most RMC effects can be hidden, except for atomic borrows, where we have interactions with explicit views, in the very same fashion as in general invariants. In [Chapter 17](#), we have combined atomic points-to with general invariants to reconstruct GPS protocols in iRC11. In [Chapter 22](#), we have combined atomic points-to, general invariants, and logical atomicity to specify and verify strong functional correctness for RMC libraries.

APPLYING THE HIGH-LEVEL LOGIC TO VERIFICATIONS. Ultimately, the designs of low-level and high-level reasoning principles were driven by the goal to verify strong specifications of concrete, realistic RMC programs. In [Chapter 12](#), we have demonstrated several verifications using the core logic of iRC11. In [Chapter 18](#) and [Chapter 19](#), we have demonstrated the semantic type-checking of [RwLock](#) and [Arc](#) using the lifetime logic and GPS protocols. In [Chapter 23](#) and [Chapter 24](#), we have demonstrated the verifications of stacks, queues, and exchangers against strong Compass specifications. The various applications together have demonstrated the generality and extensibility of our separation logic iRC11.



## Bibliography

---

- [AL88] Martín Abadi and Leslie Lamport. “The Existence of Refinement Mappings”. In: *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS '88)*, Edinburgh, Scotland, UK, July 5-8, 1988. IEEE Computer Society, 1988, pp. 165–175. DOI: [10.1109/LICS.1988.5115](https://doi.org/10.1109/LICS.1988.5115). URL: <https://doi.org/10.1109/LICS.1988.5115> (cit. on p. 297).
- [AL91] Martín Abadi and Leslie Lamport. “The Existence of Refinement Mappings”. In: *Theor. Comput. Sci.* 82.2 (1991), pp. 253–284. DOI: [10.1016/0304-3975\(91\)90224-P](https://doi.org/10.1016/0304-3975(91)90224-P). URL: [https://doi.org/10.1016/0304-3975\(91\)90224-P](https://doi.org/10.1016/0304-3975(91)90224-P) (cit. on p. 297).
- [AKY10] Yehuda Afek, Guy Korland, and Eitan Yanovsky. “Quasi-Linearizability: Relaxed Consistency for Improved Concurrency”. In: *Principles of Distributed Systems - 14th International Conference, OPODIS 2010, Tozeur, Tunisia, December 14-17, 2010. Proceedings*. Vol. 6490. Lecture Notes in Computer Science. Springer, 2010, pp. 395–410. DOI: [10.1007/978-3-642-17653-1\\_29](https://doi.org/10.1007/978-3-642-17653-1_29). URL: [https://doi.org/10.1007/978-3-642-17653-1\\_29](https://doi.org/10.1007/978-3-642-17653-1_29) (cit. on pp. 5, 297).
- [Ahm+10] Amal Ahmed, Andrew W. Appel, Christopher D. Richards, Kedar N. Swadi, Gang Tan, and Daniel C. Wang. “Semantic foundations for typed assembly languages”. In: *ACM Trans. Program. Lang. Syst.* 32.3 (2010), 7:1–7:67. DOI: [10.1145/1709093.1709094](https://doi.org/10.1145/1709093.1709094). URL: <https://doi.org/10.1145/1709093.1709094> (cit. on p. 173).
- [BDG13] Mark Batty, Mike Dodds, and Alexey Gotsman. “Library abstraction for C/C++ concurrency”. In: *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*. ACM, 2013, pp. 235–248. DOI: [10.1145/2429069.2429099](https://doi.org/10.1145/2429069.2429099). URL: <https://doi.org/10.1145/2429069.2429099> (cit. on pp. 5, 297).
- [Bat+11] Mark Batty, Scott Owens, Susmit Sarkar, Peter Sewell, and Tjark Weber. “Mathematizing C++ concurrency”. In: *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2011, Austin, TX, USA, January 26-28, 2011*. ACM, 2011, pp. 55–66. DOI: [10.1145/1926385.1926394](https://doi.org/10.1145/1926385.1926394). URL: <https://doi.org/10.1145/1926385.1926394> (cit. on pp. 2, 13, 15, 23).
- [BP19] John Bender and Jens Palsberg. “A formalization of Java’s concurrent access modes”. In: *Proc. ACM Program. Lang.* 3.OOPSLA (2019), 142:1–142:28. DOI: [10.1145/3360568](https://doi.org/10.1145/3360568). URL: <https://doi.org/10.1145/3360568> (cit. on p. 2).
- [BBW19] Frédéric Besson, Sandrine Blazy, and Pierre Wilke. “CompCertS: A Memory-Aware Verified C Compiler Using a Pointer as Integer Semantics”. In: *J. Autom. Reason.* 63.2 (2019), pp. 369–392. DOI: [10.1007/s10817-018-9496-y](https://doi.org/10.1007/s10817-018-9496-y). URL: <https://doi.org/10.1007/s10817-018-9496-y> (cit. on p. 51).
- [Bir+21] Lars Birkedal, Thomas Dinsdale-Young, Armaël Guéneau, Guilhem Jaber, Kasper Svendsen, and Nikos Tzevelekos. “Theorems for free from separation logic specifications”. In: *Proc. ACM Program. Lang.* 5.ICFP (2021), pp. 1–29. DOI: [10.1145/3473586](https://doi.org/10.1145/3473586). URL: <https://doi.org/10.1145/3473586> (cit. on p. 260).
- [BD14] Hans-Juergen Boehm and Brian Demsky. “Outlawing ghosts: avoiding out-of-thin-air results”. In: *Proceedings of the workshop on Memory Systems Performance and Correctness, MSPC '14, Edinburgh, United Kingdom, June 13, 2014*. Ed. by Jeremy Singer, Milind Kulkarni, and Tim Harris. ACM, 2014, 7:1–7:6. DOI: [10.1145/2618128.2618134](https://doi.org/10.1145/2618128.2618134). URL: <https://doi.org/10.1145/2618128.2618134> (cit. on p. 22).
- [Bor+05] Richard Bornat, Cristiano Calcagno, Peter W. O’Hearn, and Matthew J. Parkinson. “Permission accounting in separation logic”. In: *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2005, Long Beach, California, USA, January 12-14, 2005*. Ed. by Jens Palsberg and Martín Abadi. ACM, 2005, pp. 259–270. DOI: [10.1145/1040305.1040327](https://doi.org/10.1145/1040305.1040327). URL: <https://doi.org/10.1145/1040305.1040327> (cit. on p. 241).
- [Boy03] John Boyland. “Checking interference with fractional permissions”. In: *SAS. LNCS*. 2003. DOI: [10.1007/3-540-44898-5\\_4](https://doi.org/10.1007/3-540-44898-5_4) (cit. on pp. 81, 239).
- [Bro07] Stephen Brookes. “A Semantics for Concurrent Separation Logic”. In: *Theoretical Computer Science* 375.1–3 (2007). DOI: [10.1016/j.tcs.2006.12.034](https://doi.org/10.1016/j.tcs.2006.12.034) (cit. on p. 1).

- [Bur+12] Sebastian Burckhardt, Alexey Gotsman, Madanlal Musuvathi, and Hongseok Yang. “Concurrent Library Correctness on the TSO Memory Model”. In: *Programming Languages and Systems - 21st European Symposium on Programming, ESOP 2012, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2012, Tallinn, Estonia, March 24 - April 1, 2012. Proceedings*. 2012, pp. 87–107. DOI: [10.1007/978-3-642-28869-2\\_5](https://doi.org/10.1007/978-3-642-28869-2_5). URL: [https://doi.org/10.1007/978-3-642-28869-2\\_5](https://doi.org/10.1007/978-3-642-28869-2_5) (cit. on pp. 5, 297).
- [Bur+14] Sebastian Burckhardt, Alexey Gotsman, Hongseok Yang, and Marek Zawirski. “Replicated data types: specification, verification, optimality”. In: *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’14, San Diego, CA, USA, January 20-21, 2014*. ACM, 2014, pp. 271–284. DOI: [10.1145/2535838.2535848](https://doi.org/10.1145/2535838.2535848). URL: <https://doi.org/10.1145/2535838.2535848> (cit. on pp. 5, 297).
- [CRR15] Armando Castañeda, Sergio Rajsbaum, and Michel Raynal. “Specifying Concurrent Problems: Beyond Linearizability and up to Tasks - (Extended Abstract)”. In: *Distributed Computing - 29th International Symposium, DISC 2015, Tokyo, Japan, October 7-9, 2015, Proceedings*. Vol. 9363. Lecture Notes in Computer Science. Springer, 2015, pp. 420–435. DOI: [10.1007/978-3-662-48653-5\\_28](https://doi.org/10.1007/978-3-662-48653-5_28). URL: [https://doi.org/10.1007/978-3-662-48653-5\\_28](https://doi.org/10.1007/978-3-662-48653-5_28) (cit. on pp. 5, 297).
- [Cha+21] Tej Chajed, Joseph Tassarotti, Mark Theng, Ralf Jung, M. Frans Kaashoek, and Nickolai Zeldovich. “GoJournal: a verified, concurrent, crash-safe journaling system”. In: *OSDI*. USENIX Association, 2021, pp. 423–439 (cit. on p. 1).
- [CV19] Soham Chakraborty and Viktor Vafeiadis. “Grounding thin-air reads with event structures”. In: *Proc. ACM Program. Lang.* 3.POPL (2019), 70:1–70:28. DOI: [10.1145/3290383](https://doi.org/10.1145/3290383). URL: <https://doi.org/10.1145/3290383> (cit. on pp. 2, 24).
- [CL05] David Chase and Yossi Lev. “Dynamic circular work-stealing deque”. In: *SPAA 2005: Proceedings of the 17th Annual ACM Symposium on Parallelism in Algorithms and Architectures, July 18-20, 2005, Las Vegas, Nevada, USA*. ACM, 2005, pp. 21–28. DOI: [10.1145/1073970.1073974](https://doi.org/10.1145/1073970.1073974). URL: <https://doi.org/10.1145/1073970.1073974> (cit. on pp. 168, 299).
- [Cho+21a] Kyeongmin Cho, Sung-Hwan Lee, Azalea Raad, and Jeehoon Kang. “Revamping hardware persistency models: view-based and axiomatic persistency models for Intel-x86 and Armv8”. In: *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. PLDI 2021. Virtual, Canada: Association for Computing Machinery, 2021, 16–31. ISBN: 9781450383912. DOI: [10.1145/3453483.3454027](https://doi.org/10.1145/3453483.3454027). URL: <https://doi.org/10.1145/3453483.3454027> (cit. on p. 58).
- [Cho+21b] Minki Cho, Sung-Hwan Lee, Chung-Kil Hur, and Ori Lahav. “Modular data-race-freedom guarantees in the promising semantics”. In: *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. PLDI 2021. Virtual, Canada: Association for Computing Machinery, 2021, 867–882. ISBN: 9781450383912. DOI: [10.1145/3453483.3454082](https://doi.org/10.1145/3453483.3454082). URL: <https://doi.org/10.1145/3453483.3454082> (cit. on pp. 2, 57).
- [Cho+22] Minki Cho, Sung-Hwan Lee, Dongjae Lee, Chung-Kil Hur, and Ori Lahav. “Sequential reasoning for optimizing compilers under weak memory concurrency”. In: *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. PLDI 2022. San Diego, CA, USA: Association for Computing Machinery, 2022, 213–228. ISBN: 9781450392655. DOI: [10.1145/3519939.3523718](https://doi.org/10.1145/3519939.3523718). URL: <https://doi.org/10.1145/3519939.3523718> (cit. on pp. 2, 57, 58).
- [Www] *Crossbeam: Tools for concurrent programming in Rust*. Available at <https://github.com/crossbeam-rs/crossbeam>. 2021 (cit. on p. 168).
- [Dal+20] Sadegh Dalvandi, Simon Doherty, Brijesh Dongol, and Heike Wehrheim. “Owicki-Gries Reasoning for C11 RAR”. In: *34th European Conference on Object-Oriented Programming, ECOOP 2020, November 15-17, 2020, Berlin, Germany (Virtual Conference)*. Vol. 166. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020, 11:1–11:26. DOI: [10.4230/LIPIcs.ECOOP.2020.11](https://doi.org/10.4230/LIPIcs.ECOOP.2020.11). URL: <https://doi.org/10.4230/LIPIcs.ECOOP.2020.11> (cit. on p. 298).
- [DD21a] Sadegh Dalvandi and Brijesh Dongol. “Verifying C11-Style Weak Memory Libraries via Refinement”. In: *CoRR abs/2108.06944* (2021). arXiv: [2108.06944](https://arxiv.org/abs/2108.06944). URL: <https://arxiv.org/abs/2108.06944> (cit. on p. 298).
- [DD21b] Sadegh Dalvandi and Brijesh Dongol. “Verifying C11-style weak memory libraries”. In: *PPoPP ’21: 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, Virtual Event, Republic of Korea, February 27- March 3, 2021*. ACM, 2021, pp. 451–453. DOI: [10.1145/3437801.3441619](https://doi.org/10.1145/3437801.3441619). URL: <https://doi.org/10.1145/3437801.3441619> (cit. on p. 298).

- [Dan+20a] Hoang-Hai Dang, Jacques-Henri Jourdan, Jan-Oliver Kaiser, and Derek Dreyer. “RustBelt Meets Relaxed Memory”. In: *Proc. ACM Program. Lang.* 4.POPL (2020), 34:1–34:29. DOI: [10.1145/3371102](https://doi.org/10.1145/3371102). URL: <https://doi.org/10.1145/3371102> (cit. on pp. 1, 3, 9, 136, 298).
- [Dan+20b] Hoang-Hai Dang, Jacques-Henri Jourdan, Jan-Oliver Kaiser, and Derek Dreyer. *Technical Appendix: RustBelt Meets Relaxed Memory*. 2020. URL: <https://plv.mpi-sws.org/rustbelt/rbrlx/appendix.pdf> (cit. on pp. 44, 45).
- [Dan+22] Hoang-Hai Dang, Jaehwang Jung, Jaemin Choi, Duc-Thuan Nguyen, William Mansky, Jeehoon Kang, and Derek Dreyer. “Compass: strong and compositional library specifications in relaxed memory separation logic”. In: *PLDI ’22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*. Ed. by Ranjit Jhala and Isil Dillig. ACM, 2022, pp. 792–808. DOI: [10.1145/3519939.3523451](https://doi.org/10.1145/3519939.3523451). URL: <https://doi.org/10.1145/3519939.3523451> (cit. on pp. 3, 9, 10).
- [Del+17] Germán Andrés Delbianco, Ilya Sergey, Aleksandar Nanevski, and Anindya Banerjee. “Concurrent Data Structures Linked in Time”. In: *31st European Conference on Object-Oriented Programming, ECOOP 2017, June 19-23, 2017, Barcelona, Spain*. Vol. 74. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017, 8:1–8:30. DOI: [10.4230/LIPIcs.ECOOP.2017.8](https://doi.org/10.4230/LIPIcs.ECOOP.2017.8). URL: <https://doi.org/10.4230/LIPIcs.ECOOP.2017.8> (cit. on p. 297).
- [Der+14] John Derrick, Brijesh Dongol, Gerhard Schellhorn, Bogdan Tofan, Oleg Travkin, and Heike Wehrheim. “Quiescent Consistency: Defining and Verifying Relaxed Linearizability”. In: *FM 2014: Formal Methods - 19th International Symposium, Singapore, May 12-16, 2014. Proceedings*. Vol. 8442. Lecture Notes in Computer Science. Springer, 2014, pp. 200–214. DOI: [10.1007/978-3-319-06410-9\\_15](https://doi.org/10.1007/978-3-319-06410-9_15). URL: [https://doi.org/10.1007/978-3-319-06410-9\\_15](https://doi.org/10.1007/978-3-319-06410-9_15) (cit. on pp. 5, 297).
- [DY+10] Thomas Dinsdale-Young, Mike Dodds, Philippa Gardner, Matthew J. Parkinson, and Viktor Vafeiadis. “Concurrent abstract predicates”. In: *ECOOP. LNCS*. 2010. DOI: [10.1007/978-3-642-14107-2\\_24](https://doi.org/10.1007/978-3-642-14107-2_24) (cit. on p. 1).
- [Doh+18] Simon Doherty, Brijesh Dongol, Heike Wehrheim, and John Derrick. “Making Linearizability Compositional for Partially Ordered Executions”. In: *Integrated Formal Methods - 14th International Conference, IFM 2018, Maynooth, Ireland, September 5-7, 2018, Proceedings*. Vol. 11023. Lecture Notes in Computer Science. Springer, 2018, pp. 110–129. DOI: [10.1007/978-3-319-98938-9\\_7](https://doi.org/10.1007/978-3-319-98938-9_7). URL: [https://doi.org/10.1007/978-3-319-98938-9\\_7](https://doi.org/10.1007/978-3-319-98938-9_7) (cit. on pp. 5, 297).
- [Doh+19] Simon Doherty, Brijesh Dongol, Heike Wehrheim, and John Derrick. “Verifying C11 programs operationally”. In: *Proceedings of the 24th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP 2019, Washington, DC, USA, February 16-20, 2019*. Ed. by Jeffrey K. Hollingsworth and Idit Keidar. ACM, 2019, pp. 355–365. DOI: [10.1145/3293883.3295702](https://doi.org/10.1145/3293883.3295702). URL: <https://doi.org/10.1145/3293883.3295702> (cit. on p. 57).
- [DV16] Marko Doko and Viktor Vafeiadis. “A Program Logic for C11 Memory Fences”. In: *Verification, Model Checking, and Abstract Interpretation - 17th International Conference, VMCAI 2016, St. Petersburg, FL, USA, January 17-19, 2016. Proceedings*. Vol. 9583. Lecture Notes in Computer Science. Springer, 2016, pp. 413–430. DOI: [10.1007/978-3-662-49122-5\\_20](https://doi.org/10.1007/978-3-662-49122-5_20). URL: [https://doi.org/10.1007/978-3-662-49122-5\\_20](https://doi.org/10.1007/978-3-662-49122-5_20) (cit. on pp. 1, 2, 24, 25, 61, 99, 102, 124, 133, 167).
- [DV17] Marko Doko and Viktor Vafeiadis. “Tackling Real-Life Relaxed Concurrency with FSL++”. In: *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings*. Vol. 10201. Lecture Notes in Computer Science. Springer, 2017, pp. 448–475. DOI: [10.1007/978-3-662-54434-1\\_17](https://doi.org/10.1007/978-3-662-54434-1_17). URL: [https://doi.org/10.1007/978-3-662-54434-1\\_17](https://doi.org/10.1007/978-3-662-54434-1_17) (cit. on pp. 1, 2, 5, 24, 99, 102, 124, 133, 167, 251).
- [DSM18] Stephen Dolan, K. C. Sivaramakrishnan, and Anil Madhavapeddy. “Bounding data races in space and time”. In: *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018, Philadelphia, PA, USA, June 18-22, 2018*. ACM, 2018, pp. 242–255. DOI: [10.1145/3192366.3192421](https://doi.org/10.1145/3192366.3192421). URL: <https://doi.org/10.1145/3192366.3192421> (cit. on pp. 2, 6).
- [Don+18] Brijesh Dongol, Radha Jagadeesan, James Riely, and Alasdair Armstrong. “On abstraction and compositionality for weak-memory linearisability”. In: *Verification, Model Checking, and Abstract Interpretation - 19th International Conference, VMCAI 2018, Los Angeles, CA, USA, January 7-9, 2018, Proceedings*. Vol. 10747. Lecture Notes in Computer Science. Springer, 2018, pp. 183–204. DOI: [10.1007/978-3-319-73721-8\\_9](https://doi.org/10.1007/978-3-319-73721-8_9). URL: [https://doi.org/10.1007/978-3-319-73721-8\\_9](https://doi.org/10.1007/978-3-319-73721-8_9) (cit. on pp. 5, 297).

- [EE19] Michael Emmi and Constantin Enea. “Weak-consistency specification via visibility relaxation”. In: *Proc. ACM Program. Lang.* 3.POPL (2019), 60:1–60:28. DOI: [10.1145/3290373](https://doi.org/10.1145/3290373). URL: <https://doi.org/10.1145/3290373> (cit. on pp. 5, 297).
- [Fen09] Xinyu Feng. “Local Rely-Guarantee Reasoning”. In: *POPL*. 2009. DOI: [10.1145/1594834.1480922](https://doi.org/10.1145/1594834.1480922) (cit. on p. 1).
- [FFS07] Xinyu Feng, Rodrigo Ferreira, and Zhong Shao. “On the Relationship Between Concurrent Separation Logic and Assume-Guarantee Reasoning”. In: *ESOP*. 2007 (cit. on p. 1).
- [Flu+17] Shaked Flur, Susmit Sarkar, Christopher Pulte, Kyndylan Nienhuis, Luc Maranget, Kathryn E. Gray, Ali Sezgin, Mark Batty, and Peter Sewell. “Mixed-size concurrency: ARM, POWER, C/C++11, and SC”. In: *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, Paris, France, January 18-20, 2017*. Ed. by Giuseppe Castagna and Andrew D. Gordon. ACM, 2017, pp. 429–442. DOI: [10.1145/3009837.3009839](https://doi.org/10.1145/3009837.3009839). URL: <https://doi.org/10.1145/3009837.3009839> (cit. on pp. 2, 58).
- [Fra04] Keir Fraser. “Practical lock-freedom”. PhD thesis. University of Cambridge, 2004 (cit. on p. 299).
- [FKB18] Dan Frumin, Robbert Krebbers, and Lars Birkedal. “ReLoC: A Mechanised Relational Logic for Fine-Grained Concurrency”. In: *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2018, Oxford, UK, July 09-12, 2018*. ACM, 2018, pp. 442–451. DOI: [10.1145/3209108.3209174](https://doi.org/10.1145/3209108.3209174). URL: <https://doi.org/10.1145/3209108.3209174> (cit. on p. 297).
- [FKB21] Dan Frumin, Robbert Krebbers, and Lars Birkedal. “ReLoC Reloaded: A Mechanized Relational Logic for Fine-Grained Concurrency and Logical Atomicity”. In: *Logical Methods in Computer Science* Volume 17, Issue 3 (July 2021). DOI: [10.46298/lmcs-17\(3:9\)2021](https://doi.org/10.46298/lmcs-17(3:9)2021). URL: <https://lmcs.episciences.org/7708> (cit. on p. 1).
- [Fu+10] Ming Fu, Yong Li, Xinyu Feng, Zhong Shao, and Yu Zhang. “Reasoning about Optimistic Concurrency Using a Program Logic for History”. In: *CONCUR*. 2010 (cit. on p. 1).
- [G+22] Lennard Gäher, Michael Sammler, Simon Spies, Ralf Jung, Hoang-Hai Dang, Robbert Krebbers, Jeehoon Kang, and Derek Dreyer. “Simuliris: A Separation Logic Framework for Verifying Concurrent Program Optimizations”. In: *POPL*. 2022 (cit. on p. 1).
- [Got+07] Alexey Gotsman, Josh Berdine, Byron Cook, Noam Rinetzkky, and Mooly Sagiv. “Local Reasoning for Storable Locks and Threads”. In: *Programming Languages and Systems, 5th Asian Symposium, APLAS 2007, Singapore, November 29-December 1, 2007, Proceedings*. Ed. by Zhong Shao. Vol. 4807. Lecture Notes in Computer Science. Springer, 2007, pp. 19–37. DOI: [10.1007/978-3-540-76637-7\\_3](https://doi.org/10.1007/978-3-540-76637-7_3). URL: [https://doi.org/10.1007/978-3-540-76637-7\\_3](https://doi.org/10.1007/978-3-540-76637-7_3) (cit. on pp. 167, 175).
- [Haa+16] Andreas Haas, Thomas A. Henzinger, Andreas Holzer, Christoph M. Kirsch, Michael Lippautz, Hannes Payer, Ali Sezgin, Ana Sokolova, and Helmut Veith. “Local Linearizability for Concurrent Container-Type Data Structures”. In: *27th International Conference on Concurrency Theory, CONCUR 2016, August 23-26, 2016, Québec City, Canada*. Vol. 59. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016, 6:1–6:15. DOI: [10.4230/LIPIcs.CONCUR.2016.6](https://doi.org/10.4230/LIPIcs.CONCUR.2016.6). URL: <https://doi.org/10.4230/LIPIcs.CONCUR.2016.6> (cit. on pp. 5, 297).
- [Ham+24] Angus Hammond, Zongyuan Liu, Thibaut Pérami, Peter Sewell, Lars Birkedal, and Jean Pichon-Pharabod. “An Axiomatic Basis for Computer Programming on the Relaxed Arm-A Architecture: The AxSL Logic”. In: *Proc. ACM Program. Lang.* 8.POPL (2024). DOI: [10.1145/3632863](https://doi.org/10.1145/3632863). URL: <https://doi.org/10.1145/3632863> (cit. on p. 168).
- [He+18] Mengda He, Viktor Vafeiadis, Shengchao Qin, and João F. Ferreira. “GPS+: Reasoning About Fences and Relaxed Atomics”. In: *Int. J. Parallel Program.* 46.6 (2018), pp. 1157–1183. DOI: [10.1007/s10766-017-0518-x](https://doi.org/10.1007/s10766-017-0518-x). URL: <https://doi.org/10.1007/s10766-017-0518-x> (cit. on pp. 1, 133, 167).
- [HRV15] Nir Hemed, Noam Rinetzkky, and Viktor Vafeiadis. “Modular Verification of Concurrency-Aware Linearizability”. In: *Distributed Computing - 29th International Symposium, DISC 2015, Tokyo, Japan, October 7-9, 2015, Proceedings*. Vol. 9363. Lecture Notes in Computer Science. Springer, 2015, pp. 371–387. DOI: [10.1007/978-3-662-48653-5\\_25](https://doi.org/10.1007/978-3-662-48653-5_25). URL: [https://doi.org/10.1007/978-3-662-48653-5\\_25](https://doi.org/10.1007/978-3-662-48653-5_25) (cit. on pp. 5, 297).
- [HSY04] Danny Hendler, Nir Shavit, and Lena Yerushalmi. “A scalable lock-free stack algorithm”. In: *SPAA 2004: Proceedings of the Sixteenth Annual ACM Symposium on Parallelism in Algorithms and Architectures, June 27-30, 2004, Barcelona, Spain*. ACM, 2004, pp. 206–215. DOI: [10.1145/1007912.1007944](https://doi.org/10.1145/1007912.1007944). URL: <https://doi.org/10.1145/1007912.1007944> (cit. on p. 291).



- [Hen+13] Thomas A. Henzinger, Christoph M. Kirsch, Hannes Payer, Ali Sezgin, and Ana Sokolova. “Quantitative relaxation of concurrent data structures”. In: *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’13, Rome, Italy - January 23 - 25, 2013*. ACM, 2013, pp. 317–328. DOI: [10.1145/2429069.2429109](https://doi.org/10.1145/2429069.2429109). URL: <https://doi.org/10.1145/2429069.2429109> (cit. on pp. 5, 297).
- [HW90] Maurice Herlihy and Jeannette M. Wing. “Linearizability: A Correctness Condition for Concurrent Objects”. In: *ACM Trans. Program. Lang. Syst.* 12.3 (1990), pp. 463–492. DOI: [10.1145/78969.78972](https://doi.org/10.1145/78969.78972). URL: <https://doi.org/10.1145/78969.78972> (cit. on pp. 5, 266, 273).
- [HAN08] Aquinas Hobor, Andrew W. Appel, and Francesco Zappa Nardelli. “Oracle Semantics for Concurrent Separation Logic”. In: *Programming Languages and Systems, 17th European Symposium on Programming, ESOP 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings*. Ed. by Sophia Drossopoulou. Vol. 4960. Lecture Notes in Computer Science. Springer, 2008, pp. 353–367. DOI: [10.1007/978-3-540-78739-6\\_27](https://doi.org/10.1007/978-3-540-78739-6_27). URL: [https://doi.org/10.1007/978-3-540-78739-6\\_27](https://doi.org/10.1007/978-3-540-78739-6_27) (cit. on pp. 167, 175).
- [Iri22] Iris Team (The). *The Iris 3.6 Technical Appendix*. Tech. rep. 2022. URL: <https://plv.mpi-sws.org/iris/appendix-3.6.pdf> (cit. on pp. 136, 138, 143).
- [IO01] Samin S. Ishtiaq and Peter W. O’Hearn. “BI as an Assertion Language for Mutable Data Structures”. In: *Conference Record of POPL 2001: The 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, London, UK, January 17-19, 2001*. Ed. by Chris Hankin and Dave Schmidt. ACM, 2001, pp. 14–26. DOI: [10.1145/360204.375719](https://doi.org/10.1145/360204.375719). URL: <https://doi.org/10.1145/360204.375719> (cit. on p. 63).
- [Jag+13] Radha Jagadeesan, Gustavo Petri, Corin Pitcher, and James Riely. “Quarantining Weakness - Compositional Reasoning under Relaxed Memory Models (Extended Abstract)”. In: *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings*. Vol. 7792. Lecture Notes in Computer Science. Springer, 2013, pp. 492–511. DOI: [10.1007/978-3-642-37036-6\\_27](https://doi.org/10.1007/978-3-642-37036-6_27). URL: [https://doi.org/10.1007/978-3-642-37036-6\\_27](https://doi.org/10.1007/978-3-642-37036-6_27) (cit. on pp. 5, 297).
- [JR14] Radha Jagadeesan and James Riely. “Between Linearizability and Quiescent Consistency - Quantitative Quiescent Consistency”. In: *Automata, Languages, and Programming - 41st International Colloquium, ICALP 2014, Copenhagen, Denmark, July 8-11, 2014, Proceedings, Part II*. Vol. 8573. Lecture Notes in Computer Science. Springer, 2014, pp. 220–231. DOI: [10.1007/978-3-662-43951-7\\_19](https://doi.org/10.1007/978-3-662-43951-7_19). URL: [https://doi.org/10.1007/978-3-662-43951-7\\_19](https://doi.org/10.1007/978-3-662-43951-7_19) (cit. on pp. 5, 297).
- [JB12] Jonas Braband Jensen and Lars Birkedal. “Fictional Separation Logic”. In: *Programming Languages and Systems - 21st European Symposium on Programming, ESOP 2012, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2012, Tallinn, Estonia, March 24 - April 1, 2012. Proceedings*. Ed. by Helmut Seidl. Vol. 7211. Lecture Notes in Computer Science. Springer, 2012, pp. 377–396. DOI: [10.1007/978-3-642-28869-2\\_19](https://doi.org/10.1007/978-3-642-28869-2_19). URL: [https://doi.org/10.1007/978-3-642-28869-2\\_19](https://doi.org/10.1007/978-3-642-28869-2_19) (cit. on pp. 1, 73).
- [Jou18] Jacques-Henri Jourdan. *Insufficient synchronization in Arc::get\_mut*. Rust issue #51780, <https://github.com/rust-lang/rust/issues/51780>. 2018 (cit. on pp. 5, 8, 177).
- [Jun+23] Jaehwang Jung, Janggun Lee, Jaemin Choi, Jaewoo Kim, Sunho Park, and Jeehoon Kang. “Modular Verification of Safe Memory Reclamation in Concurrent Separation Logic”. In: *Proc. ACM Program. Lang.* 7.OOPSLA2 (2023). DOI: [10.1145/3622827](https://doi.org/10.1145/3622827). URL: <https://doi.org/10.1145/3622827> (cit. on p. 299).
- [Jun+18a] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. “RustBelt: Securing the Foundations of the Rust Programming Language”. In: *PACMPL* 2.POPL (2018) (cit. on pp. 3, 4, 10, 13, 47, 143, 173, 180, 181, 187, 188, 190, 191, 225).
- [Jun+16] Ralf Jung, Robbert Krebbers, Lars Birkedal, and Derek Dreyer. “Higher-order ghost state”. In: *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016, Nara, Japan, September 18-22, 2016*. ACM, 2016, pp. 256–269. DOI: [10.1145/2951913.2951943](https://doi.org/10.1145/2951913.2951943). URL: <https://doi.org/10.1145/2951913.2951943> (cit. on pp. 3, 63).
- [Jun+18b] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. “Iris from the ground up: A modular foundation for higher-order concurrent separation logic”. In: *J. Funct. Program.* 28 (2018), e20. DOI: [10.1017/S0956796818000151](https://doi.org/10.1017/S0956796818000151). URL: <https://doi.org/10.1017/S0956796818000151> (cit. on pp. 1, 3, 63, 67, 75, 174).

- [Jun+20] Ralf Jung, Rodolphe Lepigre, Gaurav Parthasarathy, Marianna Rapoport, Amin Timany, Derek Dreyer, and Bart Jacobs. “The future is ours: prophecy variables in separation logic”. In: *Proc. ACM Program. Lang.* 4.POPL (2020), 45:1–45:32. DOI: [10.1145/3371113](https://doi.org/10.1145/3371113). URL: <https://doi.org/10.1145/3371113> (cit. on pp. 6, 257, 258, 268, 297).
- [Jun+15] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. “Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning”. In: *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*. ACM, 2015, pp. 637–650. DOI: [10.1145/2676726.2676980](https://doi.org/10.1145/2676726.2676980). URL: <https://doi.org/10.1145/2676726.2676980> (cit. on pp. 1, 3, 6, 63, 73, 258).
- [Kai+17] Jan-Oliver Kaiser, Hoang-Hai Dang, Derek Dreyer, Ori Lahav, and Viktor Vafeiadis. “Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris”. In: *31st European Conference on Object-Oriented Programming, ECOOP 2017, June 19-23, 2017, Barcelona, Spain*. Vol. 74. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017, 17:1–17:29. DOI: [10.4230/LIPIcs.ECOOP.2017.17](https://doi.org/10.4230/LIPIcs.ECOOP.2017.17). URL: <https://doi.org/10.4230/LIPIcs.ECOOP.2017.17> (cit. on pp. 1, 3, 5, 8, 9, 27, 42, 43, 61, 75, 99, 115, 124, 133, 149, 159, 167, 177, 201, 208, 298).
- [Kan+17] Jeehoon Kang, Chung-Kil Hur, Ori Lahav, Viktor Vafeiadis, and Derek Dreyer. “A promising semantics for relaxed-memory concurrency”. In: *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, Paris, France, January 18-20, 2017*. ACM, 2017, pp. 175–189. DOI: [10.1145/3009837.3009850](https://doi.org/10.1145/3009837.3009850). URL: <https://doi.org/10.1145/3009837.3009850> (cit. on pp. 2, 24, 27, 32, 44, 45, 57).
- [Kan+15] Jeehoon Kang, Chung-Kil Hur, William Mansky, Dmitri Garbuzov, Steve Zdancewic, and Viktor Vafeiadis. “A formal C memory model supporting integer-pointer casts”. In: *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015*. Ed. by David Grove and Stephen M. Blackburn. ACM, 2015, pp. 326–335. DOI: [10.1145/2737924.2738005](https://doi.org/10.1145/2737924.2738005). URL: <https://doi.org/10.1145/2737924.2738005> (cit. on p. 51).
- [Khy+17] Artem Khyzha, Mike Dodds, Alexey Gotsman, and Matthew J. Parkinson. “Proving Linearizability Using Partial Orders”. In: *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings*. Vol. 10201. Lecture Notes in Computer Science. Springer, 2017, pp. 639–667. DOI: [10.1007/978-3-662-54434-1\\_24](https://doi.org/10.1007/978-3-662-54434-1_24). URL: [https://doi.org/10.1007/978-3-662-54434-1\\_24](https://doi.org/10.1007/978-3-662-54434-1_24) (cit. on p. 297).
- [KN18] Steve Klabnik and Carol Nichols. *The Rust Programming Language*. 2018. URL: <https://doc.rust-lang.org/stable/book/> (cit. on pp. 3, 173, 174).
- [Kre+18] Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, Joseph Tassarotti, Jan-Oliver Kaiser, Amin Timany, Arthur Charguéraud, and Derek Dreyer. “MoSeL: A General, Extensible Modal Framework for Interactive Proofs in Separation Logic”. In: *PACMPL 2.ICFP (2018)* (cit. on pp. 3, 63, 174).
- [Kre+17] Robbert Krebbers, Ralf Jung, Ales Bizjak, Jacques-Henri Jourdan, Derek Dreyer, and Lars Birkedal. “The Essence of Higher-Order Concurrent Separation Logic”. In: *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings*. Vol. 10201. Lecture Notes in Computer Science. Springer, 2017, pp. 696–723. DOI: [10.1007/978-3-662-54434-1\\_26](https://doi.org/10.1007/978-3-662-54434-1_26). URL: [https://doi.org/10.1007/978-3-662-54434-1\\_26](https://doi.org/10.1007/978-3-662-54434-1_26) (cit. on pp. 3, 63).
- [KTB17] Robbert Krebbers, Amin Timany, and Lars Birkedal. “Interactive Proofs in Higher-Order Concurrent Separation Logic”. In: *POPL*. 2017. DOI: [10.1145/3009837.3009855](https://doi.org/10.1145/3009837.3009855) (cit. on pp. 3, 63, 174).
- [Kri+20] Siddharth Krishna, Michael Emmi, Constantin Enea, and Dejan Jovanovic. “Verifying Visibility-Based Weak Consistency”. In: *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings*. Lecture Notes in Computer Science. Springer, 2020, pp. 280–307. DOI: [10.1007/978-3-030-44914-8\\_11](https://doi.org/10.1007/978-3-030-44914-8_11). URL: [https://doi.org/10.1007/978-3-030-44914-8\\_11](https://doi.org/10.1007/978-3-030-44914-8_11) (cit. on pp. 5, 297).
- [KSB17] Morten Krogh-Jespersen, Kasper Svendsen, and Lars Birkedal. “A relational model of types-and-effects in higher-order concurrent separation logic”. In: *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, Paris, France, January 18-20, 2017*. ACM, 2017, pp. 218–231. DOI: [10.1145/3009837.3009877](https://doi.org/10.1145/3009837.3009877). URL: <https://doi.org/10.1145/3009837.3009877> (cit. on p. 297).



- [Kro+20] Morten Krogh-Jespersen, Amin Timany, Marit Edna Ohlenbusch, Simon Oddershede Gregersen, and Lars Birkedal. “Aneris: A Mechanised Logic for Modular Reasoning about Distributed Systems”. In: *ESOP*. Vol. 12075. Lecture Notes in Computer Science. Springer, 2020, pp. 336–365 (cit. on p. 1).
- [LGV16] Ori Lahav, Nick Giannarakis, and Viktor Vafeiadis. “Taming release-acquire consistency”. In: *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*. ACM, 2016, pp. 649–662. DOI: [10.1145/2837614.2837643](https://doi.org/10.1145/2837614.2837643). URL: <https://doi.org/10.1145/2837614.2837643> (cit. on p. 27).
- [Lah+17] Ori Lahav, Viktor Vafeiadis, Jeehoon Kang, Chung-Kil Hur, and Derek Dreyer. “Repairing sequential consistency in C/C++11”. In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*. ACM, 2017, pp. 618–632. DOI: [10.1145/3062341.3062352](https://doi.org/10.1145/3062341.3062352). URL: <https://doi.org/10.1145/3062341.3062352> (cit. on pp. 2, 3, 13, 15–17, 21, 23, 27).
- [Lam79] Leslie Lamport. “How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs”. In: *IEEE Trans. Computers* 28.9 (1979), pp. 690–691. DOI: [10.1109/TC.1979.1675439](https://doi.org/10.1109/TC.1979.1675439). URL: <https://doi.org/10.1109/TC.1979.1675439> (cit. on p. 1).
- [Lê+13] Nhat Minh Lê, Antoniu Pop, Albert Cohen, and Francesco Zappa Nardelli. “Correct and efficient work-stealing for weak memory models”. In: *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP ’13, Shenzhen, China, February 23-27, 2013*. ACM, 2013, pp. 69–80. DOI: [10.1145/2442516.2442524](https://doi.org/10.1145/2442516.2442524). URL: <https://doi.org/10.1145/2442516.2442524> (cit. on p. 299).
- [Lee+18] Juneyoung Lee, Chung-Kil Hur, Ralf Jung, Zhengyang Liu, John Regehr, and Nuno P. Lopes. “Reconciling high-level optimizations and low-level code in LLVM”. In: *Proc. ACM Program. Lang.* 2.OOPSLA (2018), 125:1–125:28. DOI: [10.1145/3276495](https://doi.org/10.1145/3276495). URL: <https://doi.org/10.1145/3276495> (cit. on p. 51).
- [Lee+17] Juneyoung Lee, Yoonseung Kim, Youngju Song, Chung-Kil Hur, Sanjoy Das, David Majnemer, John Regehr, and Nuno P. Lopes. “Taming undefined behavior in LLVM”. In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*. Ed. by Albert Cohen and Martin T. Vechev. ACM, 2017, pp. 633–647. DOI: [10.1145/3062341.3062343](https://doi.org/10.1145/3062341.3062343). URL: <https://doi.org/10.1145/3062341.3062343> (cit. on pp. 30, 57).
- [Lee+23] Sung-Hwan Lee, Minki Cho, Roy Margalit, Chung-Kil Hur, and Ori Lahav. “Putting Weak Memory in Order via a Promising Intermediate Representation”. In: *PLDI (2023)* (cit. on pp. 2, 25, 57, 302).
- [Lee+20] Sung-Hwan Lee, Minki Cho, Anton Podkopaev, Soham Chakraborty, Chung-Kil Hur, Ori Lahav, and Viktor Vafeiadis. “Promising 2.0: global optimizations in relaxed memory concurrency”. In: *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*. ACM, 2020, pp. 362–376. DOI: [10.1145/3385412.3386010](https://doi.org/10.1145/3385412.3386010). URL: <https://doi.org/10.1145/3385412.3386010> (cit. on pp. 2, 57).
- [Lep+22] Rodolphe Lepigre, Michael Sammler, Kayvan Memarian, Robbert Krebbers, Derek Dreyer, and Peter Sewell. “VIP: verifying real-world C idioms with integer-pointer casts”. In: *Proc. ACM Program. Lang.* 6.POPL (2022), pp. 1–32. DOI: [10.1145/3498681](https://doi.org/10.1145/3498681). URL: <https://doi.org/10.1145/3498681> (cit. on p. 51).
- [Ler+12] Xavier Leroy, Andrew W. Appel, Sandrine Blazy, and Gordon Stewart. *The CompCert Memory Model, Version 2*. Research Report RR-7987. INRIA, June 2012, p. 26. URL: <https://hal.inria.fr/hal-00703441> (cit. on p. 30).
- [LF13] Hongjin Liang and Xinyu Feng. “Modular verification of linearizability with non-fixed linearization points”. In: *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI ’13, Seattle, WA, USA, June 16-19, 2013*. ACM, 2013, pp. 459–470. DOI: [10.1145/2491956.2462189](https://doi.org/10.1145/2491956.2462189). URL: <https://doi.org/10.1145/2491956.2462189> (cit. on p. 297).
- [Mat16] Nicholas D. Matsakis. *Introducing MIR*. 2016. URL: <https://blog.rust-lang.org/2016/04/19/MIR.html> (cit. on p. 50).
- [Mem+19] Kayvan Memarian, Victor B. F. Gomes, Brooks Davis, Stephen Kell, Alexander Richardson, Robert N. M. Watson, and Peter Sewell. “Exploring C semantics and pointer provenance”. In: *Proc. ACM Program. Lang.* 3.POPL (2019), 67:1–67:32. DOI: [10.1145/3290380](https://doi.org/10.1145/3290380). URL: <https://doi.org/10.1145/3290380> (cit. on p. 51).

- [Mem+16] Kayvan Memarian, Justus Matthiesen, James Lingard, Kyndylan Nienhuis, David Chisnall, Robert N. M. Watson, and Peter Sewell. “Into the depths of C: elaborating the de facto standards”. In: *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2016, Santa Barbara, CA, USA, June 13-17, 2016*. Ed. by Chandra Krintz and Emery D. Berger. ACM, 2016, pp. 1–15. DOI: [10.1145/2908080.2908081](https://doi.org/10.1145/2908080.2908081). URL: <https://doi.org/10.1145/2908080.2908081> (cit. on p. 51).
- [MJ21] Glen Mével and Jacques-Henri Jourdan. “Formal verification of a concurrent bounded queue in a weak memory model”. In: *Proc. ACM Program. Lang.* 5.ICFP (2021), pp. 1–29. DOI: [10.1145/3473571](https://doi.org/10.1145/3473571). URL: <https://doi.org/10.1145/3473571> (cit. on pp. 6, 257).
- [MJP20] Glen Mével, Jacques-Henri Jourdan, and François Pottier. “Cosmo: a concurrent separation logic for multicore OCaml”. In: *Proc. ACM Program. Lang.* 4.ICFP (2020), 96:1–96:29. DOI: [10.1145/3408978](https://doi.org/10.1145/3408978). URL: <https://doi.org/10.1145/3408978> (cit. on pp. 1, 5, 6, 61, 99, 106, 115, 124, 298, 302).
- [Mic04] Maged M. Michael. “Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects”. In: *IEEE Trans. Parallel Distrib. Syst.* 15.6 (June 2004), 491–504. ISSN: 1045-9219. DOI: [10.1109/TPDS.2004.8](https://doi.org/10.1109/TPDS.2004.8). URL: <https://doi.org/10.1109/TPDS.2004.8> (cit. on p. 299).
- [MS96] Maged M. Michael and Michael L. Scott. “Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms”. In: *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing, Philadelphia, Pennsylvania, USA, May 23-26, 1996*. ACM, 1996, pp. 267–275. DOI: [10.1145/248052.248106](https://doi.org/10.1145/248052.248106). URL: <https://doi.org/10.1145/248052.248106> (cit. on pp. 134, 268, 273).
- [MK23] Ike Mulder and Robbert Krebbers. “Proof Automation for Linearizability in Separation Logic”. In: *Proc. ACM Program. Lang.* 7.OOPSLA1 (2023). DOI: [10.1145/3586043](https://doi.org/10.1145/3586043). URL: <https://doi.org/10.1145/3586043> (cit. on p. 298).
- [MKG22] Ike Mulder, Robbert Krebbers, and Herman Geuvers. “Diaframe: automated verification of fine-grained concurrent programs in Iris”. In: *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2022, San Diego, CA, USA: Association for Computing Machinery, 2022*, 809–824. ISBN: 9781450392655. DOI: [10.1145/3519939.3523432](https://doi.org/10.1145/3519939.3523432). URL: <https://doi.org/10.1145/3519939.3523432> (cit. on p. 298).
- [Nan+14] Aleksandar Nanevski, Ruy Ley-Wild, Ilya Sergey, and Germán Andrés Delbianco. “Communicating state transition systems for fine-grained concurrent resources”. In: *ESOP*. LNCS. 2014. DOI: [10.1007/978-3-642-54833-8\\_16](https://doi.org/10.1007/978-3-642-54833-8_16) (cit. on p. 1).
- [Nei94] Gil Neiger. “Set-Linearizability”. In: *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing, Los Angeles, California, USA, August 14-17, 1994*. ACM, 1994, p. 396. DOI: [10.1145/197917.198176](https://doi.org/10.1145/197917.198176). URL: <https://doi.org/10.1145/197917.198176> (cit. on pp. 5, 297).
- [O’H07] Peter W. O’Hearn. “Resources, concurrency, and local reasoning”. In: *Theoretical Computer Science* 375.1-3 (2007). DOI: [10.1016/j.tcs.2006.12.035](https://doi.org/10.1016/j.tcs.2006.12.035) (cit. on p. 1).
- [OP99] Peter W. O’Hearn and David J. Pym. “The logic of bunched implications”. In: *Bull. Symb. Log.* 5.2 (1999), pp. 215–244. DOI: [10.2307/421090](https://doi.org/10.2307/421090). URL: <https://doi.org/10.2307/421090> (cit. on p. 63).
- [ON22] Joakim Öhman and Aleksandar Nanevski. “Visibility Reasoning for Concurrent Snapshot Algorithms”. In: *POPL*. 2022 (cit. on p. 297).
- [OD18] Peizhao Ou and Brian Demsky. “Towards understanding the costs of avoiding out-of-thin-air results”. In: *Proc. ACM Program. Lang.* 2.OOPSLA (2018), 136:1–136:29. DOI: [10.1145/3276506](https://doi.org/10.1145/3276506). URL: <https://doi.org/10.1145/3276506> (cit. on p. 24).
- [Par+24] Sunho Park, Jaewoo Kim, Ike Mulder, Jaehwang Jung, Janggun Lee, Robbert Krebbers, and Jeehoon Kang. “A Proof Recipe for Linearizability in Relaxed Memory Separation Logic (to appear in PLDI 2024)”. In: *PLDI* (2024) (cit. on p. 298).
- [PLV19] Anton Podkopaev, Ori Lahav, and Viktor Vafeiadis. “Bridging the gap between programming languages and hardware weak memory models”. In: *Proc. ACM Program. Lang.* 3.POPL (2019), 69:1–69:31. DOI: [10.1145/3290382](https://doi.org/10.1145/3290382). URL: <https://doi.org/10.1145/3290382> (cit. on p. 2).
- [PSN16] Anton Podkopaev, Ilya Sergey, and Aleksandar Nanevski. “Operational Aspects of C/C++ Concurrency”. In: *CoRR abs/1606.01400* (2016). URL: <http://arxiv.org/abs/1606.01400> (cit. on pp. 27, 57).
- [Pul+18] Christopher Pulte, Shaked Flur, Will Deacon, Jon French, Susmit Sarkar, and Peter Sewell. “Simplifying ARM concurrency: multicopy-atomic axiomatic and operational models for ARMv8”. In: *Proc. ACM Program. Lang.* 2.POPL (2018), 19:1–19:29. DOI: [10.1145/3158107](https://doi.org/10.1145/3158107). URL: <https://doi.org/10.1145/3158107> (cit. on pp. 2, 58, 168).

- [Pul+19] Christopher Pulte, Jean Pichon-Pharabod, Jeehoon Kang, Sung Hwan Lee, and Chung-Kil Hur. “Promising-ARM/RISC-V: a simpler and faster operational concurrency model”. In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22-26, 2019*. ACM, 2019, pp. 1–15. DOI: [10.1145/3314221.3314624](https://doi.org/10.1145/3314221.3314624). URL: <https://doi.org/10.1145/3314221.3314624> (cit. on pp. 2, 58).
- [Raa+19] Azalea Raad, Marko Doko, Lovro Rozic, Ori Lahav, and Viktor Vafeiadis. “On library correctness under weak memory consistency: specifying and verifying concurrent libraries under declarative consistency models”. In: *Proc. ACM Program. Lang.* 3.POPL (2019), 68:1–68:31. DOI: [10.1145/3290381](https://doi.org/10.1145/3290381). URL: <https://doi.org/10.1145/3290381> (cit. on pp. 5, 6, 263, 266, 276, 297).
- [RPDG14] Pedro da Rocha Pinto, Thomas Dinsdale-Young, and Philippa Gardner. “TaDA: A Logic for Time and Data Abstraction”. In: *ECOOP 2014 - Object-Oriented Programming - 28th European Conference, Uppsala, Sweden, July 28 - August 1, 2014. Proceedings*. Vol. 8586. Lecture Notes in Computer Science. Springer, 2014, pp. 207–231. DOI: [10.1007/978-3-662-44202-9\\_9](https://doi.org/10.1007/978-3-662-44202-9_9). URL: [https://doi.org/10.1007/978-3-662-44202-9\\_9](https://doi.org/10.1007/978-3-662-44202-9_9) (cit. on pp. 1, 6, 257, 258).
- [RP+16] Pedro da Rocha Pinto, Thomas Dinsdale-Young, Philippa Gardner, and Julian Sutherland. “Modular Termination Verification for Non-blocking Concurrency”. In: *ESOP*. Vol. 9632. Lecture Notes in Computer Science. Springer, 2016, pp. 176–201 (cit. on p. 1).
- [SLS05] William Scherer, Doug Lea, and Michael Scott. “A scalable elimination-based exchange channel”. In: (2005) (cit. on p. 5).
- [SNB15] Ilya Sergey, Aleksandar Nanevski, and Anindya Banerjee. “Specifying and Verifying Concurrent Algorithms with Histories and Subjectivity”. In: *Programming Languages and Systems - 24th European Symposium on Programming, ESOP 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings*. Vol. 9032. Lecture Notes in Computer Science. Springer, 2015, pp. 333–358. DOI: [10.1007/978-3-662-46669-8\\_14](https://doi.org/10.1007/978-3-662-46669-8_14). URL: [https://doi.org/10.1007/978-3-662-46669-8\\_14](https://doi.org/10.1007/978-3-662-46669-8_14) (cit. on p. 297).
- [Ser+16] Ilya Sergey, Aleksandar Nanevski, Anindya Banerjee, and Germán Andrés Delbianco. “Hoare-style specifications as correctness conditions for non-linearizable concurrent objects”. In: *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2016, part of SPLASH 2016, Amsterdam, The Netherlands, October 30 - November 4, 2016*. ACM, 2016, pp. 92–110. DOI: [10.1145/2983990.2983999](https://doi.org/10.1145/2983990.2983999). URL: <https://doi.org/10.1145/2983990.2983999> (cit. on p. 297).
- [SWT18] Ilya Sergey, James R. Wilcox, and Zachary Tatlock. “Programming and Proving with Distributed Protocols”. In: 2.POPL (2018). DOI: [10.1145/3158116](https://doi.org/10.1145/3158116). URL: <https://doi.org/10.1145/3158116> (cit. on p. 1).
- [Sew+10] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O. Myreen. “x86-TSO: a rigorous and usable programmer’s model for x86 multiprocessors”. In: *Commun. ACM* 53.7 (2010), pp. 89–97. DOI: [10.1145/1785414.1785443](https://doi.org/10.1145/1785414.1785443). URL: <https://doi.org/10.1145/1785414.1785443> (cit. on pp. 15, 17).
- [Sim+22] Ben Simner, Alasdair Armstrong, Jean Pichon-Pharabod, Christopher Pulte, Richard Grisenthwaite, and Peter Sewell. “Relaxed virtual memory in Armv8-A”. In: *Programming Languages and Systems*. Ed. by Ilya Sergey. Cham: Springer International Publishing, 2022, pp. 143–173. ISBN: 978-3-030-99336-8 (cit. on p. 58).
- [Sim+20] Ben Simner, Shaked Flur, Christopher Pulte, Alasdair Armstrong, Jean Pichon-Pharabod, Luc Maranget, and Peter Sewell. “ARMv8-A System Semantics: Instruction Fetch in Relaxed Architectures”. In: *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020. Proceedings*. Ed. by Peter Müller. Vol. 12075. Lecture Notes in Computer Science. Springer, 2020, pp. 626–655. DOI: [10.1007/978-3-030-44914-8\\_23](https://doi.org/10.1007/978-3-030-44914-8_23). URL: [https://doi.org/10.1007/978-3-030-44914-8\\_23](https://doi.org/10.1007/978-3-030-44914-8_23) (cit. on pp. 2, 58).
- [SL23] Abhishek Kr Singh and Ori Lahav. “An Operational Approach to Library Abstraction under Relaxed Memory Concurrency”. In: *Proc. ACM Program. Lang.* 7.POPL (2023). DOI: [10.1145/3571246](https://doi.org/10.1145/3571246). URL: <https://doi.org/10.1145/3571246> (cit. on p. 297).
- [SN04] Robert C. Steinke and Gary J. Nutt. “A unified theory of shared memory consistency”. In: *J. ACM* 51.5 (2004), pp. 800–849. DOI: [10.1145/1017460.1017464](https://doi.org/10.1145/1017460.1017464). URL: <https://doi.org/10.1145/1017460.1017464> (cit. on p. 27).

- [SB14] Kasper Svendsen and Lars Birkedal. “Impredicative Concurrent Abstract Predicates”. In: *Programming Languages and Systems - 23rd European Symposium on Programming, ESOP 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014, Proceedings*. Vol. 8410. Lecture Notes in Computer Science. Springer, 2014, pp. 149–168. DOI: [10.1007/978-3-642-54833-8\\_9](https://doi.org/10.1007/978-3-642-54833-8_9). URL: [https://doi.org/10.1007/978-3-642-54833-8\\_9](https://doi.org/10.1007/978-3-642-54833-8_9) (cit. on pp. 1, 6, 257, 258).
- [Sve+18] Kasper Svendsen, Jean Pichon-Pharabod, Marko Doko, Ori Lahav, and Viktor Vafeiadis. “A Separation Logic for a Promising Semantics”. In: *Programming Languages and Systems - 27th European Symposium on Programming, ESOP 2018, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2018, Thessaloniki, Greece, April 14-20, 2018, Proceedings*. Vol. 10801. Lecture Notes in Computer Science. Springer, 2018, pp. 357–384. DOI: [10.1007/978-3-319-89884-1\\_13](https://doi.org/10.1007/978-3-319-89884-1_13). URL: [https://doi.org/10.1007/978-3-319-89884-1\\_13](https://doi.org/10.1007/978-3-319-89884-1_13) (cit. on pp. 1, 24, 167).
- [TDV15] Joseph Tassarotti, Derek Dreyer, and Viktor Vafeiadis. “Verifying read-copy-update in a logic for weak memory”. In: *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015*. ACM, 2015, pp. 110–120. DOI: [10.1145/2737924.2737992](https://doi.org/10.1145/2737924.2737992). URL: <https://doi.org/10.1145/2737924.2737992> (cit. on pp. 5, 133, 167).
- [Tre86] R.K. Treiber. *Systems Programming: Coping with Parallelism*. Research Report RJ. International Business Machines Incorporated, Thomas J. Watson Research Center, 1986. URL: <https://books.google.lu/books?id=YQg3HAAACAAJ> (cit. on pp. 145, 273).
- [TDB13] Aaron Turon, Derek Dreyer, and Lars Birkedal. “Unifying refinement and hoare-style reasoning in a logic for higher-order concurrency”. In: *ACM SIGPLAN International Conference on Functional Programming, ICFP’13, Boston, MA, USA - September 25 - 27, 2013*. ACM, 2013, pp. 377–390. DOI: [10.1145/2500365.2500600](https://doi.org/10.1145/2500365.2500600). URL: <https://doi.org/10.1145/2500365.2500600> (cit. on pp. 1, 297).
- [TVD14] Aaron Turon, Viktor Vafeiadis, and Derek Dreyer. “GPS: navigating weak memory with ghosts, protocols, and separation”. In: *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA 2014, part of SPLASH 2014, Portland, OR, USA, October 20-24, 2014*. ACM, 2014, pp. 691–707. DOI: [10.1145/2660193.2660243](https://doi.org/10.1145/2660193.2660243). URL: <https://doi.org/10.1145/2660193.2660243> (cit. on pp. 1, 2, 5, 8, 99, 115, 124, 133, 134, 159, 177, 201, 245, 273).
- [VN13] Viktor Vafeiadis and Chinmay Narayan. “Relaxed separation logic: a program logic for C11 concurrency”. In: *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA 2013, part of SPLASH 2013, Indianapolis, IN, USA, October 26-31, 2013*. ACM, 2013, pp. 867–884. DOI: [10.1145/2509136.2509532](https://doi.org/10.1145/2509136.2509532). URL: <https://doi.org/10.1145/2509136.2509532> (cit. on pp. 1, 2, 24, 25, 99, 124, 133, 168).
- [VP07] Viktor Vafeiadis and Matthew J. Parkinson. “A Marriage of Rely/Guarantee and Separation Logic”. In: *CONCUR*. 2007, pp. 256–271 (cit. on p. 1).
- [VB23] Simon Friis Vindum and Lars Birkedal. “Spirea: A Mechanized Concurrent Separation Logic for Weak Persistent Memory”. In: *Proc. ACM Program. Lang.* 7.OOPSLA2 (2023). DOI: [10.1145/3622820](https://doi.org/10.1145/3622820). URL: <https://doi.org/10.1145/3622820> (cit. on p. 302).
- [Wat+20] Conrad Watt, Christopher Pulte, Anton Podkopaev, Guillaume Barbier, Stephen Dolan, Shaked Flur, Jean Pichon-Pharabod, and Shu-yu Guo. “Repairing and mechanising the JavaScript relaxed memory model”. In: *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*. ACM, 2020, pp. 346–361. DOI: [10.1145/3385412.3385973](https://doi.org/10.1145/3385412.3385973). URL: <https://doi.org/10.1145/3385412.3385973> (cit. on p. 2).
- [WRP19] Conrad Watt, Andreas Rossberg, and Jean Pichon-Pharabod. “Weakening WebAssembly”. In: *Proc. ACM Program. Lang.* 3.OOPSLA (2019), 133:1–133:28. DOI: [10.1145/3360559](https://doi.org/10.1145/3360559). URL: <https://doi.org/10.1145/3360559> (cit. on p. 2).